

208.5.0

Huawei LiteOS 开发指南

文档版本

6.0

发布日期

2023-05-15



版权所有 © 华为技术有限公司 2023。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编： 518129

网址： <https://www.huawei.com>

客户服务邮箱： support@huawei.com

客户服务电话： 4008302118

目 录

- 1 前言.....1
- 2 概述.....3
 - 2.1 背景介绍.....3
 - 2.2 支持的核.....6
 - 2.3 使用约束.....6
- 3 基础内核.....7
 - 3.1 任务.....7
 - 3.1.1 概述.....7
 - 3.1.2 开发指导.....10
 - 3.1.3 注意事项.....20
 - 3.1.4 编程实例.....21
 - 3.1.5 编程实例 (SMP).....23
 - 3.2 内存.....25
 - 3.2.1 概述.....25
 - 3.2.2 动态内存.....28
 - 3.2.2.1 开发指导.....28
 - 3.2.2.2 注意事项.....32
 - 3.2.2.3 编程实例.....33
 - 3.2.2.3.1 普通内存使用实例.....33
 - 3.2.2.3.2 多模块内存使用实例.....34
 - 3.2.3 静态内存.....35
 - 3.2.3.1 开发指导.....35
 - 3.2.3.2 注意事项.....37
 - 3.2.3.3 编程实例.....37
 - 3.3 中断.....38
 - 3.3.1 概述.....38
 - 3.3.2 开发指导.....40
 - 3.3.3 注意事项.....45
 - 3.3.4 编程实例.....46
 - 3.4 异常接管.....47
 - 3.4.1 概述.....47
 - 3.4.2 使用指南.....49

3.4.3 注意事项.....	50
3.4.4 问题定位实例.....	50
3.5 错误处理.....	52
3.5.1 概述.....	52
3.5.2 开发指导.....	53
3.5.3 注意事项.....	54
3.5.4 编程实例.....	54
3.6 队列.....	55
3.6.1 概述.....	55
3.6.2 开发指导.....	58
3.6.3 注意事项.....	62
3.6.4 编程实例.....	63
3.7 事件.....	65
3.7.1 概述.....	65
3.7.2 开发指导.....	67
3.7.3 注意事项.....	69
3.7.4 编程实例.....	70
3.7.4.1 基于事件类型的读写.....	70
3.7.4.2 基于事件条件的读写.....	72
3.8 信号量.....	75
3.8.1 概述.....	75
3.8.2 开发指导.....	76
3.8.3 注意事项.....	79
3.8.4 编程实例.....	79
3.9 读写信号量.....	81
3.9.1 概述.....	81
3.9.2 开发指导.....	82
3.9.3 注意事项.....	85
3.9.4 编程实例.....	85
3.10 互斥锁.....	88
3.10.1 概述.....	88
3.10.2 开发指导.....	89
3.10.3 注意事项.....	92
3.10.4 编程实例.....	92
3.11 软件定时器.....	94
3.11.1 概述.....	94
3.11.2 开发指导.....	95
3.11.3 注意事项.....	98
3.11.4 编程实例.....	99
3.12 自旋锁.....	100
3.12.1 概述.....	100
3.12.2 开发指导.....	101

3.12.3 注意事项.....	102
3.12.4 编程实例.....	102
3.13 分级初始化.....	105
3.13.1 概述.....	105
3.13.2 开发指导.....	105
3.13.3 注意事项.....	106
3.13.4 编程实例.....	106
3.14 其他.....	108
3.14.1 时间管理.....	108
3.14.1.1 概述.....	108
3.14.1.2 开发指导.....	108
3.14.1.3 注意事项.....	109
3.14.1.4 编程实例.....	110
3.14.2 双向链表.....	111
3.14.2.1 概述.....	111
3.14.2.2 开发指导.....	111
3.14.2.3 注意事项.....	113
3.14.2.4 编程实例.....	113
3.14.3 原子操作.....	114
3.14.3.1 概述.....	114
3.14.3.2 开发指导.....	115
3.14.3.3 注意事项.....	117
3.14.3.4 编程实例.....	117
3.14.4 位操作.....	118
3.14.4.1 概述.....	118
3.14.4.2 开发指导.....	118
3.14.4.3 注意事项.....	119
3.14.4.4 编程实例.....	119
3.14.5 环形缓冲（ringbuf）.....	119
3.14.5.1 概述.....	120
3.14.5.2 开发指导.....	121
3.14.5.3 注意事项.....	122
3.14.5.4 编程实例.....	122
3.14.6 通用内存访问.....	124
3.14.6.1 概述.....	124
3.14.6.2 开发指导.....	124
3.14.6.3 注意事项.....	125
4 扩展内核.....	126
4.1 动态加载.....	126
4.1.1 概述.....	126
4.1.2 开发指导.....	127
4.1.3 注意事项.....	135

4.1.4 编程实例.....	135
4.2 分散加载.....	136
4.2.1 概述.....	136
4.2.2 开发指导.....	137
4.2.3 注意事项.....	140
4.3 低功耗框架.....	141
4.3.1 概述.....	141
4.3.2 休眠唤醒.....	141
4.3.2.1 概述.....	141
4.3.2.2 开发指导.....	142
4.3.2.3 注意事项.....	148
4.3.3 系统休眠.....	148
4.3.3.1 概述.....	148
4.3.3.2 开发指导.....	149
4.3.3.3 注意事项.....	152
4.3.3.4 编程实例.....	152
4.3.4 Tickless.....	154
4.3.4.1 概述.....	154
4.3.4.2 开发指导.....	154
4.3.4.3 注意事项.....	155
4.4 访问权限管理.....	155
4.4.1 概述.....	155
4.4.2 PMP.....	155
4.4.2.1 概述.....	156
4.4.2.2 开发指导.....	156
4.4.2.3 注意事项.....	160
4.4.2.4 编程实例.....	160
4.4.3 MPU.....	160
4.4.3.1 概述.....	161
4.4.3.2 开发指导.....	161
4.4.3.3 注意事项.....	163
4.4.3.4 编程实例.....	163
4.4.4 MMU.....	163
4.4.4.1 概述.....	164
4.4.4.2 ARM32 MMU.....	164
4.4.4.2.1 开发指导.....	165
4.4.4.2.2 注意事项.....	166
4.4.4.2.3 编程实例.....	166
4.4.4.3 ARM64 MMU.....	169
4.4.4.3.1 开发指导.....	170
4.4.4.3.2 注意事项.....	172
4.4.4.3.3 编程实例.....	172

4.5 CPU 占用率.....	173
4.5.1 概述.....	173
4.5.2 开发指导.....	174
4.5.3 注意事项.....	177
4.5.4 编程实例.....	177
4.6 Trace.....	179
4.6.1 概述.....	179
4.6.2 开发指导.....	179
4.6.3 注意事项.....	184
4.6.4 编程实例.....	184
4.7 LMS.....	186
4.7.1 概述.....	187
4.7.2 使用指南.....	189
4.7.3 注意事项.....	191
4.8 Perf.....	191
4.8.1 概述.....	191
4.8.2 开发指导.....	191
4.8.3 注意事项.....	195
4.8.4 编程实例.....	195
4.9 C++支持.....	198
4.9.1 概述.....	198
4.9.2 开发指导.....	198
4.9.3 注意事项.....	200
4.9.4 编程实例.....	200
4.10 动态共享内存.....	202
4.10.1 概述.....	202
4.10.2 开发指导.....	203
4.10.3 注意事项.....	205
4.11 核间通信.....	205
4.11.1 概述.....	206
4.11.2 开发指导.....	206
4.11.3 注意事项.....	207
4.11.4 编程实例.....	207
5 文件系统.....	209
5.1 功能概述.....	209
5.2 VFS.....	210
5.2.1 概述.....	210
5.2.2 开发指导.....	212
5.2.3 注意事项.....	213
5.3 NFS.....	214
5.3.1 概述.....	214
5.3.2 开发指导.....	214

5.3.3 注意事项.....	216
5.4 JFFS2.....	217
5.4.1 概述.....	217
5.4.2 开发指导.....	217
5.4.3 注意事项.....	221
5.5 FAT.....	221
5.5.1 概述.....	221
5.5.2 开发指导.....	222
5.5.3 虚拟分区的兼容性.....	230
5.5.4 注意事项.....	230
5.6 YAFFS2.....	231
5.6.1 概述.....	231
5.6.2 开发指导.....	232
5.6.3 注意事项.....	234
5.7 RAMFS.....	235
5.7.1 概述.....	235
5.7.2 开发指导.....	235
5.7.3 注意事项.....	236
5.8 PROC.....	236
5.8.1 概述.....	236
5.8.2 开发指导.....	237
5.8.3 注意事项.....	239
5.8.4 编程实例.....	239
5.9 ROMFS.....	241
5.9.1 概述.....	241
5.9.2 开发指导.....	241
5.9.3 注意事项.....	242
5.10 LITTLEFS.....	243
5.10.1 概述.....	243
5.10.2 开发指导.....	243
5.10.3 注意事项.....	245
6 兼容接口.....	247
6.1 概述.....	247
6.2 POSIX 接口.....	248
6.2.1 POSIX 支持接口.....	248
6.2.2 POSIX NP 支持接口.....	263
6.2.3 POSIX 不支持接口.....	263
6.3 Libc/Libm 接口.....	276
6.3.1 Libc 支持接口.....	276
6.3.2 Libm 支持接口.....	290
6.3.3 Libc/Libm 不支持接口.....	301
6.4 C++兼容规格.....	316

6.5 CMSIS 接口.....	318
6.5.1 CMSIS v1.0.....	318
6.5.1.1 CMSIS v1.0 适配接口.....	318
6.5.1.2 CMSIS v1.0 不支持接口.....	320
6.5.1.3 CMSIS v1.0 标准接口适配差异.....	320
6.5.2 CMSIS v2.0.....	321
6.5.2.1 CMSIS v2.0 适配接口.....	321
6.5.2.2 CMSIS v2.0 不支持接口.....	324
6.5.2.3 CMSIS v2.0 标准接口适配差异.....	325
6.6 linux 适配.....	325
6.6.1 完成量（ completion ）.....	326
6.6.1.1 概述.....	326
6.6.1.2 开发指导.....	326
6.6.1.3 注意事项.....	327
6.6.1.4 编程实例.....	327
6.6.2 工作队列（ workqueue ）.....	328
6.6.2.1 概述.....	329
6.6.2.2 开发指导.....	329
6.6.2.3 注意事项.....	331
6.6.2.4 编程实例.....	331
6.6.3 等待队列（ waitqueue ）.....	332
6.6.3.1 概述.....	333
6.6.3.2 开发指导.....	333
6.6.3.3 注意事项.....	334
6.6.3.4 编程实例.....	334
6.6.4 中断.....	335
6.6.4.1 概述.....	336
6.6.4.2 开发指导.....	336
6.6.4.3 注意事项.....	337
6.6.4.4 编程实例.....	337
6.6.5 高精度定时器.....	338
6.6.5.1 概述.....	338
6.6.5.2 开发指导.....	338
6.6.5.3 注意事项.....	339
6.6.5.4 编程实例.....	339
6.6.6 linux 适配接口概览.....	340
6.6.7 linux 不支持接口概览.....	353
7 驱动开发指导.....	356
7.1 基于驱动框架的开发指导.....	356
7.1.1 概述.....	356
7.1.2 开发指导.....	358
7.1.3 注意事项.....	361

7.1.4 编程实例.....	361
7.2 适配文件操作的驱动开发.....	362
7.2.1 概述.....	362
7.2.2 开发指导.....	362
7.2.3 注意事项.....	365
7.3 MMC 驱动开发.....	365
7.3.1 概述.....	365
7.3.2 开发指导.....	366
7.3.3 注意事项.....	371
7.4 USB 驱动开发.....	372
7.4.1 概述.....	372
7.4.2 开发指导.....	373
7.4.2.1 功能.....	373
7.4.2.2 配置 USB 模块的内存.....	376
7.4.2.3 动态切换 USB 协议.....	377
7.4.2.4 模块编译.....	377
7.4.3 编程实例.....	379
7.4.3.1 Host 端.....	379
7.4.3.1.1 Host U 盘.....	379
7.4.3.1.2 Host USB 转网口&RNDIS.....	380
7.4.3.1.3 Host 4G-Modem.....	381
7.4.3.1.4 Host USB-WIFI.....	381
7.4.3.1.5 Host UVC.....	381
7.4.3.1.6 Host HID.....	382
7.4.3.2 Device 端.....	382
7.4.3.2.1 Device 虚拟 U 盘.....	382
7.4.3.2.2 Device UVC.....	385
7.4.3.2.3 Device UAC.....	386
7.4.3.2.4 Device UAV (Camera)	387
7.4.3.2.5 Device 虚拟串口.....	387
7.4.3.2.6 Device 虚拟网口.....	388
7.4.3.2.7 Device 复合设备 (串口&网口)	389
7.4.3.2.8 Device DFU.....	390
7.4.3.2.9 Device HID.....	390
7.4.4 API 参考.....	391
7.4.4.1 usbd_set_device_info.....	393
7.4.4.2 usb_init.....	395
7.4.4.3 usb_deinit.....	396
7.4.4.4 usb_is_devicemode.....	397
7.4.4.5 fmass_register_notify.....	397
7.4.4.6 fmass_unregister_notify.....	398
7.4.4.7 fmass_partition_startup.....	399

7.4.4.8 ram_mass_storage_init.....	400
7.4.4.9 ram_mass_storage_deinit.....	401
7.4.4.10 fuvc_frame_descriptors_get.....	401
7.4.4.11 uvc_open_device.....	402
7.4.4.12 uvc_close_device.....	403
7.4.4.13 uvc_wait_host.....	404
7.4.4.14 uvc_get_state.....	405
7.4.4.15 uvc_rcv_pack.....	406
7.4.4.16 uvc_video_stop.....	407
7.4.4.17 uvc_format_info_get.....	408
7.4.4.18 uvc_format_status_check.....	409
7.4.4.19 hi_camera_register_cmd.....	409
7.4.4.20 uac_wait_host.....	410
7.4.4.21 fuac_send_message.....	411
7.4.4.22 fuac_receive_message.....	412
7.4.4.23 fuac_opts_set.....	413
7.4.4.24 fuac_get_opts.....	414
7.4.4.25 fuac_rate_get.....	415
7.4.4.26 fuac2_get_opts.....	415
7.4.4.27 fuac2_set_opts.....	416
7.4.4.28 usb_dfu_init_env_entities.....	417
7.4.4.29 usb_dfu_update_status.....	418
7.4.4.30 usb_dfu_update_size_get.....	419
7.4.4.31 usb_dfu_get_entity.....	419
7.4.4.32 usb_dfu_read.....	420
7.4.4.33 hid_report_descriptor_info.....	421
7.4.4.34 hid_add_report_descriptor.....	422
7.4.4.35 fhid_send_data.....	423
7.4.4.36 fhid_rcv_data.....	423
7.4.4.37 u3g_write_from_shell.....	424
7.4.4.38 uvc_get_frame_buf.....	425
7.4.4.39 usb_dfu_free_entities.....	426
7.5 UART 驱动开发.....	427
7.5.1 概述.....	427
7.5.2 开发指导.....	428
7.5.2.1 裸读写 UART.....	428
7.5.2.2 文件方式读写 UART.....	428
7.5.2.3 模块配置.....	428
7.5.3 编程实例.....	428
7.5.3.1 simple uart.....	429
7.5.3.2 general uart.....	429
7.5.3.2.1 裸读写 UART.....	429

7.5.3.2.2 文件读写 UART.....	430
8 配置工具.....	432
8.1 概述.....	432
8.2 安装工具.....	432
8.3 使用指导.....	434
8.4 打开 menuconfig 菜单.....	434
8.5 保存 savemenuconfig 菜单.....	435
8.6 make defconfig.....	435
8.7 make allyesconfig.....	435
8.8 make allnoconfig.....	436
8.9 配置项说明.....	436
8.9.1 配置项简介.....	436
8.9.2 Compiler 配置.....	436
8.9.3 Targets 配置.....	437
8.9.4 SSP 配置.....	437
8.9.5 NX 配置.....	438
9 可维可测.....	440
9.1 Telnet.....	440
9.1.1 概述.....	440
9.1.2 开发指导.....	441
9.1.3 注意事项.....	441
9.2 Shell.....	442
9.2.1 概述.....	442
9.2.2 开发指导.....	443
9.2.3 注意事项.....	445
9.2.4 静态注册编程实例.....	445
9.2.5 动态注册编程实例.....	446
9.2.6 系统命令参考.....	446
9.2.6.1 使能系统命令.....	446
9.2.6.2 help.....	446
9.2.6.3 date.....	447
9.2.6.4 uname.....	448
9.2.6.5 task.....	449
9.2.6.6 free.....	452
9.2.6.7 memcheck.....	453
9.2.6.8 memused.....	454
9.2.6.9 hwi.....	455
9.2.6.10 queue.....	457
9.2.6.11 sem.....	458
9.2.6.12 mutex.....	459
9.2.6.13 swtmr.....	460
9.2.6.14 systeminfo.....	461

9.2.6.15 log.....	462
9.2.6.16 dmesg.....	463
9.2.6.17 stack.....	465
9.2.6.18 cpup.....	466
9.2.6.19 watch.....	467
9.2.6.20 schedstatstart.....	468
9.2.6.21 schedstatstop.....	469
9.2.6.22 schedstatinfo.....	469
9.2.7 文件命令参考.....	469
9.2.7.1 使能文件系统命令.....	469
9.2.7.2 cd.....	470
9.2.7.3 pwd.....	470
9.2.7.4 mkdir.....	471
9.2.7.5 rmdir.....	472
9.2.7.6 rm.....	473
9.2.7.7 touch.....	474
9.2.7.8 cp.....	475
9.2.7.9 dd.....	476
9.2.7.10 ls.....	477
9.2.7.11 cat.....	478
9.2.7.12 lsfd.....	479
9.2.7.13 format.....	479
9.2.7.14 mount.....	480
9.2.7.15 umount.....	481
9.2.7.16 partinfo.....	482
9.2.7.17 partition.....	483
9.2.7.18 statfs.....	483
9.2.7.19 virstatfs.....	484
9.2.7.20 sync.....	485
9.2.7.21 writeproc.....	486
9.2.8 网络命令参考.....	487
9.2.8.1 使能网络命令.....	487
9.2.8.2 ifconfig.....	487
9.2.8.3 dns.....	490
9.2.8.4 arp.....	491
9.2.8.5 ntpdate.....	493
9.2.8.6 ping.....	493
9.2.8.7 ping6.....	494
9.2.8.8 tftp.....	496
9.2.8.9 netstat.....	499
9.2.8.10 ipdebug.....	500
9.2.8.11 netdebug.....	501

9.2.8.12 telnet.....	506
9.2.8.13 iperf.....	507
9.2.9 动态加载命令参考.....	510
9.2.9.1 使能动态加载命令.....	510
9.2.9.2 mopen.....	510
9.2.9.3 mclose.....	511
9.2.9.4 findsym.....	511
9.2.9.5 call.....	512
9.2.9.6 lddrop.....	513
9.2.10 Trace 命令参考.....	513
9.2.10.1 使能 Trace 命令.....	513
9.2.10.2 trace_mask.....	514
9.2.10.3 trace_start.....	515
9.2.10.4 trace_stop.....	515
9.2.10.5 trace_dump.....	516
9.2.10.6 trace_reset.....	517
9.3 调度统计.....	517
9.4 内存调测方法.....	519
9.4.1 概述.....	520
9.4.2 多内存池机制.....	520
9.4.3 内存备份机制.....	522
9.4.4 内存合法性检查.....	523
9.4.5 内存 size 检查.....	524
9.4.6 内存泄露检测.....	527
9.5 MMC 维测方法.....	528
9.5.1 命令失败调测方法.....	528
9.5.2 DMA 传输调测方法.....	528
9.6 任务间通信调测方法.....	529
9.6.1 队列调测方法.....	529
9.6.2 死锁检测机制.....	530
9.7 异常交互.....	535
9.8 低功耗调测方法.....	536
9.8.1 低功耗调测使能.....	536
9.8.2 系统状态信息查询.....	536
9.8.3 低功耗动态开关.....	537
9.8.4 阻止系统深睡事件查询.....	537
9.8.5 系统休眠唤醒打点.....	538
9.9 临终遗言使用方法.....	539
9.10 魔法键使用方法.....	541
9.11 栈估算工具使用方法.....	542
9.12 调试案例.....	546
9.12.1 踩内存定位方法.....	546

9.12.1.1 异常信息定位踩内存方法..... 546

9.12.1.2 内存备份机制定位踩内存方法..... 546

9.12.1.3 内存合法性检查定位踩内存方法..... 546

9.12.1.4 内存 size 检查定位踩内存方法..... 546

9.12.1.5 全局变量踩内存定位方法..... 546

9.12.1.6 task 状态判断是否踩内存..... 547

10 Security..... 548

10.1 地址空间布局随机化..... 548

10.1.1 概述..... 548

10.1.2 使用指南..... 552

10.1.3 注意事项..... 560

11 附录..... 562

11.1 Huawei LiteOS Kernel 启动流程..... 562

11.2 平台兼容性适配修改要点..... 563

1 前言

目的

本文档介绍Huawei LiteOS的体系结构，并介绍如何进行Huawei LiteOS的开发和调试。

读者对象

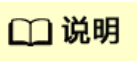
本文档主要适用于Huawei LiteOS的开发者。

本文档主要适用于以下对象：

- 物联网端侧软件开发工程师
- 物联网架构设计师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下：

符号	说明
 注意	用于警示潜在的危险情形，若不避免，可能会导致中度或轻微的人身伤害。
 须知	用于传递设备或环境安全警示信息，若不避免，可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	“说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修订记录

修改记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本的更新内容。

日期	修订版本	描述
2021年6月22日	1.0	206.x版本发布
2021年11月25日	2.0	206.1版本发布
2022年3月23日	3.0	206.2版本发布
2022年11月25日	4.0	207.0版本发布
2023年3月30日	5.0	207.1版本发布

2 概述

[2.1 背景介绍](#)

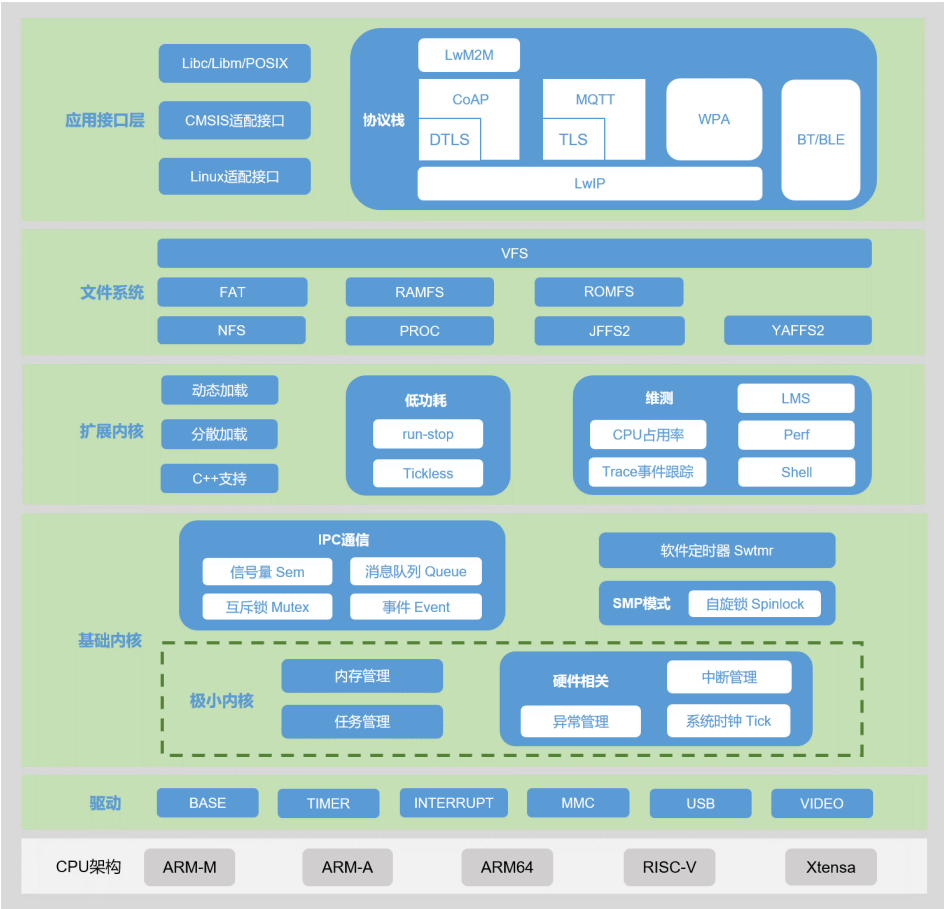
[2.2 支持的核](#)

[2.3 使用约束](#)

2.1 背景介绍

Huawei LiteOS是轻量级的实时操作系统，具备轻量级、低功耗、快速启动、组件丰富等关键能力。

图 2-1 Huawei LiteOS 的基本框架图



Huawei LiteOS基础内核包括不可裁剪的极小内核和可裁剪的其他模块。极小内核只包含任务管理和调度相关的基本功能。可裁剪的模块包括信号量、互斥锁、队列管理、事件管理、软件定时器、内存管理、中断管理、异常管理和系统时钟tick。

Huawei LiteOS支持UP（单核）与SMP（多核）模式，即支持在单核或者多核的环境上运行。

Huawei LiteOS 的优势

- 高实时性，高稳定性。
- 低功耗。
- 支持动态加载、分散加载。
- 支持功能静态裁剪。
- 快速启动。

内核各模块简介

任务管理

提供任务的创建、删除、延迟、挂起、恢复等功能，支持锁定和解锁任务调度，支持任务按优先级高低的抢占调度以及同优先级时间片轮转调度。

内存管理

- 提供静态内存和动态内存两种算法，支持内存申请、释放。目前支持的内存管理算法有固定大小的BOX算法、动态申请的bestfit算法和bestfit_little算法。
- 提供内存统计、内存越界检测功能。

硬件相关

提供中断管理、异常管理、系统时钟Tick等功能。

- 中断管理：提供中断的创建、删除、使能、禁止、请求位的清除功能。
- 异常管理：系统运行过程中发生异常后，跳转到异常处理模块，打印当前发生异常的函数调用栈信息，或者保存当前系统状态。
- Tick：Tick是操作系统调度的基本时间单位，对应的时长由每秒Tick数决定，由用户配置。

IPC通信

提供消息队列、事件、信号量和互斥锁功能。

- 消息队列：支持消息队列的创建、删除、发送和接收功能。
- 事件：支持读事件和写事件功能。
- 信号量：支持信号量的创建、删除、申请和释放功能。
- 互斥锁：支持互斥锁的创建、删除、申请和释放功能。

软件定时器

软件定时器提供了定时器的创建、删除、启动、停止功能。

自旋锁

多核场景下，支持自旋锁的初始化、申请、释放功能。

动态加载

动态加载是一种软件加载链接技术，不对组成程序的目标文件进行一次性链接加载，当程序要运行时才加载链接。

Huawei LiteOS提供支持OBJ目标文件和SO共享目标文件的动态加载机制。

分散加载

分散加载是通过重排镜像等手段，优先加载关键业务，从而缩短启动时间。

低功耗

- run-stop：即休眠唤醒，是Huawei LiteOS提供的保存系统现场镜像以及从系统现场镜像中恢复运行的机制。
- Tickless：Tickless机制通过计算下一次有意义的时钟中断的时间，来减少不必要的时钟中断，从而降低系统功耗。打开Tickless功能后，系统会在CPU空闲时启动Tickless机制。

维测

- CPU占用率：可以获取系统或者指定任务的CPU占用率。
- Trace事件跟踪：实时获取事件发生的上下文，并写入缓冲区。支持自定义缓冲区，跟踪指定模块的事件，开启/停止Trace，清除/输出trace缓冲区数据等。

- LMS：实时检测内存操作合法性，LMS能够检测的内存问题包括缓冲区溢出（buffer overflow），释放后使用（use after free），多重释放（double free）和释放野指针（wild pointer free）。
- Perf：一款性能分析工具，可以通过Perf查找性能瓶颈、定位热点代码。Huawei LiteOS支持配置采样数据缓冲区、配置采样事件、启动/停止Perf、输出采样数据等功能。
- Shell：Huawei LiteOS Shell使用串口或Telnet工具接收用户输入的命令，通过命令的方式调用、执行相应的应用程序。Huawei LiteOS Shell支持常用的基本调试功能，包含系统、文件、网络和动态加载等相关命令，同时支持用户添加自定义命令。

C++支持

Huawei LiteOS支持部分STL特性、异常和RTTI特性，其他特性由编译器支持。

2.2 支持的核

表 2-1 Huawei LiteOS 支持的核及芯片

支持的核	芯片
ARM9、Cortex-A7、Cortex-A53、Cortex-A55	Hi3516A、Hi3518EV200、Hi3556、Hi3559A、Hi3516CV300系列、Hi3516EV200系列
Cortex-M7	STM32F769
Cortex-R7、Cortex-R8	Kirin系列
Tensilica Xtensa Vision Q6 DSP、HiFi 3z DSP	Kirin系列、Hi3751V900
LingLong	Kirin Audio DSP
RISC-V、LinxCore131、LinxCore170	Kirin系列、Hi3886、Hi3931、Hi3862、Hi3892、Hi1162、Hi3321

2.3 使用约束

- Huawei LiteOS提供一套自有 OS接口，同时也支持POSIX和CMSIS接口。请勿混用这些接口。混用接口可能导致不可预知的错误，例如：用POSIX接口申请信号量，但用Huawei LiteOS接口释放信号量。
- 开发驱动程序只能用Huawei LiteOS的接口，上层APP建议用POSIX接口。

3 基础内核

- 3.1 任务
- 3.2 内存
- 3.3 中断
- 3.4 异常接管
- 3.5 错误处理
- 3.6 队列
- 3.7 事件
- 3.8 信号量
- 3.9 读写信号量
- 3.10 互斥锁
- 3.11 软件定时器
- 3.12 自旋锁
- 3.13 分级初始化
- 3.14 其他

3.1 任务

3.1.1 概述

基本概念

从系统角度看，任务是竞争系统资源的最小运行单元。任务可以使用或等待CPU、使用内存空间等系统资源，并独立于其它任务运行。

Huawei LiteOS的任务模块可以给用户提供多个任务，实现任务间的切换，帮助用户管理业务程序流程。Huawei LiteOS的任务模块具有如下特性：

- 支持多任务。
- 一个任务表示一个线程。
- 抢占式调度机制，高优先级的任务可打断低优先级任务，低优先级任务必须在高优先级任务阻塞或结束后才能得到调度。
- 相同优先级任务支持时间片轮转调度方式。
- 共有32个优先级[0, 31]，最高优先级为0，最低优先级为31。
- 任务存在两种属性(detached、joinable)。默认情况下任务创建的属性为detached。

任务相关概念

任务状态

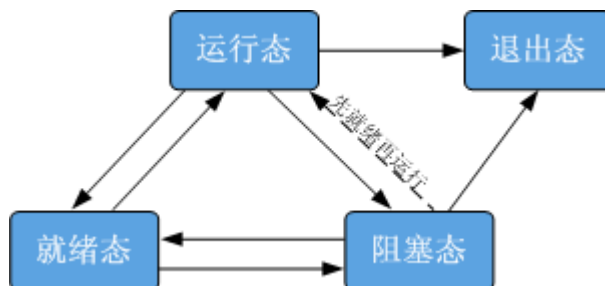
Huawei LiteOS系统中的任务有多种运行状态。系统初始化完成后，创建的任务就可以在系统中竞争一定的资源，由内核进行调度。

任务状态通常分为以下四种：

- 就绪（Ready）：该任务在就绪队列中，只等待CPU。
- 运行（Running）：该任务正在执行。
- 阻塞（Blocked）：该任务不在就绪队列中，包含任务被挂起（suspend状态）、任务被延时（delay状态）、任务正在等待信号量、读写队列或者等待事件等。
- 退出态（Dead）：该任务运行结束，等待系统回收资源，包含任务处于僵尸状态（zombie状态）、任务已经被回收（unused状态）。

任务状态迁移

图 3-1 任务状态示意图



任务状态迁移说明：

- 就绪态→运行态
任务创建后进入就绪态，发生任务切换时，就绪队列中最高优先级的任务被执行，从而进入运行态，同时该任务从就绪队列中移出。
- 运行态→阻塞态
正在运行的任务发生阻塞（挂起、延时、读信号量等）时，任务状态由运行态变成阻塞态，然后发生任务切换，取出并运行就绪队列中最高优先级任务。
- 阻塞态→就绪态（阻塞态→运行态）
阻塞的任务被恢复后（任务恢复、延时时间超时、读信号量超时或读到信号量等），此时被恢复的任务会被加入就绪队列，从而由阻塞态变成就绪态；此时如

果被恢复任务的优先级高于正在运行任务的优先级，则会发生任务切换，该任务由就绪态变成运行态。

- 就绪态→阻塞态
任务也有可能在就绪态时被阻塞（挂起），此时任务状态由就绪态变为阻塞态，该任务从就绪队列中删除，不会参与任务调度，直到该任务被恢复。
- 运行态→就绪态
有更高优先级任务创建或者恢复后，会发生任务调度，此刻就绪队列中最高优先级任务会被取出并切换到运行态，那么原先运行的任务由运行态变为就绪态，并插入到就绪队列中。
- 运行态→退出态
运行中的任务运行结束，任务状态由运行态变为退出态。退出态包含任务运行结束的正常退出状态以及僵尸状态。例如，joinable属性的任务运行结束但是没有被回收资源，对外呈现的就是僵尸状态，即退出态。
- 阻塞态→退出态
阻塞的任务调用删除接口，任务状态由阻塞态变为退出态。

任务ID

任务ID，在任务创建时通过参数返回给用户，是任务的重要标识。系统中的ID号是唯一的。用户可以通过任务ID对指定任务进行任务挂起、任务恢复、查询任务名等操作。

任务优先级

优先级表示任务执行的优先顺序。任务的优先级决定了在发生任务切换时即将要执行的任务，就绪队列中最高优先级的任务将得到执行。

任务入口函数

新任务得到调度后将执行的函数。该函数由用户实现，在任务创建时，通过任务创建结构体设置。

任务栈

每个任务都拥有一个独立的栈空间，我们称为任务栈。栈空间里保存的信息包含局部变量、寄存器、函数参数、函数返回地址等。

任务上下文

任务在运行过程中使用的一些资源，如寄存器等，称为任务上下文。当这个任务挂起时，其他任务继续执行，可能会修改寄存器等资源中的值。如果任务切换时没有保存任务上下文，可能会导致任务恢复后出现未知错误。

因此，Huawei LiteOS在任务切换时会切出任务的任务上下文信息，保存在自身的任务栈中，以便任务恢复后，从栈空间中恢复挂起时的上下文信息，从而继续执行挂起时被打断的代码。

任务控制块TCB

每个任务都含有一个任务控制块(TCB)。TCB包含了任务上下文栈指针（stack pointer）、任务状态、任务优先级、任务ID、任务名、任务栈大小等信息，可以反映出每个任务运行情况。

任务切换

任务切换包含获取就绪队列中最高优先级任务、切出任务上下文保存、切入任务上下文恢复等动作。

任务属性

任务存在两种属性(detached、joinable)，任务的不同属性可以决定任务的终止方式。joinable属性即为非分离状态，这种状态的任务退出后，不会自动释放资源，需要被其他任务通过调用LOS_TaskJoin接口等待获取其返回值，才能被释放资源。detached属性即为分离状态，这种状态的任务退出后将自动回收资源，并且这个任务不能被等待，等待是没有意义的。

调度器相关概念

调度模式

Huawei LiteOS包含两种调度模式，即全局队列调度器模式和多队列调度器模式，在不同场景下各有优势，当前系统默认使用全局调度器模式。

- 全局队列调度器模式：系统仅使用一个优先级队列保存准备运行的任务，所有核使用同一优先级队列调度，所有任务抢占同一个优先级队列，因此全局队列调度器模式拥有更高的实时性。
- 多队列调度器模式：仅多核使用，系统按照核数为每个核创建一个优先级队列，每个核仅使用自己的优先级队列进行调度，系统通过负载均衡算法将任务在不同的优先级队列上进行迁移，相比全局队列调度模式可以有效减少频繁的核间迁移。

任务迁移算法

任务迁移算法能够实现不同核中优先级队列间的均衡。由于全局队列调度器模式仅有一个优先级队列，因此不需要使用算法来实现均衡。但在多队列调度器模式下则需使用任务迁移算法，并按照一定规则将任务在不同优先级队列上进行迁移。目前，任务迁移算法有以下两种。

- Simple均衡算法：按照每个优先级队列上的任务数量进行迁移，使得不同核上等待的任务数量差值能够保持在一定范围内。该算法仅使用任务数量这一个变量进行均衡，具有负载计算简单、不依赖其他特性的特点，可以实现基本的负载均衡。
- 基于CPUP均衡算法：先通过CPUP组件计算出每个核的CPU使用率，再根据每个核的CPU使用率对任务进行迁移，使得不同核的CPU使用率差值保持在一定范围内。该算法通过消耗少量CPU来统计CPU使用率，并根据CPU使用率来进行均衡。由于该均衡算法基于真实的CPU使用率，故均衡效果较Simple均衡算法更佳。

运作机制

用户创建任务时，系统会初始化任务栈，预置上下文。此外，系统还会将“任务入口函数”地址放在相应位置。这样在任务第一次启动进入运行态时，将会执行“任务入口函数”。

3.1.2 开发指导

使用场景

任务创建后，内核可以执行锁定任务调度，解锁任务调度，挂起，恢复，延时等操作，同时也可以设置任务优先级，获取任务优先级。

功能

Huawei LiteOS 的任务管理模块提供下面几种功能，接口详细信息请参见API参考。

功能分类	接口名	描述
创建和删除任务	LOS_TaskCreateOnly	创建任务，并使该任务进入suspend状态，不对该任务进行调度。如果需要调度，可以调用LOS_TaskResume使该任务进入ready状态。
	LOS_TaskCreate	创建任务，并使该任务进入ready状态，如果就绪队列中没有更高优先级的任务，则运行该任务。
	LOS_TaskCreateOnlyStatic	创建任务，任务栈由用户传入，并使该任务进入suspend状态，不对该任务进行调度。如果需要调度，可以调用LOS_TaskResume使该任务进入ready状态。
	LOS_TaskCreateStatic	创建任务，任务栈由用户传入，并使该任务进入ready状态，如果就绪队列中没有更高优先级的任务，则运行该任务。
	LOS_TaskDelete	删除指定的任务。
	LOS_TaskJoin	以阻塞的方式等待指定的任务并回收任务资源。
控制任务状态	LOS_TaskResume	恢复挂起的任务，使该任务进入ready状态。
	LOS_TaskSuspend	挂起指定的任务，然后切换任务。
	LOS_TaskDelay	任务延时等待，释放CPU，等待时间到期后该任务会重新进入ready状态。
	LOS_TaskYield	当前任务释放CPU，并将其移到具有相同优先级的就绪任务队列的末尾。
	LOS_TaskDetach	将任务属性设置为detached。
控制任务调度	LOS_TaskLock	锁定任务调度，但任务仍可被中断打断。
	LOS_TaskUnlock	解锁任务调度。
	LOS_TaskUnlockNoSched	解锁任务调度，并继续执行本任务（建议参考注释使用在特殊场景，错误使用会导致调度时序错乱）。
控制任务优先级	LOS_CurTaskPriSet	设置当前任务的优先级。
	LOS_TaskPriSet	设置指定任务的优先级。
	LOS_TaskPriGet	获取指定任务的优先级。
设置任务亲和性	LOS_TaskCpuAffiSet	设置指定任务的运行CPU集合（该函数仅在SMP模式下支持）。
回收任务栈资源	LOS_TaskResRecycle	回收所有待回收的任务栈资源。

功能分类	接口名	描述
获取任务信息	LOS_CurTaskIDGet	获取当前任务的ID。
	LOS_TaskInfoGet	获取指定任务的信息，包括任务状态、优先级、任务栈大小、栈顶指针SP、任务入口函数和已使用的任务栈大小等。
	LOS_TaskCpuAffiGet	获取指定任务的运行CPU集合（该函数仅在SMP模式下支持）。
	LOS_TaskIsScheduled	查询当前核是否已经开始任务调度。
任务信息维测	LOS_TaskSwitchHookReg	注册任务上下文切换的钩子函数。只有开启LOSCFG_BASE_CORE_TSK_MONITOR宏开关后，这个钩子函数才会在任务发生上下文切换时被调用。
任务空闲处理回调	LOS_IdleHandlerHookReg	注册空闲任务钩子函数，当系统空闲时调用。
申请任务的安全栈	LOS_TaskAllocSecureContext	申请任务的安全栈。只有开启LOSCFG_TRUSTZONE宏开关后，该函数才可使用。

须知

- 打开菜单（如何打开请参见后续[打开menuconfig菜单](#)），配置LOSCFG_KERNEL_SMP使能多核模式（菜单路径为Kernel ---> Enable Kernel SMP）。在SMP的子菜单中，还可以设置核的数量、使能多任务的核间同步、使能函数跨核调用。
- 在多核模式下，创建任务时可以传入usCpuAffiMask来配置任务的CPU亲和性，该标志位采用1bit-1core的对应方式，详情可参见TSK_INIT_PARAM_S结构体。
- 各个任务的任务栈大小可以在创建任务时进行针对性的设置，若设置为0，则使用默认任务栈大小（LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE）作为任务栈大小。

TASK 状态

Huawei LiteOS任务的大多数状态由内核维护，唯有自删除状态对用户可见，需要用户在创建任务时传入：

定义	实际数值	描述
LOS_TASK_STATUS_JOINABLE	0x0080	任务不会自动释放资源，需要被其他任务等待获取其返回值，才能被释放资源
LOS_TASK_STATUS_DETACHED	0x0100	任务是自删除的

用户在调用创建任务接口时，可以将创建任务的TSK_INIT_PARAM_S参数的uwResved域设置为LOS_TASK_STATUS_DETACHED（detached属性）或者LOS_TASK_STATUS_JOINABLE(joinable属性）。

须知

任务joinable属性受LOSCFG_TASK_JOINABLE开关影响。

- 打开菜单（如何打开请参见后续[打开menuconfig菜单](#)），LOSCFG_TASK_JOINABLE配置位于Kernel ---> Basic Config ---> Task ---> Enable Join/Detach mechanism。
- LOSCFG_TASK_JOINABLE打开，只有将任务状态设置为LOS_TASK_STATUS_JOINABLE才能设置为joinable属性的任务，joinable属性的任务不会自删除，需要用户通过调用LOS_TaskJoin函数去回收joinable属性任务的资源，或者通过调用LOS_TaskDetach接口设置任务为detached属性，detached属性的任务运行完成之后会自动回收资源。
- LOSCFG_TASK_JOINABLE关闭，任务只有detached属性，不管TSK_INIT_PARAM_S参数的uwResved域是否设置为LOS_TASK_STATUS_JOINABLE。
- LOS_TaskDelete接口无法回收joinable属性的任务资源，只能通过LOS_TaskJoin接口。

TASK 优先级

Huawei LiteOS任务的优先级当前支持范围为（LOS_TASK_PRIORITY_HIGHEST - LOS_TASK_PRIORITY_LOWEST），数值越小，优先级越高。

定义	实际数值	描述
LOS_TASK_PRIORITY_HIGHEST	0	任务最高优先级
LOS_TASK_PRIORITY_LOWEST	31	任务最低优先级

TASK 错误码

创建任务、删除任务、挂起任务、恢复任务、延时任务等操作存在失败的可能，失败时会返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_TS K_NO_MEMORY	0x0300 0200	内存空间不足。	增大动态内存空间，有兩種方式可以实现： • 设置更大的系统动态内存池，配置项为 OS_SYS_MEM_SIZE。 • 释放一部分动态内存。 如果错误发生在LiteOS启动过程中的任务初始化，还可以通过减少系统支持的最大任务数来解决；如果错误发生在任务创建过程中，也可以减小任务栈大小来解决。
2	LOS_ERRNO_TS K_PTR_NULL	0x0200 0201	传递给任务创建接口的任务参数initParam为空指针，或者传递给任务信息获取的接口的参数为空指针。	确保传入的参数不为空指针。
3	LOS_ERRNO_TS K_STKSZ_NOT_ALIGN	0x0200 0202	暂不使用该错误码。	-
4	LOS_ERRNO_TS K_PRIOR_ERROR	0x0200 0203	创建任务或者设置任务优先级时，传入的优先级参数不正确。	检查任务优先级，必须在[0, 31]的范围内。
5	LOS_ERRNO_TS K_ENTRY_NULL	0x0200 0204	创建任务时传入的任务入口函数为空指针。	定义任务入口函数。
6	LOS_ERRNO_TS K_NAME_EMPTY	0x0200 0205	创建任务时传入的任务名为空指针。	设置任务名。
7	LOS_ERRNO_TS K_STKSZ_TOO_SMALL	0x0200 0206	创建任务时传入的任务栈太小。	增大任务的任务栈大小使之不小于系统设置最小任务栈大小（配置项为 LOS_TASK_MIN_STACK_SIZE）。
8	LOS_ERRNO_TS K_ID_INVALID	0x0200 0207	超出OS支持范围内的无效的任务ID。	检查任务ID。

序号	定义	实际数值	描述	参考解决方案
9	LOS_ERRNO_TASK_ALREADY_SUSPENDED	0x02000208	挂起任务时，发现任务已经被挂起。	等待这个任务被恢复后，再去尝试挂起这个任务。
10	LOS_ERRNO_TASK_NOT_SUSPENDED	0x02000209	恢复任务时，发现任务未被挂起。	挂起这个任务后，再去尝试恢复这个任务。
11	LOS_ERRNO_TASK_NOT_CREATED	0x0200020a	未创建任务。	创建这个任务，这个错误可能会发生在以下操作中： • 删除任务 • 恢复/挂起任务 • 设置指定任务的优先级 • 获取指定任务的信息 • 设置指定任务的运行CPU集合
12	LOS_ERRNO_TASK_DELETE_LOCKED	0x0300020b	删除任务时，任务处于锁定状态。	解锁任务之后再删除任务。
13	LOS_ERRNO_TASK_DELAY_INTERRUPT	0x0300020d	中断期间，进行任务延时。	等待退出中断后再进行延时操作。
14	LOS_ERRNO_TASK_DELAY_IN_LOCK	0x0200020e	在任务锁定状态下，延时该任务。	解锁任务之后再延时任务。
15	LOS_ERRNO_TASK_YIELD_IN_LOCK	0x0200020f	在任务锁定状态下，进行Yield操作。	任务解锁后再进行Yield操作。
16	LOS_ERRNO_TASK_YIELD_NOT_ENOUGH_TASK	0x02000210	执行Yield操作时，发现具有相同优先级的就绪任务队列中没有其他任务。	增加与当前任务具有相同优先级的任务数。
17	LOS_ERRNO_TASK_TCB_UNAVAILABLE	0x02000211	创建任务时，发现没有空闲的任务控制块可以使用。	调用LOS_TaskResRecycle接口回收空闲的任务控制块，如果回收后依然创建失败，再增加系统的任务控制块数量。

序号	定义	实际数值	描述	参考解决方案
18	LOS_ERRNO_TS K_OPERATE_SY STEM_TASK	0x0200 0214	不允许删除、挂起、延时系统级别的任务，例如idle任务、软件定时器任务，也不允许修改系统级别的任务优先级。	检查任务ID，不要操作系统任务。
19	LOS_ERRNO_TS K_SUSPEND_LO CKED	0x0300 0215	不允许将处于锁定状态的任务挂起。	任务解锁后，再尝试挂起任务。
20	LOS_ERRNO_TS K_STKSZ_TOO_ LARGE	0x0200 0220	创建任务时，设置了过大的任务栈。	减小任务栈大小。
21	LOS_ERRNO_TS K_CPU_AFFINIT Y_MASK_ERR	0x0300 0223	设置指定任务的运行CPU集合时，传入了错误的CPU集合。	检查传入的CPU掩码。
22	LOS_ERRNO_TS K_YIELD_IN_INT	0x0200 0224	不允许在中断中对任务进行Yield操作。	不要在中断中进行Yield操作。
23	LOS_ERRNO_TS K_MP_SYNC_RE SOURCE	0x0200 0225	跨核任务删除同步功能，资源申请失败。	通过设置更大的LOSCFG_BASE_IPC_SEM_LIMIT的值，增加系统支持的信号量个。
24	LOS_ERRNO_TS K_MP_SYNC_FA ILED	0x0200 0226	跨核任务删除同步功能，任务未及时删除。	需要检查目标删除任务是否存在频繁的状态切换，导致系统无法在规定的时间内完成删除的动作。
25	LOS_ERRNO_TS K_ALLOC_SECU RE_INT	0x0200 0227	不允许在中断中申请任务安全栈。	不要在中断中申请任务安全栈。
26	LOS_ERRNO_TS K_SECURE_ALR EADY_ALLOC	0x0200 0228	任务的安全栈已经申请过。	不要重复为同一个任务申请安全栈。
27	LOS_ERRNO_TS K_ALLOC_SECU RE_FAILED	0x0200 0229	申请安全栈失败。	安全世界的内存不足，建议释放部分安全世界的内存。
28	LOS_ERRNO_TS K_FREE_SECU RE_FAILED	0x0200 022a	释放安全栈失败。	当前未使用。
29	LOS_ERRNO_TS K_NOT_ALLOW _IN_INT	0x0200 022b	此接口不允许在中断上下文。	不要在中断中调用对应接口

序号	定义	实际数值	描述	参考解决方案
30	LOS_ERRNO_TSK_SCHED_LOCKE	0x0200022c	此接口可能存在阻塞，不允许关闭调度	检查调度锁逻辑
31	LOS_ERRNO_TSK_ALREADY_JOIN	0x0200022d	joinable的任务不能被多个任务同时等待	检查是否已经存在其他任务正在等待
32	LOS_ERRNO_TSK_IS_DETACHED	0x0200022e	当前任务属性为detached	检查是否join一个detached属性的任务或者重复设置任务为detached属性
33	LOS_ERRNO_TSK_NOT_JOIN_SELF	0x0200022f	任务不能join自己	检查taskid是否传入正确
34	LOS_ERRNO_TSK_IS_ZOMBIE	0x02000230	此任务处于僵尸状态	检查是否对一个处于僵尸状态的任务进行删除，设置优先级/属性/CPU亲和性，挂起，恢复

须知

- 错误码定义见[错误码简介](#)。8~15位的所属模块为任务模块，值为0x02。
- 任务模块中的错误码序号 0x16、0x1c，未被定义，不可用。

开发流程

本部分以创建任务为例，讲解开发流程。

1. 打开菜单，进入Kernel ---> Basic Config ---> Task菜单，完成任务模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_CORE_TSK_LIMIT	系统支持的最大任务数。	(0, 256]	不同平台默认值不一样	无
LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE	最小任务栈大小，一般使用默认值即可。	(0, OS_SYS_MEM_SIZE)	不同平台默认值不一样	无

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE	默认任务栈大小。	(0, OS_SYS_MEM_SIZE)	不同平台默认值不一样	无
LOSCFG_BASE_CORE_TSK_IDLE_STACK_SIZE	idle任务栈大小，一般使用默认值即可。	(0, OS_SYS_MEM_SIZE)	不同平台默认值不一样	无
LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO	默认任务优先级，一般使用默认配置即可。	[0,31]	10	无
LOSCFG_BASE_CORE_TIMESLICE	任务时间片调度开关。	YES/NO	YES	无
LOSCFG_BASE_CORE_TIMESLICE_TIMEOUT	同优先级任务最长执行时间。（单位：Tick）	(0, 65535]	不同平台默认值不一样	无
LOSCFG_OBSOLETE_API	使能后，任务参数使用旧方式UINTPTR auwArgs[4]，否则使用新的任务参数VOID *pArgs。建议关闭此开关，使用新的任务参数。	YES/NO	不同平台默认值不一样	无
LOSCFG_BASE_CORE_TSK_MONITOR	任务栈溢出检查和轨迹开关。	YES/NO	YES	无
LOSCFG_TASK_STACK_STATIC_ALLOCATION	支持创建任务时，由用户传入任务栈。	YES/NO	NO	无
LOSCFG_TASK_JOINABLE	开启内核join/detach机制	YES/NO	NO	无

2. 若启用SMP模式，可以打开菜单，进入Kernel ---> Basic Config ---> Enable Kernel SMP菜单，完成任务在多核的相关系统配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_SMP_CORE_NUM	系统使用核的数量。	[2, 4]	2	LOSCFG_KERNEL_SMP
LOSCFG_KERNEL_SMP_TASK_SYNC	多任务的核间同步开关。	YES/NO	NO	LOSCFG_KERNEL_SMP

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_SMP_CALL	函数跨核调用开关。	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_SMP_CALL_STATIC_ALLOCATION	函数调用对象的创建使用静态分配替代动态分配。	YES/NO	NO	LOSCFG_KERNEL_SMP_CALL
LOSCFG_SMP_CALL_OBJ_NUM	静态分配的函数调用对象的数量。	[1, 65535]	16	LOSCFG_SMP_CALL_STATIC_ALLOCATION
LOSCFG_KERNEL_SMP_GC_IDLE	使用IDLE任务回收待删除任务	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_KERNEL_SMP_GC_SWTMR	使用SWTMR回收待删除任务	YES/NO	YES	LOSCFG_KERNEL_SMP

3. 打开菜单，进入Kernel ---> Basic Config ---> Enable Scheduler菜单，完成任务调度相关配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_SCHED_SQ	全局队列调度器模式。	YES/NO	YES	无
LOSCFG_SCHED_MQ	多队列调度器模式。	YES/NO	NO	LOSCFG_KERNEL_SMP
LOSCFG_SCHED_LOAD_BALANCE_SIMPLE	Simple均衡算法。	YES/NO	NO	LOSCFG_SCHED_MQ
LOSCFG_SCHED_LOAD_BALANCE_CPUP	基于CPUP均衡算法。	YES/NO	NO	LOSCFG_SCHED_MQ LOSCFG_KERNEL_CPUP

- 锁任务调度LOS_TaskLock，防止高优先级任务调度。
- 创建任务LOS_TaskCreate，或静态创建任务LOS_TaskCreateStatic。（需要打开LOSCFG_TASK_STACK_STATIC_ALLOCATION宏。）
- 解锁任务LOS_TaskUnlock，让任务按照优先级进行调度。
- 延时任务LOS_TaskDelay，任务延时等待。
- 挂起指定的任务LOS_TaskSuspend，任务挂起等待恢复操作。
- 恢复挂起的任务LOS_TaskResume。

平台差异性

无。

3.1.3 注意事项

任务创建：

- 任务名是指针，并没有分配空间，在设置任务名时，禁止将局部变量的地址赋值给任务名指针。
- 创建任务时，会按照16字节大小或者sizeof(UINTPTR) * 2对齐的方式为任务栈分配内存。确定任务栈大小的原则是，够用就行，多了浪费，少了任务栈溢出。

锁任务：

- 锁任务调度，并不关中断，因此任务仍可被中断打断。
- 锁任务调度必须和解锁任务调度配合使用。

任务优先级：

- 设置任务优先级时可能会发生任务调度。
- LOS_CurTaskPriSet和LOS_TaskPriSet接口不能在中断中使用，也不能用于修改软件定时器任务的优先级。
- LOS_TaskPriGet接口传入的task ID对应的任务未创建或者超过最大任务数，统一返回0xffff。

禁止操作：

- 挂起当前任务时，如果任务已经被锁定，则无法挂起。
- idle任务及软件定时器任务不能被挂起或者删除。
- 在中断处理函数中或者在锁任务的情况下，执行LOS_TaskDelay会失败。

任务资源：

- 执行Idle任务时，会对之前已删除任务的任务控制块和任务栈进行回收。
- 可配置的系统最大任务数是指整个系统的任务总个数，而非用户能使用的任务个数。例如：系统软件定时器多占用一个任务资源，那么用户能使用的任务资源就会减少一个。
- 在删除任务时要保证任务申请的资源（如互斥锁、信号量等）已被释放。

多核模式：

- 在多核模式下，锁任务调度只能锁住当前核的调度器，其他核仍然能正常调度。
- 在多核模式下，由于跨核间任务的删除或挂起是异步执行的，因此操作的返回值并不代表最终操作的结果，仅代表上述请求已经发出。同时执行完成会存在延时。
- 在多核模式下，如果开启任务跨核删除同步的功能（LOSCFG_KERNEL_SMP_TASK_SYNC选项），则跨核删除任务时，需要等待目标任务删除后才会返回结果，如果在设定的时间内未成功将任务删除，则会返回LOS_ERRNO_TSK_MP_SYNC_FAILED错误。开启该功能后，每个任务会增加1个信号量的开销。
- 在多核模式下，如前面第二条所述，跨核任务删除是异步执行的，部分任务由回收机制执行回收。可以通过LOSCFG_KERNEL_SMP_GC_IDLE与LOSCFG_KERNEL_SMP_GC_SWTMR配置项选择回收机制执行的方式，但由于IDLE任务有较低的优先级，如果CPU一直处于忙状态，可能出现任务无法回收的情况，请谨慎使用IDLE回收方式。

3.1.4 编程实例

实例描述

本实例介绍基本的任务操作方法，包含2个不同优先级任务的创建、任务延时、任务锁与解锁调度、挂起和恢复等操作，并阐述任务优先级调度的机制以及各接口的应用。

编程示例

前提条件：在menuconfig菜单中完成任务模块的配置。

```
UINT32 g_taskHild;
UINT32 g_taskLold;
#define TSK_PRIOR_HI 4
#define TSK_PRIOR_LO 5

UINT32 Example_TaskHi(VOID)
{
    UINT32 ret;

    printf("Enter TaskHi Handler.\r\n");

    /* 延时2个Tick，延时后该任务会挂起，执行剩余任务中最高优先级的任务(g_taskLold任务) */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay Task Failed.\r\n");
        return LOS_NOK;
    }

    /* 2个Tick时间到了后，该任务恢复，继续执行 */
    printf("TaskHi LOS_TaskDelay Done.\r\n");

    /* 挂起自身任务 */
    ret = LOS_TaskSuspend(g_taskHild);
    if (ret != LOS_OK) {
        printf("Suspend TaskHi Failed.\r\n");
        return LOS_NOK;
    }
    printf("TaskHi LOS_TaskResume Success.\r\n");

    return ret;
}

/* 低优先级任务入口函数 */
UINT32 Example_TaskLo(VOID)
{
    UINT32 ret;

    printf("Enter TaskLo Handler.\r\n");

    /* 延时2个Tick，延时后该任务会挂起，执行剩余任务中最高优先级的任务(背景任务) */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay TaskLo Failed.\r\n");
        return LOS_NOK;
    }

    printf("TaskHi LOS_TaskSuspend Success.\r\n");

    /* 恢复被挂起的任务g_taskHild */
    ret = LOS_TaskResume(g_taskHild);
    if (ret != LOS_OK) {
        printf("Resume TaskHi Failed.\r\n");
        return LOS_NOK;
    }
}
```

```
printf("TaskHi LOS_TaskDelete Success.\r\n");

return ret;
}

/* 任务测试入口函数，创建两个不同优先级的任务 */
UINT32 Example_TskCaseEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S initParam;

    /* 锁任务调度，防止新创建的任务比本任务高而发生调度 */
    LOS_TaskLock();

    printf("LOS_TaskLock() Success!\r\n");

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskHi;
    initParam.usTaskPrio = TSK_PRIOR_HI;
    initParam.pcName = "TaskHi";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
    /* 创建高优先级任务，由于锁任务调度，任务创建成功后不会马上执行 */
    ret = LOS_TaskCreate(&g_taskHild, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskHi create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskHi create Success!\r\n");

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskLo;
    initParam.usTaskPrio = TSK_PRIOR_LO;
    initParam.pcName = "TaskLo";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    /* 创建低优先级任务，由于锁任务调度，任务创建成功后不会马上执行 */
    ret = LOS_TaskCreate(&g_taskLold, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskLo create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskLo create Success!\r\n");

    /* 解锁任务调度，此时会发生任务调度，执行就绪队列中最高优先级任务 */
    LOS_TaskUnlock();

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
LOS_TaskLock() Success!
Example_TaskHi create Success!
Example_TaskLo create Success!
Enter TaskHi Handler.
Enter TaskLo Handler.
TaskHi LOS_TaskDelay Done.
TaskHi LOS_TaskSuspend Success.
TaskHi LOS_TaskResume Success.
TaskHi LOS_TaskDelete Success.
```

完整实例代码

sample_task.zip

3.1.5 编程实例 (SMP)

实例描述

本实例介绍基本的任务操作方法，包含任务创建、任务延时、任务锁与解锁调度、挂起和恢复等操作，阐述任务优先级调度的机制以及各接口的应用。

1. 创建了2个任务:TaskHi和TaskLo。
2. TaskHi为高优先级任务, 绑定在当前测试任务的CPU上。
3. TaskLo为低优先级任务, 不设置亲和性即不绑核。

须知

由于TaskLo未设置亲和性，LOS_TaskLock对其没有锁任务的效果，因此"Example_TaskLo create Success!" 与 " Enter TaskLo Handler." 这两句打印并没有严格的先后顺序。

编程示例

前提条件：在menuconfig菜单中完成任务模块的配置和SMP模式使能。

```
UINT32 g_taskLoid;
UINT32 g_taskHild;
#define TSK_PRIOR_HI 4
#define TSK_PRIOR_LO 5

UINT32 Example_TaskHi(VOID)
{
    UINT32 ret;

    printf("[cpu%d] Enter TaskHi Handler.\r\n", ArchCurrCpuid());

    /* 延时2个Tick，延时后该任务会挂起，执行剩余任务中最高优先级的任务(g_taskLoid任务) */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay Task Failed.\r\n");
        return LOS_NOK;
    }

    /* 2个Tick后，该任务恢复，继续执行 */
    printf("TaskHi LOS_TaskDelay Done.\r\n");

    /* 挂起自身任务 */
    ret = LOS_TaskSuspend(g_taskHild);
    if (ret != LOS_OK) {
        printf("Suspend TaskHi Failed.\r\n");
        return LOS_NOK;
    }
    printf("TaskHi LOS_TaskResume Success.\r\n");
    return ret;
}

/* 低优先级任务入口函数 */
UINT32 Example_TaskLo(VOID)
{
```

```

    UINT32 ret;

    printf("[cpu%d] Enter TaskLo Handler.\r\n", ArchCurrCpuId());

    /* 延时2个Tick，延时后该任务会挂起，执行剩余任务中就高优先级的任务(背景任务) */
    ret = LOS_TaskDelay(2);
    if (ret != LOS_OK) {
        printf("Delay TaskLo Failed.\r\n");
        return LOS_NOK;
    }

    printf("TaskHi LOS_TaskDelete Success.\r\n");
    return ret;
}

/* 任务测试入口函数，创建两个不同优先级的任务 */
UINT32 Example_TskCaseEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S initParam = {0};

    /* 锁任务调度 */
    LOS_TaskLock();

    printf("LOS_TaskLock() Success on cpu%d!\r\n", ArchCurrCpuId());

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskHi;
    initParam.usTaskPrio = TSK_PRIOR_HI;
    initParam.pcName = "TaskHi";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
#ifdef LOSCFG_KERNEL_SMP
    /* 绑定高优先级任务到CPU1运行 */
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    /* 创建高优先级任务，由于CPU1的调度器被锁，任务创建成功后不会马上执行 */
    ret = LOS_TaskCreate(&g_taskHild, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskHi create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskHi create Success!\r\n");

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_TaskLo;
    initParam.usTaskPrio = TSK_PRIOR_LO;
    initParam.pcName = "TaskLo";
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;
#ifdef LOSCFG_KERNEL_SMP
    /* 低优先级任务不设置CPU亲和性 */
    initParam.usCpuAffiMask = 0;
#endif
    /* 创建低优先级任务，尽管锁任务调度，但是由于该任务没有绑定该处理器，任务创建成功后可以马上在其他CPU执行 */
    ret = LOS_TaskCreate(&g_taskLold, &initParam);
    if (ret != LOS_OK) {
        LOS_TaskUnlock();

        printf("Example_TaskLo create Failed!\r\n");
        return LOS_NOK;
    }

    printf("Example_TaskLo create Success!\r\n");

    /* 解锁任务调度，此时会发生任务调度，执行就绪队列中最高优先级任务 */
    LOS_TaskUnlock();
}

```

```
    return LOS_OK;  
}
```

结果验证

编译运行得到的结果为：

```
LOS_TaskLock() success on cpu1!  
Example_TaskHi create Success!  
Example_TaskLo create Success!  
[cpu2] Enter TaskLo Handler.  
[cpu1] Enter TaskHi Handler.  
TaskHi LOS_TaskDelete Success.  
TaskHi LOS_TaskDelay Done.
```

完整实例代码

sample_task_smp.zip

3.2 内存

3.2.1 概述

基本概念

内存管理模块用于管理系统的内存资源，它是操作系统的核心模块之一，主要包括内存的初始化、分配以及释放。

在系统运行过程中，内存管理模块通过对内存的申请/释放来管理用户和OS对内存的使用，使内存的利用率和使用效率达到最优，同时最大限度地解决系统的内存碎片问题。

Huawei LiteOS的内存管理分为静态内存管理和动态内存管理，提供内存初始化、分配、释放等功能。

- 动态内存：在动态内存池中分配用户指定大小的内存块，支持bestfit（也称为dlink）和bestfit_little两种内存管理算法。
 - 优点：按需分配。
 - 缺点：内存池中可能出现碎片。
- 静态内存：在静态内存池中分配用户初始化时预设（固定）大小的内存块。
 - 优点：分配和释放效率高，静态内存池中无碎片。
 - 缺点：只能申请到初始化预设大小的内存块，不能按需申请。

动态内存运作机制——bestfit

动态内存管理，即在内存资源充足的情况下，根据用户需求，从系统配置的一块比较大的连续内存（内存池，也是堆内存）中分配任意大小的内存块。当用户不需要该内存块时，又可以释放回系统供下一次使用。

与静态内存相比，动态内存管理的优点是按需分配，缺点是内存池中容易出现碎片。

bestfit内存管理结构如下图所示：

图 3-2 bestfit 动态内存管理结构图



- 第一部分
堆内存（也称内存池）的起始地址及堆区域总大小。
- 第二部分
本身是一个数组，每个元素是一个双向链表，所有free节点的控制头都会被分类挂在这个数组的双向链表中。
假设内存允许的最小节点为 $2^{\min}$ 字节，则数组的第一个双向链表存储的是所有size为 $2^{\min} \leq \text{size} < 2^{\min+1}$ 的free节点，第二个双向链表存储的是所有size为 $2^{\min+1} \leq \text{size} < 2^{\min+2}$ 的free节点，依次类推第n个双向链表存储的是所有size为 $2^{\min+n-1} \leq \text{size} < 2^{\min+n}$ 的free节点。每次申请内存的时候，会从这个数组检索最合适大小的free节点以分配内存。每次释放内存时，会将该内存作为free节点存储至这个数组以便下次再使用。
- 第三部分
占用内存池极大部分的空间，是用于存放各节点的实际区域。以下是LosMemDynNode节点结构体申明以及简单介绍：

```
typedef struct {
    union {
        LOS_DL_LIST freeNodeInfo;    /* Free memory node */
        struct {
            UINT32 magic;
            UINT32 taskid : 16;
        };
    };
    struct tagLosMemDynNode *preNode;
    UINT32 sizeAndFlag;
} LosMemCtlNode;

typedef struct tagLosMemDynNode {
    LosMemCtlNode selfNode;
} LosMemDynNode;
```

图 3-3 LosMemDynNode 结构体介绍图

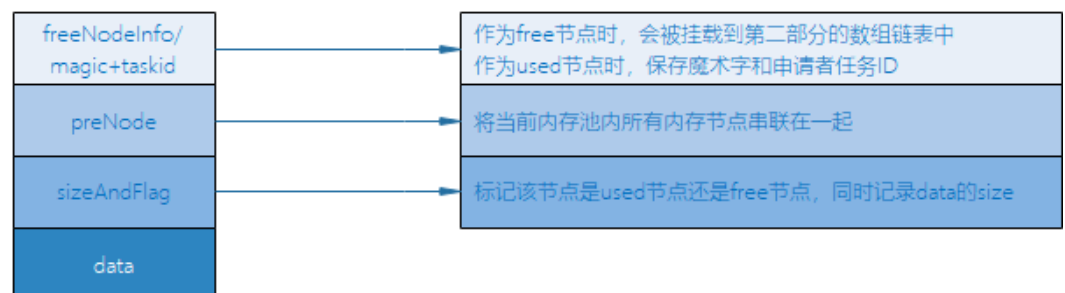


图 3-4 对齐方式申请内存结果示意



当申请到的节点包含的数据空间首地址不符合对齐要求时，需要进行对齐，可以通过增加Gap域确保返回的指针符合对齐要求。

动态内存运作机制——bestfit_little

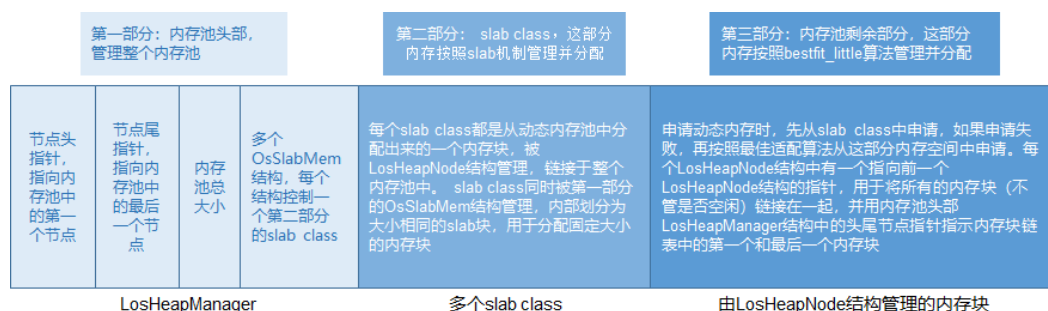
bestfit_little算法是在最佳适配算法的基础上加入slab机制形成的算法。最佳适配算法使得每次分配内存时，都会选择内存池中最小最适合的内存块进行分配，而slab机制可以用于分配固定大小的内存块，从而减小产生内存碎片的可能性。

Huawei LiteOS内存管理中的slab机制支持配置slab class数目及每个class的最大空间。

现以内存池中共有4个slab class，每个slab class的最大空间为512字节为例说明slab机制。这4个slab class是从内存池中按照最佳适配算法分配出来的。第一个slab class被分为32个16字节的slab块，第二个slab class被分为16个32字节的slab块，第三个slab class被分为8个64字节的slab块，第四个slab class被分为4个128字节的slab块。

初始化内存模块时，首先初始化内存池，然后在初始化后的内存池中按照最佳适配算法申请4个slab class，再逐个按照slab内存管理机制初始化4个slab class。

每次申请内存时，先在满足申请大小的最佳slab class中申请（比如用户申请20字节内存，就在slab块大小为32字节的slab class中申请），如果申请成功，就将slab内存块整块返回给用户，释放时整块回收。需要注意的是，如果满足条件的slab class中已无可以分配的内存块，则从内存池中按照最佳适配算法申请，而不会继续从有着更大slab块空间的slab class中申请。释放内存时，先检查释放的内存块是否属于slab class，如果是则还回对应的slab class中，否则还回内存池中。

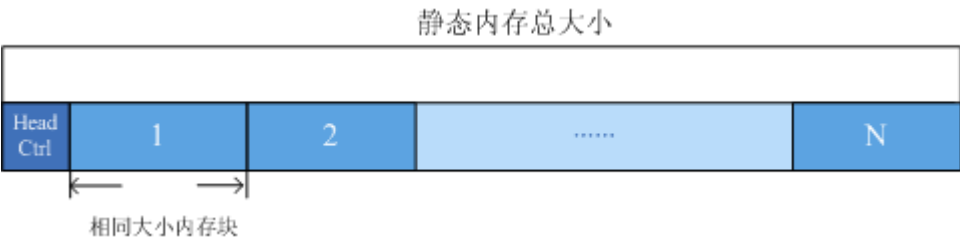


静态内存运作机制

静态内存实质上是一个静态数组，静态内存池内的块大小在初始化时设定，初始化后块大小不可变更。

静态内存池由一个控制块和若干相同大小的内存块构成。控制块位于内存池头部，用于内存块管理。内存块的申请和释放以块大小为粒度。

图 3-5 静态内存示意图



3.2.2 动态内存

3.2.2.1 开发指导

使用场景

动态内存管理的主要工作是动态分配并管理用户申请到的内存区间。

动态内存管理主要用于用户需要使用大小不等的内存块的场景。当用户需要使用内存时，可以通过操作系统的动态内存申请函数索取指定大小的内存块，一旦使用完毕，通过动态内存释放函数归还所占用内存，使之可以重复使用。

功能

Huawei LiteOS系统中的动态内存管理模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
初始化和删除内存池	LOS_MemInit	初始化一块指定的动态内存池，大小为size。
	LOS_MemDelnit	删除指定内存池，仅打开LOSCFG_MEM_MUL_POOL时有效。
申请、释放动态内存	LOS_MemAlloc	从指定动态内存池中申请size长度的内存。
	LOS_MemFree	释放已申请的内存。
	LOS_MemRealloc	按size大小重新分配内存块，并将原内存块内容拷贝到新内存块。如果新内存块申请成功，则释放原内存块。
	LOS_MemAllocAlign	从指定动态内存池中申请长度为size且地址按boundary字节对齐的内存。
获取内存池信息	LOS_MemPoolSizeGet	获取指定动态内存池的总大小。
	LOS_MemTotalUsedGet	获取指定动态内存池的总使用量大小。

功能分类	接口名	描述
	LOS_MemInfoGet	获取指定内存池的内存结构信息，包括空闲内存大小、已使用内存大小、空闲内存块数量、已使用的内存块数量、最大的空闲内存块大小。
	LOS_MemPoolList	打印系统中已初始化的所有内存池，包括内存池的起始地址、内存池大小、空闲内存总大小、已使用内存总大小、最大的空闲内存块大小、空闲内存块数量、已使用的内存块数量。仅打开LOSCFG_MEM_MUL_POOL时有效。
获取内存块信息	LOS_MemFreeBlocksGet	获取指定内存池的空闲内存块数量。
	LOS_MemUsedBlocksGet	获取指定内存池已使用的内存块数量。
	LOS_MemTaskIdGet	获取申请了指定内存块的任务ID，仅LOSCFG_MEM_DEBUG或LOSCFG_MEM_TASK_STAT时生效。
	LOS_MemLastUsedGet	获取内存池尾节点使用的最后一个字节后的第一个字节的地址。 如果尾节点是已使用节点，则尾节点使用的最后一个字节为节点尾部。 如果尾节点是空节点，则尾节点使用的最后一个字节为节点头结构体。
	LOS_MemNodeSizeCheck	获取指定内存块的总大小和可用大小，仅打开LOSCFG_BASE_MEM_NODE_SIZE_CHECK时有效。
	LOS_MemFreeNodeShow	打印指定内存池的空闲内存块的大小及数量。
检查指定内存池的完整性	LOS_MemIntegrityCheck	对指定内存池做完整性检查，仅打开LOSCFG_BASE_MEM_NODE_INTEGRITY_CHECK时有效。
设置、获取内存检查级别（仅打开LOSCFG_BASE_MEM_NODE_SIZE_CHECK时有效）	LOS_MemCheckLevelSet	设置内存检查级别。
	LOS_MemCheckLevelGet	获取内存检查级别。
为指定模块申请、释放动态内存（仅打开LOSCFG_MEM_	LOS_MemMalloc	从指定动态内存池分配size长度的内存给指定模块，并纳入模块统计。
	LOS_MemMfree	释放已经申请的内存块，并纳入模块统计。

功能分类	接口名	描述
MUL_MODULE (时有效)	LOS_MemMallocAlign	从指定动态内存池中申请长度为size且地址按boundary字节对齐的内存给指定模块，并纳入模块统计。
	LOS_MemMrealloc	按size大小重新分配内存块给指定模块，并将原内存块内容拷贝到新内存块，同时纳入模块统计。如果新内存块申请成功，则释放原内存块。
获取指定模块的内存使用量（仅打开LOSCFG_MEM_MUL_MODULE时有效）	LOS_MemMusedGet	获取指定模块的内存使用量，单位为Byte。

须知

- 动态内存提供了内存调测功能，具体使用方法见[内存调测方法](#)。
- 对于bestfit_little算法，只支持[多内存池机制](#)和[内存合法性检查](#)，不支持其他内存调测功能。
- 上述接口中，通过宏开关控制的都是内存调测功能相关的接口。
- 通过LOS_MemMallocAlign/LOS_MemMallocAlign申请的内存进行LOS_MemRealloc/LOS_MemMrealloc操作后，不能保障新的内存首地址保持对齐。
- 对于bestfit_little算法，不支持对LOS_MemMallocAlign申请的内存进行LOS_MemRealloc操作，否则将返回失败。

说明

Huawei LiteOS提供了多模块内存统计功能，该功能基于普通内存接口的封装接口，增加模块ID作为入参。不同业务模块进行内存操作时，调用对应封装接口，可统计各模块的内存使用情况，并通过模块ID获取指定模块的内存使用情况。

应用场景：系统业务模块化清晰，用户需统计各模块的内存占用情况。

依赖：目前只有bestfit内存管理算法支持该功能，需要使能LOSCFG_KERNEL_MEM_BESTFIT。

使用方法：

1. 该功能配置项为LOSCFG_MEM_MUL_MODULE，配置路径是Kernel ---> Memory Management ---> Dynamic Memory Management ---> Enable Memory module statistics；
2. 每个业务模块配置唯一module ID，模块代码中在内存操作时调用对应接口，并传入相应模块ID；
3. 通过LOS_MemMusedGet接口获取指定模块的内存使用情况，可用于模块内存占用优化分析。

注意事项：

1. 模块ID由宏MEM_MODULE_MAX限定，当系统模块个数超过该值时，需修改MEM_MODULE_MAX；
2. 模块中所有内存操作都需调用LOS_MemM开头的接口，否则可能导致统计不准确。

开发流程

本节介绍使用动态内存的典型场景开发流程。

1. 在los_config.h文件中配置项动态内存池起始地址与大小。

配置项	含义	取值范围	默认值	依赖
OS_SYS_MEM_ADDR	系统动态内存起始地址	[0, n)	&m_aucSysMem1[0]	无
OS_SYS_MEM_SIZE	系统动态内存池的大小（DDR自适应配置），以byte为单位	[0, n)	从bss段末尾至系统DDR末尾	无

- OS_SYS_MEM_ADDR：一般使用默认值即可。
- OS_SYS_MEM_SIZE：一般使用默认值即可。

2. 打开菜单，进入Kernel ---> Memory Management菜单，完成动态内存管理模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_MEM_BESTFIT	选择bestfit内存管理算法	YES/NO	YES	无
LOSCFG_KERNEL_MEM_BESTFIT_LITTLE	选择bestfit_little内存管理算法	YES/NO	NO	无
LOSCFG_KERNEL_MEM_SLAB_EXTENSION	使能slab功能，可以降低系统持续运行过程中内存碎片化的程度	YES/NO	NO	无
LOSCFG_KERNEL_MEM_SLAB_AUTO_EXPANSION_MODE	slab自动扩展，当分配给slab的内存不足时，能够自动从系统内存池中申请新的空间进行扩展	YES/NO	NO	LOSCFG_KERNEL_MEM_SLAB_EXTENSION
LOSCFG_KERNEL_TASK_STAT	使能任务内存统计	YES/NO	YES	LOSCFG_KERNEL_MEM_BESTFIT或 LOSCFG_KERNEL_MEM_BESTFIT_LITTLE

配置项	含义	取值范围	默认值	依赖
LOSCFG_MEM_MULTODULE	多模块内存统计	YES/NO	NO	LOSCFG_KERNEL_MEM_BESTFIT

3.
初始化LOS_MemInit。
- 初始一个内存池后如图，生成一个EndNode，并且剩余的内存全部被标记为FreeNode节点。注：EndNode作为内存池末尾的节点，size为0。



4.
申请任意大小的动态内存LOS_MemAlloc。
- 判断动态内存池中是否存在申请量大小的空间，若存在，则划出一块内存块，以指针形式返回，若不存在，返回NULL。

调用三次LOS_MemAlloc函数可以创建三个节点，假设分别为UsedA，UsedB，UsedC，大小分别为sizeA，sizeB，sizeC。因为刚初始化内存池的时候只有一个大的FreeNode，所以这些内存块是从这个FreeNode中切割出来的。



- 当内存池中存在多个FreeNode的时候进行malloc，将会适配最合适大小的FreeNode用来新建内存块，减少内存碎片。若新建的内存块不等于被使用的FreeNode的大小，则在新建内存块后，多余的内存又会被标记为一个新的FreeNode。
5.
释放动态内存LOS_MemFree。
- 回收内存块，供下一次使用。

假设调用LOS_MemFree释放内存块UsedB，则会回收内存块UsedB，并且将其标记为FreeNode。在回收内存块时，相邻的FreeNode会自动合并。



平台差异性

无。

3.2.2.2 注意事项

- 内存分配：
- 对齐分配内存接口LOS_MemAllocAlign/LOS_MemMallocAlign因为要进行地址对齐，可能会额外消耗部分内存，故存在一些遗失内存，当系统释放该对齐内存时，同时回收由于对齐导致的遗失内存。
 - 重新分配内存接口LOS_MemRealloc/LOS_MemMrealloc如果分配成功，系统会自己判定是否需要释放原来申请的内存，并返回重新分配的内存地址。如果重新

分配失败，原来的内存保持不变，并返回NULL。禁止使用pPtr = LOS_MemRealloc(pool, pPtr, uwSize)，即：不能使用原来的旧内存地址pPtr变量来接收返回值。

内存释放：

- 对同一块内存多次调用LOS_MemFree/LOS_MemMfree时，第一次会返回成功，但对同一块内存多次重复释放会导致非法指针操作，结果不可预知。

其他限制：

- 由于动态内存管理需要管理控制块数据结构来管理内存，这些数据结构会额外消耗内存，故实际用户可使用内存总量小于配置项OS_SYS_MEM_SIZE的大小。
- 由于动态内存管理的内存节点控制块结构体LosMemDynNode中，成员sizeAndFlag的数据类型为UINT32，高两位为标志位，余下的30位表示内存结点大小，因此用户初始化内存池的大小不能超过1G，否则会出现不可预知的结果。

3.2.2.3 编程实例

3.2.2.3.1 普通内存使用实例

实例描述

前提条件：在menuconfig菜单中完成动态内存的配置。

本实例执行以下步骤：

1. 初始化一个动态内存池。
2. 从动态内存池中申请一个内存块。
3. 在内存块中存放一个数据。
4. 打印出内存块中的数据。
5. 释放该内存块。

编程示例

```
#define TEST_POOL_SIZE (2*1024*1024)
UINT8 g_testPool[TEST_POOL_SIZE];

VOID Example_DynMem(VOID)
{
    UINT32 *mem = NULL;
    UINT32 ret;

    ret = LOS_MemInit(g_testPool, TEST_POOL_SIZE);
    if (LOS_OK == ret) {
        dprintf("内存池初始化成功!\n");
    } else {
        dprintf("内存池初始化失败!\n");
        return;
    }

    /*分配内存*/
    mem = (UINT32 *)LOS_MemAlloc(g_testPool, 4);
    if (NULL == mem) {
        dprintf("内存分配失败!\n");
        return;
    }
    dprintf("内存分配成功\n");
}
```



```
/*赋值*/
*mem = 828;
dprintf("*mem = %d\n", *mem);

/*释放内存*/
ret = LOS_MemFree(g_testPool, mem);
if (LOS_OK == ret) {
    dprintf("内存释放成功!\n");
} else {
    dprintf("内存释放失败!\n");
}

return;
}
```

结果验证

输出结果如下：

```
内存池初始化成功!
内存分配成功
*mem = 828
内存释放成功!
```

完整实例代码

sample_mem.c

3.2.2.3.2 多模块内存使用实例

实例描述

前提条件：在menuconfig菜单中完成多模块内存统计的配置。

本实例执行以下步骤：

1. 从动态内存池中为模块0申请一个内存块。
2. 获取模块0的内存使用量。
3. 从动态内存池中为模块1申请一个内存块。
4. 获取模块1的内存使用量。
5. 释放模块0和模块1的内存。

编程示例

```
void test(void)
{
    void *ptr = NULL;
    void *ptrTmp = NULL;
    UINT32 size = 0x10;
    UINT32 module = 0;
    UINT32 memUsed = 0;

    ptr = LOS_MemMalloc(OS_SYS_MEM_ADDR, size, module);
    if (ptr == NULL) {
        PRINTK("module %d malloc failed\n", module);
    } else {
        PRINTK("module %d malloc succeeded\n", module);
    }

    memUsed = LOS_MemUsedGet(module);
    PRINTK("module %d mem used size %d\n", module, memUsed);
}
```

```
module = 1;
ptrTmp = LOS_MemMalloc(OS_SYS_MEM_ADDR, size, module);
if (ptrTmp == NULL) {
    PRINTK("module %d malloc failed\n", module);
} else {
    PRINTK("module %d malloc succeeded\n", module);
}

memUsed = LOS_MemUsedGet(module);
PRINTK("module %d mem used size %d\n", module, memUsed);

module = 0;
(VOID)LOS_MemMfree(OS_SYS_MEM_ADDR, ptr, module);
module = 1;
(VOID)LOS_MemMfree(OS_SYS_MEM_ADDR, ptrTmp, module);
}
```

结果验证

输出结果如下：

```
module 0 malloc succeeded
module 0 mem used size 32
module 1 malloc succeeded
module 1 mem used size 32
```

完整实例代码

sample_module_mem.rar

3.2.3 静态内存

3.2.3.1 开发指导

使用场景

当用户需要使用固定长度的内存时，可以通过静态内存分配的方式获取内存，一旦使用完毕，通过静态内存释放函数归还所占用内存，使之可以重复使用。

功能

Huawei LiteOS的静态内存管理主要为用户提供以下功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
初始化静态内存池	LOS_MemboxInit	初始化一个静态内存池，根据入参设定其起始地址、总大小及每个内存块大小
清除静态内存块内容	LOS_MemboxClr	清零指定静态内存块的内容
申请、释放静态内存	LOS_MemboxAlloc	从指定的静态内存池中申请一块静态内存块
	LOS_MemboxFree	释放指定的一块静态内存块

功能分类	接口名	描述
获取、打印静态内存池信息	LOS_MemboxStatisticsGet	获取指定静态内存池的信息，包括内存池中总内存块数量、已经分配出去的内存块数量、每个内存块的大小
	LOS_ShowBox	打印指定静态内存池所有节点信息（打印等级是LOS_INFO_LEVEL），包括内存池起始地址、内存块大小、总内存块数量、每个空闲内存块的起始地址、所有内存块的起始地址

开发流程

本节介绍使用静态内存的典型场景开发流程。

1. 打开菜单，进入Kernel ---> Memory Management菜单，完成静态内存管理模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_MEMBOX	使能membox内存管理	YES/NO	YES	无
LOSCFG_KERNEL_MEMBOX_STATIC	选择静态内存方式实现membox	YES/NO	YES	LOSCFG_KERNEL_MEMBOX
LOSCFG_KERNEL_MEMBOX_DYNAMIC	选择动态内存方式实现membox	YES/NO	NO	LOSCFG_KERNEL_MEMBOX

2. 规划一片内存区域作为静态内存池。
3. 调用LOS_MemboxInit初始化静态内存池。
初始化会将入参指定的内存区域分割为N块（N值取决于静态内存总大小和块大小），将所有内存块挂到空闲链表，在内存起始处放置控制头。
4. 调用LOS_MemboxAlloc接口分配静态内存。
系统将会从空闲链表中获取第一个空闲块，并返回该内存块的起始地址。
5. 调用LOS_MemboxClr接口。
将入参地址对应的内存块清零。
6. 调用LOS_MemboxFree接口。
将该内存块加入空闲链表。

平台差异性

无。

3.2.3.2 注意事项

静态内存池区域，如果是通过动态内存分配方式获得的，在不需要静态内存池时，需要释放该段内存，避免发生内存泄露。

3.2.3.3 编程实例

实例描述

前提条件：在menuconfig菜单中完成静态内存的配置。

本实例执行以下步骤：

1. 初始化一个静态内存池。
2. 从静态内存池中申请一块静态内存。
3. 在内存块存放一个数据。
4. 打印出内存块中的数据。
5. 清除内存块中的数据。
6. 释放该内存块。

编程示例

```
VOID Example_StaticMem(VOID)
{
    UINT32 *mem = NULL;
    UINT32 blkSize = 10;
    UINT32 boxSize = 100;
    UINT32 boxMem[1000];
    UINT32 ret;

    ret = LOS_MemboxInit(&boxMem[0], boxSize, blkSize);
    if(ret != LOS_OK) {
        printf("内存池初始化失败!\n");
        return;
    } else {
        printf("内存池初始化成功!\n");
    }

    /*申请内存块*/
    mem = (UINT32 *)LOS_MemboxAlloc(boxMem);
    if (NULL == mem) {
        printf("内存分配失败!\n");
        return;
    }
    printf("内存分配成功\n");

    /*赋值*/
    *mem = 828;
    printf("**mem = %d\n", *mem);

    /*清除内存内容*/
    LOS_MemboxClr(boxMem, mem);
    printf("清除内存内容成功\n *mem = %d\n", *mem);

    /*释放内存*/
    ret = LOS_MemboxFree(boxMem, mem);
    if (LOS_OK == ret) {
        printf("内存释放成功!\n");
    } else {
        printf("内存释放失败!\n");
    }
}
```

```
    return;  
}
```

结果验证

输出结果如下：

```
内存池初始化成功!  
内存分配成功  
*mem = 828  
清除内存内容成功  
*mem = 0  
内存释放成功!
```

完整实例代码

sample_membox.c

3.3 中断

3.3.1 概述

基本概念

中断是指出现需要时，CPU暂停执行当前程序，转而执行新程序的过程。即在程序运行过程中，出现了一个必须由CPU立即处理的事务。此时，CPU暂时中止当前程序的执行转而处理这个事务，这个过程就叫做中断。

外设可以在没有CPU介入的情况下完成一定的工作，但某些情况下也需要CPU为其执行一定的工作。通过中断机制，在外设不需要CPU介入时，CPU可以执行其它任务，而当外设需要CPU时，将通过产生中断信号使CPU立即中断当前任务来响应中断请求。这样可以使CPU避免把大量时间耗费在等待、查询外设状态的操作上，有效提高系统实时性以及执行效率。

Huawei LiteOS的中断特性：

- 中断共享，且可配置。
- 中断嵌套，即高优先级的中断可抢占低优先级的中断，且可配置。
- 使用独立中断栈，可配置。
- 可配置支持的中断优先级个数。
- 可配置支持的中断数。
- 中断底半部，且可配置。

中断相关的硬件介绍

与中断相关的硬件可以划分为三类：设备、中断控制器、CPU本身。

- 设备
发起中断的源，当设备需要请求CPU时，产生一个中断信号，该信号连接至中断控制器。
- 中断控制器

中断控制器是CPU众多外设中的一个，它一方面接收其它外设中断引脚的输入，另一方面，它会发出中断信号给CPU。可以通过对中断控制器编程来打开和关闭中断源、设置中断源的优先级和触发方式。常用的中断控制器有VIC（Vector Interrupt Controller）、GIC（General Interrupt Controller）、PLIC（Platform-Level Interrupt Controller）和CLIC（Core-Local Interrupt Controller）。在ARM Cortex-M系列中使用的中断控制器是NVIC（Nested Vector Interrupt Controller）。在ARM Cortex-A7中使用的中断控制器是GIC。在RISC-V中使用的中断控制器是PLIC和CLIC。

- CPU

CPU会响应中断源的请求，中断当前正在执行的任务，转而执行中断处理程序。

中断相关概念

中断号

每个中断请求信号都会有特定的标志，使得计算机能够判断是哪个设备提出的中断请求，这个标志就是中断号。

中断请求

“紧急事件”需向CPU提出申请（发一个电脉冲信号），要求中断，及要求CPU暂停当前执行的任务，转而处理该“紧急事件”，这一申请过程称为中断请求。

中断优先级

为使系统能够及时响应并处理所有中断，系统根据中断时间的重要性和紧迫程度，将中断源分为若干个级别，称作中断优先级。

中断处理程序

当外设产生中断请求后，CPU暂停当前的任务，转而响应中断申请，即执行中断处理程序。产生中断的每个设备都有相应的中断处理程序。

中断嵌套

中断嵌套也称为中断抢占，指的是正在执行一个中断处理程序时，如果有另一个优先级更高的中断源提出中断请求，这时会暂时终止当前正在执行的优先级较低的中断源的中断处理程序，转而去处理更高优先级的中断请求，待处理完毕，再返回到之前被中断的处理程序中继续执行。

中断触发

中断源向中断控制器发送中断信号，中断控制器对中断进行仲裁，确定优先级，将中断信号送给CPU。中断源产生中断信号的时候，会将中断触发器置“1”，表明该中断源产生了中断，要求CPU去响应该中断。

中断触发类型

外部中断申请通过一个物理信号发送到NVIC/GIC，可以是电平触发或边沿触发。

中断向量

中断服务程序的入口地址。

中断向量表

存储中断向量的存储区，中断向量与中断号对应，中断向量在中断向量表中按照中断号顺序存储。

中断共享

当外设较少时，可以实现一个外设对应一个中断号，但为了支持更多的硬件设备，可以让多个设备共享一个中断号，共享同一个中断号的中断处理程序形成一个链表。当外部设备产生中断申请时，系统会遍历执行中断号对应的中断处理程序链表直到找到对应设备的中断处理程序。在遍历执行过程中，各中断处理程序可以通过检测设备ID，判断是否是这个中断处理程序对应的设备产生的中断。

核间中断

对于多核系统，中断控制器允许一个CPU的硬件线程去中断其他CPU的硬件线程，这种方式被称为核间中断。核间中断的实现基础是多CPU内存共享，采用核间中断可以减少某个CPU负荷过大，有效提升系统效率。目前只有GIC中断控制器与LingLong中断控制器支持核间中断。

中断底半部

区别于中断处理程序，中断底半部运行在任务上下文，用户可在中断处理程序中，将实时性要求不高，非紧急的业务添加到中断底半部执行。

运作机制

- Huawei LiteOS的中断机制支持中断共享：
中断共享的实现依赖于链表，对应每一个中断号创建一个链表，链表节点中包含注册的中断处理函数和函数入参。当对同一中断号多次创建中断时，将中断处理函数和函数入参添加到中断号对应的链表中，因此当硬件产生中断时，通过中断号查找到其对应的链表，遍历执行链表直到找到对应设备的中断处理函数。
- Huawei LiteOS的中断嵌套：
GIC与NVIC的中断嵌套由硬件实现。
RISC-V的中断嵌套实现机制为：中断嵌套下，中断A触发后会将当前的操作进行压栈，调用中断处理程序前，将MIE设置为1，允许新的中断被响应。在A执行中断处理程序的过程中，如果有更高优先级的中断B被触发，B会将当前的操作即中断A相关的操作进行压栈，然后执行B的中断处理程序。待B的中断处理程序执行完后，会暂时的将mstatus寄存器中的MIE域置为0，关闭中断响应，将中断A相关的操作进行出栈，将MIE设置为1，允许处理器再次响应中断，中断B结束，继续执行中断A。
- Huawei LiteOS的中断底半部：
中断底半部依赖链表、任务与事件机制。
在中断处理程序中，用户添加底半部处理函数到队列中，待中断退出后，在中断底半部任务中逐个执行用户添加的底半部处理函数。

3.3.2 开发指导

使用场景

当有中断请求产生时，CPU暂停当前的任务，转而去响应外设请求。根据需要，用户通过中断申请，注册中断处理程序，可以指定CPU响应中断请求时所执行的具体操作。

功能

Huawei LiteOS 的中断模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
创建和删除中断	LOS_HwiCreate	中断创建，注册中断号、中断触发模式、中断优先级、中断处理程序。中断被触发时，handleIrq会调用该中断处理程序
	LOS_HwiDelete	删除中断，请注意不要在注册的中断 handler中调用此接口，共享模式下，可能会访问非法内存
打开和关闭所有中断	LOS_IntUnlock	打开当前处理器所有中断响应
	LOS_IntLock	关闭当前处理器所有中断响应
	LOS_IntRestore	恢复到使用LOS_IntLock关闭所有中断之前的状态
使能和屏蔽指定中断	LOS_HwiDisable	中断屏蔽（通过设置寄存器，禁止CPU响应该中断）
	LOS_HwiEnable	中断使能（通过设置寄存器，允许CPU响应该中断）
设置中断优先级	LOS_HwiSetPriority	设置中断优先级
触发中断	LOS_HwiTrigger	触发中断（通过写中断控制器的相关寄存器模拟外部中断）
清除中断寄存器状态	LOS_HwiClear	清除中断号对应的中断寄存器的状态位，此接口依赖中断控制器版本，非必需
核间中断	LOS_HwiSendIpi	向指定核发送核间中断，此接口依赖中断控制器版本和cpu架构，该函数仅在SMP模式下支持
设置中断亲和性	LOS_HwiSetAffinity	设置中断的亲和性，即设置中断在固定核响应（该函数仅在SMP模式下支持）
设置中断前处理钩子函数	LOS_HwiPreHookReg	设置中断前处理钩子函数
设置中断后处理钩子函数	LOS_HwiPostHookReg	设置中断后处理钩子函数
获取中断响应次数	LOS_HwiRespCntGet	获取中断的响应次数
添加中断底半部函数	LOS_HwiBottomWorkAdd	添加中断底半部函数

HWI 错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_HWI_NUM_INVALID (OS_ERRNO_HWI_NUM_INVALID)	0x02000900	创建或删除中断时，传入了无效中断号。老版本的错误码 OS_ERRNO_HWI_NUM_INVALID后续不再支持。	检查中断号，给定有效中断号。
2	LOS_ERRNO_HWI_PROC_FUNC_NULL (OS_ERRNO_HWI_PROC_FUNC_NULL)	0x02000901	创建中断时，传入的中断处理程序指针为空；如果调用其他接口返回此错误码则表示该接口功能不支持。老版本的错误码 OS_ERRNO_HWI_PROC_FUNC_NULL后续不再支持。	传入非空中断处理程序指针。
3	LOS_ERRNO_HWI_NO_MEMORY (OS_ERRNO_HWI_NO_MEMORY)	0x02000903	创建中断时，出现内存不足的情况。老版本的错误码 OS_ERRNO_HWI_NO_MEMORY后续不再支持。	增大动态内存空间，有两种方式可以实现： • 设置更大的系统动态内存池，配置项为 OS_SYS_MEM_SIZE • 释放一部分动态内存
4	LOS_ERRNO_HWI_ALREADY_CREATED (OS_ERRNO_HWI_ALREADY_CREATED)	0x02000904	创建中断时，发现要注册的中断号已经创建。老版本的错误码 OS_ERRNO_HWI_ALREADY_CREATED后续不再支持。	对于非共享中断号的情况，检查传入的中断号是否已经被创建；对于共享中断号的情况，检查传入中断号的链表中是否已经有匹配函数参数的设备ID。
5	LOS_ERRNO_HWI_PRIORITY_INVALID (OS_ERRNO_HWI_PRIORITY_INVALID)	0x02000905	创建中断时，传入的中断优先级无效。老版本的错误码 OS_ERRNO_HWI_PRIORITY_INVALID后续不再支持。	传入有效中断优先级。优先级有效范围依赖于硬件，外部可配。

序号	定义	实际数值	描述	参考解决方案
6	LOS_ERRNO_HWI_INTERRUPT (OS_ERRNO_HWI_INTERRUPT)	0x02000908	在中断中调用 request_irq接口或 bus_request_intr接口。老版本的错误码 OS_ERRNO_HWI_INTERRUPT后续不再支持。注意开源版本中没有这两个接口。	查看request_irq、bus_request_intr接口的使用是否正确。
7	LOS_ERRNO_HWI_SHARED_ERROR (OS_ERRNO_HWI_SHARED_ERROR)	0x02000909	创建中断时：发现 hwiMode指定创建共享中断，但是未设置设备ID；或hwiMode指定创建非共享中断，但是该中断号之前已创建为共享中断；或配置 LOSCFG_SHARED_IRQ为NO，但是创建中断时，入参指定创建共享中断。 删除中断时：设备号创建时指定为共享中断，删除时未设置设备ID，删除错误。老版本的错误码 OS_ERRNO_HWI_SHARED_ERROR后续不再支持。	检查入参，创建时参数 hwiMode与irqParam保持一致，hwiMode为0，表示不共享，此时 irqParam应为NULL；当 hwiMode为 IRQF_SHARED时表示共享，irqParam需设置设备ID；LOSCFG_SHARED_IRQ为NO时，即非共享中断模式下，只能创建非共享中断。删除中断时 irqParam要与创建中断时的参数一样。
8	LOS_ERRNO_HWI_HWINUM_UNCREATE (OS_ERRNO_HWI_HWINUM_UNCREATE)	0x0200090b	中断共享情况下，删除中断时，中断号对应的链表中，无法匹配到相应的设备ID。老版本的错误码 OS_ERRNO_HWI_HWINUM_UNCREATE后续不再支持。	对于共享中断号的情况，检查传入中断号的链表中是否已经有匹配函数参数的设备ID。
9	LOS_ERRNO_HWI_NO_CPUP_MEMORY	0x0200090c	使能CPUP开关 LOSCFG_CPUP_CB_NUM_CONFIGURABLE后，创建中断时，如果CPUP的中断相关控制块数目不够，会返回此错误。	根据实际的中断个数，而非中断最大标号，确定配置宏 LOSCFG_CPUP_IRQ_CB_NUM，最大不超过 LOSCFG_PLATFORM_HWI_LIMIT。
10	LOS_ERRNO_HWI_NOT_INTERRUPT_CONTEXT	0x0200090d	调用 LOS_HwiBhworkAdd时不在中断上下文。	在中断上下文中调用 LOS_HwiBhworkAdd。

序号	定义	实际数值	描述	参考解决方案
11	LOS_ERRNO_HWI_PTR_NULL	0x0200090e	函数入参指针为空。	入参传入有效的非空指针。
12	LOS_ERRNO_HWI_ARG_NOT_ENABLED	0x0200090f	没有开启LOSCFG_HWI_WITH_ARG时，LOS_HwiCreate最后一个参数传入了非空指针。	LOS_HwiCreate最后一参数传入NULL或者开启LOSCFG_HWI_WITH_ARG。

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为中断模块，值为0x09。

开发流程

1. 打开菜单，进入Kernel ---> Interrupt Management菜单，完成中断模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_ARCH_INTERRUPT_PREEMPTION	使能中断嵌套	YES/NO	NO	GICv3、RISC-V
LOSCFG_IRQ_USE_STANDALONE_STACK	使用独立中断栈	YES/NO	YES	依赖CPU架构，目前Cortex-A、Cortex-R、ARM64、LingLong支持可选，RISC-V默认使用独立中断栈
LOSCFG_SHARED_IRQ	使能中断共享	YES/NO	YES	无
LOSCFG_PLATFORM_HWI_LIMIT	最大中断使用数	根据芯片手册适配	根据芯片手册适配	无
LOSCFG_HWI_PRIORITY_LIMIT	可设置的中断优先级个数	根据芯片手册适配	根据芯片手册适配	无
LOSCFG_IRQ_STACK_SIZE	中断栈大小	根据板子自适应	0x2000	目前只有RISC-V可配置
LOSCFG_NMI_STACK_SIZE	不可屏蔽中断栈大小	根据板子自适应	0x800	目前只有RISC-V可配置

配置项	含义	取值范围	默认值	依赖
LOSCFG_HWI_PRE_POST_PROCESS	中断前后处理钩子函数	YES/NO	NO	无
LOSCFG_HWI_BOTTOM_HALF	使能中断底半部	YES/NO	NO	LOSCFG_BASE_IPC_EVENT
LOSCFG_HWI_BOTTOM_HALF_WORK_LIMIT	中断底半部队列最多缓存的个数	根据板子自适应	16	LOSCFG_HWI_BOTTOM_HALF

- 调用中断创建接口LOS_HwiCreate创建中断。
- 如果是SMP模式，调用LOS_HwiSetAffinity设置中断的亲中性，否则直接进入步骤4。
- 调用LOS_HwiEnable接口使能指定中断。
- 调用LOS_HwiTrigger接口触发指定中断（该接口通过写中断控制器的相关寄存器模拟外部中断，一般的外设设备，不需要执行这一步）。
- 在中断处理函数中，调用LOS_HwiBhworkAdd接口添加中断底半部处理函数（如果无底半部处理函数，则可以不执行这一步）。
- 调用LOS_HwiDisable接口屏蔽指定中断，此接口根据实际情况使用，判断是否需要屏蔽中断。
- 调用LOS_HwiDelete接口删除指定中断，此接口根据实际情况使用，判断是否需要删除中断。

3.3.3 注意事项

中断处理：

- 中断处理程序耗时不能过长，否则会影响CPU对中断的及时响应。
- 中断响应过程中不能执行引起调度的函数。

中断共享：

- 中断共享机制，支持不同的设备使用相同的中断号注册同一中断处理程序，但中断处理程序的入参pDevId（设备号）必须唯一，代表不同的设备。即同一中断号，同一-dev只能挂载一次；但同一中断号，同一中断处理程序，dev不同则可以重复挂载。

中断触发：

- RISC-V himideerV200不支持LOS_HwiTrigger接口，因为RISC-V himideerV200中断控制器pending寄存器不可写；用户需要模拟中断触发进行测试，请使用软中断或外设中断验证中断触发流程。

中断寄存器状态清除：

- RISC-V himideerV200开启中断嵌套时，对于电平触发方式，需要在中断处理程序结束时调用LOS_HwiClear接口，主动清除中断寄存器状态。即对于RISC-V himideerV200，LOS_HwiClear接口只有在中断嵌套场景生效；非嵌套场景用户无需清除中断控制器状态，LOS_HwiClear接口无清除中断状态效果。

中断底半部：

- 中断底半部用户处理函数中禁止调用延时接口，避免调用阻塞类接口。中断底半部任务运行时具有不确定性，只有当中断底半部任务优先级最高，才会运行。

兼容接口：

- Huawei LiteOS有兼容Linux的中断接口，request_irq、disable_irq、enable_irq、free_irq，详见[中断](#)。开发者可根据自身需要，选择适合的接口。

其他限制：

- 根据具体硬件，配置支持的最大中断数及可设置的中断优先级个数。
- 中断恢复LOS_IntRestore()的入参必须是与之对应的LOS_IntLock()的返回值（即关中断之前的CPSR值）。
- Cortex-M系列处理器中0-15中断为内部使用，Cortex-A7中0-31中断为内部使用，因此不建议用户去申请和创建。

3.3.4 编程实例

实例描述

本实例实现如下功能：

1. 创建中断。
2. 设置中断亲和性。
3. 使能中断。
4. 触发中断。
5. 屏蔽中断。
6. 删除中断。

编程示例

前提条件：menuconfig菜单中配置中断使用最大数、配置可设置的中断优先级个数。

代码实现如下：

```
#include "los_hwi.h"
#include "los_typedef.h"
#include "los_task.h"

STATIC UINT32 g_BhworkCnt = 0;

STATIC VOID HwiBhfunc(VOID *data)
{
    UINT32 *count = (UINT32 *data);
    (*count)++;
}

STATIC VOID HwiUsrIrq(VOID)
{
    UINT32 ret;
    ret = LOS_HwiBhworkAdd(HwiBhfunc, &g_BhworkCnt);
    if (ret != LOS_OK) {
        printf("\n hwi bhwork add failed, %x \n", ret);
    }
    printf("\n in the func HwiUsrIrq \n");
}

/* cpu0 trigger, cpu0 response */
UINT32 It_Hwi_001(VOID)
```

```
{
    UINT32 ret;
    HWI_HANDLE_T irqNum = 26;
    HWI_PRIOR_T irqPri = 0x3;

    ret = LOS_HwiCreate(irqNum, irqPri, 0, (HWI_PROC_FUNC)HwiUsrIrq, 0);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

#ifdef LOSCFG_KERNEL_SMP
    ret = LOS_HwiSetAffinity(irqNum, CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
#endif
    ret = LOS_HwiEnable(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    ret = LOS_HwiTrigger(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    LOS_TaskDelay(1);

    ret = LOS_HwiDisable(irqNum);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    ret = LOS_HwiDelete(irqNum, NULL);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    return LOS_OK;
}
```

完整实例

sample_hwi.rar

3.4 异常接管

3.4.1 概述

基本概念

异常接管是操作系统对运行期间发生的异常情况（芯片硬件异常）进行处理的一系列动作，例如打印异常发生时当前函数的调用栈信息、CPU现场信息、任务的堆栈情况等。

异常接管作为一种调测手段，可以在系统发生异常时给用户提供有用的异常信息，譬如异常类型、发生异常时的系统状态等，方便用户定位分析问题。

Huawei LiteOS的异常接管，在系统发生异常时的处理动作为：显示异常发生时正在运行的任务信息（包括任务名、任务号、堆栈大小等），以及CPU现场等信息。针对某些RISC-V架构的芯片，对内存size要求较高的场景，Huawei LiteOS提供了极小特性宏

LOSCFG_EXC_SIMPLE_INFO (menuconfig菜单项为: Kernel --> Exception Management --> Enable Exception Simple Info) , 用于裁剪多余的异常提示字符串信息, 但是仍然保留发生异常时的CPU执行环境的所有信息。

运作机制

每个函数都有自己的栈空间, 称为栈帧。调用函数时, 会创建子函数的栈帧, 同时将函数入参、局部变量、寄存器入栈。栈帧从高地址向低地址生长。

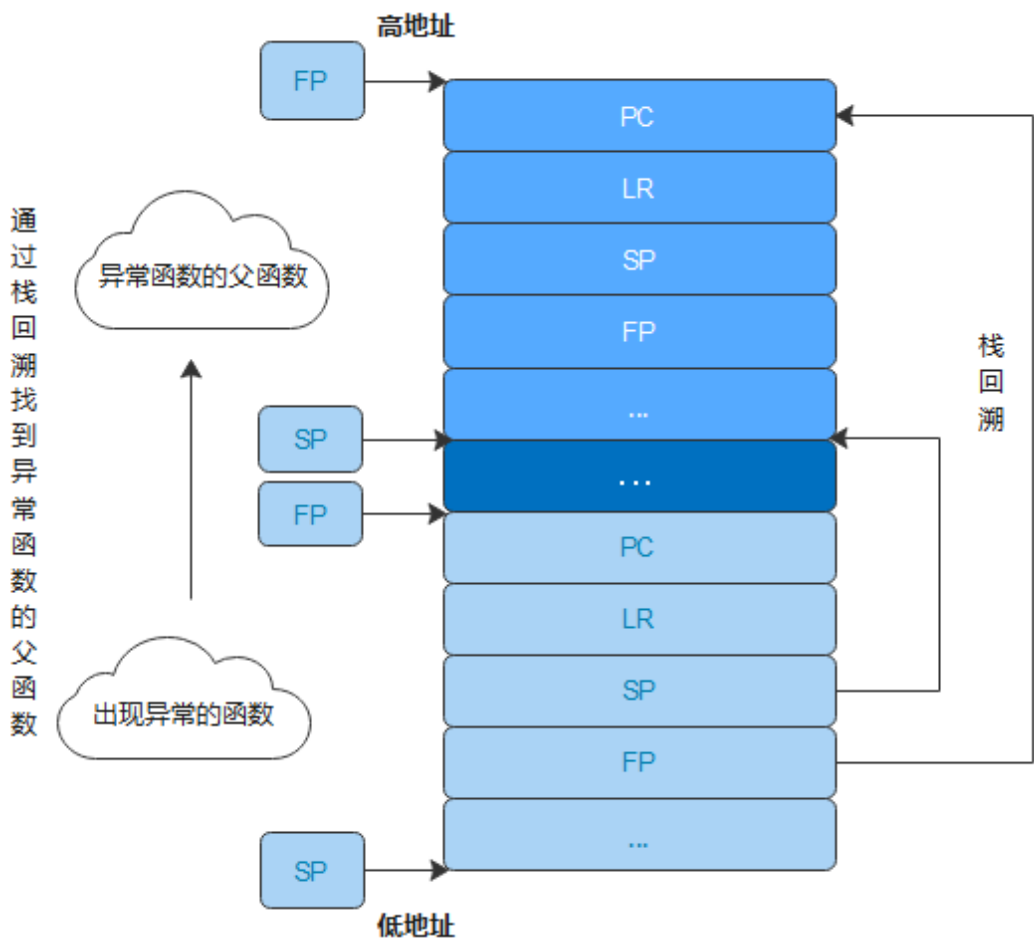
以ARM32 CPU架构为例, 每个栈帧中都会保存PC、LR、SP和FP寄存器的历史值。

堆栈分析

- LR寄存器 (Link Register) , 链接寄存器, 指向函数的返回地址。
- R11: 可以用作通用寄存器, 在开启特定编译选项时可以用作帧指针寄存器FP, 用来实现栈回溯功能。
GNU编译器 (gcc) 默认将R11作为存储变量的通用寄存器, 因而默认情况下无法使用FP的栈回溯功能。为支持调用栈解析功能, 需要在编译参数中添加-fno-omit-frame-pointer选项, 提示编译器将R11作为FP使用。
- FP寄存器 (Frame Point) , 帧指针寄存器, 指向当前函数的父函数的栈帧起始地址。利用该寄存器可以得到父函数的栈帧, 从栈帧中获取父函数的FP, 就可以得到祖父函数的栈帧, 以此类推, 可以追溯程序调用栈, 得到函数间的调用关系。
当系统发生异常时, 系统打印异常函数的栈帧中保存的寄存器内容, 以及父函数、祖父函数的栈帧中的LR、FP寄存器内容, 用户就可以据此追溯函数间的调用关系, 定位异常原因。

堆栈分析原理如下图所示, 实际堆栈信息根据不同CPU架构有所差异, 此处仅做示意。

图 3-6 堆栈分析原理示意图



图中不同颜色的寄存器表示不同的函数。可以看到函数调用过程中，寄存器的保存。通过FP寄存器，栈回溯到异常函数的父函数，继续按照规律对栈进行解析，推出函数调用关系，方便用户定位问题。

3.4.2 使用指南

功能

异常接管对系统运行期间发生的芯片硬件异常进行处理，不同芯片的异常类型存在差异，具体异常类型可以查看芯片手册。

定位流程

异常接管一般的定位步骤如下：

- 1. 打开编译后生成的镜像反汇编（asm）文件。
- 2. 搜索PC指针（指向当前正在执行的指令）在asm中的位置，找到发生异常的函数。
- 3. 根据LR值查找异常函数的父函数。
- 4. 重复步骤3，得到函数间的调用关系，找到异常原因。

具体的定位方法会在实例中举例说明。

配置项

打开菜单，进入Kernel ---> Exception Management菜单，完成异常接管模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_EXC_SIMPLE_INFO	使能后将减少异常输出的信息	YES/NO	NO	无
LOSCFG_EXC_STACK_SIZE	异常栈大小	根据板子自适应	0x800	目前只有RISC-V可配置

3.4.3 注意事项

要查看调用栈信息，必须添加编译选项宏-fno-omit-frame-pointer支持stack frame，否则编译时FP寄存器是关闭的。

3.4.4 问题定位实例

在某ARM32平台上通过错误释放内存，触发系统异常。系统异常被挂起后，能在串口中看到异常调用栈打印信息和关键寄存器信息，如下所示，其中excType表示异常类型，此处值为4表示数据终止异常，其它数值可以查看芯片手册。通过这些信息可以定位到异常所在函数和其调用栈关系，为分析异常原因提供第一手资料。

```
excType: 4
taskName = MNT_send
taskId = 6
task stackSize = 12288
excBuffAddr pc = 0x8034d3cc
excBuffAddr lr = 0x8034d3cc
excBuffAddr sp = 0x809ca358
excBuffAddr fp = 0x809ca36c
*****backtrace begin*****
traceback 0 -- lr = 0x803482fc
traceback 0 -- fp = 0x809ca38c
traceback 1 -- lr = 0x80393e34
traceback 1 -- fp = 0x809ca3a4
traceback 2 -- lr = 0x8039e0d0
traceback 2 -- fp = 0x809ca3b4
traceback 3 -- lr = 0x80386bec
traceback 3 -- fp = 0x809ca424
traceback 4 -- lr = 0x800a6210
traceback 4 -- fp = 0x805da164
```

定位步骤如下：

1. 打开编译后生成的 asm 反汇编文件（默认生成在Huawei_LiteOS/out/<platform>目录下，其中的platform为具体的平台名）。
2. 搜索PC指针 8034d3cc 在asm文件中的位置（去掉0x）。

PC地址指向发生异常时程序正在执行的指令。在当前执行的二进制文件对应的asm文件中，查找PC值8034d3cc，找到当前CPU正在执行的指令行，得到如下图所示结果。

```

8034d3bc <osSlabMemFree>:
8034d3bc: e92d48f0    push    {r4, r5, r6, r7, fp, lr}
8034d3c0: e1a04001    mov     r4, r1
8034d3c4: e28db014    add     fp, sp, #20
8034d3c8: ebfff21e    bl      80349c48 <osSlabCtrlHdrGet>
8034d3cc: e15420b4    ldrh    r2, [r4, #-4]
8034d3d0: e59f308c    ldr     r3, [pc, #140] ; 8034d464 <osSlabMemFree+0xa8>
8034d3d4: e1520003    cmp     r2, r3

```

从图中可以看到：

- 异常时CPU正在执行的指令是ldrh r2, [r4, #-4]。
- 异常发生在函数osSlabMemFree中。

结合ldrh指令分析，此指令是从内存的(r4-4)地址中读值，将其load到寄存器r2中。再结合异常时打印的寄存器信息，查看此时r4的值。下图是异常时打印的寄存器信息，可以看到，r4此时值是0xffffffff。

```

R0      = 0x807243c8
R1      = 0xffffffff
R2      = 0x80000013
R3      = 0x80623de0
R4      = 0xffffffff
R5      = 0x80000013
R6      = 0x807243c0
R7      = 0x1
R8      = 0x809ca490
R9      = 0x4e4
R10     = 0x11
R11     = 0x809ca36c
R12     = 0x800000d3
SP      = 0x809ca358
LR      = 0x8034d3cc
PC      = 0x8034d3cc
CPSR    = 0xa00000d3

```

显然，r4的值超出了内存范围，故CPU执行到该指令时发生了数据终止异常。根据汇编知识，从asm文件可以看到，r4是从r1 mov过来，而r1是函数第二个入参，于是可以确认，在调用osSlabMemFree时传入了0xffffffff（或-1）这样一个错误入参。

接下来，需要查找谁调用了osSlabMemFree函数。

- 根据 LR链接寄存器值 查找调用栈。

从异常信息的backtrace begin开始，打印的是调用栈信息。在asm文件中查找backtrace 0对应的LR，如下图所示。

```
803482c4 <LOS_MemFree>:
803482c4: e92d4bf0 push {r4, r5, r6, r7, r8, r9, fp, lr}
803482c8: e28db01c add fp, sp, #28
803482cc: e10f5000 mrs r5, CPSR
803482d0: e385c0c0 orr ip, r5, #192 ; 0xc0
803482d4: e121f00c msr CPSR_c, ip
803482d8: e3510000 cmp r1, #0
803482dc: 13500000 cmpne r0, #0
803482e0: 1a000002 bne 803482f0 <LOS_MemFree+0x2c>
803482e4: e3a00001 mov r0, #1
803482e8: e121f005 msr CPSR_c, r5
803482ec: e8bd8bf0 pop {r4, r5, r6, r7, r8, r9, fp, pc}
803482f0: e1a04001 mov r4, r1
803482f4: e1a06000 mov r6, r0
803482f8: eb00142f bl 8034d3bc <osSlabMemFree>
803482fc: e3500001 cmp r0, #1
80348300: 1afffff8 bne 803482e8 <LOS_MemFree+0x24>
```

可见，是LOS_MemFree调用了osSlabMemFree。依此方法，可得到异常时函数调用关系如下：MNT_buf_send(业务函数) -> free -> LOS_MemFree -> osSlabMemFree。

最终，通过排查业务中MNT_buf_send实现，发现其中存在错误使用指针的问题，导致free了一个错误地址，引发上述异常。

3.5 错误处理

3.5.1 概述

基本概念

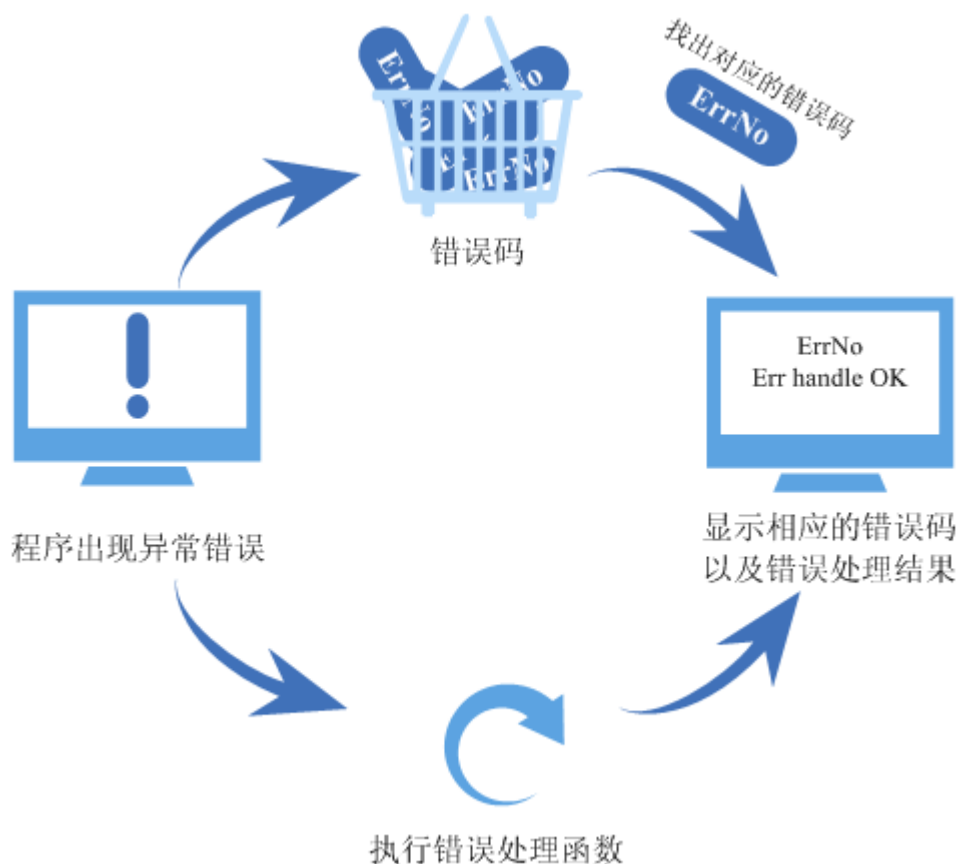
错误处理指程序运行错误时，调用错误处理模块的接口函数，上报错误信息，并调用注册的钩子函数进行特定处理，保存现场以便定位问题。

通过错误处理，可以控制和提示程序中的非法输入，防止程序崩溃。

运作机制

错误处理是一种机制，用于处理异常状况。当程序出现错误时，会显示相应的错误码。此外，如果注册了相应的错误处理函数，则会执行这个函数。

图 3-7 错误处理示意图



3.5.2 开发指导

错误码简介

调用API接口时可能会出现错误，此时接口会返回对应的错误码，以便快速定位错误原因。

错误码是一个32位的无符号整型数，31~24位表示错误等级，23~16位表示错误码标志（当前该标志值为0），15~8位代表错误码所属模块，7~0位表示错误码序号。

例如将任务模块中的错误码LOS_ERRNO_TSK_NO_MEMORY定义为致命级别的错误，模块ID为LOS_MOD_TSK，错误码序号为0，其定义如下：

```
#define LOS_ERRNO_TSK_NO_MEMORY LOS_ERRNO_OS_FATAL(LOS_MOD_TSK, 0x00)
#define LOS_ERRNO_OS_FATAL(MID, ERRNO) \
    (LOS_ERRTYPE_FATAL | LOS_ERRNO_OS_ID | ((UINT32)(MID) << 8) | ((UINT32)(ERRNO)))
```

说明

- LOS_ERRTYPE_FATAL：错误等级为致命，值为0x03000000。
- LOS_ERRNO_OS_ID：错误码标志，值为0x000000。
- MID：所属模块，LOS_MOD_TSK的值为0x2。
- ERRNO：错误码序号。

所以LOS_ERRNO_TSK_NO_MEMORY的值为0x03000200。

错误码接管

有时只靠错误码不能快速准确的定位问题，为方便用户分析错误，错误处理模块支持注册错误处理的钩子函数，发生错误时，用户可以调用LOS_ErrHandle接口以执行错误处理函数。

Huawei LiteOS 的错误处理模块为用户提供下面几个接口，接口详细信息请参见API参考。

接口名	描述	参数	备注
LOS_RegErrHandle	注册错误处理钩子函数。	func：错误处理钩子函数	-
LOS_ErrHandle	调用钩子函数，处理错误。	fileName：存放错误日志的文件名	系统内部调用时，入参为"os_unspecific_file"。
		lineNo：发生错误的代码行号	系统内部调用时，若值为0xa1b2c3f8，表示未传递行号。
		errnoNo：错误码	-
		paraLen：入参para的长度	系统内部调用时，入参为0。
		para：错误标签	系统内部调用时，入参为NULL。

须知

系统内部会在某些难以定位的错误处，主动调用注册的钩子函数（目前只在互斥锁模块和信号量模块中主动调用了钩子函数）。

3.5.3 注意事项

系统中有且仅有一个错误处理的钩子函数。当多次注册钩子函数时，最后一次注册的钩子函数会覆盖前一次注册的函数。

3.5.4 编程实例

实例描述

本实例演示功能如下：

- 1. 注册错误处理钩子函数。
- 2. 执行错误处理函数。

编程示例

代码实现如下：

```
#include "los_err.h"
#include "los_typedef.h"
#include <stdio.h>

void Test_ErrHandle(CHAR *fileName, UINT32 lineNo, UINT32 errorNo, UINT32 paraLen, VOID *para)
{
    printf("err handle ok\n");
}

static UINT32 TestCase(VOID)
{
    UINT32 errNo = 0;
    UINT32 ret;
    UINT32 errLine = 16;

    LOS_RegErrHandle(Test_ErrHandle);

    ret = LOS_ErrHandle("os_unspecific_file", errLine, errNo, 0, NULL);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
Huawei LiteOS # err handle ok
```

完整实例代码

sample_err.c

3.6 队列

3.6.1 概述

基本概念

队列又称消息队列，是一种常用于任务间通信的数据结构。队列接收来自任务或中断的不固定长度消息，并根据不同的接口确定传递的消息是否存放在队列空间中。

任务能够从队列里面读取消息，当队列中的消息为空时，挂起读取任务；当队列中有新消息时，挂起的读取任务被唤醒并处理新消息。任务也能够往队列里写入消息，当队列已经写满消息时，挂起写入任务；当队列中有空闲消息节点时，挂起的写入任务被唤醒并写入消息。如果将读队列和写队列的超时时间设置为0，则不会挂起任务，接口会直接返回，这就是非阻塞模式。

消息队列提供了异步处理机制，允许将一个消息放入队列，但不立即处理。同时队列还有缓冲消息的作用。

Huawei LiteOS中使用队列实现任务异步通信，具有如下特性：

- 消息以先进先出的方式排队，支持异步读写。
- 读队列和写队列都支持超时机制。

- 每读取一条消息，就会将该消息节点设置为空闲。
- 发送消息类型由通信双方约定，可以允许不同长度（不超过队列的消息节点大小）的消息。
- 一个任务能够从任意一个消息队列接收和发送消息。
- 多个任务能够从同一个消息队列接收和发送消息。
- 创建队列时所需的队列空间，默认支持接口内系统自行动态申请内存的方式，同时也支持将用户分配的队列空间作为接口入参传入的方式。

运作机制

队列控制块

```
typedef enum {
    OS_QUEUE_READ = 0,
    OS_QUEUE_WRITE = 1,
    OS_QUEUE_N_RW = 2
} QueueReadWrite;

/**
 * Queue information block structure
 */
typedef struct
{
    UINT8    *queueHandle;           /* 队列指针 */
    UINT8    queueState;             /* 队列状态 */
    UINT8    queueMemType;           /* 创建队列时内存分配的方式 */
    UINT16   queueLen;               /* 队列中消息节点个数，即队列长度 */
    UINT16   queueSize;              /* 消息节点大小 */
    UINT32   queueID;                /* 队列ID */
    UINT16   queueHead;              /* 消息头节点位置（数组下标） */
    UINT16   queueTail;              /* 消息尾节点位置（数组下标） */
    UINT16   readWriteableCnt[OS_QUEUE_N_RW]; /* 数组下标0的元素表示队列中可读消息数，
                                           数组下标1的元素表示队列中可写消息数 */
    LOS_DL_LIST readWriteList[OS_QUEUE_N_RW]; /* 读取或写入消息的任务等待链表，
                                           下标0：读取链表，下标1：写入链表 */
    LOS_DL_LIST memList;              /* CMSIS-RTOS中的MailBox模块使用的内存块链表 */
} LosQueueCB;
```

每个队列控制块中都含有队列状态，表示该队列的使用情况：

- OS_QUEUE_UNUSED：队列没有被使用。
- OS_QUEUE_INUSED：队列被使用中。

每个队列控制块中都含有创建队列时的内存分配方式：

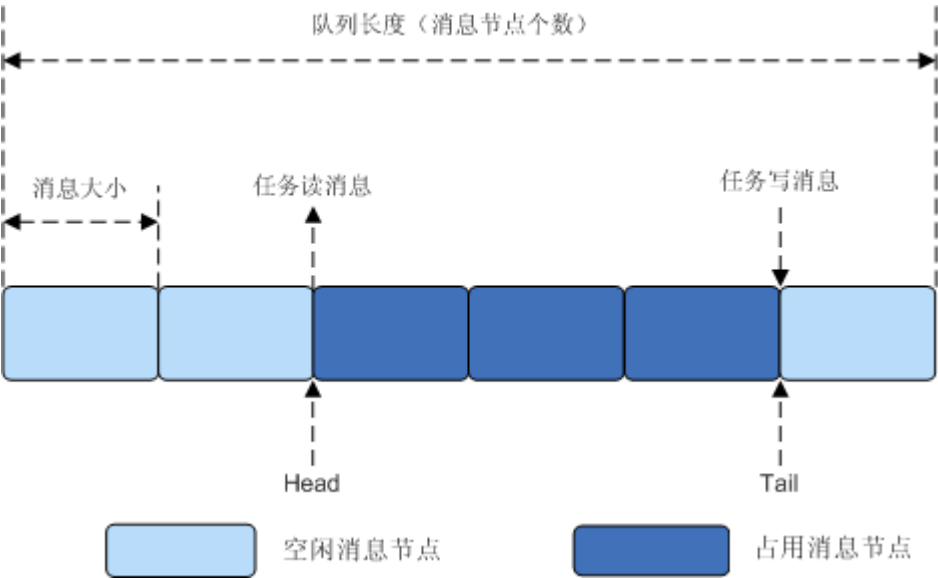
- OS_QUEUE_ALLOC_DYNAMIC：创建队列时所需的队列空间，由系统自行动态申请内存获取。
- OS_QUEUE_ALLOC_STATIC：创建队列时所需的队列空间，由接口调用者自行申请后传入接口。

队列运作原理

表 3-1

功能	运作原理描述
创建队列	创建队列成功会返回队列ID。 在队列控制块中维护着一个消息头节点位置Head和一个消息尾节点位置Tail，用于表示当前队列中消息的存储情况。 • Head表示队列中被占用的消息节点的起始位置。 • Tail表示被占用的消息节点的结束位置，也是空闲消息节点的起始位置。 （注：队列刚创建时，Head和Tail均指向队列起始位置。）
写队列	根据readWriteableCnt[1]判断队列是否可以写入，不能对已满（readWriteableCnt[1]为0）队列进行写操作。 写队列支持以下2种写入方式。 • 向队列尾节点写入：尾节点写入时，根据Tail找到起始空闲消息节点作为数据写入对象，如果Tail已经指向队列尾部则采用回卷方式。 • 向队列头节点写入：头节点写入时，将Head的前一个节点作为数据写入对象，如果Head指向队列起始位置则采用回卷方式。
读队列	根据readWriteableCnt[0]判断队列是否有消息需要读取，对全部空闲（readWriteableCnt[0]为0）队列进行读操作会引起任务挂起。 • 如果队列可以读取消息，则根据Head找到最先写入队列的消息节点进行读取。 • 如果Head已经指向队列尾部则采用回卷方式。
删除队列	根据队列ID找到对应队列，把队列状态置为未使用，把队列控制块置为初始状态。如果是通过系统动态申请内存方式创建的队列，还会释放队列所占内存。

图 3-8 队列读写数据操作示意图



上图对读写队列做了示意，图中只画了尾节点写入方式，没有画头节点写入，但是两者是类似的。

3.6.2 开发指导

使用场景

队列用于任务间通信，可以实现消息的异步处理。同时消息的发送方和接收方不需要彼此联系，两者间是解耦的。

功能

Huawei LiteOS中的队列模块提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
创建/删除消息队列	LOS_QueueCreate	创建一个消息队列，由系统动态申请队列空间
	LOS_QueueCreateStatic	创建一个消息队列，由用户分配队列内存空间传入接口
	LOS_QueueDelete	根据队列ID删除一个指定队列
读/写队列（不带拷贝）	LOS_QueueRead	读取指定队列头节点中的数据（队列节点中的数据实际上是一个地址）
	LOS_QueueWrite	向指定队列尾节点中写入入参bufferAddr的值（即buffer的地址）
	LOS_QueueWriteHead	向指定队列头节点中写入入参bufferAddr的值（即buffer的地址）

功能分类	接口名	描述
读/写队列（带拷贝）	LOS_QueueReadCopy	读取指定队列头节点中的数据
	LOS_QueueWriteCopy	向指定队列尾节点中写入入参 <code>bufferAddr</code> 中保存的数据
	LOS_QueueWriteHeadCopy	向指定队列头节点中写入入参 <code>bufferAddr</code> 中保存的数据
获取队列信息	LOS_QueueInfoGet	获取指定队列的信息，包括队列ID、队列长度、消息节点大小、头节点、尾节点、可读节点数量、可写节点数量、等待读操作的任务、等待写操作的任务、等待mail操作的任务

队列错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_QUEUE_NO_MEMORY	0x02000601	队列初始化时，从动态内存池申请内存失败	设置更大的系统动态内存池，配置项为OS_SYS_MEM_SIZE，或减少系统支持的最大队列数
2	LOS_ERRNO_QUEUE_CREATE_NO_MEMORY	0x02000602	创建队列时，从动态内存池申请内存失败	设置更大的系统动态内存池，配置项为OS_SYS_MEM_SIZE，或减少要创建队列的队列长度和消息节点大小
3	LOS_ERRNO_QUEUE_SIZE_TOO_BIG	0x02000603	创建队列时消息节点大小超过上限	更改入参消息节点大小，使之不超过上限
4	LOS_ERRNO_QUEUE_CB_UNAVAILABLE	0x02000604	创建队列时，系统中已经没有空闲队列	增加系统支持的最大队列数
5	LOS_ERRNO_QUEUE_NOT_FOUND	0x02000605	传递给删除队列接口的队列ID大于等于系统支持的最大队列数	确保队列ID是有效的
6	LOS_ERRNO_QUEUE_PENDING_IN_LOCK	0x02000606	当任务被锁定时，禁止在队列中阻塞等待写消息或读消息	使用队列前解锁任务

序号	定义	实际数值	描述	参考解决方案
7	LOS_ERRNO_QUEUE_TIMEOUT	0x02000607	等待处理队列超时	检查设置的超时时间是否合适
8	LOS_ERRNO_QUEUE_IN_TSKUSE	0x02000608	队列存在阻塞任务而不能被删除	使任务能够获得资源而不是在队列中被阻塞
9	LOS_ERRNO_QUEUE_WRITE_INTERRUPT	0x02000609	在中断处理程序中不能以阻塞模式写队列	将写队列设为非阻塞模式，即将写队列的超时时间设置为0
10	LOS_ERRNO_QUEUE_NOT_CREATE	0x0200060a	队列未创建	创建该队列，或更换为一个已经创建的队列
11	LOS_ERRNO_QUEUE_IN_TSKWRITE	0x0200060b	队列读写不同步	同步队列的读写，即多个任务不能并发读写同一个队列
12	LOS_ERRNO_QUEUE_CREAT_PTR_NULL	0x0200060c	对于创建队列接口，保存队列ID的入参为空指针	确保传入的参数不为空指针
13	LOS_ERRNO_QUEUE_PARAM_ISZERO	0x0200060d	对于创建队列接口，入参队列长度或消息节点大小为0	传入正确的队列长度和消息节点大小
14	LOS_ERRNO_QUEUE_INVALID_ID	0x0200060e	传递给读队列或写队列或获取队列信息接口的队列ID大于等于系统支持的最大队列数	确保队列ID有效
15	LOS_ERRNO_QUEUE_READ_PTR_NULL	0x0200060f	传递给读队列接口的指针为空	确保传入的参数不为空指针
16	LOS_ERRNO_QUEUE_READ_SIZE_IS_INVALID_ID	0x02000610	传递给读队列接口的缓冲区大小为0或者大于0xFFFFB	传入的一个正确的缓冲区大小需要大于0且小于0xFFFFC
17	LOS_ERRNO_QUEUE_WRITE_PTR_NULL	0x02000612	传递给写队列接口的缓冲区指针为空	确保传入的参数不为空指针
18	LOS_ERRNO_QUEUE_WRITE_SIZE_ISZERO	0x02000613	传递给写队列接口的缓冲区大小为0	传入正确的缓冲区大小

序号	定义	实际数值	描述	参考解决方案
19	LOS_ERRNO_QUEUE_WRITE_SIZE_TOO_BIG	0x02000615	传递给写队列接口的缓冲区大小比队列的消息节点大小要大	减小缓冲区大小，或增大队列的消息节点大小
20	LOS_ERRNO_QUEUE_ISFULL	0x02000616	写队列时没有可用的空闲节点	写队列之前，确保在队列中存在可用的空闲节点，或者使用阻塞模式写队列，即设置大于0的写队列超时时间
21	LOS_ERRNO_QUEUE_PTR_NULL	0x02000617	传递给获取队列信息接口的指针为空	确保传入的参数不为空指针
22	LOS_ERRNO_QUEUE_READ_IN_INTERRUPT	0x02000618	在中断处理程序中不能以阻塞模式读队列	将读队列设为非阻塞模式，即将读队列的超时时间设置为0
23	LOS_ERRNO_QUEUE_MAIL_HANDLE_INVALID	0x02000619	CMSIS-RTOS 1.0 中的mail队列，释放内存块时，发现传入的mail队列ID无效	确保传入的mail队列ID是正确的
24	LOS_ERRNO_QUEUE_MAIL_PTR_INVALID	0x0200061a	CMSIS-RTOS 1.0 中的mail队列，释放内存块时，发现传入的mail内存池指针为空	传入非空的mail内存池指针
25	LOS_ERRNO_QUEUE_MAIL_FREE_ERROR	0x0200061b	CMSIS-RTOS 1.0 中的mail队列，释放内存块失败	传入非空的mail队列内存块指针
26	LOS_ERRNO_QUEUE_ISEMPTY	0x0200061d	队列已空	读队列之前，确保队列中存在未读的消息，或者使用阻塞模式读队列，即设置大于0的读队列超时时间
27	LOS_ERRNO_QUEUE_READ_SIZE_TOO_SMALL	0x0200061f	传递给读队列接口的读缓冲区大小小于队列消息节点大小	增加缓冲区大小，或减小队列消息节点大小

须知

- 错误码定义见[错误码简介](#)。8~15位的所属模块为队列模块，值为0x06。
- 队列模块中的错误码序号0x11、0x14未被定义，不可用。

开发流程

使用队列模块的典型流程如下：

- 1. 打开菜单，进入Kernel ---> Enable Queue菜单，完成队列模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_IPC_QUEUE	队列模块裁剪开关	YES/NO	YES	无
LOSCFG_QUEUE_STATIC_ALLOCATION	支持以用户分配内存的方式创建队列	YES/NO	NO	LOSCFG_BASE_IPC_QUEUE
LOSCFG_BASE_IPC_QUEUE_LIMIT	系统支持的最大队列数	<65535 >	1024	LOSCFG_BASE_IPC_QUEUE

- 2. 创建队列。创建成功后，可以得到队列ID。
- 3. 写队列。
- 4. 读队列。
- 5. 获取队列信息。
- 6. 删除队列。

平台差异性

不同平台上，对数据对齐的要求可能有所差异，例如在3559a平台上，写入队列的数据需要进行八字节对齐。

3.6.3 注意事项

队列创建：

- 系统支持的最大队列数是指：整个系统的队列资源总个数，而非用户能使用的个数。例如：系统软件定时器多占用一个队列资源，那么用户能使用的队列资源就会减少一个。
- 创建队列时传入的队列名和flags暂时未使用，作为以后的预留参数。

队列读写：

- LOS_QueueReadCopy和LOS_QueueWriteCopy及LOS_QueueWriteHeadCopy是一组接口，LOS_QueueRead和LOS_QueueWrite及LOS_QueueWriteHead是一组接口，两组接口需要配套使用。
- 鉴于LOS_QueueWrite和LOS_QueueWriteHead和LOS_QueueRead这组接口实际操作的是数据地址，用户必须保证调用LOS_QueueRead获取到的指针所指向的内存区域在读队列期间没有被异常修改或释放，否则可能导致不可预知的后果。
- 鉴于LOS_QueueWrite和LOS_QueueWriteHead和LOS_QueueRead这组接口实际操作的是数据地址，也就意味着实际写和读的消息长度仅仅是一个指针数据，因此用户使用这组接口之前，需确保创建队列时的消息节点大小为一个指针的长度，避免不必要的浪费和读取失败。

其他：

- 队列接口函数中的入参timeout是相对时间。
- 当队列使用结束后，如果存在动态申请的内存，需要及时释放这些内存。

3.6.4 编程实例

实例描述

创建一个队列，两个任务。任务1调用写队列接口发送消息，任务2通过读队列接口接收消息。

1. 通过LOS_TaskCreate创建任务1和任务2。
2. 通过LOS_QueueCreate创建一个消息队列。
3. 在任务1 send_Entry中发送消息。
4. 在任务2 recv_Entry中接收消息。
5. 通过LOS_QueueDelete删除队列。

编程示例

前提条件：在menuconfig菜单中完成队列模块的配置。

```
#include "los_task.h"
#include "los_queue.h"

static UINT32 g_queue;
#define BUFFER_LEN 50

VOID send_Entry(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    CHAR abuf[] = "test is message x";
    UINT32 len = sizeof(abuf);

    while (i < 5) {
        abuf[len - 2] = '0' + i;
        i++;

        ret = LOS_QueueWriteCopy(g_queue, abuf, len, 0);
        if (ret != LOS_OK) {
            dprintf("send message failure, error: %x\n", ret);
        }

        LOS_TaskDelay(5);
    }
}

VOID recv_Entry(VOID)
{
    UINT32 ret = 0;
    CHAR readBuf[BUFFER_LEN] = {0};
    UINT32 readLen = BUFFER_LEN;

    while (1) {
        ret = LOS_QueueReadCopy(g_queue, readBuf, &readLen, 0);
        if (ret != LOS_OK) {
            dprintf("recv message failure, error: %x\n", ret);
            break;
        }

        dprintf("recv message: %s\n", readBuf);
    }
}
```

```

        LOS_TaskDelay(5);
    }

    while (LOS_OK != LOS_QueueDelete(g_queue)) {
        LOS_TaskDelay(1);
    }

    dprintf("delete the queue success!\n");
}

UINT32 Example_CreateTask(VOID)
{
    UINT32 ret = 0;
    UINT32 task1, task2;
    TSK_INIT_PARAM_S initParam;

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)send_Entry;
    initParam.usTaskPrio = 9;
    initParam.uwStackSize = LOS_TASK_MIN_STACK_SIZE;
    initParam.pcName = "sendQueue";
#ifdef LOSCFG_KERNEL_SMP
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    LOS_TaskLock();
    ret = LOS_TaskCreate(&task1, &initParam);
    if(ret != LOS_OK) {
        dprintf("create task1 failed, error: %x\n", ret);
        return ret;
    }

    initParam.pcName = "recvQueue";
    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)recv_Entry;
    ret = LOS_TaskCreate(&task2, &initParam);
    if(ret != LOS_OK) {
        dprintf("create task2 failed, error: %x\n", ret);
        return ret;
    }

    ret = LOS_QueueCreate("queue", 5, &g_queue, 0, BUFFER_LEN);
    if(ret != LOS_OK) {
        dprintf("create queue failure, error: %x\n", ret);
    }

    dprintf("create the queue success!\n");
    LOS_TaskUnlock();
    return ret;
}

```

结果验证

```

create the queue success!
recv message: test is message 0
recv message: test is message 1
recv message: test is message 2
recv message: test is message 3
recv message: test is message 4
recv message failure, error: 200061d
delete the queue success!

```

完整实例代码

sample_queue.c

3.7 事件

3.7.1 概述

基本概念

事件（Event）是一种任务间通信的机制，可用于任务间的同步。

多任务环境下，任务之间往往需要同步操作，一个等待即是一个同步。事件可以提供一对多、多对多的同步操作。

- 一对多同步模型：一个任务等待多个事件的触发。可以是任意一个事件发生时唤醒任务处理事件，也可以是几个事件都发生后才唤醒任务处理事件。
- 多对多同步模型：多个任务等待多个事件的触发。

Huawei LiteOS提供的事件具有如下特点：

- 任务通过创建事件控制块来触发事件或等待事件。
- 事件间相互独立，内部通过一个32位无符号整型来实现，每一位标识一种事件类型。第25位不可用，因此最多可支持31种事件类型。
- 事件仅用于任务间的同步，不支持数据传输。
- 若多次向事件控制块写入同一事件类型，在被清零前等效于只写入一次。
- 多个任务可以对同一事件进行读写操作。
- 支持事件读写超时机制。

事件控制块

```
/**
 * Event control structure
 */
typedef struct tagEvent {
    UINT32 uwEventID; /* 事件ID，每一位标识一种事件类型 */
    LOS_DL_LIST stEventList; /* 读取事件的任务链表 */
} EVENT_CB_S, *PEVENT_CB_S;
```

uwEventID：用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0表示该事件类型未发生、1表示该事件类型已经发生），一共31种事件类型。（第25位系统保留。）

事件读取模式

在读事件时，可以选择读取模式。读取模式如下：

- 所有事件（LOS_WAITMODE_AND）：逻辑与，基于接口传入的事件类型掩码eventMask，只有这些事件都已经发生才能读取成功，否则该任务将阻塞等待或者返回错误码。
- 任一事件（LOS_WAITMODE_OR）：逻辑或，基于接口传入的事件类型掩码eventMask，只要这些事件中有任一种事件发生就可以读取成功，否则该任务将阻塞等待或者返回错误码。
- 清除事件（LOS_WAITMODE_CLR）：这是一种附加读取模式，需要与所有事件模式或任一事件模式结合使用（LOS_WAITMODE_AND | LOS_WAITMODE_CLR 或 LOS_WAITMODE_OR | LOS_WAITMODE_CLR）。在这种模式下，当设置的所

有事件模式或任一事件模式读取成功后，会自动清除事件控制块中对应的事件类型位。

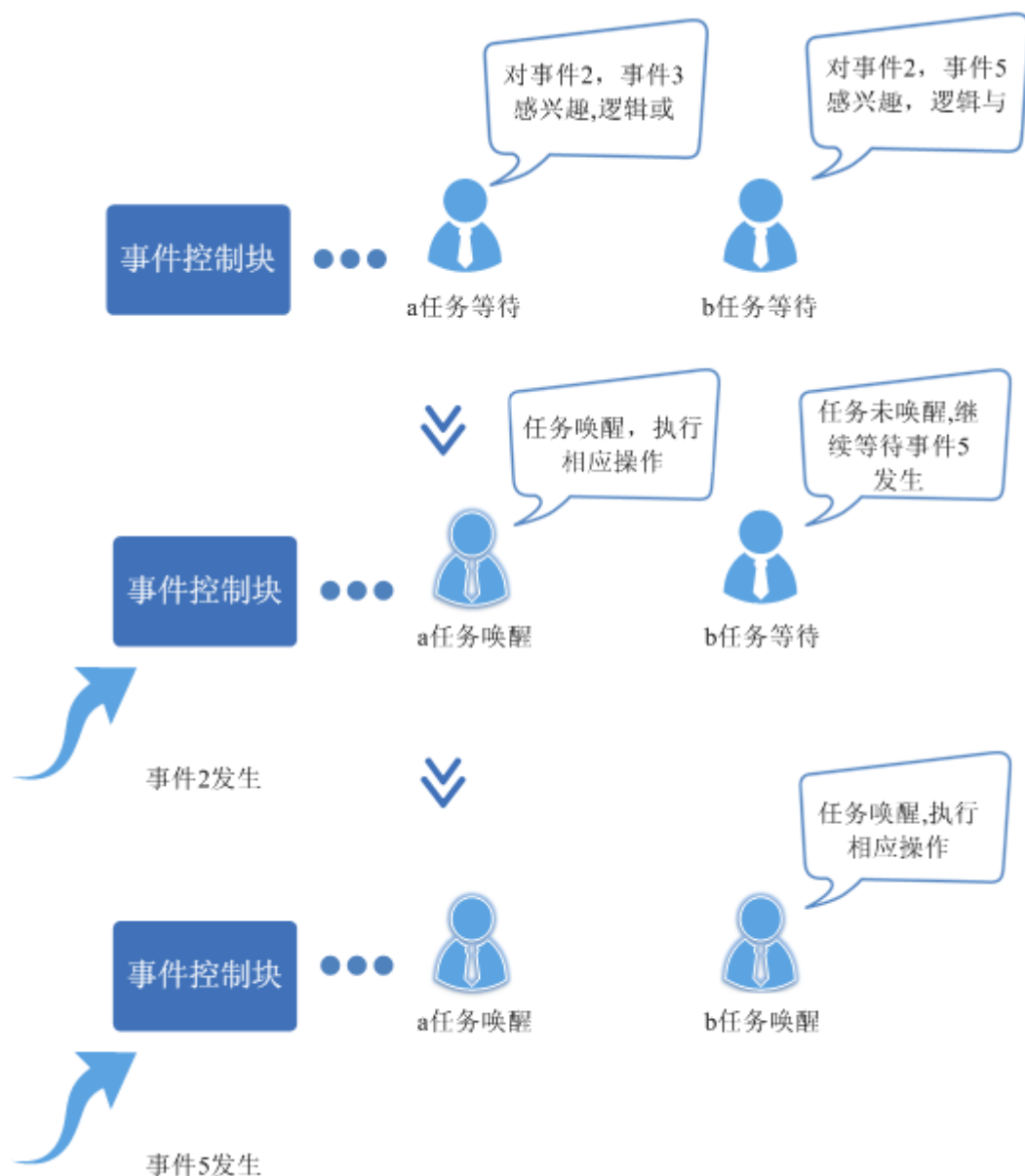
运作机制

任务在调用LOS_EventRead接口读事件时，可以根据入参事件掩码类型eventMask读取事件的单个或者多个事件类型。事件读取成功后，如果设置LOS_WAITMODE_CLR会清除已读取到的事件类型，反之不会清除已读到的事件类型，需显式清除。可以通过入参选择读取模式，读取事件掩码类型中所有事件还是读取事件掩码类型中任意事件。

任务在调用LOS_EventWrite接口写事件时，对指定事件控制块写入指定的事件类型，可以一次同时写多个事件类型。写事件会触发任务调度。

任务在调用LOS_EventClear接口清除事件时，根据入参事件和待清除的事件类型，对事件对应位进行清零操作。

图 3-9 事件唤醒任务示意图



3.7.2 开发指导

使用场景

事件可应用于多种任务同步场景，在某些同步场景下可替代信号量。

功能

Huawei LiteOS的事件模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
初始化事件	LOS_EventInit	初始化一个事件控制块。
读/写事件	LOS_EventRead	读取指定事件类型，超时时间为相对时间：单位为Tick。
	LOS_EventWrite	写指定的事件类型。
	LOS_EventCondRead	读取满足指定条件的事件，超时时间为相对时间：单位为Tick，与LOS_EventCondWrite成对使用。
	LOS_EventCondWrite	写指定的事件，与LOS_EventCondRead成对使用。
清除事件	LOS_EventClear	清除指定的事件类型。
校验事件掩码	LOS_EventPoll	根据用户传入的事件ID、事件掩码及读取模式，返回用户传入的事件是否符合预期。
销毁事件	LOS_EventDestroy	销毁指定的事件控制块。

Event 错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际值	描述	参考解决方案
1	LOS_ERRNO_EVENT_SETBIT_INVALID	0x02001c00	写事件时，将事件ID的第25个bit设置为1。这个比特位OS内部保留，不允许设置为1。	事件ID的第25bit置为0。
2	LOS_ERRNO_EVENT_READ_TIMEOUT	0x02001c01	读事件超时。	增加等待时间或者重新读取。
3	LOS_ERRNO_EVENT_EVENTMASK_INVALID	0x02001c02	入参的事件ID是无效的。	传入有效的事件ID参数。
4	LOS_ERRNO_EVENT_READ_INTERRUPT	0x02001c03	在中断中读取事件。	启动新的任务来获取事件。
5	LOS_ERRNO_EVENT_FLAGS_INVALID	0x02001c04	读取事件的mode无效。	传入有效的mode参数。

序号	定义	实际值	描述	参考解决方案
6	LOS_ERRNO_EVENT_READ_IN_LOCK	0x02001c05	任务锁住，不能读取事件。	解锁任务，再读取事件。
7	LOS_ERRNO_EVENT_PTR_NULL	0x02001c06	传入的参数为空指针。	传入非空入参。
8	LOS_ERRNO_EVENT_SHOULD_NOT_DESTROY	0x02001c08	事件链表上仍有任务，无法被销毁。	先检查事件链表是否为空。

须知

错误码定义参见[错误码简介](#)。8~15位的所属模块为事件模块，值为0x1c。

开发流程

使用事件模块的典型流程如下：

1. 打开菜单，进入Kernel ---> Enable Event菜单，完成事件模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_IPC_EVENT	事件功能的裁剪开关	YES/NO	YES	无

2. 调用事件初始化LOS_EventInit接口，初始化事件等待队列。
3. 写事件LOS_EventWrite或LOS_EventCondWrite，写入指定的事件类型或事件。
4. 读事件LOS_EventRead或LOS_EventCondRead，读取指定的事件类型或条件事件。
5. 清除事件LOS_EventClear，清除指定的事件类型，该接口只适用于事件类型的事件。
6. 销毁事件LOS_EventDestroy。

平台差异性

无。

3.7.3 注意事项

- 在系统初始化之前不能调用读写事件接口。如果调用，系统会运行异常。
- 在中断中，可以对事件对象进行写操作，但不能进行读操作。
- 在锁定任务调度状态下，禁止任务阻塞于读事件。

- 为了区别LOS_EventRead接口返回的是事件还是错误码，事件掩码的第25位不能使用。
- LOS_EventClear入参值是：要清除的指定事件类型的反码（~events）。
- LOS_EventWrite和LOS_EventRead成对使用，LOS_EventCondWrite和LOS_EventCondRead成对使用，这两组接口的区别在于，前面的用于读取或写入指定事件类型的事件；而后面的一组接口，用于读取或写入指定事件条件的事件，事件条件由用户实现，写事件任务不用关心事件条件，而是广播式告知读事件任务，即唤醒读任务，读任务判断读取的事件条件是否成功，这样可能会带来一定的性能开销，但是可简化写事件任务。

3.7.4 编程实例

3.7.4.1 基于事件类型的读写

实例描述

示例中，任务Example_TaskEntry创建一个任务Example_Event，Example_Event读事件阻塞，Example_TaskEntry向该任务写事件。可以通过示例日志中打印的先后顺序理解事件操作时伴随的任务切换。

1. 在任务Example_TaskEntry创建任务Example_Event，其中任务Example_Event优先级高于Example_TaskEntry。
2. 在任务Example_Event中读事件0x00000001，阻塞，发生任务切换，执行任务Example_TaskEntry。
3. 在任务Example_TaskEntry向任务Example_Event写事件0x00000001，发生任务切换，执行任务Example_Event。
4. Example_Event得以执行，直到任务结束。
5. Example_TaskEntry得以执行，直到任务结束。

编程示例

前提条件：在menuconfig菜单中完成事件模块的配置。

代码实现如下：

```
#include "los_event.h"
#include "los_task.h"
#include "securec.h"

/* 任务ID */
UINT32 g_testTaskId;

/* 事件控制结构体 */
EVENT_CB_S g_exampleEvent;

/* 等待的事件类型 */
#define EVENT_WAIT 0x00000001

/* 用例任务入口函数 */
VOID Example_Event(VOID)
{
    UINT32 ret;
    UINT32 event;

    /* 超时等待方式读事件,超时时间为100 ticks, 若100 ticks后未读取到指定事件，读事件超时，任务直接唤醒 */
    printf("Example_Event wait event 0x%x\n", EVENT_WAIT);
```

```
event = LOS_EventRead(&g_exampleEvent, EVENT_WAIT, LOS_WAITMODE_AND, 100);
if (event == EVENT_WAIT) {
    printf("Example_Event,read event :0x%x\n", event);
} else {
    printf("Example_Event,read event timeout\n");
}
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;

    /* 事件初始化 */
    ret = LOS_EventInit(&g_exampleEvent);
    if (ret != LOS_OK) {
        printf("init event failed .\n");
        return -1;
    }

    /* 创建任务 */
    (VOID)memset_s(&task1, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Event;
    task1.pcName = "EventTsk1";
    task1.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = 5;
    ret = LOS_TaskCreate(&g_testTaskId, &task1);
    if (ret != LOS_OK) {
        printf("task create failed .\n");
        return LOS_NOK;
    }

    /* 写g_testTaskId 等待事件 */
    printf("Example_TaskEntry write event .\n");

    ret = LOS_EventWrite(&g_exampleEvent, EVENT_WAIT);
    if (ret != LOS_OK) {
        printf("event write failed .\n");
        return LOS_NOK;
    }

    /* 清标志位 */
    printf("EventMask:%d\n", g_exampleEvent.uwEventID);
    LOS_EventClear(&g_exampleEvent, ~g_exampleEvent.uwEventID);
    printf("EventMask:%d\n", g_exampleEvent.uwEventID);

    /* 删除任务 */
    ret = LOS_TaskDelete(g_testTaskId);
    if (ret != LOS_OK) {
        printf("task delete failed .\n");
        return LOS_NOK;
    }

    return LOS_OK;
}
```

结果验证

编译运行结果如下：

```
Example_Event wait event 0x1
Example_TaskEntry write event .
Example_Event,read event :0x1
EventMask:1
EventMask:0
```

完整实例代码

sample_event.c

3.7.4.2 基于事件条件的读写

实例描述

示例中，任务ExampleTaskEntry中创建两个读事件任务readTask1和readTask2和一个写事件任务writeTask，由writeTask依次将readTask1和readTask2等待的条件置为真，并写事件，readTask1和readTask2读取的事件符合预期，结束阻塞。可以通过示例日志中打印的先后顺序理解事件操作时伴随的任务切换。

1. 在任务ExampleTaskEntry中创建两个读事件任务readTask1和readTask2；
2. 任务readTask1等待的条件EventCondition1当前为假，当前未读取到目的事件，且未超时，任务阻塞；
3. 任务readTask2等待的条件EventCondition2当前为假，当前未读取到目的事件，且未超时，任务阻塞；
4. 在任务ExampleTaskEntry中创建写事件任务writeTask；
5. writeTask将readTask1等待的条件置为真，写事件；
6. readTask1和readTask2都得到唤醒，但因只有readTask1的条件为真，所以readTask1读事件成功，向下执行，readTask2读事件失败，继续阻塞；
7. writeTask将readTask2等待的条件置为真，写事件；
8. readTask2得到唤醒，条件为真，所以readTask2读事件成功，向下执行。

编程示例

前提条件：在menuconfig菜单中完成事件模块的配置。

代码实现如下：

```
#include "los_event.h"
#include "los_task.h"
#include "securec.h"

/* 任务ID */
static UINT32 g_readTskId1;
static UINT32 g_readTskId2;
static UINT32 g_writeTskId;

/* g_readTskId1和g_readTskId2等待的条件 */
static int g_testData1 = 0;
static int g_testData2 = 0;

/* 事件控制结构体 */
static EVENT_CB_S g_event;

/* 写事件任务入口函数 */
static VOID WriteTaskFunc(VOID)
{
    UINT32 ret;

    g_testData1 = 1;
    printf("\nWriteTaskFunc: write for ReadTaskFunc1\n\n");
    ret = LOS_EventCondWrite(&g_event);
    if (ret != LOS_OK) {
        printf("WriteTaskFunc failed: ret = %u\n", ret);
        return;
    }
}
```

```

    }
    LOS_TaskDelay(2); /* 2: delay 2 ticks */

    printf("\nWriteTaskFunc: write for ReadTaskFunc2\n\n");
    g_testData2 = 1;
    ret = LOS_EventCondWrite(&g_event);
    if (ret != LOS_OK) {
        printf("WriteTaskFunc failed: ret = %u\n", ret);
        return;
    }
    LOS_TaskDelay(2); /* 2: delay 2 ticks */

    return;
}

/* 读事件任务1预期想读取的事件条件 */
static BOOL EventCondition1(VOID)
{
    BOOL ret;

    ret = (g_testData1 == 1) ? TRUE : FALSE;
    printf("EventCondition1: ret = %s\n", (ret ? "TRUE" : "FALSE"));
    return ret;
}

/* 读事件任务1入口函数 */
static VOID ReadTaskFunc1(VOID)
{
    UINT32 ret;

    ret = LOS_EventCondRead(&g_event, EventCondition1(), LOS_WAIT_FOREVER);
    if (ret != LOS_OK) {
        printf("ReadTaskFunc1 read failed!\n");
        return;
    }

    printf("ReadTaskFunc1 read succeeded!\n");
    return;
}

/* 读事件任务2预期想读取的事件条件 */
static BOOL EventCondition2(VOID)
{
    BOOL ret;

    ret = (g_testData2 == 1) ? TRUE : FALSE;
    printf("EventCondition2: ret = %s\n", (ret ? "TRUE" : "FALSE"));
    return ret;
}

/* 读事件任务2入口函数 */
static VOID ReadTaskFunc2(VOID)
{
    UINT32 ret;

    ret = LOS_EventCondRead(&g_event, EventCondition2(), LOS_WAIT_FOREVER);
    if (ret != LOS_OK) {
        printf("ReadTaskFunc2 read failed!\n");
        return;
    }

    printf("ReadTaskFunc2 read succeeded!\n");
    return;
}

static VOID InitTaskParam(TSK_INIT_PARAM_S *task, char *taskName, TSK_ENTRY_FUNC entry, UINT16
prio, UINT16 affi)
{
    task->pcName = taskName;

```



```
task->pfnTaskEntry = entry;
task->usTaskPrio = prio;
task->uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
task->uwResved = LOS_TASK_STATUS_DETACHED;

#ifdef LOSCFG_KERNEL_SMP
    task->usCpuAffiMask = affi;
#endif
}

static UINT32 ExampleTaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S readTask1 = {0};
    TSK_INIT_PARAM_S readTask2 = {0};
    TSK_INIT_PARAM_S writeTask = {0};

    ret = LOS_EventInit(&g_event);
    if (ret != LOS_OK) {
        return ret;
    }

    InitTaskParam(&readTask1, "EventReadTsk1", (TSK_ENTRY_FUNC)ReadTaskFunc1, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuid()));
    ret = LOS_TaskCreate(&g_readTskId1, &readTask1);
    if (ret != LOS_OK) {
        return ret;
    }

    InitTaskParam(&readTask2, "EventReadTsk2", (TSK_ENTRY_FUNC)ReadTaskFunc2, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuid()));
    ret = LOS_TaskCreate(&g_readTskId2, &readTask2);
    if (ret != LOS_OK) {
        return ret;
    }

    InitTaskParam(&writeTask, "EventWriteTsk45", (TSK_ENTRY_FUNC)WriteTaskFunc, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuid()));
    ret = LOS_TaskCreate(&g_writeTskId, &writeTask);
    if (ret != LOS_OK) {
        return ret;
    }

    (VOID)LOS_TaskDelay(6); /* 6: delay 6 ticks */

    (VOID)LOS_EventDestroy(&g_event);
    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
EventCondition1: ret = FALSE
EventCondition2: ret = FALSE

WriteTaskFunc: write for ReadTaskFunc1

EventCondition1: ret = TRUE
ReadTaskFunc1 read succeeded!
EventCondition2: ret = FALSE

WriteTaskFunc: write for ReadTaskFunc2

EventCondition2: ret = TRUE
EventCondition2: ret = TRUE
ReadTaskFunc2 read succeeded!
```

完整实例代码

sample_event_cond.rar

3.8 信号量

3.8.1 概述

基本概念

信号量（Semaphore）是一种实现任务间通信的机制，可以实现任务间同步或共享资源的互斥访问。

一个信号量的数据结构中，通常有一个计数值，用于对有效资源数的计数，表示剩下的可被使用的共享资源数，其值的含义分两种情况：

- 0，表示该信号量当前不可获取，因此可能存在正在等待该信号量的任务。
- 正值，表示该信号量当前可被获取。

以同步为目的的信号量和以互斥为目的的信号量在使用上有如下不同：

- 用作互斥时，初始信号量计数值不为0，表示可用的共享资源个数。在需要使用共享资源前，先获取信号量，然后使用一个共享资源，使用完毕后释放信号量。这样在共享资源被取完，即信号量计数减至0时，其他需要获取信号量的任务将被阻塞，从而保证了共享资源的互斥访问。另外，当共享资源数为1时，建议使用二值信号量。二值信号量的计数值最大为1，是一种类似于互斥锁的机制。
- 用作同步时，初始信号量计数值为0。任务1获取信号量而阻塞，直到任务2或者某中断释放信号量，任务1才得以进入Ready或Running态，从而达到了任务间的同步。

运作机制

信号量控制块

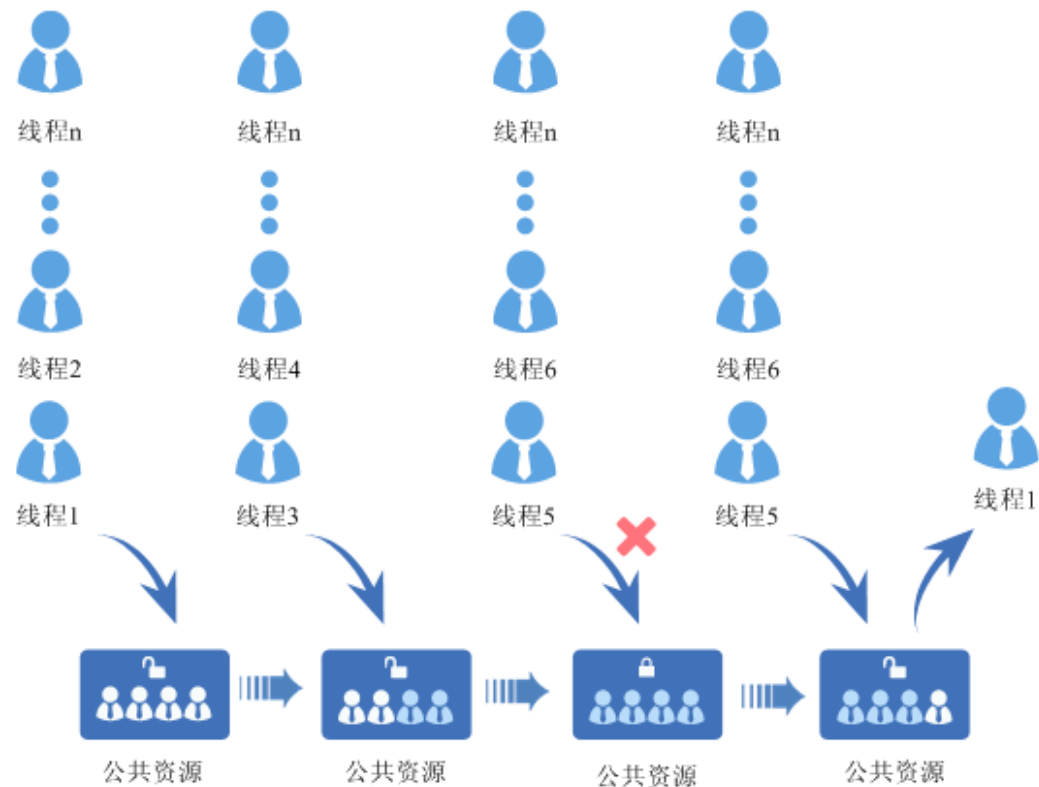
```
/**
 * Semaphore control structure.
 */
typedef struct {
    UINT8      semStat;      /* 是否使用标志位 */
    UINT8      semType;      /* 信号量类型 */
    UINT16     semCount;      /* 信号量计数 */
    UINT32     semId;        /* 信号量索引号 */
    LOS_DL_LIST semList;     /* 挂接阻塞于该信号量的任务 */
} LosSemCB;
```

信号量运作原理

- 信号量初始化：为配置的N个信号量申请内存（N值可以由用户自行配置，通过LOSCFG_BASE_IPC_SEM_LIMIT宏实现），并把所有信号量初始化成未使用，加入到未使用链表中供系统使用。
- 信号量创建：从未使用的信号量链表中获取一个信号量，并设定初值。
- 信号量申请：若其计数器值大于0，则直接减1返回成功。否则任务阻塞，等待其它任务释放该信号量，等待的超时时间可设定。当任务被一个信号量阻塞时，将该任务挂到信号量等待任务队列的队尾。

- 信号量释放：若没有任务等待该信号量，则直接将计数器加1返回。否则唤醒该信号量等待任务队列上的第一个任务。
- 信号量删除：将正在使用的信号量置为未使用信号量，并挂回到未使用链表。
- 信号量允许多个任务在同一时刻访问共享资源，但会限制同一时刻访问此资源的最大任务数目。当访问资源的任务数达到该资源允许的最大数量时，会阻塞其他试图获取该资源的任务，直到有任务释放该信号量。

图 3-10 信号量运作示意图



3.8.2 开发指导

使用场景

在多任务系统中，信号量是一种非常灵活的同步方式，可以运用在多种场合中，实现锁、同步、资源计数等功能，也能方便的用于任务与任务，中断与任务的同步中。信号量常用于协助一组相互竞争的任务访问共享资源。

功能

Huawei LiteOS 的信号量模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
创建/删除信号量	LOS_SemCreate	创建信号量，返回信号量ID

功能分类	接口名	描述
	LOS_BinarySemCreate	创建二值信号量，其计数值最大为1
	LOS_SemDelete	删除指定的信号量
申请/释放信号量	LOS_SemPend	申请指定的信号量，并设置超时时间
	LOS_SemPost	释放指定的信号量

说明

信号量有三种申请模式：无阻塞模式、永久阻塞模式、定时阻塞模式。

- 无阻塞模式：即任务申请信号量时，入参timeout等于0。若当前信号量计数值不为0，则申请成功，否则立即返回申请失败。
- 永久阻塞模式：即任务申请信号量时，入参timeout等于0xFFFFFFFF。若当前信号量计数值不为0，则申请成功。否则该任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，直到有其他任务释放该信号量，阻塞任务才会重新得以执行。
- 定时阻塞模式：即任务申请信号量时，0<timeout<0xFFFFFFFF。若当前信号量计数值不为0，则申请成功。否则，该任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，超时前如果有其他任务释放该信号量，则该任务可成功获取信号量继续执行，若超时前未获取到信号量，接口将返回超时错误码。

信号量错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_SEM_NO_MEMORY	0x02000700	初始化信号量时，内存空间不足	调整OS_SYS_MEM_SIZE以确保有足够的内存供信号量使用，或减小系统支持的最大信号量数 LOSCFG_BASE_IPC_SEM_LIMIT
2	LOS_ERRNO_SEM_INVALID	0x02000701	信号量ID不正确或信号量未创建	传入正确的信号量ID或创建信号量后再使用
3	LOS_ERRNO_SEM_PTR_NULL	0x02000702	传入空指针	传入合法指针
4	LOS_ERRNO_SEM_ALL_BUSY	0x02000703	创建信号量时，系统中已经没有未使用的信号量	及时删除无用的信号量或增加系统支持的最大信号量数 LOSCFG_BASE_IPC_SEM_LIMIT

序号	定义	实际数值	描述	参考解决方案
5	LOS_ERRNO_SEM_UNAVAILABLE	0x02000704	无阻塞模式下未获取到信号量	选择阻塞等待或根据该错误码适当处理
6	LOS_ERRNO_SEM_PEND_INTERRUPT	0x02000705	中断期间非法调用LOS_SemPend申请信号量	中断期间禁止调用LOS_SemPend
7	LOS_ERRNO_SEM_PEND_IN_LOCK	0x02000706	任务被锁，无法获得信号量	在任务被锁时，不能调用LOS_SemPend申请信号量
8	LOS_ERRNO_SEM_TIMEOUT	0x02000707	获取信号量超时	将时间设置在合理范围内
9	LOS_ERRNO_SEM_OVERFLOW	0x02000708	信号量计数值已达到最大值，无法再继续释放该信号量	根据该错误码适当处理
10	LOS_ERRNO_SEM_PENDED	0x02000709	等待信号量的任务队列不为空	唤醒所有等待该信号量的任务后，再删除该信号量

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为信号量模块，值为0x07。

开发流程

信号量的开发典型流程：

1. 打开菜单，进入Kernel ---> Enable Sem菜单，完成信号量的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_IPC_SEM	信号量模块裁剪开关	YES/NO	YES	无
LOSCFG_BASE_IPC_SEM_LIMIT	系统支持的信号量最大数	[0, 65535]	1024	无

2. 创建信号量LOS_SemCreate，若要创建二值信号量则调用LOS_BinarySemCreate。
3. 申请信号量LOS_SemPend。
4. 释放信号量LOS_SemPost。
5. 删除信号量LOS_SemDelete。

平台差异性

无。

3.8.3 注意事项

由于中断不能被阻塞，因此不能在中断中使用阻塞模式申请信号量。

3.8.4 编程实例

实例描述

本实例实现如下功能：

1. 测试任务Example_TaskEntry创建一个信号量，锁任务调度，创建两个任务Example_SemTask1、Example_SemTask2，Example_SemTask2优先级高于Example_SemTask1，两个任务中申请同一信号量，解锁任务调度后两任务阻塞，测试任务Example_TaskEntry释放信号量。
2. Example_SemTask2得到信号量，被调度，然后任务休眠20Tick，Example_SemTask2延迟，Example_SemTask1被唤醒。
3. Example_SemTask1定时阻塞模式申请信号量，等待时间为10Tick，因信号量仍被Example_SemTask2持有，Example_SemTask1挂起，10Tick后仍未得到信号量，Example_SemTask1被唤醒，试图以永久阻塞模式申请信号量，Example_SemTask1挂起。
4. 20Tick后Example_SemTask2唤醒，释放信号量后，Example_SemTask1得到信号量被调度运行，最后释放信号量。
5. Example_SemTask1执行完，40Tick后任务Example_TaskEntry被唤醒，执行删除信号量，删除两个任务。

编程示例

前提条件：在menuconfig菜单中完成信号量的配置。

代码实现如下：

```
#include "los_sem.h"
#include "securec.h"

/* 任务ID */
static UINT32 g_testTaskId01;
static UINT32 g_testTaskId02;
/* 测试任务优先级 */
#define TASK_PRIO_TEST 5
/* 信号量结构体id */
static UINT32 g_semId;

VOID Example_SemTask1(VOID)
{
    UINT32 ret;

    printf("Example_SemTask1 try get sem g_semId,timeout 10 ticks.\n");
    /* 定时阻塞模式申请信号量，定时时间为10ticks */
    ret = LOS_SemPend(g_semId, 10);

    /* 申请到信号量 */
    if (ret == LOS_OK) {
        LOS_SemPost(g_semId);
        return;
    }
}
```

```

    }
    /* 定时时间到, 未申请到信号量 */
    if (ret == LOS_ERRNO_SEM_TIMEOUT) {
        printf("Example_SemTask1 timeout and try get sem g_semId wait forever.\n");
        /*永久阻塞模式申请信号量*/
        ret = LOS_SemPend(g_semId, LOS_WAIT_FOREVER);
        printf("Example_SemTask1 wait_forever and get sem g_semId .\n");
        if (ret == LOS_OK) {
            LOS_SemPost(g_semId);
            return;
        }
    }
}

VOID Example_SemTask2(VOID)
{
    UINT32 ret;
    printf("Example_SemTask2 try get sem g_semId wait forever.\n");
    /* 永久阻塞模式申请信号量 */
    ret = LOS_SemPend(g_semId, LOS_WAIT_FOREVER);

    if (ret == LOS_OK) {
        printf("Example_SemTask2 get sem g_semId and then delay 20ticks .\n");
    }

    /* 任务休眠20 ticks */
    LOS_TaskDelay(20);

    printf("Example_SemTask2 post sem g_semId .\n");
    /* 释放信号量 */
    LOS_SemPost(g_semId);
    return;
}

UINT32 ExampleTaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;
    TSK_INIT_PARAM_S task2;

    /* 创建信号量 */
    LOS_SemCreate(0,&g_semId);

    /* 锁任务调度 */
    LOS_TaskLock();

    /* 创建任务1 */
    (VOID)memset_s(&task1, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SemTask1;
    task1.pcName = "TestTsk1";
    task1.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = TASK_PRIO_TEST;
    ret = LOS_TaskCreate(&g_testTaskId01, &task1);
    if (ret != LOS_OK) {
        printf("task1 create failed .\n");
        return LOS_NOK;
    }

    /* 创建任务2 */
    (VOID)memset_s(&task2, sizeof(TSK_INIT_PARAM_S), 0, sizeof(TSK_INIT_PARAM_S));
    task2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SemTask2;
    task2.pcName = "TestTsk2";
    task2.uwStackSize = OS_TSK_DEFAULT_STACK_SIZE;
    task2.usTaskPrio = (TASK_PRIO_TEST - 1);
    ret = LOS_TaskCreate(&g_testTaskId02, &task2);
    if (ret != LOS_OK) {
        printf("task2 create failed .\n");
        return LOS_NOK;
    }
}

```

```
/* 解锁任务调度 */
LOS_TaskUnlock();

ret = LOS_SemPost(g_semId);

/* 任务休眠40 ticks */
LOS_TaskDelay(40);

/* 删除信号量 */
LOS_SemDelete(g_semId);

/* 删除任务1 */
ret = LOS_TaskDelete(g_testTaskId01);
if (ret != LOS_OK) {
    printf("task1 delete failed .\n");
    return LOS_NOK;
}
/* 删除任务2 */
ret = LOS_TaskDelete(g_testTaskId02);
if (ret != LOS_OK) {
    printf("task2 delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
Example_SemTask2 try get sem g_semId wait forever.
Example_SemTask1 try get sem g_semId ,timeout 10 ticks.
Example_SemTask2 get sem g_semId and then delay 20ticks .
Example_SemTask1 timeout and try get sem g_semId wait forever.
Example_SemTask2 post sem g_semId .
Example_SemTask1 wait_forever and get sem g_semId .
```

完整实例代码

sample_sem.c

3.9 读写信号量

3.9.1 概述

基本概念

读写信号量（rwsem）与普通信号量的功能相似，常用于共享资源的互斥访问和任务同步。读写信号量将持有者分为读者和写者，如果一个持有者需要对该信号量保护的共享资源进行写操作，那么将这类持有者定义为写者；如果持有者只是访问共享资源，而不修改，那么将这类持有者定义为读者。

读写信号量支持一个读写信号量同时拥有不受上限的读者数，即读者不排他，可并发；但是一个读写量同时只能有一个写者，写者是排他性的，独占性的，写写互斥、读写互斥。

运作机制

读写信号量控制块

```
typedef struct {  
    UINT8    rwsemState;    /* 是否使用标志位 */  
    INT16    rwsemCount;    /* 读写信号量拥有者的数量 */  
    UINT32    rwsemId;      /* 读写信号量索引号 */  
    LOS_DL_LIST waitList;   /* 等待队列，用于记录阻塞于该读写信号量的任务 */  
} OsRwsemCB;
```

读写信号量运作原理

- 读写信号量资源初始化：为配置的N个读写信号量申请内存（N值可以由用户自行配置，通过LOSCFG_BASE_IPC_RWSEM_LIMIT宏实现），并把所有读写信号量初始化成未使用，加入到未使用链表中供系统使用。
- 创建读写信号量：从未使用链表中获取一个读写信号量。
- 申请读信号量：若该信号量还未被持有，或者正在被其他读者持有，返回成功；否则任务阻塞，等待其它任务释放该信号量，等待的超时时间可设定。当任务被一个读写信号量阻塞时，该任务会被挂到读写信号量等待队列的队尾。
- 申请写信号量：若该信号量还未被持有，返回成功；否则任务阻塞，等待其它任务释放该信号量，等待的超时时间可设定。当任务被一个读写信号量阻塞时，该任务会被挂到读写信号量等待队列的队尾。
- 释放读信号量：若没有任务等待该读写信号量，则直接返回；当信号量处于读状态，等待队列中的队首任务只能是写者，唤醒这一个写者，然后返回。
- 释放写信号量：若没有任务等待该读写信号量，则直接返回；如果等待队列的队首任务是写者，则唤醒这一个写者，然后返回；如果等待队列的队首任务是读者，则唤醒这个读者及后面连续的多个读者，直到等待队列为空，或者遇到写者停止。
- 降级写信号量为读信号量：信号量变更为读信号量，若没有任务等待该读写信号量或者等待队列的队首任务是写者，则直接返回；否则唤醒等待队列的队首读者及后面连续的多个读者，直到等待队列为空，或者遇到写者停止。
- 删除读写信号量：将正在使用的读写信号量设置为未使用的状态，并挂回到未使用链表。

说明

读写信号量允许多个任务在同一时刻访问共享资源，但同一时刻只能有一个任务可修改共享资源，这样对于读任务多于写任务的场景，可有效提高系统的整体性能。

3.9.2 开发指导

使用场景

读写信号量是基于普通信号量优化的一种多任务同步互斥机制，可以运用在多种场合中，实现锁、同步、资源计数等功能，也能方便的用于任务与任务同步中，适用于读任务多于写任务的场景。

功能

Huawei LiteOS的读写信号量模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
创建/删除读写信号量	LOS_RwsemCreate	创建读写信号量，返回读写信号量ID
	LOS_RwsemDelete	删除指定的读写信号量
申请/释放读写信号量	LOS_RwsemPendRead	申请指定的读写信号量为读信号量，支持超时等待获取
	LOS_RwsemPostRead	释放指定的读信号量
	LOS_RwsemPendWrite	申请指定的读写信号量为写信号量，支持超时等待获取
	LOS_RwsemPostWrite	释放指定的写信号量
	LOS_RwsemDowngradeWrite	将指定的写信号量降级为读信号量

 说明

读写信号量支持三种申请模式：无阻塞模式、永久阻塞模式、定时阻塞模式。

- 无阻塞模式：即任务申请读写信号量时，入参timeout等于0。若当前读写信号量可获取，则申请成功，否则立即返回申请失败。
- 永久阻塞模式：即任务申请读写信号量时，入参timeout等于0xFFFFFFFF。若当前读写信号量可获取，则申请成功。否则该任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，直到其他任务释放该读写信号量，使得该读写信号量可重新被获取，阻塞任务才会重新得以执行。
- 定时阻塞模式：即任务申请读写信号量时，0<timeout<0xFFFFFFFF。若当前读写信号量可获取，则申请成功。否则，该任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，如果超时前其他任务释放该信号量，使得该读写信号量可重新被获取，则该任务可成功获取读写信号量继续执行，若超时前未获取到读写信号量，接口将返回超时错误码。

读写信号量错误码

对于存在失败可能性的操作，系统会返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_RWSEM_INVALID	0x02002200	读写信号量ID不正确或读写信号量未创建	传入正确的读写信号量ID或创建读写信号量后再使用
2	LOS_ERRNO_RWSEM_PTR_NULL	0x02002201	传入空指针	传入合法指针

序号	定义	实际数值	描述	参考解决方案
3	LOS_ERRNO_RWSEM_ALL_BUSY	0x02002202	创建读写信号量时，系统中已经没有未使用的读写信号量	及时删除无用的读写信号量或增加系统支持的最大读写信号量数 LOSCFG_BASE_IPC_RWSEM_LIMIT
4	LOS_ERRNO_RWSEM_PEND_INTERR	0x02002203	中断期间非法调用API，例如以阻塞的方式申请读写信号量	合理调用API，不要在中断期间以阻塞方式获取读写信号量
5	LOS_ERRNO_RWSEM_PEND_IN_LOCK	0x02002204	任务被锁，无法获得读写信号量	在任务被锁时，不要调用API获取读写信号量
6	LOS_ERRNO_RWSEM_TIMEOUT	0x02002205	获取读写信号量超时	将时间设置在合理范围内
7	LOS_ERRNO_RWSEM_PENDED	0x02002206	等待读写信号量的任务队列不为空	唤醒所有等待该读写信号量的任务后，再删除该读写信号量
8	LOS_ERRNO_RWSEM_INVALID_STATUS	0x02002207	释放或降级读写信号量时，信号量的状态不正确	请正确释放和降级读写信号量
9	LOS_ERRNO_RWSEM_UNAVAILABLE	0x02002208	无阻塞模式下未获取到读写信号量	选择阻塞等待或根据该错误码适当处理

须知

错误码定义见[开发指导](#)。8~15位的所属模块为信号量模块，值为0x22。

开发流程

读写信号量的典型开发流程

1. 打开菜单，进入Kernel ---> Enable Rwsem菜单，完成读写信号量的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_IPC_RWSEM	读写信号量模块裁剪开关	YES/NO	YES	无
LOSCFG_BASE_IPC_RWSEM_LIMIT	系统支持的读写信号量最大数	[0, 65535]	1024	LOSCFG_BASE_IPC_RWSEM

2. 创建读写信号量LOS_RwsemCreate。
3. 申请读信号量LOS_RwsemPendRead。
4. 释放读信号量LOS_RwsemPostRead。
5. 删除读写信号量LOS_RwsemDelete。

平台差异性

无。

3.9.3 注意事项

由于中断不能被阻塞，因此不能在中断中使用阻塞模式申请读写信号量。

3.9.4 编程实例

实例描述

本实例实现如下功能：

1. 测试任务ExampleTaskEntry创建一个读写信号量，并申请写信号量，锁任务调度，创建三个任务ExampleReadTask1、ExampleReadTask2和ExampleWriteTask2；ExampleReadTask1和ExampleReadTask2的优先级高于ExampleWriteTask2任务，并申请读信号量；ExampleWriteTask2申请写信号量。
2. ExampleTaskEntry解锁任务调度后，ExampleReadTask1、ExampleReadTask2获取读信号量失败，永久阻塞等待，ExampleWriteTask2等待1tick获取写信号量失败，改为永久阻塞等待获取写信号量。
3. ExampleTaskEntry降级写信号量为读信号量，ExampleReadTask1和ExampleReadTask2获取读信号量成功，ExampleWriteTask2继续阻塞。
4. 在ExampleReadTask1、ExampleReadTask2和ExampleTaskEntry释放读信号量之后，ExampleWriteTask2获取写信号量成功。
5. ExampleWriteTask2释放写信号量。

编程示例

前提条件：在menuconfig菜单中完成读写信号量的配置。

代码实现如下：

```
#include "los_rwsem.h"
#include "securec.h"

static UINT32 g_rwsemId; /* 读写信号量结构体id */

VOID ExampleReadTask1(VOID)
{
    UINT32 ret;

    /* 定时时间到，未申请到读信号量 */
    printf("[ExampleReadTask1] try get rwsem as read, wait forever.\n");
    /* 永久阻塞模式申请读信号量 */
    ret = LOS_RwsemPendRead(g_rwsemId, LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        printf("[ExampleReadTask1] get rwsem success.\n");
        LOS_TaskDelay(3);
        printf("[ExampleReadTask1] post rwsem.\n");
    }
```

```

        LOS_RwsemPostRead(g_rwsemId);
        return;
    }
}

VOID ExampleReadTask2(VOID)
{
    UINT32 ret;

    /* 定时时间到，未申请到读信号量 */
    printf("[ExampleReadTask2] try get rwsem as read, wait forever.\n");
    /* 永久阻塞模式申请读信号量 */
    ret = LOS_RwsemPendRead(g_rwsemId, LOS_WAIT_FOREVER);
    if (ret == LOS_OK) {
        printf("[ExampleReadTask2] get rwsem success.\n");
        LOS_TaskDelay(3);
        printf("[ExampleReadTask2] post rwsem.\n");
        LOS_RwsemPostRead(g_rwsemId);
        return;
    }
}

VOID ExampleWriteTask2(VOID)
{
    UINT32 ret;

    printf("[ExampleWriteTask2] try get rwsem as write, timeout 1 tick.\n");
    /* 定时阻塞模式申请写信号量，定时时间为1tick */
    ret = LOS_RwsemPendWrite(g_rwsemId, 1);

    /* 申请到写信号量 */
    if (ret == LOS_OK) {
        LOS_RwsemPostWrite(g_rwsemId);
        return;
    }
    /* 定时时间到，未申请到写信号量 */
    if (ret == LOS_ERRNO_RWSEM_TIMEOUT) {
        printf("[ExampleWriteTask2] timeout and try get rwsem as write, wait forever.\n");
        /* 永久阻塞模式申请写信号量 */
        ret = LOS_RwsemPendWrite(g_rwsemId, LOS_WAIT_FOREVER);
        if (ret == LOS_OK) {
            printf("[ExampleWriteTask2] get rwsem success.\n");
            LOS_TaskDelay(3);
            printf("[ExampleWriteTask2] post rwsem.\n");
            LOS_RwsemPostWrite(g_rwsemId);
            return;
        }
    }
    return;
}

static VOID InitTaskParam(TSK_INIT_PARAM_S *task, char *taskName, TSK_ENTRY_FUNC entry, UINT16
prio, UINT16 affi)
{
    task->pcName      = taskName;
    task->pfnTaskEntry = entry;
    task->usTaskPrio   = prio;
    task->uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task->uwResved     = LOS_TASK_STATUS_DETACHED;

#ifdef LOSCFG_KERNEL_SMP
    task->usCpuAffiMask = affi;
#endif
}

UINT32 ExampleTaskEntry(VOID)
{
    UINT32 ret;
    UINT32 readTsk1Id, readTsk2Id, writeTsk2Id;

```

```
TSK_INIT_PARAM_S readTask1 = {0};
TSK_INIT_PARAM_S readTask2 = {0};
TSK_INIT_PARAM_S writeTask2 = {0};

/* 创建读写信号量 */
ret = LOS_RwsemCreate(&g_rwsemId);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] create rwsem failed.\n");
    return LOS_NOK;
}

ret = LOS_RwsemPendWrite(g_rwsemId, 0);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] get rwsem failed.\n");
    return LOS_NOK;
}
printf("[ExampleTaskEntry] get rwsem success.\n");

/* 锁任务调度 */
LOS_TaskLock();

/* 创建读任务1 */
InitTaskParam(&readTask1, "resemReadTsk1", (TSK_ENTRY_FUNC)ExampleReadTask1, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
ret = LOS_TaskCreate(&readTsk1Id, &readTask1);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] readTask1 create failed.\n");
    return LOS_NOK;
}

/* 创建读任务2 */
InitTaskParam(&readTask2, "resemReadTsk2", (TSK_ENTRY_FUNC)ExampleReadTask2, 5,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
ret = LOS_TaskCreate(&readTsk2Id, &readTask2);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] readTask2 create failed.\n");
    return LOS_NOK;
}

/* 创建写任务2 */
InitTaskParam(&writeTask2, "resemWriteTsk2", (TSK_ENTRY_FUNC)ExampleWriteTask2, 9,
CPUID_TO_AFFI_MASK(ArchCurrCpuId()));
ret = LOS_TaskCreate(&writeTsk2Id, &writeTask2);
if (ret != LOS_OK) {
    printf("[ExampleTaskEntry] writeTask2 create failed.\n");
    return LOS_NOK;
}

/* 解锁任务调度 */
LOS_TaskUnlock();

/* 让出CPU 5tick,等待readTask1、readTask2、writeTask2先执行并阻塞 */
LOS_TaskDelay(5);

/* 写信号量降级为读信号量 */
printf("[ExampleTaskEntry] downgrade rwsem.\n");
LOS_RwsemDowngradeWrite(g_rwsemId);

/* 任务休眠5ticks */
LOS_TaskDelay(5);

LOS_RwsemPostRead(g_rwsemId);

/* 任务休眠10ticks */
LOS_TaskDelay(10);

/* 删除读写信号量 */
LOS_RwsemDelete(g_rwsemId);
```

```
    return LOS_OK;
}
```

结果验证

编译运行所得结果如下：

```
[ExampleTaskEntry] get rwsem success.
[ExampleReadTask1] try get rwsem as read, wait forever.
[ExampleReadTask2] try get rwsem as read, wait forever.
[ExampleWriteTask2] try get rwsem as write, timeout 1 tick.
[ExampleWriteTask2] timeout and try get rwsem as write, wait forever.
[ExampleTaskEntry] downgrade rwsem.
[ExampleReadTask1] get rwsem success.
[ExampleReadTask2] get rwsem success.
[ExampleReadTask1] post rwsem.
[ExampleReadTask2] post rwsem.
[ExampleWriteTask2] get rwsem success.
[ExampleWriteTask2] post rwsem.
```

完整实例代码

sample_rwsem_cond.rar

3.10 互斥锁

3.10.1 概述

基本概念

互斥锁又称互斥型信号量，是一种特殊的二值性信号量，用于实现对临界资源的独占式处理。另外，互斥锁可以解决信号量存在的优先级翻转问题。

任意时刻互斥锁只有两种状态，开锁或闭锁。当任务持有时，这个任务获得该互斥锁的所有权，互斥锁处于闭锁状态。当该任务释放锁后，任务失去该互斥锁的所有权，互斥锁处于开锁状态。当一个任务持有互斥锁时，其他任务不能再对该互斥锁进行开锁或持有。

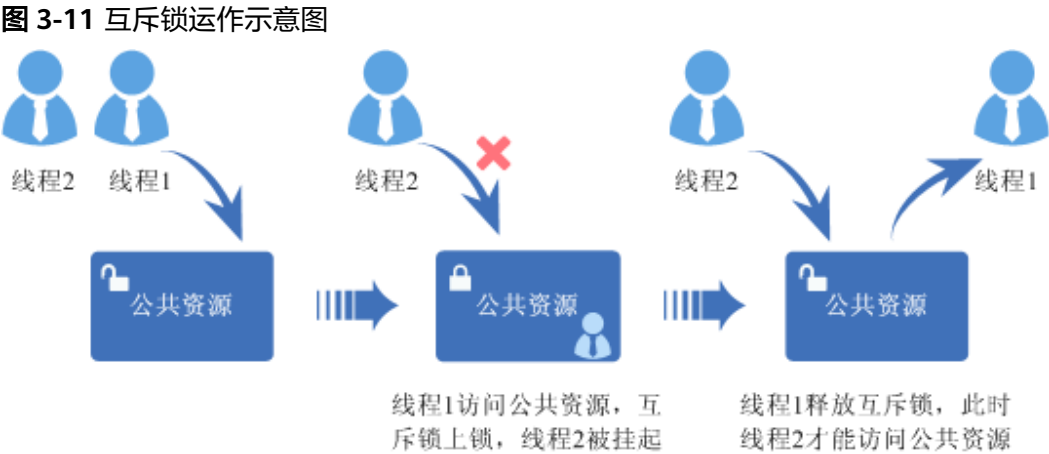
Huawei LiteOS提供的互斥锁具有如下特点：

- 通过优先级继承算法，解决优先级翻转问题。
- 多任务阻塞等待同一个锁的场景，支持基于任务优先级等待和FIFO两种模式。

运作机制

多任务环境下会存在多个任务访问同一公共资源的场景，而有些公共资源是非共享的临界资源，只能被独占使用。互斥锁怎样来避免这种冲突呢？

用互斥锁处理临界资源的同步访问时，如果有任务访问该资源，则互斥锁为加锁状态。此时其他任务如果想访问这个临界资源则会被阻塞，直到互斥锁被持有该锁的任务释放后，其他任务才能重新访问该公共资源，此时互斥锁再次上锁，如此确保同一时刻只有一个任务正在访问这个临界资源，保证了临界资源操作的完整性。



3.10.2 开发指导

使用场景

多任务环境下往往存在多个任务竞争同一临界资源的应用场景，互斥锁可以提供任务间的互斥机制，防止两个任务在同一时刻访问相同的临界资源，从而实现独占式访问。

功能

Huawei LiteOS 的互斥锁模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
创建/删除互斥锁	LOS_MuxCreate	创建互斥锁
	LOS_MuxDelete	删除指定互斥锁
申请/释放互斥锁	LOS_MuxPend	申请指定互斥锁
	LOS_MuxPost	释放指定互斥锁

说明

申请互斥锁有三种模式：无阻塞模式、永久阻塞模式、定时阻塞模式。

- 无阻塞模式：即任务申请互斥锁时，入参timeout等于0。若当前没有任务持有该互斥锁，或者持有该互斥锁的任务和申请该互斥锁的任务为同一个任务，则申请成功，否则立即返回申请失败。
- 永久阻塞模式：即任务申请互斥锁时，入参timeout等于0xFFFFFFFF。若当前没有任务持有该互斥锁，则申请成功。否则，任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，直到有其他任务释放该互斥锁，阻塞任务才会重新得以执行。
- 定时阻塞模式：即任务申请互斥锁时， $0 < \text{timeout} < 0xFFFFFFFF$ 。若当前没有任务持有该互斥锁，则申请成功。否则该任务进入阻塞态，系统切换到就绪任务中优先级最高者继续执行。任务进入阻塞态后，超时前如果有其他任务释放该互斥锁，则该任务可成功获取互斥锁继续执行，若超时前未获取到该互斥锁，接口将返回超时错误码。

释放互斥锁：

- 如果有任务阻塞于该互斥锁，则唤醒被阻塞任务中优先级最高的，该任务进入就绪态，并进行任务调度。
- 如果没有任务阻塞于该互斥锁，则互斥锁释放成功。

互斥锁错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_MUX_NO_MEMORY	0x02001d00	初始化互斥锁模块时，内存不足	设置更大的系统动态内存池，配置项为OS_SYS_MEM_SIZE，或减少系统支持的最大互斥锁个数
2	LOS_ERRNO_MUX_INVALID	0x02001d01	互斥锁不可用	传入有效的互斥锁ID
3	LOS_ERRNO_MUX_PTR_NULL	0x02001d02	创建互斥锁时，入参为空指针	传入有效指针
4	LOS_ERRNO_MUX_ALL_BUSY	0x02001d03	创建互斥锁时，系统中已经没有可用的互斥锁	增加系统支持的最大互斥锁个数
5	LOS_ERRNO_MUX_UNAVAILABLE	0x02001d04	申请互斥锁失败，因为锁已经被其他线程持有	等待其他线程解锁或者设置等待时间
6	LOS_ERRNO_MUX_PENDING_INTERRUPT	0x02001d05	在中断中使用互斥锁	禁止在中断中申请/释放互斥锁
7	LOS_ERRNO_MUX_PENDING_LOCK	0x02001d06	锁任务调度时，不允许以阻塞模式申请互斥锁	以非阻塞模式申请互斥锁，或使能任务调度后再阻塞申请互斥锁

序号	定义	实际数值	描述	参考解决方案
8	LOS_ERRNO_MUX_TIMEO UT	0x0200 1d07	申请互斥锁超时	增加等待时间，或采用一直等待模式
9	LOS_ERRNO_MUX_PENDE D	0x0200 1d09	删除正在使用的互斥锁	等待解锁后再删除该互斥锁

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为互斥锁模块，值为0x1d。

开发流程

互斥锁典型场景的开发流程：

1. 打开菜单，进入Kernel ---> Enable Mutex菜单，完成互斥锁的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_IPC_MUX	互斥锁模块裁剪开关	YES/NO	YES	无
LOSCFG_MUTEX_WAITMODE_PRIO	互斥锁基于任务优先级的等待模式	YES/NO	YES	LOSCFG_BASE_IPC_MUX
LOSCFG_MUTEX_WAITMODE_FIFO	互斥锁基于FIFO的等待模式	YES/NO	NO	LOSCFG_BASE_IPC_MUX
LOSCFG_BASE_IPC_MUX_LIMIT	系统支持的最大互斥锁个数	<65535	1024	LOSCFG_BASE_IPC_MUX

2. 创建互斥锁LOS_MuxCreate。
3. 申请互斥锁LOS_MuxPend。
4. 释放互斥锁LOS_MuxPost。
5. 删除互斥锁LOS_MuxDelete。

平台差异性

无。

3.10.3 注意事项

- 互斥锁不能在中断服务程序中使用。
- Huawei LiteOS作为实时操作系统需要保证任务调度的实时性，尽量避免任务的长时间阻塞，因此在获得互斥锁之后，应该尽快释放互斥锁。
- 持有互斥锁的过程中，不得再调用LOS_TaskPriSet等接口更改持有互斥锁任务的优先级。
- 互斥锁不支持多个相同优先级任务翻转的场景。

3.10.4 编程实例

实例描述

本实例实现如下流程。

1. 任务Example_TaskEntry创建一个互斥锁，锁任务调度，创建两个任务Example_MutexTask1、Example_MutexTask2。Example_MutexTask2优先级高于Example_MutexTask1，解锁任务调度，然后Example_TaskEntry任务休眠300Tick。
2. Example_MutexTask2被调度，以永久阻塞模式申请互斥锁，并成功获取到该互斥锁，然后任务休眠100Tick，Example_MutexTask2挂起，Example_MutexTask1被唤醒。
3. Example_MutexTask1以定时阻塞模式申请互斥锁，等待时间为10Tick，因互斥锁仍被Example_MutexTask2持有，Example_MutexTask1挂起。10Tick超时时间到达后，Example_MutexTask1被唤醒，以永久阻塞模式申请互斥锁，因互斥锁仍被Example_MutexTask2持有，Example_MutexTask1挂起。
4. 100Tick休眠时间到达后，Example_MutexTask2被唤醒，释放互斥锁，唤醒Example_MutexTask1。Example_MutexTask1成功获取到互斥锁后，释放锁。
5. 300Tick休眠时间到达后，任务Example_TaskEntry被调度运行，删除互斥锁，删除两个任务。

编程示例

前提条件：打开菜单完成互斥锁的配置。

代码实现如下：

```
/* 互斥锁句柄id */
UINT32 g_testMux;
/* 任务ID */
UINT32 g_testTaskId01;
UINT32 g_testTaskId02;

VOID Example_MutexTask1(VOID)
{
    UINT32 ret;

    printf("task1 try to get mutex, wait 10 ticks.\n");
    /* 申请互斥锁 */
    ret = LOS_MuxPend(g_testMux, 10);

    if (ret == LOS_OK) {
        printf("task1 get mutex g_testMux.\n");
        /* 释放互斥锁 */
        LOS_MuxPost(g_testMux);
        return;
    }
}
```

```

    } else if (ret == LOS_ERRNO_MUX_TIMEOUT ) {
        printf("task1 timeout and try to get mutex, wait forever.\n");
        /* 申请互斥锁 */
        ret = LOS_MuxPend(g_testMux, LOS_WAIT_FOREVER);
        if (ret == LOS_OK) {
            printf("task1 wait forever, get mutex g_testMux.\n");
            /* 释放互斥锁 */
            LOS_MuxPost(g_testMux);
            return;
        }
    }
    return;
}

VOID Example_MutexTask2(VOID)
{
    printf("task2 try to get mutex, wait forever.\n");
    /* 申请互斥锁 */
    (VOID)LOS_MuxPend(g_testMux, LOS_WAIT_FOREVER);

    printf("task2 get mutex g_testMux and suspend 100 ticks.\n");

    /* 任务休眠100Ticks */
    LOS_TaskDelay(100);

    printf("task2 resumed and post the g_testMux\n");
    /* 释放互斥锁 */
    LOS_MuxPost(g_testMux);
    return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task1;
    TSK_INIT_PARAM_S task2;

    /* 创建互斥锁 */
    LOS_MuxCreate(&g_testMux);

    /* 锁任务调度 */
    LOS_TaskLock();

    /* 创建任务1 */
    memset(&task1, 0, sizeof(TSK_INIT_PARAM_S));
    task1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_MutexTask1;
    task1.pcName = "MutexTsk1";
    task1.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task1.usTaskPrio = 5;
    ret = LOS_TaskCreate(&g_testTaskId01, &task1);
    if (ret != LOS_OK) {
        printf("task1 create failed.\n");
        return LOS_NOK;
    }

    /* 创建任务2 */
    memset(&task2, 0, sizeof(TSK_INIT_PARAM_S));
    task2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_MutexTask2;
    task2.pcName = "MutexTsk2";
    task2.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task2.usTaskPrio = 4;
    ret = LOS_TaskCreate(&g_testTaskId02, &task2);
    if (ret != LOS_OK) {
        printf("task2 create failed.\n");
        return LOS_NOK;
    }

    /* 解锁任务调度 */
    LOS_TaskUnlock();
}

```

```
/* 休眠300Ticks */
LOS_TaskDelay(300);

/* 删除互斥锁 */
LOS_MuxDelete(g_testMux);

/* 删除任务1 */
ret = LOS_TaskDelete(g_testTaskId01);
if (ret != LOS_OK) {
    printf("task1 delete failed .\n");
    return LOS_NOK;
}
/* 删除任务2 */
ret = LOS_TaskDelete(g_testTaskId02);
if (ret != LOS_OK) {
    printf("task2 delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
task2 try to get  mutex, wait forever.
task2 get mutex g_testMux and suspend 100 ticks.
task1 try to get  mutex, wait 10 ticks.
task1 timeout and try to get mutex, wait forever.
task2 resumed and post the g_testMux
task1 wait forever,get mutex g_testMux.
```

完整实例代码

sample_mutex.c

3.11 软件定时器

3.11.1 概述

基本概念

软件定时器，是基于系统Tick时钟中断且由软件来模拟的定时器。当经过设定的Tick数后，会触发用户自定义的回调函数。

硬件定时器受硬件的限制，数量上不足以满足用户的实际需求。因此为了满足用户需求，提供更多的定时器，Huawei LiteOS提供了软件定时器功能，支持如下特性：

- 创建软件定时器。
- 启动软件定时器。
- 停止软件定时器。
- 删除软件定时器。
- 获取软件定时器剩余Tick数。
- 可配置支持的软件定时器个数。

运作机制

软件定时器是系统资源，在模块初始化的时候已经分配了一块连续内存。

软件定时器使用了系统的一个队列和一个任务资源，软件定时器的触发遵循队列规则，先进先出。定时时间短的定时器总是比定时时间长的靠近队列头，满足优先触发的准则。

软件定时器以Tick为基本计时单位，当创建并启动一个软件定时器时，Huawei LiteOS会根据当前系统Tick时间及设置的定时时长确定该定时器的到期Tick时间，并将该定时器控制结构挂入计时全局链表。

当Tick中断到来时，在Tick中断处理函数中扫描软件定时器的计时全局链表，检查是否有定时器超时，若有则将超时的定时器记录下来。Tick中断处理函数结束后，软件定时器任务（优先级为最高）被唤醒，在该任务中调用已经记录下来的定时器的回调函数。

定时器状态

- `OS_SWTMR_STATUS_UNUSED`（定时器未使用）
系统在定时器模块初始化时，会将系统中所有定时器资源初始化成该状态。
- `OS_SWTMR_STATUS_TICKING`（定时器处于计数状态）
在定时器创建后调用`LOS_SwtmrStart`接口启动，定时器将变成该状态，是定时器运行时的状态。
- `OS_SWTMR_STATUS_CREATED`（定时器创建后未启动，或已停止）
定时器创建后，不处于计数状态时，定时器将变成该状态。
- `OS_SWTMR_STATUS_DELETING`（定时器被删除了）
定时器类型是单次触发类型，在定时器超时后，定时器将变成该状态。

定时器模式

软件定时器提供了三类模式：

- 单次触发定时器，这类定时器在启动后只会触发一次定时器事件，然后定时器自动删除。
- 周期触发定时器，这类定时器会周期性的触发定时器事件，直到用户手动停止定时器，否则将永远持续执行下去。
- 单次触发定时器，但这类定时器超时触发后不会自动删除，需要调用定时器删除接口删除定时器。

3.11.2 开发指导

使用场景

- 创建一个单次触发的定时器，超时后执行用户自定义的回调函数。
- 创建一个周期性触发的定时器，超时后执行用户自定义的回调函数。

功能

Huawei LiteOS的软件定时器模块为用户提供下面几种功能，接口详细信息可以查看API参考。

表 3-2

功能分类	接口名	描述
创建/删除定时器	LOS_SwtmrCreate	创建定时器，设置定时器的定时时长、定时器模式、回调函数，并返回定时器ID
	LOS_SwtmrDelete	异步删除定时器，接口在入参检测后，立即返回，不保障接口返回时软件定时器资源已经被释放，软件定时器资源将在目标定时器所有回调执行完毕后释放
	LOS_SwtmrSyncDelete	同步删除定时器，接口在入参检测后，会等待目标定时器的所有回调执行完毕，而后删除定时器，释放软件定时器资源
启动/停止定时器	LOS_SwtmrStart	启动定时器
	LOS_SwtmrStop	停止定时器
获得软件定时器剩余Tick数	LOS_SwtmrTimeGet	获得软件定时器剩余Tick数

软件定时器错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_SWTMR_PTR_NULL	0x02000300	软件定时器回调函数为空	定义软件定时器回调函数
2	LOS_ERRNO_SWTMR_INTERVAL_NOT_SUITED	0x02000301	软件定时器的定时时长为0	重新定义定时器的定时时长
3	LOS_ERRNO_SWTMR_MODE_INVALID	0x02000302	不正确的软件定时器模式	确认软件定时器模式，范围为[0,2]
4	LOS_ERRNO_SWTMR_RET_PTR_NULL	0x02000303	入参的软件定时器ID指针为NULL	定义ID变量，传入有效指针
5	LOS_ERRNO_SWTMR_MAXSIZE	0x02000304	软件定时器个数超过最大值	重新设置软件定时器最大个数，或者等待一个软件定时器释放资源
6	LOS_ERRNO_SWTMR_ID_INVALID	0x02000305	入参的软件定时器ID不正确	确保入参合法

序号	定义	实际数值	描述	参考解决方案
7	LOS_ERRNO_SWTMR_NOT_CREATED	0x02000306	软件定时器未创建	创建软件定时器
8	LOS_ERRNO_SWTMR_NO_MEMORY	0x02000307	初始化软件定时器模块时，内存不足	调整OS_SYS_MEM_SIZE，以确保有足够的内存供软件定时器使用
9	LOS_ERRNO_SWTMR_HWI_ACTIVE	0x02000309	在中断中使用定时器	修改源代码确保不在中断中使用
10	LOS_ERRNO_SWTMR_QUEUE_CREATE_FAILED	0x0200030b	在软件定时器初始化时，创建定时器队列失败	调整OS_SYS_MEM_SIZE，以确保有足够的内存供软件定时器创建队列
11	LOS_ERRNO_SWTMR_TASK_CREATE_FAILED	0x0200030c	在软件定时器初始化时，创建定时器任务失败	调整OS_SYS_MEM_SIZE，以确保有足够的内存供软件定时器创建任务
12	LOS_ERRNO_SWTMR_NOT_STARTED	0x0200030d	未启动软件定时器	启动软件定时器
13	LOS_ERRNO_SWTMR_STATUS_INVALID	0x0200030e	不正确的软件定时器状态	检查确认软件定时器状态
14	LOS_ERRNO_SWTMR_TICK_PTR_NULL	0x02000310	用以获取软件定时器剩余Tick数的入参指针为NULL	定义有效变量以传入有效指针
15	LOS_ERRNO_SWTMR_SORTLINK_CREATE_FAILED	0x02000311	在软件定时器初始化时，创建定时器链表失败	调整OS_SYS_MEM_SIZE，以确保有足够的内存供软件定时器创建链表
16	LOS_ERRNO_SWTMR_INVALID_SYNCDEL	0x02000312	在中断或软件定时器回调中尝试同步删除定时器，删除失败	调整代码逻辑，避免在中断或软件定时器回调中同步删除软件定时器

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为软件定时器模块，值为0x03。

开发流程

软件定时器的典型开发流程：

1. 打开菜单，进入Kernel ---> Enable Software Timer菜单，完成软件定时器的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_CORE_SWTMR	软件定时器裁剪开关	YES/NO	YES	LOSCFG_BASE_IPC_QUEUE
LOSCFG_BASE_CORE_SWTMR_LIMIT	最大支持的软件定时器数	<65535	1024	LOSCFG_BASE_CORE_SWTMR
LOSCFG_BASE_CORE_SWTMR_IN_ISR	在中断中直接执行回调函数	YES/NO	NO	LOSCFG_BASE_CORE_SWTMR
LOSCFG_BASE_CORE_TASK_SWTMR_STACK_SIZE (菜单路径为：Kernel ---> Basic Config ---> Task)	软件定时器任务栈大小	[LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE, OS_SYS_MEM_SIZE)	24576	LOSCFG_BASE_CORE_SWTMR

2. 创建定时器LOS_SwtmrCreate，设置定时器的定时时长、定时器模式、超时后的回调函数。
3. 启动定时器LOS_SwtmrStart。
4. 获得软件定时器剩余Tick数LOS_SwtmrTimeGet。
5. 停止定时器LOS_SwtmrStop。
6. 删除定时器LOS_SwtmrDelete。

3.11.3 注意事项

- 软件定时器的回调函数中不应执行过多操作，不建议使用可能引起任务挂起或者阻塞的接口或操作，如果使用会导致软件定时器响应不及时，可能会影响正常业务功能。
- 如果没有使能LOSCFG_BASE_CORE_SWTMR_IN_ISR宏，软件定时器将使用系统的一个任务资源。软件定时器任务的优先级设定为0，且不允许修改。
- 系统可配置的软件定时器个数是指：整个系统可使用的软件定时器总个数，并非用户可使用的软件定时器个数。例如：系统多占用一个软件定时器，那么用户能使用的软件定时器资源就会减少一个。
- 创建单次不自删除属性的定时器，用户需要自行调用定时器删除接口删除定时器，回收定时器资源，避免资源泄露。
- 软件定时器的定时精度与系统Tick时钟的周期有关。

3.11.4 编程实例

实例描述

在下面的例子中，演示如下功能：

1. 软件定时器创建、启动、停止、删除操作。
2. 单次软件定时器，周期软件定时器使用方法。

编程示例

前提条件：在menuconfig菜单中完成软件定时器的配置。

代码实现如下：

```
UINT32 g_timerCount1 = 0;
UINT32 g_timerCount2 = 0;

VOID Timer1_CallBack(UINT32 arg)
{
    UINT64 lastTick;

    g_timerCount1++;
    lastTick=(UINT32)LOS_TickCountGet();
    dprintf("g_timerCount1=%d\n", g_timerCount1);
    dprintf("tick_last1=%d\n", lastTick);
}

VOID Timer2_CallBack(UINT32 arg)
{
    UINT64 lastTick;

    lastTick=(UINT32)LOS_TickCountGet();
    g_timerCount2++;
    dprintf("g_timerCount2=%d\n", g_timerCount2);
    dprintf("tick_last2=%d\n", lastTick);
}

VOID Timer_example(VOID)
{
    UINT16 id1;    // Timer1 id
    UINT16 id2;    // Timer2 id
    UINT32 tick;

    LOS_SwtmrCreate(1000, LOS_SWTMR_MODE_ONCE, Timer1_CallBack, &id1, 1);
    LOS_SwtmrCreate(100, LOS_SWTMR_MODE_PERIOD, Timer2_CallBack, &id2, 1);
    dprintf("create Timer1 success\n");

    LOS_SwtmrStart(id1);
    dprintf("start Timer1 sucess\n");
    LOS_TaskDelay(200);
    LOS_SwtmrTimeGet(id1, &tick);
    dprintf("tick =%d\n", tick);
    LOS_SwtmrStop(id1);
    dprintf("stop Timer1 sucess\n");

    LOS_SwtmrStart(id1);
    LOS_TaskDelay(1000);
    LOS_SwtmrDelete(id1);
    dprintf("delete Timer1 sucess\n");

    LOS_SwtmrStart(id2);
    dprintf("start Timer2\n");
    LOS_TaskDelay(1000);
    LOS_SwtmrStop(id2);
}
```

```
    LOS_SwtmrDelete(id2);  
}
```

结果验证

得到的结果为：

```
Create Timer1 success  
start Timer1 sucess  
tick =800  
Stop Timer1 sucess  
g_timerCount1=1201  
tick_last1=1201  
delete Timer1 success  
Start Timer2  
g_timerCount2=1  
tick_last2=1301  
g_timerCount2=2  
tick_last2=1401  
g_timerCount2=3  
tick_last2=1501  
g_timerCount2=4  
tick_last2=1601  
g_timerCount2=5  
tick_last2=1701  
g_timerCount2=6  
tick_last2=1801  
g_timerCount2=7  
tick_last2=1901  
g_timerCount2=8  
tick_last2=2001  
g_timerCount2=9  
tick_last2=2101  
g_timerCount2=10  
tick_last2=2201
```

完整实例代码

sample_Timer.c

3.12 自旋锁

3.12.1 概述

在多核环境中，由于使用相同的内存空间，存在对同一资源进行访问的情况，所以需要互斥访问机制来保证同一时刻只有一个核进行操作。自旋锁就是这样的一种机制。

自旋锁是指当一个线程在获取锁时，如果锁已经被其它线程获取，那么该线程将循环等待，并不断判断是否能够成功获取锁，直到获取到锁才会退出循环。因此建议保护耗时较短的操作，防止对系统整体性能有明显的影晌。

自旋锁与互斥锁比较类似，它们都是为了解决对共享资源的互斥使用问题。无论是互斥锁，还是自旋锁，在任何时刻，最多只能有一个持有者。但是两者在调度机制上略有不同，对于互斥锁，如果锁已经被占用，锁申请者会被阻塞；但是自旋锁不会引起调用者阻塞，会一直循环检测自旋锁是否已经被释放。

3.12.2 开发指导

使用场景

自旋锁可以提供任务之间的互斥访问机制，用来防止两个任务在同一时刻访问相同的共享资源。

功能

Huawei LiteOS 的自旋锁模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述	注意事项
初始化自旋锁	LOS_SpinInit	动态初始化自旋锁	由调用者保证入参合法
	SPIN_LOCK_INIT	静态初始化自旋锁	不涉及
申请/释放自旋锁	LOS_SpinLock	申请指定的自旋锁，如果无法获取锁，会一直循环等待	由调用者保证入参合法
	LOS_SpinTrylock	尝试申请指定的自旋锁，如果无法获取锁，直接返回失败，而不会一直循环等待	由调用者保证入参合法
	LOS_SpinUnlock	释放指定的自旋锁	由调用者保证入参合法
	LOS_SpinUnlockNoSchedule	释放指定的自旋锁，并继续执行本任务（建议参考注释使用在特殊场景，错误使用会导致调度时序错乱）。	由调用者保证入参合法
申请/释放自旋锁（同时进行关中断保护）	LOS_SpinLockSave	关中断后，再申请指定的自旋锁	由调用者保证入参合法
	LOS_SpinUnlockRestore	先释放指定的自旋锁，再恢复中断状态	由调用者保证入参合法
获取自旋锁持有状态	LOS_SpinHeld	检查自旋锁是否已经被持有	由调用者保证入参合法

开发流程

自旋锁的开发典型流程：

1. 自旋锁依赖于SMP，打开菜单，配置项的菜单路径为：Kernel ---> Enable Kernel SMP。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERN EL_SMP	SMP控制开关	YES/NO	YES	硬件支持 多核
LOSCFG_KERN EL_SMP_CORE _NUM	多核core数量	与架构相 关	2	无

2. 创建自旋锁：使用LOS_SpinInit初始化自旋锁，或者使用SPIN_LOCK_INIT初始化静态内存的自旋锁。
3. 申请自旋锁：使用接口LOS_SpinLock/LOS_SpinTrylock/LOS_SpinLockSave申请指定的自旋锁，申请成功就继续往后执行锁保护的代码；申请失败在自旋锁申请中忙等，直到申请到自旋锁为止。
4. 释放自旋锁：使用LOS_SpinUnlock/LOS_SpinUnlockRestore接口释放自旋锁。锁保护代码执行完毕后，释放对应的自旋锁，以便其他核申请自旋锁。

3.12.3 注意事项

- 主要注意函数入参的合法性由调用者保证。
- 同一个任务不能对同一把自旋锁进行多次加锁，否则会导致死锁。
- 自旋锁中会执行本核的锁任务操作，因此需要等到最外层完成解锁后本核才会进行任务调度。
- LOS_SpinLock与LOS_SpinUnlock允许单独使用，即可以不进行关中断，但是用户需要保证使用的接口只会在任务或中断中使用。如果接口同时会在任务和中断中被调用，请使用LOS_SpinLockSave与LOS_SpinUnlockRestore，因为在未关中断的情况下使用LOS_SpinLock可能会导致死锁。
- 耗时的操作谨慎选用自旋锁，可使用互斥锁进行保护。
- 未开启SMP的单核场景下，自旋锁功能无效，只有LOS_SpinLockSave与LOS_SpinUnlockRestore接口有关闭恢复中断功能。
- 建议LOS_SpinLock和LOS_SpinUnlock，LOS_SpinLockSave和LOS_SpinUnlockRestore配对使用，避免出错。

3.12.4 编程实例

实例描述

本实例实现如下流程。

1. 任务Example_TaskEntry初始化自旋锁，创建两个任务Example_SpinTask1、Example_SpinTask2，分别运行于两个核。
2. Example_SpinTask1、Example_SpinTask2中均执行申请自旋锁的操作，同时为了模拟实际操作，在持有自旋锁后进行延迟操作，最后释放自旋锁。
3. 300Tick后任务Example_TaskEntry被调度运行，删除任务Example_SpinTask1和Example_SpinTask2。

须知

多核的运行时序不是固定的，因此也存在任务执行顺序不同的情况。

编程示例

前提条件：在menuconfig中，将LOSCFG_KERNEL_SMP配置项打开，并设置多核core数量。

代码实现如下：

```
#include "los_spinlock.h"
#include "los_task.h"

/* 自旋锁句柄id */
SPIN_LOCK_S g_testSpinlock;
/* 任务ID */
UINT32 g_testTaskId01;
UINT32 g_testTaskId02;

VOID Example_SpinTask1(VOID)
{
    UINT32 i;
    UINTPTR intSave;

    /* 申请自旋锁 */
    dprintf("task1 try to get spinlock\n");
    LOS_SpinLockSave(&g_testSpinlock, &intSave);
    dprintf("task1 got spinlock\n");
    for(i = 0; i < 5000; i++) {
        asm volatile("nop");
    }

    /* 释放自旋锁 */
    dprintf("task1 release spinlock\n");
    LOS_SpinUnlockRestore(&g_testSpinlock, intSave);

    return;
}

VOID Example_SpinTask2(VOID)
{
    UINT32 i;
    UINTPTR intSave;

    /* 申请自旋锁 */
    dprintf("task2 try to get spinlock\n");
    LOS_SpinLockSave(&g_testSpinlock, &intSave);
    dprintf("task2 got spinlock\n");
    for(i = 0; i < 5000; i++) {
        asm volatile("nop");
    }

    /* 释放自旋锁 */
    dprintf("task2 release spinlock\n");
    LOS_SpinUnlockRestore(&g_testSpinlock, intSave);

    return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S stTask1;
    TSK_INIT_PARAM_S stTask2;
```

```
/* 初始化自旋锁 */
LOS_SpinInit(&g_testSpinlock);

/* 创建任务1 */
memset(&stTask1, 0, sizeof(TSK_INIT_PARAM_S));
stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SpinTask1;
stTask1.pcName = "SpinTask1";
stTask1.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
stTask1.usTaskPrio = 5;
#ifdef LOSCFG_KERNEL_SMP
/* 绑定任务到CPU0运行 */
stTask1.usCpuAffiMask = CPUID_TO_AFFI_MASK(0);
#endif
ret = LOS_TaskCreate(&g_testTaskId01, &stTask1);
if(ret != LOS_OK) {
    dprintf("task1 create failed .\n");
    return LOS_NOK;
}

/* 创建任务2 */
memset(&stTask2, 0, sizeof(TSK_INIT_PARAM_S));
stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_SpinTask2;
stTask2.pcName = "SpinTask2";
stTask2.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
stTask2.usTaskPrio = 5;
#ifdef LOSCFG_KERNEL_SMP
/* 绑定任务到CPU1运行 */
stTask1.usCpuAffiMask = CPUID_TO_AFFI_MASK(1);
#endif
ret = LOS_TaskCreate(&g_testTaskId02, &stTask2);
if(ret != LOS_OK) {
    dprintf("task2 create failed .\n");
    return LOS_NOK;
}

/* 任务休眠300Ticks */
LOS_TaskDelay(300);

/* 删除任务1 */
ret = LOS_TaskDelete(g_testTaskId01);
if(ret != LOS_OK) {
    dprintf("task1 delete failed .\n");
    return LOS_NOK;
}
/* 删除任务2 */
ret = LOS_TaskDelete(g_testTaskId02);
if(ret != LOS_OK) {
    dprintf("task2 delete failed .\n");
    return LOS_NOK;
}

return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
task2 try to get spinlock
task2 got spinlock
task1 try to get spinlock
task2 release spinlock
task1 got spinlock
task1 release spinlock
```

完整实例代码

sample_spinlock.zip

3.13 分级初始化

3.13.1 概述

基本概念

分级初始化将Kernel的启动过程分成多个阶段，阶段之间存在依赖关系，后初始化的阶段可能会依赖前初始化阶段的接口。对于Kernel启动过程新增的调用接口，不再需要修改启动代码添加函数调用，可通过分级初始化注册宏进行接口注册，Kernel在启动时，按照顺序自动执行所有注册的初始化接口。

运作机制

- 每一个需要在Kernel初始化时执行的函数，通过使用分级启动宏都会生成一个函数指针变量，指向该函数。
- 编译生成镜像时，链接脚本将上述的函数指针变量冠段。
- Kernel初始化，从冠段地址取出所有函数指针，依次执行所有初始化函数。

3.13.2 开发指导

使用场景

分级初始化，可以实现Kernel初始化模块与其他模块的充分解耦。Kernel初始化模块不再感知具体模块的初始化接口，初始化接口也不用关心如何被调用。

功能

Huawei LiteOS中的分级初始化模块提供下面几种功能，接口详细信息可以查看API参考。

接口名	描述
LOS_SYS_INIT	注册一个初始化函数，在Kernel阶段会运行该函数

层级划分：分级初始化时，下表中越靠前的层级，优先级越高，越先得到执行。

启动层级宏	描述
SYS_INIT_LEVEL_EARLY	Kernel启动早期阶段，该阶段仅供用户使用，并且注册到该阶段的初始化函数不可依赖Kernel资源
SYS_INIT_LEVEL_AHEAD	Kernel启动预备阶段，该阶段仅供系统模块使用，为内核初始化做准备
SYS_INIT_LEVEL_ARCH	该阶段仅供系统模块使用，架构相关初始化

启动层级宏	描述
SYS_INIT_LEVEL_KERNEL	该阶段仅供系统模块使用，内核基础子模块初始化
SYS_INIT_LEVEL_KERNEL_ADDITION	该阶段仅供系统模块使用，扩展内核模块初始化
SYS_INIT_LEVEL_COMPONENT	该阶段仅供系统模块使用，组件相关初始化
SYS_INIT_LEVEL_APP	应用程序初始化
SYS_INIT_LEVEL_RESERVE	Kernel启动后期阶段，该阶段供用户使用，注册到该阶段的初始化函数可依赖Kernel资源

优先级划分：同一层级的初始化函数之间，可能存在依赖关系。sync优先级只能用于设置同一层级内的函数优先级。sync值越小，优先级越高，越先执行。

优先级宏	描述
SYS_INIT_SYNC_0	sync 0优先级，在同一层级内最先执行
SYS_INIT_SYNC_1	sync 1优先级
SYS_INIT_SYNC_2	sync 2优先级
SYS_INIT_SYNC_3	sync 3优先级
SYS_INIT_SYNC_4	sync 4优先级

开发流程

1. 确定初始化接口的依赖关系，进而确定层级及优先级。
2. 调用LOS_SYS_INIT注册初始化接口。

3.13.3 注意事项

- 分级初始化适用于类型为 typedef unsigned int (*SysInitcallFunc)(void)的初始化函数。
- 由于不会在代码中直接调用初始化函数，一些编译器可能会因此优化掉初始化函数，导致注册失败，建议使用-u链接选项。
- 同一个层级、sync优先级内，可以注册多个初始化接口，但这些接口之间不能存在依赖关系。

3.13.4 编程实例

实例描述

在下面的实例中，描述了分级初始化注册接口的使用方法：

1. 确定初始化接口的依赖关系，层级及优先级。
2. 注册初始化接口。

编程示例

```
#include "stdio.h"
#include "los_init.h"
#include "los_list.h"
#include "los_hwi.h"

#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

UINT32 g_count;
LOS_DL_LIST g_DLlist;

UINT32 InitLevelTest1(VOID)
{
    dprintf("InitLevelTest1 init\n");
    g_count = 10;
    LOS_ListInit(&g_DLlist);

    return LOS_OK;
}
/* InitLevelTest1 has no dependence, set SYS_INIT_LEVEL_EARLY level */
LOS_SYS_INIT(InitLevelTest1, SYS_INIT_LEVEL_EARLY, SYS_INIT_SYNC_0);

STATIC VOID HwiUsrIrq(VOID)
{
    g_count++;
}

UINT32 InitLevelTest2(VOID)
{
    UINT32 ret;
    HWI_HANDLE_T irqNum = 26;
    HWI_PRIOR_T irqPri = 0x3;

    dprintf("InitLevelTest2 init\n");
    ret = LOS_HwiCreate(irqNum, irqPri, 0, (HWI_PROC_FUNC)HwiUsrIrq, 0);
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
    return ret;
}
/* InitLevelTest2 rely on hwi module, set SYS_INIT_LEVEL_RESERVE level */
LOS_SYS_INIT(InitLevelTest2, SYS_INIT_LEVEL_RESERVE, SYS_INIT_SYNC_0);

#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */
```

结果验证

编译运行得到的结果为：

```
InitLevelTest1 init
InitLevelTest2 init
```

完整实例代码

sample_initlevel.rar

3.14 其他

3.14.1 时间管理

3.14.1.1 概述

基本概念

时间管理以系统时钟为基础，给应用程序提供所有和时间有关的服务。

系统时钟是由定时器/计数器产生的输出脉冲触发中断产生的，一般定义为整数或长整数。输出脉冲的周期叫做一个“时钟滴答”。系统时钟也称为时标或者Tick。

用户以秒、毫秒为单位计时，而操作系统以Tick为单位计时，当用户需要对系统进行操作时，例如任务挂起、延时等，此时需要时间管理模块对Tick和秒/毫秒进行转换。

Huawei LiteOS的时间管理模块提供时间转换、统计、延迟功能。

相关概念

- Cycle
系统最小的计时单位。Cycle的时长由系统主时钟频率决定，系统主时钟频率就是每秒钟的Cycle数。
- Tick
Tick是操作系统的基本时间单位，由用户配置的每秒Tick数决定。

3.14.1.2 开发指导

使用场景

用户需要了解当前系统运行的时间以及Tick与秒、毫秒之间的转换关系等。

功能

Huawei LiteOS的时间管理提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
时间转换	LOS_MS2Tick	采用向上取整的计算方法将毫秒转换成Tick
	LOS_Tick2MS	Tick转化为毫秒
时间统计	LOS_CyclePerTickGet	每个Tick多少Cycle数
	LOS_TickCountGet	获取自系统启动以来的Tick数

功能分类	接口名	描述
	LOS_GetCpuCycle	获取自系统启动以来的Cycle数
	LOS_CurrNanosec	获取自系统启动以来的纳秒数
延时管理	LOS_Udelay	以us为单位的忙等，但可以被优先级更高的任务抢占
	LOS_Mdelay	以ms为单位的忙等，但可以被优先级更高的任务抢占

时间管理错误码

时间转换存在出错的可能性，需要返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_TICK_CFG_INVALID	0x02000400	无效的系统Tick配置	在板级配置适配时配置有效的系统主时钟频率OS_SYS_CLOCK，打开菜单配置有效的LOSCFG_BASE_CORE_TICK_PER_SECOND

开发流程

时间管理的典型开发流程：

1. 根据实际需求，在板级配置适配时确认是否使能LOSCFG_BASE_CORE_TICK_HW_TIME宏选择外部定时器，并配置系统主时钟频率OS_SYS_CLOCK（单位Hz）。OS_SYS_CLOCK的默认值基于硬件平台配置。
2. 打开菜单配置LOSCFG_BASE_CORE_TICK_PER_SECOND，配置项的菜单路径为：Kernel--->Basic Config--->Task。

配置项	含义	取值范围	默认值	依赖
LOSCFG_BASE_CORE_TICK_PER_SECOND	每秒Tick数	(0, 1000]	100	无

3. 调用时钟转换/统计接口。

3.14.1.3 注意事项

- 时间管理不是单独的功能模块，依赖于OS_SYS_CLOCK和LOSCFG_BASE_CORE_TICK_PER_SECOND两个配置选项。

- 系统的Tick数在关中断的情况下不进行计数，故系统Tick数不能作为准确时间使用。

3.14.1.4 编程实例

实例描述

在下面的例子中，介绍了时间管理的基本方法，包括：

1. 时间转换：将毫秒数转换为Tick数，或将Tick数转换为毫秒数。
2. 时间统计：每Tick的Cycle数、自系统启动以来的Tick数和延迟后的Tick数。

编程示例

前提条件：

- 使用每秒的Tick数LOSCFG_BASE_CORE_TICK_PER_SECOND的默认值100。
- 配好OS_SYS_CLOCK系统主时钟频率。

时间转换：

```
VOID Example_TransformTime(VOID)
{
    UINT32 ms;
    UINT32 tick;

    tick = LOS_MS2Tick(10000); // 10000ms转换为tick
    dprintf("tick = %d \n", tick);
    ms = LOS_Tick2MS(100); // 100tick转换为ms
    dprintf("ms = %d \n", ms);
}
```

时间统计和时间延迟：

```
VOID Example_GetTime(VOID)
{
    UINT32 cyclePerTick;
    UINT64 tickCount;

    cyclePerTick = LOS_CyclePerTickGet();
    if(0 != cyclePerTick) {
        dprintf("LOS_CyclePerTickGet = %d \n", cyclePerTick);
    }

    tickCount = LOS_TickCountGet();
    if(0 != tickCount) {
        dprintf("LOS_TickCountGet = %d \n", (UINT32)tickCount);
    }

    LOS_TaskDelay(200);
    tickCount = LOS_TickCountGet();
    if(0 != tickCount) {
        dprintf("LOS_TickCountGet after delay = %d \n", (UINT32)tickCount);
    }
}
```

结果验证

编译运行得到的结果为：

时间转换：

```
tick = 1000
ms = 1000
```

时间统计和时间延迟：

```
LOS_CyclePerTickGet = 495000
LOS_TickCountGet = 1
LOS_TickCountGet after delay = 201
```

完整实例代码

```
sample_time.c
```

3.14.2 双向链表

3.14.2.1 概述

基本概念

双向链表是指含有往前和往后两个方向的链表，即每个结点中除存放下一个节点指针外，还增加一个指向前一个节点的指针。其头指针head是唯一确定的。

从双向链表中的任意一个结点开始，都可以很方便地访问它的前驱结点和后继结点，这种数据结构形式使得双向链表在查找时更加方便，特别是大量数据的遍历。由于双向链表具有对称性，能方便地完成各种插入、删除等操作，但需要注意前后方向的操作。

3.14.2.2 开发指导

功能

Huawei LiteOS的双向链表模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述	注意事项
初始化链表	LOS_ListInit	将指定节点初始化为双向链表节点	由调用者保证入参合法
	LOS_DL_LIST_HEAD	定义一个节点并初始化为双向链表节点	不涉及
增加节点	LOS_ListAdd	将指定节点插入到双向链表头端	由调用者保证入参合法
	LOS_ListHeadInsert	将指定节点插入到双向链表头端，同LOS_ListAdd	由调用者保证入参合法
	LOS_ListTailInsert	将指定节点插入到双向链表尾端	由调用者保证入参合法
删除节点	LOS_ListDelete	将指定节点从链表中删除	由调用者保证入参合法

功能分类	接口名	描述	注意事项
	LOS_ListDeInit	将指定节点从链表中删除，并使用该节点初始化链表	由调用者保证入参合法
判断双向链表是否为空	LOS_ListEmpty	判断链表是否为空	由调用者保证入参合法
获取节点	LOS_DL_LIST_LAST	获取指定节点的前驱结点	不涉及
	LOS_DL_LIST_FIRST	获取指定节点的后继结点	不涉及
获取结构体信息	LOS_DL_LIST_ENTRY	获取包含链表的结构体地址，接口的第一个入参表示的是链表中的某个节点，第二个入参是要获取的结构体名称，第三个入参是链表在该结构体中的名称	不涉及
	LOS_OFF_SET_OF	获取指定结构体内的成员相对于结构体起始地址的偏移量	不涉及
遍历双向链表	LOS_DL_LIST_FOR_EACH	遍历双向链表	不涉及
	LOS_DL_LIST_FOR_EACH_SAFE	遍历双向链表，并存储当前节点的后继节点用于安全校验	不涉及
遍历包含双向链表的结构体	LOS_DL_LIST_FOR_EACH_ENTRY	遍历指定双向链表，获取包含该链表节点的结构体地址	不涉及
	LOS_DL_LIST_FOR_EACH_ENTRY_SAFE	遍历指定双向链表，获取包含该链表节点的结构体地址，并存储包含当前节点的后继节点的结构体地址	不涉及
	LOS_DL_LIST_FOR_EACH_ENTRY_HOOK	遍历指定双向链表，获取包含该链表节点的结构体地址，并在每次循环中调用钩子函数	不涉及

开发流程

双向链表的典型开发流程：

- 1. 调用LOS_ListInit/LOS_DL_LIST_HEAD初始双向链表。
- 2. 调用LOS_ListAdd/LOS_ListHeadInsert向链表头部插入节点。
- 3. 调用LOS_ListTailInsert向链表尾部插入节点。
- 4. 调用LOS_ListDelete删除指定节点。

5. 调用LOS_ListEmpty判断链表是否为空。
6. 调用LOS_ListDelInit删除指定节点并以此节点初始化链表。

3.14.2.3 注意事项

- 需要注意函数入参的合法性由调用者保证。
- 需要注意节点指针前后方向的操作。
- 链表操作接口为底层接口，不对入参进行判空，需要使用者确保传参合法。
- 如果链表节点的内存是动态申请的，删除节点时，要注意释放内存。

3.14.2.4 编程实例

实例描述

本实例实现如下功能：

1. 初始化双向链表。
2. 增加节点。
3. 删除节点。
4. 测试操作是否成功。

编程示例

代码实现如下：

```
#include "stdio.h"
#include "los_list.h"

#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

static UINT32 DList_sample(VOID)
{
    LOS_DL_LIST DList = {NULL,NULL};
    LOS_DL_LIST DListNode01 = {NULL,NULL};
    LOS_DL_LIST DListNode02 = {NULL,NULL};
    LOS_DL_LIST DListNode03 = {NULL,NULL};

    dprintf("Initial head\n");
    LOS_ListInit(&DList);

    LOS_ListAdd(&DList, &DListNode01);
    if (DListNode01.pstNext == &DList && DListNode01.pstPrev == &DList) {
        dprintf("Add DListNode01 success \n");
    }

    LOS_ListTailInsert(&DList, &DListNode02);
    if (DListNode02.pstNext == &DList && DListNode02.pstPrev == &DListNode01) {
        dprintf("Tail insert DListNode02 success \n");
    }

    LOS_ListHeadInsert(&DListNode02, &DListNode03);
    if (DListNode03.pstNext == &DList && DListNode03.pstPrev == &DListNode02) {
        dprintf("Head insert DListNode03 success \n");
    }
}
```



```
    LOS_ListDelInit(&DLlistNode03);
    LOS_ListDelete(&DLlistNode01);
    LOS_ListDelete(&DLlistNode02);

    if (LOS_ListEmpty(&DLlist)) {
        dprintf("Delete success \n");
    }

    return LOS_OK;
}

#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */
```

结果验证

编译运行得到的结果为：

```
Initial head
Add DLlistNode01 success
Tail insert DLlistNode02 success
Head insert DLlistNode03 success
Delete success
```

3.14.3 原子操作

3.14.3.1 概述

基本概念

在支持多任务的操作系统中，修改一块内存区域的数据需要“读取-修改-写入”三个步骤。然而同一内存区域的数据可能同时被多个任务访问，如果在修改数据的过程中被其他任务打断，就会造成该操作的执行结果无法预知。

使用开关中断的方法固然可以保证多任务执行结果符合预期，但这种方法显然会影响系统性能。

ARMv6架构引入了LDREX和STREX指令，以支持对共享存储器更缜密的非阻塞同步。由此实现的原子操作能确保对同一数据的“读取-修改-写入”操作在它的执行期间不会被打断，即操作的原子性。

运作机制

Huawei LiteOS通过对ARMv6架构中的LDREX和STREX进行封装，向用户提供了一套原子操作接口。

- **LDREX Rx, [Ry]**

读取内存中的值，并标记对该段内存为独占访问：

- 读取寄存器Ry指向的4字节内存数据，保存到Rx寄存器中。
- 对Ry指向的内存区域添加独占访问标记。

- **STREX Rf, Rx, [Ry]**

检查内存是否有独占访问标记，如果有则更新内存值并清空标记，否则不更新内存：

- 有独占访问标记
 - i. 将寄存器Rx中的值更新到寄存器Ry指向的内存。
 - ii. 标志寄存器Rf置为0。
- 没有独占访问标记
 - i. 不更新内存。
 - ii. 标志寄存器Rf置为1。
- 判断标志寄存器
 - 标志寄存器为0时，退出循环，原子操作结束。
 - 标志寄存器为1时，继续循环，重新进行原子操作。

3.14.3.2 开发指导

使用场景

有多个任务对同一个内存数据进行加减或交换操作时，使用原子操作保证结果的可预知性。

功能

Huawei LiteOS的原子数据包含两种类型Atomic（有符号32位数）与 Atomic64（有符号64位数）。原子操作模块为用户提供下面几种功能，接口详细信息可以查看API参考。

表 3-3 Atomic 原子操作功能列表

功能分类	接口名	描述	注意事项
读	LOS_AtomicRead	读取内存数据	由调用者保证入参合法
写	LOS_AtomicSet	写入内存数据	由调用者保证入参合法
加	LOS_AtomicAdd	对内存数据做加法	由调用者保证入参合法
	LOS_AtomicSub	对内存数据做减法	由调用者保证入参合法
	LOS_AtomicInc	对内存数据加1	由调用者保证入参合法
	LOS_AtomicIncRet	对内存数据加1并返回运算结果	由调用者保证入参合法
减	LOS_AtomicDec	对内存数据减1	由调用者保证入参合法
	LOS_AtomicDecRet	对内存数据减1并返回运算结果	由调用者保证入参合法

功能分类	接口名	描述	注意事项
交换	LOS_AtomicXchg32bits	交换内存数据，原内存中的值以返回值的方式返回	由调用者保证入参合法
	LOS_AtomicCmpXchg32bits	比较并交换内存数据，返回比较结果	由调用者保证入参合法

表 3-4 Atomic64 原子操作功能列表

功能分类	接口名	描述	注意事项
读	LOS_Atomic64Read	读取64位内存数据	由调用者保证入参合法
写	LOS_Atomic64Set	写入64位内存数据	由调用者保证入参合法
加	LOS_Atomic64Add	对64位内存数据做加法	由调用者保证入参合法
	LOS_Atomic64Sub	对64位内存数据做减法	由调用者保证入参合法
	LOS_Atomic64Inc	对64位内存数据加1	由调用者保证入参合法
	LOS_Atomic64IncRet	对64位内存数据加1并返回运算结果	由调用者保证入参合法
减	LOS_Atomic64Dec	对64位内存数据减1	由调用者保证入参合法
	LOS_Atomic64DecRet	对64位内存数据减1并返回运算结果	由调用者保证入参合法
交换	LOS_AtomicXchg64bits	交换64位内存数据，原内存中的值以返回值的方式返回	由调用者保证入参合法
	LOS_AtomicCmpXchg64bits	比较并交换64位内存数据，返回比较结果	由调用者保证入参合法

须知

原子操作中，操作数及其结果不能超过函数所支持位数的最大值。

平台差异性

无。

3.14.3.3 注意事项

- 需注意函数入参的合法性由调用者保证。
- 目前原子操作接口只支持整型数据。
- 原子操作只能保证操作的原子性，不能保证操作不会出现翻转溢出。

3.14.3.4 编程实例

实例描述

调用原子操作相关接口，观察结果：

步骤1 创建两个任务。

1. 任务一用LOS_AtomicInc对全局变量加100次。
2. 任务二用LOS_AtomicDec对全局变量减100次。

步骤2 子任务结束后在主任务中打印全局变量的值。

----结束

编程示例

```
#include "los_hwi.h"
#include "los_atomic.h"
#include "los_task.h"

UINT32 g_testTaskId01;
UINT32 g_testTaskId02;
Atomic g_sum;
Atomic g_count;

UINT32 Example_Atomic01(VOID)
{
    int i = 0;
    for(i = 0; i < 100; ++i) {
        LOS_AtomicInc(&g_sum);
    }

    LOS_AtomicInc(&g_count);
    return LOS_OK;
}

UINT32 Example_Atomic02(VOID)
{
    int i = 0;
    for(i = 0; i < 100; ++i) {
        LOS_AtomicDec(&g_sum);
    }

    LOS_AtomicInc(&g_count);
    return LOS_OK;
}

UINT32 Example_TaskEntry(VOID)
{
    TSK_INIT_PARAM_S stTask1={0};
    stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Atomic01;
    stTask1.pcName       = "TestAtomicTsk1";
    stTask1.uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    stTask1.usTaskPrio   = 4;
    stTask1.uwResved     = LOS_TASK_STATUS_DETACHED;
```

```
TSK_INIT_PARAM_S stTask2={0};
stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Atomic02;
stTask2.pcName      = "TestAtomicTsk2";
stTask2.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
stTask2.usTaskPrio  = 4;
stTask2.uwResved    = LOS_TASK_STATUS_DETACHED;

LOS_TaskLock();
LOS_TaskCreate(&g_testTaskId01, &stTask1);
LOS_TaskCreate(&g_testTaskId02, &stTask2);
LOS_TaskUnlock();

while(LOS_AtomicRead(&g_count) != 2);
dprintf("g_sum = %d\n", g_sum);

return LOS_OK;
}
```

结果验证

```
g_sum = 0
```

完整实例代码

```
sample_atomic.c
```

3.14.4 位操作

3.14.4.1 概述

基本概念

位操作是指对二进制数的bit位进行操作。程序可以设置某一变量为状态字，状态字中的每一bit位（标志位）可以具有自定义的含义。

3.14.4.2 开发指导

使用场景

系统提供标志位的置1和清0操作，可以改变标志位的内容，同时还提供获取状态字中标志位为1的最高位和最低位的功能。用户也可以对系统的寄存器进行位操作。

功能

Huawei LiteOS的位操作模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
置1/清0标志位	LOS_BitmapSet	对状态字的某一标志位进行置1操作
	LOS_BitmapClr	对状态字的某一标志位进行清0操作
获取标志位为1的bit位	LOS_HighBitGet	获取状态字中为1的最高位
	LOS_LowBitGet	获取状态字中为1的最低位

3.14.4.3 注意事项

无。

3.14.4.4 编程实例

实例描述

对数据实现位操作，本实例实现如下功能：

1. 某一标志位置1。
2. 获取标志位为1的最高bit位。
3. 某一标志位清0。
4. 获取标志位为1的最低bit位。

编程示例

```
#include "los_bitmap.h"
#include "los_printf.h"

static UINT32 Bit_Sample(VOID)
{
    UINT32 flag = 0x10101010;
    UINT16 pos;

    dprintf("\nBitmap Sample!\n");
    dprintf("The flag is 0x%8x\n", flag);

    pos = 8;
    LOS_BitmapSet(&flag, pos);
    dprintf("LOS_BitmapSet:\t pos : %d, the flag is 0x%0+8x\n", pos, flag);

    pos = LOS_HighBitGet(flag);
    dprintf("LOS_HighBitGet:\t The highest one bit is %d, the flag is 0x%0+8x\n", pos, flag);

    LOS_BitmapClr(&flag, pos);
    dprintf("LOS_BitmapClr:\t pos : %d, the flag is 0x%0+8x\n", pos, flag);

    pos = LOS_LowBitGet(flag);
    dprintf("LOS_LowBitGet:\t The lowest one bit is %d, the flag is 0x%0+8x\n\n", pos, flag);

    return LOS_OK;
}
```

结果验证

编译运行得到结果：

```
Bitmap Sample!
The flag is 0x10101010
LOS_BitmapSet: pos : 8, the flag is 0x10101110
LOS_HighBitGet:The highest one bit is 28, the flag is 0x10101110
LOS_BitmapClr: pos : 28, the flag is 0x00101110
LOS_LowBitGet: The lowest one bit is 4, the flag is 0x00101110
```

3.14.5 环形缓冲（ringbuf）

3.14.5.1 概述

基本概念

环形缓冲（ringbuf）作为数据的环形缓冲机制，可以实现数据流以先入先出的方式进行缓存。

ringbuf 控制块

```
typedef struct {  
    UINT32 startIdx;           /* ringbuf读下标 */  
    UINT32 endIdx;            /* ringbuf写下标 */  
    UINT32 size;               /* ringbuf总大小，单位为字节 */  
    UINT32 remain;             /* ringbuf剩余可用大小，单位为字节 */  
    SPIN_LOCK_S lock;          /* ringbuf自旋锁 */  
    RingbufStatus status       /* ringbuf状态 */  
    RingbufFlag flag;          /* Ringbuf flag */  
    CHAR *fifo;                /* ringbuf缓冲区起始地址 */  
} Ringbuf;
```

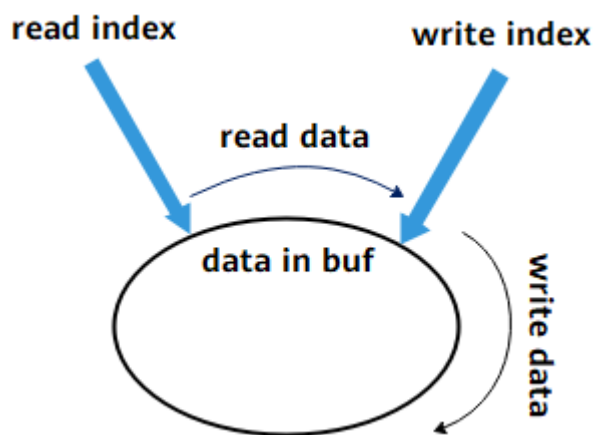
每个ringbuf有两种状态：

- RBUF_UNINIT：ringbuf未初始化。
- RBUF_INITED：ringbuf已初始化。

每个ringbuf有两种模式：

- RBUF_NORMAL：ringbuf正常模式。
- RBUF_OVERWRITE：ringbuf可覆盖写模式。

运作原理



ringbuf控制块中维护着缓冲区的起始地址，和读/写指针的相对偏移。startIdx是读指针相对于ringbuf缓冲区起始地址的偏移，endIdx是写指针的相对偏移。startIdx与endIdx之间为缓冲的数据。ringbuf刚创建时，读写指针偏移均为0。

写入数据到ringbuf时，先判断ringbuf标志。标志为RBUF_NORMAL时，处于正常模式；标志为RBUF_OVERWRITE时，处于覆盖写模式。

正常模式：写入数据到ringbuf时，先判断ringbuf是否还有剩余可用内存。当还有可用内存时才会继续写入，endIdx增长，并减少ringbuf的剩余可用内存；否则丢弃写入数据，可根据LOS_RingbufWrite()的返回值判断写入的数据量。如果ringbuf已经存满数据，因为不会覆盖旧数据，所以无法继续写入。

可覆盖写模式：写入数据到ringbuf时，先判断ringbuf要写入的数据。当要写入数据大于size时，清空ringbuf，仅写入size长度数据；当要写入数据小于size且大于剩余内存remain时，清理一部分最早缓存的数据，再写入数据；当要写入数据小于剩余内存remain时，直接写入；最后均返回实际写入长度。

读取数据时，会按照先入先出的方式，读取最早缓存的数据，startIdx增长，并增加ringbuf的剩余可用内存。读取数据后，因为读指针向后移动，所以无法再次读取相同的数据。

endIdx增长到等于startIdx时，表示ringbuf已经写满，所以无法继续写入，需要读取数据才能继续写数据到ringbuf。

3.14.5.2 开发指导

使用场景

ringbuf用于数据缓冲，遵循先入先出的原则。

功能

Huawei LiteOS中的ringbuf模块提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
初始化/重置环形缓冲	LOS_RingbufInit	初始化环形缓冲区，缓冲区内内存、大小和标志通过入参传入
	LOS_RingbufReset	重置缓冲区的状态，读写偏移重置为0，且清除缓冲区的数据
读/写环形缓冲	LOS_RingbufRead	从环形缓冲区中读取指定大小的数据到指定地址空间，并返回读取到的数据量
	LOS_RingbufPeek	从环形缓冲区中预读指定大小的数据到指定地址空间，并返回读取到的数据量
	LOS_RingbufWrite	写入指定大小的数据到环形缓冲区，并返回成功写入到缓冲区的数据量
获取缓冲数据量	LOS_RingbufUsedSize	获取环形缓冲区中已缓存的数据量

开发流程

使用ringbuf模块开发的典型流程如下：

1. 打开菜单，进入Kernel ---> Enable Ringbuf菜单，完成ringbuf模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_RINGBUF	ringbuf裁剪开关	YES/NO	NO	无

2. 为ringbuf申请一块内存;
3. 通过LOS_RingbufInit初始化ringbuf。
4. 写ringbuf。
5. 在合适时机读ringbuf。

3.14.5.3 注意事项

- ringbuf写接口行为由LOS_RingbufInit接口在初始化时决定;
- 通过LOS_RingbufRead接口读取缓冲区的数据后, 该数据无法被再次读取;
- LOS_RingbufPeek接口仅预读缓冲区数据, 未消费该数据, 该数据仍然可以被读取;
- 正常模式下, 缓冲区写满时, 无法继续写入, 请检查LOS_RingbufWrite的返回值以确认实际写入的数据量;
- 使用完ringbuf后, 需要用户自己释放申请的内存。

3.14.5.4 编程实例

实例描述

创建一个ringbuf, 两个任务。任务1定时写入数据到ringbuf, 任务2在ringbuf数据达到一定量时, 取出数据。

1. 锁任务调度, 通过LOS_TaskCreate创建任务1和任务2;
2. 申请一块内存用作环形缓冲区;
3. 通过LOS_RingbufInit初始化一个缓冲区;
4. 解锁任务调度;
5. 任务1往环形缓冲区中写入数据;
6. 任务2判断数据达到水线后, 读取数据;

编程示例

```
static Ringbuf g_Ringbuf = {0};
#define READ_TRIGGER_THRESHOLD 32
#define RINGBUF_SIZE 128

VOID WriteTask(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    CHAR data[READ_TRIGGER_THRESHOLD * 2 + 1] = "LiteOS test data, send in WriteTask, receive & check in ReadTask";
    for (;;) {
        ret = LOS_RingbufWrite(&g_Ringbuf, data + i, READ_TRIGGER_THRESHOLD / 2);
        if (ret != READ_TRIGGER_THRESHOLD / 2) {
            PRINT_ERR("real write count is %u\n", ret);
        }
        i += ret;
        if (i >= strlen(data)) {
            break;
        } else {
            LOS_Msleep(1000); /* 1000: sleep 1000ms */
        }
    }
}
```

```

VOID ReadTask(VOID)
{
    UINT32 i = 0;
    UINT32 ret = 0;
    UINT32 count = 0;
    CHAR data[READ_TRIGGER_THRESHOLD * 2 + 1] = {0};
    for (;;) {
        count = LOS_RingbufUsedSize(&g_Ringbuf);
        if (count >= READ_TRIGGER_THRESHOLD) {
            ret = LOS_RingbufRead(&g_Ringbuf, data + i, count);
            i += ret;
            PRINTK("read data count %u \n", i);
            if (i == READ_TRIGGER_THRESHOLD * 2) {
                PRINTK("data receive done: %s\n", data);
                break;
            }
        } else {
            PRINTK("info: data count %u < %u\n", count, READ_TRIGGER_THRESHOLD);
        }
        LOS_Msleep(1000); /* 1000: sleep 1000ms */
    }
}

UINT32 SampleTaskCreat(VOID)
{
    UINT32 ret = 0;
    UINT32 task1, task2;
    CHAR *buff = NULL;
    TSK_INIT_PARAM_S initParam;

    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)WriteTask;
    initParam.usTaskPrio = LOSCFG_BASE_CORE_TSK_DEFAULT_PRIO;
    initParam.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    initParam.pcName = "WriteTask";
#ifdef LOSCFG_KERNEL_SMP
    initParam.usCpuAffiMask = CPUID_TO_AFFI_MASK(ArchCurrCpuId());
#endif
    initParam.uwResved = LOS_TASK_STATUS_DETACHED;

    LOS_TaskLock();
    ret = LOS_TaskCreate(&task1, &initParam);
    if (ret != LOS_OK) {
        PRINT_ERR("create WriteTask failed, error: %x\n", ret);
        return ret;
    }

    initParam.pcName = "ReadTask";
    initParam.pfnTaskEntry = (TSK_ENTRY_FUNC)ReadTask;
    ret = LOS_TaskCreate(&task2, &initParam);
    if (ret != LOS_OK) {
        PRINT_ERR("create ReadTask failed, error: %x\n", ret);
        return ret;
    }

    buff = (CHAR *)LOS_MemAlloc(m_aucSysMem0, RINGBUF_SIZE);
    if (buff == NULL) {
        PRINT_ERR("malloc failed");
        return LOS_NOK;
    }
    ret = LOS_RingbufInit(&g_Ringbuf, buff, RINGBUF_SIZE, RBUF_NORMAL);
    if (ret != LOS_OK) {
        PRINT_ERR("ringbuf init failed");
        return LOS_NOK;
    }

    PRINTK("create the ringbuf success!\n");
    LOS_TaskUnlock();
    return ret;
}

```

结果验证

```
create the ringbuf success!
info: data count 16 < 32
read data count 32
info: data count 16 < 32
read data count 64
data receive done: LiteOS test data, send in WriteTask, receive & check in ReadTask
```

完整实例代码

```
sample_ringbuf.c
```

3.14.6 通用内存访问

3.14.6.1 概述

基本概念

当寄存器或内存位于内存空间时，称为IO内存。Huawei LiteOS提供了一组通用接口来操作I/O内存。

3.14.6.2 开发指导

使用场景

系统提供读写（1，2，4，8）字节寄存器操作，方便编写驱动代码。

功能

Huawei LiteOS为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
读/写64位数据	WRITE64(addr, value)	将64位数据(value)写入64位寄存器(addr)
	READ64(addr, value)	读取64位寄存器(addr)
	WRITE64_MB(addr, value)	将64位数据(value)写入64位寄存器(addr)，写入前使用数据同步屏障同步
	READ64_MB(addr, value)	读取64位寄存器(addr)，读取前使用数据同步屏障同步
读/写32位数据	WRITE32(addr, value)	将32位数据(value)写入32位寄存器(addr)
	READ32(addr, value)	读取32位寄存器(addr)
	WRITE32_MB(addr, value)	将32位数据(value)写入32位寄存器(addr)，写入前使用数据同步屏障同步

功能分类	接口名	描述
	READ32_MB(addr, value)	读取32位寄存器(addr)，读取前使用数据同步屏障同步
读/写16位数据	WRITE16(addr, value)	将16位数据(value)写入16位寄存器(addr)
	READ16(addr, value)	读取16位寄存器(addr)
	WRITE16_MB(addr, value)	将16位数据(value)写入16位寄存器(addr)，写入前使用数据同步屏障同步
	READ16_MB(addr, value)	读取16位寄存器(addr)，读取前使用数据同步屏障同步
读/写8位数据	WRITE8(addr, value)	将8位数据(value)写入8位寄存器(addr)
	READ8(addr, value)	读取8位寄存器(addr)
	WRITE8_MB(addr, value)	将8位数据(value)写入8位寄存器(addr)，写入前使用数据同步屏障同步
	READ8_MB(addr, value)	读取8位寄存器(addr)，读取前使用数据同步屏障同步

- 32位ARM架构写64位寄存器如果通过str指令访问会导致高32位寄存器地址和低32位寄存器地址的数据不能同时生效，因此通过strd指令实现。

3.14.6.3 注意事项

无

4 扩展内核

- [4.1 动态加载](#)
- [4.2 分散加载](#)
- [4.3 低功耗框架](#)
- [4.4 访问权限管理](#)
- [4.5 CPU占用率](#)
- [4.6 Trace](#)
- [4.7 LMS](#)
- [4.8 Perf](#)
- [4.9 C++支持](#)
- [4.10 动态共享内存](#)
- [4.11 核间通信](#)

4.1 动态加载

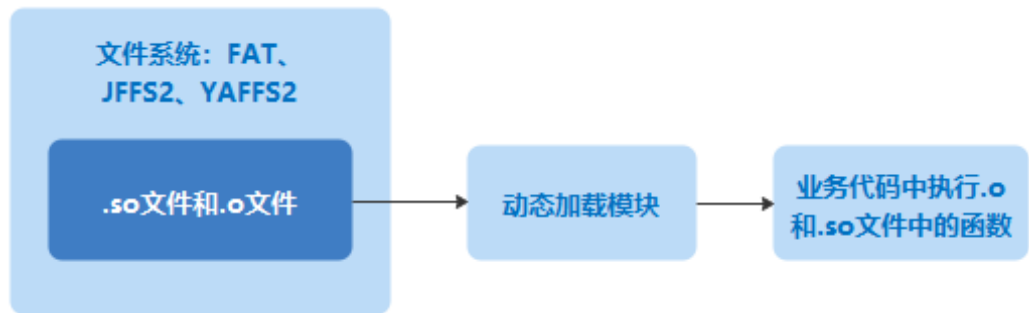
4.1.1 概述

基本概念

动态加载是一种程序加载技术。

静态链接是在链接阶段将程序各模块文件链接成一个完整的可执行文件，运行时作为整体一次性加载进内存。动态加载允许用户将程序各模块编译成独立的.o文件和.so文件，而不将它们链接成一个可执行文件，在需要使用某模块时再动态地将其加载进内存。

Huawei LiteOS支持动态加载OBJ目标文件（.o文件）和SO共享目标文件（.so文件），需要目标文件、系统二进制镜像文件配合使用。



动态加载相关概念

符号表

符号表在表现形式上是记录了符号名及其所在内存地址的数组。符号表在动态加载模块初始化时被载入到动态加载模块的符号管理结构中。在加载用户模块进行符号重定位时，动态加载模块通过查找符号管理结构得到相应符号所在地址，对相应符号进行重定位。

运作机制

对于so文件，可能以ZIP（压缩）或者NOZIP（非压缩）的形式存放在存储介质中，相应的有两种加载策略。

- NOZIP策略：普通的so文件可以通过多次lseek与read获取相应的文件信息。首先根据so文件的 SegmentHeader信息，计算出对应的可Load段的大小，然后分配出相应的空间，将相应的可Load段加载到内存并完成动态加载工作。因为每次仅仅读取文件的一小部分，不存在内存浪费问题。
- ZIP策略：读取ZIP压缩文件时，必须一次性将文件全部读取到内存中（mem1），但是第一次读取时并不知道对应的so文件中可Load段的大小，所以需要二次分配内存（mem2）。因为mem2中已经具备了本次动态加载所需的所有信息，如果让mem1和mem2并存就会造成内存浪费并导致内存峰值过高。解决办法是分两次读取ZIP文件，第一次读取的目的仅仅是计算最终所需的内存大小（计算完毕，立刻释放相应内存），第二次读取的目的才是根据相应信息完成动态加载工作。可见，ZIP加载策略是以牺牲指令为代价来避免内存峰值过高的一种策略。

4.1.2 开发指导

使用场景

静态链接将程序各模块文件链接成一个整体，运行时一次性加载进内存，具有代码装载速度快等优点。但当程序规模较大，模块变更升级较为频繁时，会存在内存和磁盘空间浪费、模块更新困难等问题。

动态加载技术可以较好地解决上述静态链接中存在的问题，在程序需要执行外部模块中的代码时，动态地将外部模块加载进内存，不需要该模块时再卸载，可以实现公共代码的共享以及模块的平滑升级等功能。

功能

Huawei LiteOS 的动态加载模块为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
销毁动态加载	LOS_LdDestroy	销毁动态加载模块
动态加载模块文件	LOS_SoLoad	动态加载一个so模块（依赖文件系统）
	LOS_MemLoad	动态加载一个so或obj模块（不依赖文件系统，从指定内存空间加载）
	LOS_ObjLoad	动态加载一个obj模块（依赖文件系统）
查找符号地址	LOS_FindSymByName	在模块或系统符号表中查找符号地址
卸载模块	LOS_ModuleUnload	卸载一个模块
设置动态加载的搜索路径/参数/内存池地址	LOS_PathAdd	添加模块的搜索路径
	LOS_DynParamReg	设置so模块的动态加载参数，使用该接口需要打开LOSCFG_DYNLOAD_DYN_FROM_FS宏开关，即只支持从文件系统中加载so模块时才能设置其动态加载参数
	LOS_DynMemPoolSet	设置动态加载使用的内存池地址

📖 说明

LOS_DynMemPoolSet接口入参必须是经过LOS_MemInit初始化的内存池地址，即通过Huawei LiteOS内存管理算法管理，并且保证该内存池与系统内存池不重合。该接口需在加载so或者.o文件前使用。

开发流程

动态加载主要有以下几个步骤：

1. [准备编译环境](#)
2. [编译待加载的模块文件](#)
3. [编写动态加载的业务代码](#)
4. [编译系统镜像](#)
5. [准备系统环境](#)

准备编译环境

步骤1 添加.o和.so模块编译选项。

- .o模块的编译选项中需要添加-nostdlib -fno-PIC选项
- .so模块的编译选项中需要添加-nostdlib -fPIC -shared选项

 说明

以下列出的编译选项，可以根据实际需要自行选择。

-z max-page-size=value

设置.o和.so模块可加载的program segment的对齐参数为value值，添加该选项，可以有效减少各相邻可加载的segment的虚拟地址之间由于对齐需要而产生的空白区域。如果不添加该选项，默认对齐参数为0x10000。

须知

- IPC的动态加载需要用户保证所提供的模块文件中所有LD_SHT_PROGBITS、LD_SHT_NOBITS类型的section起始地址都是4字节对齐，否则拒绝加载该模块。
- 生成.o和.so模块时禁止链接编译器中的标准库（即不允许使用-ldl，-lpthread，-lc等），且务必使用编译选项 -nostdlib。

.o和.so模块编译选项添加示例如下：

```
RM = -rm -rf
CC = arm-himix100-linux-gcc

SRCS = $(wildcard *.c)
OBS = $(patsubst %.c,%.o,$(SRCS))
SOS = $(patsubst %.c,%.so,$(SRCS))

all: $(SOS)

$(OBS): %.o : %.c
    @$(CC) -nostdlib -c $< -fno-PIC -o $@

$(SOS): %.so : %.c
    @$(CC) -nostdlib $< -fPIC -shared -o $@

clean:
    @$(RM) $(SOS) $(OBS)

.PHONY: all clean
```

步骤2 修改系统的Makefile。

编译系统镜像的Makefile必须include根目录下的**config.mk**文件，并使用config.mk中的**LITEOS_CFLAGS**或**LITEOS_CXXFLAGS**编译选项，示例如下：

```
LITEOSTOPDIR ?= ../..
SAMPLE_OUT = .

include $(LITEOSTOPDIR)/config.mk
RM = -rm -rf

LITEOS_LIBDEPS := --start-group $(LITEOS_LIBDEP) --end-group

SRCS = $(wildcard sample.c)

OBS = $(patsubst %.c,$(SAMPLE_OUT)/%.o,$(SRCS))

all: $(OBS)

clean:
    @$(RM) *.o sample *.bin *.map *.asm
```



```
$(OBS): $(SAMPLE_OUT)/%.o : %.c
$(CC) $(LITEOS_CFLAGS) -c $< -o $@

$(LD) $(LITEOS_LDFLAGS) -uinit_jffspar_param --gc-sections -Map=$(SAMPLE_OUT)/sample.map -o $
$(SAMPLE_OUT)/sample ./ $@ $(LITEOS_LIBDEPS) $(LITEOS_TABLES_LDFLAGS) $(LITEOS_DYNLDFLAGS)
$(OBJCOPY) -O binary $(SAMPLE_OUT)/sample $(SAMPLE_OUT)/sample.bin
$(OBJDUMP) -d $(SAMPLE_OUT)/sample >$(SAMPLE_OUT)/sample.asm
```

----结束

编译待加载的模块文件

请严格按如下步骤进行编译。以makefile为例介绍。

步骤1 编译.o和.so模块，并将运行所需的所有.o和.so文件拷贝到同一目录下。

说明

1. 如果a.so需要调用b.so中的函数，或者a.so引用到了b.so中的数据，则称a.so依赖b.so。
2. 当a.so依赖b.so，且需要在加载a.so时自动将b.so加载进系统，则在编译a.so时需要将b.so作为编译参数，否则需要在加载a.so之前事先加载b.so。

步骤2 进入Huawei_LiteOS/build/scripts/dynload_tools目录执行sym.sh脚本，示例如下：

```
$ ./sym.sh /home/wmin/customer/out out/dynload_sym
```

说明

- sym.sh脚本的入参/home/wmin/customer/out，是.o和.so文件所在的目录的绝对路径。out/dynload_sym为指定los_dynload_gsymbol.c生成目录的绝对路径，建议指定路径为工程的编译生成目录。
- 注意如果所需加载的.o或.so被更新了，需要重新执行该脚本。该脚本会提取.o和.so文件中的所有系统符号（非用户定义的符号），以便在编译系统镜像时由编译器计算出对应符号的地址。
- 必须要在Huawei_LiteOS/build/scripts/dynload_tools目录下执行该脚本。
- 执行完sym.sh脚本后，确保out/dynload_sym目录下生成了los_dynload_gsymbol.c，并且保证该文件在编译阶段参与编译，以及生成的目标文件参与链接。否则，在加载.o和.so过程中会出现符号不能定位的问题。

----结束

编写动态加载的业务代码

步骤1 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Dynamic Load Feature菜单，完成动态加载模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_DYNLOAD	动态加载模块的裁剪开关	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_KERNEL_DYNLOAD_DYN	使能so文件加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD
LOSCFG_DYNLOAD_DYN_FROM_FS	使能so文件从文件系统加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_DYN

配置项	含义	取值范围	默认值	依赖
LOSCFG_DYNLOAD_DYNFROM_MEM	使能so文件从指定内存中加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_DYNAMIC
LOSCFG_KERNEL_DYNLOAD_REL	使能obj文件加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD
LOSCFG_DYNLOAD_REL_FROM_FS	使能obj文件从文件系统加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_REL
LOSCFG_DYNLOAD_REL_FROM_MEM	使能obj文件从指定内存中加载	YES/NO	YES	LOSCFG_KERNEL_DYNLOAD_REL
LOSCFG_KERNEL_DYN_HEAP_SIZE	配置动态加载使用的堆大小，单位为M	[0, OS_SYS_MEMORY_SIZE)	2	LOSCFG_KERNEL_NX && LOSCFG_KERNEL_DYNLOAD，且依赖Cortex-A芯片

须知

如果同时使能LOSCFG_KERNEL_NX（数据段不可执行，依赖Cortex-A芯片，其在menuconfig中对应的菜单项为：Kernel ---> Enable Data Sec NX Feature）和动态加载模块，则只支持动态加载so文件，其他形式的加载可能会有异常，同时需要配置动态加载使用的堆大小（位于系统动态内存池的尾部）。

步骤2 设置动态加载使用的内存池地址（可选）。

调用LOS_DynMemPoolSet接口可以设置动态加载使用的内存池。如果不设置，默认使用系统内存池。

步骤3 设置so文件的动态加载策略。

在不同的应用场景下，so文件可能以ZIP或者NOZIP的形式存放于存储介质中。如果需从文件系统中加载so文件，由于压缩文件与非压缩文件读写操作的差异性，在初始化动态加载模块之前需要指明具体的加载策略。

```
DYNLOAD_PARAM_S dynloadParam = {ZIP}; // 设置ZIP或NOZIP策略
LOS_DynParamReg(&dynloadParam);      // 设置具体的加载策略
```

说明

以ZIP格式存储的so文件必须采用ZIP加载策略，而普通的so文件使用上述两种策略都可以加载成功，建议使用NOZIP策略。如果没有设置，默认采用NOZIP加载策略。

步骤4 使用相对路径（可选）。

如果在动态加载模块时想使用相对路径，可以通过LOS_PathAdd接口添加so和.o文件所在的绝对路径：

```
ret = LOS_PathAdd("/yaffs/bin/dynload");
if (ret != LOS_OK) {
    printf("add relative path failed");
    return 1;
}
```

添加路径后，调用LOS_SoLoad、LOS_ObjLoad、LOS_MemLoad接口时传入文件名即可，动态加载会在添加的路径下查找指定文件。

说明

- 只有在调用LOS_PathAdd接口添加路径后，才能在调用动态加载模块接口时使用相对路径。
- 可以多次调用LOS_PathAdd接口添加多个相对路径。
- 如果添加的多个路径下有相同文件名的模块，则在加载模块时按照添加的先后依次在所有路径中查找，且只加载第一个查找到的文件。

步骤5 加载用户模块。

动态加载模块支持加载.o和.so模块。

- 使用LOS_ObjLoad接口动态加载obj文件（注意只能从文件系统中加载obj模块）：

```
if ((handle = LOS_ObjLoad("/yaffs/bin/dynload/foo.o")) == NULL) {
    printf("load module ERROR!!!!!!\n");
    return 1;
}
```

- 使用LOS_SoLoad接口动态加载so文件：

```
if ((handle = LOS_SoLoad("/yaffs/bin/dynload/foo.so")) == NULL) {
    printf("load module ERROR!!!!!!\n");
    return 1;
}
```

说明

对于so文件的动态加载：

- 如果模块A需要模块B，即模块A依赖模块B，如果明确指明了A.so依赖B.so（编译A.so时将B.so作为编译参数），那么加载A模块时会自动将B模块也加载进来。如果没有明确指明A模块与B模块的依赖关系，那么在加载A模块之前，必须保证B模块已经被成功加载。
- 对于不支持文件系统的平台，支持直接从指定内存空间动态加载so，可以调用LOS_MemLoad接口实现。

步骤6 获取用户模块中的符号地址。

- 在特定用户模块中查找符号

需要在某个特定用户模块中查找符号地址时，调用LOS_FindSymByName接口，并将LOS_FindSymByName的第一个参数置为需要查找的用户模块的句柄。

```
if ((magic = LOS_FindSymByName(handle, "os_symbol_table")) == NULL) {
    printf("symbol not found\n");
    return 1;
}
```

- 在全局符号表中查找符号

需要在全局符号表（即OS模块，包括本模块和所有其他用户模块）中查找某个符号地址时，调用LOS_FindSymByName接口，并将LOS_FindSymByName的第一个参数置**NULL**。

```
if ((funTestCase0 = LOS_FindSymByName(NULL, "printf")) == NULL) {
    printf("symbol not found\n");
    return 1;
}
```

步骤7 使用获取到的符号地址。

LOS_FindSymByName返回符号地址（VOID *指针），对该符号地址转换类型后，可以使用该符号。下面针对数据类型符号和函数类型符号举例说明。

- 整数类型符号

现有待加载的test.c，有一全局变量UINT32 g_test = 0，可以通过如下代码获取g_test的地址。

```
const char *g_pscOsOSSymtblFilePath = "/yaffs/bin/dynload/test.so";
UINT32 *g_testPtr = NULL;
INT8 *ptr = (INT8 *)NULL;
if ((pOSSymtblHandler = LOS_SoLoad(g_pscOsOSSymtblFilePath)) == NULL) {
    return LOS_NOK;
}
if ((ptr = LOS_FindSymByName(pOSSymtblHandler, "g_test")) == NULL) {
    printf("g_uwTest not found\n");
    return LOS_NOK;
}
g_testPtr = (UINT32 *)ptr; /* 强制类型转换成真实的指针类型 */
```

- 函数类型符号

foo.c中定义了一个无参的函数test_0和一个有两个参数的函数test_2，编译生成foo.o。

```
foo.c:
int test_0(void)
{
    return 0;
}
int test_2(int i, int j)
{
    return 0;
}
```

以下代码演示在demo.c中获取foo.o模块中的函数并调用。

```
demo.c:
typedef int (* TST_CASE_FUNC)(); /* 无形参函数指针类型声明 */
typedef int (* TST_CASE_FUNC1)(UINT32); /* 单形参函数指针类型声明 */
typedef int (* TST_CASE_FUNC2)(UINT32, UINT32); /* 双形参函数指针类型声明 */

TST_CASE_FUNC funTestCase0 = NULL; /* 函数指针定义 */
TST_CASE_FUNC2 funTestCase2 = NULL;
int ret;

handle = LOS_ObjLoad("/yaffs/bin/dynload/foo.o");
funTestCase0 = LOS_FindSymByName(handle, "test_0");
if (funTestCase0 == NULL) {
    printf("can not find the function name\n");
    return 1;
}
ret = funTestCase0();

funTestCase2 = LOS_FindSymByName(NULL, "test_2");
if (funTestCase2 == NULL){
    printf("can not find the function name\n");
    return 1;
}
ret = funTestCase2(42, 57);
```

步骤8 卸载模块。

调用LOS_ModuleUnload接口卸载某个模块，将需要卸载的模块句柄作为参数传入该接口。对于已被加载过的obj或so文件的句柄，卸载时统一使用LOS_ModuleUnload接口。

```
ret = LOS_ModuleUnload(handle);
if (ret != LOS_OK) {
```

```
printf("unload module failed");  
return 1;  
}
```

步骤9 销毁动态加载模块。

不再需要动态加载功能时，调用LOS_LdDestroy接口，销毁动态加载模块。

须知

- 销毁动态加载模块时会自动卸载所有已被加载的模块，销毁后模块不能再使用。
- 销毁动态加载模块前需确认模块不再使用。

----结束

编译系统镜像

在Huawei LiteOS源码根目录下执行make，编译系统镜像。

编译完成后，在根目录out/平台名的目录下，可以看到生成的系统镜像vs_server.bin文件。

说明

如果待加载的.o和.so中包含了未定义的外部符号（既没有定义在这些.o和.so文件中，也不是一个合法的系统全局符号），在编译系统镜像文件时会提示相应错误，需排查错误信息，确保系统镜像编译正确。

准备系统环境

.so文件（或.o文件）需要和系统镜像文件配合使用。

- 如果选择从文件系统加载模块，则模块文件必须放置在文件系统中，例如jffs2、yaffs、fat等文件系统。
- 如果选择从指定内存空间加载，则忽略下文的步骤2，只需要将模块文件烧写到指定内存空间即可。

建议操作顺序：

步骤1 烧写系统镜像文件到开发板的flash中。

步骤2 将.so文件（或.o文件）存储到开发板的flash中，这里分为两种情况：

- 如果模块文件保存在可热拔插的SD卡设备上，直接将SD卡插到开发板上即可。如果需要更新.so文件（或.o文件），可将SD卡插到电脑上更新。
- 如果模块文件保存在不可热插拔的存储设备上，可通过如下两种方式更新文件：
 - a. 将模块文件编译进文件系统镜像，然后烧写文件系统镜像到开发板的flash中。
 - b. 启动Huawei LiteOS系统后，通过tftp命令下载.so文件（或.o文件），示例命令如下：

```
tftp -g -l /yaffs/bin/dynload/foo.so -r foo.so 10.67.211.235
```

步骤3 启动系统进行验证。

----结束

Shell 调试

在Shell里封装了一系列与动态加载有关的命令，方便用户进行调试。具体的Shell命令详细说明参见[动态加载命令参考](#)。

4.1.3 注意事项

编译选项：

- .o模块的编译选项中需要添加-nostdlib -fno-PIC选项。
- .so模块的编译选项中需要添加-nostdlib -fPIC -shared选项。

安全性：

- 对于加载的模块文件，需要提供者保证它的来源可靠并且安全。强烈建议禁止应用系统从SD卡/U盘等外部高风险介质加载文件，如果确实需要，则由提供者/应用层保证文件的可靠性，如果由此产生损失或者问题，华为不承担任何责任。
- 如果被加载的文件有问题，可能导致包括但不限于设备损坏、数据泄露/被篡改等一系列的问题，对此华为不承担任何责任。

其他限制：

- 在编译系统镜像之前必须确保所需加载的.o与.so已经就绪，这样才能保证后续在编译系统镜像文件时，这些.o和.so所调用到的外部符号信息已经集成到最终的系统镜像中。
- 在加载.o和.so模块时，一旦发现该模块引用的外部符号被重复定义在其它没有依赖关系的模块中时，会拒绝对本次引用做重定位，导致加载该模块失败。为了避免使用异常，用户应该事先确保所有待加载的模块内部没有重复定义的符号（变量或函数接口）。

4.1.4 编程实例

实例描述

示例中，sample_foo.c中定义了一个无参的函数test_0和一个有两个参数的函数test_2，编译生成foo.o。在sample_Dynamic_loading.c中演示获取foo.o模块中的函数并调用。

编程示例

前提条件：在menuconfig菜单中完成动态加载模块的配置。

sample_foo.c代码实现如下：

```
int test_0(void)
{
    printf("test_0\n");
    return 0;
}
int test_2(int i, int j)
{
    printf("test_2: %d %d\n", i, j);
    return 0;
}
```

sample_Dynamic_loading.c代码实现如下：

```
typedef int (* TST_CASE_FUNC)(VOID); /* 无形参函数指针类型声明 */
typedef int (* TST_CASE_FUNC2)(UINT32, UINT32); /* 双形参函数指针类型声明 */
```

```
TST_CASE_FUNC funTestCase0 = NULL;          /* 函数指针定义 */
TST_CASE_FUNC2 funTestCase2 = NULL;
int ret;
void *handle = NULL;
handle = LOS_ObjLoad("/jffs0/bin/dynload/dynload_test.o");
funTestCase0 = LOS_FindSymByName(handle, "test_0");
if (funTestCase0 == NULL) {
    printf("can not find the function name\n");
    return;
}
ret = funTestCase0();                        /* 调用该函数指针 */
funTestCase2 = LOS_FindSymByName(NULL, "test_2");
if (funTestCase2 == NULL){
    printf("can not find the function name\n");
    return;
}
ret = funTestCase2(42, 57);                 /* 调用该函数指针 */
ret = LOS_ModuleUnload(handle);
if (ret != LOS_OK) {
    printf("unload module failed");
    return;
}
LOS_LdDestroy();
```

结果验证

编译运行得到的结果为：

```
Huawei LiteOS#
test_0
test_2: 42 57
```

完整实例代码

sample_foo.c
sample_Dynamic_loading.rar

4.2 分散加载

4.2.1 概述

基本概念

分散加载是一种实现特定代码快速启动的技术，通过优先加载特定代码到内存，缩短从系统开机到特定代码执行的时间，可用于实现关键业务的快速启动。

嵌入式系统通过uboot加载flash上的镜像文件到内存并执行，而镜像文件可能较大，由于flash读取速度的限制，将镜像全部加载完再执行可能无法满足时间敏感的业务对启动速度的要求。

分散加载的思想是先加载部分镜像并执行，这部分镜像包含了时间敏感的关键业务，从而达到快速启动关键业务的效果。

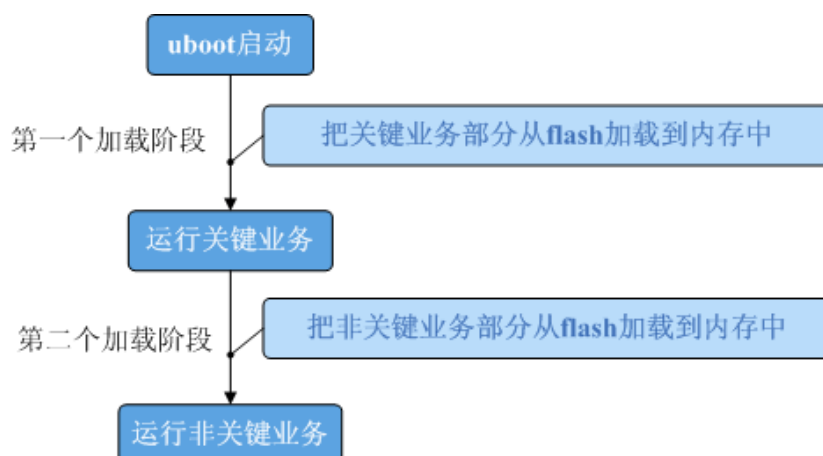
运作机制

Huawei LiteOS的分散加载的主体思想是将部分时间敏感的关键业务提前加载执行，具体手段是通过合理布局镜像，将这些业务相关的数据、代码段布局到镜像文件的前端。在镜像加载时将加载过程分为两个阶段：

- 第一阶段通过uboot将包含时间敏感业务的这段前端镜像加载到内存并执行。
- 第二阶段加载剩余部分镜像到内存继续执行，具体实现为：在第一阶段的代码中调用分散加载接口加载剩余部分镜像并运行镜像。

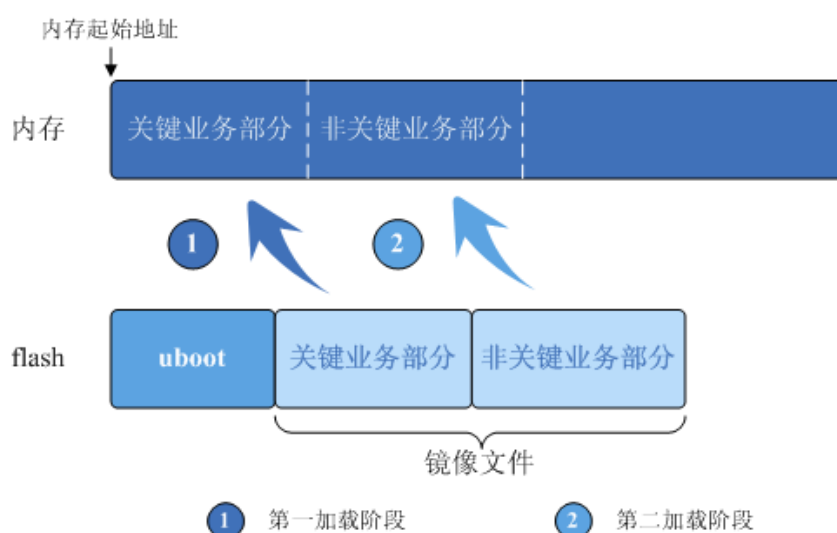
因为第一阶段加载部分镜像的速度会快于加载完整镜像的速度，所以缩短了系统启动到运行时间敏感业务的时间。

图 4-1 分散加载流程图



分散加载的内部原理图如下图2所示，图中的运作顺序可参照上图1的流程说明。

图 4-2 分散加载内部原理图



4.2.2 开发指导

使用场景

应用分散加载技术的典型场景是快速启动对时间敏感的业务。

嵌入式系统中可能存在某些业务对启动时间要求比较高，例如某项目对开机到录制预览的时间要求较高，可以利用分散加载技术实现录制预览业务的快速启动，从开机启动到录制可以在1s内，超越Linux版本的3s-4.5s。

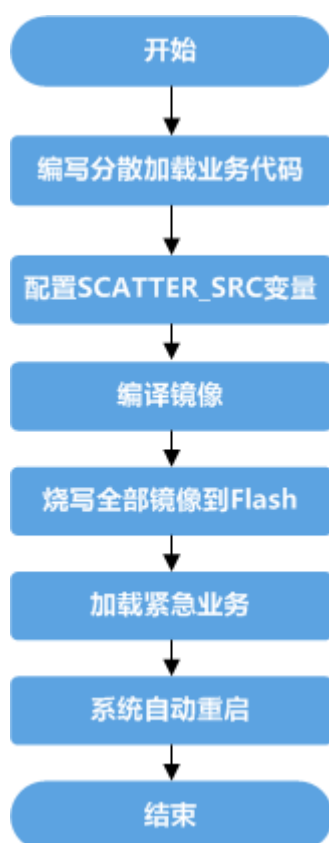
功能

Huawei LiteOS的分散加载模块为用户提供如下功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
分散加载接口	LOS_ScatterLoad	在分散加载第一阶段的代码最后调用此接口，从flash中加载剩余非紧急业务到内存中，参数readAlignSize为flash读取数据时的块对齐大小，应根据flash的实际读写特性设置

开发流程

分散加载的开发流程如下图所示。



以hi3516a平台为例，分散加载使用需要以下几个步骤，以makefile为编译工具介绍：

步骤1 编写分散加载业务代码，调用接口**LOS_ScatterLoad**。

业务代码的入口函数为app_init，该函数位于Huawei LiteOS源码根目录的“targets/(LITEOS_PLATFORM)/os_adapt/os_adapt.c”文件中。在紧急业务代码后调用

LOS_ScatterLoad函数进行分散加载，并通过MAKE_SCATTER_IMAGE宏控制非紧急业务代码的编译，示例代码如下。

```
void app_init() {
    proc_fs_init();
    hi_uartdev_init();
    system_console_init("/dev/uartdev-0");
    LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end,
    BEFORE_SCATTER);
    LOS_ScatterLoad(0x100000, flash_read, NAND_READ_ALIGN_SIZE);

    /* 以下为非紧急业务 */
    #ifndef MAKE_SCATTER_IMAGE
        LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end,
        AFTER_SCATTER);

        extern unsigned int osShellInit(void);
        osShellInit();
        rdk_fs_init();
        SDK_init();

        hi_product_driver_init();
        char *apszArgv[3]={"vs_server","./higv.bin","-i"};
        vs_server(3, apszArgv);
    #endif
}
```

步骤2 配置SCATTER_SRC变量。

在Huawei LiteOS源码根目录的build/mk/wow_scatter.mk中配置**SCATTER_SRC**，将变量定义为调用分散加载LOS_ScatterLoad函数的业务源文件路径，如下所示，其中LITEOSTOPDIR指代Huawei LiteOS源码根目录。

```
SCATTER_SRC := $(LITEOSTOPDIR)/targets/$(LITEOS_PLATFORM)/os_adapt/os_adapt.c
```

步骤3 打开菜单，选择Kernel ---> Enable Extend Kernel ---> Enable Scatter Feature，使能分散加载（相应的宏开关为**LOSCFG_KERNEL_SCATTER**）。

当前Huawei LiteOS的分散加载依赖于Flash，在使能分散加载前需要先使能Flash，菜单路径为：Driver ---> Enable MTD。Nand Flash的宏开关为**LOSCFG_DRIVERS_MTD_NAND**，spi_nor flash的宏开关为**LOSCFG_DRIVERS_MTD_SPI_NOR**。

步骤4 执行make，编译系统镜像。

- 在根目录下执行如下命令，编译全部业务代码。

```
Huawei_LiteOS$ make
```

编译后，命令行界面会返回包含紧急业务的镜像大小信息，如下图所示。

```
#####
#####Calculate the size of scatter#####
the size is 0x4E0000
#####end#####
```

- 编译完成后，检查镜像段的排布，如果镜像中生成了分散加载相关的段则表明分散加载的镜像生成成功。进入系统镜像生成目录（hi3516a平台的镜像生成目录为out/hi3516a，其他类推），可以看到生成的系统ELF文件vs_server，执行**readelf -S vs_server**命令查看section头信息，结果如下图所示。

[15]	.fast_ropdata	PROGBITS	80159000	088868	0a6adc
[16]	.fast_text	PROGBITS	80200000	12f868	27ece0
[17]	.fast_data	PROGBITS	8047f000	3ae868	067b78
[18]	.got	PROGBITS	804e6b78	4163e0	000504
[19]	.text	PROGBITS	804e8000	417868	1b1b90
[20]	.rodata	PROGBITS	8069a000	5c9868	09f4d8
[21]	.data	PROGBITS	8073a000	669868	01cd7c
[22]	.bss	NOBITS	80757000	6865e4	6d74e0

图中显示了与分散加载相关的段信息（包括段的名称、起始地址及偏移大小），其中.fast_ropdata为分散加载镜像的只读数据段，.fast_text为代码段，.fast_data为数据段。

- 查看分散加载链接脚本Huawei_LiteOS/tools/scripts/ld/scatter.ld的.text段，新增了scatter.o(*.text*)，实现了将分散加载的快速启动部分代码相关符号整合到同一个段中，如下图所示。

```
.fast_text ALIGN (0x1000): { __fast_text_start = ABSOLUTE(.); . = .;
scatter.o(*.text*);
}

. = (ABSOLUTE(.) + (0x1000 - 1)) & ~ (0x1000 - 1);
__fast_text_end = ABSOLUTE(.);
```

步骤5 烧写全部镜像到Flash。

进入串口工具界面，可以参考如下命令，将全部镜像烧写到Flash的0x100000地址位。

```
tftp 0x82000000 vs_server.bin;nand erase 0x100000 0x700000;nand write 0x82000000 0x100000 0x700000;
```

其中，vs_server.bin为系统镜像文件名，先将其烧写到内存中高地址位0x82000000。然后烧写到Flash，起始地址为0x100000，烧写长度为0x700000，即烧写的镜像文件大小不能超过7M，可根据实际镜像大小调整长度值。

说明

用户可以根据实际环境自主选择镜像烧写工具及烧写方法。

步骤6 加载紧急业务。

可以参考如下命令，从Flash的0x100000地址处读取包含紧急业务的镜像，加载到内存地址0x80008000处。镜像大小在步骤4执行make后的命令行界面上可以看到，示例中为0x4E0000。

```
nand read 0x80008000 0x100000 0x4E0000;
```

执行如下命令，从内存地址0x80008000处启动系统。

```
go 0x80008000;
```

步骤7 系统自动重启。

系统自动重启，在0x80008000地址处加载镜像。

----结束

4.2.3 注意事项

- 分散加载第一阶段拷贝过少，或者拷贝偏移地址没有根据存储介质的差异进行对齐都会导致系统异常，因此使用时要按照步骤4编译最后给出的大小进行uboot加载镜像。

- 分散加载依赖于存储介质，当前Huawei LiteOS系统依赖Flash，因此该功能依赖Flash，如果关闭Flash支持，分散加载功能也会关闭。如果需要在其他存储介质上使能分散加载，请参考当前Flash的配置进行相应适配。

4.3 低功耗框架

4.3.1 概述

低功耗框架是低功耗模块的统一管理框架，提供对休眠唤醒、系统休眠、Tickless等多种低功耗能力的支持。

低功耗框架默认使能休眠唤醒和Tickless。此外，由于低功耗特性与具体业务和硬件有较强的相关性，为了更好地适配硬件、扩展功能，低功耗框架开放了相关功能接口方便开发者适配。同时，在默认的低功耗框架中也可以自定义具体策略。

表 4-1 接口列表

接口名	描述
LOS_PowerMgrInit	初始化低功耗框架，入参为框架参数指针，可以自定义低功耗策略及具体休眠动作。注意入参不能为空，否则将注册失败。
LOS_LowpowerHookReg	注册低功耗框架休眠入口函数，系统idle时调用该休眠函数

4.3.2 休眠唤醒

4.3.2.1 概述

基本概念

休眠唤醒（RunStop）是一套保存系统现场镜像以及从现场镜像中快速恢复运行的机制。

用户在系统运行的某一时刻调用休眠唤醒接口，可以将系统此时的状态快照（CPU现场及内存）保存到Flash上。系统重启后，可以通过快照恢复系统，使系统从快照保存的状态处继续运行。

运作机制

业务模块初始化并保持稳定运行后，调用休眠唤醒接口保存系统快照，这份快照包含了业务模块所需的所有CPU现场及内存状态。系统运行一段时间后，业务模块空闲时，主核主动下电进入低功耗模式。当有业务需要处理时，MCU发送特定数据包唤醒主核上电，硬件状态恢复后，系统从Flash快照处恢复运行，这样系统重新上电后无需重新初始化业务模块，从而实现业务模块的快速唤醒。

4.3.2.2 开发指导

使用场景

如果希望系统运行一段时间后进入低功耗模式，当有业务到来时，系统能够快速恢复到低功耗模式前的状态，从而快速恢复业务处理能力。对于这种应用场景，可以通过休眠唤醒实现。

在某项目中，休眠唤醒被用于实现芯片的休眠唤醒：系统业务运行稳定后，保存当前系统状态的快照。当系统进入空闲状态时，主核主动下电进入低功耗模式，当业务消息到来时MMU对主核上电，系统从快照处快速启动，恢复执行。以WIFI芯片为例，RunStop可以实现主核在低功耗模式下WIFI状态不断连，以及WIFI状态的快速唤醒。

功能

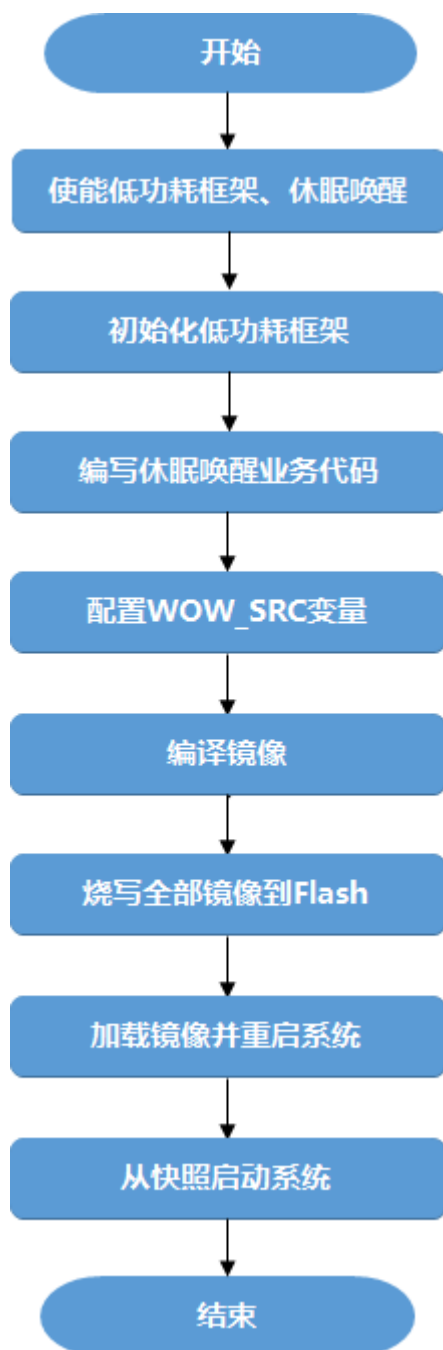
休眠唤醒模块为用户提供如下功能，接口详细信息可以查看API参考。

表 4-2 功能列表

接口名	描述
LOS_MakeImage	将系统状态快照保存到指定介质中

开发流程

图 4-3 休眠唤醒流程图



步骤1 使能低功耗框架和休眠唤醒。

打开菜单，相关配置项的菜单路径为：Kernel ---> Enable Extend Kernel ---> Enable Low Power Management Framework。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_LOWPOWER	低功耗框架裁剪开关	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_KERNEL_RUNSTOP	休眠唤醒开关	YES/NO	NO	休眠唤醒依赖Flash、Uboot和bestfit内存管理算法。Nand Flash的宏开关为LOSCFG_DRIVERS_MTD_NAND，spi_nor flash的宏开关为LOSCFG_DRIVERS_MTD_SPI_NOR

步骤2 初始化低功耗框架。

入口函数main中，在OsStart前调用 LOS_PowerMgrInit(NULL) 初始化低功耗框架为系统默认框架，示例代码如下：

```
#ifndef LOSCFG_KERNEL_LOWPOWER
#include "los_lowpower.h"
const PowerMgrRunOps runOps = {
    .getSleepMode = NULL,
};

UINT32 OsPreSystemConfig(VOID)
{
    PowerMgrParameter param;
    param.runOps = runOps;
    return LOS_PowerMgrInit(&param);
}

#endif
LITE_OS_SEC_TEXT_INIT int main(void)
{
    .....
#ifdef LOSCFG_KERNEL_LOWPOWER
    ret = OsPreSystemConfig();
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
#endif
    OsStart();
    .....
}
```

步骤3 编写快照业务代码。

业务代码入口函数为app_init，该函数位于源码根目录的“targets/board/\$ (LITEOS_PLATFORM)/os_adapt/os_adapt.c”文件中。

在快照业务代码后调用LOS_MakeImage将系统当前的状态快照保存到指定介质中，并通过MAKE_WOW_IMAGE宏控制非快照业务代码的编译，以实现快照镜像和全部镜像的编译，示例代码如下。

```
void wakeup_callback(void)
{
#ifdef LOSCFG_KERNEL_CPUP
    LOS_CpupReset();
#endif
    (VOID)LOS_HwiEnable(83);
}
```

```

}
#define NOR_ERASE_ALIGN_SIZE (64 * 1024)
#define NOR_READ_ALIGN_SIZE 1
#define NOR_WRITE_ALIGN_SIZE 1
int flash_read(void *memaddr, size_t start, size_t size)
{
    return spiflash_read(memaddr, start, size);
}
int flash_write(void *memaddr, size_t start, size_t size)
{
    spiflash_erase(start, size);
    return spiflash_write(memaddr, start, size);
}
void app_init(void)
{
#ifdef LOSCFG_KERNEL_RUNSTOP
    RUNSTOP_PARAM_S stRunstopParam; // 定义一个LOS_MakeImage的参数变量
#endif
#ifdef LOSCFG_DRIVERS_MTD_SPI_NOR
    (void)spinor_init();
    mtd = get_mtd("spinor");
#endif
    if defined(LOSCFG_KERNEL_RUNSTOP) && defined(LOSCFG_DRIVERS_MTD_SPI_NOR)
        memset(&stRunstopParam, 0, sizeof(RUNSTOP_PARAM_S));
    /* LOS_MakeImage参数配置 */
    stRunstopParam.pfFlashReadFunc = flash_read; // 指定介质读函数
    stRunstopParam.pfFlashWriteFunc = flash_write; // 指定介质写函数
    stRunstopParam.pfIdleWakeupCallback = idle_wakeup_callback; // 指定从快照恢复后在idle任务中需要
    执行的回调函数
    stRunstopParam.pfImageDoneCallback = NULL; // 指定快照镜像生成后的回调函数,此处为
    NULL
    stRunstopParam.pfWakeupCallback = wakeup_callback; // 指定从快照恢复后需要执行的回调函
    数
    stRunstopParam.uwFlashEraseAlignSize = NOR_ERASE_ALIGN_SIZE; // 介质的擦除对齐参数
    stRunstopParam.uwFlashWriteAlignSize = NOR_WRITE_ALIGN_SIZE; // 介质的写对齐参数
    stRunstopParam.uwFlashReadAlignSize = NOR_READ_ALIGN_SIZE; // 介质的读对齐参数
    stRunstopParam.uwImageFlashAddr = 0x100000; // 烧写系统全镜像的介质起始地址
    stRunstopParam.uwWowFlashAddr = 0x800000; // 保存快照的介质起始地址
    LOS_MakeImage(&stRunstopParam);
#endif
#ifdef MAKE_WOW_IMAGE
#ifdef LOSCFG_KERNEL_RUNSTOP
    PRINTK("Image length 0x%x\n", OsWowImageSizeGet());
#endif
#endif /* MAKE_WOW_IMAGE */
}

```

步骤4 配置WOW_SRC变量。

在源码根目录build/mk/wow_scatter.mk文件中配置WOW_SRC变量，将变量定义为调用休眠唤醒函数的业务源文件路径，如下所示，其中LITEOSTOPDIR指代Huawei_LiteOS源码根目录。

```
WOW_SRC := $(LITEOSTOPDIR)/targets/board/$(LITEOS_PLATFORM)/os_adapt/os_adapt.c
```

步骤5 编译系统镜像。

1. 在根目录下执行make命令，编译全部业务代码。
Huawei_LiteOS\$ make
2. 编译完成后，查看镜像段的排布，以验证快照镜像是否生成成功。进入系统镜像生成目录（例如hi3556v200平台的镜像生成目录为out/hi3556v200，其他类推），可以看到生成的系统ELF文件vs_server，执行readelf -S vs_server命令查看section头信息，结果如下图所示。

图 4-4 系统 ELF 文件 vs_server readelf 结果

[12]	.wow_rodata	PROGBITS	80008020	0000d8	00b550	00	A	0	0	8
[13]	.wow_text	PROGBITS	80013570	00b628	04bb44	00	AX	0	0	8
[14]	.wow_data	PROGBITS	8005f0b4	05716c	001c3c	00	WA	0	0	8
[15]	.wow_bss	NOBITS	80060cf0	058da8	008e1c	00	WA	0	0	8

图中显示了与休眠唤醒有关的段信息（包括段的名称、起始地址及偏移大小）。其中.wow_rodata为休眠唤醒镜像的只读数据段，.wow_text为代码段，.wow_data为数据段，.wow_bss为bss段。

- 查看休眠唤醒链接脚本Huawei_LiteOS/tools/scripts/ld/wow.ld的.text段，可以看到新增了wow.O(*.text*)，它实现了将休眠唤醒的快照部分代码相关符号整合到同一个段中，如下图所示。

图 4-5 链接脚本.text 段

```
.wow_text ALIGN (0x4): {
    wow.O(*.text*);
}

.wow_data ALIGN (0x4): {
    . = ALIGN (4); KEEP(wow.O( SORT (.liteos.table.*.wow.*))) ; . = ALIGN (4); wow.O(*.data*);
}

__wow_bss_start = ABSOLUTE (.);
```

步骤6 将全部镜像烧写到Flash。

进入串口工具界面，可以参考如下命令，将全部镜像烧写到Flash的0x100000地址位。

```
tftp 0x81000000 vs_server.bin;sf probe 0;sf erase 0x100000 0x300000;sf write 0x81000000 0x100000 0x300000;
```

其中，vs_server.bin为系统镜像文件名，先将其烧写到内存中一段高地址位0x81000000。然后烧写到Flash，起始地址为0x100000，烧写长度为0x300000，即烧写的镜像文件大小不能超过3M，请根据实际的镜像大小调整长度值。

说明

用户可以根据实际环境自主选择镜像烧写工具及烧写方法。

步骤7 加载全部镜像。

可以参考如下命令，从Flash的0x100000地址处读取全部镜像，加载到内存地址0x80200000处。

```
sf probe 0;sf read 0x80200000 0x100000 0x300000;
```

执行如下命令，从内存地址0x80200000处启动系统。

```
go 0x80200000;
```

步骤8 根据系统反馈的快照镜像大小，从快照处恢复运行。

图 4-6 系统启动时的打印信息

```
Relocation Offset is: 0d6d7000
Relocating to 8fd7000, new gd at 8fe36ef0, sp at 8fe36ed0
SPI Nor: Check Flash Memory Controller v100 ... Found
SPI Nor ID Table Version 1.0
SPI Nor(cs 0) ID: 0xc2 0x20 0x18
Block:64KB Chip:16MB Name:"MX25L128XX"
SPI Nor total size: 16MB
NAND: 0 MiB
MMC:
In: serial
Out: serial
Err: serial
Net: eth0
Hit any key to stop autoboot: 2 □□□ 1 □□□ 0
device 0 offset 0x100000, size 0x300000

SF: 3145728 bytes @ 0x100000 Read: OK
## Starting application at 0x80200000 ...

*****Hello Huawei LiteOS*****

Processor : Cortex-A7 * 2
Run Mode : SMP
GIC Rev : GICv2
build time : Aug 1 2019 09:47:25

*****

main core booting up...
osAppInit
releasing 1 secondary cores
cpu 0 entering scheduler
cpu 1 entering scheduler
Spi Nor ID:0xc2 0x20 0x18 0xc2 0x20 0x18 0xc2 0x20
Spi Nor Flash Info:
Name:"MX25L128XX" Size:16MB Block:64KB
Image length 0x170000

Huawei LiteOS # VPP_V300R003C28SPC006 (lwIP) [Aug 1 2019][09:47:40]
###topip_init done####
Ethernet start.hisi_eth: User did not set phy mode, use default=rmi
hisi_eth: User did not set phy addr, auto scan...
Detected phy addr 1, phyid: 0x1cc816
Link is Up - 100Mbps/Full
```

根据系统启动时打印的信息，可以获取休眠唤醒快照镜像大小，如上图红框中所示，本示例中快照大小为0x170000。快照被写到介质的0x800000地址处（从[步骤3编写的快照业务代码](#)中可以获得保存快照的介质的起始地址为0x800000）。

重启系统，可以参考如下命令在uboot中执行，从保存快照的介质的0x800000地址处读取0x170000大小的快照镜像，加载到内存地址0x80200000处，然后执行go 0x80200000从快照启动系统。

```
sf probe 0;sf read 0x80200000 0x800000 0x170000;
go 0x80200000;
```

启动效果如下图：

图 4-7 uboot 执行命令后的打印信息

```
Relocation Offset is: 0d6d7000
Relocating to 8fed7000, new gd at 8fe36ef0, sp at 8fe36ed0
SPI Nor: Check Flash Memory Controller v100 ... Found
SPI Nor ID Table Version 1.0
SPI Nor(cs 0) ID: 0xc2 0x20 0x18
Block:64KB Chip:16MB Name:"MX25L128XX"
SPI Nor total size: 16MB
NAND: 0 MiB
MMC:
In: serial
Out: serial
Err: serial
Net: eth0
Hit any key to stop autoboot: 2 □□□ 1 □□□ 0
device 0 offset 0x800000, size 0x180000
SF: 1572864 bytes @ 0x800000 Read: OK
## Starting application at 0x80200000 ...
Image length 0x170000

Huawei LiteOS # VPP_V300R003C28SPC006 (lwIP) [Aug 1 2019][09:47:40]
###tcpip_init done####
Ethernet start.hisi_eth: User did not set phy mode, use default=rmii
hisi_eth: User did not set phy addr, auto scan...
Detected phy addr 1, phyid: 0x1cc816
Link is Up - 100Mbps/Full
```

系统成功从快照启动。

----结束

4.3.2.3 注意事项

- 用户在编写休眠唤醒业务时，需要保证#ifdef MAKE_WOW_IMAGE前涵盖了快照的所有代码和数据，否则从快照恢复后的系统中不会包含所有休眠唤醒的业务。
- 休眠唤醒依赖于存储介质，当前LiteOS依赖Flash，因此该功能依赖Flash，如果关闭Flash支持，休眠唤醒功能也会关闭。如果需要在其他存储介质上使能休眠唤醒，请参考当前Flash的配置进行适配。

4.3.3 系统休眠

4.3.3.1 概述

基本概念

系统休眠是一种兼顾系统性能和能耗的低功耗机制，用户可通过自行配置休眠策略，在待机场景下调整系统时钟频率、调整外设工作模式或下电等，从而节省功耗，提升续航能力。

系统休眠的低功耗管控主要支持：

1. 睡眠模式。
2. 睡眠投票机制。
3. 系统调频。

低功耗流程注册结构体

目前，系统休眠支持浅睡、深睡两种模式。系统休眠默认包含浅睡逻辑，若用户有自定义休眠逻辑或深睡的需求，可以注册低功耗流程结构体struct PowerMgrParameter。

```
typedef struct {
    PowerMgrRunOps runOps;          /* power manager run operations */
} PowerMgrParameter;

typedef struct {
#ifdef LOSCFG_KERNEL_DYN_FREQ
    UINT32 (*changeFreq)(LosFreqMode freq); /**< Tune system frequency */
#endif
#ifdef defined (LOSCFG_KERNEL_LIGHTSLEEP) || defined (LOSCFG_KERNEL_DEEPSLEEP)
    LosSleepMode (*getSleepMode)(VOID); /**< GetSleepMode, provided for special needs before entering sleep */
#endif
#ifdef LOSCFG_KERNEL_LIGHTSLEEP
    VOID (*enterLightSleep)(VOID); /**< Enter light sleep */
#endif
#ifdef LOSCFG_KERNEL_DEEPSLEEP
#ifdef LOSCFG_LOWPOWER_SLEEP_USERCONFIG
    VOID (*userPreConfig)(VOID); /**< UserPreconfig, provided for special needs before entering sleep */
    VOID (*userPostConfig)(VOID); /**< UserPostconfig, provided for special needs after wakeup */
#endif
    VOID (*systemWakeup)(VOID); /**< System wakeup */
    VOID (*suspendConfig)(VOID); /**< Suspend device before entering deep sleep */
    VOID (*resumeConfig)(VOID); /**< Resume device after wakeup from deep sleep */
#endif
#ifdef LOSCFG_KERNEL_SMP
    VOID (*otherCoreResume)(VOID); /**< Other core Resume for multi-core scenes */
#endif
    VOID (*enterDeepSleep)(VOID); /**< Enter deep sleep */
    VOID (*setWakeUpTimer)(UINT32 timeout); /**< Set wakeup timer */
    VOID (*withdrawWakeUpTimer)(VOID); /**< Withdraw wakeup timer */
#ifdef LOSCFG_KERNEL_RAM_SAVE
    VOID (*contextSave)(VOID); /**< Context save */
    VOID (*contextRestore)(VOID); /**< Context restore */
#endif
#ifdef LOSCFG_LOWPOWER_SLEEP_WFI
    VOID (*enterWFI)(VOID); /**< Enter WFI mode */
#endif
} PowerMgrRunOps;
```

4.3.3.2 开发指导

用户可根据具体业务场景制定休眠策略，定义禁止/允许进入的睡眠模式、选择不同的睡眠模式。由于深睡等模式可能涉及到具体硬件和时钟，所以用户需自行实现相应函数以适配硬件。同时系统休眠模块在流程初始和结束阶段均有预留接口，对于有特殊需求的场景，可以进行睡眠流程的扩展。

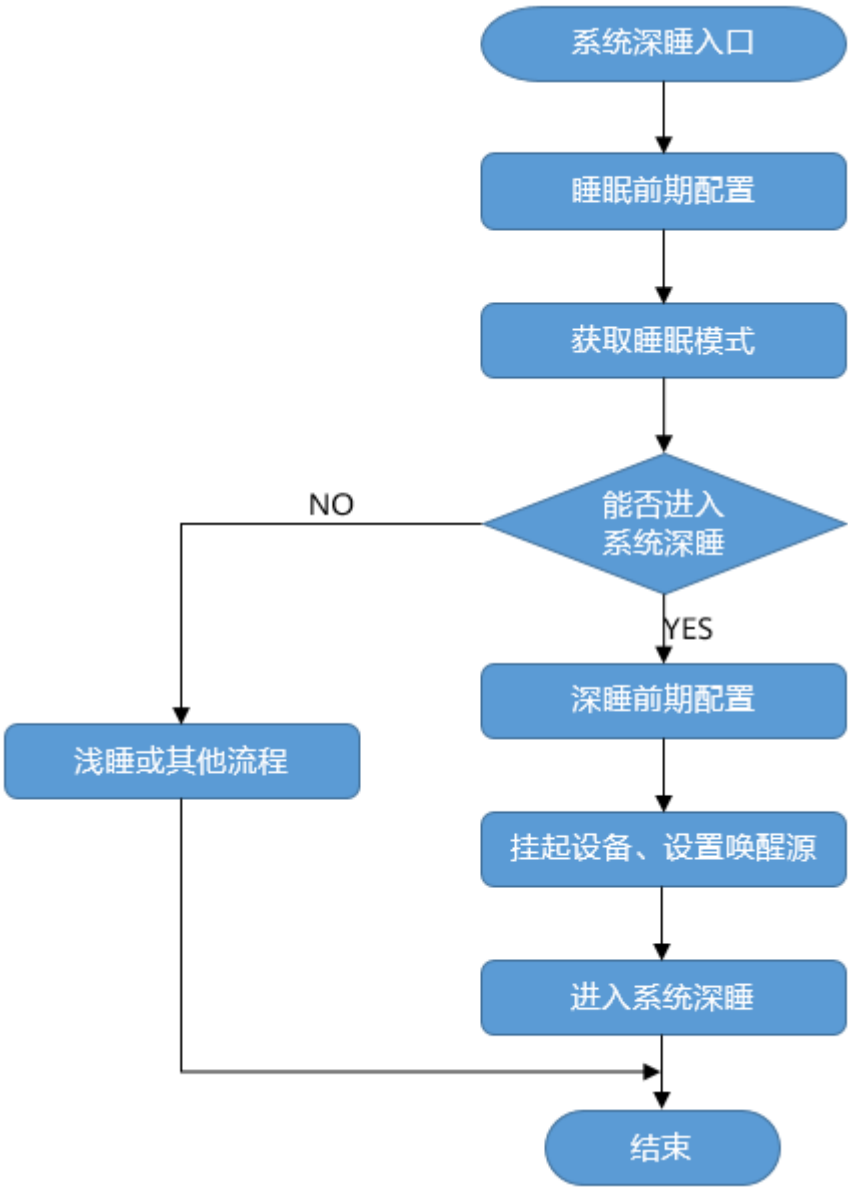
系统休眠模块为用户提供如下功能，接口详细信息可以查看API参考。

表 4-3 功能列表

功能分类	接口名	描述
系统休眠开关	LOS_EnableLowPower	开启系统休眠，仅在debug模式生效
	LOS_DisableLowPower	关闭系统休眠，仅在debug模式生效

功能分类	接口名	描述
系统休眠	LOS_PowerMgrSleepLock	禁止进入指定的睡眠模式
	LOS_PowerMgrSleepUnlock	允许进入指定的睡眠模式
	LOS_LowpowerHookReg	注册系统休眠的入口函数
系统调频	LOS_PowerMgrChangeFreq	系统调频

默认的系统休眠流程，大致如下图所示。



- 在进入系统休眠时，首先进行睡眠前期的配置，根据系统可睡眠时间 & 用户定义的休眠策略，获取睡眠模式。
- 若为深睡模式，则会执行深睡前期的配置，例如屏蔽中断、保存设备状态等。随后将挂起设备并设置唤醒源，系统进入深睡模式。
 - 若为浅睡或其他模式，则执行对应流程。通常情况下，Tickless将配合系统休眠浅睡共同使用，以屏蔽Tick中断。用户也可使用自定义的Tick屏蔽和恢复机制。默认浅睡仅进入Tickless+WFI模式。

开发流程

系统休眠的典型开发流程：

1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Low Power Management Framework ---> Low Power Management Configure菜单。

配置项	含义	取值范围	默认值	依赖
config LOSCFG_KERNEL_LIGHTSLEEP	系统休眠的浅睡模式裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_KERNEL_DEEPSLEEP	系统休眠的深睡模式裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER，且需要硬件平台支持
LOSCFG_KERNEL_RAM_SAVE	系统休眠时深睡模式下的内存保存开关	YES/NO	NO	LOSCFG_KERNEL_DEEPSLEEP，且需要硬件平台支持
LOSCFG_LOWPOWER_SLEEP_USERCONFIG	系统休眠时前/后处理开关	YES/NO	NO	LOSCFG_KERNEL_DEEPSLEEP
LOSCFG_KERNEL_DYN_FREQ	系统休眠的调频裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER，且需要硬件平台支持
LOSCFG_LOWPOWER_SLEEP_VETO	系统休眠的投票裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_LOWPOWER_SLEEP_WFI	系统休眠WFI模式自定义配置开关	YES/NO	NO	LOSCFG_KERNEL_LOWPOWER
LOSCFG_MAX_SLEEP_TIME	系统休眠最大睡眠阈值配置	INT	10000	LOSCFG_KERNEL_LOWPOWER

配置项	含义	取值范围	默认值	依赖
LOSCFG_MIN_LIGHT_SLEEP_TIME	系统休眠最小浅睡阈值配置	INT	50	LOSCFG_KERNEL_LIGHTSLEEP
LOSCFG_MIN_DEEP_SLEEP_TIME	系统休眠最小深睡阈值配置	INT	2000	LOSCFG_KERNEL_DEEPSLEEP
LOSCFG_DELTA_TICKS	系统休眠深睡tick补偿配置	INT	1	LOSCFG_KERNEL_DEEPSLEEP
LOSCFG_EXT_TIMER_FREQ	系统休眠深睡外部时钟频率配置	INT	1	LOSCFG_KERNEL_DEEPSLEEP

2. 在Menuconfig中配置系统休眠唤醒策略，包括最小浅睡/深睡阈值、最大睡眠阈值；如果开启深睡模式，根据需要选择是否开启自定义WFI、休眠前后处理、深睡tick补偿以及外部时钟频率等。
3. 适配对应睡眠模式的休眠接口，详见[低功耗流程注册结构体](#)。
4. 当系统业务繁忙时，调用LOS_PowerMgrSleepLock接口禁止进入指定的睡眠模式（深睡、浅睡）；当系统业务空闲时，调用LOS_PowerMgrSleepUnLock接口允许进入指定的睡眠模式。注意这两接口应该是配套使用的。
5. 系统待机时，根据业务投票及系统实际可休眠时长，自动进入相应休眠模式进行休眠。

4.3.3.3 注意事项

1. 开启系统深睡模式时，建议在[低功耗流程注册结构体](#)的子结构体成员struct PowerMgrRunOps的enterDeepSleep钩子函数中将cache写回内存。
2. 低功耗提供的调频机制，如果在系统启动之后和业务首次调频之前进入空闲状态，系统会默认把频率调到最高。

4.3.3.4 编程实例

实例描述

本实例实现如下功能：

1. 注册低功耗流程框架，适配深睡、RTC使能等功能接口；
2. 创建用户任务，并在任务中延时5000Ticks，使系统进入深睡模式。

编程示例

前提条件：在menuconfig菜单中完成系统休眠深睡的配置，同时配置用户休眠阈值：LOSCFG_MIN_SLEEP_TIME、LOSCFG_MAX_SLEEP_TIME以及LOSCFG_MIN_DEEP_SLEEP_TIME。

部分代码实现如下：

1. 注册低功耗框架

```

#ifdef LOSCFG_KERNEL_LOWPOWER
#include "los_lowpower.h"
#endif
const PowerMgrParameter gPowerMgrParameter = {    // 用户自定义系统休眠配置，成员函数用户自行按需实现
    .runOps = {
#ifdef LOSCFG_KERNEL_LIGHTSLEEP
        .enterLightSleep = EnterLightSleep,        // 浅睡动作，此处设置为进入WFI模式
        .getSleepMode = GetSleepMode,              // 获取用户休眠模式
#endif
#ifdef LOSCFG_KERNEL_DEEPSLEEP
        .suspendConfig = SuspendDevice,            // 关闭外设动作
        .resumeConfig = ResumeDevice,              // 恢复外设
        .enterDeepSleep = EnterDeepSleep,          // 深睡动作，此处设置为STM32的standby模式
        .setWakeUpTimer = SetWakeUpTimer,          // 设置唤醒源，此处设置为根据入参（休眠时间）使
        能外部RTC
        .withdrawWakeUpTimer = WithdrawWakeUpTimer, // 取消唤醒源，此处设置为停止RTC计时
#endif
    }
};
INT32 main(VOID)
{
    UINT32 ret;
#ifdef LOSCFG_KERNEL_LOWPOWER
    // 注册低功耗流程框架，若仅需默认的Tickless及浅睡模式，可调用LOS_PowerMgrInit(NULL)注册系统默认框架。
    LOS_PowerMgrInit(gPowerMgrParameter);
#endif
    HardwareInit();
    printf("[main]: Hello, welcome to liteos !\n");
    PRINT_RELEASE("*****Hello Huawei LiteOS *****\n"
        "\nversion : %s\nopen-version : %s %u.%u.%u\n"
        "build data : %s %s\n\n"
        "*****\n",
        HW_LITEOS_VER, HW_LITEOS_SYSNAME, MAJ_V, MIN_V, REL_V, __DATE__,
        __TIME__);
    OsSetMainTask();
    OsCurrTaskSet(OsGetMainTask());
    ret = OsMain();
    if (ret != LOS_OK) {
        return LOS_NOK;
    }
    RTC_Start();
    g_kernellnit = 1;
    OsStart();
    return LOS_OK;
}

```

2. 创建任务，并在任务入口函数中执行延时操作：

```

STATIC VOID task_demo1(VOID)
{
    while (1) {
        osDelay(5000); // 任务延时5000Ticks
    }
}

```

结果验证

由于设置的深睡阈值为2000，任务延时5000Ticks，所以系统会进入深睡模式：


```
[09:12:46.954]收←◆[main]: Hello, welcome to liteos !
*****Hello Huawei LiteOS *****

version : Huawei LiteOS V200R005C20B050-SMP
open-version : Huawei LiteOS 5.1.0
build data : May 11 2021 09:12:27

*****
UsAppInit
cpu 0 entering scheduler
[app_init]: Hello, welcome to liteos!

[09:12:52.183]收←◆resume from standby
[09:12:57.387]收←◆resume from standby
[09:13:02.594]收←◆resume from standby
```

如上图所示，系统每隔5s将会从standby模式唤醒。

完整实例代码

demo.rar

4.3.4 Tickless

4.3.4.1 概述

基本概念

Tickless 机制是一种新的定时机制，它使用动态时钟中断替代周期性时钟定时。

任务调度通常通过时钟中断触发。在周期性定时机制下，每一次时钟中断都会检测当前调度条件是否满足，但能触发任务调度的时钟中断往往只占很小一部分。软件定时器任务必须通过时钟中断触发才能实现，每一次时钟中断也会检测软件定时器是否到期，若到期则调度该定时器的回调函数，而大部分时钟中断发生时并没有软件定时器到期。Tickless机制是通过预期省略不必要的时钟中断，确定下一次“有意义”的时钟中断的产生时刻，而不让“无意义”的时钟中断产生。通过省去CPU空闲时期无意义的时钟中断，可以有效降低系统的功耗。

运作机制

LiteOS的Tickless机制，采用的是“idle模式”，即在CPU空闲时期（idle任务中）开启Tickless机制。通过计算下一次任务调度的时间和下一次软件定时器到期的时间，将两者中的较小值设定为下一次时钟中断到来的时间，从而减少无意义的时钟中断。

4.3.4.2 开发指导

使用场景

系统CPU长时间处于空闲状态，且系统对功耗要求较高。

开发流程

Tickless的典型开发流程：

1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Low Power Management Framework菜单，使能Tickless。
- | 配置项 | 含义 | 取值范围 | 默认值 | 依赖 |
|------------------------|---------------|--------|-----|------------------------|
| LOSCFG_KERNEL_TICKLESS | Tickless的裁剪开关 | YES/NO | YES | LOSCFG_KERNEL_LOWPOWER |
2. 当系统长时间不运行业务时，进入idle任务后根据可休眠时间自动进入tickless模式。

平台差异性

无。

4.3.4.3 注意事项

由于Tickless机制与系统时间都通过Tick中断定时器计时，因此在时间精度要求极高的场景下不建议使用。

各平台由于Systick寄存器位数差异，为避免寄存器溢出，在配置tickless休眠时间时，需要根据寄存器可接受最大数值、系统时钟频率、LOSCFG_BASE_CORE_TICK_PER_SECOND计算tickless可正常运行的最大tick数。例如cortex-m4平台由于SysTick->LOAD寄存器为24位，系统时钟频率为120M，配置LOSCFG_BASE_CORE_TICK_PER_SECOND为1000，则配置tickless休眠时间应不超过对应阈值： $(2^{24} - 1) / (120000000 / 1000) = 139$ ticks。

4.4 访问权限管理

4.4.1 概述

访问权限管理，即通过配置硬件的内存保护/内存管理单元，实现CPU在不同运行态时，对目标地址的访问权限控制。

 注意

不同目标平台的内存保护/内存管理单元设计可能存在差异，在使用本章节提供接口前，请务必阅读目标平台的相关文档以理解平台硬件能力和使用限制。

使用该功能前，请确保已经使能访问权限控制（其对应的宏开关为LOSCFG_APC_ENABLE，可以打开菜单，选择Targets ---> Enable Access Permission Control使能），使能后系统会根据不同架构使能对应的内存管理单元，包含RISC-V下的PMP，Cortex-M架构下的MPU，Cortex-A架构下的MMU。

4.4.2 PMP

4.4.2.1 概述

基本概念

为了提高系统安全性，希望通过软件的方式配置物理地址的访问权限，RISC-V提供了标准的控制状态寄存器（CSR）来支持内存保护的功能。一般RISC-V平台中，物理内存保护（Physical Memory Protection，简称PMP）模块支持16个 PMP entry，通过两类寄存器来对PMP entry进行配置，分别为：pmpcfg0-3（8-bit for each entry），pmpaddr0-15。华为自研芯片中，可能有额外的一类寄存器memattr/memattrh(4-bit for each entry)。每一块entry都有自己可编程的起始地址，大小（pmpaddrX），及设置（pmpcfgX），另外还支持一些背景内存区域特性（memattr/memattrh）。

PMP提供的功能包括：

- 将SRAM或RAM空间定义为Non-Writable，防止代码被篡改。
- 将SRAM或RAM空间定义为Non-Executable，防止代码被注入攻击。
- 通过PMP memattr/memattrh定义存储器的属性。

4.4.2.2 开发指导

功能

Huawei LiteOS PMP为用户提供下面几种功能。

表 4-4 PMP 支持的接口列表

功能分类	接口名	描述
禁用PMP区域	ArchRegionDisable	禁用地址保护区域
设置PMP区域	ArchProtectionRegionSet	设置地址保护区域
使能Cache	ArchICacheEnable	使能ICache并设置Cache size
	ArchDCacheEnable	使能DCache并设置Cache size
	ArchICachePrefetchEnable	使能ICache预取
	ArchDCachePrefetchEnable	使能DCache预取
刷新所有Cache	ArchICacheFlush	刷新所有的ICache line
	ArchDCacheFlush	刷新所有的DCache line
按指定空间大小刷新Cache	ArchDCacheFlushByVa	按指定起始地址及指定空间大小刷新DCache

功能分类	接口名	描述
刷新指定地址区间的Cache	ArchICacheFlushByVa	按指定起始地址及指定空间大小刷新ICache
	ArchICacheFlushByAddr	刷新指定地址区间的ICache
	ArchDCacheFlushByAddr	刷新指定地址区间的DCache
无效Cache	ArchDCacheCleanByAddr	刷新指定地址区间的DCache后，无效该区间的DCache
	ArchDCacheInvalidateByVa	按指定起始地址及指定空间大小无效DCache
	ArchDCacheInvalidateByAddr	无效指定地址区间的DCache

PMP 错误码

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_PMP_PTR_NULL	0x02001200	空指针检测错误	确保传入正确参数
2	LOS_ERRNO_PMP_INVALID_BASE_ADDRESS	0x02001201	配置Region时，基址未保证4字节对齐	确保基址对齐
3	LOS_ERRNO_PMP_INVALID_CAPACITY	0x02001202	配置NAPOT模式的Region时，未保证Size是2的整数幂	确保保护范围是2的整数幂
4	LOS_ERRNO_PMP_ATTR_INVALID	0x02001203	配置Region的内存属性时，使用了非法的值	确保传入的内存属性是当前硬件平台支持的
5	LOS_ERRNO_PMP_INVALID_NUMBER	0x02001204	无效的Region号	选择使用合法的Region
6	LOS_ERRNO_PMP_TOR_INVALID_MATCH_MODE	0x02001205	修改TOR/OFF模式Region至非TOR/OFF模式时，检测到下一个Region已经配置为TOR模式	不调整模式

序号	定义	实际数值	描述	参考解决方案
7	LOS_ERRNO_PMP_NAPOT_INVALID_MATCH_MODE	0x02001206	修改NAPOT模式Region至非NAPOT模式时，检测到下一个Region已经配置为TOR模式	不调整模式
8	LOS_ERRNO_PMP_REGION_INVALID_ACCESS	0x02001207	暂不使用该错误码	-
9	LOS_ERRNO_PMP_BASE_ADDRESS_NOT_IN_RANGE	0x02001208	配置Region时，Region基址不在系统允许的范围	调整至合理的保护范围
10	LOS_ERRNO_PMP_REGION_SIZE_IS_TOO_BIG	0x02001209	配置Region时，Region范围过大	调整至合理的保护范围
11	LOS_ERRNO_PMP_ENTRY_IS_LOCKED	0x0200120a	尝试操作的Region已经被锁定	选择使用其他Region
12	LOS_ERRNO_PMP_INVALID_MATCH	0x0200120b	配置模式为NA4，但硬件不支持	选用其他模式
13	LOS_ERRNO_PMP_NAPOT_ENCODING_NON_MATCHED	0x0200120c	配置NAPOT模式的Region时，范围不合理	配置合理的保护范围
14	LOS_ERRNO_PMP_TOR_LOWER_BOUND_LOCKED	0x0200120d	配置TOR模式的Region时，前一个Region已经被锁定	确认前一个Region是否已经锁定，并选择合适的Region
15	LOS_ERRNO_PMP_TOR_LOWER_BOUND_NONMATCHED	0x0200120e	配置TOR模式的Region时，Region低地址不合法	确认Region低地址配置是否合理，若不合理，则调整
16	LOS_ERRNO_PMP_SECTL_INVALID	0x0200120f	配置非法的sectl	设置合法的sectl

须知

错误码定义见[错误码简介](#)。PMP模块的模块ID和MPU模块共用，值为0x12。

PMP entry 结构体定义（针对 lx131）

```
typedef struct {
    /* Index of PMP entry */
    UINT8 ucNumber;

    /* whether lock, Works in any supported mode */
    BOOL bLocked;

    /*
     * Address matching mode of PMP configuration,
     * Currently supported modes: PMP_RGN_ADDR_MATCH_OFF,
     * PMP_RGN_ADDR_MATCH_TOR and PMP_RGN_ADDR_MATCH_NAPOT.
     */
    UINT8 ucAddressMatch;

    /*
     * access permission of read, write and instruction execution.
     * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
     */
    PmpAccInfo accPermission;

    /*
     * set the base address of the protected region,
     * the base address must be in the range of RAM.
     * It should be cache line aligned under the cache.
     */
    UINT32 uwBaseAddress;

    /*
     * PMP entry region size: It is valid only in PMP_RGN_ADDR_MATCH_NAPOT mode,
     * requiring >= 8byte and aligned power-of-2 byte.
     * It should be cache line aligned under the cache.
     */
    UINT32 uwRegionSize;

    /*
     * PMP memory attribute register information:
     * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
     */
    PmpMemAttrInfo memAttr;

    /*
     * Security control register information:
     * The changed member is not valid in PMP_RGN_ADDR_MATCH_OFF mode.
     */
    PmpSeControlInfo seCtl;
} PMP_ENTRY_INFO;
```

该数据结构中主要分为5个部分：

- PMP entry number
- PMP configuration register: Lock bit + Address Matching + Access Permission(R/W/X)
- PMP address register: base address + region size(or end address)
- PMP memory attribute register: memory attribute , including cacheability and allocation policies
- PMP security control register: non-secure or secure

PMP 相关资料

请参考RISC-V官方相关[Specifications](#)。

华为自研芯片请参考相关配套资料。

4.4.2.3 注意事项

- 若PMP 寄存器出现重叠，且同一个存储器位置落在两个PMP entry的寄存器中，存储器访问属性和权限则会基于编号最小的寄存器。
- 当前Huawei LiteOS中只存在 Machine-Mode (M-Mode)，并且在M-Mode下对PMP进行配置。一般而言，设置PMP entry的访问权限只在User-Mode (U-Mode) 生效，仅当Lock 该PMP entry时，该PMP entry 访问权限才会对M-Mode生效。

4.4.2.4 编程实例

实例描述

本实例设置PMP entry 3，将起始地址为0x1000000，大小为16MB的内存区域设为可读可写可执行，内存属性设为write-back write and read allocate。地址模式设置为NAPOT，不需要依赖前一个entry 2的pmpaddr设置。

编程示例

前提条件：打开菜单使能访问权限控制。

```
static UINT32 SetPmpEntry3(VOID)
{
    UINT32 ret;

    pstPmpInfo.ucNumber = 3;

    // set pmpXcfg register
    pstPmpInfo.bLocked = TRUE;
    pstPmpInfo.ucAddressMatch = PMP_RGN_ADDR_MATCH_NAPOT;
    pstPmpInfo.accPermission.readAcc = E_MEM_RD_ACC_RD;
    pstPmpInfo.accPermission.writeAcc = E_MEM_WR_ACC_WR;
    pstPmpInfo.accPermission.excuteAcc = E_MEM_EX_ACC_EX;

    // set pmpaddrX register
    pstPmpInfo.uwBaseAddress = 0x1000000;
    pstPmpInfo.uwRegionSize = PMP_REGION_SIZE_16MB;

    // set memattr register
    pstPmpInfo.memAttr = MEM_ATTR_WB_RW_ALLOC;
    pstPmpInfo.seCtl = SEC_CONTRLO_SECURE_NOMMU;

    ret = LOS_ProtectionRegionSet(&pstPmpInfo);
    if(ret != LOS_OK) {
        printf("set protection region failed.\n");
        return LOS_NOK;
    }

    return LOS_OK;
}
```

4.4.3 MPU

4.4.3.1 概述

在Cortex-M处理器中可以选配一个存储器保护单元（MPU），实现对存储器（主要是内存和外设寄存器）的保护，从而使软件更加健壮可靠。如果打算启用MPU，在使用前必须根据需要对其编程。如果没有启用MPU，则等同于系统中没有配MPU。MPU有如下能力可以提高系统的可靠性：

- 阻止用户应用程序破坏操作系统使用的数据。
- 阻止任务访问其它任务的数据区，从而将任务隔离开。
- 可以把关键数据区设置为只读，从根本上消除了被破坏的可能。
- 检测异常的存储访问，例如：堆栈溢出，数组越界。
- MPU可以设置存储器regions的其它访问属性，例如：是否buffable，是否cachable等。

4.4.3.2 开发指导

功能

Huawei LiteOS MPU为用户提供下面几种功能。

表 4-5 MPU 支持的接口列表

接口名	描述
ArchRegionDisable	禁用地址保护区域
ArchProtectionRegionSet	设置地址保护区域

MPU 错误码

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_MPU_PTR_NULL	0x02001200	空指针检测错误	确保传入正确参数
2	LOS_ERRNO_MPU_INVALID_BASE_ADDRESS	0x02001201	配置Region时，使用了非法的基地址	确保使用合法的基地址
3	LOS_ERRNO_MPU_INVALID_CAPACITY	0x02001202	配置Region时，使用了非法的Region size	确保使用合法的Region size
4	LOS_ERRNO_MPU_NOT_CONFIGURED	0x02001203	硬件平台不支持MPU	确保硬件平台支持MPU

序号	定义	实际数值	描述	参考解决方案
5	LOS_ERRNO_M PU_INVALID_N UMBER	0x020 01204	无效的Region号	确保操作合法的Region
6	LOS_ERRNO_M PU_REGION_IS_ ENABLED	0x020 01205	配置了一个已经使能的Region	确保操作正确的Region
7	LOS_ERRNO_M PU_REGION_IS_ DISABLED	0x020 01206	关闭一个未使能的Region	确保操作正确的Region
8	LOS_ERRNO_M PU_REGION_IN VALID_ACCESS	0x020 01207	配置的Region权限不合法	确保配置的权限是合法的
9	LOS_ERRNO_M PU_BASE_ADDR ESS_NOT_IN_R AM	0x020 01208	暂不使用该错误码	-
10	LOS_ERRNO_M PU_REGION_SIZ E_IS_TOO_BIG	0x020 01209	暂不使用该错误码	-

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为MPU模块，值为0x12。

MPU region 结构体定义

```
typedef struct {
    UINT8 number; /* number of MPU register to be checked */
    UINT32 baseAddress; /* set the base address of the protected region,
                        the base address must be in the range of RAM */
    UINT32 accessPermission; /* privilege level and user level access permission */
    BOOL sharable; /* whether to share */
    BOOL cachable; /* whether cache */
    BOOL buffable; /* whether buffer */
    UINT32 regionSize; /* region size */
    BOOL hfnmiena; /* Whether in the NMI and hard fault service routine
                  is not mandatory in addition to MPU */
    BOOL xn; /* To indicate whether instructions are fetchable in this region,
            0 fetchable; 1: otherwise */
} MPU_REGION_INFO;
```

该数据结构中主要分为4个部分：

- MPU Region number
- MPU Access Permission(R/W/X)
- MPU address register: base address + region size(or end address)

- MPU memory attribute register: memory attribute , including cacheability and allocation policies

MPU 相关资料

请参考[Arm官方相关文档](#)。

4.4.3.3 注意事项

- 若存储器访问权限与MPU定义的访问权限冲突，或者访问的存储器位置未在MPU寄存器中定义，数据传输会被阻止且触发一次异常。
- 若MPU寄存器出现重叠，且同一个存储器位置落在两个MPU区域中，存储器访问属性和权限则会基于编号最大的寄存器。
- 当前Huawei LiteOS中只存在Privileged，并且只能在Privileged下对MPU进行配置。

4.4.3.4 编程实例

实例描述

本实例配置了Region0和Region1。对Region0配置0x00000000开始的512KB地址空间的内存属性和访问权限，对Region1配置0x20000000开始的512KB地址空间的内存属性和访问权限。

编程示例

前提条件：打开菜单使能访问权限控制。

```
VOID MPU_Config(VOID)
{
    MPU_REGION_INFO mpuInfo;

    mpuInfo.ucNumber = 0;
    mpuInfo.uwBaseAddress = 0x00000000; // start addr of text
    mpuInfo.uwAccessPermission = MPU_DEFS_RASR_AP_RO;
    mpuInfo.bSharable = 1;
    mpuInfo.bCachable = 1;
    mpuInfo.bBuffable = 1;
    mpuInfo.uwRegionSize = MPU_REGION_SIZE_512KB;
    mpuInfo.bHfnmiena = 1;
    mpuInfo.bXN = 0;
    LOS_SetProtectionRegion(&mpuInfo);

    mpuInfo.ucNumber = 1;
    mpuInfo.uwBaseAddress = 0x20000000; // start addr of data
    mpuInfo.uwAccessPermission = MPU_DEFS_RASR_AP_FULL_ACCESS;
    mpuInfo.bSharable = 1;
    mpuInfo.bCachable = 1;
    mpuInfo.bBuffable = 1;
    mpuInfo.uwRegionSize = MPU_REGION_SIZE_512KB;
    mpuInfo.bHfnmiena = 1;
    mpuInfo.bXN = 1;
    LOS_SetProtectionRegion(&mpuInfo);
}
```

4.4.4 MMU

4.4.4.1 概述

基本概念

在Cortex-A处理器中可以选配一个内存管理单元（MMU，Memory Management Unit），实现虚拟地址映射为物理地址（即虚拟内存管理）以及提供硬件机制的内存保护。

使用场景

系统内部有些内存不希望被修改，否则会造成不可预测的后果，此时可以用MMU修改该段内存的访问权限为只读。访问该段内存时会检查访问权限，若权限不正确则会触发异常，起到保护该段内存数据的作用。

通过cache+buffer可控制内存缓存状态，包括：关闭cache、写通、写回。

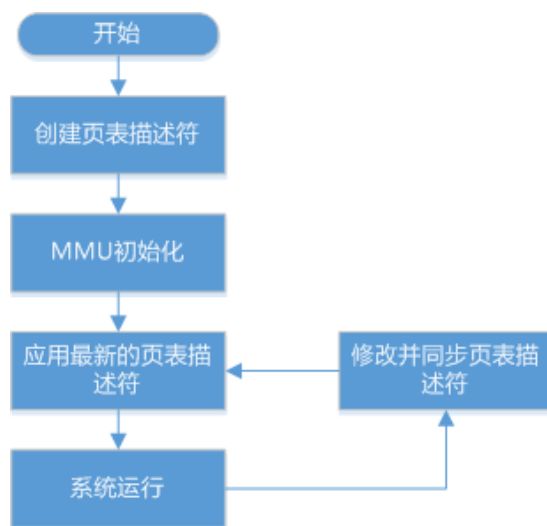
4.4.4.2 ARM32 MMU

运作机制

建立页表描述符号表，将物理地址映射成虚拟地址，以虚拟地址为媒介来操作和管理实际的物理内存。

页表描述符号表，是根据所使用的主芯片的页表描述格式创建、修改和管理的。内存依据页表描述符号表进行映射、权限控制等。页表描述符号表在创建或修改后，需要写入协处理器CP15才能生效，而协处理CP15正是内存管理的实际执行者。

综合来说，MMU操作就是通过修改页表描述符和控制CP15协处理器实现的，具体运作流程如下图所示。



Huawei LiteOS的MMU有两个方面的作用：

- 提供硬件机制的内存cache/nocache属性的控制接口。
- 提供硬件机制的内存访问权限控制接口。

功能

Huawei LiteOS的MMU模块为用户提供以下接口。

接口名	描述
ArchSecPageEnable	建立一级页表和传入的二级页表的地址关系，并初始化内存读写权限
ArchMMUParamSet	修改指定地址段的内存cache状态、buffer状态、读写权限状态

接口的入参数据类型说明：

```
typedef struct
{
    UINT32 startAddr;    /* 起始地址 */
    UINT32 endAddr;      /* 结束地址 */
    UINT32 uwFlag;        /* 配置属性：
        * bit0: ACCESS_PERM_RW_RW/ACCESS_PERM_RO_RO(1/0)
        * bit1: CACHE_ENABLE/CACHE_DISABLE(1/0)
        * bit2: BUFFER_ENABLE/BUFFER_DISABLE(1/0)
        * bit3: EXEC_DISENABLE/EXEC_ENABLE(1/0)
        * bit4: FIRST_SECTION/SECOND_PAGE(1/0)
        * bit5~7: ignore
        */
    SENCOND_PAGE *stPage; /* 指定进行操作的二级页表，如果uwFlag配置为FIRST_SECTION属性，stPage将会被忽略 */
} MMU_PARAM;
```

二级页表结构体说明：

```
typedef struct
{
    UINT32 page_addr;      /* 使用二级页表描述的内存起始地址，必须1M对齐 */
    UINT32 page_length;    /* 使用二级页表描述的内存长度，必须1M对齐 */
    UINT32 page_descriptor_addr; /* 二级页表表项存放地址，必须1K对齐 */
    UINT32 page_type;      /* 二级页表属性，包括小页（4K），大页（64K） */
}SENCOD_PAGE;
```

举例：

```
void code_protect(void)
{
    MMU_PARAM mPara;
    mPara.startAddr = (unsigned long)&__text_start;
    mPara.endAddr = (unsigned long)&__rodata1_end;
    mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RO_RO;
    mPara.stPage = (SENCOD_PAGE *)&osPage; /* 标记A */
    ArchMMUParamSet(&mPara);
}
```

描述：

将__text_start，__rodata1_end这两个地址之间的内存设置为（写回）启用cache、buffer、只读；系统中可能存在多张二级页表，标记A处是用来指明接口操作的对象页表。

4.4.4.2.1 开发指导

开发流程

下面是MMU的典型开发流程：

步骤1 定义一个二级页表结构体变量，一般定义为静态或全局变量。

```
SECONCOND_PAGE osPage = {0};
__attribute__((aligned(MMU_1K))) UINT8 second_page_table_os[MMU_16K]; //定义二级页表存储表，大小是16k
```

步骤2 配置二级页表属性。

```
osPage.page_addr = SYS_MEM_BASE; // SYS_MEM_BASE是内存起始地址，一般也是kernel text段起始地址
osPage.page_length = (((UINTPTR)&__ram_data_start - SYS_MEM_BASE) + MMU_1M - 1) & ~(MMU_1M - 1); //__ram_data_start是数据段起始地址
osPage.page_descriptor_addr = (UINTPTR)second_page_table_os;
osPage.page_type = MMU_SECOND_LEVEL_SMALL_PAGE_TABLE_ID; //指定页面大小是4K，非64K
```

步骤3 使能指定的二级页表。

```
ArchSecPageEnable(&osPage, BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RW_RW);
```

步骤4 修改内存属性。

```
void test_func(void)
{
    MMU_PARAM mPara;
    mPara.startAddr = (unsigned long)&__text_start;
    mPara.endAddr = (unsigned long)&__rodata1_end;
    mPara.uwFlag = CACHE_ENABLE | ACCESS_PERM_RO_RO;
    mPara.stPage = (SECONCOND_PAGE *)&osPage;
    ArchMMUParamSet(&mPara);
}
```

----结束

4.4.4.2.2 注意事项

相比一级页表配置，二级页表配置的内存地址翻译效率更低，且占用更多的TLB资源，所以对于经常被访问的内存建议尽量使用一级页表。

4.4.4.2.3 编程实例

实例描述

调用接口ArchMMUParamSet，修改内存区域的读写权限，再通过在该内存区域进行写操作，查看是否已经正确修改读写权限。

步骤1 修改一段区间的内存读写权限为只读。

步骤2 在该段区间的内存中进行简单的写操作。

系统进入异常，说明将该内存设置为“只读”成功。

步骤3 注释步骤2中的写操作，调用接口将内存读写权限重新修改为可读可写后，再进行写操作。

系统不进入异常，说明将该内存设置为“可读可写”成功。

----结束

编程示例

```
extern char __text_start;
extern char __text_end;

UINT32 MMU_Sample()
{
    UINT32 *alignAddr;
    extern SECONCOND_PAGE osPage;
```

```
MMU_PARAM mPara;

mPara.startAddr = (unsigned long)&__text_start;
mPara.endAddr = (unsigned long)&__text_end;
mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RO_RO;
mPara.stPage = (SECONDD_PAGE *)&osPage;

PRINTK("---- TEST START ----\n");
alignAddr = (UINT32 *)&__text_start;
PRINTK(">>1\n");
ArchMMUParamSet(&mPara);
*alignAddr = 0xa;      //if write in the memory, an exception will be triggered.

PRINTK(">>2\n");
mPara.uwFlag = BUFFER_ENABLE | CACHE_ENABLE | ACCESS_PERM_RW_RW;
ArchMMUParamSet(&mPara);
*alignAddr = 0xb;

PRINTK(">>3\n");
PRINTK("---- TEST END ----\n");
return LOS_OK;
}
```

运行结果

不注释代码 “*alignAddr = 0xa;”

```

---- TEST START ----
>>1
uwExcType = 0x4
puwExcBuffAddr pc = 0x8000adc8
puwExcBuffAddr lr = 0x8000aa78
puwExcBuffAddr sp = 0x80093e30
puwExcBuffAddr fp = 0x80093e44
*****backtrace begin*****
traceback 0 -- lr = 0x8000b1e4
traceback 0 -- fp = 0x80093e84
traceback 1 -- lr = 0x8000b2c4
traceback 1 -- fp = 0x80093e9c
traceback 2 -- lr = 0x8000ae8c
traceback 2 -- fp = 0x80093eac
traceback 3 -- lr = 0x8000ac4c
traceback 3 -- fp = 0x80093ebc
traceback 4 -- lr = 0x800104fc
traceback 4 -- fp = 0x80093ecc
traceback 5 -- lr = 0x80026e1c
traceback 5 -- fp = 0x11111111

Name                TID    Priority  Status
----
Swt_Task             0x0     0         QueuePend
IdleCore000          0x1    31         Ready
IT_TST_INI           0x2    25         Running

R0 = 0x800040c0
R1 = 0x800040a8
R2 = 0x800040a4
R3 = 0x0
R4 = 0x8000a000
R5 = 0x0
R6 = 0x8002a000
R7 = 0x800302e4
R8 = 0x80051df4
R9 = 0x8002c020
R10 = 0x80051df4
R11 = 0x80093e44
R12 = 0x2a
SP = 0x80093e30
LR = 0x8000aa78
PC = 0x8000adc8
CPSR = 0x200f0013
pcTaskName = IT_TST_INI
TaskID = 2
Task StackSize = 86016
system mem addr:0x800561c0

stack name          stack addr      total size    used size
-----
undef_stack_addr    0x8002c820      0x20          0x0
abt_stack_addr      0x8002c840      0x20          0xb
irq_stack_addr      0x8002c880      0x40          0x10
fiq_stack_addr      0x8002c8c0      0x40          0x0
svc_stack_addr      0x8002d8c0      0x1000        0x14f
startup_stack_addr  0x800563a0      0x200         0x138

```

注释掉代码 “*alignAddr = 0xa;”

```

---- TEST START ----
>>1
>>2
>>3
---- TEST END ----

```

完整实例代码

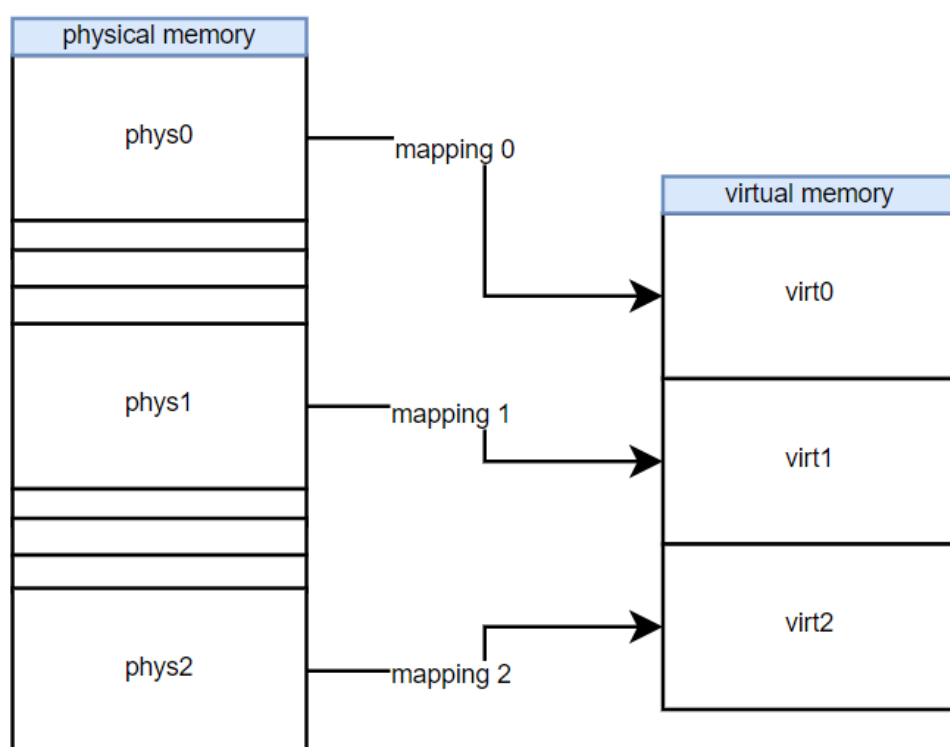
sample_MMU.c

4.4.4.3 ARM64 MMU

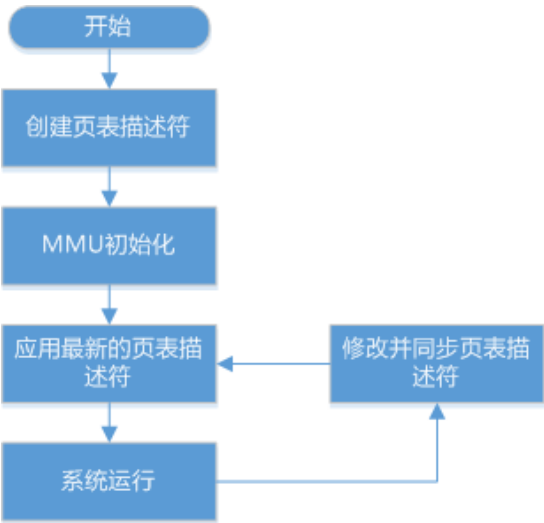
运作机制

建立页表描述符号表，将物理地址映射成虚拟地址，以虚拟地址为媒介来操作和管理实际的物理内存。页表描述符号表，是根据所使用的主芯片的页表描述格式创建、修改和管理的。内存依据页表描述符号表进行映射、权限控制等。

调用ArchMmuMap接口一次只能映射一块物理地址，比如下图中Phys0->virt0。如果有多块不连续物理地址要映射，用户就需要多次调用ArchMmuMap接口建立映射关系，并保证映射的虚拟地址连续，比如下图中Phys0->virt0，Phys1->virt1，Phys2->virt2。



与ARM32 MMU 操作一致，通过维护页表描述符和控制系统寄存器实现。具体运作流程如下图：



4.4.4.3.1 开发指导

功能

Huawei LiteOS的arm64 MMU 模块为用户提供以下接口。

接口名	描述
ArchMmuMap	建立物理页与虚拟地址映射关系
ArchMmuUnmap	解除物理页与虚拟地址映射关系
ArchMmuTableWalk	遍历一段虚拟地址空间内的虚实映射关系
ArchMmuRegionInsert	插入一段映射关系，在内核启动时（MMU使能前）建立页表
ArchMmuTableStatistics	统计页表使用率
ArchMmuLogLevelCfg	设置信息打印等级

MMU 错误码

对存在失败可能性的操作返回对应的错误码，以便快速定位错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_MMU_INPUT_OVERFLOW	0x02000d00	输入虚拟地址或物理地址过大	检查输入

序号	定义	实际数值	描述	参考解决方案
2	LOS_ERRNO_MMU_INP_UT_ZERO	0x02000d01	内存大小为零	检查输入
3	LOS_ERRNO_MMU_INP_UT_FLAG_INV	0x02000d02	页表属性非法	检查页表属性
4	LOS_ERRNO_MMU_PAGETABLE_OOM	0x02000d03	页表内存耗尽	分配更多MMU页表内存
5	LOS_ERRNO_MMU_PTE_BUSY	0x02000d04	页表项之前已映射	解除映射
6	LOS_ERRNO_MMU_REGION_NOT_ENOUGH	0x02000d05	区域数量不够	增加区域数量
7	LOS_ERRNO_MMU_REGION_OVERLAPPED	0x02000d06	输入区域地址范围与之前的区域地址范围重叠	检查输入地址
8	LOS_ERRNO_MMU_INP_UT_NULL	0x02000d07	输入地址为空	检查 ArchMmuTableStatistics 函数输入
9	LOS_ERRNO_MMU_INP_UT_ADDR_INVALID	0x02000d08	虚拟地址与物理地址不同	- 检查虚拟地址与物理地址 - 检查是否需要开启 LOSCFG_ARCH_MMU_VA_MANAGE 选项

MMU 页表属性

页表属性	含义
LOS_MMU_FLAGS_RW_NX_NORMAL_NC	可读可写不可执行，Non-cacheable_NC
LOS_MMU_FLAGS_RO_X_NORMAL_WB	只读可执行，Write-Back Cacheable_WB

页表属性	含义
LOS_MMU_FLAGS_RW_NX_DEVICE_MAP	可读可写不可执行，Device-nGnRE memory
LOS_MMU_FLAGS_RO_NX_NORMAL_WB	只读不可执行，Write-Back Cacheable

 说明

提供了debug、info、error、emg多种日志级别，方便用户定位问题。API接口 ArchMmuLogLevelCfg可以改变MMU模块的日志级别，默认打印等级为error。

开发流程

1. 打开菜单，进入Targets菜单，使能Enable mmu选项完成MMU模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_ARCH_MMU_ENABLE	使能MMU	YES/NO	YES	LOSCFG_ARCH_ARM_ARCH64
LOSCFG_ARCH_MMU_VA_MANAGE	支持虚拟地址与物理地址不同	YES/NO	NO	LOSCFG_ARCH_MMU_ENABLE
LOSCFG_ARCH_MMU_PAGE_TBL_NUM	MMU页表数	[4, 512]	512	LOSCFG_ARCH_MMU_ENABLE
LOSCFG_ARCH_ARM64_PA_BITS	物理地址位宽	{32, 36, 40, 42, 44, 48}	40	LOSCFG_ARCH_MMU_ENABLE
LOSCFG_ARCH_ARM64_VA_BITS	虚拟地址位宽	{39, 48}	39	LOSCFG_ARCH_MMU_ENABLE

2. 建立虚拟地址与物理地址映射。
3. 解除虚拟地址与物理地址映射。

4.4.4.3.2 注意事项

当前设计，仅支持4KB和39bit虚拟地址；开启LOSCFG_ARCH_MMU_VA_MANAGE功能时，用户需通过链接脚本合理划分虚拟地址空间布局，保证虚拟地址不重叠。

4.4.4.3.3 编程实例

实例描述

本实例实现以下功能：

1. 统计页表使用情况。
2. 设置信息打印等级。
3. 建立虚拟地址物理地址映射。
4. 遍历一段内存映射关系。
5. 解除虚拟地址物理地址映射。

编程示例

代码实现如下：

```
#include "arch/mmu.h"
#define STRIDE_MEM_SIZE    0x100000
#define PHYS_MEM_START1    SYS_MEM_END
#define PHYS_MEM_START2    (SYS_MEM_END + 2 * STRIDE_MEM_SIZE)
#define VIRT_MEM_START1    0x048000000
#define VIRT_MEM_START2    (VIRT_MEM_START1 + STRIDE_MEM_SIZE)
VOID SampleMmu(VOID)
{
    PageTablesStat ptr;
    ArchMmuLogLevelCfg(LOS_INFO_LEVEL); // 设置信息显示级别
    ArchMmuTableStatistics(&ptr);        // 统计页表使用情况
    ArchMmuMap(VIRT_MEM_START1, PHYS_MEM_START1, STRIDE_MEM_SIZE,
MMU_RO_NX_NORMAL_WB);
    ArchMmuTableWalk(VIRT_MEM_START1, STRIDE_MEM_SIZE);
    ArchMmuMap(VIRT_MEM_START2, PHYS_MEM_START2, STRIDE_MEM_SIZE,
MMU_RO_NX_NORMAL_WB);
    ArchMmuUnmap(VIRT_MEM_START1, STRIDE_MEM_SIZE); //解除内存映射
}
```

结果验证

编译运行得到的结果为：

```
Total page num:512 and pte num:262144, used page num:5 and pte num:652
Mapping [usr name]: vAddr 0x480000000 pAddr 0x52000000 size 0x100000 attr 0x6000000000007a8
Allocating new page table index[5], table addr:0x42020000
Allocating new page table index[6], table addr:0x42021000
Unmapping: vAddr 0x480000000 size 0x200000
Recycling page table index[6], pte addr:0x420217f8, table addr:0x42021000
Recycling page table index[5], pte addr:0x42020000, table addr:0x42020000
```

完整实例代码

sample_mmu.zip

4.5 CPU 占用率

4.5.1 概述

基本概念

Huawei LiteOS的CPUP (Central Processing Unit Percent, CPU占用率) 分为系统CPU占用率、任务CPU占用率和中断CPU占用率。

系统CPU占用率是指周期时间内系统的CPU占用率，用于表示系统一段时间内的闲忙程度，也表示CPU的负载情况。

任务CPU占用率指单个任务的CPU占用率，用于表示单个任务在一段时间内的闲忙程度。

中断CPU占用率指周期时间内的中断占用率，用于表示系统一段时间内的中断处理时间。

运作机制

Huawei LiteOS的CPUP采用任务级记录的方式，在任务切换时，记录任务的切入时间，在下一次任务切换时，记录任务的切出时间，以此计算出任务本次的运行时间，累加到任务的总占用时间上。

CPU占用率的计算方法：

系统CPU占用率=系统中除idle任务外其他任务运行总时间/系统运行总时间。

任务CPU占用率=任务运行总时间/系统运行总时间。

中断CPU占用率=中断运行总时间/系统运行总时间

4.5.2 开发指导

使用场景

通过系统CPU占用率，判断当前系统负载是否超出设计规格。

通过系统中各个任务的CPU占用率，判断各个任务的CPU占用率是否符合设计的预期。

功能

Huawei LiteOS的CPU占用率模块为用户提供下面几种功能，接口详细信息可以查看API参考。

表 4-6 功能列表

功能分类	接口名	描述
获取系统CPU占用率	LOS_HistorySysCpuUsage	获取系统CPU占用率，不包含idle任务。
获取任务CPU占用率	LOS_HistoryTaskCpuUsage	获取指定任务的CPU占用率。
	LOS_AllCpuUsage	使能LOSCFG_CPUP_INCLUDE_IRQ且设置入参flag为0时，获取所有中断的CPU占用率。设置入参flag为非0，或者关闭LOSCFG_CPUP_INCLUDE_IRQ后，获取所有任务的CPU占用率，这里的任务也包含了idel任务。
重置CPU占用率	LOS_CpupReset	重置CPU占用率数据，包含系统和任务的CPU占用率。

功能分类	接口名	描述
暂停CPU占用率统计	LOS_CpupStop	暂停CPU占用率统计，包含中断和任务的CPU占用率不再更新。
开启CPU占用率统计	LOS_CpupStart	开启CPU占用率统计，包含中断和任务的CPU占用率将先清0，然后以此为起点，重新统计。

说明

- 通过上述接口获取到的CPU占用率是千分比，所以CPU占用率的有效表示范围为0~1000。系统CPU占用率为1000，表示系统满负荷运转。任务CPU占用率为1000，表示在一段时间内系统一直在运行该任务。
- 获取CPU占用率有三种模式，通过入参mode设置：
 - CPUP_LAST_MULTIRECORD（值为0，原为CPUP_LAST_TEN_SECONDS）：表示获取最近多次采样记录内的CPU占用率，默认为10s。
 - CPUP_LAST_ONE_RECORD（值为1，原为CPUP_LAST_ONE_SECONDS）：表示获取最近1次采样记录的CPU占用率，默认为1s。
 - CPUP_ALL_TIME（值为0xffff），或除0和1之外的其他值：表示获取自系统启动以来的CPU占用率。

CPUP 错误码

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_CPUP_NO_MEMORY	0x02001e00	初始化CPU占用率模块时，内存不足。	调整OS_SYS_MEM_SIZE，以确保有足够的内存供CPU占用率模块使用。
2	LOS_ERRNO_CPUP_TASK_PTR_NULL	0x02001e01	入参存在空指针。	传入正确的指针。
3	LOS_ERRNO_CPUP_NO_INIT	0x02001e02	CPU占用率模块没有初始化。	初始化CPU占用率模块后，再使用该模块。
4	LOS_ERRNO_CPUP_MAXNUM_INVALID	0x02001e03	传入LOS_AllCpuUsage接口的入参（最大线程数或中断数）是一个无效值。	传入正确的最大线程数或中断数。
5	LOS_ERRNO_CPUP_THREAD_NO_CREATED	0x02001e04	获取指定任务的CPU占用率时，发现该任务未创建。	系统只能统计已经被创建的任务的CPU占用率，检查传入的任务ID对应的任务是否已经创建。
6	LOS_ERRNO_CPUP_TSK_ID_INVALID	0x02001e05	调用获取指定任务的CPU占用率的接口时，传入了无效的任务ID。	检查传入的任务ID的正确性。

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为CPUP模块，值为0x1e。

开发流程

CPU占用率的典型开发流程：

- 1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Cpus菜单，完成CPU占用率模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_CPUP	CPUP模块的裁剪开关。	YES/NO	YES	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_CPU_P_START_STOP	CPUP动态启停开关。	YES/NO	NO	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP
LOSCFG_CPU_P_INCLUDE_IRQ	使能该配置项后，可以在LOS_HistoryTaskCpuUsage和LOS_AllCpuUsage接口中获取中断的CPU占用率。关闭该配置项后，只能获取任务的CPU占用率。	YES/NO	YES	LOSCFG_KERNEL_CPUP && LOSCFG_ARCH_INTERRUPT_TAKEOVER
LOSCFG_CPU_P_CB_NUM_CONFIGURABLE	使能该配置项，CPUP的中断相关控制块的资源数量可单独配置，不再依赖LOSCFG_PLATFORM_HWI_LIMIT，可优化CPUP的size。	YES/NO	NO	LOSCFG_CPUP_INCLUDE_IRQ
LOSCFG_CPU_P_IRQ_CB_NUM	CPUP的中断相关控制块的资源数量，根据实际的中断个数来配置，最大不超过LOSCFG_PLATFORM_HWI_LIMIT。	[1, LOSCFG_PLATFORM_HWI_LIMIT]	LOSCFG_PLATFORM_HWI_LIMIT	LOSCFG_CPUP_CB_NUM_CONFIGURABLE

配置项	含义	取值范围	默认值	依赖
LOSCFG_CPUP_SAMPLE_PERIOD	CPUP的采样周期，时间单位是tick，表示CPUP软件定时器的周期运行时间间隔。	[10, 1000]，实际还会限制时间 [100ms, 10s]	LOSCFG_BASE_CORE_TICK_PERIOD_SECONDS	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP
LOSCFG_CPUP_HISTORY_RECORD_NUM	CPUP的最大采样记录数目，实际为环形缓冲buffer，表示用户最多可查看最近几次采样记录的CPU占用率信息。	[1,10]	10	LOSCFG_KERNEL_CPUP && ! LOSCFG_SCHED_LOAD_BALANCE_CPUP

2. 获取系统CPU使用率LOS_HistorySysCpuUsage。
3. 获取指定任务或中断的CPU使用率LOS_HistoryTaskCpuUsage。
4. 获取所有任务或所有中断的CPU使用率LOS_AllCpuUsage。

平台差异性

无。

4.5.3 注意事项

- CPU占用率对性能有一定影响，而一般只有在产品开发时需要了解各个任务的占用率，因此建议在发布产品时，关闭CPU占用率。
- 关闭配置项LOSCFG_CPUP_INCLUDE_IRQ后，系统中的中断耗时会被统计到中断发生的任务中，即被中断打断的任务中。

4.5.4 编程实例

实例描述

本实例实现如下功能：

1. 创建一个测试CPUP的任务。
2. 获取系统最近1s内所有任务或中断的CPUP。
3. 获取系统（除idle任务外）最近10s内的总CPU占用率。
4. 获取CPUP测试任务的CPUP。

编程示例

前提条件：打开菜单配置好CPU占用率模块。

代码实现如下：

```
#include <unistd.h>
#include "los_task.h"
#include "los_cpup.h"

#define MAXTASKNUM 32

UINT32 cpupUse;
UINT32 g_cpuTestTaskId;

VOID Example_CPUP(VOID)
{
    printf("entry cpup test example\n");
    while(1) {
        usleep(100);
    }
}

UINT32 Example_CPUP_Test(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S cpupTestTask;
    CPUP_INFO_S cpupInfo[MAXTASKNUM];

    /* 创建测试CPUP的任务 */
    memset(&cpupTestTask, 0, sizeof(TSK_INIT_PARAM_S));
    cpupTestTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_CPUP;
    cpupTestTask.pcName = "TestCpupTsk"; /* 测试任务名称 */
    cpupTestTask.uwStackSize = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    cpupTestTask.usTaskPrio = 5;
    cpupTestTask.uwResved = LOS_TASK_STATUS_DETACHED;

    ret = LOS_TaskCreate(&g_cpuTestTaskId, &cpupTestTask);
    if(ret != LOS_OK) {
        printf("cpupTestTask create failed.\n");
        return LOS_NOK;
    }
    usleep(100);

    /* 系统中运行的任务或者中断数量 */
    UINT16 maxNum = MAXTASKNUM;

    /* 获取系统所有任务或中断最近1s的CPU占用率 */
    cpupUse = LOS_AllCpuUsage(maxNum, cpupInfo, CPUP_LAST_ONE_SECONDS, 0);
    printf("the system cpu usage in last 1s: %d\n", cpupUse);

    /* 获取最近10s内系统（除idle任务外）总CPU占用率 */
    cpupUse = LOS_HistorySysCpuUsage(CPUP_LAST_TEN_SECONDS);
    printf("the history system cpu usage in last 10s: %d\n", cpupUse);

    /* 获取指定任务在最近1s的CPU占用率，该测试例程中指定的任务为上面创建的CPUP测试任务 */
    cpupUse = LOS_HistoryTaskCpuUsage(g_cpuTestTaskId, CPUP_LAST_ONE_SECONDS);
    printf("cpu usage of the cpupTestTask in last 1s:\n TaskID: %d\n usage: %d\n", g_cpuTestTaskId,
        cpupUse);

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
the system cpu usage in last 1s: 15
the history system cpu usage in last 10s: 3
cpu usage of the cpupTestTask in last 1s:
TaskID: 10
usage: 0
```

完整实例代码

sample_cpup.rar

4.6 Trace

4.6.1 概述

基本概念

Trace即追踪，实时记录系统行为，类似系统“录像”功能。在系统发生异常后，能辅助用户查看历史事件，定位问题。

运作机制

Huawei LiteOS的Trace采用静态代码打桩和缓冲区记录方式，在桩被执行时，获取事件发生的上下文，并写入到缓冲区。

在插桩函数的入参中需要提供追踪的事件类型、事件操作的主体对象和事件的参数，这些信息会被写入缓冲区，同时缓冲区中也会记录事件发生的时间、系统中的任务信息等事件上下文信息。

4.6.2 开发指导

使用场景

- 通过Trace了解系统运转的轨迹，理解系统运行机制。
- 通过Trace分析系统发生异常前的操作，定位死机问题。

功能

Huawei LiteOS的Trace模块为用户提供下面几种功能，接口详细信息可以查看API参考。

表 4-7 功能列表

功能分类	接口名	描述
配置Trace缓冲区	LOS_TraceInit	配置Trace缓冲区的地址和大小
开启/停止Trace事件记录	LOS_TraceStart	开启事件记录
	LOS_TraceStop	停止事件记录
操作Trace记录的数据	LOS_TraceRecordDump	输出Trace缓冲区数据
	LOS_TraceRecordGet	获取Trace缓冲区的首地址
	LOS_TraceReset	清除Trace缓冲区中的事件

功能分类	接口名	描述
过滤Trace记录的模块	LOS_TraceEventMaskSet	设置事件掩码，仅记录某些模块的事件
屏蔽特定中断号事件	LOS_TraceHwiFilterHookReg	注册过滤特定中断号事件的钩子函数
插桩函数	LOS_TRACE_EASY	简易插桩
	LOS_TRACE	标准插桩

说明

1. `LOS_TRACE_EASY(TYPE, IDENTITY, params...)` 简易插桩。
 - 一句话插桩，用户在目标源代码中插入该接口即可。
 - `TYPE`有效取值范围为[0, 0xF]，表示不同的事件类型。
 - `IDENTITY`类型`UINTPTR`，表示事件操作的主体对象。
 - `Params`类型`UINTPTR`，表示事件的参数。
 - 示例：


```
LOS_TRACE_EASY(1, userId0, userParam1, userParam2);
LOS_TRACE_EASY(2, userId0);
LOS_TRACE_EASY(1, userId1, userParam1);
LOS_TRACE_EASY(2, userId1);
LOS_TRACE_EASY(3, userParam1, userParam2, userParam3); // params 可复用IDENTITY字段
(当IDENTITY不必要时)
```
2. `LOS_TRACE(TYPE, IDENTITY, params...)` 标准插桩。
 - 相比简易插桩，支持动态过滤事件和参数裁剪，但使用上需要用户按规则来扩展。
 - `TYPE`用于设置具体的事件类型，可以在头文件`los_trace.h`中的`enum LOS_TRACE_TYPE`中自定义事件类型。定义方法和规则可以参考其他事件类型。
 - `IDENTITY`和`Params`的类型及含义同简易插桩。
 - 示例：
 - 1.在`enum LOS_TRACE_MASK`中定义事件掩码，即模块级别的事件类型。定义规范为`TRACE_#MOD#_FLAG`，`#MOD#`表示模块名，例如：


```
TRACE_FS_FLAG = 0x2000
```
 - 2.在`enum LOS_TRACE_TYPE`中定义具体事件类型。定义规范为`#TYPE# = TRACE_#MOD#_FLAG | NUMBER`，例如：


```
FS_READ = TRACE_FS_FLAG | 0; // 读文件
FS_WRITE = TRACE_FS_FLAG | 1; // 写文件
```
 - 3.定义事件参数。定义规范为`#TYPE#_PARAMS(IDENTITY, parma1...) IDENTITY, ...`其中的`#TYPE#`就是上面2中的`#TYPE#`，例如：


```
#define FS_READ_PARAMS(fp, fd, flag, size) fp, fd, flag, size
```

 宏定义的参数对应于Trace缓冲区中记录的事件参数，用户可对任意参数字段进行裁剪；当定义为空时，表示不追踪该类型事件：


```
#define FS_READ_PARAMS(fp, fd, flag, size) // 不追踪文件读事件
```
 - 4.在适当位置插入代码桩。定义规范为`LOS_TRACE(#TYPE#, #TYPE#_PARAMS(IDENTITY, parma1...))`

```
LOS_TRACE(FS_READ, fp, fd, flag, size); // 读文件的代码桩，#TYPE#之后的入参就是上面3中的FS_READ_PARAMS函数的入参。
```
3. Huawei LiteOS预置的Trace事件及参数均可以通过上述方式进行裁剪，参数详见`kernel\include\los_trace.h`。
4. Trace Mask事件过滤接口`LOS_TraceEventMaskSet(UINT32 mask)`，其入参`mask`仅高28位生效（对应`LOS_TRACE_MASK`中某模块的使能位），仅用于模块的过滤，暂不支持针对某个特定事件的细粒度过滤。例如：`LOS_TraceEventMaskSet(0x202)`，则实际设置生效的`mask`为`0x200`（`TRACE_QUE_FLAG`），`QUE`模块的所有事件均会被采集。一般建议使用的方法为：


```
LOS_TraceEventMaskSet(TRACE_EVENT_FLAG | TRACE_MUX_FLAG | TRACE_SEM_FLAG | TRACE_QUE_FLAG);
```
5. 如果仅需要过滤简易插桩事件，通过设置Trace Mask为`TRACE_MAX_FLAG`即可。
6. Trace缓冲区大小有限，事件写满之后会覆盖写，用户可通过`LOS_TraceRecordDump`中打印的`CurEvtIndex`识别最新事件。

Trace 错误码

为快速定位可能导致Trace操作失败的原因，初始化Trace和启动Trace均需要返回对应的错误码。其他无返回值的接口如停止Trace、清除与Dump Trace 数据，错误原因均为Trace状态不合法，系统会直接打印错误原因。

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_TRACE_ERROR_STATUS	0x02001400	初始化Trace或启动Trace时状态不正确。	- 删除重复的Trace初始化 - 启动Trace前先初始化Trace
2	LOS_ERRNO_TRACE_NO_MEMORY	0x02001401	初始化Trace时，缓冲区申请失败。	解决方案有两个： - 配置 LOSCFG_TRACE_BUFFER_SIZE减小Trace缓冲区的大小，可以通过menuconfig菜单项修改：Kernel --> Enable Extend Kernel --> Enable Trace Feature --> Trace work mode (Offline mode) --> Trace record buffer size - 配置 LOSCFG_BASE_CORE_TSK_LIMIT减少最大任务数，可以通过menuconfig菜单项修改：Kernel --> Basic Config --> Task Max Task Number
3	LOS_ERRNO_TRACE_BUF_TOO_SMALL	0x02001402	初始化Trace时，缓冲区size设置过小。	配置 LOSCFG_TRACE_BUFFER_SIZE增大Trace缓冲区的大小，menuconfig菜单项为：Kernel --> Enable Extend Kernel --> Enable Trace Feature --> Trace work mode (Offline mode) --> Trace record buffer size
4	LOS_ERRNO_TRACE_BUF_IS_NULL	0x02001403	在离线模式下初始化Trace，且动态内存关闭，指定缓冲区为NULL。	有如下解决方案： - 初始化Trace时传入有效的缓冲区地址 - 开启动态内存 - 使用在线模式

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为Trace模块，值为0x14。

开发流程

Trace的典型开发流程：

1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Trace Feature菜单，完成Trace的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_TRACE	Trace模块的裁剪开关	YES/NO	NO	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_RECORDER_MODE_OFFLINE	Trace工作模式为离线模式	YES/NO	YES	LOSCFG_KERNEL_TRACE
LOSCFG_RECORDER_MODE_ONLINE	Trace工作模式为在线模式	YES/NO	NO	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_CLIENT_INTERACT	使能与Trace IDE（Huawei LiteOS Studio）的交互，包括数据可视化和流程控制	YES/NO	NO	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_PIPELINE_SERIAL	选择串口作为IDE数据可视化通道	YES/NO	YES	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_CONTROL_VIA_SHELL	选择shell作为IDE流程控制的方式	YES/NO	YES	LOSCFG_TRACE_CLIENT_INTERACT && LOSCFG_SHELL
LOSCFG_TRACE_CONTROL_AGENT	选择agent作为IDE流程控制的方式	YES/NO	NO	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_NO_CONTROL	不启用IDE的流程控制	YES/NO	NO	LOSCFG_TRACE_CLIENT_INTERACT
LOSCFG_TRACE_MSG_EXTEND	记录系统更多的扩展信息	YES/NO	NO	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_FRAME_CORE_MSG	记录CUID、中断状态、锁任务状态	YES/NO	NO	LOSCFG_TRACE_MSG_EXTEND

配置项	含义	取值范围	默认值	依赖
LOSCFG_TRACE_FRAME_EVENT_COUNT	记录事件的次序编号	YES/NO	NO	LOSCFG_TRACE_MSG_EXTEND
LOSCFG_TRACE_FRAME_MAX_PARAMS	配置记录事件的最大参数个数	INT	3	LOSCFG_KERNEL_TRACE
LOSCFG_TRACE_BUFFER_SIZE	配置Trace的缓冲区大小	INT	2048	LOSCFG_RECORDER_MODE_OFFLINE

- 2. (可选)预置事件参数和事件桩（或使用系统默认的事件参数配置和事件桩）。
- 3. (可选)调用LOS_TraceStop停止Trace后，清除缓冲区LOS_TraceReset（系统默认已启动trace）。
- 4. (可选)调用LOS_TraceEventMaskSet设置需要追踪的事件掩码（系统默认的事件掩码仅使能中断与任务切换），事件掩码参见los_trace.h 中的LOS_TRACE_MASK定义。
- 5. 在需要记录事件的代码起始点调用LOS_TraceStart。
- 6. 在需要记录事件的代码结束点调用LOS_TraceStop。
- 7. 调用LOS_TraceRecordDump输出缓冲区数据（函数的入参为布尔型，FALSE表示格式化输出，TRUE表示输出到Windows客户端）。

 说明

上述第4-7步中的接口，均封装有对应的shell命令，对应关系如下，命令介绍参见[Trace命令参考](#)：

- LOS_TraceEventMaskSet —— trace_mask
- LOS_TraceStart —— trace_start
- LOS_TraceStop —— trace_stop
- LOS_TraceRecordDump —— trace_dump

平台差异性

无。

4.6.3 注意事项

由于Trace会影响系统性能，同时考虑到一般只有在产品开发时才需要了解系统发生的事件，因此建议在产品发布时关闭Trace。

4.6.4 编程实例

实例描述

本实例实现如下功能：

- 1. 创建一个用于Trace测试的任务。

2. 设置事件掩码。
3. 启动Trace。
4. 停止Trace。
5. 格式化输出Trace数据。

编程示例

前提条件：在menuconfig菜单中完成Trace模块的配置。

代码实现如下：

```
#include "los_trace.h"

UINT32 g_traceTestTaskId;
VOID Example_Trace(VOID)
{
    UINT32 ret;

    LOS_TaskDelay(10);

    /* 开启trace */
    ret = LOS_TraceStart();
    if (ret != LOS_OK) {
        dprintf("trace start error\n");
        return;
    }

    /* 触发任务切换的事件 */
    LOS_TaskDelay(1);

    LOS_TaskDelay(1);

    LOS_TaskDelay(1);

    /* 停止trace */
    LOS_TraceStop();

    LOS_TraceRecordDump(FALSE);
}

UINT32 Example_Trace_test(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S traceTestTask;

    /* 创建用于trace测试的任务 */
    memset(&traceTestTask, 0, sizeof(TSK_INIT_PARAM_S));
    traceTestTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Trace;
    traceTestTask.pcName = "TestTraceTsk"; /* 测试任务名称 */
    traceTestTask.uwStackSize = 0x800;
    traceTestTask.usTaskPrio = 5;
    traceTestTask.uwResved = LOS_TASK_STATUS_DETACHED;
    ret = LOS_TaskCreate(&g_traceTestTaskId, &traceTestTask);
    if (ret != LOS_OK) {
        dprintf("TraceTestTask create failed .\n");
        return LOS_NOK;
    }

    /* 系统默认情况下已启动trace, 因此可先关闭trace, 然后清除缓存区后, 再重启trace */
    LOS_TraceStop();
    LOS_TraceReset();

    /* 开启任务模块事件记录 */
    LOS_TraceEventMaskSet(TRACE_TASK_FLAG);

    return LOS_OK;
}
```


结果验证

编译运行得到的结果为：

```
*****TraceInfo begin*****
clockFreq = 50000000
CurEvtIndex = 7
Index  Time(cycles)  EventType  CurTask  Identity  params
0      0x366d5e88    0x45      0x1     0x0     0x1f    0x4    0x0
1      0x366d74ae    0x45      0x0     0x1     0x0     0x8    0x1f
2      0x36940da6    0x45      0x1     0xc     0x1f    0x4    0x9
3      0x3694337c    0x45      0xc     0x1     0x9     0x8    0x1f
4      0x36eea56e    0x45      0x1     0xc     0x1f    0x4    0x9
5      0x36eec810    0x45      0xc     0x1     0x9     0x8    0x1f
6      0x3706f804    0x45      0x1     0x0     0x1f    0x4    0x0
7      0x37070e59    0x45      0x0     0x1     0x0     0x8    0x1f
*****TraceInfo end*****
```

输出的事件信息包括：发生时间、事件类型、事件发生在哪个任务中、事件操作的主体对象、事件的其他参数。

- Time(cycles)：表示事件发生的时间，值是时钟的cycle数。
- EventType：表示的具体事件可查阅头文件los_trace.h中的enum LOS_TRACE_TYPE。
- CurTask：表示当前运行在哪个任务中，值为taskId。
- Identity：表示事件操作的主体对象，可查阅头文件los_trace.h中的#TYPE#_PARAMS。
- params：表示的事件参数可查阅头文件los_trace.h中的#TYPE#_PARAMS。

下面以序号为0的输出项为例，进行说明。

Index	Time(cycles)	EventType	CurTask	Identity	params
0	0x366d5e88	0x45	0x1	0x0	0x1f 0x4 0x0

- Time(cycles)可换算成时间，换算公式为cycles/clockFreq，单位为s。
- 0x45为TASK_SWITCH事件，即任务切换事件，当前运行任务的taskId为0x1。
- Identity和params的含义需要查看TASK_SWITCH_PARAMS宏定义：
#define TASK_SWITCH_PARAMS(taskId, oldPriority, oldTaskStatus, newPriority, newTaskStatus) \ taskId, oldPriority, oldTaskStatus, newPriority, newTaskStatus

因为#TYPE#_PARAMS(IDENTITY, parma1...) IDENTITY, ..., 所以Identity为taskId (0x0)，第一个参数为oldPriority (0x1f) ...

说明

params的个数由menuconfig中Enable Trace Feature --> Record max params配置，默认为3，超出的参数不会被记录，用户应自行合理配置该值。

综上所述，任务由0x1切换到0x0，0x1任务的优先级为0x1f，状态为0x4，0x0任务的优先级为0x0。

完整实例代码

sample_trace.zip

4.7 LMS

4.7.1 概述

基本概念

LMS全称为Lite Memory Sanitizer，是一种实时检测内存操作合法性的调测工具。LMS能够实时检测缓冲区溢出（buffer overflow），释放后使用（use after free）和释放野指针（wild pointer），在异常发生的第一时间通知操作系统，结合操作系统Backtrace等定位手段，能准确定位到产生内存问题的代码行，极大提升内存问题定位效率。

运作机制

LMS使用影子内存映射标记系统内存的状态，一共可标记为三个状态：可读写，不可读写，已释放。影子内存存放在内存池的尾部。

- 内存从堆上申请后，会将数据区的影子内存设置为“可读写”状态，并将头结点区的影子内存设置为“不可读写”状态。
- 内存从堆上被释放时，会将被释放内存的影子内存设置为“已释放”状态。
- 编译代码时，会在代码中的读写指令前插入检测函数，对地址的合法性进行检验。主要是检测访问内存的影子内存的状态值，若检测到影子内存为不可读写，则会报溢出错误；若检测到影子内存为已释放，则会报释放后使用（Use After Free）错误。
- 在内存释放时，会检测被释放地址的影子内存状态值，若检测到影子内存非可读写，则会报重复释放（Double Free）错误。

功能

Huawei LiteOS的LMS模块为用户提供下面几种功能，接口详细信息可以查看API参考。

1. 支持多内存池检测。
2. 支持LiteOS所有内存算法的检测，包括bestfit、bestfit-little、slab、slab extend。
3. 支持LOS_MemAlloc、LOS_MemAllocAlign、LOS_MemRealloc申请出的内存检测。
4. 支持安全函数的访问检测。
5. 支持libc 高频函数的访问检测，包括：memset、memcpy、memmove、strcat、strcpy、strncat、strncpy。
6. 支持libc全量检测，可通过LMS的menuconfig选项LOSCFG_LMS_LIBC_FULL_CHECK开启。

可动态设置的功能如下表所示。

表 4-8 API 列表

功能分类	接口名	描述
添加指定内存池被检测	LOS_LmsAddCheckPool	将指定内存池的地址范围添加到LMS的内存检测链表上，当访问的地址在链表范围内时，LMS才进行合法性校验；且LOS_MemInit接口会调用该接口，默认将初始化的内存池挂入到检测链表中。
删除指定内存池不被检测	LOS_LmsDelCheckPool	不检测指定内存池地址范围内的合法性校验
使能指定内存段锁保护	LOS_LmsAddrProtect	为某段内存地址上锁，设置为不可读写，一旦访问则报错
去能指定内存段锁保护	LOS_LmsAddrDisableProtect	为某段内存地址解锁，设置为可读写

7. 可裁剪的功能如下：

打开菜单，进入Debug ---> Enable Lite Memory Sanitizer菜单，完成LMS的配置和裁剪。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_LMS	LMS模块的裁剪开关	YES/NO	NO	NA
LOSCFG_LMS_MAX_RECORD_POOL_NUM	LMS支持的检测内存池最大个数	INT	10	LOSCFG_KERNEL_LMS
LOSCFG_LMS_LOAD_CHECK	LMS内存读检测的裁剪开关	YES/NO	YES	LOSCFG_KERNEL_LMS
LOSCFG_LMS_STORE_CHECK	LMS内存写检测的裁剪开关	YES/NO	YES	LOSCFG_KERNEL_LMS
LOSCFG_LMS_CHECK_STRICT	LMS内存逐字节严格检测的裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LMS
LOSCFG_LMS_LIBC_FULL_CHECK	LMS使能全量libc检测的裁剪开关	YES/NO	NO	LOSCFG_KERNEL_LMS

📖 说明

1. 开启全量libc检测后libc库生成的镜像中text段的体积膨胀约17%。

SizeTable sorted by text+data			
liba_file	text	data	bss
total	453284	1818	1093745
libc.a	79820	26	647

SizeTable sorted by text+data			
liba_file	text	data	bss
total	466664	1818	1093745
libc.a	93200	26	647

2. 用户可配仅读检测或写检测，用于提升检测效率。
3. 用户可通过打开配置宏LOSCFG_LMS_CHECK_STRICT对内存地址进行逐字节的合法性校验，但是会降低检测性能，系统默认开启首尾字节检测。

4.7.2 使用指南

使用场景

- 疑似系统内存问题需要定位时。
- 开发过程中需要验证是否存在内存问题时。

使用流程

1. 在被检测模块的Makefile文件里，增加LMS检测编译选项-fsanitize=kernel-address。
2. 为避免编译器优化，通过menuconfig配置编译器不优化：
LOSCFG_COMPILER_OPTIMIZE_NONE=y 【 Compiler --> Optimize Option --> Optimize None 】
3. 为输出Backtrace信息，需要开启配置项：
LOSCFG_BACKTRACE=y 【 Debug --> Enable Backtrace 】
4. 开启LMS配置项：
LOSCFG_KERNEL_LMS=y 【 Debug --> Enable Lite Memory Sanitizer 】
5. 重新编译，查看串口输出。如果检测到内存问题，会输出检测结果。LMS输出的检测信息类似下图所示：

```
[ERR] **** Kernel Address Sanitizer Error Detected Start ****
[ERR] Heap buffer overflow error detected 1
[ERR] Illegal WRITE address at: [0x1206a8] 2
[ERR] Shadow memory address: [0xf20252 : 4] Shadow memory value: [2] 3
runTask->taskName = IT_TST_INI
runTask->taskId = 3 4
fp:0x0010cb3c
*****backtrace begin*****
traceback 0 -- lr = 0x218bc    fp = 0x10cb4c 5
traceback 1 -- lr = 0x2364c    fp = 0x10cb7c
traceback 2 -- lr = 0x237e8    fp = 0x10cb8c
traceback 3 -- lr = 0x21ea4    fp = 0x10cba4
traceback 4 -- lr = 0x139b4    fp = 0x10cbb4
traceback 5 -- lr = 0x12c1c    fp = 0x10cd1c
traceback 6 -- lr = 0x12b64    fp = 0x10cd2c
traceback 7 -- lr = 0x12a20    fp = 0x10cd4c
traceback 8 -- lr = 0x13a04    fp = 0x10cd64
traceback 9 -- lr = 0x1393c    fp = 0x10cd6c
traceback 10 -- lr = 0x11bf0    fp = 0x10cd74
traceback 11 -- lr = 0x11c30    fp = 0x10cd7c
traceback 12 -- lr = 0x11f04    fp = 0x10cd9c
traceback 13 -- lr = 0x1b200    fp = 0x10cdb4
traceback 14 -- lr = 0x39a30    fp = 0xb0b0b0b
*****backtrace end*****

[LMS] Dump info around address [0x 1206a8]:

[0x120628]: 0b 0b 0b 0b 30 9a 03 00 | [0xf2024a | 4]: 0 0
[0x120630]: 00 00 00 00 cb f9 ed ff | [0xf2024b | 0]: 2 2
[0x120638]: 03 00 00 00 20 fe 11 00 | [0xf2024b | 4]: 2 2
[0x120640]: 74 00 00 80 00 00 00 00 | [0xf2024c | 0]: 2 0
[0x120648]: 00 00 00 00 00 00 00 00 | [0xf2024c | 4]: 0 0
[0x120650]: 00 00 00 00 00 00 00 00 | [0xf2024d | 0]: 0 0
[0x120658]: 00 00 00 00 00 00 00 00 | [0xf2024d | 4]: 0 0
[0x120660]: 00 00 00 00 00 00 00 00 | [0xf2024e | 0]: 0 0
[0x120668]: 00 00 00 00 00 00 00 00 | [0xf2024e | 4]: 0 0
[0x120670]: 00 00 00 00 00 00 00 00 | [0xf2024f | 0]: 0 0
[0x120678]: 00 00 00 00 00 00 00 00 | [0xf2024f | 4]: 0 0
[0x120680]: 00 00 00 00 00 00 00 00 | [0xf20250 | 0]: 0 0
[0x120688]: 00 00 00 00 00 00 00 00 | [0xf20250 | 4]: 0 0
[0x120690]: 00 00 00 00 00 00 00 00 | [0xf20251 | 0]: 0 0
[0x120698]: 00 00 00 00 00 00 00 00 | [0xf20251 | 4]: 0 0
[0x1206a0]: 00 00 00 00 00 00 00 00 | [0xf20252 | 0]: 0 0
[0x1206a8]: [30] d4 0f 00 30 d4 0f 00 | [0xf20252 | 4]: [2] 2 6
[0x1206b0]: 34 06 12 00 48 d8 df 00 | [0xf20253 | 0]: 2 2
[0x1206b8]: 00 00 00 00 00 00 00 00 | [0xf20253 | 4]: 3 3
```

LMS检测信息包含下述几类信息：

- 检测到的内存问题，示例图中为访问越界Heap buffer overflow。
- 发生问题的内存地址，示例图中为0x1206a8，操作类型为非法写。
- 发生问题的内存地址对应的影子内存及该影子内存的值，示例图中影子内存地址为0xf20252，字节内偏移为4，该地址的值为2，2为不可读写（0为可读写、3为已释放）。
- 当前运行的任务，示例图中为IT_TST_INI，任务ID为3。
- 发生问题时的backtrace回溯栈（backtrace begin~ backtrace end），调用顺序为tracetrack 14->tracetrack 13->...->tracetrack 0，每一层的调用函数地址为lr的值，通过查阅asm文件，可定位到具体的函数。

- f. 发生问题的内存地址前后内存的取值，及对应影子内存的取值。

平台差异性

无。

4.7.3 注意事项

不支持项：

- LMS不支持栈上内存、全局变量的溢出检测，需要编译器支持。

其他限制：

- LMS属于调测特性，在产品发布时，需关闭LMS模块的裁剪开关 LOSCFG_KERNEL_LMS，删除LMS检测编译选项-fsanitize=kernel-address，恢复为开启LMS特性打开的其他配置项，参见[使用流程](#)。
- 如果被检测的任务有大量堆内存读写操作，需要增加任务栈大小。
- 内存模块和LMS模块不需要增加LMS检测编译选项-fsanitize=kernel-address。

4.8 Perf

4.8.1 概述

基本概念

Perf为性能分析工具，依赖PMU（Performance Monitoring Unit）对采样事件进行计数和上下文采集，统计出其算法的热点（hot spot）分布和热路径（hot path）。

运作机制

基于事件采样原理，以性能事件为基础，当事件发生时，相应的事件计数器溢出发生中断，在中断处理函数中记录事件信息，包括当前的PC、当前运行的任务ID以及调用栈等信息。

4.8.2 开发指导

使用场景

通过Perf查找性能瓶颈和定位热点代码，更高效地优化系统性能。

功能

Huawei LiteOS的Perf模块为用户提供下面几种功能，接口详细信息可参见API参考。

表 4-9 功能列表

功能分类	接口名	描述
初始化Perf	LOS_PerfInit	初始化PMU以及采样缓冲区

功能分类	接口名	描述
开启/停止Perf采样	LOS_PerfStart	开启采样
	LOS_PerfStop	停止采样
配置Perf采样事件	LOS_PerfConfig	配置采样事件的类型、周期等
读取采样数据	LOS_PerfDataRead	读取采样数据到指定地址
注册采样数据缓冲区的钩子函数	LOS_PerfNotifyHookReg	注册缓冲区水线到达的处理钩子
	LOS_PerfFlushHookReg	注册缓冲区刷cache的钩子
去初始化Perf	LOS_PerfDeinit	去初始化PMU

⚠ 注意

采样数据缓冲区为环形buffer，buffer中读过的区域可以覆盖写，未被读过的区域不能被覆盖写。

Perf 错误码

为快速定位可能导致Perf操作失败的错误原因，Perf初始化和Perf采样配置接口均需要返回对应的错误码。具体描述如下表所示：

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_PERF_STATUS_INVALID	0x02002000	初始化Perf或配置采样事件时的状态不正确	禁止重复初始化Perf，只能在Perf处于停止状态时才能配置采样事件。
2	LOS_ERRNO_PERF_HW_INIT_ERROR	0x02002001	初始化Perf时硬件PMU初始化失败	确认平台是否具备硬件PMU单元，以及PMU的中断号是否正确。
3	LOS_ERRNO_PERF_TIMED_INIT_ERROR	0x02002002	初始化Perf时高精度Hrtimer PMU初始化失败	确认平台是否具备高精度定时器单元。
4	LOS_ERRNO_PERF_SW_INIT_ERROR	0x02002003	初始化Perf时软件PMU初始化失败	确认是否开启软件PMU功能。

序号	定义	实际数值	描述	参考解决方案
5	LOS_ERRNO_PERF_BUF_ERROR	0x02002004	初始化Perf时buffer初始化失败	配置的Perf buffer size过大或过小，可通过修改los_config.h中LOS_PERF_BUFFER_SIZE解决。
6	LOS_ERRNO_PERF_INVALID_PMU	0x02002005	配置采样事件的PMU未注册	各类PMU可通过menuconfig进行选配，确保对应的PMU已开启。
7	LOS_ERRNO_PERF_PMU_CONFIG_ERROR	0x02002006	配置采样事件的事件参数不合法	检查LOS_PerfConfig中事件类型、事件ID、事件周期是否合法。 事件类型见enum PerfEventType，包括： • 硬件事件 事件ID见enum PmuHwId； 周期范围 [0x101,0xFFFFFFFF]； (注：armv8-m架构 cycle counter周期范围 [0x101,0xFFFFFFFF]，event counter周期范围 [0x101,0xFFFF]) • 软件事件 事件ID见enum PmuSwId； 周期范围>0； • 周期事件 事件ID见enum PmuTimedId； 周期范围>0；
8	LOS_ERRNO_PERF_CONFIG_NULL	0x02002007	配置采样事件的入参为空指针	LOS_PerfConfig入参不能为空指针。
9	LOS_ERRNO_PERF_CONFIG_TASK_FILTER_ERROR	0x02002008	配置任务白名单失败	检查拷贝白名单时memcpy_s入参是否正确。
10	LOS_ERRNO_PERF_INIT_NO_MEMORY	0x02002009	初始化Perf时内存不足	检查当前系统内存剩余或者关闭分任务统计功能以减少内存使用。

须知

更多错误码定义信息，请参见[错误码简介](#)。8~15位的所属模块为Perf模块，值为0x20。

开发流程

Perf的典型开发流程：

1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Perf Feature菜单，完成Perf的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_PERF	Perf模块的裁剪开关	YES/NO	NO	LOSCFG_KERNEL_EXTKERNEL
LOSCFG_PERF_CALC_TIME_BY_TICK	Perf模块使用Tick作为计时单位	YES/NO	YES	LOSCFG_KERNEL_PERF
LOSCFG_PERF_CALC_TIME_BY_CYCLE	Perf模块使用系统Cycle作为计时单位	YES/NO	NO	LOSCFG_KERNEL_PERF
LOSCFG_PERF_HW_PMU	使能硬件PMU采样事件	YES/NO	NO	LOSCFG_KERNEL_PERF&& LOSCFG_ARCH_PMU
LOSCFG_PERF_TIMED_PMU	使能高精度周期PMU采样事件	YES/NO	NO	LOSCFG_KERNEL_PERF && LOSCFG_COMPAT_LINUX_HRTIMER
LOSCFG_PERF_SW_PMU	使能软件PMU采样事件	YES/NO	YES	LOSCFG_KERNEL_PERF
LOSCFG_KERNEL_PERF_PER_TASK	Perf分任务统计模式开关	YES/NO	NO	LOSCFG_KERNEL_PERF
LOSCFG_KERNEL_PERF_SEPERATE_IRQ	Perf独立中断统计模式开关	YES/NO	NO	LOSCFG_KERNEL_PERF && LOSCFG_ARCH_INTERRUPT_TAKEOVER

2. 调用LOS_PerfConfig配置需要采样的事件。
Perf提供2种模式的配置， 及3大类型的事件配置：

- 2种模式：计数模式（仅统计事件发生次数）、采样模式（收集上下文如任务ID、PC、backtrace等）。
 - 3种事件类型：CPU硬件事件（cycle、branch、icache、dcache等）、OS软件事件（task switch、mux pend、irq等）、高精度周期事件（cpu clock）。
3. 在需要采样的代码起始点调用LOS_PerfStart。
 4. 在需要采样的代码结束点调用LOS_PerfStop。
 5. 调用输出缓冲区数据的接口LOS_PerfDataRead读取采样数据，并使用LiteOS Studio工具解析（单击调测工具里的性能分析tab页签）。

平台差异性

针对类型为硬件事件的采样，其依赖PMU硬件单元，且具有平台差异性。目前Huawei LiteOS Perf支持armv7、armv8、risc-v、x86_64与LingLong架构的PMU。

4.8.3 注意事项

Perf用于性能调测，对系统性能有一定影响，建议在产品发布时，关闭Perf模块的裁剪开关LOSCFG_KERNEL_PERF。

4.8.4 编程实例

实例描述

本实例实现如下功能：

1. 配置采样事件。
2. 启动Perf。
3. 执行需要统计的算法。
4. 停止Perf。
5. 输出统计结果。

编程示例

前提条件：在menuconfig菜单中完成Perf模块的配置。

打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Perf Feature，然后选中Enable Hardware Pmu Events for Sampling, Enable Perf Per-Task mode和Enable Perf Separated IRQ mode，使能PMU对硬件事件监测，并使能perf分任务统计以及中断独立统计。

代码实现如下：

```
#include "los_perf.h"
#include "los_memory.h"

#define TEST_PERF_BUFFER_SIZE 1024
#define OUTPUT_BUFFER_SIZE 0x200

#ifdef LOSCFG_KERNEL_MEM_ALLOC
STATIC CHAR g_buffer[TEST_PERF_BUFFER_SIZE] = {0};
#endif

STATIC VOID OsPrintBuff(const CHAR *buf, UINT32 num)
{
```

```

    UINT32 i;
    dprintf("num: ");
    for (i = 0; i < num; i++) {
        dprintf(" %02u", i);
    }
    dprintf("\n");
    dprintf("hex: ");
    for (i = 0; i < num; i++) {
        dprintf(" %02x", buf[i]);
    }
    dprintf("\n");
}

VOID PerfTestHWEvent(VOID)
{
    UINT32 ret;
    CHAR buf[OUTPUT_BUFFER_SIZE] = {0};
    UINT32 len;
    CHAR *buffer = NULL;

    PerfConfigAttr attr1 = {
        .eventsCfg = {
            .type = PERF_EVENT_TYPE_HW,
            .events = {
                [0] = {PERF_COUNT_HW_CPU_CYCLES, 0xFFFF},
                [1] = {PERF_COUNT_HW_BRANCH_INSTRUCTIONS, 0xFFFFF00},
            },
            .eventsNr = 2,
            .predivided = 1, /* cycle counter increase every 64 cycles */
        },
        .taskIds = {0},
        .taskIdsNr = 0,
        .needSample = 0,
        .sampleType = PERF_RECORD_IP | PERF_RECORD_CALLCHAIN,
    };
#ifdef LOSCFG_KERNEL_MEM_ALLOC
    buffer = (CHAR *)LOS_MemAlloc(m_aucSysMem1, TEST_PERF_BUFFER_SIZE);
#else
    buffer = g_buffer;
#endif

    ret = LOS_PerfInit(buffer, TEST_PERF_BUFFER_SIZE);
    if (ret != LOS_OK) {
        PRINT_ERR("perf init error %u\n", ret);
        return;
    }

    ret = LOS_PerfConfig(&attr1);
    if (ret != LOS_OK) {
        PRINT_ERR("perf config error %u\n", ret);
        return;
    }

    PRINTK("-----count mode-----\n");
    LOS_PerfStart(0);
    test(); /* code snippets concerned by users */
    LOS_PerfStop();

    PRINTK("-----sample mode----- \n");
    attr1.needSample = 1;
    LOS_PerfConfig(&attr1);
    LOS_PerfStart(2);
    test(); /* code snippets concerned by users */
    LOS_PerfStop();

    len = LOS_PerfDataRead(buf, OUTPUT_BUFFER_SIZE); /* get sample data */
    OsPrintBuff(buf, len); /* print data */
    memset(buf, 0, sizeof(buf));
#ifdef LOSCFG_KERNEL_MEM_ALLOC

```

```
(VOID)LOS_MemFree(m_aucSysMem1, buffer);
#endif
    LOS_PerfDeinit();
}
```

结果验证

编译运行得到的结果为：

```
-----count mode-----
[EMG] perf count [tid 0 / Swt_Task]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 1 / IdleCore000]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 2 / app_Task]:
[EMG] [cycles] eventType: 0xff count: 40043481344
[EMG] [branches] eventType: 0xc count: 10002153167
[EMG] perf count [tid 3 / Swt_Task]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 4 / IdleCore000]:
[EMG] [cycles] eventType: 0xff count: 100718720
[EMG] [branches] eventType: 0xc count: 5182967
[EMG] perf count [tid 5 / SerialShellTask]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [tid 6 / SerialEntryTask]:
[EMG] [cycles] eventType: 0xff count: 0
[EMG] [branches] eventType: 0xc count: 0
[EMG] perf count [irq@core 0]:
[EMG] [cycles] eventType: 0xff count: 891017
[EMG] [branches] eventType: 0xc count: 2948096
[EMG] perf count [irq@core 1]:
[EMG] [cycles] eventType: 0xff count: 889686
[EMG] [branches] eventType: 0xc count: 2948154
[EMG] time used: 44668000(us)
-----sample mode-----
[EMG] time used: 44673000(us)
num: 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 ...
hex: 00 ef ef ef 00 00 00 00 14 00 00 00 60 00 00 00 02 00 00 00 90 2d 20 80 02 00 00 00 d4 5f 20 80 54
55 21 80 90 2d 20 80 02 ...
```

针对计数模式，系统在perf stop后会打印如下信息：

- 当开启分任务统计功能后，会先打印每个任务的头信息，其后的事件计数为此任务的计数
perf count [tid 2 / app_Task]
任务ID(2)，任务名称(app_Task)
- 当关闭分任务统计功能后，会先打印每个核的头信息，其后的事件计数为此核的计数
perf count [core 0]
核ID(core 0)
- 当开启中断独立统计功能后，会先打印每个核中断的头信息，其后的事件计数为此核中断的计数
perf count [irq@core 0]
核ID(core 0)
- 事件计数的信息：
[cycles] eventType: 0xff count: 40043481344
事件名称(cycles)、事件类型(0xff)、事件发生的次数(40043481344)。

说明

当采样事件为硬件PMU事件时，打印的事件类型为实际的硬件事件ID，非enum PmuHwId中定义的抽象类型。硬件事件ID详见各架构手册规定。

- 针对采样模式，系统会将采样数据记录到用户指定的内存中，即LOS_PerfInit的入参。用户可以自行打印该内存的地址，然后通过JTAG口导出该片内存，再使用LiteOS Studio线下工具解析。用户也可以通过LOS_PerfDataRead将数据读到指定地址，进行查看或进一步处理。示例中OsPrintBuff为测试接口，其按字节打印Read到的采样数据，num表示第几个字节，hex表示该字节中的数值。

注意

- 在RISC-V架构中，使用采样模式需要在定时中断处理函数中调用OsPerfCheckEventCount()函数。一般使用Tick中断处理函数。
- Perf独立中断统计模式下，PMU中断不会被单独统计。另外，由于Timed PMU同样依赖于中断工作，计数可能与预期不一致。如果需要使用Timed PMU，请关闭Perf独立中断。

完整实例代码

sample_perf.zip

4.9 C++支持

4.9.1 概述

基本概念

C++作为目前使用最广泛的编程语言之一，支持类、封装、重载等特性，是在C语言基础上开发的一种面向对象的编程语言。

运作机制

STL（Standard Template Library）标准模板库，是一些“容器”的集合，也是算法和其他一些组件的集合。其目的是标准化组件，使用标准化组件后可以不用重新开发，直接使用现成的组件。

4.9.2 开发指导

功能

功能分类	接口名	描述
使用C++特性的前置条件	LOS_CppSystemInit	初始化C++构造函数

说明

该函数有3个入参：

- 第一个参数：init_array段的起始地址。
- 第二个参数：init_array段的结束地址。
- 第三个参数：标记调用C++特性时的场景，包括BEFORE_SCATTER（在分散加载快速启动阶段使用C++特性）、AFTER_SCATTER（在分散加载非快速启动阶段使用C++特性）、NO_SCATTER（在非分散加载特性中使用C++特性，或者在分散加载中不使用C++特性）。

开发流程

1. 打开菜单，选择Kernel ---> Enable Extend Kernel ---> C++ Support，使能C++。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_CPPSUPPORT	C++特性的裁剪开关	YES/ NO	YES	LOSCFG_KERNEL_EXTKERNEL

2. 使用C++特性之前，需要调用函数LOS_CppSystemInit，初始化C++构造函数。

由于在分散加载应用场景下，C++初始化中涉及到的相关代码、数据段加载的时机会有所不同，所以在开启和不开启分散加载这两种情况下，C++初始化函数LOS_CppSystemInit的调用有所不同。其中构造函数的初始化存在init_array这个段中，段区间通过变量__init_array_start、__init_array_end传递。

不开启分散加载特性

在不开启分散加载特性的情况下，需要在调用C++代码前，以NO_SCATTER（表示并未开启分散加载特性）参数调用LOS_CppSystemInit：

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, NO_SCATTER);
```

开启分散加载特性

- 如果需要在分散加载的快速启动阶段调用相关C++代码：

1. 在分散加载的快速启动阶段调用C++代码之前，以BEFORE_SCATTER（表示在分散加载快速启动阶段调用）参数调用LOS_CppSystemInit：

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, BEFORE_SCATTER);
```

2. 在分散加载的非快速启动阶段，再以AFTER_SCATTER（表示在分散加载非快速启动阶段调用）参数调用LOS_CppSystemInit：

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, AFTER_SCATTER);
```

- 如果无需在分散加载的快速启动阶段调用相关C++代码：

以NO_SCATTER参数调用一次LOS_CppSystemInit：

```
LOS_CppSystemInit((unsigned long)&__init_array_start, (unsigned long)&__init_array_end, NO_SCATTER);
```

3. C函数与C++函数混合调用。

在C++中调用C程序的函数，代码需加入C++包含的宏：

```
#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */
```

```
/* code */
...
#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */
```

4.9.3 注意事项

- Huawei LiteOS的C++功能需要编译器适配才能支持，编译器编译链接C++代码时需要使用Huawei LiteOS提供的C库。
- C++的cout函数依赖console功能。

4.9.4 编程实例

实例描述

本实例实现了在Huawei LiteOS上运行C++的代码。

1. 编写C++代码。
2. 在运行C++代码之前，先在app_init函数里初始化C++构造函数。此处未开启分散加载特性，所以只需以NO_SCATTER参数调用一次LOS_CppSystemInit即可。
3. 在LOS_CppSystemInit之后调用编写好的C++代码。

编程示例

前提条件：打开菜单使能C++支持。

C++代码实现如下：

```
#ifdef __cplusplus
#if __cplusplus
extern "C" {
#endif /* __cplusplus */
#endif /* __cplusplus */

using namespace std;

class TestClass {
public:
    TestClass(int arg);
    ~TestClass(void);
    void PrintTest(void);
    void StringTest(void);
    void MapTest(void);
private:
    int intTest;
    string stringTest;
    map<string, int> mapTest;
};

TestClass::TestClass(int arg)
{
    cout << "TestClass is constructed here, arg = " << arg << endl;
    intTest = arg;
}

TestClass::~~TestClass(void)
{
    cout << "TestClass is destructed" << endl;
}

void TestClass::PrintTest(void)
{
```

```

    cout << __FUNCTION__ << " enter" << endl;
    cout << " intTest = " << this->intTest << endl;
}
void TestClass::StringTest(void)
{
    cout << __FUNCTION__ << " enter" << endl;
    string a("Lite");
    string b("OS");
    string c("LiteOS");
    if (a != b) {
        cout << " " << a << " != " << b << endl;
    }
    a += b;
    if (a == c) {
        cout << " " << a << " == " << c << endl;
    }
}
void TestClass::MapTest(void)
{
    cout << __FUNCTION__ << " enter" << endl;
    mapTest.insert(pair<string, int>("Huawei", 1));
    mapTest.insert(pair<string, int>("LiteOS", 2));
    mapTest.insert(pair<string, int>("Open", 3));
    mapTest.insert(pair<string, int>("Source", 4));
    cout << " show map key&value" << endl;
    for (auto &it : mapTest) {
        cout << " " << it.first << " " << it.second << endl;
    }
    mapTest["LiteOS"] = 8; /* 8: new value */
    cout << " change value of \"LiteOS\" key" << endl;
    for (auto &it : mapTest) {
        cout << " " << it.first << " " << it.second << endl;
    }
}

void CppTestEntry(void)
{
    cout << "LiteOS cpp sample start" << endl;
    TestClass test(123);
    test.PrintTest();
    test.StringTest();
    test.MapTest();
    cout << "LiteOS cpp sample stop" << endl;
}

#ifdef __cplusplus
#if __cplusplus
}
#endif /* __cplusplus */
#endif /* __cplusplus */

```

app_init中初始化C++构造函数后，调用C++代码，实现如下：

```

void app_init(void)
{
    .....
    /* 初始化C++构造函数 */
    LOS_CppSystemInit((UINT32)&__init_array_start, (UINT32)&__init_array_end, NO_SCATTER);
    /* 调用C++代码 */
    CppTestEntry();
    .....
}

```

结果验证

运行CppTestEntry()函数，结果如下：

```

LiteOS cpp sample start
TestClass is constructed here, arg = 123

```



```
PrintTest enter
intTest = 123
StringTest enter
Lite != OS
LiteOS == LiteOS
MapTest enter
show map key&value
Huawei 1
LiteOS 2
Open 3
Source 4
change value of "LiteOS" key
Huawei 1
LiteOS 8
Open 3
Source 4
LiteOS cpp sample stop
TestClass is destructed
```

完整实例代码

TestClass.zip

4.10 动态共享内存

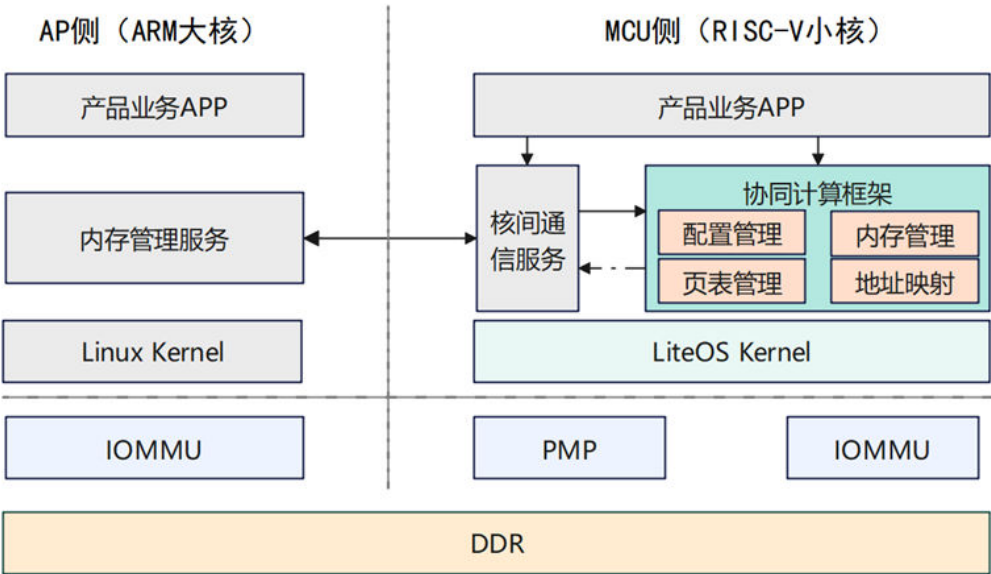
4.10.1 概述

基本概念

Huawei LiteOS基于IOMMU（Input-Output Memory Management Unit，输入输出内存管理单元）和PMP（Physical Memory Protection，物理内存保护单元）设计了AP侧（Android/Linux，ARM大核）和MCU侧（LiteOS，RISC-V小核）之间的动态共享内存管理框架，能够做到大小核数据交互零拷贝，提升性能。

运作机制

总体框架设计如下图所示：



共享DDR的实际物理地址空间由AP大核的内存管理服务（Memory Management Service，简称MMS）统一管理、分配。MCU上的业务有DDR共享内存需求时，通过核间通信服务向AP核的内存管理服务请求内存；MMS分配一块虚地址（iommuAddr），并在页表中建立起对应DDR物理页映射关系，经核间通信服务将内存地址和大小传递给协同计算框架的动态共享内存管理模块；动态共享内存管理模块在MCU的虚拟地址空间内分配空闲虚拟地址（virAddr）来映射MMS分配的物理地址。动态共享内存主要包括配置管理、地址映射、内存管理和页表管理，具体功能如下：

- a) 配置管理：动态共享内存管理模块的配置管理和初始化
- b) 地址映射：AP侧IOMMU虚拟地址到MCU侧虚拟地址的映射(map)与解映射(unmap)
- c) 内存管理：MCU侧虚拟内存和共享DDR物理内存的申请和释放
- d) 页表管理：AP侧IOMMU虚拟地址与MCU侧虚拟地址的相互转换、页表的重建和清除操作

4.10.2 开发指导

使用场景

- 大小核共享内存交互。
- 虚实地址转换。

功能

Huawei LiteOS的动态共享内存模块为用户提供下面几种功能，接口详细信息可以查看API参考。

表 4-10 功能列表

功能分类	接口名	描述
配置管理初始化	LOS_DynShmInit	动态共享内存模块初始化
地址映射	LOS_DynShmMap	AP侧虚拟地址->MCU侧虚拟地址建立映射
	LOS_DynShmUnmap	销毁MCU侧虚拟地址
地址转换	LOS_DynShmAddrTranslate	AP侧虚拟地址、MCU侧虚拟地址和内存物理地址总共3类地址之间的相互转换
内存管理，共享物理内存申请和释放	LOS_DynShmBulkMalloc	向AP侧批量申请共享DDR物理内存，并在MCU侧建立虚实映射
	LOS_DynShmBulkFree	向AP侧批量释放共享DDR物理内存，并在MCU侧销毁虚实映射

说明

- 1. LOS_DynShmInit接口用来初始化动态共享内存模块，重复初始化将会失败报错。
- 2. 只有在调用LOS_DynShmInit接口初始化模块后，才能调用其他API接口，否则将会失败。
- 3. LOS_DynShmBulkMalloc和LOS_DynShmBulkFree内部涉及到AP侧和MCU侧的核间通信，耗时较长，建议用户传入的size大小数值较大，达到一次性批量申请的目的。

动态共享内存错误码

序号	定义	实际数值	描述	参考解决方案
1	LOS_ERRNO_DYN_SHM_NO_MEMORY	0x02002300	没有足够的管理资源或者虚拟内存	调整配置项LOSCFG_KERNEL_DYN_SHM_VM_BLOCK_NUM或者增加虚拟内存空间大小
2	LOS_ERRNO_DYN_SHM_INPUT_INVALID	0x02002301	入参非法	核实传入参数正确性
3	LOS_ERRNO_DYN_SHM_INIT_FAILED	0x02002302	初始化失败	核实内部管理资源Mutex是否成功创建和PMP是否成功配置
4	LOS_ERRNO_DYN_SHM_ALREADY_MAPPED	0x02002303	重复映射	核实入参起始地址和大小，是否和以前冲突
5	LOS_ERRNO_DYN_SHM_INIT_REPEATEDLY	0x02002304	LOS_DynShmInit接口重复初始化	核实代码是否有多次调用可能性
6	LOS_ERRNO_DYN_SHM_INTERRUPT	0x02002305	在中断上半部调用	核实代码是否在中断上半部调用
7	LOS_ERRNO_DYN_SHM_NOT_READY	0x02002306	尚未初始化模块，就调用地址映射、地址转换或者内存分配释放接口	核实代码是否存在未初始化就调用情形

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为动态共享内存模块，值为0x23。

开发流程

动态共享内存的典型开发流程：

- 1. 打开菜单，进入Kernel ---> Enable Extend Kernel ---> Enable Dynamic Shared Memory菜单和Driver ---> Enable IOMMU完成动态共享内存模块的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_KERNEL_DYNSHM	动态共享内存模块总开关	YES/NO	NO	无
LOSCFG_KERNEL_DYNSHM_SECURE_ENBALE	安全访问是否支持	YES/NO	NO	硬件需要支持安全机制，LOSCFG_KERNEL_DYNSHM配置宏需要打开
LOSCFG_KERNEL_DYNSHM_VM_BLOCK_NUM	虚拟区域数量大小	正整数	1000	LOSCFG_KERNEL_DYNSHM
LOSCFG_DRIVERS_IOMMU	IOMMU驱动模块总开关	YES/NO	NO	LOSCFG_KERNEL_DYNSHM配置宏需要打开
LOSCFG_DRIVERS_HISI_CUSTOM_IOMMU	IOMMU hisi定制驱动	YES/NO	YES	LOSCFG_DRIVERS_IOMMU配置项需要打开
LOSCFG_DRIVERS_IOMMU_VA_BITS	IOMMU虚拟地址位数	36bit	36bit	LOSCFG_DRIVERS_IOMMU配置项需要打开

- 调用初始化接口LOS_DynShmInit完成模块初始化。
- 调用LOS_DynShmMap/LOS_DynShmUnmap接口，完成映射/解映射。
- 调用LOS_DynShmAddrTranslate接口可查询虚实地址转换关系。
- 调用LOS_DynShmBulkMalloc/LOS_DynShmBulkFree可申请/释放共享物理内存，当传入参数noNeedMap为FALSE时，还会进行虚实地址映射/解映射，否则仅仅申请/释放共享物理内存。

平台差异性

有，需要硬件支持IOMMU和RISC-V PMP，当前支持的平台是Hi3751。

4.10.3 注意事项

- 当前安全访问尚未支持，使用LOSCFG_KERNEL_DYNSHM_SECURE_ENBALE宏隔离，默认关闭，后续支持。
- 动态共享内存模块基于hisi定制IOMMU和RISC-V PMP实现，所以对具体硬件有要求。

4.11 核间通信

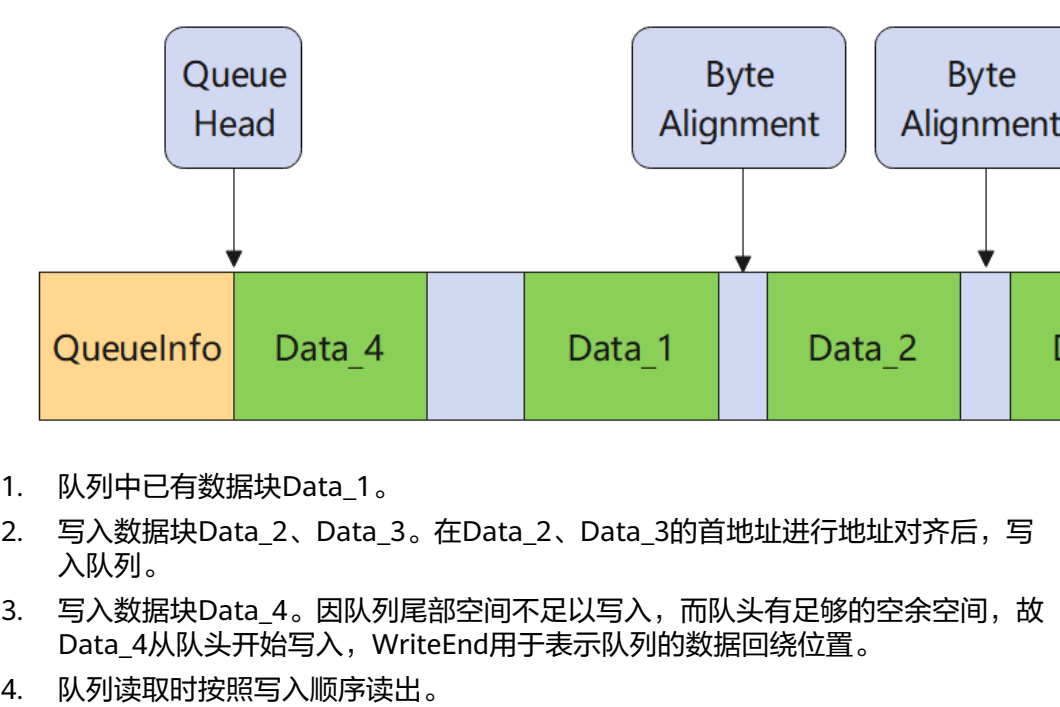
4.11.1 概述

基本概念

核间通信是用于CPU与CPU之间的通信方式，目的在于充分利用硬件资源和机制，实现高效的通信方式，给特定的应用提供简洁易用的API接口。

本模块采用循环队列来实现数据收发，保证性能与可靠性的同时，也可以最大程度的利用内存空间。通过连续地址写入和自动字节对齐，对齐大小按8字节对齐，数据接收端可以无需拷贝直接在原地址处理数据，且支持读和写同时进行。

运作机制



4.11.2 开发指导

表 4-11 功能列表

功能分类	接口名	描述
初始化	LOS_RpqueueInit	核间通信队列初始化
数据发送和接收	LOS_RpqueueSend	写数据到核间通信队列，数据整包写入到队列且地址连续
	LOS_RpqueueReceive	从核间通信队列读取数据，从数据接收回调传回的数据地址始终以8字节对齐
获取可发送长度	LOS_RpqueueMaxSendSizeGet	获取队列当前最大的可发送长度

说明

1. 请在数据发送端调用LOS_RpqueueInit进行队列初始化。
2. 同一个通信队列只允许一个线程进行数据接收，一旦对同一队列进行数据接收时发生重入，可能会导致程序崩溃。
3. LOS_RpqueueSend不支持多线程，多个线程向同一个队列写数据时需要配合互斥锁等任务间通信手段实现多线程操作。
4. 通信队列允许发送长度为0的数据包，所以LOS_RpqueueMaxSendSizeGet接口获取到可发送长度为0并非错误。

4.11.3 注意事项

- 用于核间通信的内存禁止使用cache，以免发生数据不一致。
- 核间通信模块无法校验地址合法性，请使用有效的共享内存地址进行初始化及读写操作。

4.11.4 编程实例

实例描述

本实例实现如下功能：

1. 核1从通道1接收数据，再将收到的数据，对cmd+1后从通道2发送出去。
2. 核2从通道1主动发送数据，再从通道2等待接收。

编程示例

前提条件：打开菜单使能核间通信模块。

代码实现如下：

核1代码：

```
#include "los_rpqueue.h"

STATIC UINT8 *g_rpqueueCh1 = (UINT8 *)0x38000000U;
STATIC UINT8 *g_rpqueueCh2 = (UINT8 *)0x38001000U;

STATIC VOID RpqueueSmapleRecv(UINT8 cmd, UINT8 *data, UINT16 len)
{
    LosRpqueueSendCfg queCfg;
    printf("Core 1 Rpqueue Smaple Receive cmd = 0x%02X, len = %d\n", cmd, len);
    queCfg.cmd = cmd + 1;
    queCfg.option = 0;
    queCfg.timeout = 0;
    LOS_RpqueueSend(g_rpqueueCh2, data, len, &queCfg);
}

UINT32 RpqueueSmapleCase(VOID)
{
    printf("Core 1 Rpqueue Smaple Start!\n");

    LOS_RpqueueInit(g_rpqueueCh2, 0x1000, 0);
    while (1) {
        LOS_RpqueueRecv(g_rpqueueCh1, RpqueueSmapleRecv);
    }
}
```

```
    return LOS_OK;
}
```

核2代码：

```
#include "los_rpqueue.h"

#define SAMPLE_SEND_LEN 3

STATIC UINT8 *g_rpqueueCh1 = (UINT8 *)0x38000000U;
STATIC UINT8 *g_rpqueueCh2 = (UINT8 *)0x38001000U;

STATIC VOID RpqueueSmapleRecv(UINT8 cmd, UINT8 *data,  UINT16 len)
{
    printf("Core 2 Rpqueue Smaple Receive cmd = 0x%02X, len = %d, data :", cmd, len);
    for (int i = 0; i < len; i++) {
        printf(" %02X", data[i]);
    }
    printf("\n");
}

UINT32 RpqueueSmapleCase(VOID)
{
    UINT32 ret;
    LosRpqueueSendCfg queCfg = {0};
    UINT8 data[SAMPLE_SEND_LEN] = {0x11, 0x22, 0x33};

    printf("Core 2 Rpqueue Smaple Start!\n");

    LOS_RpqueueInit(g_rpqueueCh1, 0x1000, 0);

    queCfg.cmd = 0x55;
    ret = LOS_RpqueueSend(g_rpqueueCh1, data, SAMPLE_SEND_LEN, &queCfg);
    if (ret != LOS_OK) {
        printf("Core 2 LOS_RpqueueSend Error!\n");
        return LOS_NOK;
    }

    do {
        if (LOS_RpqueueRecv(g_rpqueueCh2, RpqueueSmapleRecv) == LOS_OK) {
            break;
        }
    } while(1);
    printf("Core 2 Rpqueue Smaple END!\n");

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
核1:
Core 1 Rpqueue Smaple Start!
Core 1 Rpqueue Smaple Receive cmd = 0x55, len = 3
核2:
Core 2 Rpqueue Smaple Start!
Core 2 Rpqueue Smaple Receive cmd = 0x56, len = 3, data : 11 22 33
Core 2 Rpqueue Smaple END!
```

完整实例代码

Sample_Rpqueue_Core1.zip

Sample_Rpqueue_Core2.zip

5 文件系统

5.1 功能概述

5.2 VFS

5.3 NFS

5.4 JFFS2

5.5 FAT

5.6 YAFFS2

5.7 RAMFS

5.8 PROC

5.9 ROMFS

5.10 LITTLEFS

5.1 功能概述

本章以3556V200为例，介绍Huawei LiteOS目前支持的文件系统，包括VFS、NFS、JFFS2、FAT、YAFFS2、RAMFS、PROC、ROMFS和LITTLEFS。

各个文件系统功能概述

表 5-1 文件系统功能概述

文件系统	功能特点概述
VFS	VFS是Virtual File System（虚拟文件系统）的缩写。通过采用标准的Unix系统接口，VFS可以访问位于不同物理介质上的不同文件系统，为各类文件系统提供一个统一的操作方式。
NFS	NFS是Network File System（网络文件系统）的缩写。它可以通过网络，实现在不同的机器、不同的操作系统间的文件分享

文件系统	功能特点概述
JFFS2	JFFS2是Journalling Flash File System Version 2（日志文件系统）的缩写，属于日志型文件系统的一种，主要应用于NOR Flash闪存的文件管理。Huawei LiteOS的JFFS2支持多分区。
FAT	FAT文件系统是File Allocation Table（文件配置表）的简称，有FAT12、FAT16、FAT32。在可移动存储介质（U盘、SD卡、移动硬盘等）上使用FAT文件系统，可以使设备在Windows、Linux等桌面系统之间保持很好的兼容性。
YAFFS2	YAFFS是Yet Another Flash File System的简称，是一种针对Nand Flash的开源嵌入式文件系统，适用于大容量的存储设备，使得Nand Flash具有高效性和健壮性。 YAFFS2为文件系统提供了损耗平衡和掉电保护，保证在对文件系统修改的过程中，不会因发生意外而造成数据受损。Huawei LiteOS的YAFFS2支持多分区。
RAMFS	RAMFS（RAM File System）是基于RAM的动态文件系统的一种存储缓冲机制。RAMFS文件系统把所有的文件都放在RAM中，使得读/写操作发生在RAM中，既避免了对存储器的读写损耗，又提高了数据读写速度。
PROC	PROC文件系统是一种伪文件系统，只存在内存当中，而不占用外存空间。它以文件系统的方式为访问系统内核数据提供接口。
ROMFS	ROMFS（ROM File System）是一种只读文件系统，其功能比较简单，占用系统资源也比较小。ROMFS不支持文件的写入，只支持读取。设计它的目的是在启动盘等场合下，提供一个比普通文件系统更加节省空间的文件系统。
LITTLEFS	LITTLEFS（Little File System）是一种基于块的故障保护文件系统，使用小日志存储元数据，使用较大的写时复制（COW）结构存储文件数据，主要应用于NOR Flash闪存的文件管理。 LITTLEFS为文件系统提供了磨损平衡和掉电保护，保证在对文件系统修改的过程中，不会因发生意外而造成数据受损。

5.2 VFS

5.2.1 概述

基本概念

VFS是Virtual File System（虚拟文件系统）的缩写，是一个异构文件系统之上的软件粘合层，为用户提供统一的Unix文件操作接口。

由于不同类型的文件系统接口不统一，若系统中有多多个文件系统类型，访问不同的文件系统则需要使用不同的非标准接口。但是，通过在系统中添加VFS层来提供统一的抽象接口，便可以屏蔽底层异构类型的文件系统差异，使得访问文件系统时无需关注底层的存储介质和文件系统类型，从而提高开发效率。

Huawei LiteOS中，VFS框架通过在内存中的树结构实现，树的每个结点都是一个inode结构体。设备注册和文件系统挂载后会根据路径在树中生成相应的结点。

VFS的两大主要功能如下：

1. 查找节点。
2. 统一调用（标准）。

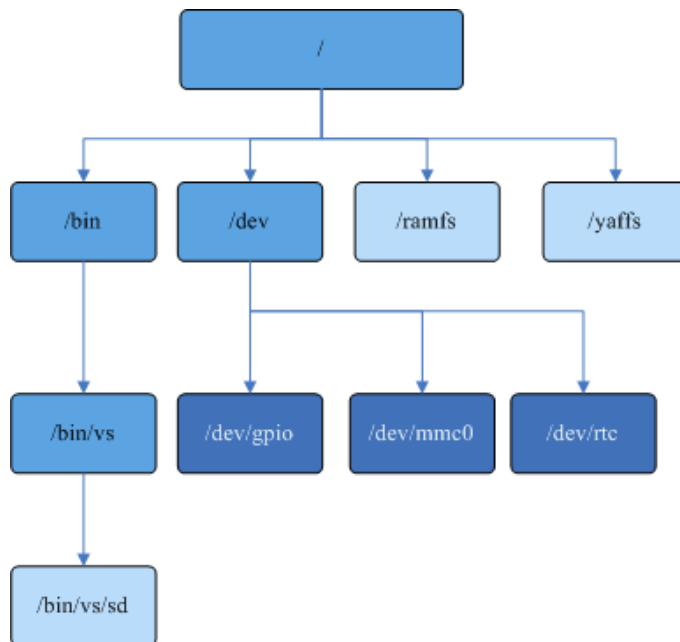
运作机制

通过VFS层，可以使用标准的Unix文件操作函数（如open、read、write等）实现对不同介质上不同文件系统的访问。

VFS框架内存中的inode的关键在于u和i_private字段，一个是函数方法结构体的指针，一个是数据指针。inode树结点有三种类型：

1. 虚拟结点：作为VFS框架的虚拟文件，保持树的连续性，如/bin，/bin/vs，/dev。
2. 设备结点：/dev目录下，对应一个设备，如/dev/mmc0。
3. 挂载点：调用mount函数后生成，如/bin/vs/sd，/ramfs，/yaffs。

图 5-1 VFS 树节点结构



系统初始化过程中会调用los_vfs_init()，将"/"作为root_inode。

文件描述符

本设计中文件描述符分两种，采用动态内存数组的形式来对普通文件描述符进行管理，采用全局数组管理网络描述符，这两个数组在内存上并不连续。

- File描述符，即普通文件描述符，其中0保留作为系统stdin标准输入、1保留作为系统stdout标准输出、2保留作为系统stderr标准错误输出。用户可以分配的文件描述符从3开始到CONFIG_NFILE_DESCRIPTOR - 1。
- Socket描述符，该描述符的分配从CONFIG_NFILE_DESCRIPTOR开始。

5.2.2 开发指导

接口说明

头文件	接口名	描述
open_source/ incubator- nuttx/ liteos/fs/ include/fs/fs.h	register_driver	注册字符驱动到文件系统。
	unregister_driver	取消字符驱动注册。
	register_blockdriver	注册块设备驱动到文件系统。
	unregister_blockdriver	取消块设备驱动注册。
	open_blockdriver	打开块设备。
	close_blockdriver	关闭块设备。
	find_blockdriver	查找块设备。
fs/include/ los_fs.h	chattr	改变文件系统中的文件属性，目前仅FAT文件系统支持，修改后的文件属性目前支持4种：F_RDO（只读）、F_HID（隐藏）、F_SYS（系统文件）、F_ARC（存档）。
	ls	列出目录内容。
	rindex	从指定字符串的右面开始搜索第一次出现指定字符的位置。

说明

关于文件属性：

- 当前只支持修改FAT文件系统的文件属性，其他文件系统对只读等属性有各自的处理方式。
- F_RDO（只读）、F_HID（隐藏）、F_SYS（系统文件）和F_ARC（存档）这4种属性并不冲突，可以任意修改。
- 只读属性文件/目录不允许被删除。
- 只读属性文件/目录允许重命名。
- 只读文件不允许以O_CREAT、O_TRUNC，以及有含有写权限的方式打开。
- 在Huawei LiteOS中隐藏属性文件可见，在Windows中不可见（不显示隐藏文件属性的情况下）。
- 在Huawei LiteOS中文件加上隐藏属性，在Windows中只能通过命令行找到（在显示/不显示隐藏文件属性的情况下，该文件在Windows中都不可见）。

开发流程

推荐驱动开发人员使用VFS框架来注册/卸载设备，在应用层使用open()、read()操作字符设备文件来调用驱动。

1. 打开菜单，选择FileSystem ---> Enable VFS，使能VFS。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_VFS	VFS框架的裁剪开关	YES/NO	YES	无

- 调用register_driver(), register_blockdriver()接口生成设备结点, 调用mount()接口生成文件系统挂载结点。生成设备节点的开发流程详见适配文件操作的[开发指导](#)。

5.2.3 注意事项

- VFS下所有文件系统创建的目录名和文件名长度最多为255个字符, 若超过则创建失败。
- VFS层支持的文件和目录最大全路径长度为259个字符, 若超过, 则创建失败
- 所有文件系统下的文件名和目录名中只支持“-”与“_”两种特殊字符, 使用其他特殊字符可能造成不可预知的后果, 请谨慎为之。
- VFS下的所有文件系统, 在创建模式或只写模式打开文件, 此时文件权限为open接口mode入参值默认为0666, 因VFS下的文件系统对文件权限模式支持不全, 推荐使用0666入参。
- VFS在优化迭代过程中, 对部分POSIX接口(rename、dup、opendir等)的异常错误分支的errno返回值会有更改, 以便更贴近于POSIX标准, 业务在使用errno返回值时请谨慎处理。
- 对于块设备节点(例如/dev/mmc0), 如果块设备驱动的bpos实现unlink回调函数且返回值为OK, 则误调用unlink("/dev/mmc0")会导致块设备节点被删除和节点消失, 并影响正常业务功能; 如果想保持块设备节点常驻, 不实现unlink回调即可。
- ls命令会显示块设备节点文件的size, size值为块设备的容量值, 如果不想显示对应的设备容量, 块设备驱动的bpos不实现geometry回调函数即可。
- VFS可以直接递归创建多级目录。
- 设备分为字符设备和块设备, 为了设备上的数据安全, 需挂载相应文件系统后通过文件系统接口操作数据。
- los_vfs_init() 只能调用一次, 多次调用将会造成文件系统异常。
- open打开一个文件时, 参数O_RDWR、O_WRONLY、O_RDONLY互斥, 只能出现一个, 若出现2个或以上作为open的参数, 会出现不可预知的错误, 禁止使用。
- 不支持open一个目录直接获取目录数据。如果要获取目录信息, 可以调用opendir接口打开目录。
- open()、read()、write()等为非线程安全接口, 在多线程场景下, 建议使用fopen()、fread()、fwrite()等接口。
- 挂载文件系统请严格按照手册进行, 错误挂载可能损坏设备及系统。
- 挂载点必须为空目录, 不能重复挂载至同一挂载点或挂载至其他挂载点下的目录或文件。
- 不要在休眠唤醒或者分散加载阶段挂载一个不存在的文件系统。
- 在unregister_driver卸载设备之前, 要确保字符设备已经正确的close, 即open和close的次数要对应, 否则会卸载失败。

- 在umount之前，需确保此文件系统中的所有目录及文件全部关闭，否则umount会失败。如果强制umount，可能导致包括但不限于文件系统损坏、设备损坏等问题，对此华为不承担任何责任。
- SD卡移除前，需确保所有目录及文件已经全部关闭，并进行umount操作。如果强制拔卡，可能导致包括但不限于SD数据丢失、SD卡损坏等问题，对此华为不承担任何责任。

5.3 NFS

5.3.1 概述

基本概念

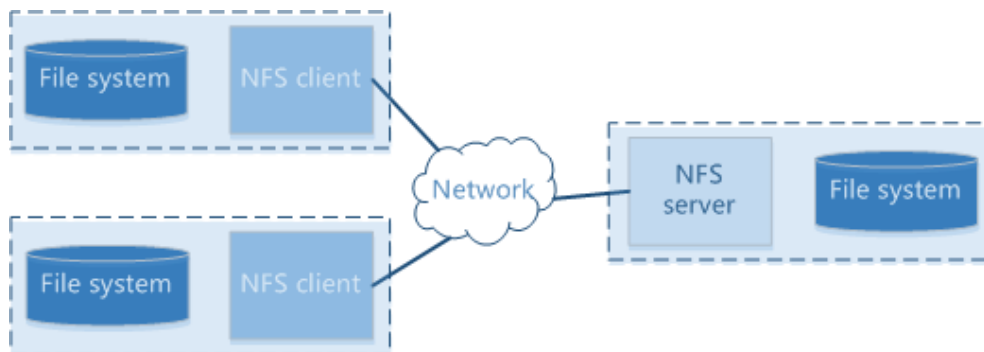
NFS是Network File System（网络文件系统）的缩写。在NFS文件系统中，可以通过网络让不同机器、不同操作系统彼此分享文件。因此，可以将它看做是一种对文件系统的服务，在一定程度上相当于Windows环境下的共享文件夹。

NFS客户端用户，能够将网络远程的NFS服务器端分享的目录挂载到客户端的机器或设备中，运行程序和共享文件。对于客户端而言，拥有远程服务器端的目录就像是增加了一个磁盘，且该磁盘不会占用客户端的系统资源，有效节省了客户端本地磁盘的使用空间。此外，软驱、CDROM和 Zip（一种高存储密度的磁盘驱动器与磁盘）之类的存储设备也能够网络上被其他机器使用，从而减少了整个网络中可移动介质设备的数量。

运作机制

一个典型的NFS组网如下图所示。

图 5-2 NFS 组网图



Huawei LiteOS的NFS支持将服务器端的NFS目录（如/home/Huawei_LiteOS/nfs）映射到客户端目录（如/nfs），两者内容相同并保持同步。

5.3.2 开发指导

开发流程

使用NFS功能，需要执行以下步骤（各步骤详细操作见下述分解）：

1. 打开菜单，选择FileSystem ---> Enable NFS，使能NFS。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_NFS	使能NFS的开关	YES/NO	NO	LOSCFG_FS_VFS=y && LOSCFG_NET_LWIP_SACK=n

2. [搭建NFS服务器](#)
3. [搭建NFS客户端](#)
4. [利用NFS共享文件](#)

搭建 NFS 服务器

这里以Ubuntu操作系统为例，说明搭建服务器的步骤。

步骤1 安装NFS服务器软件。

设置好Ubuntu系统的下载源，保证网络连接好的情况下执行：

```
sudo apt-get install nfs-kernel-server
```

步骤2 设置NFS server。

修改NFS配置文件/etc/exports，添加如下一行：

```
/home/yourusername *(rw,no_root_squash,async)
```

其中/home/yourusername是NFS共享的根目录，可以根据实际情况设置。

步骤3 启动NFS server。

执行以下命令启动NFS server：

```
sudo /etc/init.d/nfs-kernel-server start
```

如果需要重启NFS server，可以执行以下命令重启：

```
sudo /etc/init.d/nfs-kernel-server restart
```

----结束

搭建 NFS 客户端

本指导中的NFS客户端指运行Huawei LiteOS的设备。

步骤1 设置硬件连接。

使NFS客户端可以连接到NFS服务器：设置两者IP，使其处于同一网段。比如，设置NFS服务器的IP为10.67.212.178，Huawei LiteOS设备IP为10.67.212.3（根据子网掩码位数，将两个IP的前几段设置成相同，则这两个IP处于同一网段）。

NFS客户端的IP信息可通过ifconfig命令查看。

步骤2 启动网络，确保NFS客户端到NFS服务器之间的网络通畅。

启动以太网或者其他类型网络，使用ping命令检查到服务器的网络是否通畅。

```
Huawei LiteOS # ping 10.67.212.178  
[0]Reply from 10.67.212.178: time=1ms TTL=63
```

```
[1]Reply from 10.67.212.178: time=0ms TTL=63
[2]Reply from 10.67.212.178: time=1ms TTL=63
[3]Reply from 10.67.212.178: time=1ms TTL=63
--- 10.67.212.178 ping statistics ---
4 packets transmitted, 4 received, 0 loss
```

步骤3 初始化NFS客户端。

运行如下命令，将服务器端的共享目录（如/home/sqbin/nfs）挂载到客户端本地目录（如/nfs）上：

```
Huawei LiteOS# mount 10.67.212.178:/home/sqbin/nfs /nfs nfs 1011 1000
```

从串口得到如下回应信息，表明初始化NFS客户端成功。

```
Huawei LiteOS# mount 10.67.212.178:/home/sqbin/nfs /nfs nfs 1011 1000
Mount nfs on 10.67.212.178:/home/sqbin/nfs
Mount nfs finished.
```

说明

mount命令的格式为：

```
mount [SERVER_IP:SERVER_PATH] [CLIENT_PATH] nfs <uid gid>
```

- SERVER_IP表示服务器的IP地址
- SERVER_PATH表示服务器端的NFS共享目录，注意该路径盘符小写，且不带冒号
- CLIENT_PATH表示客户端的NFS路径，即将服务端共享目录挂载到该客户端目录下。
- uid指服务器端的用户ID和用户组ID，用于获取NFS服务器端共享目录的权限。如果不想限制NFS的访问权限，请在NFS服务器上将共享目录的权限设置成777：
chmod -R 777 /home/sqbin/nfs

至此，NFS文件系统已成功挂载，NFS客户端设置完毕。

----结束

利用 NFS 共享文件

在NFS服务器端的共享目录下新建目录dir，并保存。在NFS客户端中执行ls命令查看共享目录下的文件：

```
Huawei LiteOS# ls /nfs
```

可从串口得到如下回应：

```
Huawei LiteOS#
Huawei LiteOS# ls /nfs
Directory /nfs:
dir          <DIR>dir          <DIR>dir          <DIR>dir          <DIR>
```

可见，刚刚在NFS服务器上新建的dir目录已同步到客户端的/nfs目录，两者保持同步。同样地，在客户端上创建文件/目录，在NFS服务器端共享目录下也可以访问。

平台差异性

目前，NFS客户端仅支持NFS v3规范要求，因此对于规范支持不全的服务器，无法完全兼容。在开发测试过程中，建议使用Linux的NFS server，因为其对NFS支持很完善。

5.3.3 注意事项

- NFS属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。

- open打开一个文件，o_flags必须带有O_RDONLY、O_WRONLY、O_RDWR中的一种属性，o_flags设置的读写权限必须与创建文件时设置的mode匹配。
- 当前NFS文件不支持读阻塞和写阻塞。
- 当前NFS文件不支持信号功能。
- 当前NFS默认使用TCP做通信socket。
- 当前NFS文件系统mount路径长度（不包含IP的长度）不超过259个字符，超过时返回ENAMETOOLONG错误。
- 当前NFS文件系统支持的操作有：open，close，read，write，seek，dup，opendir，closedir，readdir，rewinddir，statfs，remove，mkdir，rmdir，rename，stat，stat64，seek64。
- open打开一个文件且参数有O_TRUNC时，必须同时拥有写权限才能将文件内容清空。
- 在文件未关闭的情况下使用rename()函数重命名A为B之后，读B文件将会读到原A文件中的内容。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

5.4 JFFS2

5.4.1 概述

基本概念

JFFS2是Journalling Flash File System Version 2（日志文件系统）的缩写。JFFS2主要应用于NOR Flash闪存，其特点包括：

- 可读写
- 支持数据压缩
- 提供崩溃/掉电安全保护
- 提供“写平衡”支持等

闪存与磁盘介质有许多差异，因此直接将磁盘文件系统运行在闪存上存在性能和安全性上的不足。为解决这一问题，需要实现一个针对闪存的文件系统，JFFS2就是这样的文件系统。

Huawei LiteOS的JFFS2主要应用于NOR Flash闪存的文件管理，并且支持多分区，支持dentry cache目录项缓存提升访问性能，最大支持同时缓存128个目录项。

5.4.2 开发指导

接口说明

头文件	接口名	描述
fs/include/ mtd_partition. h	add_mtd_partition	添加JFFS2分区，该函数会自动为设备节点命名，对于JFFS2，其命名规则是“/dev/spinorblk”加上分区号。

头文件	接口名	描述
	delete_mtd_partition	删除已经卸载的分区。

 说明

- add_mtd_partition添加JFFS2分区时，系统会自动对起始地址和分区大小根据设备block大小进行对齐。该函数有四个参数：
 - 第一个参数表示介质，支持“nand”和“spinor”。JFFS2分区在“spinor”上使用，YAFFS2在“nand”上使用。
 - 第二个参数表示分区的起始地址，以16进制的形式传入。
 - 第三个参数表示分区大小，以16进制的形式传入。
 - 最后一个参数表示分区号，有效值为0~19。
- delete_mtd_partition函数有两个参数：
 - 第一个参数是分区号。
 - 第二个参数为介质类型，该函数与add_mtd_partition()函数对应。

开发流程

使用JFFS2功能，涉及以下步骤（各步骤详细操作见下述分解）：

- 打开菜单，选择FileSystem ---> Enable JFFS2，使能JFFS2文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_JFFS	使能JFFS2的开关	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVER_S_MTD_SPI_NO R && LOSCFG_BASE_IPC_MUX && LOSCFG_BASE_IPC_EVENT
LOSCFG_FS_JFFS_DCACHE	使能JFFS2目录项缓存功能	YES/NO	YES	LOSCFG_FS_JFFS
LOSCFG_FS_JFFS_GC_LAZY	使能JFFS2垃圾回收缓慢模式	YES/NO	NO	LOSCFG_FS_JFFS

- [制作JFFS2文件系统镜像并烧录（可选）](#)
- [添加JFFS2分区](#)
- [挂载JFFS2分区](#)
- [卸载JFFS2分区](#)
- [删除JFFS2分区](#)

制作 JFFS2 文件系统镜像并烧录（可选）

目前支持使用mkfs.jffs2工具制作文件系统镜像，可以参考如下命令制作镜像：

```
./mkfs.jffs2 -s 0x1000 -e 0x10000 -p 0x100000 -d rootfs/ -o rootfs_64k.jffs2
```

表 5-2 参数描述

参数	描述
-s	设置页大小
-e	设置eraseblock大小
-p	设置镜像大小
-d	设置要制作成镜像的源目录
-o	设置镜像名称

说明

用户可根据实际情况修改命令中参数值，其它参数项可通过查询mkfs.jffs2查看。

制作好镜像后，便可进行镜像烧录。

添加 JFFS2 分区

调用add_mtd_partition函数添加JFFS2分区。

```
if (ret = add_mtd_partition("spinor", 0x100000, 0x800000, 0) != 0) {  
    dprintf("add jffs partition 0 failed, return %d\n", ret);  
} else {  
    dprintf("add jffs partition 0 successfully.\n");  
}  
if (ret = add_mtd_partition("spinor", 0x900000, 0x200000, 1) != 0) {  
    dprintf("add jffs partition 1 failed, return %d\n", ret);  
} else {  
    dprintf("add jffs partition 1 successfully.\n");  
}
```

成功后，可以在Shell中使用partition spinor命令查看spinor flash分区信息。

```
Huawei LiteOS# partition spinor  
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:(null), startaddr:0x0100000,length:0x0800000  
spinor partition num:1, dev name:/dev/spinorblk1, mountpt:(null), startaddr:0x0900000,length:0x0200000
```

挂载 JFFS2 分区

调用mount()函数实现设备节点的挂载。

```
ret = mount("/dev/spinorblk0", "/jffs0", "jffs", 0, NULL);  
if (ret) {  
    dprintf("mount jffs err %d\n", ret);  
    dprintf("Mount jffs2 on nor finished.\n");  
}
```

说明

mount()函数有五个参数：

- 第一个参数表示设备节点，这个参数需要和add_mtd_partition()函数对应起来。
- 第二个参数表示挂载点。
- 第三个参数表示文件系统类型。
- 第四个参数表示挂载标志，默认为0。
- 最后一个参数表示私有数据，默认为NULL。

调用mount()函数是否成功，可以在Shell中执行ls查看当前根目录下的目录。得到如下回应信息，表明挂载成功。

```
Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
jffs0        <DIR>
ramfs        <DIR>
yaffs0       <DIR>
```

除了在源码中调用mount()函数，也可以在Shell中使用mount命令实现，前面3个参数同mount()函数，最后两个参数不需要给出：

```
Huawei LiteOS# mount /dev/spinorblk1 /jffs1 jffs
```

```
Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
jffs0        <DIR>
jffs1        <DIR>
ramfs        <DIR>
yaffs0       <DIR>
```

挂载成功后，就能对NOR Flash进行读写操作。

卸载 JFFS2 分区

调用umount()函数卸载分区，只需要给出正确的挂载点即可。也可以在Shell中使用umount命令实现：

```
Huawei LiteOS# umount /jffs1
```

若得到如下回应信息，则表明卸载成功。

```
Huawei LiteOS# umount /jffs1
umount ok
```

删除 JFFS2 分区

调用delete_mtd_partition删除已经卸载的分区。

```
ret =delete_mtd_partition(1, "spinor");
if (ret != 0) {
    printf("delte jffs error\n");
} else {
    printf("delete jffs ok\n");
}
```

可以在Shell中使用partition spinor命令查看分区是否删除成功。

```
Huawei LiteOS# partition spinor
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:/jffs0, startaddr:0x0100000,length:0x0800000
```

5.4.3 注意事项

- 目前JFFS2文件系统用于NOR Flash，需要调用NOR Flash驱动接口，因此使用JFFS2文件系统之前要确保硬件上有NOR Flash，且驱动初始化成功（spinor_init()返回0）。
- 目前JFFS2文件系统支持的操作有：open，close，read，write，seek，seek64，opendir，closedir，readdir，rewinddir，statfs，unlink，mkdir，rmdir，rename，stat，stat64。
- open打开一个文件，参数有O_TRUNC时，会将文件中的内容清空。
- 目前JFFS2不支持open带 O_DIRECTORY属性的操作，打开目录请使用opendir()。
- JFFS2支持挂载只读属性，当mount函数的入参mountflags为MS_RDONLY时，JFFS2将开启只读属性，所有带有写操作的接口，如write、mkdir、unlink，以及通过非O_RDONLY属性打开的文件，均将被拒绝，并传出EACCESS错误码。
- open打开一个文件，o_flags必须带有O_RDONLY、O_WRONLY、O_RDWR中的一种属性，o_flags设置的读写权限必须与创建文件时设置的mode匹配。

5.5 FAT

5.5.1 概述

基本概念

FAT文件系统是File Allocation Table（文件配置表）的简称，FAT文件系统有FAT12、FAT16、FAT32。FAT文件系统将硬盘分为MBR区、DBR区、FAT区、DIR区、DATA区等5个区域。

FAT文件系统支持多种介质，特别是在可移动存储介质（U盘、SD卡、移动硬盘等）上广泛使用。而且，FAT文件系统可以使嵌入式设备和Windows、Linux等桌面系统保持很好的兼容性，方便用户管理操作文件。

Huawei LiteOS的FAT文件系统的特点有：

- 代码量和资源占用小、可裁切、支持多种物理介质。
- 与Windows、Linux等系统保持兼容。
- 支持多设备、多分区识别等功能。
- 支持硬盘多分区，可以在主分区以及逻辑分区上进行文件操作。
- 支持识别出硬盘上其他类型的文件系统，比如NTFS。
- 支持基于FAT32 的虚拟分区特性。

当开启虚拟分区特性时，用户可通过调用虚拟分区的设置接口

los_set_virpartparam()，对特定的物理分区配置虚拟分区的数量、空间百分比大小、虚拟分区的入口名等，并会根据设置的入口名，在根目录建立对应的文件夹作为虚拟分区入口。

- 在对应的虚拟分区内部进行操作，即视为在对应的虚拟分区内进行操作。
- 若所有虚拟分区空间百分比总和未达到100%，则允许在虚拟分区入口目录外进行写入操作，此时所有虚拟分区入口外部的总可写入空间，即为百分比总和的剩余空间。若所有虚拟分区空间百分比达到了100%，则在虚拟分区入口目录外部将会以空间不足的理由，拒绝写入操作。

5.5.2 开发指导

FAT 文件系统的接口说明

头文件	接口名	描述
fs/include/ los_fs.h	getlabel	获取盘符
	set_label	设置盘符
	format	格式化磁盘
	los_set_systime_status	设置FAT文件当前系统时间有效或无效
	LOS_SetSyncThreadPrio	设置FAT文件系统缓存异步刷新任务的优先级
	LOS_SetSyncThreadInterval	设置FAT文件系统缓存异步刷新的时间间隔
	LOS_GetSyncThreadInterval	获取FAT文件系统缓存异步刷新的时间间隔
	LOS_SetDirtyRatioThreshold	设置FAT文件系统缓存异步刷新的脏数据块比例阈值
	LOS_GetDirtyRatioThreshold	获取FAT文件系统缓存异步刷新的脏数据块比例阈值
	LOS_SetBlockExpireInterval	设置FAT文件系统块的最大过期时间间隔
	LOS_GetBlockExpireInterval	获取FAT文件系统块的最大过期时间间隔

说明

- 使能FAT文件系统缓存（Cache）异步刷新后，系统会创建一个任务刷Cache，线程被触发唤醒进行脏数据刷新的条件如下：1. 周期性5s唤醒。2. 脏数据停留时间超过3s。3. 脏数据量超过50%时，触发任何一个条件，线程都会立刻把脏数据写回磁盘。
- 在open_source/FatFs/source/ffconf.h文件中可以配置FAT文件系统，其中FF_FS_LOCK为最多支持同时打开的文件（文件夹）数。

虚拟分区的接口说明

头文件	接口名	接口功能描述
fs/include/ los_fs.h	los_set_virpartparam	设置虚拟分区的参数
	virstatfs	获取指定虚拟分区的状态

虚拟分区的调配结构体定义如下所示：

```
#define _MAX_ENTRYLENGTH 16
#define _MAX_VIRVOLUMES 5

typedef struct virtual_partition_info
{
    char *devpartpath;
    int virpartnum;
    double virpartpercent[_MAX_VIRVOLUMES];
    char virpartname[_MAX_VIRVOLUMES][_MAX_ENTRYLENGTH + 1];
} virpartinfo;
```

成员	描述	注意事项
devpartpath	设备节点路径	所选定的设备节点的名称。必须是/dev下的设备节点路径，上限是DISK_NAME。
virpartnum	虚拟分区数量	最小值为1，最大值为_MAX_VIRVOLUMES，即为5。
virpartpercent	虚拟分区百分比	总和必须大于0.00，小于等于1.00。每个成员的值必须大于0.00，小于1.00。
virpartname	虚拟分区入口名	入口名有效字符最多为_MAX_ENTRYLENGTH，即16个字符，不得为空。传入的集合不可以有重名项。入口名不得含有非法名称字符。

须知

- 在Huawei LiteOS中，虚拟分区特性仅支持特定的单个物理分区进行，可通过调配虚拟分区参数接口设置设备节点。
- 设置指定物理设备节点后，会对设置进行上锁，不允许再次更改设置。挂载时就会采用设置的参数进行配置。当卸载后，才会对设置进行解锁。
- 卸载后，需要通过调配虚拟分区参数接口重新设置虚拟分区参数。
- 当插入的物理分区中已经设置了虚拟分区，但是其配置参数与当前系统通过调配接口所设置的参数不一致时，则将按照已有的配置参数对此分区进行管理。

虚拟分区涉及的接口行为变化

开启虚拟分区特性后，部分接口的行为会在虚拟分区下有部分变化。下述这些接口的行为变化只对于已成功应用了虚拟分区特性的物理分区，对于未能应用虚拟分区的基本FAT文件系统的物理分区，以及其他的文件系统，这些接口行为不涉及变化。

下述的接口只是在执行功能上的行为针对虚拟分区有所变化，函数的调用方式、参数等均不受影响。

序号	接口名	行为变化	返回值与错误码说明
1	mount	挂载了基本FAT分区后，会尝试识别当前物理分区是否已经应用过虚拟分区，并尝试对该物理分区按照其配置的信息加载虚拟分区，检查并尝试修复虚拟分区入口；若当前物理分区未应用过虚拟分区并符合启动虚拟分区的条件，则会自动对该物理分区应用虚拟分区，并建立虚拟分区。	返回值为-1表示基本FAT分区挂载失败，此时会触发libc错误码指示发生了何种错误； 返回值为0表示基本FAT分区挂载成功，此时错误码若为0表示虚拟分区挂载成功，若错误码为VIRERR_MODIFIED、VIRERR_CHAIN_ERR、VIRERR_OCCUPIED、VIRERR_NOTCLEAR、VIRERR_NOTFIT、VIRERR_NOPARAM表示虚拟分区挂载失败
2	format	当FAT分区已挂载后，若格式化成为FAT32文件系统，则会在格式化完成后对该物理分区应用虚拟分区；若非FAT32文件系统，则会完成格式化操作，并清除该物理分区的虚拟分区；若FAT分区未挂载，无论何种文件系统，则均会完成格式化操作后清除该物理分区的虚拟分区。	返回值为-1表示基本FAT分区格式化失败，此时会触发libc错误码指示发生了何种错误； 返回值为0表示基本FAT分区格式化完成。若此时分区并未挂载，则错误码则会为VIRERR_NOTMOUNT；若格式化的分区非调配接口所设定设备节点路径，或者调配接口并未设置参数，则此时会指示VIRERR_NOPARAM。若分区已挂载，错误码为0表示应用或重新应用虚拟分区成功，或者错误码可能为VIRERR_NOTFIT。
3	open	请参阅 虚拟分区的兼容性的说明	无
4	mkdir	请参阅 虚拟分区的兼容性的说明	无
5	rmdir	当物理分区已经成功应用虚拟分区时，不允许删除虚拟分区入口目录。	无
6	remove	当物理分区已经成功应用虚拟分区时，不允许删除虚拟分区入口目录。	无
7	rename	当物理分区已经成功应用虚拟分区时，不允许重命名虚拟分区入口目录。	无

虚拟分区的错误码

应用虚拟分区发生错误，只会发生于挂载时或者格式化时。当基本FAT文件系统的对应操作完成后，进行虚拟分区的配置与应用。此时若发生某种问题或者触发某种条件，

虚拟分区会将错误码传递给上层，以分辨应用虚拟分区时发生了什么情况。同时该分区将会以基本FAT文件系统进行管理，而不会在此基础上采用虚拟分区管理。

说明

对于挂载和格式化操作的返回值，指示的是基本FAT文件系统的成功或失败。

- 当接口的返回值为0时，表示对应的基本FAT文件系统操作成功，此时可通过访问errno获取虚拟分区的错误码，以检查应用虚拟分区过程中是否有异常产生。
- 当接口的返回值为-1时，表示于对应的基本FAT文件系统操作失败，此时的errno为libc的错误码，表示基本FAT操作失败的原因，不再表示虚拟分区的提示信息。

序号	定义	实际数值	描述	触发情况
1	VIRERR_MODIFIED	0x10000001	虚拟分区保留扇区信息遭到破坏	是否被重新分区、是否调整过分区大小、或者SD卡是否曾遭到破坏。
2	VIRERR_CHAIN_ERR	0x10000002	虚拟分区入口目录已被移出所在的虚拟分区	虚拟分区入口目录在其他平台上被删除后重建，有一定概率会触发该问题。
3	VIRERR_OCCUPIED	0x10000003	虚拟分区入口被同名文件占据	物理分区根目录有同名文件，阻碍了虚拟分区入口的创建。
4	VIRERR_NOT_CLEAR	0x10000004	物理分区非空	在对新物理分区应用虚拟分区时，分区非空。
5	VIRERR_NOT_FIT	0x10000005	物理分区非FAT32文件系统	物理分区是否为FAT32文件系统。
6	VIRERR_NOT_MOUNT	0x10000006	设备未挂载	执行格式化时设备未挂载。
7	VIRERR_INTERRUPT_ERR	0x10000007	意外错误	应用虚拟分区时发生意外错误。
8	VIRERR_NOPARAM	0x10000008	未对虚拟分区进行全局配置	在执行挂载/格式化操作之前，需要先调用调配接口。
9	VIRERR_PARAMETER_LOCKED	0x10000009	设置已上锁	设置已上锁，不允许再次设置。
10	VIRERR_PARAMETER_NUMERR	0x1000000A	设置接口VirPartNum字段错误	检查VirPartNum的设定值是否正确。
11	VIRERR_PARAMETER_PERCENTERR	0x1000000B	设置接口VirPartPercent字段错误	检查VirPartPercent字段的值是否正确。
12	VIRERR_PARAMETER_NAMEERR	0x1000000C	设置接口VirPartName字段错误	检查VirPartName字段的设定是否正确。

序号	定义	实际数值	描述	触发情况
13	VIRERR_PAR MDEVERR	0x100000 0D	设置接口 DevPartPath字段错 误	检查DevPartPath字段是 否正确。

开发流程

使用FAT功能，涉及以下几个步骤，其中对虚拟分区的操作为可选部分，只有在FAT使用虚拟分区功能的情况下才涉及，请根据实际业务情况选择（各步骤详细操作见下述分解）：

1. 打开菜单，进入FileSystem ---> Enable FAT菜单，完成FAT文件系统的配置。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_FAT	使能FAT文件系统的开关	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVE R_DISK
LOSCFG_FS_FAT_C ACHE	使能FAT文件系 统缓存	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_C ACHE_SYNC_THRE AD	使能FAT文件系 统缓存异步刷 新	YES/NO	YES	LOSCFG_FS_FAT _CACHE
LOSCFG_FS_FAT_C ACHE_PROC	使能FAT文件系 统缓存统计	YES/NO	YES	LOSCFG_FS_FAT _CACHE
LOSCFG_FS_FAT_C HINESE	使能FAT文件系 统支持中文	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_D CACHE	使能FAT文件系 统目录项缓存	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_T RIM	使能FAT文件系 统介质TRIM	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_U NI_TRIM	使能FAT文件系 统介质合并 TRIM	YES/NO	YES	LOSCFG_FS_FAT
LOSCFG_FS_FAT_D IRECT_TRIM	使能FAT文件系 统介质直接 TRIM	YES/NO	NO	LOSCFG_FS_FAT
LOSCFG_FS_FAT_V IRTUAL_PARTITIO N	使能FAT文件系 统支持虚拟分 区	YES/NO	NO	LOSCFG_FS_FAT

说明

- 当虚拟分区特性启用后，整个Huawei LiteOS的FAT文件系统将优先采用虚拟分区特性进行管理，虚拟分区不支持FAT目录项缓存。
- 当启用FAT目录项缓存后，会消耗一部分内存用来缓存FAT文件系统的目录项，从而减少部分接口的再次访问耗时。
- 当启用FAT Trim功能后，可以有效减缓磁盘的write amplification行为，提升数据写入速率的稳定性。

2. [识别设备](#)
3. [挂载FAT文件系统](#)
4. [卸载FAT文件系统](#)
5. [检查虚拟分区的可操作条件（可选）](#)
6. [调配虚拟分区（可选）](#)
7. [挂载虚拟分区（可选）](#)
8. [卸载虚拟分区（可选）](#)
9. [创建虚拟分区（可选）](#)
10. [删除虚拟分区（可选）](#)

识别设备

在open_source/FatFs/source/ffconf.h文件中可以使能多分区和多设备。使能多分区后，可以操作设备上的多个分区，否则只能操作第一个分区。使能多设备后，可以将多个设备挂载成FAT文件系统。具体配置项为：

- 配置FF_MULTI_PARTITION为1，使能多分区功能。
- 配置FF_VOLUMES大于1时，使能多设备功能。

系统自动识别插入的SD卡，自动注册设备节点，如下所示。其中的mmcblk0和mmcblk1为卡0和卡1，是独立的主设备，mmcblk0p0和mmcblk0p1为卡0的两个分区，mmcblk1p0为卡1的分区，这些分区可作为分区设备使用。**存在分区设备的情况下，禁止使用主设备。**

```
Huawei LiteOS# ls
Directory /dev:
acodec          0
adec            0
aenc            0
ai              0
ao              0
console         0
fb0             0
hi_gpio         0
hi_mipi         0
hi_rtc          0
hi_tde          0
i2c-0           0
i2c-1           0
i2c-2           0
isp_dev         0
lcd             0
logmpp          0
mem             0
mmcblk0         0
mmcblk0p0       0
mmcblk0p1       0
mmcblk1         0
mmcblk1p0       0
```

可以在Shell中使用partinfo命令查看设备的分区信息：

```
Huawei LiteOS # partinfo /dev/sdap0
part info :
disk id      : 3
part_id in system: 0
part no in disk : 0
part no in mbr : 1
part filesystem : 0C
part dev name : sdap0
part sec start : 2048
part sec count : 167794688
```

挂载 FAT 文件系统

调用mount()函数实现设备节点的挂载。

```
ret = mount("/dev/mmcbk0p0", "/bin/vs/sd", "vfat", 0, NULL);
if (ret) {
    dprintf("mount fat filesystem err %d\n", ret);
}
```

说明

mount()函数有五个参数：

- 第一个参数表示设备节点。
- 第二个参数表示挂载点。
- 第三个参数表示文件系统类型。
- 第四个参数表示挂载标志，默认为0。
- 最后一个参数表示私有数据，默认为NULL。

除了在源码中调用mount()函数，也可以在Shell中使用mount命令实现挂载，前面3个参数同mount()函数，最后两个参数不需要设置。得到如下回应信息，表明挂载成功：

```
Huawei LiteOS# mount /dev/mmcbk0p0 /bin/vs/sd vfat
mount ok
```

卸载 FAT 文件系统

调用umount()函数卸载分区，只需要给出正确的挂载点即可。也可以在Shell中使用umount命令实现，得到如下回应信息，表明卸载成功：

```
Huawei LiteOS# umount /bin/vs/sd
umount ok
```

检查虚拟分区的可操作条件（可选）

- 对于未在Huawei LiteOS中应用过虚拟分区特性的存储设备（如SD卡、U盘、移动硬盘等）物理分区，需要保证：
 - 该物理分区为FAT32分区。
 - 该物理分区下无任何文件。
- 对于曾经在LiteOS中已经应用过虚拟分区特性的存储设备物理分区，需要保证：
 - 该物理分区下的虚拟分区入口目录没有被删除、重命名或者其他形式的破坏。
 - 该物理分区的保留扇区内数据完整。

若有任一条件不满足，则无法对该物理分区应用虚拟分区特性，只能应用常规FAT文件系统。

调配虚拟分区（可选）

虚拟分区特性针对的是某个特定场景下的物理分区，需要通过调配接口 `los_set_virpartparam()` 指定分区设备，并设置该分区的配置参数。调配虚拟分区参数接口的主要目的是允许用户建立符合场景的虚拟分区。

系统开机后，若想应用虚拟分区特性，需要先调用 `los_set_virpartparam()`，对当前系统的虚拟分区数量、对应虚拟分区的关键参数进行配置。若未配置，则将导致后续虚拟分区应用失败。

- 对未在Huawei LiteOS中应用过虚拟分区特性的存储设备物理分区，在进行虚拟分区应用之前，可以通过调配接口设置需要的参数。正常情况下，当成功调配后，对该物理分区执行挂载操作，即可完成虚拟分区的应用。
- 若操作的物理分区已经应用过虚拟分区特性，只要该分区已设置的虚拟分区校验正确，则可忽略调配接口设置的参数，而无条件遵循该分区内部的参数对其管理。

挂载虚拟分区（可选）

在完成挂载FAT文件系统后会自动对已经挂载的分区进行虚拟分区的应用。若虚拟分区应用成功，则会启动虚拟分区对该物理分区进行管理；若虚拟分区应用失败，则会以普通FAT文件系统对该物理分区进行管理，并传出特定的错误码表明在应用虚拟分区时发生了何种错误。

虚拟分区成功挂载后，会在分区根目录下建立对应数量以及对应名称的目录，即作为虚拟分区入口。在虚拟分区入口内部创建文件，即相当于在对应的虚拟分区内创建文件。每个虚拟分区入口内的最大可用空间，即为设置的虚拟分区百分比大小。

卸载虚拟分区（可选）

卸载虚拟分区可以认为是挂载的逆过程，不再进行重复。

创建虚拟分区（可选）

当对一个新物理分区应用虚拟分区时，执行挂载操作即可完成虚拟分区的创建，并且可以在Huawei LiteOS中进行重复使用。创建的虚拟分区的基本参数，以调配接口所设置的参数为准。

某些情况下，需要对某个分区重新应用虚拟分区时，可在已经挂载的分区上格式化该磁盘为FAT32文件系统。在格式化操作完成后，则会对该物理分区重新应用虚拟分区。

须知

执行格式化操作会导致用户数据丢失，在执行此操作时请小心。

删除虚拟分区（可选）

当某些情况下，需要撤销对物理分区使用虚拟分区特性，可以在未挂载的分区上进行格式化，即可清除该分区上的虚拟分区信息。

须知

执行格式化操作会导致用户数据丢失，在执行此操作时请小心。

5.5.3 虚拟分区的兼容性

虚拟分区特性是Huawei LiteOS独有的基于FAT的文件管理模式。在其他平台上对已经应用过虚拟分区的物理分区进行操作时，务必了解以下兼容性特征，以免使用时发生问题。

- 重新对存储介质分区、调整存储介质的大小等操作，会直接导致物理分区的虚拟分区特性失效。
- 在其他平台上对已经应用虚拟分区的物理分区建立文件夹时，需要注意：
 - 若建立的文件夹在虚拟分区入口外，且全部虚拟分区的百分比总和为100%，则下次在Huawei LiteOS中在该文件夹内不允许创建新的文件或者文件夹。
- 在其他平台上对已经应用虚拟分区的物理分区写文件时，需要注意：
 - 若写入的文件在虚拟分区入口外，且全部虚拟分区的百分比总和为100%，则下次在Huawei LiteOS中这些文件只允许以只读方式访问。
 - 若写入的文件在虚拟分区入口内，而且文件整体的簇链均在对应的虚拟分区的簇链管理范围内，则下次在Huawei LiteOS中这些文件会以正常权限访问。
 - 若写入的文件在虚拟分区入口内，而且文件整体的簇链有一部分或者整体不在虚拟分区的管理范围内，则下次在Huawei LiteOS中这些文件只允许以只读权限访问。
- 在其他平台上删除虚拟分区入口目录，会导致对应的虚拟分区意外丢失。下次在Huawei LiteOS中进行挂载时，会尝试重建丢失的虚拟分区。
 - 若删除后用户创建一个同名的文件夹，且文件夹的整体簇链均落在了对应的虚拟分区的管理范围内，则下次在Huawei LiteOS中进行挂载时，该虚拟分区入口将被正常识别。
 - 若删除后用户创建一个同名的文件夹，且整体簇链一部分或者整体落在了对应虚拟分区的管辖范围外，则下次在Huawei LiteOS中进行挂载时，会导致虚拟分区入口校验失败，拒绝此次的虚拟分区应用。
- 在其他平台上重命名虚拟分区入口目录，会导致对应的虚拟分区意外丢失。下次在Huawei LiteOS中进行挂载时，会尝试将丢失的虚拟分区重建。原有的文件不会丢失，但这些文件将不会再受到虚拟分区特性的管理，只允许以只读权限访问。

5.5.4 注意事项

- FAT文件系统中，单个文件不能大于4G。
- 带绝对路径的文件名，除去挂载点之后的长度总和不能大于252个字节。
- 目前只支持FAT32格式，如果使用其他文件格式，需要先将介质格式化为FAT32再对接到LiteOS。
- FAT文件系统支持的操作有：open, close, read, write, seek, sync, opendir, closedir, readdir, rewinddir, statfs, virstatfs, unlink, mkdir, rmdir, rename, stat, lstat, stat64, utime, chattr, seek64, getlabel, set_label, format, fallocation, fallocation64, truncate, truncate64, fscheck。

- 以可写方式打开一个文件后，未close前再次打开会失败。同时打开同一文件，必须全部以只读方式打开。长时间打开一个文件，没有close时数据会丢失，必须close才能保存。
- open打开一个文件，参数有O_TRUNC时，会将文件中的内容清空。
- 若open未使用mode入参，则无法通过mode入参来控制文件权限。
- 目前FAT文件系统暂不支持open带O_DIRECTORY属性的操作，打开目录请使用opendir()。
- FAT文件系统的读写指针没有分离，所以以O_APPEND（追加写）方式打开文件后，读指针也在文件尾，读文件前需要用户手动置位。
- FAT文件系统的stat及lstat函数获取的文件时间只是文件的修改时间，暂不支持获取文件创建时间和最后访问时间。微软FAT协议不支持1980年以前的时间。
- 在format操作之前，若FAT文件系统已挂载，需确保所有目录及文件已经全部关闭，否则format会失败。
- FAT支持挂载只读属性：
 - 当mount函数的入参为MS_RDONLY时，FAT将开启只读属性，所有带写入操作的接口（format接口除外），如write、mkdir、unlink，以及通过非O_RDONLY属性打开的文件，均被拒绝，并传出EACCESS错误码。
 - 当mount函数的入参为MS_NOSYNC时，FAT不会主动将Cache的内容写回存储器件。FAT的如下接口（open、close、unlink、rename、mkdir、rmdir、chattr、truncate）不会自动进行sync操作，虽然可以提升速度，但是需要上层主动调用sync来同步数据，否则下电可能会导致数据丢失。
- 当前Cache的默认大小为16个块，每个块256个扇区。可以在fs/include/vfs_config.h文件中修改Cache的块数量CONFIG_FS_FAT_BLOCK_NUMS 和每个块的扇区数CONFIG_FS_FAT_SECTOR_PER_BLOCK，注意CONFIG_FS_FAT_SECTOR_PER_BLOCK必须为32的倍数，且不能大于2048。
- 当有两张SD卡插槽时，卡0和卡1不固定，先插入的为卡0，后插入的为卡1。
- 为避免SD卡使用异常和内存泄漏，在SD卡使用过程中拔卡，必须先关闭正处于打开状态的文件和目录，然后umount挂载节点。

5.6 YAFFS2

5.6.1 概述

基本概念

YAFFS是Yet Another Flash File System的简称，是一种开源的、针对Nand Flash的嵌入式文件系统。在YAFFS中，最小的存储单位为page。

目前YAFFS文件系统有YAFFS和YAFFS2两个版本，主要区别在于page读写size的大小，YAFFS2可支持到2K per page，远高于YAFFS的512 bytes，因此YAFFS2能够更好地支持大容量的Nand Flash芯片。此外YAFFS2还有64bytes的SPARE区域，用于存储坏块信息、ECC校验等。

- YAFFS2专为Nand Flash设计，适用于大容量的存储设备，同时也使得Nand Flash具有高效性和健壮性。
- YAFFS2实现对2K per page的读写支持，同时在内存空间占用、垃圾回收速度、读写速度等方面均有大幅提升。

- YAFFS2为文件系统提供了损耗平衡和掉电保护，保证在修改文件系统中的数据时，即使发生意外也不损坏数据。
- Huawei LiteOS的YAFFS2文件系统支持多分区，采用双向链表结构实现。

5.6.2 开发指导

接口说明

头文件	接口名	描述
fs/include/ mtd_partition. h	add_mtd_partition	添加YAFFS2分区，该函数会自动为设备节点命名，对于YAFFS2，其命名规则是“/dev/nandblk”加上分区号
	delete_mtd_partition	删除已经卸载的分区

说明

1. add_mtd_partition添加YAFFS2分区时，系统会自动对起始地址和分区大小根据设备block大小进行对齐。该函数有四个参数：
 - 第一个参数表示介质，支持“nand”和“spinor”。YAFFS2分区在“nand”上使用，JFFS2在“spinor”上使用。
 - 第二个参数表示分区的起始地址，以16进制的形式传入。
 - 第三个参数表示分区大小，以16进制的形式传入。
 - 最后一个参数表示分区号，有效值为0~19。
2. delete_mtd_partition函数有两个参数：
 - 第一个参数是分区号。
 - 第二个参数为介质类型，该函数与add_mtd_partition()函数对应。

开发流程

使用Huawei LiteOS的YAFFS2文件系统，涉及以下几个步骤（各步骤详细操作见下述分解）：

1. 打开菜单，选择FileSystem ---> Enable YAFFS2，使能YAFFS2文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_YAF FS	使能YAFFS2的 开关	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVER S_MTD_NAND

2. [制作YAFFS2文件系统镜像并烧录（可选）](#)
3. [调用add_mtd_partition创建分区](#)
4. [调用mount函数挂载分区](#)
5. [调用umount函数卸载分区](#)
6. [调用delete_mtd_partition函数删除分区](#)

制作 YAFFS2 文件系统镜像并烧录（可选）

目前支持使用mkyaffs2image工具制作文件系统镜像，可以参考如下命令制作镜像：

```
./mkyaffs2image rootfs rootfs.yaffs2 1 2
```

表 5-3 参数含义表

参数	含义
rootfs	要制作成镜像的源目录
rootfs.yaffs2	镜像名称
1	Nand Flash器件的页大小，1表示2K Bytes pagesize
2	Nand Flash器件的ECC校验类型，2表示4bit/512

说明

用户可根据实际情况修改命令中参数值。

制作好镜像后，就可以烧录此镜像了。

调用 add_mtd_partition 创建分区

创建分区的示例代码如下：

```
if (ret = add_mtd_partition("nand", 0x900000, 0x200000, 0) < 0) {
    dprintf("add yaffs partition 0 failed, return %d\n", ret);
}

if (ret = add_mtd_partition("nand", 0xb00000, 0x200000, 1) < 0) {
    dprintf("add yaffs partition 1 failed, return %d\n", ret);
}
```

创建成功后，在Shell中可以使用partition nand命令查看Nand Flash的分区信息：

```
Huawei LiteOS# partition nand
nand partition num:0, dev name:/dev/nandblk0, mountpt:(null), startaddr:0x0900000,length:0x0200000
nand partition num:1, dev name:/dev/nandblk1, mountpt:(null), startaddr:0x0b00000,length:0x0200000
```

调用 mount 函数挂载分区

调用mount()函数实现设备节点的挂载。

```
ret = mount("/dev/nandblk0", "/yaffs0", "yaffs", 0, NULL);
if (ret) {
    dprintf("mount yaffs err %d\n", ret);
}
```

mount()函数的入参说明参见[挂载JFFS2分区](#)。

说明

mount()函数有五个参数：

- 第一个参数表示设备节点，这个参数需要和add_mtd_partition()函数对应起来。
- 第二个参数表示挂载点。
- 第三个参数表示文件系统类型。
- 第四个参数表示挂载标志，默认为0。
- 最后一个参数表示私有数据，默认为NULL。

除了在源码中调用mount()函数，也可以在Shell中使用mount命令实现挂载，前面3个参数同mount()函数，最后两个参数不需要给出，回显“mount ok”表明挂载成功：

```
Huawei LiteOS# mount /dev/nandblk1 /yaffs1 yaffs
mount ok
```

挂载成功后，在Shell中可以使用partition nand命令看到设备节点的挂载点信息：

```
Huawei LiteOS# partition nand
nand partition num:0, dev name:/dev/nandblk0, mountpt:/yaffs0, startaddr:0x0900000,length:0x0200000
nand partition num:1, dev name:/dev/nandblk1, mountpt:/yaffs1, startaddr:0x0b00000,length:0x0200000
```

调用 umount 函数卸载分区

调用umount()函数卸载分区，只需要正确给出挂载点即可。这一操作也可以在Shell中使用umount命令实现，回显“umount ok”表明命令执行成功：

```
Huawei LiteOS# umount /yaffs1
umount ok

Huawei LiteOS# umount /yaffs0
umount ok
```

卸载成功后，在Shell中可以使用partition nand命令看到设备节点的挂载点变为空。

```
Huawei LiteOS# partition nand
nand partition num:0, dev name:/dev/nandblk0, mountpt:(null), startaddr:0x0900000,length:0x0200000
nand partition num:1, dev name:/dev/nandblk1, mountpt:(null), startaddr:0x0b00000,length:0x0200000
```

调用 delete_mtd_partition 函数删除分区

调用delete_mtd_partition删除分区前要卸载分区。

```
ret = delete_mtd_partition(1, "nand");
if (ret != 0) {
    printf("delete yaffs error\n");
} else {
    printf("delete yaffs ok\n");
}
```

5.6.3 注意事项

- 对于YAFFS2的多分区特性，分区起始地址可以灵活配置，但分区间地址不能重叠，用户需要根据Flash使用情况，合理配置空间，防止与其它文件系统或其它不适合挂载YFFAS2文件系统的空间发生冲突。
- 最小可添加的分区为一个block大小，但是最小可挂载的分区为9个block大小（由YAFFS2特性决定），要注意区分这两个概念。
- 建议分区使用前先进行擦除操作。
- Huawei LiteOS的YAFFS2文件系统只允许连续打开20个目录。

- YAFFS2文件系统支持的操作有：open, close, read, write, seek, sync, dup, opendir, closedir, readdir, rewinddir, statfs, unlink, mkdir, rmdir, rename, stat, stat64, seek64。
- open打开一个文件，参数有O_TRUNC时，必须同时拥有写的权限，才会将文件中的内容清空。
- YAFFS2当前不支持获取文件的atime、mtime、ctime，使用stat接口获取出来的值为0。
- 目前YAFFS2暂不支持open带 O_DIRECTORY属性的操作，打开目录请使用opendir()。
- YAFFS2支持挂载只读属性，当mount函数的入参为MS_RDONLY时，YAFFS2将开启只读属性，所有带写操作的接口，如write、mkdir、unlink，以及通过非O_RDONLY属性打开的文件，均被拒绝，并传出EACCES错误码。
- open打开一个文件，o_flags必须带有O_RDONLY、O_WRONLY、O_RDWR中的一种属性，o_flags设置的读写权限必须与创建文件时设置的mode匹配。

5.7 RAMFS

5.7.1 概述

基本概念

RAMFS是一个基于RAM且可动态调整大小的文件系统。RAMFS没有后备存储源。虽然在RAMFS中写文件操作也会分配目录项和页缓存，但是数据并不写回到任何其他存储介质上，掉电后数据则会丢失。

RAMFS文件系统把所有的文件都放在RAM中，故读/写操作均发生在RAM中，可以用RAMFS来存储一些临时性或经常要修改的数据，例如/tmp和/var目录，这样既避免了对存储器的读写损耗，也提高了数据读写速度。

Huawei LiteOS的RAMFS是一个简单的文件系统：

- 基于RAM的动态文件系统，是一种缓冲机制。
- 基于虚拟文件系统层（VFS），不能格式化。

5.7.2 开发指导

开发流程

使用RAMFS文件系统，涉及以下几个步骤（各步骤具体操作见下述分解）：

1. 打开菜单，选择FileSystem ---> Enable RAMFS，使能RAMFS文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_RAMFS	使能RAMFS的开关	YES/NO	YES	LOSCFG_FS_VFS

2. [挂载RAMFS文件系统](#)
3. [卸载RAMFS文件系统](#)

挂载 RAMFS 文件系统

调用mount()函数实现ramfs的挂载。

```
ret = mount((const char *)NULL, "/ramfs", "ramfs", 0, NULL);
if (ret) {
    dprintf("mount ramfs err %d\n", ret);
}
```

mount()函数的入参说明参见[挂载FAT文件系统](#)。

除了在源码中调用mount()函数，也可以在Shell中使用mount命令实现，前面3个参数同mount()函数，最后两个参数不需要给出，如下回应信息，表明挂载成功：

```
Huawei LiteOS# mount 0 /ramfs ramfs
mount ok
```

卸载 RAMFS 文件系统

调用umount()函数卸载分区，只需要给出正确的挂载点即可，也可以在Shell中使用umount命令实现，若得到如下回应信息，则表明卸载成功：

```
Huawei LiteOS# umount /ramfs
umount ok
```

5.7.3 注意事项

- RAMFS属于调测功能，默认配置为打开，但在正式商用产品中需禁止使用该功能。
- RAMFS只能挂载一次，一次挂载成功后，后面不能再挂载到其他目录。
- RAMFS文件数量受信号量资源限制，不能超过LOSCFG_BASE_IPC_SEM_LIMIT。
- RAMFS文件系统支持的操作有：open, close, read, write, seek, opendir, closedir, readdir, rewinddir, statfs, unlink, mkdir, rmdir, rename, stat, stat64, seek64。
- open打开一个文件且参数有O_TRUNC时，会将文件中的内容清空。
- 若open未使用mode入参，则无法通过mode入参来控制文件权限。
- RAMFS文件系统的读写指针没有分离，所以以O_APPEND（追加写）方式打开文件后，读指针也在文件尾，读文件前需要手动置位。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

5.8 PROC

5.8.1 概述

基本概念

PROC文件系统是一个伪文件系统，它只存在内存当中，而不占用外存空间。它以文件系统的方式为系统内核数据的操作提供接口。

Huawei LiteOS的PROC是一个虚拟文件系统，不支持多线程操作。

5.8.2 开发指导

接口说明

头文件	接口名	描述
fs/include/ proc_fs.h	proc_fs_init	初始化PROC文件系统
	create_proc_entry	创建PROC文件节点
	remove_proc_entry	删除PROC节点
	proc_mkdir	创建PROC目录节点
	proc_create	创建PROC节点并赋值操作函数

开发流程

使用PROC文件系统涉及以下几个步骤（各步骤详细操作见下述分解）：

1. 打开菜单，选择FileSystem ---> Enable PROC，使能PROC文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_PRO C	使能PROC的开 关	YES/NO	YES	LOSCFG_FS_VFS

2. [初始化PROC文件系统](#)
3. [创建PROC节点](#)
4. [查看PROC节点信息](#)
5. [修改PROC节点信息](#)
6. [删除PROC节点](#)

初始化 PROC 文件系统

调用proc_fs_init()接口实现PROC文件系统的初始化，在Huawei LiteOS系统启动时若看到如下显示，则表示PROC文件系统已初始化成功：

```
Mount procfs finished
```

创建 PROC 节点

- 调用proc_mkdir函数创建目录节点，第一个参数为要创建的节点的名字，第二个参数表示要创建的节点的父节点，传NULL默认为父节点是“/proc”。函数的返回值就是创建出来的proc目录节点。

```
struct proc_dir_entry *handle;

handle = proc_mkdir("test", NULL);
if (handle == NULL) {
    dprintf("create test error!\n");
    return;
}
```

- 调用create_proc_entry函数创建文件节点，第一个参数为要创建的节点的名字，第二个参数为文件的模式（文件的类型和文件的权限），第三个参数表示要创建的节点的父节点，传入NULL默认为父节点是“/proc”，如果父节点当前不存在，需要先调用proc_mkdir函数创建父节点目录。create_proc_entry函数的返回值就是创建出来的proc文件节点。

```
struct proc_dir_entry *handle;

handle = create_proc_entry("mounts", 0, NULL);
if (handle == NULL) {
    dprintf("create mounts error!\n");
    return;
}
```

- 在节点创建成功后，可以给它赋予相应的操作函数，例如：

```
handle->proc_fops = &mounts_proc_fops;
```

若没有赋予操作函数，则会使用系统默认的操作函数。

查看 PROC 节点信息

在Shell中执行cat命令查看节点信息，示例如下：

```
Huawei LiteOS # cat /proc/test
Huawei LiteOS # -----LOG BUFFER STATE-----
MaxLen ReadPos WritePos ButtPos
64(KB) 0 113 65536

-----CURRENT LOG LEVEL-----
vb : 3
sys : 3
region : 3
chnl : 3
vpss : 3
venc : 3
vda : 3
h264e : 3
jpege : 3
vou : 3
viu : 3
rc : 3
aio : 3
ai : 3
ao : 3
aenc : 3
adec : 3
isp : 3
ive : 3
tde : 3
vgs : 3
h265e : 3
```

修改 PROC 节点信息

在Shell中执行writeproc命令修改节点信息，示例如下：

```
Huawei LiteOS # writeproc 'sys=2' >> /proc/test
```

得到如下回应信息：

```
sys=2 >> /proc/test
```

再次cat打印节点信息，会发现sys等级已修改。

```
Huawei LiteOS # cat /proc/test
Huawei LiteOS # -----LOG BUFFER STATE-----
MaxLen ReadPos WritePos ButtPos
64(KB) 0 0 65536
```

```
-----CURRENT LOG LEVEL-----  
vb : 3  
sys : 2  
region : 3  
chnl : 3  
vpss : 3  
venc : 3  
vda : 3  
h264e : 3  
jpege : 3  
vou : 3  
viu : 3  
rc : 3  
aio : 3  
ai : 3  
ao : 3  
aenc : 3  
adec : 3  
isp : 3  
ive : 3  
tde : 3  
vgs : 3  
h265e : 3
```

删除 PROC 节点

调用remove_proc_entry()函数删除节点，第一个参数为要删除的节点的名字，第二个参数为要删除的节点的父节点，传NULL默认为父节点是“/proc”。

5.8.3 注意事项

- PROC文件系统属于调测功能，默认配置为打开，正式商用产品中禁止使用该功能。
- PROC文件系统的文件和目录名长度最大为32。
- proc_fs_init() 只能调用一次，多次调用将会造成PROC文件系统异常。
- PROC文件系统初始化后，ls查看PROC下的文件，大小全是0，只有使用cat命令时能获取系统内核的实时信息，并显示。
- PROC文件系统不支持卸载，不支持创建或删除文件/目录。
- PROC文件系统支持的操作有：open，close，read，write，seek，opendir，closedir，readdir，rewinddir，statfs，unlink，stat，stat64，seek64。
- open未使用mode入参，无法通过mode入参来控制文件权限。
- PROC文件系统仅支持以读写属性open文件。
- PROC文件系统在使用fseek函数时，不支持SEEK_END，即不支持从文件尾开始偏移。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

5.8.4 编程实例

示例描述

在/proc目录下创建一个proc_example文件。可以对该文件进行open/read/write/seek操作。

1. 调用proc_example_init()初始化PROC文件系统，在/proc目录下创建一个proc_example文件，并定义proc_example文件的操作函数。

- 打开文件时，执行example_proc_open()函数，并在该函数中注册example_proc_show()。example_proc_show()函数定义读文件后可以读取的内容。
 - 读取文件时，执行seq_read()函数。
2. 调用proc_test()打开并读取/proc/proc_example，此时seq_read()会先执行example_proc_show()，然后将seq_file中的内容拷贝到buf，最后通过buf回传。

编程示例

```
#include "proc_fs.h"
#include "linux/seq_file.h"

#include "stdio.h"
#include "stdlib.h"
#include "fcntl.h"

static int example_proc_show(struct seq_file *m, void *v)
{
    PRINTK("this is example_proc_show\n");
    seq_printf(m, "%s%s\n", "my test proc file1!", "f");
    return 0;
}

static int example_proc_open(struct inode *inode_ptr, struct proc_file *pf)
{
    PRINTK("this is example_proc_open\n");
    return single_open(pf, example_proc_show, NULL);
}

static const struct proc_file_operations example_proc_fops = {
    .open    = example_proc_open,
    .read    = seq_read,
    .llseek  = seq_lseek,
    .release = seq_release,
    .write   = NULL,
};

void proc_example_init(void)
{
    struct proc_dir_entry *handler;

    PRINTK("proc fs init ...\n");
    proc_fs_init();
    handler = create_proc_entry("proc_example", 0, (struct proc_dir_entry *)NULL);
    if (handler == NULL) {
        PRINT_ERR("creat mounts error!\n");
        return;
    }
    PRINTK("proc_example_init end\n");

    handler->proc_fops = &example_proc_fops;
}

void proc_test(void)
{
    char buf[50] = {0};
    int fd = open("/proc/proc_example", O_NONBLOCK | O_CREAT | O_RDWR, 0777);
    if (fd < 0) {
        PRINTK("fd open err\n");
        return;
    }

    int len = read(fd, buf, 50);
    PRINTK("read data:%s\n", buf);
    close(fd);
}
```

结果验证

```
proc fs init ...
Mount procfs finished.
proc_example_init end
this is example_proc_open
this is example_proc_show
read data:my test proc file1!f
```

完整实例代码

```
proc_test.c
```

5.9 ROMFS

5.9.1 概述

ROMFS (ROM File System) 是一种只读文件系统，其功能比较简单，占用系统资源较小，不支持文件的写入，只支持读取。设计ROMFS的目的是在启动盘等场合下，提供一个比普通文件系统更加节省空间的文件系统。

Huawei LiteOS ROMFS支持SD卡/U盘或者NOR Flash。创建ROMFS文件系统需要使用genromfs工具。在Linux上可以使用该工具，将目录制作成ROMFS镜像并存储到SD卡/U盘、或者烧写到NOR Flash中，然后在Huawei LiteOS上挂载后读取内容。

5.9.2 开发指导

开发流程

使用ROMFS文件系统所涉及的步骤所示如下（各步骤操作见下述分解）。

1. 打开菜单，选择FileSystem ---> Enable ROMFS，使能ROMFS文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_ROMFS	使能ROMFS的开关	YES/NO	YES	LOSCFG_FS_VFS

2. [制作ROMFS文件系统镜像](#)
3. [烧写ROMFS镜像](#)
4. [挂载ROMFS文件系统](#)
5. [卸载ROMFS文件系统](#)

制作 ROMFS 文件系统镜像

- SD卡或U盘

将SD卡或者U盘插入到Linux系统的电脑上，系统识别出SD卡或者U盘后，会在/dev/目录下会增加一个设备，如/dev/sda1。在Linux系统上创建一个目录，将需要制作成ROMFS文件系统的文件拷贝到该目录。然后使用genromfs工具将目录制作成ROMFS镜像，如下所示（假设目录名为romfs_dir，设备名为/dev/sda1）。


```
sudo genromfs -d romfs_dir -f /dev/sda1
```

- NOR Flash

在Linux系统电脑上创建一个目录，将需要制作成ROMFS文件系统的文件拷贝到该目录。然后使用genromfs工具将目录制作成ROMFS镜像。如下所示（假设目录名为romfs_dir，制作镜像名为romfs.img）。

```
sudo genromfs -d romfs_dir -f romfs.img
```

烧写 ROMFS 镜像

- SD卡或U盘

将SD卡或者U盘插到开发板。

- NOR Flash

将制作出的ROMFS镜像烧到NOR Flash中。

挂载 ROMFS 文件系统

- SD卡或U盘

插入SD卡或者U盘后，Huawei LiteOS会自动识别出设备，如/dev/mmcbk0p0，并在Shell中运行命令挂载SD卡或者U盘为ROMFS文件系统：

```
Huawei LiteOS# mount /dev/mmcbk0p0 /romfs romfs
```

若得到如下回应信息，则表明挂载成功。

```
mount ok
```

- NOR Flash

首先调用add_mtd_partition函数添加分区，该函数会自动为设备节点命名，其命名规则是“/dev/spinorblk”加上分区号。注意分区起始地址和长度要与烧写ROMFS镜像时一致。如下所示：

```
if (ret = add_mtd_partition("spinor", 0x100000, 0x500000, 0) != 0) {  
    dprintf("add partition failed, return %d\n", ret);  
}
```

添加分区后在Huawei LiteOS上可以识别出NOR Flash分区，如/dev/spinorblk0，运行mount命令挂载ROMFS文件系统，如下所示：

```
Huawei LiteOS# mount /dev/spinorblk0 /romfs romfs
```

若得到如下回应信息，则表明挂载成功。

```
mount ok
```

卸载 ROMFS 文件系统

运行命令：

```
Huawei LiteOS# umount /romfs
```

若从串口得到如下回应信息，则表明卸载成功。

```
umount ok
```

5.9.3 注意事项

- ROMFS文件系统总大小最大支持2G。
- ROMFS文件系统单个文件大小最大支持256M。
- ROMFS文件系统文件名和目录名长度不能超过16字节。
- open一个文件时，参数必须为O_RDONLY，否则会打开失败。

- 若open未使用mode入参，则无法通过mode入参来控制文件权限。
- ROMFS文件系统支持的操作有：open，close，read，seek，dup，opendir，closedir，readdir，rewinddir，statfs，stat，stat64，seek64。
- ROMFS不支持软链接，支持硬链接。
- 使用fread接口进行读操作，默认开启缓冲区，每次读512个字节，可能会导致读速率慢的问题。可用setvbuf接口设置为无缓冲区模式，第三个参数传_IONBF即可。
- ROMFS可用于SD卡/U盘或者Nor Flash，默认为Nor Flash，目前仅支持源码配置。

5.10 LITTLEFS

5.10.1 概述

基本概念

LITTLEFS是LITTLE File System（微型文件系统）的缩写，是一种专门为微控制器设计的故障保护文件系统，主要应用于NOR Flash闪存，其特点包括：

- 可读写
- 提供损耗平衡
- 提供掉电安全保护，在修改文件系统中的数据时，即使发生意外也不损坏数据。
- 内存和代码很精简，占用很少的RAM和FLASH。

LITTLEFS是基于块的文件系统，磁盘被分成大小均匀的块阵列，作为存储的逻辑单元，这些块在使用时由公共块分配器进行分配。LITTLEFS使用小日志存储元数据，并使用较大的写时复制（COW）结构存储文件数据。

Huawei LiteOS的LITTLEFS主要应用于NOR Flash闪存的文件管理，且支持多分区。

5.10.2 开发指导

接口说明

头文件	接口名	描述
fs/include/ mtd_partition. h	add_mtd_partition	添加文件系统分区，该函数会自动为设备节点命名，对于LITTLEFS，其命名规则是“/dev/spinorblk0”
	delete_mtd_partition	删除已经卸载的分区
open_source/ musl/ include/sys/ mount.h	mount	挂载文件系统
	umount	卸载文件系统

开发流程

- 1. 使能LITTLEFS
- 2. 添加LITTLEFS分区
- 3. 挂载LITTLEFS分区
- 4. 卸载LITTLEFS分区
- 5. 删除LITTLEFS分区

使能 LITTLEFS

打开菜单，选择FileSystem ---> Enable LITTLEFS，使能LITTLEFS文件系统。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_LITTLEFS	使能LITTLEFS的开关	YES/NO	YES	LOSCFG_FS_VFS && LOSCFG_DRIVERS_MTD_SPI_NOR

添加 LITTLEFS 分区

调用add_mtd_partition()函数添加LITTLEFS分区。

```
if (add_mtd_partition("spinor", 0x100000, 0x500000, 0) != 0) {
    dprintf("add spinor partition failed\n");
}
```

说明

add_mtd_partition()添加LITTLEFS分区时，系统会自动根据设备block大小将分区起始地址和大小对齐。

该函数有四个参数：

- 第一个参数表示介质，支持“nand”和“spinor”。LITTLEFS分区在“spinor”上使用。
- 第二个参数表示分区的起始地址，以16进制的形式传入。
- 第三个参数表示分区大小，以16进制的形式传入。
- 最后一个参数表示分区号，有效值为0~19。

成功后，可以在Shell中使用partition spinor命令查看spinor flash分区信息，更多请详见[partition](#)。

```
Huawei LiteOS# partition spinor
spinor partition num:0, dev name:/dev/spinorblk0, mountpt:(null), startaddr:0x0100000, length:0x0500000
```

挂载 LITTLEFS 分区

调用mount()函数实现设备节点的挂载。

```
ret = mount("/dev/spinorblk0", "/littlefs0", "littlefs", 0, NULL);
if (ret != 0) {
    dprintf("ERROR:mount littlefs failed\n");
}
```

📖 说明

mount()函数有五个参数：

- 第一个参数表示设备节点，这个参数须与add_mtd_partition()函数对应。
- 第二个参数表示挂载点。
- 第三个参数表示文件系统类型。
- 第四个参数表示挂载标志，默认为0。
- 最后一个参数表示私有数据，默认为NULL。

调用mount()函数是否成功，可以在Shell中执行ls查看当前根目录下的目录。

```
Huawei LiteOS# ls
Directory /:
bin          <DIR>
dev          <DIR>
littlefs0    <DIR>
```

挂载成功后，可以对NOR Flash进行读写操作。

卸载 LITTLEFS 分区

调用umount()函数卸载分区，只需要给出正确的挂载点即可。

```
ret = umount("/littlefs0");
if (ret != 0) {
    dprintf("umount littlefs err\n");
}
```

📖 说明

umount()函数只有一个“表示挂载点”的参数。

删除 LITTLEFS 分区

调用delete_mtd_partition()删除已经卸载的分区。

```
delete_mtd_partition(0, "spinor");
```

📖 说明

delete_mtd_partition()函数有两个参数：

- 第一个参数是分区号。
- 第二个参数为介质类型，该参数与add_mtd_partition()函数对应。

5.10.3 注意事项

- 如果以O_TRUNC方式open一个文件时，会将文件中的内容清空。
- 若open未使用mode入参，则无法通过mode入参来控制文件权限。
- LITTLEFS文件系统的读写指针分离，以O_APPEND（追加写）方式打开文件后，读指针在文件头，写指针在文件尾。
- LITTLEFS文件系统用于NOR Flash，最终调用NOR Flash驱动接口，因此使用LITTLEFS文件系统之前需要保证存在NOR Flash闪存，且驱动已初始化成功（spinor_init()返回0）。
- 目前LITTLEFS文件系统支持的操作有：open, close, read, write, seek, opendir, closedir, readdir, rewinddir, unlink, mkdir, rmdir, rename, stat, stat64, seek64。

- 当前LITTLEFS读写cache的默认大小为1024字节，可以在fs/littlefs/include/vfs_littlefs.h文件中修改LFS_CACHE_SIZE的大小，注意必须为LFS_MIN_CACHE_SIZE的倍数，且不能大于65536字节。
- 打开文件并写入数据，若在未close的情况下rename，数据会丢失。
- 目前access操作只能判断文件是否存在，不能判断文件是否具有指定的权限。
- 目前stat操作只能获取文件名、大小、文件类型（文件或目录），无法获取文件权限。
- 目前LITTLEFS不支持open带O_DIRECTORY属性的操作，打开目录请使用opendir()。
- 若同时打开同一文件，必须全部以只读方式打开。若以可写方式同时打开一个文件并写入数据，必须close才能保存，且只保存最后一次close的数据。
- LITTLEFS支持挂载只读属性，当mount函数的入参mountflags为MS_RDONLY时，LITTLEFS将开启只读属性，所有带有写操作的接口，如write、mkdir、unlink，以及通过非O_RDONLY属性打开的文件，均将被拒绝，并传出EACCESS错误码。

6 兼容接口

6.1 概述

6.2 POSIX接口

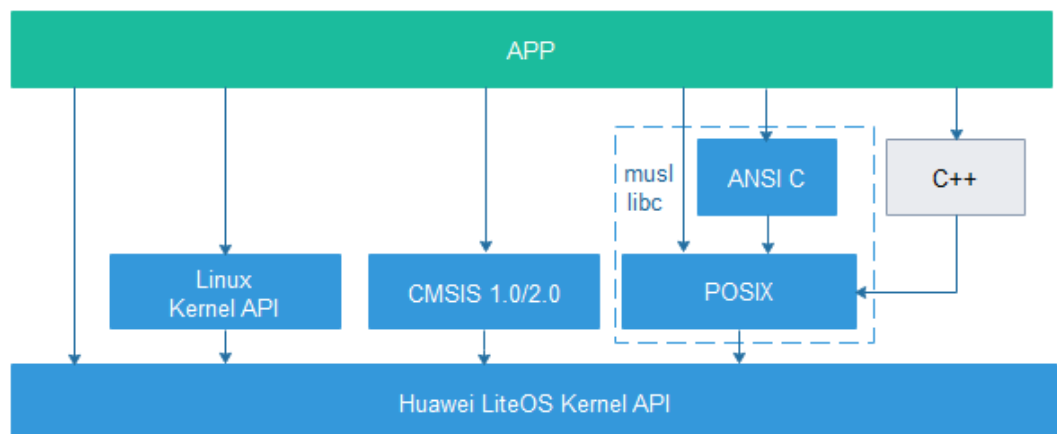
6.3 Libc/Libm接口

6.4 C++兼容规格

6.5 CMSIS接口

6.6 linux适配

6.1 概述



Huawei LiteOS提供了一套自有OS接口，同时也支持musl libc库、C++标准库、CMSIS接口和Linux接口。上层APP建议调用musl libc库等兼容接口，不建议直接调用Huawei LiteOS的自有OS接口。下面章节将列出Huawei LiteOS支持的兼容接口。

⚠ 注意

LitesOS适配的libc库函数，遵循开源musl libc的实现形式，musl开源代码遵循标准C规范ISO/IEC 9899:1999和POSIX标准1003.1™-2008：作为最简单的底层库函数，内部不对指针类型参数做非空判断，由使用者保证入参的合法性。

其中LiteOS实现的获取时间类接口都是自系统起调度算起，与接口标准定义不一致，使用时需特别注意。

6.2 POSIX 接口

6.2.1 POSIX 支持接口

Huawei LiteOS提供一套POSIX适配接口，具体的规格参见下表。

头文件	接口名	类型	说明
arpa/inet.h	inet_addr	网络函数	转换字符串IP为数字格式
arpa/inet.h	inet_ntoa	网络函数	将网络二进制的数字转换成网络地址
arpa/inet.h	inet_ntop	网络函数	将二进制整数转为点分十进制
arpa/inet.h	inet_pton	网络函数	将点分十进制转为二进制整数
ctype.h	isalnum_l	数据判断	判断入参是否为字母或数字
ctype.h	isalpha_l	数据判断	判断入参是否为字母
ctype.h	isascii	数据判断	判断入参是否为ascii码
ctype.h	isblank	数据判断	判断入参是否为空格或tab
ctype.h	isblank_l	数据判断	判断入参是否为空格或tab
ctype.h	iscntrl_l	数据判断	判断入参是否为控制字符
ctype.h	isdigit_l	数据判断	判断入参是否为十进制数
ctype.h	isgraph_l	数据判断	判断入参是否有图形表示

头文件	接口名	类型	说明
ctype.h	islower_l	数据判断	判断入参是否为小写字母
ctype.h	isprint_l	数据判断	判断入参是否可以打印
ctype.h	ispunct_l	数据判断	判断入参是否为标点符号
ctype.h	isspace_l	数据判断	判断入参是否为空格
ctype.h	isupper_l	数据判断	判断入参是否为大写字母
ctype.h	isxdigit_l	数据判断	判断入参是否为十六进制数
ctype.h	toascii	数据转换函数	将字符转换成ascii码
ctype.h	tolower_l	数据转换函数	将大写字母转换成小写字母
ctype.h	toupper_l	数据转换函数	将小写字母转换成大写字母
dirent.h	alphasort	目录操作	依字母顺序排序目录结构
dirent.h	closedir	目录操作	关闭目录
dirent.h	opendir	目录操作	打开目录
dirent.h	readdir	目录操作	读取目录
dirent.h	rewinddir	目录操作	重设读取目录的位置为开头位置
dirent.h	scandir	目录操作	扫描目录完成条件过滤
dirent.h	seekdir	目录操作	设置下回读取目录的位置
dirent.h	telldir	目录操作	取得目录流的读取位置
dlfcn.h	dlclose	动态加载	卸载一个模块
dlfcn.h	dlopen	动态加载	动态加载一个so模块

头文件	接口名	类型	说明
dlfcn.h	dlsym	动态加载	在模块或者系统符号表中查找符号
fcntl.h	creat	文件操作函数	创建一个文件
fcntl.h	fcntl	文件操作函数	文件描述词操作
fcntl.h	open	文件操作函数	打开文件
langinfo.h	nl_langinfo	系统函数	返回指定的本地信息
langinfo.h	nl_langinfo_l	系统函数	返回指定的本地信息
libgen.h	basename	文件路径	返回文件名
mqqueue.h	mq_close	消息队列	关闭消息队列
mqqueue.h	mq_getattr	消息队列	获取消息队列属性
mqqueue.h	mq_open	消息队列	打开消息队列
mqqueue.h	mq_receive	消息队列	接受一个消息队列中的消息
mqqueue.h	mq_send	消息队列	发送一个消息到消息队列
mqqueue.h	mq_setattr	消息队列	设置消息队列属性
mqqueue.h	mq_timedreceive	消息队列	定时接收消息
mqqueue.h	mq_timedsend	消息队列	定时发送消息
mqqueue.h	mq_unlink	消息队列	移除消息队列
net/if.h	if_freenameindex	网络函数	通过if_nameindex()获取完毕接口名称与索引后，调用该函数以释放动态分配的内存区域
net/if.h	if_indextoname	数据转换函数	将网卡序号转为网卡名

头文件	接口名	类型	说明
net/if.h	if_nameindex	网络函数	返回动态分配的struct if_nameindex结构数组，数组中的每一个元素分别对应一个本地网络接口
net/if.h	if_nametoindex	数据转换函数	将网卡名转为网卡序号
netdb.h	freeaddrinfo	网络函数	释放getaddrinfo申请的内存空间
netdb.h	getaddrinfo	网络函数	将主机名和服务名转换到socket地址
netdb.h	getnameinfo	网络函数	将socket地址转换到主机名和服务名
poll.h	poll	文件操作函数	监视文件描述符
pthread.h	pthread_attr_destroy	pthread	删除线程的属性
pthread.h	pthread_attr_get_detachstate	pthread	获取脱离状态的属性
pthread.h	pthread_attr_get_inheritsched	pthread	获取任务调度方式
pthread.h	pthread_attr_get_schedparam	pthread	获取任务调度参数
pthread.h	pthread_attr_get_schedpolicy	pthread	获取任务调度策略属性，目前仅支持SCHED_RR调度策略，不支持SCHED_OTHERSCHED_FIFO SCHED_RR
pthread.h	pthread_attr_get_scope	pthread	获取任务范围属性，任务使用范围目前只支持PTHREAD_SCOPE_SYSTEM，不支持PTHREAD_SCOPE_PROCESS
pthread.h	pthread_attr_get_stackaddr	pthread	获取任务堆栈的起始地址
pthread.h	pthread_attr_get_stacksize	pthread	获取任务属性堆栈大小
pthread.h	pthread_attr_init	pthread	初始化任务属性
pthread.h	pthread_attr_set_detachstate	pthread	设置任务属性分离状态
pthread.h	pthread_attr_set_inheritsched	pthread	设置任务调度方式

头文件	接口名	类型	说明
pthread.h	pthread_attr_set schedparam	pthread_d	设置任务调度参数，注意在LiteOS中，任务优先级的值越小，任务在系统中的优先级就越高，与标准库函数相反，且优先级仅支持0~31。注意：需要将pthread_attr_t任务属性的inheritsched字段设置为PTHREAD_EXPLICIT_SCHED，否则设置的任务调度优先级将不会生效，系统默认设置为PTHREAD_INHERIT_SCHED
pthread.h	pthread_attr_set schedpolicy	pthread_d	设置任务调度策略属性，目前仅支持SCHED_RR调度策略，不支持SCHED_OTHERSCHED_FIFO SCHED_RR
pthread.h	pthread_attr_set scope	pthread_d	设置任务范围属性，任务使用范围目前只支持PTHREAD_SCOPE_SYSTEM，不支持PTHREAD_SCOPE_PROCESS
pthread.h	pthread_attr_set stackaddr	pthread_d	设置任务堆栈的起始地址
pthread.h	pthread_attr_set stacksize	pthread_d	设置任务属性堆栈大小
pthread.h	pthread_cancel	pthread_d	取消任务，仅支持先设置PTHREAD_CANCEL_ASYNCHRONOUS状态，再调用pthread_cancel取消任务
pthread.h	pthread_cond_broadcast	pthread_d	唤醒所有被阻塞在条件变量上的线程
pthread.h	pthread_cond_destroy	pthread_d	释放条件变量
pthread.h	pthread_cond_init	pthread_d	初始化条件变量
pthread.h	pthread_cond_signal	pthread_d	释放被阻塞在条件变量上的一个线程
pthread.h	pthread_cond_timedwait	pthread_d	超时时限内等待一个条件变量，当超时等待时间为相对时间，LiteOS不能处理早已超时的情况
pthread.h	pthread_cond_wait	pthread_d	等待一个条件变量
pthread.h	pthread_condattr_destroy	pthread_d	删除存储并使属性对象无效
pthread.h	pthread_condattr_getclock	pthread_d	获取任务时钟

头文件	接口名	类型	说明
pthread.h	pthread_condattr_getpshared	pthread_t	获取条件变量属性，目前只支持获取 PTHREAD_PROCESS_PRIVATE 条件变量属性
pthread.h	pthread_condattr_init	pthread_t	初始化条件变量属性
pthread.h	pthread_condattr_setclock	pthread_t	设置任务时钟，只支持 CLOCK_MONOTONIC 或 CLOCK_REALTIME 模式
pthread.h	pthread_condattr_setpshared	pthread_t	设置条件变量属性，只支持 PTHREAD_PROCESS_PRIVATE 属性
pthread.h	pthread_create	pthread_t	创建任务
pthread.h	pthread_detach	pthread_t	分离任务
pthread.h	pthread_equal	pthread_t	判断是否为同一任务
pthread.h	pthread_exit	pthread_t	任务退出
pthread.h	pthread_getschedparam	pthread_t	获取任务优先级及调度参数，目前仅支持 SCHED_RR 调度策略，不支持 SCHED_OTHER SCHED_FIFO SCHED_RR
pthread.h	pthread_getspecific	pthread_t	获取调用线程的键绑定
pthread.h	pthread_join	pthread_t	阻塞任务
pthread.h	pthread_key_create	pthread_t	分配用于标识进程中线程特定数据的键
pthread.h	pthread_key_delete	pthread_t	销毁现有线程特定数据键
pthread.h	pthread_mutex_destroy	pthread_t	删除互斥锁
pthread.h	pthread_mutex_getprioceiling	pthread_t	获取互斥锁的优先级上限
pthread.h	pthread_mutex_init	pthread_t	初始化互斥锁
pthread.h	pthread_mutex_lock	pthread_t	申请互斥锁（阻塞操作）

头文件	接口名	类型	说明
pthread.h	pthread_mutex_setprioceiling	pthread_t	设置互斥锁的优先级上限
pthread.h	pthread_mutex_timedlock	pthread_t	申请互斥锁（只在设定时间内阻塞）
pthread.h	pthread_mutex_trylock	pthread_t	尝试申请互斥锁（非阻塞）
pthread.h	pthread_mutex_unlock	pthread_t	释放互斥锁
pthread.h	pthread_mutexattr_destroy	pthread_t	销毁互斥锁属性对象
pthread.h	pthread_mutexattr_getprioceiling	pthread_t	获取互斥锁属性的优先级上限
pthread.h	pthread_mutexattr_getprotocol	pthread_t	获取互斥锁属性的协议属性
pthread.h	pthread_mutexattr_gettype	pthread_t	获取互斥锁的类型属性
pthread.h	pthread_mutexattr_init	pthread_t	初始化互斥锁属性对象
pthread.h	pthread_mutexattr_setprioceiling	pthread_t	设置互斥锁属性的优先级上限
pthread.h	pthread_mutexattr_setprotocol	pthread_t	设置互斥锁属性的协议属性
pthread.h	pthread_mutexattr_settype	pthread_t	设置互斥锁的类型属性
pthread.h	pthread_once	pthread_t	一次性操作任务
pthread.h	pthread_self	pthread_t	获取任务ID
pthread.h	pthread_setcancelstate	pthread_t	任务cancel功能开关
pthread.h	pthread_setcanceltype	pthread_t	设置任务cancel类型
pthread.h	pthread_setschedparam	pthread_t	设置任务优先级及调度策略，目前仅支持SCHED_RR调度策略，不支持SCHED_OTHER和SCHED_FIFO SCHED_RR，任务优先级仅支持0~31

头文件	接口名	类型	说明
pthread.h	pthread_setschedprio	pthread_t	设置任务优先级，任务优先级仅支持0~31
pthread.h	pthread_setspecific	pthread_t	设置线程数据
pthread.h	pthread_testcancel	pthread_t	cancel任务
sched.h	sched_get_priority_max	调度函数	获取系统支持的最大的优先级值
sched.h	sched_get_priority_min	调度函数	获取系统支持的最小的优先级值
sched.h	sched_yield	调度函数	使当前线程放弃占用CPU
semaphore.h	sem_destroy	信号量	销毁无名信号量
semaphore.h	sem_getvalue	信号量	获取指定信号量的值
semaphore.h	sem_init	信号量	初始化无名信号量
semaphore.h	sem_post	信号量	释放一个指定的无名信号量
semaphore.h	sem_timedwait	信号量	申请一个超时等待的无名信号量，当超时等待时间为相对时间时，LiteOS不能处理早已超时的情况
semaphore.h	sem_trywait	信号量	尝试申请一个无名信号量
semaphore.h	sem_wait	信号量	申请等待一个无名信号量
stdio.h	dprintf	标准IO操作	打印函数
stdio.h	fdopen	文件操作函数	取一个现存的文件描述符，并使一个标准的I/O流与该描述符相结合
stdio.h	fileno	文件操作函数	取得参数stream指定的文件流所使用的文件描述符，如果文件流已经关闭，则返回-1，并设置错误码
stdio.h	fseeko	文件操作函数	移动文件流的读写位置
stdio.h	ftello	文件操作函数	获取文件流的读取位置

头文件	接口名	类型	说明
stdlib.h	initstate	随机数函数	初始化随机数生成状态
stdlib.h	lrand48	随机数函数	取得一个正的长整型的随机数
stdlib.h	mkstemp	文件处理函数	建立唯一的临时文件
stdlib.h	rand48	随机数函数	产生正随机数
stdlib.h	posix_memalign	内存操作	对齐申请内存，返回内存指针
stdlib.h	random	随机数函数	产生0~RAND_MAX范围内的随机数
stdlib.h	realpath	文件处理函数	相对路径转换成绝对路径，入参 resolved_path 的长度必须不小于 PATH_MAX
stdlib.h	seed48	随机数函数	设置48位随机数种子
stdlib.h	setstate	随机数函数	设置随机数状态
stdlib.h	srand48	随机数函数	设置48位随机数种子
stdlib.h	srandom	随机数函数	产生随机数种子
string.h	strcpy	字符串处理函数	赋值字符串到数组
string.h	strncpy	字符串处理函数	赋值字符串到数组
string.h	strcoll_l	字符串处理函数	根据 LC_COLLATE 比较字符串
string.h	strdup	字符串处理函数	拷贝字符串到新申请内存中
string.h	strerror_l	字符串处理函数	返回一个指向错误消息字符串的指针

头文件	接口名	类型	说明
string.h	strerror_r	字符串处理函数	返回一个指向错误消息字符串的指针（可重入）
string.h	strlen	字符串处理函数	返回字符串长度
string.h	strtok_r	字符串处理函数	分割字符串
string.h	strxfrm_l	字符串处理函数	根据LC_COLLATE来转换字符串src，若传入n的长度小于字符串src的长度，则不作转换，直接返回字符串src的长度
strings.h	ffs	位操作函数	查找一个整数中的第一个置位值
strings.h	strcasecmp	字符串处理函数	字符串比较
strings.h	strcasecmp_l	字符串处理函数	字符串比较
strings.h	strncasecmp	字符串处理函数	字符串比较
strings.h	strncasecmp_l	字符串处理函数	字符串比较
sys/ioctl.h	ioctl	文件操作函数	执行指定操作
sys/select.h	select	文件操作函数	监视文件描述符，用户需要保证待监测的文件描述符为小于FD_SETSIZE的正值，否则可能产生不确定的行为
sys/socket.h	accept	网络函数	接受套接字上的新连接
sys/socket.h	bind	网络函数	将套接字绑定到一个地址
sys/socket.h	connect	网络函数	建立套接字连接
sys/socket.h	getpeername	网络函数	获取与某个套接字关联的外地协议地址

头文件	接口名	类型	说明
sys/ socket.h	getsockname	网络函数	获取套接字的名字
sys/ socket.h	getsockopt	网络函数	获取套接字的状态
sys/ socket.h	listen	网络函数	将套接字修改为监听状态，等待连接
sys/ socket.h	recv	网络函数	从连接的套接字接收消息
sys/ socket.h	recvfrom	网络函数	从连接或者非连接套接字接收消息
sys/ socket.h	recvmsg	网络函数	经过指定的套接字接收消息
sys/ socket.h	send	网络函数	经套接字发送消息
sys/ socket.h	sendmsg	网络函数	经过指定的套接字发送消息
sys/ socket.h	sendto	网络函数	经过指定的套接字发送消息，若连接发送给对端，若非连接发送给参数指定的套接字
sys/ socket.h	setsockopt	网络函数	设置套接字状态
sys/ socket.h	shutdown	网络函数	终止套接字通信
sys/ socket.h	socket	网络函数	创建一个套接字
sys/stat.h	fstat	文件操作接口	获取文件属性
sys/stat.h	lstat	文件操作接口	获取文件属性
sys/stat.h	mkdir	目录操作接口	创建目录
sys/stat.h	stat	文件操作接口	获取文件属性
sys/time.h	gettimeofday	时间函数	获取当前时间，以秒和微秒的格式返回
sys/times.h	times	时间函数	获取cpu tick到buff中

头文件	接口名	类型	说明
sys/uio.h	readv	文件操作函数	读文件
sys/uio.h	writew	文件操作函数	写文件
sys/utsname.h	uname	系统函数	获取系统信息，可配置LOSCFG_LIB_VENDORNAME选项改变返回值的nodename成员，不返回硬件标识符和域名信息
syslog.h	syslog	系统函数	输出系统日志，不支持第一个入参按等级输出日志
time.h	asctime_r	时间函数	将时间和日期以字符串形式表示
time.h	clock_getres	时间函数	获取对应时钟类型能够提供的精确度
time.h	clock_gettime	时间函数	获取指定时钟的时间
time.h	clock_settime	时间函数	设置指定时钟的时间
time.h	ctime_r	时间函数	将时间和日期以字符串形式表示
time.h	gmtime_r	时间函数	取得目前的时间和日期
time.h	localtime_r	时间函数	取得当地目前的时间和日期
time.h	nanosleep	时间函数	进程以纳秒为单位休眠，但目前只支持tick（默认10ms）休眠，第二个参数不支持，且传参所设置的秒数不能大于4292秒
time.h	strftime_l	时间函数	格式化日期和时间，格式化字符不支持%z或%Z的格式化转换
time.h	strptime	时间函数	按照特定时间格式将字符串转换为时间类型
time.h	timer_create	时间函数	创建定时器，只支持SIGEV_THREAD在线程内处理（ LiteOS有两种实现方式，在线程或者在中断中处理timer的回调函数，具体采用哪种方式由宏开关LOSCFG_BASE_CORE_SWTMR_IN_ISR控制）； clock_id只支持CLOCK_REALTIME

头文件	接口名	类型	说明
time.h	timer_delete	时间函数	删除定时器
time.h	timer_getoverrun	时间函数	获取定时器超时次数
time.h	timer_gettime	时间函数	获得一个定时器剩余时间
time.h	timer_settime	时间函数	初始化或者撤销定时器， <i>it_interval</i> 参数为0时，定时器使用完毕同样需要手动删除
unistd.h	access	文件操作函数	判断文件权限
unistd.h	chdir	目录操作接口	改变当前工作目录
unistd.h	close	文件操作接口	关闭文件
unistd.h	dup	文件操作函数	复制文件
unistd.h	dup2	文件操作函数	复制文件
unistd.h	fsync	文件操作函数	文件同步
unistd.h	ftruncate	文件操作函数	改变文件大小
unistd.h	getcwd	目录操作接口	获取当前工作目录
unistd.h	getopt	系统函数	分析命令行参数
unistd.h	getpid	系统函数	获取任务ID
unistd.h	isatty	文件操作函数	判断文件是否为终端设备
unistd.h	lseek	文件操作函数	改变文件读写指针
unistd.h	pread	文件操作函数	读文件
unistd.h	pwrite	文件操作函数	写文件

头文件	接口名	类型	说明
unistd.h	read	文件操作函数	读文件
unistd.h	rmdir	目录操作接口	删除目录
unistd.h	sleep	时间函数	以秒为单位阻塞进程
unistd.h	sync	文件操作函数	文件同步
unistd.h	sysconf	系统函数	获取系统参数
unistd.h	unlink	文件操作函数	删除文件
unistd.h	write	文件操作函数	写文件
utime.h	utime	文件操作函数	修改文件时间
wchar.h	mbsnrtowcs	宽字符处理函数	多字节序列转换为宽字符（有最大长度限制）
wchar.h	wcscoll_l	宽字符处理函数	采用目前区域的字符排列次序来比较宽字符串
wchar.h	wcsnlen	宽字符处理函数	返回多字节字符的长度（有最大长度限制）
wchar.h	wcsnrtombs	转换函数	把数组中存储的编码转换为多字节字符（有最大长度限制）
wchar.h	wcsxfrm_l	宽字符处理函数	根据程序当前的区域选项中的 LC_COLLATE 来转换宽字符串的前 n 个字符
wctype.h	iswalnum_l	宽字符处理函数	判断宽字符是否字母或数字
wctype.h	iswalpha_l	宽字符处理函数	判断宽字符是否为英文字母
wctype.h	iswblank	宽字符处理函数	判断宽字符是否为 TAB 或者空格

头文件	接口名	类型	说明
wctype.h	iswblank_l	宽字符处理函数	判断宽字符是否为TAB或者空格
wctype.h	iswcntrl_l	宽字符处理函数	判断宽字符是否为控制字符
wctype.h	iswctype_l	宽字符处理函数	判断是否为宽字符类型
wctype.h	iswdigit_l	宽字符处理函数	判断宽字符是否为阿拉伯字符0到9
wctype.h	iswgraph_l	宽字符处理函数	判断宽字符是否为除空格外的可打印字符
wctype.h	iswlower_l	宽字符处理函数	判断宽字符是否为小写英文字母
wctype.h	iswprint_l	宽字符处理函数	判断宽字符是否为可打印字符
wctype.h	iswpunct_l	宽字符处理函数	判断宽字符是否为标点符号或特殊符号
wctype.h	iswspace_l	宽字符处理函数	判断宽字符是否为空格字符
wctype.h	iswupper_l	宽字符处理函数	判断宽字符是否为大写英文字母
wctype.h	iswxdigit_l	宽字符处理函数	判断宽字符是否为16进制数字
wctype.h	tolower_l	宽字符处理函数	将宽字符转换成小写字母
wctype.h	toupper_l	宽字符处理函数	将宽字符转换成大写字母

头文件	接口名	类型	说明
wctype.h	wctype_l	宽字符处理函数	判断是否为宽字符类型

6.2.2 POSIX NP 支持接口

Huawei LiteOS还提供了一套POSIX NP（nonportable）适配接口，以支持部分SMP（多核）的NP接口。NP接口为非POSIX标准接口，作为POSIX接口的补充。以下接口只有在开启多核模式下可以操作亲和性，在单核模式下直接返回ENOERR。

文件名	接口名	说明
pthread.h	pthread_attr_setaffinity_np	设置attr亲和性属性
pthread.h	pthread_attr_getaffinity_np	获取attr亲和性属性
pthread.h	pthread_setaffinity_np	设置pthread任务的亲和性
pthread.h	pthread_getaffinity_np	获取pthread任务的亲和性

6.2.3 POSIX 不支持接口

Huawei LiteOS的POSIX接口中，有一些未支持，具体参见下表。

头文件	接口名
aio.h	aio_cancel
aio.h	aio_error
aio.h	aio_fsync
aio.h	aio_read
aio.h	aio_return
aio.h	aio_suspend
aio.h	aio_write
aio.h	lio_listio
dirent.h	dirfd
dirent.h	fdopendir

头文件	接口名
dirent.h	readdir_r
dlfcn.h	dLError
fcntl.h	openat
fcntl.h	posix_fadvise
fcntl.h	posix_fallocate
fmtmsg.h	fmtmsg
fnmatch.h	fnmatch
ftw.h	ftw
ftw.h	nftw
glob.h	glob
glob.h	globfree
grp.h	endgrent
grp.h	getgrent
grp.h	getgrgid
grp.h	getgrgid_r
grp.h	getgrnam
grp.h	getgrnam_r
grp.h	setgrent
iconv.h	iconv
iconv.h	iconv_close
iconv.h	iconv_open
inttypes.h	imaxabs
inttypes.h	imaxdiv
libgen.h	dirname
locale.h	duplocale
locale.h	freelocale
locale.h	newlocale
locale.h	uselocale
monetary.h	strfmon
monetary.h	strfmon_l

头文件	接口名
mqqueue.h	mq_notify
netdb.h	endhostent
netdb.h	endnetent
netdb.h	endprotoent
netdb.h	endservent
netdb.h	gai_strerror
netdb.h	gethostent
netdb.h	getnetbyaddr
netdb.h	getnetbyname
netdb.h	getnetent
netdb.h	getprotobyname
netdb.h	getprotobynumber
netdb.h	getprotoent
netdb.h	getservbyname
netdb.h	getservbyport
netdb.h	getservent
netdb.h	sethostent
netdb.h	setnetent
netdb.h	setprotoent
netdb.h	setservent
nl_types.h	catclose
nl_types.h	catgets
nl_types.h	catopen
pthread.h	pthread_atfork
pthread.h	pthread_attr_getguardsize
pthread.h	pthread_attr_getstack
pthread.h	pthread_attr_setguardsize
pthread.h	pthread_attr_setstack
pthread.h	pthread_barrier_destroy
pthread.h	pthread_barrier_init

头文件	接口名
pthread.h	pthread_barrier_wait
pthread.h	pthread_barrierattr_destroy
pthread.h	pthread_barrierattr_getpshared
pthread.h	pthread_barrierattr_init
pthread.h	pthread_barrierattr_setpshared
pthread.h	pthread_getattr_default_np
pthread.h	pthread_getattr_np
pthread.h	pthread_getconcurrency
pthread.h	pthread_getcpuclockid
pthread.h	pthread_getname_np
pthread.h	pthread_mutex_consistent
pthread.h	pthread_mutexattr_getpshared
pthread.h	pthread_mutexattr_getrobust
pthread.h	pthread_mutexattr_setpshared
pthread.h	pthread_mutexattr_setrobust
pthread.h	pthread_rwlock_destroy
pthread.h	pthread_rwlock_init
pthread.h	pthread_rwlock_rdlock
pthread.h	pthread_rwlock_timedrdlock
pthread.h	pthread_rwlock_timedwrlock
pthread.h	pthread_rwlock_tryrdlock
pthread.h	pthread_rwlock_trywrlock
pthread.h	pthread_rwlock_unlock
pthread.h	pthread_rwlock_wrlock
pthread.h	pthread_rwlockattr_destroy
pthread.h	pthread_rwlockattr_getpshared
pthread.h	pthread_rwlockattr_init
pthread.h	pthread_rwlockattr_setpshared
pthread.h	pthread_setattr_default_np
pthread.h	pthread_setconcurrency

头文件	接口名
pthread.h	pthread_setname_np
pthread.h	pthread_spin_destroy
pthread.h	pthread_spin_init
pthread.h	pthread_spin_lock
pthread.h	pthread_spin_trylock
pthread.h	pthread_spin_unlock
pthread.h	pthread_timedjoin_np
pthread.h	pthread_tryjoin_np
pwd.h	endpwent
pwd.h	getpwent
pwd.h	getpwnam
pwd.h	getpwnam_r
pwd.h	getpwuid
pwd.h	getpwuid_r
pwd.h	setpwent
regex.h	regcomp
regex.h	regerror
regex.h	regexexec
regex.h	regfree
setjmp.h	siglongjmp
setjmp.h	sigsetjmp
sched.h	sched_getparam
sched.h	sched_getscheduler
sched.h	sched_rr_get_interval
sched.h	sched_setparam
sched.h	sched_setscheduler
search.h	hcreate
search.h	hdestroy
search.h	hsearch
search.h	insque

头文件	接口名
search.h	lfind
search.h	lsearch
search.h	remque
search.h	tdelete
search.h	tfind
search.h	tsearch
search.h	twalk
semaphore.h	sem_close
semaphore.h	sem_open
semaphore.h	sem_unlink
signal.h	kill
signal.h	killpg
signal.h	psiginfo
signal.h	psignal
signal.h	pthread_kill
signal.h	pthread_sigmask
signal.h	sigaction
signal.h	sigaddset
signal.h	sigaltstack
signal.h	sigdelset
signal.h	sigemptyset
signal.h	sigfillset
signal.h	sighold
signal.h	sigignore
signal.h	siginterrupt
signal.h	sigismember
signal.h	sigpause
signal.h	sigpending
signal.h	sigprocmask
signal.h	sigqueue

头文件	接口名
signal.h	sigrelse
signal.h	sigsuspend
signal.h	sigtimedwait
signal.h	sigwait
signal.h	sigwaitinfo
spawn.h	posix_spawn
spawn.h	posix_spawn_file_actions_addclose
spawn.h	posix_spawn_file_actions_adddup2
spawn.h	posix_spawn_file_actions_addopen
spawn.h	posix_spawn_file_actions_destroy
spawn.h	posix_spawn_file_actions_init
spawn.h	posix_spawnattr_destroy
spawn.h	posix_spawnattr_getflags
spawn.h	posix_spawnattr_getpgroup
spawn.h	posix_spawnattr_getschedparam
spawn.h	posix_spawnattr_getschedpolicy
spawn.h	posix_spawnattr_getsigdefault
spawn.h	posix_spawnattr_getsigmask
spawn.h	posix_spawnattr_init
spawn.h	posix_spawnattr_setflags
spawn.h	posix_spawnattr_setpgroup
spawn.h	posix_spawnattr_setschedparam
spawn.h	posix_spawnattr_setschedpolicy
spawn.h	posix_spawnattr_setsigdefault
spawn.h	posix_spawnattr_setsigmask
spawn.h	posix_spawnnp
stdio.h	ctermid
stdio.h	flockfile
stdio.h	fmemopen
stdio.h	ftrylockfile

头文件	接口名
stdio.h	funlockfile
stdio.h	getc_unlocked
stdio.h	getchar_unlocked
stdio.h	getdelim
stdio.h	getline
stdio.h	open_memstream
stdio.h	pclose
stdio.h	popen
stdio.h	putc_unlocked
stdio.h	putchar_unlocked
stdio.h	renameat
stdio.h	tempnam
stdlib.h	_Exit
stdlib.h	a64l
stdlib.h	drand48
stdlib.h	erand48
stdlib.h	exit
stdlib.h	getsubopt
stdlib.h	grantpt
stdlib.h	jrand48
stdlib.h	l64a
stdlib.h	lcong48
stdlib.h	mkdtemp
stdlib.h	mrnd48
stdlib.h	posix_openpt
stdlib.h	ptsname
stdlib.h	putenv
stdlib.h	rand_r
stdlib.h	setenv
stdlib.h	setkey

头文件	接口名
stdlib.h	unlockpt
stdlib.h	unsetenv
string.h	memccpy
string.h	strndup
string.h	strsignal
stropts.h	isastream
sys/ipc.h	ftok
sys/mman.h	mlock
sys/mman.h	mlockall
sys/mman.h	mmap
sys/mman.h	mprotect
sys/mman.h	msync
sys/mman.h	munlock
sys/mman.h	munlockall
sys/mman.h	munmap
sys/mman.h	posix_madvise
sys/mman.h	shm_open
sys/mman.h	shm_unlink
sys/msg.h	msgctl
sys/msg.h	msgget
sys/msg.h	msgrcv
sys/msg.h	msgsnd
sys/resource.h	getpriority
sys/resource.h	getrlimit
sys/resource.h	getrusage
sys/resource.h	setpriority
sys/resource.h	setrlimit
sys/select.h	pselect
sys/sem.h	semctl
sys/sem.h	semget

头文件	接口名
sys/sem.h	semop
sys/shm.h	shmat
sys/shm.h	shmctl
sys/shm.h	shmdt
sys/shm.h	shmget
sys/socket.h	socketmark
sys/socket.h	socketpair
sys/stat.h	chmod
sys/stat.h	fchmod
sys/stat.h	fchmodat
sys/stat.h	fstatat
sys/stat.h	futimens
sys/stat.h	mkdirat
sys/stat.h	mkfifo
sys/stat.h	mkfifoat
sys/stat.h	mknod
sys/stat.h	mknodat
sys/stat.h	umask
sys/stat.h	utimensat
sys/time.h	getitimer
sys/time.h	setitimer
sys/time.h	utimes
sys/wait.h	wait
sys/wait.h	waitid
sys/wait.h	waitpid
syslog.h	closelog
syslog.h	openlog
syslog.h	setlogmask
termios.h	cfgetispeed
termios.h	cfgetospeed

头文件	接口名
termios.h	cfsetispeed
termios.h	cfsetospeed
termios.h	tcdrain
termios.h	tcflow
termios.h	tcflush
termios.h	tcgetattr
termios.h	tcgetsid
termios.h	tcsendbreak
termios.h	tcsetattr
time.h	clock_getcpuclockid
time.h	clock_nanosleep
time.h	getdate
time.h	tzset
ulimit.h	ulimit
unistd	_exit
unistd.h	alarm
unistd.h	chown
unistd.h	confstr
unistd.h	crypt
unistd.h	ctermid
unistd.h	encrypt
unistd.h	execl
unistd.h	execle
unistd.h	execlp
unistd.h	execv
unistd.h	execve
unistd.h	execvp
unistd.h	faccessat
unistd.h	fchdir
unistd.h	fchown

头文件	接口名
unistd.h	fchownat
unistd.h	fdatasync
unistd.h	fexecve
unistd.h	fork
unistd.h	fpathconf
unistd.h	getegid
unistd.h	geteuid
unistd.h	getgid
unistd.h	getgroups
unistd.h	gethostid
unistd.h	gethostname
unistd.h	getlogin
unistd.h	getlogin_r
unistd.h	getpgid
unistd.h	getpgrp
unistd.h	getppid
unistd.h	getsid
unistd.h	getuid
unistd.h	lchown
unistd.h	link
unistd.h	linkat
unistd.h	nice
unistd.h	pathconf
unistd.h	pause
unistd.h	pipe
unistd.h	readlink
unistd.h	readlinkat
unistd.h	setegid
unistd.h	seteuid
unistd.h	setgid

头文件	接口名
unistd.h	setpgid
unistd.h	setpgrp
unistd.h	setregid
unistd.h	setreuid
unistd.h	setsid
unistd.h	setuid
unistd.h	swab
unistd.h	symlink
unistd.h	symlinkat
unistd.h	tcgetpgrp
unistd.h	tcsetpgrp
unistd.h	truncate
unistd.h	ttyname
unistd.h	ttyname_r
unistd.h	unlinkat
utmpx.h	endutxent
utmpx.h	getutxent
utmpx.h	getutxid
utmpx.h	getutxline
utmpx.h	pututxline
utmpx.h	setutxent
wchar.h	open_wmemstream
wchar.h	wcpcpy
wchar.h	wcpncpy
wchar.h	wcscasecmp
wchar.h	wcscasecmp_l
wchar.h	wcsdup
wchar.h	wcsncasecmp
wchar.h	wcsncasecmp_l
wchar.h	wcswidth

头文件	接口名
wchar.h	wcwidth
wctype.h	towctrans_l
wctype.h	wctrans_l
wordexp.h	wordexp
wordexp.h	wordfree

6.3 Libc/Libm 接口

6.3.1 Libc 支持接口

Huawei LiteOS支持部分Libc接口，具体如下表所示。

头文件	接口名	类型	说明
arpa/inet.h	inet_aton	网络函数	转换字符串IP为数字格式，并存储在传参地址中
arpa/inet.h	htonl	网络函数	将主机数转换成无符号长整型的网络字节顺序
arpa/inet.h	htons	网络函数	将整型变量从主机字节顺序转变成网络字节顺序
arpa/inet.h	ntohl	网络函数	将一个无符号长整形数从网络字节顺序转换为主机字节顺序
arpa/inet.h	ntohs	网络函数	将一个16位数由网络字节顺序转换为主机字节顺序
assert.h	assert	诊断宏函数	如果它的条件返回错误，则终止程序执行，用于调测
ctype.h	isalnum	数据判断	判断入参是否为字母或数字
ctype.h	isalpha	数据判断	判断入参是否为字母
ctype.h	iscntrl	数据判断	判断入参是否为控制字符
ctype.h	isdigit	数据判断	判断入参是否为十进制数
ctype.h	isgraph	数据判断	判断入参是否有图形表示
ctype.h	islower	数据判断	判断入参是否为小写字母
ctype.h	isprint	数据判断	判断入参是否可以打印
ctype.h	ispunct	数据判断	判断入参是否为标点符号
ctype.h	isspace	数据判断	判断入参是否为空格

头文件	接口名	类型	说明
ctype.h	isupper	数据判断	判断入参是否为大写字母
ctype.h	isxdigit	数据判断	判断入参是否为十六进制数
ctype.h	tolower	数据转换函数	将大写字母转换成小写字母
ctype.h	toupper	数据转换函数	将小写字母转换成大写字母
err.h	err	错误提示	输出错误信息
err.h	errx	错误提示	输出错误信息
err.h	verr	错误提示	输出错误信息
err.h	verrx	错误提示	输出错误信息
err.h	vwarn	错误提示	输出警告信息
err.h	vwarnx	错误提示	输出警告信息
err.h	warn	错误提示	输出警告信息
err.h	warnx	错误提示	输出警告信息
errno.h	errno	错误信息	错误类型
errno.h	get_errno	错误信息	获取错误类型
errno.h	set_errno	错误信息	设置错误类型
fcntl.h	creat64	文件操作函数	创建一个文件（64位）
fcntl.h	fallcoate	文件操作函数	为文件预分配物理空间
fcntl.h	fallcoate64	文件操作函数	为文件预分配物理空间（64位）
fcntl.h	open64	文件操作函数	打开文件（64位）
ifaddrs.h	freeifaddrs	网络函数	释放本地IP地址
ifaddrs.h	getifaddrs	网络函数	获取本地IP地址
inttypes.h	strtoimax	数据转换函数	解析字符串中的整数值
inttypes.h	strtoumax	数据转换函数	解析字符串中的整数值
inttypes.h	wcstoimax	数据转换函数	解析字符串中的整数值

头文件	接口名	类型	说明
inttypes.h	wcstoumax	数据转换函数	解析字符串中的整数值
locale.h	setlocale	系统函数	设置或读取地域化信息。读取地域信息时返回C.UTF-8，因为LiteOS不支持国际化，所以设置地域信息无效。
malloc.h	calloc	内存操作	申请内存，返回内存指针
malloc.h	free	内存操作	释放原先分配的动态内存
malloc.h	malloc	内存操作	申请内存，返回内存指针
malloc.h	memalign	内存操作	对齐申请内存，返回内存指针
malloc.h	realloc	内存操作	调整已经申请的内存，返回内存指针
malloc.h	zalloc	内存操作	申请内存并清零内存区域，返回内存指针
netdb.h	gethostbyname	网络函数	获取对应于给定主机名的主机信息
netdb.h	gethostbyname_r	网络函数	获取对应于给定主机名的主机信息，可重入
poll.h	notify_poll	文件操作函数	监视文件描述符
poll.h	notify_poll_with_key	文件操作函数	监视文件描述符
resolv.h	dn_comp	网络函数	压缩域名
sched.h	sched_getaffinity	调度函数	获取指定任务对的CPU亲和性
sched.h	sched_setaffinity	调度函数	设置指定任务对的CPU亲和性
setjmp.h	longjmp	系统函数	长跳转
setjmp.h	setjmp	系统函数	承担非局部标号和goto作用
stdio.h	asprintf	格式化字符串函数	格式化字符串的复制
stdio.h	clearerr	错误处理函数	清除错误标志（加锁）
stdio.h	clearerr_unlocked	错误处理函数	清除错误标志（不加锁）
stdio.h	fclose	文件操作函数	关闭文件，文件关闭后再操作，将操作失败，并且设置错误码

头文件	接口名	类型	说明
stdio.h	feof	文件操作函数	检查文件流是否读到了文件尾
stdio.h	feof_unlocked	文件操作函数	检查文件流是否读到了文件尾（不加锁）
stdio.h	ferror	文件操作函数	检查文件流是否有错误
stdio.h	ferror_unlocked	文件操作函数	检查文件流是否有错误（不加锁）
stdio.h	fflush	文件操作函数	更新缓冲区
stdio.h	fflush_unlocked	文件操作函数	更新缓冲区（不加锁）
stdio.h	fgetc	文件操作函数	从文件中读取一个字符
stdio.h	fgetpos	文件操作函数	取得文件流的读取位置
stdio.h	fgets	文件操作函数	从文件中读取一个字符串
stdio.h	fgets_unlocked	文件操作函数	从文件中读取一个字符串（不加锁）
stdio.h	fileno_unlocked	文件操作函数	取得参数stream指定的文件流所使用的文件描述符（不加锁），如果文件流已经关闭，则返回-1，并设置错误码
stdio.h	fopen	文件操作函数	打开文件，不支持以rx模式打开一个文件，否则返回NULL
stdio.h	fopen64	文件操作函数	打开文件（64位）
stdio.h	fprintf	文件操作函数	格式化输出数据至文件
stdio.h	fputc	文件操作函数	将字符写入文件流中
stdio.h	fputs	文件操作函数	将字符串写入文件流中
stdio.h	fputs_unlocked	文件操作函数	将字符串写入文件流中（不加锁）

头文件	接口名	类型	说明
stdio.h	fread	文件操作函数	从文件流读取数据，读取文件的size未做对齐处理，需用户自行保证size对齐
stdio.h	fread_unlocked	文件操作函数	从文件流读取数据（不加锁）
stdio.h	freopen	文件操作函数	关闭已打开的文件，按照传入的模式打开新文件后将新文件流关联到旧文件流
stdio.h	freopen64	文件操作函数	关闭已打开的文件，按照传入的模式打开新文件后将新文件流关联到旧文件（64位）
stdio.h	fscanf	文件操作函数	读取格式化输入
stdio.h	fseek	文件操作函数	移动文件流的读取位置
stdio.h	fseeko64	文件操作函数	移动文件流的读写位置（64位）
stdio.h	fsetpos	文件操作函数	移动文件流的读取位置
stdio.h	fsetpos64	文件操作函数	移动文件流的读取位置（64位）
stdio.h	ftell	文件操作函数	获取文件流的读取位置
stdio.h	ftello64	文件操作函数	获取文件流的读取位置（64位）
stdio.h	fwrite	文件操作函数	将数据写入文件流，写入文件的size未做对齐处理，需用户自行保证size对齐，有入参合法性校验
stdio.h	fwrite_unlocked	文件操作函数	将数据写入文件流（不加锁）
stdio.h	getc	标准IO操作	由文件中读取一个字符
stdio.h	getchar	标准IO操作	由文件中读取一个字符
stdio.h	gets	标准IO操作	由文件中读取一个字符串
stdio.h	perror	标准IO操作	向标准错误流输出错误信息

头文件	接口名	类型	说明
stdio.h	printf	标准IO操作	格式化输出数据
stdio.h	putc	标准IO操作	将一指定字符写入文件
stdio.h	putchar	标准IO操作	将一指定字符写标准输出流
stdio.h	puts	标准IO操作	将指定的字符串写到标准输出流
stdio.h	putw	标准IO操作	将一个整数写入文件
stdio.h	remove	文件操作函数	删除
stdio.h	rename	文件操作函数	重命名
stdio.h	rewind	文件操作函数	移动文件流的读取位置到起始位置
stdio.h	scanf	格式化输入输出函数	格式化字符串输入
stdio.h	setbuf	文件操作函数	设置文件流的缓冲区
stdio.h	setbuffer	文件操作函数	设置文件流的缓冲区
stdio.h	setvbuf	文件操作函数	设置文件流的缓冲区，当入参size小于0时，返回-1，并设置错误码。当入参buf为NULL，则会动态申请默认大小空间作为文件缓冲区
stdio.h	snprintf	格式化输入输出函数	格式化字符串
stdio.h	sprintf	格式化输入输出函数	格式化字符串
stdio.h	sscanf	格式化输入输出函数	格式化输入字符串
stdio.h	tmpnam	文件操作函数	返回指向唯一文件名的指针

头文件	接口名	类型	说明
stdio.h	ungetc	文件操作函数	将一指定字符写回文件流中
stdio.h	vasprintf	格式化输入输出函数	格式化字符串
stdio.h	vfprintf	格式化输入输出函数	格式化输出数据至文件
stdio.h	vfscanf	格式化输入输出函数	从文件流读取字符串，根据参数 format 转化并格式化数据
stdio.h	vprintf	格式化输入输出函数	格式化输出
stdio.h	vscanf	格式化输入输出函数	格式化字符串输入
stdio.h	vsprintf	格式化输入输出函数	格式化字符串
stdio.h	vsscanf	格式化输入输出函数	格式化输入字符串
stdlib.h	abort	系统函数	产生异常终止系统
stdlib.h	abs	数学计算函数	求整形绝对值
stdlib.h	atof	数据转换函数	将字符串转换成浮点型数
stdlib.h	atoi	数据转换函数	将字符串转换成整型数
stdlib.h	atol	数据转换函数	将字符串转换成长整型数
stdlib.h	atoll	数据转换函数	将字符串转换成长长整型数
stdlib.h	bsearch	数据结构函数	二元搜索

头文件	接口名	类型	说明
stdlib.h	labs	数学计算函数	求长整形绝对值
stdlib.h	llabs	数学计算函数	求长整形绝对值
stdlib.h	mblen	多字节字符函数	多字节字符串长度
stdlib.h	mbstowcs	多字节字符函数	转换多字节字符串到宽字符串
stdlib.h	mbtowc	多字节字符函数	转换多字节字符到宽字符
stdlib.h	mkostemps	文件处理函数	产生唯一的临时文件名
stdlib.h	mkstemps	文件处理函数	建立唯一的临时文件
stdlib.h	qsort	排序函数	利用快速排序法排列数组
stdlib.h	qsort_r	排序函数	利用快速排序法排列数组
stdlib.h	rand	随机数函数	产生随机数
stdlib.h	srand	随机数函数	设置随机数种子
stdlib.h	strtod	数据转换函数	将字符串转换成浮点型数
stdlib.h	strtof	字符串处理函数	字符串转单精度浮点型数
stdlib.h	strtol	数据转换函数	将字符串转换成长整型数
stdlib.h	strtold	字符串处理函数	字符串转长双精度浮点型数
stdlib.h	strtoll	字符串处理函数	字符串转双长整型数
stdlib.h	strtoul	数据转换函数	将字符串转换成无符号长整型数
stdlib.h	strtoull	字符串处理函数	字符串转无符号双长整数
stdlib.h	wctomb	多字节字符函数	转换宽字符到多字节字符

头文件	接口名	类型	说明
string.h	memchr	字符串处理函数	在某一内存范围内查找一特定字符
string.h	memcmp	字符串处理函数	比较内存内容
string.h	memcpy	字符串处理函数	拷贝内存内容
string.h	memmove	字符串处理函数	拷贝内存内容
string.h	memrchr	字符串处理函数	查找字符首次出现位置
string.h	memset	字符串处理函数	将一段内存空间填入某值
string.h	strcasestr	字符串处理函数	在一字符串中查找指定字符串，忽略大小写
string.h	strcat	字符串处理函数	连接两个字符串
string.h	strchr	字符串处理函数	查找字符首次出现位置
string.h	strchrnul	字符串处理函数	查找字符首次出现位置
string.h	strcmp	字符串处理函数	比较字符串
string.h	strcoll	字符串处理函数	根据 LC_COLLATE 比较字符串
string.h	strcpy	字符串处理函数	拷贝字符串
string.h	strcspn	字符串处理函数	入参1字符串中连续有几个字符都不属于入参2字符串
string.h	strerror	字符串处理函数	返回一个指向错误消息字符串的指针
string.h	strncpy	字符串处理函数	拷贝字符串
string.h	strlen	字符串处理函数	返回字符串长度
string.h	strncat	字符串处理函数	连接两个字符串
string.h	strncmp	字符串处理函数	比较字符串

头文件	接口名	类型	说明
string.h	strncpy	字符串处理函数	拷贝字符串
string.h	strpbrk	字符串处理函数	查找字符串中第一个出现的指定字符
string.h	strrchr	字符串处理函数	查找字符串中最后出现的指定字符
string.h	strsep	字符串处理函数	分解字符串为一组字符串
string.h	strspn	字符串处理函数	返回字符串中连续不含指定字符串内容的字符数
string.h	strstr	字符串处理函数	在一字符串中查找指定字符串
string.h	strtok	字符串处理函数	分割字符串
string.h	strxfrm	字符串处理函数	根据LC_COLLATE来转换字符串src，若传入n的长度小于字符串src的长度，则不作转换，直接返回字符串src的长度
strings.h	bcmp	内存操作	同memcmp
strings.h	bcopy	内存操作	同memcpy
strings.h	bzero	内存操作	同memset
strings.h	rindex	字符处理函数	字符处理
sys/mount.h	mount	文件系统操作	挂载文件系统
sys/mount.h	umount	文件系统操作	卸载文件系统
sys/prctl.h	prctl	pthread	线程操作，目前只支持PR_SET_NAME操作修改线程名
sys/stat.h	fstat64	文件操作接口	获取文件属性（64位）
sys/statfs.h	statfs	文件操作接口	获取文件系统属性
sys/time.h	adjtime	时间函数	按时间值delta来微调系统时间
sys/time.h	settimeofday	时间函数	设置系统时间
threads.h	tss_get	pthread	获取调用线程的键绑定，同pthread_getspecific

头文件	接口名	类型	说明
time.h	asctime	时间函数	将时间和日期以字符串形式表示
time.h	clock	时间函数	获取处理器时钟所使用的时间
time.h	ctime	时间函数	将时间和日期以字符串形式表示
time.h	difftime	时间函数	返回两个时间之间的差(秒)
time.h	gmtime	时间函数	取得目前的时间和日期
time.h	localtime	时间函数	取得当地目前的时间和日期
time.h	mktime	时间函数	将时间结构数据转换成经过的秒数
time.h	stime	时间函数	设置时间
time.h	strftime	时间函数	格式化日期和时间，格式化字符不支持%z或%Z的格式化转换
time.h	time	时间函数	获取当前的系统时间
time64.h	asctime64	时间函数	将时间和日期以字符串形式表示
time64.h	asctime64_r	时间函数	将时间和日期以字符串形式表示
time64.h	ctime64	时间函数	将时间和日期以字符串形式表示
time64.h	ctime64_r	时间函数	将时间和日期以字符串形式表示
time64.h	gmtime64	时间函数	取得目前的时间和日期
time64.h	gmtime64_r	时间函数	取得目前的时间和日期
time64.h	gettimeofday64	时间函数	取得当前系统时间，Huawei LiteOS重新实现了该接口，用于在32位平台下获取2038年以后的系统时间
time64.h	localtime64	时间函数	取得当地目前的时间和日期
time64.h	localtime64_r	时间函数	取得当地目前的时间和日期
time64.h	mktime64	时间函数	将时间结构数据转换成经过的秒数
time64.h	settimeofday64	时间函数	设置当前时间，Huawei LiteOS重新实现了该接口，用于在32位平台下设置当前系统时间为2038年以后的时间
tzdst.h	settimezone	时间函数	设置时区
tzdst.h	dst_disable	时间函数	失能夏令时
tzdst.h	dst_enable	时间函数	使能夏令时
tzdst.h	dst_inquire	时间函数	查询夏令时
uchar.h	c32rtomb	宽字符处理函数	将32位宽字符转换为多字节字符

头文件	接口名	类型	说明
uchar.h	mbrtoc32	宽字符处理函数	将多字节字符转换为32位宽字符
unistd.h	ftruncate64	文件操作函数	改变文件大小
unistd.h	getentropy	随机数函数	获取指定长度的随机数，若硬件不能提供随机数，则返回失败
unistd.h	getpagesize	系统函数	获取页大小
unistd.h	lseek64	文件操作函数	改变文件读写指针
unistd.h	pread64	文件操作函数	读文件
unistd.h	pwrite64	文件操作函数	写文件
unistd.h	usleep	时间函数	以微秒为单位阻塞进程
wchar.h	btowc	宽字符处理函数	把单个字节转换为宽字符
wchar.h	fgetwc	宽字符处理函数	把单个字节转换为宽字符
wchar.h	fgetwc_unlocked	宽字符处理函数	把单个字节转换为宽字符(不加锁)
wchar.h	fputwc	宽字符处理函数	写入一个宽字符到文件流
wchar.h	fputwc_unlocked	宽字符处理函数	写入一个宽字符到文件流（不加锁）
wchar.h	fwide	文件操作函数	设置查询文件流的方向
wchar.h	getwc	宽字符处理函数	从文件中读取一个宽字符
wchar.h	getwc_unlocked	宽字符处理函数	从文件中读取一个宽字符（不加锁）
wchar.h	mbrlen	宽字符处理函数	返回多字节字符的长度
wchar.h	mbrtowc	宽字符处理函数	多字节序列转换为宽字符
wchar.h	mbsinit	宽字符处理函数	测试初始的转换状态

头文件	接口名	类型	说明
wchar.h	mbsrtowcs	宽字符处理函数	多字节序列转换为宽字符
wchar.h	putwc	宽字符处理函数	将一指定宽字符写入文件中
wchar.h	putwc_unlocked	宽字符处理函数	将一指定宽字符写入文件中（不加锁）
wchar.h	swprintf	格式化输入输出函数	格式化宽字符串
wchar.h	ungetwc	宽字符处理函数	将宽字符写回指定的文件流
wchar.h	vfwprintf	格式化输入输出函数	格式化输出宽字符数据至文件
wchar.h	vfwscanf	格式化输入输出函数	从输入流中格式化宽字符串
wchar.h	vswprintf	格式化输入输出函数	格式化宽字符串
wchar.h	wcrtomb	宽字符处理函数	将宽字符代码转换为字符
wchar.h	wcschr	宽字符处理函数	查找宽字符串中第一个出现的指定宽字符
wchar.h	wcscmp	宽字符处理函数	比较宽字符串
wchar.h	wcscoll	宽字符处理函数	采用目前区域的字符排列次序来比较宽字符串
wchar.h	wcsftime	宽字符处理函数	格式化时间
wchar.h	wcsftime_l	宽字符处理函数	格式化时间
wchar.h	wcslen	宽字符处理函数	返回宽字符串长度
wchar.h	wcsncmp	宽字符处理函数	比较两个宽字符串指定个数的字符
wchar.h	wcsncpy	宽字符处理函数	复制宽字符串

头文件	接口名	类型	说明
wchar.h	wcsrtombs	转换函数	把数组中存储的编码转换为多字节字符
wchar.h	wcsstr	宽字符处理函数	在一个宽字符串中搜索另一个宽字符串
wchar.h	wcstol	宽字符处理函数	将宽字符串转换为长整数
wchar.h	wcstoll	宽字符处理函数	将宽字符串转换为长长整数
wchar.h	wcstoul	宽字符处理函数	将宽字符串转换为无符号长整数
wchar.h	wcstoull	宽字符处理函数	将宽字符串转换为无符号长长整数
wchar.h	wcsxfrm	宽字符处理函数	根据程序当前的区域选项中的 LC_COLLATE 来转换宽字符串的前 n 个字符
wchar.h	wctob	宽字符处理函数	把宽字符转换为单字节字符
wchar.h	wmemchr	宽字符处理函数	在一个宽字符数组中搜索
wchar.h	wmemcmp	宽字符处理函数	比较两个宽字符数组
wchar.h	wmemcpy	宽字符处理函数	拷贝宽字符数组
wchar.h	wmemmove	宽字符处理函数	拷贝宽字符数组
wchar.h	wmemset	宽字符处理函数	填充一个宽字符数组
wctype.h	iswalnum	宽字符处理函数	判断宽字符是否字母或数字
wctype.h	iswalpha	宽字符处理函数	判断宽字符是否为英文字母
wctype.h	iswcntrl	宽字符处理函数	判断宽字符是否为控制字符
wctype.h	iswctype	宽字符处理函数	判断是否为宽字符类型
wctype.h	iswdigit	宽字符处理函数	判断宽字符是否为阿拉伯字符0到9

头文件	接口名	类型	说明
wctype.h	iswgraph	宽字符处理函数	判断宽字符是否为除空格外的可打印字符
wctype.h	iswlower	宽字符处理函数	判断宽字符是否为小写英文字母
wctype.h	iswprint	宽字符处理函数	判断宽字符是否为可打印字符
wctype.h	iswpunct	宽字符处理函数	判断宽字符是否为标点符号或特殊符号
wctype.h	iswspace	宽字符处理函数	判断宽字符是否为空格字符
wctype.h	iswupper	宽字符处理函数	判断宽字符是否为大写英文字母
wctype.h	iswxdigit	宽字符处理函数	判断宽字符是否为16进制数字
wctype.h	towlower	宽字符处理函数	将宽字符转换成小写字母
wctype.h	towupper	宽字符处理函数	将宽字符转换成大写字母
wctype.h	wctype	宽字符处理函数	判断是否为宽字符类型

6.3.2 Libm 支持接口

Huawei LiteOS提供一套Libm开源接口，具体的规格参见下表。

须知

Libm不支持设置错误返回码。

头文件	接口名	类型	说明
complex.h	cabs	复数计算函数	计算复数的绝对值
math.h	acos	数学计算函数	取反余弦函数值
math.h	acosf	数学计算函数	求反余弦函数

头文件	接口名	类型	说明
math.h	acosh	数学计算函数	求反双曲余弦值
math.h	acoshf	数学计算函数	求反双曲余弦值
math.h	acoshl	数学计算函数	求反双曲余弦值
math.h	acosl	数学计算函数	求反余弦函数
math.h	asin	数学计算函数	取反正弦函数值
math.h	asinf	数学计算函数	求反正弦函数
math.h	asinh	数学计算函数	求反双曲正弦值
math.h	asinhf	数学计算函数	求反双曲正弦值
math.h	asinhf	数学计算函数	求反双曲正弦值
math.h	asinl	数学计算函数	求反正弦函数
math.h	atan	数学计算函数	取反正切函数值
math.h	atan2	数学计算函数	取得反正切函数值
math.h	atan2f	数学计算函数	求反正切的值(以弧度表示)
math.h	atan2l	数学计算函数	求反正切的值(以弧度表示)
math.h	atanf	数学计算函数	求反正切函数
math.h	atanh	数学计算函数	求反双曲线正切函数
math.h	atanhf	数学计算函数	求反双曲线正切函数
math.h	atanhl	数学计算函数	求反双曲线正切函数
math.h	atanl	数学计算函数	求反正切函数

头文件	接口名	类型	说明
math.h	cbrt	数学计算函数	求立方根函数
math.h	cbrtf	数学计算函数	求立方根函数
math.h	cbrtl	数学计算函数	求立方根函数
math.h	ceil	数学计算函数	计算最小整数值
math.h	ceilf	数学计算函数	计算最小整数值
math.h	ceilfll	数学计算函数	计算最小整数值
math.h	copysign	数学计算函数	返回x，但是符号位变为y的符号位
math.h	copysignf	数学计算函数	返回x，但是符号位变为y的符号位
math.h	copysignll	数学计算函数	返回x，但是符号位变为y的符号位
math.h	cos	数学计算函数	求余弦函数
math.h	cosf	数学计算函数	求余弦函数
math.h	cosh	数学计算函数	求双曲线余弦函数
math.h	coshf	数学计算函数	求双曲线余弦函数
math.h	coshll	数学计算函数	求双曲线余弦函数
math.h	cosl	数学计算函数	求余弦函数
math.h	drem	数学计算函数	求余数(double)
math.h	dremf	数学计算函数	求余数(float)
math.h	erf	数学计算函数	求误差函数
math.h	erfc	数学计算函数	求误差补函数

头文件	接口名	类型	说明
math.h	erfcf	数学计算函数	求误差补函数
math.h	erfcl	数学计算函数	求误差补函数
math.h	erff	数学计算函数	求误差函数
math.h	erfl	数学计算函数	求误差函数
math.h	exp	数学计算函数	计算指数
math.h	exp10	数学计算函数	计算以10为底的x次方值
math.h	exp10l	数学计算函数	计算以10为底的x次方值
math.h	exp2	数学计算函数	计算以2为底的x次方值
math.h	exp2f	数学计算函数	计算以2为底的x次方值
math.h	exp2l	数学计算函数	计算以2为底的x次方值
math.h	expf	数学计算函数	计算以e为底的x次方值
math.h	expl	数学计算函数	计算以e为底的x次方值
math.h	expm1	数学计算函数	$\exp(x) - 1.0$
math.h	expm1f	数学计算函数	$\exp(x) - 1$
math.h	expm1l	数学计算函数	$\exp(x) - 1$
math.h	fabs	数学计算函数	计算浮点型数的绝对值
math.h	fabsf	数学计算函数	求浮点数的绝对值
math.h	fabsl	数学计算函数	求浮点数的绝对值
math.h	fdim	数学计算函数	计算入参的差值的绝对值

头文件	接口名	类型	说明
math.h	fdimf	数学计算函数	计算入参的差值的绝对值
math.h	fdiml	数学计算函数	计算入参的差值的绝对值
math.h	finite	数学计算函数	判断数据是否有效
math.h	finitef	数学计算函数	判断数据是否有效
math.h	floor	数学计算函数	向下取整
math.h	floorf	数学计算函数	向下取整
math.h	floorl	数学计算函数	向下取整
math.h	fma	数学计算函数	浮点运算 $(x \cdot y) + z$
math.h	fmaf	数学计算函数	浮点运算 $(x \cdot y) + z$
math.h	fmal	数学计算函数	浮点运算 $(x \cdot y) + z$
math.h	fmax	数学计算函数	返回两个浮点入参的较大值
math.h	fmaxf	数学计算函数	返回两个浮点入参的较大值
math.h	fmaxl	数学计算函数	返回两个浮点入参的较大值
math.h	fmin	数学计算函数	返回两个浮点入参的较小值
math.h	fminf	数学计算函数	返回两个浮点入参的较小值
math.h	fminl	数学计算函数	返回两个浮点入参的较小值
math.h	fmod	数学计算函数	计算余数
math.h	fmodf	数学计算函数	计算余数
math.h	fmodl	数学计算函数	计算余数

头文件	接口名	类型	说明
math.h	frexp	数学计算函数	将浮点型数分为尾数与指数
math.h	frexpf	数学计算函数	把浮点数分解成尾数和指数
math.h	frexpl	数学计算函数	把浮点数分解成尾数和指数
math.h	hypot	数学计算函数	返回直角三角形斜边长度
math.h	hypotf	数学计算函数	返回直角三角形斜边长度
math.h	hypotl	数学计算函数	返回直角三角形斜边长度
math.h	ilogb	数学计算函数	与logb相同，但是返回有符号数的整数
math.h	ilogbf	数学计算函数	与logb相同，但是返回有符号数的整数
math.h	ilogbl	数学计算函数	与logb相同，但是返回有符号数的整数
math.h	isnan	数据判断	判断参数是否为不可表示的值
math.h	j0	数学计算函数	返回x的一类0阶的Bessel函数
math.h	j0f	数学计算函数	返回x的一类0阶的Bessel函数
math.h	j1	数学计算函数	返回x的一类1阶的Bessel函数
math.h	j1f	数学计算函数	返回x的一类1阶的Bessel函数
math.h	jn	数学计算函数	返回x的一类n阶的Bessel函数
math.h	jnf	数学计算函数	返回x的一类n阶的Bessel函数
math.h	ldexp	数学计算函数	返回x乘以2的exp次幂
math.h	ldexpf	数学计算函数	返回x乘以2的exp次幂
math.h	ldexpl	数学计算函数	返回x乘以2的exp次幂

头文件	接口名	类型	说明
math.h	lgamma	数学计算函数	返回gamma函数的绝对值的自然对数，gamma函数为： $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$
math.h	lgamma_r	数学计算函数	lgamma_r与lgamma类似，但它将中间结果的符号存储在第二个入参指向的变量中，而不是存储在全局变量中。这个接口是可重入的。
math.h	lgammaf	数学计算函数	返回gamma函数的绝对值的自然对数，gamma函数为： $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$
math.h	lgammaf_r	数学计算函数	lgamma_r与lgamma类似，但它将中间结果的符号存储在第二个入参指向的变量中，而不是存储在全局变量中。这个接口是可重入的。
math.h	lgammal	数学计算函数	返回gamma函数的绝对值的自然对数，gamma函数为： $\gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$
math.h	lgammal_r	数学计算函数	lgamma_r与lgamma类似，但它将中间结果的符号存储在第二个入参指向的变量中，而不是存储在全局变量中。这个接口是可重入的。
math.h	llrint	数学计算函数	根据当前的舍入方式，将x舍入为整数值
math.h	llrintf	数学计算函数	根据当前的舍入方式，将x舍入为整数值
math.h	llrintl	数学计算函数	根据当前的舍入方式，将x舍入为整数值
math.h	llround	数学计算函数	四舍五入为整数
math.h	llroundf	数学计算函数	四舍五入为整数
math.h	llroundl	数学计算函数	四舍五入为整数
math.h	log	数学计算函数	计算以e为底的对数值
math.h	log10	数学计算函数	计算以10为底的对数值
math.h	log10f	数学计算函数	计算以10为底的x对数值
math.h	log10l	数学计算函数	计算以10为底的x对数值

头文件	接口名	类型	说明
math.h	log1p	数学计算函数	$\log(1+x)$
math.h	log1pf	数学计算函数	$\log(1+x)$
math.h	log1pl	数学计算函数	$\log(1+x)$
math.h	log2	数学计算函数	计算以2为底的x对数值
math.h	log2f	数学计算函数	计算以2为底的x对数值
math.h	log2l	数学计算函数	计算以2为底的x对数值
math.h	logb	数学计算函数	使用FLT_RADIX作为对数的底数，返回x绝对值的对数
math.h	logbf	数学计算函数	使用FLT_RADIX作为对数的底数，返回x绝对值的对数
math.h	logbl	数学计算函数	使用FLT_RADIX作为对数的底数，返回x绝对值的对数
math.h	logf	数学计算函数	计算以e为底的x对数值
math.h	logl	数学计算函数	计算以e为底的x对数值
math.h	lrint	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	lrintf	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	lrintl	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	lround	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	lroundf	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	lroundl	数学计算函数	根据当前的舍入方式，将x舍入为整数值，返回值为long int型
math.h	modf	数学计算函数	将浮点型数分解成整数与小数，返回小数部分
math.h	modff	数学计算函数	将一个浮点值分为整数和小数部分

头文件	接口名	类型	说明
math.h	modfl	数学计算函数	将一个浮点值分为整数和小数部分
math.h	nan	数据判断	返回非数
math.h	nanf	数据判断	返回非数
math.h	nanl	数据判断	返回非数
math.h	nearbyint	数学计算函数	根据当前舍入模式，将x舍入为整数值
math.h	nearbyintf	数学计算函数	根据当前舍入模式，将x舍入为整数值
math.h	nearbyintl	数学计算函数	根据当前舍入模式，将x舍入为整数值
math.h	nextafter	数学计算函数	取两个双精度浮点数之间与第一个数相邻的浮点数，两数相等，则返回第二个数
math.h	nextafterf	数学计算函数	取两个浮点数之间与第一个数相邻的浮点数，两数相等，则返回第二个数
math.h	nextafterl	数学计算函数	返回x在y方向上的下一个可表示的数
math.h	nexttoward	数学计算函数	返回x在y方向上的上一个可表示的数
math.h	nexttowardf	数学计算函数	返回x在y方向上的上一个可表示的数
math.h	nexttowardl	数学计算函数	返回x在y方向上的上一个可表示的数
math.h	pow	数学计算函数	计算次方值
math.h	pow10	数学计算函数	计算10的x次方
math.h	pow10f	数学计算函数	计算10的x次方
math.h	pow10l	数学计算函数	计算10的x次方
math.h	powf	数学计算函数	求 x 的 y 次幂（次方）
math.h	powl	数学计算函数	求 x 的 y 次幂（次方）

头文件	接口名	类型	说明
math.h	remainder	数学计算函数	计算余数，舍入到最接近的整数
math.h	remainderf	数学计算函数	计算余数，舍入到最接近的整数
math.h	remainderl	数学计算函数	计算余数，舍入到最接近的整数
math.h	remquo	数学计算函数	计算余数
math.h	remquof	数学计算函数	计算余数
math.h	remquol	数学计算函数	计算余数
math.h	rint	数学计算函数	将浮点数舍入到最接近的整数
math.h	rintf	数学计算函数	将浮点数舍入到最接近的整数
math.h	rintl	数学计算函数	将浮点数舍入到最接近的整数
math.h	round	数学计算函数	返回x的四舍五入整数值
math.h	roundf	数学计算函数	返回x的四舍五入整数值
math.h	roundl	数学计算函数	返回x的四舍五入整数值
math.h	scalb	数学计算函数	返回 $x * 2^{\text{exp}}$
math.h	scalbf	数学计算函数	返回 $x * 2^{\text{exp}}$
math.h	scalbln	数学计算函数	返回 $x * 2^{\text{exp}}$
math.h	scalblnf	数学计算函数	返回 $x * 2^{\text{exp}}$
math.h	scalblnl	数学计算函数	返回 $x * 2^{\text{exp}}$
math.h	scalbn	数学计算函数	返回x乘以FLT_RADIX的n次幂
math.h	scalbnf	数学计算函数	返回x乘以FLT_RADIX的n次幂

头文件	接口名	类型	说明
math.h	scalbnl	数学计算函数	返回x乘以FLT_RADIX的n次幂
math.h	significand	数学计算函数	返回范围[1,2)的尾数, <code>scalb (x, (double) -ilogb (x))</code> .
math.h	significandf	数学计算函数	返回范围[1,2)的尾数, <code>scalb (x, (double) -ilogb (x))</code> .
math.h	sin	数学计算函数	取正弦函数值
math.h	sincos	数学计算函数	取正弦余弦值
math.h	sincosf	数学计算函数	取正弦余弦值
math.h	sincosl	数学计算函数	取正弦余弦值
math.h	sinf	数学计算函数	求正弦函数
math.h	sinh	数学计算函数	取双曲线正弦函数值
math.h	sinhf	数学计算函数	求双曲线正弦函数
math.h	sinhl	数学计算函数	求双曲线正弦函数
math.h	sinl	数学计算函数	求正弦函数
math.h	sqrt	数学计算函数	计算平方根值
math.h	sqrtf	数学计算函数	计算平方根值
math.h	sqrtl	数学计算函数	计算平方根值
math.h	tan	数学计算函数	求正切函数
math.h	tanf	数学计算函数	求正切函数
math.h	tanh	数学计算函数	取双曲线正切函数值
math.h	tanhf	数学计算函数	求双曲线正切函数

头文件	接口名	类型	说明
math.h	tanh	数学计算函数	求双曲线正切函数
math.h	tanl	数学计算函数	求正切函数
math.h	tgamma	数学计算函数	参数为arg的gamma函数返回
math.h	tgammaf	数学计算函数	参数为arg的gamma函数返回
math.h	tgammal	数学计算函数	参数为arg的gamma函数返回
math.h	trunc	时间函数	截取日期或数字，返回指定的值
math.h	truncf	时间函数	截取日期或数字，返回指定的值
math.h	truncl	时间函数	截取日期或数字，返回指定的值
math.h	y0	数学计算函数	返回x的第二类0阶Bessel函数
math.h	y0f	数学计算函数	返回x的第二类0阶Bessel函数
math.h	y1	数学计算函数	返回x的第二类1阶Bessel函数
math.h	y1f	数学计算函数	返回x的第二类1阶Bessel函数
math.h	yn	数学计算函数	返回x的第二类n阶Bessel函数
math.h	ynf	数学计算函数	返回x的第二类n阶Bessel函数

6.3.3 Libc/Libm 不支持接口

Huawei LiteOS的Libc/Libm接口中，有一些未支持，具体参见下表。

头文件	接口名
alloca.h	alloca
arpa/inet.h	inet_lnaof
arpa/inet.h	inet_makeaddr

头文件	接口名
arpa/inet.h	inet_netof
arpa/inet.h	inet_network
arpa/nameser.h	ns_get16
arpa/nameser.h	ns_get32
arpa/nameser.h	ns_initparse
arpa/nameser.h	ns_name_uncompress
arpa/nameser.h	ns_parserr
arpa/nameser.h	ns_put16
arpa/nameser.h	ns_put32
arpa/nameser.h	ns_skiprr
byteswap.h	bswap_16
byteswap.h	bswap_32
byteswap.h	bswap_64
complex.h	cabsf
complex.h	cabsl
complex.h	cacos
complex.h	cacosf
complex.h	cacosh
complex.h	cacoshf
complex.h	cacoshl
complex.h	cacosl
complex.h	carg
complex.h	cargf
complex.h	cargl
complex.h	casin
complex.h	casinf
complex.h	casinh
complex.h	casinhf
complex.h	casinhl
complex.h	casinl

头文件	接口名
complex.h	catan
complex.h	catanf
complex.h	catanh
complex.h	catanhf
complex.h	catanhl
complex.h	catanl
complex.h	ccos
complex.h	ccosf
complex.h	ccosh
complex.h	ccoshf
complex.h	ccoshl
complex.h	ccosl
complex.h	cexp
complex.h	cexpf
complex.h	cexpl
complex.h	cimag
complex.h	cimagf
complex.h	cimagl
complex.h	clog
complex.h	clogf
complex.h	clogl
complex.h	conj
complex.h	conjf
complex.h	conjl
complex.h	cpow
complex.h	cpowf
complex.h	cpowl
complex.h	cproj
complex.h	cprojf
complex.h	cprojl

头文件	接口名
complex.h	creal
complex.h	crealf
complex.h	creall
complex.h	csin
complex.h	csinf
complex.h	csinh
complex.h	csinhf
complex.h	csinhl
complex.h	csinl
complex.h	csqrt
complex.h	csqrtf
complex.h	csqrtl
complex.h	ctan
complex.h	ctanf
complex.h	ctanh
complex.h	ctanhf
complex.h	ctanhl
complex.h	ctanl
crypt.h	crypt
crypt.h	crypt_r
dirent.h	getdents
dirent.h	versionsort
dlfcn.h	dladdr
dlfcn.h	dlopen
fcntl.h	lockf
fcntl.h	lockf64
fcntl.h	name_to_handle_at
fcntl.h	open_by_handle_at
fcntl.h	openat64
fcntl.h	posix_fadvise64

头文件	接口名
fcntl.h	posix_fallocate64
fcntl.h	readahead
fcntl.h	splice
fcntl.h	sync_file_range
fcntl.h	tee
fcntl.h	vmsplice
fenv.h	feclearexcept
fenv.h	fegetenv
fenv.h	fegetexceptflag
fenv.h	fegetround
fenv.h	feholdexcept
fenv.h	feraiseexcept
fenv.h	fesetenv
fenv.h	fesetexceptflag
fenv.h	fesetround
fenv.h	fetestexcept
fenv.h	feupdateenv
getopt.h	getopt_long
getopt.h	getopt_long_only
libintl.h	bind_textdomain_codeset
libintl.h	bindtextdomain
libintl.h	dcgettext
libintl.h	dcngettext
libintl.h	dgettext
libintl.h	dngettext
libintl.h	gettext
libintl.h	ngettext
libintl.h	textdomain
link.h	dl_iterate_phdr
locale.h	localeconv

头文件	接口名
malloc.h	malloc_usable_size
malloc.h	valloc
mntent.h	addmntent
mntent.h	endmntent
mntent.h	getmntent
mntent.h	getmntent_r
mntent.h	hasmntopt
mntent.h	setmntent
netdb.h	gethostbyaddr
netdb.h	gethostbyaddr_r
netdb.h	gethostbyname2
netdb.h	gethostbyname2_r
netdb.h	getservbyname_r
netdb.h	getservbyport_r
netdb.h	herror
netdb.h	hstrerror
netinet/ether.h	ether_aton
netinet/ether.h	ether_aton_r
netinet/ether.h	ether_hostton
netinet/ether.h	ether_line
netinet/ether.h	ether_ntoa
netinet/ether.h	ether_ntoa_r
netinet/ether.h	ether_ntohost
poll.h	ppoll
pty.h	forkpty
pty.h	openpty
pwd.h	fgetpwent
pwd.h	putpwent
resolv.h	dn_expand
resolv.h	dn_skipname

头文件	接口名
resolv.h	res_init
resolv.h	res_mkquery
resolv.h	res_query
resolv.h	res_querydomain
resolv.h	res_search
resolv.h	res_send
sched.h	clone
sched.h	sched_getcpu
sched.h	setns
sched.h	unshare
search.h	hcreate_r
search.h	hdestroy_r
search.h	hsearch_r
search.h	tdestroy
shadow.h	endspent
shadow.h	fgetspent
shadow.h	getspent
shadow.h	getspnam
shadow.h	getspnam_r
shadow.h	lckpword
shadow.h	putspent
shadow.h	setspent
shadow.h	sgetspent
shadow.h	ulckpword
signal.h	raise
signal.h	sigandset
signal.h	sigisemptyset
signal.h	sigorset
spawn.h	posix_spawn
spawn.h	posix_spawn_file_actions_addchdir_np

头文件	接口名
spawn.h	posix_spawn_file_actions_addclose
spawn.h	posix_spawn_file_actions_adddup2
spawn.h	posix_spawn_file_actions_addfchdir_np
spawn.h	posix_spawn_file_actions_addopen
spawn.h	posix_spawn_file_actions_destroy
spawn.h	posix_spawn_file_actions_init
spawn.h	posix_spawnattr_destroy
spawn.h	posix_spawnattr_getflags
spawn.h	posix_spawnattr_getpgroup
spawn.h	posix_spawnattr_getschedparam
spawn.h	posix_spawnattr_getschedpolicy
spawn.h	posix_spawnattr_getsigdefault
spawn.h	posix_spawnattr_getsigmask
spawn.h	posix_spawnattr_init
spawn.h	posix_spawnattr_setflags
spawn.h	posix_spawnattr_setpgroup
spawn.h	posix_spawnattr_setschedparam
spawn.h	posix_spawnattr_setschedpolicy
spawn.h	posix_spawnattr_setsigdefault
spawn.h	posix_spawnattr_setsigmask
spawn.h	posix_spawnnp
stdio.h	cuserid
stdio.h	fgetc_unlocked
stdio.h	fgetln
stdio.h	fopencookie
stdio.h	fputc_unlocked
stdio.h	getw
stdio.h	setlinebuf
stdio.h	tmpfile
stdio.h	tmpfile64

头文件	接口名
stdlib.h	aligned_alloc
stdlib.h	at_quick_exit
stdlib.h	div
stdlib.h	ecvt
stdlib.h	fcvt
stdlib.h	gcvt
stdlib.h	getenv
stdlib.h	getloadavg
stdlib.h	ldiv
stdlib.h	lldiv
stdlib.h	mkdtemp
stdlib.h	mkostemp
stdlib.h	ptsname_r
stdlib.h	quick_exit
stdlib.h	reallocarray
stdlib.h	system
stdlib.h	tcgetwinsize
stdlib.h	tcsetwinsize
stdlib.h	valloc
stdlib.h	wcstombs
string.h	explicit_bzero
string.h	memmem
string.h	memcpy
string.h	strlcat
string.h	strverscmp
strings.h	ffsl
strings.h	ffsll
strings.h	index
sys/acct.h	acct
sys/auxv.h	getauxval

头文件	接口名
sys/cachectl.h	cachectl
sys/cachectl.h	cacheflush
sys/epoll.h	epoll_create
sys/epoll.h	epoll_create1
sys/epoll.h	epoll_ctl
sys/epoll.h	epoll_pwait
sys/epoll.h	epoll_wait
sys/eventfd.h	eventfd
sys/eventfd.h	eventfd_read
sys/eventfd.h	eventfd_write
sys/fanotify.h	fanotify_init
sys/fanotify.h	fanotify_mark
sys/file.h	flock
sys/fsuid.h	setfsuid
sys/fsuid.h	setfsuid
sys/inotify.h	inotify_add_watch
sys/inotify.h	inotify_init
sys/inotify.h	inotify_init1
sys/inotify.h	inotify_rm_watch
sys/io.h	ioperm
sys/io.h	iopl
sys/klog.h	klogctl
sys/membarrier.h	membarrier
sys/mman.h	madvise
sys/mman.h	memfd_create
sys/mman.h	mincore
sys/mman.h	mlock2
sys/mman.h	mremap
sys/mman.h	remap_file_pages
sys/mount.h	umount2

头文件	接口名
sys/personality.h	personality
sys/ptrace.h	ptrace
sys/quota.h	quotactl
sys/random.h	getrandom
sys/reboot.h	reboot
sys/resource.h	prlimit
sys/sem.h	semtimedop
sys/sendfile.h	sendfile
sys/signalfd.h	signalfd
sys/socket.h	accept4
sys/socket.h	recvmsg
sys/socket.h	sendmsg
sys/stat.h	fstatat64
sys/stat.h	lchmod
sys/stat.h	lstat64
sys/statfs.h	fstatfs
sys/statvfs.h	fstatvfs
sys/statvfs.h	statvfs
sys/swap.h	swapoff
sys/swap.h	swapon
sys/sysinfo.h	get_avphys_pages
sys/sysinfo.h	get_nprocs
sys/sysinfo.h	get_nprocs_conf
sys/sysinfo.h	get_phys_pages
sys/sysinfo.h	sysinfo
sys/time.h	futimes
sys/time.h	futimesat
sys/time.h	lutimes
sys/timeb.h	ftime
sys/timerfd.h	timerfd_create

头文件	接口名
sys/timerfd.h	timerfd_gettime
sys/timerfd.h	timerfd_settime
sys/timex.h	adjtimex
sys/timex.h	clock_adjtime
sys/uio.h	preadv
sys/uio.h	process_vm_readv
sys/uio.h	process_vm_writev
sys/uio.h	pwritev
sys/wait.h	wait3
sys/wait.h	wait4
sys/xattr.h	fgetxattr
sys/xattr.h	flistxattr
sys/xattr.h	fremovexattr
sys/xattr.h	fsetxattr
sys/xattr.h	getxattr
sys/xattr.h	lgetxattr
sys/xattr.h	listxattr
sys/xattr.h	llistxattr
sys/xattr.h	lremovexattr
sys/xattr.h	lsetxattr
sys/xattr.h	removexattr
sys/xattr.h	setxattr
syslog.h	vsyslog
termios.h	cfmakeraw
termios.h	cfsetspeed
threads.h	call_once
threads.h	cnd_broadcast
threads.h	cnd_destroy
threads.h	cnd_init
threads.h	cnd_signal

头文件	接口名
threads.h	cnd_timedwait
threads.h	cnd_wait
threads.h	mtx_destroy
threads.h	mtx_init
threads.h	mtx_lock
threads.h	mtx_timedlock
threads.h	mtx_trylock
threads.h	mtx_unlock
threads.h	thrd_create
threads.h	thrd_current
threads.h	thrd_detach
threads.h	thrd_equal
threads.h	thrd_exit
threads.h	thrd_join
threads.h	thrd_sleep
threads.h	thrd_yield
threads.h	tss_create
threads.h	tss_delete
threads.h	tss_set
time.h	timegm
time.h	timespec_get
time64.h	timegm64
time64.h	timelocal64
uchar.h	c16rtomb
uchar.h	mbrtoc16
ucontext.h	getcontext
ucontext.h	makecontext
ucontext.h	setcontext
ucontext.h	swapcontext
unistd.h	_Fork

头文件	接口名
unistd.h	acct
unistd.h	brk
unistd.h	chroot
unistd.h	copy_file_range
unistd.h	daemon
unistd.h	dup3
unistd.h	eaccess
unistd.h	endusershell
unistd.h	execvpe
unistd.h	get_current_dir_name
unistd.h	getdomainname
unistd.h	getdtablesize
unistd.h	gettid
unistd.h	getusershell
unistd.h	issetugid
unistd.h	pipe2
unistd.h	posix_close
unistd.h	sbrk
unistd.h	setdomainname
unistd.h	setresgid
unistd.h	setresuid
unistd.h	setusershell
unistd.h	syncfs
unistd.h	syscall
unistd.h	truncate64
unistd.h	ualarm
unistd.h	vfork
unistd.h	vhangup
utmp.h	endutent
utmp.h	getutent

头文件	接口名
utmp.h	getutid
utmp.h	getutline
utmp.h	login_tty
utmp.h	pututline
utmp.h	setutent
utmp.h	updwtmp
utmp.h	utmpname
utmpx.h	updwtmpx
utmpx.h	utmpxname
wchar.h	fgetws
wchar.h	fgetws_unlocked
wchar.h	fputws
wchar.h	fputws_unlocked
wchar.h	fwprintf
wchar.h	fwscanf
wchar.h	getwchar
wchar.h	getwchar_unlocked
wchar.h	putwchar
wchar.h	putwchar_unlocked
wchar.h	swscanf
wchar.h	vswscanf
wchar.h	vwprintf
wchar.h	vwscanf
wchar.h	wcscat
wchar.h	wcscpy
wchar.h	wcscspn
wchar.h	wcsncat
wchar.h	wcspbrk
wchar.h	wcsrchr
wchar.h	wcsspn

头文件	接口名
wchar.h	wcstod
wchar.h	wcstof
wchar.h	wcstok
wchar.h	wcstold
wchar.h	wcswcs
wchar.h	wprintf
wchar.h	wscanf
wctype.h	towctrans
wctype.h	wctrans

须知

自Huawei LiteOS V200R005C00版本起不再支持stdlib.h头文件中提供的atexit()。

6.4 C++兼容规格

下述表格为C++标准库/STL兼容规格。

说明

支持异常和RTTI特性，其他特性由编译器支持；对于STL的特性，支持清单如下，其它特性暂不支持。

- 支持语言支持功能

头文件	描述
<limits>	提供与基本数据类型相关的定义，例如，对于每个数值数据类型，它定义了可以表示出来的最大值和最小值以及二进制数字的位数
<new>	支持动态内存分配

- 支持工具函数

头文件	描述
<utility>	定义重载的关系运算符，简化关系运算符的写入，它还定义了pair类型，该类型是一种模板类型，可以存储一对值。这些功能在库的其他地方使用

头文件	描述
<functional.>	定义了许多函数对象类型和支持函数对象的功能，函数对象是支持operator函数调用运算符的任意对象
<memory>	给容器、管理内存的函数和auto_ptr模板类定义标准内存分配器

- 支持字符串处理

头文件	描述
<string>	为字符串类型提供支持和定义，包括单字节字符串(由char组成)的string和多字节字符串(由wchar_t组成)

- 支持容器类模板

头文件	描述
<vector>	定义vector序列模板，这是一个大小可以重新设置的数组类型，比普通数组更安全、更灵活
<list>	定义list序列模板，这是一个序列的链表，常常在任意位置插入和删除元素
<deque>	定义deque序列模板，支持在开始和结尾的高效插入和删除操作
<queue>	为队列(先进先出)数据结构定义序列适配器queue和priority_queue
<stack>	为堆栈(后进先出)数据结构定义序列适配器stack
<map>	map是一个关联容器类型，允许根据唯一键值，按照升序存储
<set>	set是一个关联容器类型，用于以升序方式存储唯一值
<bitset>	为固定长度的位序列定义bitset模板，它可以看作固定长度的紧凑型bool数组

- 支持迭代器

头文件	描述
<iterator>	给迭代器提供定义和支持

- 支持算法

头文件	描述
<algorithm>	提供一组基于算法的函数，包括置换、排序、合并和搜索

- 支持数值操作

头文件	描述
<complex>	支持复杂数值的定义和操作
<valarray>	支持数值矢量的操作
<numeric>	在数值序列上定义一组一般数学操作，例如accumulate和inner_product

- 支持线程处理

头文件	描述
<mutex>	支持互斥锁的操作
<thread>	支持线程函数的操作
<condition_variable>	支持条件变量的操作

6.5 CMSIS 接口

6.5.1 CMSIS v1.0

本部分简要介绍Huawei LiteOS适配CMSIS v1.0接口的情况，更多关于接口声明、入参介绍、返回值类型、接口使用条件等详细介绍，请参见《[CMSIS v1.0接口官方手册](#)》。

6.5.1.1 CMSIS v1.0 适配接口

LiteOS目前已支持大部分CMSIS v1.0接口，接口声明在open_source/CMSIS/CMSIS/RTOS/LiteOS/INC/cmsis_os1.h，接口描述详见下表。

接口名	类型	描述
osKernelInitialize	内核类接口	初始化操作系统
osKernelStart	内核类接口	启动操作系统
osKernelRunning	内核类接口	判断系统是否启动
osKernelSysTick	内核类接口	获取系统启动后时间（单位：tick）

接口名	类型	描述
osThreadCreate	任务/线程类接口	创建一个任务
osThreadGetId	任务/线程类接口	获取当前的任务句柄
osThreadTerminate	任务/线程类接口	终止某个任务
osThreadYield	任务/线程类接口	切换至同优先级的就绪任务
osThreadSetPriority	任务/线程类接口	设置任务优先级
osThreadGetPriority	任务/线程类接口	获取任务优先级
osDelay	延时类接口	任务延时处理
osTimerCreate	定时器类接口	创建定时器
osTimerStart	定时器类接口	启动定时器（若定时器正在计时会先停止该定时器）
osTimerRestart	定时器类接口	重新启动定时器
osTimerStop	定时器类接口	停止定时器
osTimerDelete	定时器类接口	删除定时器
osSignalSet	信号类接口	设置信号
osSignalClear	信号类接口	清除信号
osSignalWait	信号类接口	等待信号
osMutexCreate	互斥锁类接口	创建互斥锁
osMutexWait	互斥锁类接口	获取互斥锁（在规定时间内阻塞等待）
osMutexRelease	互斥锁类接口	释放互斥锁
osMutexDelete	互斥锁类接口	删除互斥锁
osSemaphoreCreate	信号量类接口	创建信号量
osSemaphoreWait	信号量类接口	获取信号量（阻塞等待）
osSemaphoreRelease	信号量类接口	释放信号量
osSemaphoreDelete	信号量类接口	删除信号量
osPoolCreate	块状内存类接口	创建块状内存池
osPoolAlloc	块状内存类接口	申请内存

接口名	类型	描述
osPoolCAlloc	块状内存类接口	申请内存并清零
osPoolFree	块状内存类接口	释放内存
osMessageCreate	指针消息类接口	创建消息队列（不带内容，一般为数据指针）
osMessagePut	指针消息类接口	往消息队列里放入消息
osMessageGet	指针消息类接口	从消息队列里获取消息
osMailCreate	内容消息类接口	创建消息队列（带内容，可理解为在osMessage的基础上增加块状内存创建）
osMailAlloc	内容消息类接口	申请内存（用于存放消息内容）
osMailCAlloc	内容消息类接口	申请内存并清零（用于存放消息内容）
osMailPut	内容消息类接口	往消息队列里放入消息
osMailGet	内容消息类接口	从消息队列里获取消息
osMailFree	内容消息类接口	释放已申请的内存

6.5.1.2 CMSIS v1.0 不支持接口

CMSIS v1.0不支持接口描述如下表所示。

接口名	类型	描述
osWait	延时类接口	等待任一信号或消息发生

6.5.1.3 CMSIS v1.0 标准接口适配差异

考虑接口的易用性和Huawei LiteOS内部机制与CMSIS标准接口的差异，在适配CMSIS v1.0接口时，对部分接口进行了修改，详见下表。

接口名	类型	描述
osTimerStart	定时器类接口	增加一种定时器类型osTimerDelay，与osTimerOnce同为单次定时器，差别在于osTimerOnce超时会删除定时器，但osTimerDelay不会，可以通过osTimerStart重复启动
osBinarySemaphoreCreate	信号量类接口	新增接口，用于创建二值信号量

6.5.2 CMSIS v2.0

本部分简要介绍Huawei LiteOS适配CMSIS v2.0接口的情况，更多关于接口声明、入参介绍、返回值类型、接口使用条件等详细介绍，请参见《[CMSIS v2.0接口官方手册](#)》。

6.5.2.1 CMSIS v2.0 适配接口

LiteOS目前已支持大部分CMSIS v2.0接口，接口声明在open_source/CMSIS/CMSIS/RTOS2/Include/cmsis_os2.h，接口描述详见下表。

接口名	类型	描述
osKernelInitialize	内核类接口	初始化操作系统
osKernelGetInfo	内核类接口	获取系统版本信息
osKernelGetState	内核类接口	获取系统状态（具体状态可参考osThreadState_t）
osKernelStart	内核类接口	启动操作系统
osKernelLock	内核类接口	锁内核（锁调度）
osKernelUnlock	内核类接口	解锁内核（解锁调度）
osKernelRestoreLock	内核类接口	恢复内核锁状态
osKernelGetTickCount	内核类接口	获取系统启动后时间（单位：tick）
osKernelGetTickFreq	内核类接口	获取每秒的tick数
osKernelGetSysTimerCount	内核类接口	获取系统启动后时间（单位：cycle）
osKernelGetSysTimerFreq	内核类接口	获取每秒的CPU cycle数
osThreadNew	任务/线程类接口	创建任务
osThreadJoin	任务/线程类接口	等待指定的任务并回收任务资源（需要开启内核LOSCFG_TASK_JOINABLE配置）
osThreadDetach	任务/线程类接口	设置任务为detached属性（需要开启内核LOSCFG_TASK_JOINABLE配置）
osThreadGetName	任务/线程类接口	获取任务名
osThreadGetId	任务/线程类接口	获取任务句柄

接口名	类型	描述
osThreadGetState	任务/线程类接口	获取任务状态
osThreadGetStackSize	任务/线程类接口	获取任务栈大小
osThreadGetStackSpace	任务/线程类接口	获取未使用过的任务栈空间
osThreadSetPriority	任务/线程类接口	设置任务优先级
osThreadGetPriority	任务/线程类接口	获取任务优先级
osThreadYield	任务/线程类接口	切换至同优先级的就绪任务
osThreadSuspend	任务/线程类接口	挂起任务（恢复前无法得到调度）
osThreadResume	任务/线程类接口	恢复任务
osThreadTerminate	任务/线程类接口	终止任务（建议不要主动终止任务）
osThreadGetCount	任务/线程类接口	获取已创建的任务数量
osThreadFlagsSet	任务事件类接口	写入指定事件
osThreadFlagsClear	任务事件类接口	清除指定事件
osThreadFlagsGet	任务事件类接口	获取当前任务事件
osThreadFlagsWait	任务事件类接口	等待指定事件
osDelay	延时类接口	任务延时（单位：tick）
osDelayUntil	指针消息类接口	延时至某一时刻（单位：tick）
osTimerNew	定时器类接口	创建定时器
osTimerGetName	定时器类接口	获取定时器名称（目前固定返回NULL）
osTimerStart	定时器类接口	启动定时器（若定时器正在计时会先停止该定时器）
osTimerStop	定时器类接口	停止定时器
osTimerIsRunning	定时器类接口	定时器是否在计月中

接口名	类型	描述
osTimerDelete	定时器类接口	删除定时器
osEventFlagsNew	事件类接口	创建事件（与任务事件ThreadFlags的差别在于有独立的句柄和控制块）
osEventFlagsGetName	事件类接口	获取事件名称（目前固定返回NULL）
osEventFlagsSet	事件类接口	写入指定事件
osEventFlagsClear	事件类接口	清除指定事件
osEventFlagsGet	事件类接口	获取当前事件值
osEventFlagsWait	事件类接口	等待指定事件
osEventFlagsDelete	事件类接口	删除事件
osMutexNew	互斥锁类接口	创建互斥锁
osMutexGetName	互斥锁类接口	获取互斥锁名称（目前固定返回NULL）
osMutexAcquire	互斥锁类接口	获取互斥锁（阻塞等待）
osMutexRelease	互斥锁类接口	释放互斥锁
osMutexGetOwner	互斥锁类接口	获取持有该互斥锁的任务句柄
osMutexDelete	互斥锁类接口	删除互斥锁
osSemaphoreNew	信号量类接口	创建信号量
osSemaphoreGetName	信号量类接口	获取信号量名称（目前固定返回NULL）
osSemaphoreAcquire	信号量类接口	获取信号量（阻塞等待）
osSemaphoreRelease	信号量类接口	释放信号量
osSemaphoreGetCount	信号量类接口	获取信号量的计数值
osSemaphoreDelete	信号量类接口	删除信号量
osMessageQueueNew	消息队列类接口	创建消息队列
osMessageQueueGetName	消息队列类接口	获取消息队列名称（目前固定返回NULL）

接口名	类型	描述
osMessageQueuePut	消息队列类接口	往消息队列里放入消息
osMessageQueueGet	消息队列类接口	从消息队列里获取消息
osMessageQueueGetCapacity	消息队列类接口	获取消息队列节点数量
osMessageQueueGetMsgSize	消息队列类接口	获取消息队列节点大小
osMessageQueueGetCount	消息队列类接口	获取当前消息队列里的消息数量
osMessageQueueGetSpace	消息队列类接口	获取当前消息队列里的剩余消息数量
osMessageQueueDelete	消息队列类接口	删除消息队列

6.5.2.2 CMSIS v2.0 不支持接口

CMSIS v2.0不支持接口描述如下表所示。

接口名	类型	描述
osKernelSuspend	任务/线程类接口	挂起内核阻止调度，一般用于低功耗处理，目前Huawei LiteOS已提供Tickless、Runstop等低功耗机制，暂未适配该接口
osKernelResume	任务/线程类接口	同osKernelSuspend描述
osThreadExit	任务/线程类接口	Huawei LiteOS任务支持自删除，任务退出前不需要调用osThreadExit
osThreadEnumerate	任务/线程类接口	获取已创建的任务列表，目前未适配该接口，用户可以调用Huawei LiteOS的LOS_TaskInfoGet等维测接口获取任务状态
osMemoryPoolAlloc	块状内存类接口	接口需要支持超时时间内申请内存块，目前Huawei LiteOS暂未提供这种机制
osMemoryPoolFree	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolNew	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolGetName	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现

接口名	类型	描述
osMemoryPoolGetCapacity	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolGetBlockSize	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolGetCount	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolGetSpace	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMemoryPoolDelete	块状内存类接口	由于osMemoryPoolAlloc未实现，剩余osMemPool类接口暂不实现
osMessageQueueReset	消息队列类接口	操作系统暂不提供清空队列接口，由用户对队列内容进行操作，避免资源泄漏或其他异常

6.5.2.3 CMSIS v2.0 标准接口适配差异

考虑接口的易用性和Huawei LiteOS内部机制与CMSIS标准接口的差异，在适配CMSIS v2.0接口时，对部分接口进行了修改，详见下表。

接口名	类型	描述
osKernelGetTick2ms	内核类接口	新增接口，用于获取系统启动后时间（单位：ms）。接口声明在compat/cmsis/cmsis_os.h
osMs2Tick	内核类接口	新增接口，用于毫秒与系统Tick转换。接口声明在compat/cmsis/cmsis_os.h
osThreadNew	任务/线程类接口	优先级范围仅支持[osPriorityLow3, osPriorityHigh]
osThreadSetPriority	任务/线程类接口	优先级范围仅支持[osPriorityLow3, osPriorityHigh]

6.6 linux 适配

Huawei LiteOS目前适配的是linux-4.19.90版本。Linux适配支持裁剪，如果需要使用Linux适配功能，可以打开菜单，选择Compat ---> Enable Linux以使能Linux适配模块。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX	Linux适配模块的裁剪开关	YES/NO	YES	无

6.6.1 完成量（completion）

6.6.1.1 概述

基本概念

完成量（completion）是一种轻量级任务同步机制。完成量是对信号量的一种补充，Linux的信号量机制，在同一个信号量上允许并发执行up()和down()，在多CPU运行场景下，up()可能试图访问一个不存在的信号量，为了防止这种错误，专门设计了完成量这种机制。

当一个任务需要等待另一个任务完成某操作后才能执行时，通过完成量允许被等待任务在完成指定操作后通知等待任务，从而唤醒等待任务，实现任务同步。

6.6.1.2 开发指导

使用场景

完成量应用于多任务同步场合，实现任务间的同步。

功能

功能分类	接口名	描述	兼容性
初始化完成量	init_completion	初始化一个完成量	不完全兼容，如果入参为空指针会退出，不做初始化
等待完成量	wait_for_completion	永久等待完成量	不完全兼容，如果入参为空指针或接口使用场景不正确会退出
	wait_for_completion_timeout	超时等待完成量，超时时间单位为Tick	不完全兼容，接口实现中增加了非法判断，使用时请查看API参考
唤醒等待完成量的任务	complete	唤醒等待该完成量的任务队列中的首个任务	不完全兼容，如果入参为空会退出
	complete_all	唤醒所有等待该完成量的任务	不完全兼容，如果入参为空会退出

开发流程

完成量的典型开发流程：

1. 打开菜单，选择Compat ---> Enable Linux ---> Enable completion，使能完成量。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX_COMPLETION	使能Linux完成量	YES/NO	NO	无

2. 初始化完成量init_completion，该接口会创建一个完成量。
3. 等待完成量wait_for_completion_timeout/wait_for_completion。
4. 唤醒等待完成量的任务complete/complete_all。

6.6.1.3 注意事项

- 完成量类似于信号量机制，参考信号量机制[注意事项](#)，即不能在中断中以阻塞模式（永久阻塞和定时阻塞）操作完成量，因为中断不能被阻塞。
- 需要用户保证完成量接口的完成量指针入参的合法性，即入参必须为合法的完成量指针。

6.6.1.4 编程实例

实例描述

示例中，任务Example_TaskEntry创建了任务Example_Completion。Example_Completion等待完成量而阻塞，Example_TaskEntry唤醒该完成量。通过打印的先后顺序可以理解完成量操作时伴随的任务切换。

1. 在任务Example_TaskEntry中创建任务Example_Completion，其中任务Example_Completion优先级高于Example_TaskEntry。
2. 在任务Example_Completion中等待完成量，阻塞，发生任务切换，执行任务Example_TaskEntry。
3. 在任务Example_TaskEntry中唤醒完成量，发生任务切换，执行任务Example_Completion。
4. Example_Completion得以执行，直到该任务结束。
5. Example_TaskEntry得以执行，直到任务结束。

编程示例

前提条件：在menuconfig菜单中已经使能Linux适配模块。

代码实现如下：

```
#include "linux/completion.h"
#include "los_task.h"

/* 任务ID */
UINT32 g_testTaskId;

/* 完成量 */
struct completion exampleCompletion;

/* 用例任务入口函数 */
VOID Example_Completion(VOID)
{
    UINT32 ret;
```

```
/* 以超时等待方式等待完成量，超时时间为100Ticks */
printf("Example_Completion wait completion\n");
ret = wait_for_completion_timeout(&exampleCompletion, 100);

if(ret == 0) {
    printf("Example_Completion, wait completion timeout\n");
} else {
    printf("Example_Completion, wait completion success\n");
}
return;
}

UINT32 Example_TaskEntry(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S task;

    /* 完成量初始化 */
    init_completion(&exampleCompletion);

    /* 创建任务 */
    memset(&task, 0, sizeof(TSK_INIT_PARAM_S));
    task.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Completion;
    task.pcName       = "EventTsk1";
    task.uwStackSize  = LOSCFG_BASE_CORE_TSK_DEFAULT_STACK_SIZE;
    task.usTaskPrio   = 8;
    ret = LOS_TaskCreate(&g_testTaskId, &task);
    if(ret != LOS_OK) {
        printf("task create failed \n");
        return LOS_NOK;
    }

    /* 唤醒完成量 */
    printf("Example_TaskEntry complete\n");
    complete(&exampleCompletion);

    printf("Delete Task.\n");
    /* 删除任务 */
    ret = LOS_TaskDelete(g_testTaskId);
    if(ret != LOS_OK) {
        printf("task delete failed\n");
        return LOS_NOK;
    }

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
Example_Completion wait completion
Example_TaskEntry complete
Example_Completion, wait completion success
Delete Task.
```

完整实例代码

sample_completion.c

6.6.2 工作队列（workqueue）

6.6.2.1 概述

基本概念

工作队列提供一种机制：任何系统模块都可以将工作项挂载到工作队列，由工作队列按照FIFO方式处理挂载在上面的工作项。

工作队列通过工作队列任务处理挂载到工作队列中的工作项。队列处理任务允许被重新调度和睡眠，从而节省系统资源。

Huawei LiteOS的工作队列机制提供了丰富的对外接口以管理工作队列。当有工作项挂载到工作队列中时，工作队列处理任务将被唤醒去处理工作队列中的工作项，直到所有工作项都被处理完，工作队列处理任务将重新进入睡眠状态。

6.6.2.2 开发指导

使用场景

工作队列运用在对时间要求不高的场景，原因是工作队列里的工作只有在工作队列处理任务得到运行时才能被逐一处理，而工作队列处理任务是否能被及时唤醒调度取决于其他内核任务，另外在中断里也不能调度，只有在退出中断后才能得到调度。

功能

功能分类	接口名	描述	兼容性
创建/销毁工作队列	create_singlethread_workqueue	在当前CPU上创建一个工作队列	完全兼容
	create_workqueue	Huawei LiteOS只在当前CPU上创建一个工作队列，而对于多CPU场景，linux会在每个CPU上都创建一个工作队列	不完全兼容
	destroy_workqueue	销毁一个工作队列	完全兼容
初始化工作	INIT_WORK	初始化一个工作，将该工作与指定的处理函数绑定。当该工作被执行时，就会执行这个处理函数	完全兼容
	INIT_DELAYED_WORK	初始化一个延迟工作，将该工作与指定的处理函数绑定，该工作将延迟一段时间后才会被执行。当该工作被执行时，就会执行这个处理函数	完全兼容

功能分类	接口名	描述	兼容性
挂载工作项到工作队列	queue_work	挂载一个工作项到指定工作队列	不完全兼容，有参数非法性检查，详见API参考
	queue_delayed_work	挂载一个延迟工作项到指定工作队列	不完全兼容，有参数非法性检查，详见API参考
	schedule_work	挂载一个工作项到系统默认工作队列	完全兼容
	schedule_delayed_work	挂载一个延迟工作项到系统默认工作队列	不完全兼容，有参数非法性检查，详见API参考
查询工作的状态	work_busy	查询一个指定工作当前的状态	不完全兼容，返回值不同，详见API参考
执行工作	flush_work	阻塞等待一个指定的非延迟工作项执行，完成后返回	完全兼容
	flush_delayed_work	取消延迟，并阻塞等待这个工作项执行，完成后返回	完全兼容
取消工作	cancel_delayed_work	取消一个延迟工作，如果有其他任务正在取消该工作，则返回失败	不完全兼容，返回值不同
	cancel_delayed_work_sync	取消一个延迟工作项，须等待工作项执行结束再返回。如果有其他任务正在取消该工作，那么等待其执行完毕	不完全兼容，返回值不同
	cancel_work_sync	取消一个工作项，须等待工作项执行结束再返回。如果有其他任务正在取消该工作，那么等待其执行完毕	不完全兼容，返回值不同

说明

- 创建工作队列时，会初始化信号量，用于工作队列处理任务的睡眠和唤醒，以及初始化重要结构体并搭建工作队列链表。
- 销毁工作队列时，系统先对资源进行加锁操作，再删除工作队列处理任务，释放信号量资源，释放内存空间等，最后解锁。
- 有两种类型的工作：非延迟工作和延迟工作。工作可以立即得到挂载，而延迟工作需要延时指定时间后才能被挂载到工作队列中。

开发流程

工作队列的典型流程：

- 1. 打开菜单，选择Compat ---> Enable Linux ---> Enable workqueue，使能工作队列。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX_WORKQUEUE	使能Linux工作队列	YES/NO	NO	LOSCFG_COMPAT_LINUX_TIMER

- 2. 调用create_workqueue接口创建工作队列。
- 3. 调用INIT_WORK/INIT_DELAYED_WORK初始化工作。
- 4. 调用queue_work/queue_delayed_work接口挂载工作项到工作队列。
- 5. 调用flush_work/flush_delayed_work接口立即执行工作。
- 6. 调用cancel_delayed_work_sync/cancel_work_sync接口取消工作。
- 7. 调用destroy_workqueue接口删除工作队列。

6.6.2.3 注意事项

- 挂载到工作队列的工作项由工作队列处理任务进行处理，而任务的调度运行在时间上具有不确定性，所以对时间要求高的工作不建议使用工作队列进行处理。
- Huawei LiteOS使用工作队列名对工作队列进行标识，所以不能存在相同名称的工作队列。
- 可以使用schedule_work/schedule_delayed_work接口向系统默认的工作队列挂载工作，而不用再创建工作队列。

6.6.2.4 编程实例

实例描述

本实例实现如下功能：

- 1. 创建名称为wq_test的工作队列。
- 2. 分配工作内存并且初始化工作。
- 3. 向工作队列中挂载工作。
- 4. 立即执行工作。
- 5. 销毁工作队列。

编程示例

前提条件：在menuconfig菜单中已经使能Linux适配模块。

代码实现如下：

```
#include "los_printf.h"
#include "linux/workqueue.h"
```

```
static void Work_Func(struct work_struct *work)
{
    int i;
    for (i = 0; i < 2; i++) {
        dprintf("workqueue function is been called!..%d..%d..\n", i, work->work_status);
    }
}

UINT32 Example_Workqueue(VOID)
{
    struct workqueue_struct *wq;
    struct work_struct *work;
    UINT32 ret = LOS_OK;

    wq = create_workqueue("test1008");
    dprintf("create the workqueue successfully!\n\n");

    work =(struct work_struct *)malloc(sizeof(struct work_struct));
    if (!work) {
        ret = LOS_NOK;
    }
    dprintf("create work ok!\n\n");

    INIT_WORK(work, Work_Func);
    dprintf("init the work ok!\n\n");

    ret = queue_work(wq, work);
    dprintf("mount the work into workqueue successfully!\n\n");

    ret = flush_work(work);
    dprintf("flush the work ok!\n\n");

    destroy_workqueue(wq);
    dprintf("destroy the work ok!\n\n");

    return ret;
}
```

结果验证

```
create the workqueue successfully!

create work ok!

init the work ok!

mount the work into workqueue successfully!

workqueue function is been called!..0..3..
workqueue function is been called!..1..3..
flush the work ok!

destroy the work ok!
```

完整实例代码

sample_workqueue.zip

6.6.3 等待队列（waitqueue）

6.6.3.1 概述

基本概念

等待队列常用于线程等待资源。

当一个线程必须获取某些资源后才能进行下一步操作，且不占用CPU资源的时候，可以将该线程放入等待队列中，然后系统会切换到其他进程，等资源准备好后再唤醒等待队列。等待队列可以概括为两种等待情况：超时等待和循环等待。

Huawei LiteOS的等待队列机制提供了丰富的对外接口以管理等待队列，可以满足linux系统等待队列的绝大多数功能。

6.6.3.2 开发指导

使用场景

与工作队列相比，等待队列运用在对时间要求较高的场景，尤其是超时等待机制，常见场景有：同步系统资源访问、中断底半部和异步事件通知等。

功能

功能分类	接口名	描述	兼容性	注意事项
初始化等待队列头	DECLARE_WAIT_QUEUE_HEAD	定义并初始化等待队列头	完全兼容	不涉及
	init_waitqueue_head	初始化等待队列头	完全兼容	不涉及
获取等待队列合法性	waitqueue_active	判断等待队列是否可用	完全兼容	用户保证入参是合法指针，不能是空指针
等待事件	wait_event	等待事件	完全兼容	不涉及
	wait_event_interruptible	等待事件	不完全兼容，该事件不可被信号中断	不涉及
	wait_event_interruptible_timeout	超时等待事件	不完全兼容，该事件不可被信号中断，返回值不同，入参范围有限制，详见API参考	不涉及
唤醒事件	wake_up_interruptible	唤醒事件	不完全兼容，入参合法时直接返回	不涉及
	wake_up_interruptible_poll	唤醒事件	完全兼容	不涉及

功能分类	接口名	描述	兼容性	注意事项
	wake_up	唤醒事件	完全兼容	不涉及

说明

有两种类型的等待队列：

- 循环等待，只有等到资源后才会退出等待。
- 超时等待，即使未等到资源，在等待时间到达之后，也会退出等待。

开发流程

等待队列的典型开发流程：

1. 打开菜单，选择Compat ---> Enable Linux ---> Enable waitqueue，使能等待队列。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX_WAITQUEUE	使能Linux等待队列	YES/NO	NO	LOSCFG_BASE_IPC_EVENT

2. 调用DECLARE_WAIT_QUEUE_HEAD/init_waitqueue_head接口初始化等待队列头。
3. 调用wait_event/wait_event_interruptible/wait_event_interruptible_timeout等待资源。
4. 调用wake_up/wake_up_interruptible/wake_up_interruptible_poll唤醒等待队列。

6.6.3.3 注意事项

1. Huawei LiteOS的等待队列不支持linux内核signal打断的机制。
2. Huawei LiteOS不支持在一个等待队列中增加多个等待项，一个等待队列中只能有一个等待项。

6.6.3.4 编程实例

实例描述

本实例实现如下功能：

1. 定义并初始化一个等待队列头。
2. 创建一个任务，在任务中挂起事件，等待被唤醒。
3. 唤醒事件。

编程示例

前提条件：在menuconfig菜单中已经使能Linux适配模块。

代码实现如下：

```
#include "los_printf.h"
#include "los_task.h"
#include "linux/wait.h.h"

static UINT32 g_TestTaskID;
static DECLARE_WAIT_QUEUE_HEAD(g_waitQueue);
static BOOL g_condition = FALSE;

static VOID Example_Task(VOID)
{
    UINT32 timeout = 10; // 10 ticks timeout
    UINT32 ret;

    ret = wait_event_interruptible_timeout(g_waitQueue, g_condition, timeout);
    if (ret > 0) {
        g_TestCount++;
    }
}

UINT32 Example_Waitqueue(VOID)
{
    UINT32 ret;
    TSK_INIT_PARAM_S stTask = {0};

    stTask.pfnTaskEntry = (TSK_ENTRY_FUNC)Example_Task;
    stTask.usTaskPrio = TSK_PRIOR_HI;
    stTask.pcName = "wait_task";
    stTask.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
    stTask.uwResved = LOS_TASK_STATUS_DETACHED;

    g_TestCount = 0;

    ret = LOS_TaskCreate(&g_TestTaskID, &stTask);
    if (ret != LOS_OK) {
        printf("Example_Task create Failed!\r\n");
        return LOS_NOK;
    }

    LOS_TaskDelay(2);
    g_condition = TRUE;
    wake_up_interruptible(&g_waitQueue);
    LOS_TaskDelay(1);

    if (g_TestCount == 1) {
        printf("Wakeup succeeded!\r\n");
        return LOS_OK;
    } else {
        printf("Wakeup failed!\r\n");
        return LOS_NOK;
    }
}
```

结果验证

编译运行得到的结果为：

```
Wakeup succeeded!
```

完整实例代码

sample_waitqueue.rar

6.6.4 中断

6.6.4.1 概述

基本概念

Linux内核中有专门的中断函数。为提高系统使用的友好度，Huawei LiteOS基于自身的中断机制，适配了Linux中的相关中断接口。

Huawei LiteOS的中断支持：

- 中断申请
- 中断删除
- 中断使能
- 中断屏蔽
- 中断底半部（基于workqueue）

中断顶半部和底半部的概念

中断处理程序耗时应尽可能短，以满足中断的快速响应，为了平衡中断处理程序的性能与工作量，将中断处理程序分解为两部分：顶半部和底半部。

顶半部完成尽可能少的比较紧急的任务，它往往只是简单地读取寄存器中的中断状态并清除中断标志位，即进行“登记工作”，将耗时的底半部处理程序挂到系统的底半部执行队列中去。

6.6.4.2 开发指导

功能

Huawei LiteOS 中断模块适配的linux接口如下表，接口详细信息可以查看API参考。

接口名	描述	兼容性
request_irq	申请中断，将中断处理程序注册到指定中断号对应的链表中，支持一个中断号注册多个中断处理程序	不完全兼容，详见API参考
free_irq	删除中断	完全兼容
enable_irq	使能指定中断	完全兼容
disable_irq	屏蔽指定中断	完全兼容

说明

中断底半部的功能，可利用workqueue实现，在上半部中初始化一个工作项，将其添加到workqueue中，系统会在空闲时执行workqueue里的底半部，详细使能用法请查阅[工作队列（workqueue）](#)。

开发流程

1. 打开菜单，选择Compat ---> Enable Linux ---> Enable irq interrupt，使能中断。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX_IRQ	使能Linux中断	YES/NO	NO	LOSCFG_ARCH_INTERRUPT_TAKEOVER

2. 调用申请中断接口request_irq。
3. 调用disable_irq接口屏蔽指定中断。
4. 调用enable_irq接口使能指定中断。
5. 调用free_irq接口删除中断。

6.6.4.3 注意事项

- 调用request_irq()申请中断时，中断处理程序入参要符合(int, void*)标准格式。直接调用LOS_HwiCreate()创建中断时，中断处理程序入参可以为NULL。
- 中断处理程序中禁止调用request_irq()和free_irq()接口。
- 当中断ID为共享中断时，不能通过LOS_HwiCreate()创建中断，且request_irq()入参dev指针不能为NULL，应与处理程序一一对应。直接调用LOS_HwiCreate()创建的中断，不能通过free_irq()删除。

6.6.4.4 编程实例

实例描述

本实例实现如下功能：

1. 申请中断。
2. 删除中断。

编程示例

前提条件：

- 在menuconfig菜单中使能Linux适配模块。
- 在menuconfig菜单中配置中断使用最大数和可设置的中断优先级个数，详见[配置中断模块](#)。

代码实现如下：

```
#include "los_hwi.h"
#include "linux/interrupt.h"

#define HWI_NUM_INT50 50
void Uart_Irqhandle(int irq, void *dev)
{
    printf("\nuart0:the function1 \n");
}
void Hwi_Test(void)
{
    int a = 1;
    void *dev = &a;
    unsigned long flags = 0;
    const char * name = "hwiTest";
```



```
request_irq(HWI_NUM_INT50, Uart_Irqhandle, flags, name, dev); // 创建中断
free_irq(HWI_NUM_INT50, dev);                               // 删除中断
}
```

完整实例

```
sample_irq.c
```

6.6.5 高精度定时器

6.6.5.1 概述

基本概念

高精度软件定时器利用一定的硬件定时器资源，通过算法实现更高精度的软件定时器，满足了对精确时间的需求。

Huawei LiteOS高精度软件定时器设计了一套软件架构，提供了微秒级的定时精度，以满足对精确时间有迫切需求的应用程序或内核驱动。

6.6.5.2 开发指导

使用场景

内核软件定时器的定时精度为毫秒级别，定时精度并不高，无法满足对定时精度要求更高的场合。高精度软件定时器弥补了内核软件定时器在这方面的不足，提供了精度到达微秒级别的定时接口。

功能

接口名	描述	兼容性
hrtimer_init	初始化高精度软件定时器资源，这部分功能已经在内核初始化阶段实现，所以这个接口只是适配linux接口，并无实现	完全兼容
hrtimer_create	创建高精度软件定时器，设置定时时间及回调函数	Huawei LiteOS自定义接口
hrtimer_start	启动高精度定时器	不完全兼容，返回值不同，详见API参考
hrtimer_cancel	取消还未超时的高精度定时器	不完全兼容，返回值不同，详见API参考
hrtimer_forward	修改还未超时的高精度定时器的定时时间，重新设定超时时间	不完全兼容，入参不同，详见API参考
hrtimer_is_queued	如果高精度定时器已创建，则返回0，否则返回1	不完全兼容，返回值不同，详见API参考

开发流程

高精度软件定时器使用的典型流程：

1. 打开菜单，选择Compat ---> Enable Linux ---> Enable Linux Hrtimer，使能高精度定时器。

配置项	含义	取值范围	默认值	依赖
LOSCFG_COMPAT_LINUX_HRTIMER	使能Linux高精度定时器	YES/NO	YES	LOSCFG_COMPAT_LINUX

2. 定义一个hrtimer变量和一个ktime变量。
3. 接着根据定时时间，给ktime变量赋值，分别为sec及usec字段。
4. 然后调用hrtimer_create接口，创建高精度定时器，设置其定时器时间及回调函数。
5. 最后调用hrtimer_start接口，启动高精度软件定时器。
6. 调用hrtimer_cancel与hrtimer_forward接口取消或修改定时器的超时时间。
7. hrtimer_is_queued接口辅助用户判断高精度定时器是否已经创建。

6.6.5.3 注意事项

- 高精度软件定时器接口涉及的hrtimer变量，需要用户提供，接口不会为其申请任何资源。
- hrtimer_init接口没有实现，只是适配linux接口，所以使用时可不调用。
- 目前高精度软件定时器只支持相对时间模式，不支持绝对时间模式。
- 高精度软件定时器依赖硬件，使用前需要确认硬件是否支持。
- 高精度定时器存在指令时间消耗，所以实际定时触发时间会大于设置的时间。

6.6.5.4 编程实例

实例描述

本实例实现如下功能：

1. 用户定义swtmr变量及time变量。
2. 调用hrtimer_create接口创建高精度定时器，设置其定时器时间及回调函数。
3. 调用hrtimer_start接口启动定时器。
4. 调用hrtimer_forward接口修改定时时间，由原来的80ms修改为40ms。
5. 调用LOS_Msleep函数使任务延迟50ms。
6. 再调用hrtimer_cancel接口取消定时器，发现此时定时器已超时不存在了。

编程示例

前提条件：已使能高精度定时器。

代码实现如下：

```
#include "los_base.h"
#include "linux/hrtimer.h"

static enum hrtimer_restart Hrtimer_Func(struct hrtimer *arg)
{
    dprintf("The hrtimer is timeout!!!\n");
    return HRTIMER_NORESTART;
}

static UINT32 TestCase(VOID)
{
    struct hrtimer swtmr;
    union ktime time;
    union ktime interval = { 0 };
    interval.tv.sec = 0;
    interval.tv.usec = 40000;
    int ret;

    dprintf("----->test start<-----\n");

    time.tv.sec = 0;
    time.tv.usec = 80000;
    ret = hrtimer_create(&swtmr, time, Hrtimer_Func);
    if (ret == 0) {
        dprintf("Hrtimer create successfully!\n");
    }

    ret = hrtimer_start(&swtmr, time, HRTIMER_MODE_REL);
    if (ret == 0) {
        dprintf("Hrtimer start successfully!\n");
    }

    ret = hrtimer_forward(&swtmr, interval);

    LOS_Msleep(50);

    ret = hrtimer_cancel(&swtmr);
    if (ret == 0) {
        dprintf("Hrtimer already timeout!\n");
    }

    dprintf("----->test end<-----\n");

    return LOS_OK;
}
```

结果验证

编译运行得到的结果为：

```
----->test start<-----
Hrtimer create successfully!
Hrtimer start successfully!
The hrtimer is timeout!!!
Hrtimer already timeout!
----->test end<-----
```

完整实例代码

sample_hrtimer.c

6.6.6 linux 适配接口概览

Huawei LiteOS提供的linux适配接口如下表所示。

序号	文件名	接口名	说明	兼容性	注意事项
1	atomic.h	atomic_add	原子变量做加法	完全兼容	不涉及
2	atomic.h	atomic_add_return	原子变量做加法，并返回最新值	完全兼容	不涉及
3	atomic.h	atomic_dec	原子变量减1	完全兼容	不涉及
4	atomic.h	atomic_dec_and_test	递减原子变量，并测试是否为0	完全兼容	不涉及
5	atomic.h	atomic_dec_return	原子变量减1，并返回最新值	完全兼容	不涉及
6	atomic.h	atomic_inc	原子变量加1	完全兼容	不涉及
7	atomic.h	atomic_inc_return	原子变量加1，并返回最新值	完全兼容	不涉及
8	atomic.h	atomic_read	读取原子变量的值	完全兼容	不涉及
9	atomic.h	atomic_set	设置原子变量的值	完全兼容	不涉及
10	atomic.h	atomic_sub	原子变量做减法	完全兼容	不涉及
11	checksum.h	csum_partial	数据校验	完全兼容	不涉及
12	checksum.h	csum_partial_copy_nocheck	复制和数据校验	完全兼容	不涉及
13	checksum.h	in_cksum	计算校验和	Huawei LiteOS自定义接口	不涉及
14	checksum.h	in_cksum_copy	计算校验和并复制	Huawei LiteOS自定义接口	不涉及
15	clock.h	sched_clock	获取系统时间	完全兼容	不涉及
16	completion.h	complete	唤醒一个等待completion的线程	不完全兼容，如果入参为空会退出	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
17	completion.h	complete_all	唤醒所有等待 completion 的线程	不完全兼容，如果入参为空会退出	不涉及
18	completion.h	init_completion	初始化 completion 结构	不完全兼容，如果入参为空指针会退出，不做初始化	不涉及
19	completion.h	wait_for_completion	等待 completion	不完全兼容，如果入参为空指针或接口使用场景不正确会退出	不涉及
20	completion.h	wait_for_completion_timeout	等待 completion，若超时则结束等待	不完全兼容，接口实现中增加了非法判断，详见API参考	不涉及
21	crc32.h	crc32	计算crc	完全兼容	不涉及
22	delay.h	msleep	延时，程序暂停若干时间（单位ms）	完全兼容	不涉及
23	delay.h	udelay	忙等延迟。只用于极小时间（微秒级）的延迟	完全兼容	不涉及
24	delay.h	mdelay	忙等延迟，延迟时间以毫秒为单位	完全兼容	不涉及
25	hrtimer.h	hrtimer_cancel	取消计时器并等待处理程序完成	不完全兼容，返回值不同，而且依赖平台，需要适配后才能实现，详见API参考	不涉及
26	hrtimer.h	hrtimer_create	设置时间段并创建回调函数	Huawei LiteOS自定义接口，依赖平台，需要适配后才能实现	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
27	hrtimer.h	hrtimer_forward	转发计时器到期	不完全兼容，入参不同，而且依赖平台，需要适配后才能实现，详见API参考	不涉及
28	hrtimer.h	hrtimer_init	将计时器初始化为给定的时钟	依赖平台，需要适配后才能实现	不涉及
29	hrtimer.h	hrtimer_is_queued	帮助程序，检查计时器是否在队列之一上	不完全兼容，返回值不同，而且依赖平台，需要适配后才能实现，详见API参考	不涉及
30	hrtimer.h	hrtimer_start	（重新）在当前CPU上启动hrtimer	不完全兼容，返回值不同，而且依赖平台，需要适配后才能实现，详见API参考	不涉及
31	interrupt.h	disable_irq	屏蔽中断	完全兼容	不涉及
32	interrupt.h	enable_irq	使能中断	完全兼容	不涉及
33	interrupt.h	free_irq	释放中断服务	完全兼容	不涉及
34	interrupt.h	request_irq	注册中断服务	完全兼容	不涉及
35	io.h	readb	从I/O读取1字节	完全兼容	不涉及
36	io.h	readl	从I/O读取4字节	完全兼容	不涉及
37	io.h	readw	从I/O读取2字节	完全兼容	不涉及
38	io.h	writeb	向I/O写入1字节	完全兼容	不涉及
39	io.h	writel	向I/O写入4字节	完全兼容	不涉及
40	io.h	writew	向I/O写入2字节	完全兼容	不涉及
41	jiffies.h	get_jiffies_64	获取自系统启动以来的滴答数	完全兼容	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
42	jiffies.h	jiffies_to_msecs	将滴答数转换成毫秒	功能完全兼容，参数有范围限制 (0, 0xFFFFFFFF * LOSCFG_BASE_CORE_TICK_PER_SECOND / OS_SYS_MS_PER_SECOND]	不涉及
43	jiffies.h	msecs_to_jiffies	将毫秒转换成滴答数	不完全兼容，目前只支持 LOSCFG_BASE_CORE_TICK_PER_SECOND ≤ 1000且是1000的约数	不涉及
44	kernel.h	div_s64	除法运算	完全兼容	不涉及
45	kernel.h	div_s64_rem	除法运算	完全兼容	用户保证入参是合法指针，不能是空指针
46	kernel.h	div_u64	除法运算	完全兼容	不涉及
47	kernel.h	div_u64_rem	除法运算	完全兼容	用户保证入参是合法指针，不能是空指针
48	kernel.h	div64_s64	除法运算	完全兼容	不涉及
49	kernel.h	div64_u64	除法运算	完全兼容	不涉及
50	kernel.h	div64_u64_rem	除法运算	完全兼容	用户保证入参是合法指针，不能是空指针
51	kernel.h	do_div_imp	除法运算	完全兼容	不涉及
52	kernel.h	do_div_s64_imp	除法运算	完全兼容	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
53	kernel.h	ERR_PTR	错误码操作	完全兼容	不涉及
54	kernel.h	IS_ERR	错误码判断	完全兼容	不涉及
55	kernel.h	PTR_ERR	错误码操作	完全兼容	不涉及
56	list.h	LIST_HEAD_INIT	初始化双链表（入参非指针类型）	完全兼容	不涉及
57	list.h	LIST_HEAD	声明变量，并初始化一个链表头	完全兼容	不涉及
58	list.h	INIT_LIST_HEAD	初始化双链表（入参为指针类型）	不完全兼容，Huawei LiteOS为宏函数，linux早期版本是宏函数，4.9.37版本之后变更为inline函数	用户保证入参是合法指针，不能是空指针
59	list.h	list_add	在节点后面添加一个新节点	完全兼容	用户保证入参是合法指针，不能是空指针
60	list.h	list_add_tail	在节点前面添加一个新节点	完全兼容	用户保证入参是合法指针，不能是空指针
61	list.h	list_is_last	判断节点是否是双链表的尾节点	完全兼容	用户保证入参是合法指针，不能是空指针
62	list.h	list_empty	判断链表是否为空	完全兼容	用户保证入参是合法指针，不能是空指针
63	list.h	list_entry	通过链表的地址，去获取包含这个链表的结构体的地址	完全兼容	用户保证入参是合法指针，不能是空指针

序号	文件名	接口名	说明	兼容性	注意事项
64	list.h	list_first_entry	获取包含链表首个节点的结构体指针	完全兼容	用户保证入参是合法指针，不能是空指针
65	list.h	list_for_each	遍历链表	完全兼容	用户保证入参是合法指针，不能是空指针
66	list.h	list_for_each_entry	遍历双链表，获取包含该链表节点的结构体地址	完全兼容	用户保证入参是合法指针，不能是空指针
67	list.h	list_for_each_entry_reverse	反向遍历双链表，获取包含该链表节点的结构体地址	完全兼容	用户保证入参是合法指针，不能是空指针
68	list.h	list_for_each_entry_safe	遍历双链表，获取包含该链表节点的结构体地址，并存储当前节点的后继节点用于下一次的循环，当遍历的同时有删除当前节点的操作时，可以防止找不到后继节点	完全兼容	用户保证入参是合法指针，不能是空指针
69	list.h	list_for_each_safe	遍历双向链表，并存储当前节点的后继节点用于下一次的循环，当遍历的同时有删除当前节点的操作时，可以防止找不到后继节点	完全兼容	用户保证入参是合法指针，不能是空指针
70	list.h	list_del	从双链表上删除一个节点	完全兼容	用户保证入参是合法指针，不能是空指针
71	list.h	list_move	将节点从原链表上删除，并作为头节点插入到另一个链表	完全兼容	用户保证入参是合法指针，不能是空指针

序号	文件名	接口名	说明	兼容性	注意事项
72	list.h	list_del_init	删除节点，并重新初始化这个节点	完全兼容	用户保证入参是合法指针，不能是空指针
73	platform_device.h	platform_device_register	注册一个设备	完全兼容	不涉及
74	platform_device.h	platform_device_unregister	注销设备	完全兼容	不涉及
75	platform_driver.h	platform_driver_register	注册一个驱动	完全兼容	不涉及
76	platform_driver.h	platform_driver_unregister	注销驱动	不完全兼容，返回值不一样，详见API参考	不涉及
77	platform_device.h	platform_get_drvdata	获取驱动私有数据	完全兼容	不涉及
78	platform_device.h	platform_get_irq	获取设备中断号	完全兼容	不涉及
79	platform_device.h	platform_get_resource	获取设备资源	完全兼容	不涉及
80	platform_device.h	platform_ioremap_resource	获取设备内存基址	完全兼容	不涉及
81	platform_device.h	platform_set_drvdata	设置驱动私有数据	完全兼容	不涉及
82	rbtree.h	rb_erase	删除节点	完全兼容	不涉及
83	rbtree.h	rb_first	求第一个节点	完全兼容	不涉及
84	rbtree.h	rb_insert_color	新插入节点着色	完全兼容	不涉及
85	rbtree.h	rb_next	求后继节点	完全兼容	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
86	rbtree.h	rb_prev	求前驱节点	完全兼容	不涉及
87	rbtree.h	rb_replace_node	替换节点	完全兼容	不涉及
88	rtc.h	is_leap_year	是否是闰年	完全兼容	不涉及
89	rtc.h	rtc_time_to_tm	绝对时间转换为年月日分秒	完全兼容	不涉及
90	rtc.h	rtc_tm_to_time	年月日分秒转换为绝对时间	完全兼容	不涉及
91	rwsem.h	DECLARE_RWSEM	定义并初始化一个读写信号量	完全兼容	不涉及
92	rwsem.h	init_rwsem	初始化一个读写信号量	完全兼容	不涉及
93	rwsem.h	down_read	获取一个读信号量，如果获取不到则会睡眠等待	完全兼容	不涉及
94	rwsem.h	down_read_trylock	尝试得到读信号量，不会睡眠	完全兼容	不涉及
95	rwsem.h	down_write	获取一个写信号量，如果获取不到则会睡眠等待	完全兼容	不涉及
96	rwsem.h	down_write_trylock	尝试得到写信号量，不会睡眠	完全兼容	不涉及
97	rwsem.h	up_read	释放一个读信号量	完全兼容	不涉及
98	rwsem.h	up_write	释放一个写信号量	完全兼容	不涉及
99	rwsem.h	downgrade_write	将写信号量降级为读信号量	完全兼容	不涉及
100	scatterlist.h	sg_init_one	初始化集散序列	完全兼容	用户保证入参是合法指针，不能是空指针
101	scatterlist.h	sg_init_table	初始化集散序列	完全兼容	用户保证入参是合法指针，不能是空指针

序号	文件名	接口名	说明	兼容性	注意事项
102	scatterlist.h	sg_mark_end	标记集散序列结束	完全兼容	用户保证入参是合法指针，不能是空指针
103	scatterlist.h	sg_set_buf	设置集散序列	完全兼容	用户保证入参是合法指针，不能是空指针
104	sched.h	schedule_timeout	task延时#timeout长的时间，时间到达后唤醒。Timeout为0时，由于timer定时器的时间误差，实际会delay 1 tick	完全兼容	不涉及
105	semaphore.h	DEFINE_SEMAPHORE	根据入参定义一个struct semaphore类型的变量作为信号量，并初始化该信号量	完全兼容	不涉及
106	semaphore.h	sema_init	动态初始化一个信号量	不完全兼容，对于计数值#val<0或#val>1的，系统不再统一设置为0，不做初始化，有错误打印	不涉及
107	semaphore.h	down	获取信号量，如果获取不到则进入睡眠状态，且不可唤醒	不完全兼容，没有实现计数值>1的情况	不涉及
108	semaphore.h	down_interruptible	获取信号量，否则进入睡眠状态，等待信号量被释放后，激活该程	不完全兼容，没有实现计数值>1的情况；无signal概念；返回值不一样，详见API参考	不涉及
109	semaphore.h	down_trylock	试图获取信号量，否则立刻返回非零值，调用者不会睡眠	不完全兼容，没有实现计数值>1的情况	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
110	semaphore.h	up	释放一个信号量	不完全兼容，没有实现计数值>1的情况	不涉及
111	semaphore.h	sema_destory	销毁信号量，回收内核内存资源	非标准接口	不涉及
112	seq_file.h	seq_lseek	偏移seq流指针	完全兼容	不涉及
113	seq_file.h	seq_open	打开seq流	完全兼容	不涉及
114	seq_file.h	seq_printf	向seq流方式写格式化信息	完全兼容	不涉及
115	seq_file.h	seq_read	读seq流	完全兼容	不涉及
116	seq_file.h	seq_release	释放seq流所分配的动态内存空间	完全兼容	不涉及
117	seq_file.h	single_open	打开seq流	完全兼容	不涉及
118	seq_file.h	single_release	释放seq流所分配的动态内存空间	完全兼容	不涉及
119	slab.h	kfree	释放内存空间	完全兼容	不涉及
120	slab.h	kmalloc	分配内存空间	完全兼容	不涉及
121	slab.h	kzalloc	分配内存空间并初始化	完全兼容	不涉及
122	slab.h	vfree	释放内存空间	完全兼容	不涉及
123	slab.h	vmalloc	分配内存空间	完全兼容	不涉及
124	spinlock_types.h	DEFINE_SPINLOCK	根据入参定义一个spinlock_t类型的变量作为自旋锁，并初始化该自旋锁	完全兼容	不涉及
125	spinlock.h	spin_lock_init	动态初始化指定的自旋锁	完全兼容	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
126	spinlock.h	spin_lock	获取指定的自旋锁	完全兼容	不涉及
127	spinlock.h	spin_unlock	释放指定的自旋锁	完全兼容	不涉及
128	spinlock.h	spin_lock_irqsave	保存本地中断的当前状态，禁止本地中断，并获取指定的锁	完全兼容	不涉及
129	spinlock.h	spin_unlock_irqrestore	释放指定的锁，并让本地中断恢复到以前状态	完全兼容	不涉及
130	string.h	strncpy	拷贝字符串	完全兼容	不涉及
131	timer.h	add_timer	添加定时器	功能完全兼容，对参数 <i>timer->expires</i> 有范围限制，[0, 0xFFFFFFFF]	不涉及
132	timer.h	del_timer	删除定时器	完全兼容	不涉及
133	timer.h	del_timer_sync	删除定时器	完全兼容	不涉及
134	timer.h	init_timer	初始化定时器	完全兼容	不涉及
135	timer.h	mod_timer	更改定时器的到期时间	完全兼容	不涉及
136	wait.h	init_waitqueue_head	初始化一个已经存在的等待队列头	完全兼容	不涉及
137	wait.h	wait_event	等待事件	完全兼容	不涉及
138	wait.h	wait_event_interruptible	等待事件	不完全兼容，该事件不可被信号中断	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
139	wait.h	wait_event_interruptible_timeout	超时等待事件	不完全兼容，入参范围有限制，返回值有差异，详见API参考	不涉及
140	wait.h	waitqueue_active	检查等待队列是否为空	完全兼容	不涉及
141	wait.h	wake_up	唤醒任务	完全兼容	不涉及
142	wait.h	wake_up_interruptible	负责唤醒状态为TASK_INTERRUPTIBLE的任务	不完全兼容，无signal唤醒方式	不涉及
143	wait.h	wake_up_interruptible_poll	唤醒任务	完全兼容	不涉及
144	wait.h	DECLARE_WAIT_QUEUE_HEAD	定义并初始化一个等待队列头	完全兼容	不涉及
145	workqueue.h	cancel_delayed_work	取消挂到执行队列上的delayed work	完全兼容	不涉及
146	workqueue.h	cancel_delayed_work_sync	同步取消挂到执行队列上的delayed work	不完全兼容工作项非法、工作项正在执行或已执行时返回错误	不涉及
147	workqueue.h	cancel_work_sync	同步取消一个work并等待其执行完毕	不完全兼容，工作项非法、工作项已执行、工作项正在执行	不涉及
148	workqueue.h	create_singlethread_workqueue	创建工作队列	完全兼容	不涉及
149	workqueue.h	create_workqueue	创建工作队列	不完全兼容，Huawei LiteOS只在当前CPU上创建一个工作队列，而对于多CPU场景，linux会在每个CPU上都创建一个工作队列	不涉及

序号	文件名	接口名	说明	兼容性	注意事项
150	workqueue.h	destroy_workqueue	销毁工作队列	不完全兼容，有参数合法性检测	不涉及
151	workqueue.h	flush_delayed_work	等待一个delayed work执行完毕	完全兼容	不涉及
152	workqueue.h	flush_work	等待一个work执行完毕	完全兼容	不涉及
153	workqueue.h	INIT_DELAYED_WORK	初始化delay_work	完全兼容	不涉及
154	workqueue.h	INIT_WORK	初始化work	完全兼容	不涉及
155	workqueue.h	queue_delayed_work	往指定工作队列中插入一个delayed work	完全兼容	不涉及
156	workqueue.h	queue_work	往指定工作队列中插入一个work	不完全兼容，有参数合法性检测	不涉及
157	workqueue.h	schedule_delayed_work	往系统默认工作队列中插入一个delayed work	不完全兼容，有参数合法性检测	不涉及
158	workqueue.h	schedule_work	往系统默认工作队列中插入一个work	不完全兼容，有参数合法性检测	不涉及
159	workqueue.h	work_busy	检测work状态	不完全兼容，有参数合法性检测	不涉及

说明

完全兼容表示功能与linux一致，错误码返回值要以实际代码为准，部分兼容表示部分功能与linux一致。

6.6.7 linux 不支持接口概览

Huawei LiteOS的linux适配接口中，有一些未支持，包括但不限于下表所示。

头文件	接口名	说明	兼容性
bug.h	BUG	提供断言并输出信息	不支持
compiler.h	likely	判断语句，值为真可能性更大	不支持
compiler.h	unlikely	判断语句，值为假可能性更大	不支持
fb.h	fb_alloc_cmap	分配颜色表内存空间	不支持
fb.h	fb_cmap_to_user	复制颜色表	不支持
fb.h	fb_copy_cmap	复制颜色表	不支持
fb.h	fb_dealloc_cmap	释放颜色表内存空间	不支持
fb.h	fb_default_cmap	设置默认颜色表	不支持
fb.h	fb_pan_display	刷新操作屏	不支持
fb.h	fb_set_cmap	设定颜色表	不支持
fb.h	fb_set_user_cmap	设定颜色表	不支持
fb.h	fb_set_var	设置fbinfo显示模式和可变参数	不支持
fb.h	framebuffer_alloc	向内核申请一块空间	不支持
fb.h	framebuffer_release	向内核释放一块空间	不支持
fb.h	register_framebuffer	将fbinfo结构注册到内核	不支持
fb.h	unregister_framebuffer	注销fbinfo	不支持
kernel.h	copy_from_user	将用户空间的数据传送到内核空间	不支持
kernel.h	copy_to_user	从内核区中读取数据到用户区	不支持
kernel.h	ioremap_cached	内核虚拟地址映射	不支持
kernel.h	ioremap_nocache	内核虚拟地址映射	不支持
kernel.h	iounmap	取消ioremap函数所做的映射	不支持
module.h	module_put	减少模块使用计数	不支持
module.h	try_module_get	增加模块使用计数	不支持

头文件	接口名	说明	兼容性
pm.h	dpm_resume_end	resume所有非系统设备，即执行所有注册设备的resume回调函数	不支持
pm.h	dpm_suspend_start	suspend所有非系统设备，即调用所有注册设备的suspend回调函数	不支持
sched.h	cond_resched	调度一个新程序投入运行	不支持
sched.h	signal_pending	检查当前进程是否有信号处理	不支持
semaphore.h	init_MUTEX	初始化信号量值为1	不支持
semaphore.h	init_MUTEX_LOCKED	初始化信号量值为0	不支持
string.h	simple_strtol	将一个字符串转换成unsigend long long型数据	不支持
suspend.h	pm_resume	用于resume系统	不支持
suspend.h	pm_suspend	用于挂起系统	不支持
timer.h	timer_pending	判断一个处在定时器管理队列中的定时器对象是否已经被调度执行	不支持
wait.h	add_wait_queue	将进程插入队列尾部	不支持
wait.h	DECLARE_WAITQUEUE	定义并初始化一个等待队列	不支持
wait.h	remove_wait_queue	将等待队列wait从附属的等待队列头q指向的等待队列链表中移除	不支持
wakelock.h	wake_lock	激活锁	不支持
wakelock.h	wake_lock_active	判断锁当前是否有效	不支持
wakelock.h	wake_lock_init	初始化一个锁	不支持
wakelock.h	wake_unlock	解锁使之成为无效锁	不支持

7 驱动开发指导

本章以3556V200为例，使能开发者操作控制与ETH、USB2.0 DRD和SD/MMC/eMMC卡等驱动模块相连的Huawei LiteOS外围驱动设备。

说明

HI3556V200/HI3559V200无ETH模块。

[7.1 基于驱动框架的开发指导](#)

[7.2 适配文件操作的驱动开发](#)

[7.3 MMC驱动开发](#)

[7.4 USB驱动开发](#)

[7.5 UART驱动开发](#)

7.1 基于驱动框架的开发指导

7.1.1 概述

基本概念

如下图1所示，驱动开发就是按一定的规格实现硬件功能并进行抽象，提供给开发者一套统一的设备访问接口，方便开发者进行上层业务开发。

1. 驱动设备

系统将所有外部设备看成是一类特殊文件，称为“设备文件”。外部设备可分为三类：

- 字符设备：设备以字符形式发送和接收数据，不支持随机访问。
- 块设备：利用一块系统内存作为缓冲区，以整个数据缓冲区的形式传输数据。块设备主要针对Flash等慢速设备，能避免CPU耗费过多的时间等待操作完成。
- 网络设备：所有对网络硬件的访问都通过接口进行。接口提供一个对所有类型的网络硬件一体化的操作集合，来发送和接收基本数据。

2. 驱动程序

内核与外部设备之间的接口。驱动程序向应用程序屏蔽了硬件在实现上的细节。在应用程序看来，外部设备只是一个设备文件，使应用程序可以像操作普通文件一样来操作外部设备，如open ()、close ()、read ()、write () 等。

3. 驱动框架管理

对驱动设备和驱动程序进行系统管理的模块。

图 7-1 基本设备驱动框架



运作机制

驱动框架涉及到2个比较重要的数据结构**LosDevice**（用于描述驱动设备）和**LosDriver**（用于描述驱动程序）。系统中有两个全局双向链表，一个是全局设备链表，管理挂载的所有驱动设备，另一个是全局驱动链表，管理所有挂载的驱动程序。

这两种数据结构介绍如下。

1. LosDevice

LosDevice代表一个驱动设备。结构体中重要的成员包括：

- const CHAR *name：设备名，用以跟驱动配对。
- struct LosDriver *driver：表示该驱动设备的驱动程序。
- struct LosDeviceConfig cfg：设备io起始地址、io大小及中断号配置。
- LOS_DL_LIST driverNode：对应的驱动节点，用以挂载到该驱动结构体中的deviceList上。
- LOS_DL_LIST deviceItem：该驱动设备节点，用以挂载到全局设备链表中。

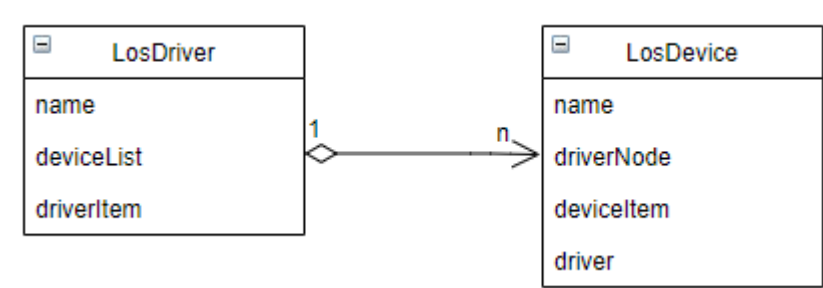
2. **LosDriver**

LosDriver代表一个设备驱动程序。结构体中重要的成员包括：

- const CHAR *name：驱动名，用以跟设备配对。
- LOS_DL_LIST deviceList：使用当前这个驱动程序的驱动设备的双向链表。
- LOS_DL_LIST driverItem：该驱动程序节点，用以挂载到全局驱动链表里。
- struct LosDriverOps ops：驱动实现的操作接口，如probe、remove等。

LosDevice和LosDriver之间的关系如下图2所示：

图 7-2 LosDevice 和 LosDriver 的关系图



LosDriver挂在全局驱动链表下，包含了一个deviceList链表，表示这个驱动程序操作（或控制）的所有设备。

LosDevice挂在全局设备链表下，包含了一个driver指针，表示这个设备对应的设备驱动程序。

一个驱动设备只对应一个驱动程序，而一个驱动程序可以支持多个驱动设备。当对应的设备/驱动挂载的时候，会根据名字去全局驱动链表/全局设备链表中寻找已经注册的对应的驱动/设备，执行注册过程。

7.1.2 开发指导

功能

Huawei LiteOS的驱动框架为用户提供下面几种功能，接口详细信息可以查看API参考。

功能分类	接口名	描述
操作驱动设备	LOS_DeviceRegister	注册一个设备，添加到全局设备链表
	LOS_DeviceUnregister	注销一个设备，从全局设备链表中移除
	LOS_DeviceDataGet	获取指定设备的私有数据
	LOS_DeviceRegBaseGet	获取指定设备的io起始地址

功能分类	接口名	描述
	LOS_DeviceRegSizeGet	获取指定设备的io大小
	LOS_DeviceIrqNumGet	获取指定设备的中断号
注册/注销驱动程序	LOS_DriverRegister	注册设备驱动程序，并和相应的设备绑定
	LOS_DriverUnregister	注销设备驱动程序
低功耗功能	LOS_PmSuspend	挂起所有设备
	LOS_PmResume	恢复所有设备

开发流程

下面以注册USB驱动为例，讲解基于驱动框架的驱动开发流程。

步骤1 打开菜单，选择Driver ---> Enable Driver Base，使能驱动框架。

配置项	含义	取值范围	默认值	依赖
LOSCFG_DRIVERS_BASE	使能驱动框架的开关	YES/NO	YES	无

步骤2 注册驱动设备。

1. 定义USB驱动设备实例。

```
struct LosDeviceRegs hiudc3_regs[] = {
{
    .base = DWC_USB3_PORT1_BASE_ADDR,
    .size = DWC_USB3_PORT1_ADDR_OFFSET,
},
};
static struct LosDevice hiudc3_device = {
    .name      = "hi_udc3",
    .id        = -1,
    .cfg       = {

        .irqNum = NUM_HAL_INTERRUPT_USB_DEV,

        .numRegs = ARRAY_SIZE(hiudc3_regs),
        .regs = hiudc3_regs,
    },
};
```
2. 将驱动设备注册到全局设备链表。

```
LOS_DeviceRegister(&hiudc3_device);
```
3. 预加载处理。

通过调用linux/module.h（在根目录compat/linux/include目录下）里面的预加载挂载函数，将驱动设备注册函数挂载到系统中。当系统启动时，系统会按照挂载函数优先级遍历预加载节点，从而注册驱动设备。系统提供如下预加载挂载函数，优先级从高到低：

```
pure_initcall(f)    //0
core_initcall(f)    //1
postcore_initcall(f) //2
arch_initcall(f)    //3
subsys_initcall(f)  //4
fs_initcall(f)      //5
device_initcall(f)  //6
late_initcall(f)    //7
```

具体调用请参见[编程实例](#)。

须知

执行预加载可能会影响到系统的某些功能，例如，当驱动程序初始化耗时较长时，会对快速启动（分散加载）的启动时间造成冲击。目前系统只支持优先级小于等于3的预加载函数。

步骤3 注册设备驱动程序。

1. 定义USB驱动程序实例。

```
static struct LosDriver hiudc3_driver = {
    .name = "hi_udc3",
    .ops = {
        .probe = hiudc3_probe,
        .remove = hiudc3_remove,
    },
    .pmOps = {
        .suspend = hiudc3_suspend,
        .resume = hiudc3_resume,
    },
};
```

参数描述如下表1所示：

表 7-1 参数描述

参数	描述
probe	该函数参数一般用于初始化驱动程序
remove	该函数参数一般用于去初始化驱动程序
name	驱动程序的属性name必须和对应驱动设备的name一致
suspend/resume	用于挂起/恢复驱动设备

2. 注册USB驱动程序。

```
LOS_DriverRegister(&hiudc3_driver);
```

步骤4 将加载的驱动注册到文件系统。

为方便用户使用，将驱动实例注册到文件系统，使对驱动的操作抽象为对文件的操作。具体实现请参见[适配文件操作的驱动开发](#)。

----结束

7.1.3 注意事项

调用预加载处理的驱动函数需要在build/mk/liteos_tables_ldflags.mk中的LITEOS_TABLES_DRIVER_LDFLAGS添加-u声明，否则可能会导致链接器优化掉驱动初始化模块，造成不可预估的错误。

```
LITEOS_TABLES_DRIVER_LDFLAGS := \  
-ui2c_init \  
-ugpio_init \  
-uregulator_init \  
-umtd_init_list \  

```

7.1.4 编程实例

实例描述

添加USB驱动。

编程实例

前提条件：在menuconfig菜单中使能驱动框架。

步骤1 在指定开发板的board.c目录下定义驱动设备。

```
/* 定义驱动设备实例 */  
#ifdef LOSCFG_DRIVERS_USB3_DEVICE_CONTROLLER  
struct LosDeviceRegs hiudc3_regs[] = {  
    {  
        .base = DWC_USB3_PORT1_BASE_ADDR,  
        .size = DWC_USB3_PORT1_ADDR_OFFSET,  
    },  
};  
  
static struct LosDevice hiudc3_device = {  
    .name = "hi_udc3",  
    .id = -1,  
    .cfg = {  
        .irqNum = NUM_HAL_INTERRUPT_USB_DEV,  
        .numRegs = ARRAY_SIZE(hiudc3_regs),  
        .regs = hiudc3_regs,  
    },  
};  
#endif  
  
int machine_init(void)  
{  
    #ifdef LOSCFG_DRIVERS_USB3_DEVICE_CONTROLLER  
        (void)LOS_DeviceRegister(&hiudc3_device);  
    #endif  
}  
arch_initcall(machine_init); //预加载驱动设备注册。内核初始化执行时，会预加载驱动设备注册
```

步骤2 在drivers\usb\adapt_liteos\controller\usb_device目录下定义USB设备驱动程序。

```
/* 驱动程序实现的操作接口 */  
static int hiudc3_probe(struct LosDevice *pdev)  
{  
    .....  
}  
  
static int hiudc3_remove(struct LosDevice *pdev)  
{  
    .....  
}  
  
static int hiudc3_suspend(struct LosDevice *pst_dev)
```



```
{
    .....
}

static int hiudc3_resume(struct LosDevice *pst_dev)
{
    .....
}

/* 定义设备驱动程序实例 */
static struct LosDriver hiudc3_driver = {
    .name = "hi_udc3",
    .ops = {
        .probe = hiudc3_probe,
        .remove = hiudc3_remove,
    },
    .pmOps = {
        .suspend = hiudc3_suspend,
        .resume = hiudc3_resume,
    },
};

/* 初始化驱动程序，即注册驱动程序 */
int hiudc3_init(void)
{
    return (int)LOS_DriverRegister(&hiudc3_driver);
}

/* 注销驱动程序 */
int hiudc3_exit(void)
{
    return (int)LOS_DriverUnregister(&hiudc3_driver);
}
```

----结束

7.2 适配文件操作的驱动开发

7.2.1 概述

基本概念

驱动开发就是把硬件的功能按操作系统的规格实现并抽象出来，提供给应用程序开发人员调用。

当在新的芯片上移植系统时，需要根据该芯片特性对支持的外部设备进行驱动开发。

Huawei LiteOS的驱动初始化函数主要初始化设备对应的驱动结构，提供给上层应用来注册设备驱动的控制节点。

7.2.2 开发指导

开发流程

推荐驱动开发人员使用VFS框架来注册/卸载设备驱动，基于文件系统的驱动开发主要涉及下面几步：

1. 打开菜单，选择FileSystem ---> Enable VFS，使能VFS。

配置项	含义	取值范围	默认值	依赖
LOSCFG_FS_VFS	VFS框架的裁剪开关	YES/NO	YES	无

- 2. 初始化驱动。
- 3. 操作驱动节点。

初始化驱动

在Huawei LiteOS上进行驱动开发的第一步，是编写驱动初始化函数。驱动初始化函数用于初始化设备所需的驱动结构，以及生成驱动的控制节点。

驱动初始化函数中需要调用设备驱动注册函数，用于注册并生成驱动节点。

```
register_driver(FAR const char *path, FAR const struct file_operations_vfs *fops, mode_t mode, FAR void *priv)
register_blockdriver(FAR const char *path, FAR const struct block_operations *bops, mode_t mode, FAR void *priv)
```

参数描述如下表1所示：

表 7-2 参数描述

参数	描述
path	驱动节点路径，应用程序可通过该路径访问到驱动节点，进而访问设备驱动提供的操作接口
fops/bops	驱动操作结构体，为应用程序提供操作函数集。fops和bops分别表示字符设备和块设备的驱动操作集
mode	读写该驱动节点的权限，暂时无效，后续或提供支持
priv	驱动节点注册过程需要传入的参数

编写完驱动初始化函数后，需要在适当的地方引导该初始化函数执行。简单的引导可以参考开发包中sample/sample_osdrv/sample.c的代码。用户可以在app_init函数里调用已编写好的驱动初始化函数引导设备初始化。

操作驱动节点

驱动初始化后，生成的设备驱动节点为应用提供操作设备的接口，下面以i2c设备驱动程序为例，说明用户程序与驱动操作函数的调用关系。

操作函数集

驱动操作函数集对于应用程序与驱动操作函数的调用关系非常重要。在编写驱动程序时，操作函数集需要实现硬件设备的各项机制，并在设备驱动注册时传入。操作函数集会成为应用程序需求的最终实现。

i2c设备驱动提供的操作函数集如下表2所示：

表 7-3 i2c 设备驱动的操作函数集

操作函数集	对应的应用层接口
i2cdev_open	open
i2cdev_release	close
i2cdev_read	read
i2cdev_write	write
i2cdev_ioctl	ioctl

• **open操作**

应用程序打开节点文件时，系统最终会在该驱动节点注册过程中调用驱动操作函数集中的open函数。

open函数对目标驱动设备函数结构体先进行实例化，再进行必要的初始化。在成功打开节点文件后，应用程序能够获取到对应驱动节点的文件描述符，并通过该文件描述符对驱动程序进行访问。

• **read/write操作**

read/write操作是常用的访问设备的接口。不同类型的设备与驱动提供的read/write操作的功能互有差异。对于i2c设备，应用程序通过调用read/write接口可以实现对i2c设备进行读写。

```
i2cdev_read(struct file * filep, char __user *buf, size_t count)
i2cdev_write(struct file * filep, const char __user *buf, size_t count)
```

参数描述如下表3所示：

表 7-4 参数描述

参数	描述
filep	文件描述结构体指针
buf	读出/写入数据的缓冲区指针
count	读出/写入数据的长度

• **ioctl操作**

ioctl操作提供对驱动设备函数的配置管理。通过执行相应的命令，完成对设备属性的配置或访问。

i2c设备中，使用命令I2C_16BIT_REG、I2C_16BIT_DATA与I2C_TIMEOUT分别对应设置i2c设备的传输寄存器位宽、传输数据位宽与超时时间。

```
i2cdev_ioctl(struct file *filep, int cmd, unsigned long arg)
```

参数描述如下表4所示：

表 7-5 参数描述

参数	描述
filep	文件描述结构体指针
cmd	操作命令
arg	附加参数

- close操作**
 close操作对应着驱动操作函数集里的release函数。release函数对驱动程序的资源进行释放。

7.2.3 注意事项

无。

7.3 MMC 驱动开发

7.3.1 概述

MMC（Multi Media Card）是 MMC协会订立、主要针对手机或平板电脑等产品的内嵌式存储器标准规格。MMC驱动用于处理对SD存储卡（Secure Digital Memory Card）和EMMC卡（Embedded Multi Media Card）的识别和读写等操作，并通过SDIO协议支持扩展外设（如蓝牙、WiFi、GPS等）。

Huawei LiteOS MMC驱动支持的设备包括TF卡、EMMC存储器和WiFi扩展外设。

- TF（T-Flash）卡，又叫micro SD卡
 TF卡主要通过挂载为FAT文件系统实现对设备的读写操作，挂载之前首先需要完成设备的识别，也可以通过裸读写接口对TF卡进行裸读写。

须知

目前Huawei LiteOS不支持用户自定义TF卡分区。如有需要，用户需提前在PC端对TF卡做好分区操作。

- EMMC存储器**
 EMMC存储器主要通过挂载为FAT文件系统实现对设备的读写操作，挂载之前需要先调用接口自定义分区，然后进行设备的识别，也可以通过裸读写接口对EMMC存储器进行裸读写。
- SDIO扩展外设**
 目前支持的外围设备只有WiFi模组。设备通过WiFi模组进行通信和数据传输，设备识别前需要根据硬件要求做好管脚复用配置。

Huawei LiteOS MMC驱动模块包括协议层和控制层：

- 协议层支持SD、EMMC、SDIO协议规范。

- 控制层支持Himci和Sdhci控制器，不同的芯片平台根据硬件IP选择对应的控制器驱动。

7.3.2 开发指导

MMC 设备识别

Huawei LiteOS中的MMC设备分为嵌入式和非嵌入式两类：

- 嵌入式设备包括EMMC存储器和SDIO扩展外设，均不支持带电插拔。EMMC设备目前仅支持首次识别，不支持多次识别；SDIO扩展外设可以调用hisi_sdio_rescan接口实现对设备的卸载和加载识别（依赖于扩展外设的软件上下电）。
- 非嵌入式设备为TF卡。TF卡支持热插拔，通过线程轮询方式实现设备的动态卸载和加载。

须知

除SDIO扩展外设可以调用sdio_rescan接口实现设备识别外，其他MMC设备的识别均在同一线程内实现，当存在多个MMC设备时，将根据控制器ID的顺序串行识别设备。

驱动初始化接口

接口名	描述
SD_MMC_Host_init	遍历平台所有控制器并进行初始化
MMC_HostInitById	初始化平台指定控制器，控制器由入参ID确定（可减少MMC驱动初始化的整体耗时）

须知

MMC驱动初始化成功后，如果系统支持proc文件系统，会生成proc/mci/mci_info节点，通过cat proc/mci/mci_info可查看设备信息。此两个初始化接口不可混用，且MMC_HostInitById调用需保证单线程调用或者时间上互斥。

添加 EMMC 分区接口

接口名	接口参数	参数描述
add_mmc_partition	struct disk_divide_info *info	入参，是一个设备信息结构体，这里表示EMMC的分区信息，当前只支持传入全局变量struct disk_divide_info emmc
	size_t sectorStart	入参，表示分区起始扇区
	size_t sectorCount	入参，表示分区扇区数

```
设备信息结构体定义如下所示：
#define MAX_DIVIDE_PART_PER_DISK 16      // 每个设备可支持的最大逻辑分区个数
#define MAX_PRIMARY_PART_PER_DISK 4      // 每个设备可支持的最大主分区个数

struct disk_divide_info{
    UINT64  sector_count;                // 分区块个数
    UINT32  part_count;                  // 分区号
    UINT32  sector_size;                 // 分区大小
    struct partition_info part[MAX_DIVIDE_PART_PER_DISK + MAX_PRIMARY_PART_PER_DISK]; // 分区信息结构体
};
```

裸读写接口

- TF卡裸读写接口说明

接口名	接口参数	参数描述
mmc_direct_write mmc_direct_read	uint32_t host_idx	入参，表示TF卡所在的控制器编号
	char * buffer	入参，表示读写数据保存的buffer地址（以CACHE_ALIGNED_SIZE对齐）
	uint32_t start_sector	入参，表示读写的起始块地址
	uint32_t nsectors	入参，表示读写的块数量

- emmc裸读写接口说明

接口名	接口参数	参数描述
emmc_raw_write emmc_raw_read	char * buffer	入参，表示读写数据保存的buffer地址（以CACHE_ALIGNED_SIZE对齐）
	uint32_t start_sector	入参，表示读写的起始块
	uint32_t nsectors	入参，表示读写的块数量

- MMC异常（中断）上下文读写接口说明

接口名	接口参数	参数描述
mmc_write_in_exception mmc_read_in_exception	uint32_t host_idx	入参，表示mmc设备所在的控制器编号（支持TF卡与emmc）
	char * buffer	入参，表示读写数据保存的buffer地址（以CACHE_ALIGNED_SIZE对齐）
	uint32_t start_sector	入参，表示读写的起始块地址

接口名	接口参数	参数描述
	uint32_t nsectors	入参，表示读写的块数量

须知

MMC异常（中断）上下文读写接口支持TF卡与emmc但不支持SDIO设备，不推荐在非异常（中断）中调用，在调用前用户需保证系统不处于MMC逻辑执行流程中，否则可能会导致接口读写失败。

SDIO 接口

Huawei LiteOS提供了一套完整的SDIO对外接口，包括发送命令，收发数据等。SDIO扩展外设通过调用这套接口实现对扩展外设的操作。

接口名	描述
sdio_get_func	获得一个sdio_func，此sdio_func在SDIO设备识别后产生，通过func号、厂商ID、设备ID（此三个参数根据SDIO外设获得）唯一标识
sdio_en_func	使能，该接口通过CMD52命令写0x2地址实现
sdio_dis_func	禁止指定的sdio_func，该接口通过CMD52命令写0x2地址实现
sdio_require_irq	给指定的sdio_func注册SDIO中断处理函数，该中断处理函数需要用户自行实现，函数类型定义为：void (sdio_irq_handler_t)(struct sdio_func *)
sdio_release_irq	释放指定sdio_func中注册的SDIO中断处理函数
sdio_read_byte	从指定sdio_func的指定地址addr读取一字节数据，并返回读取的数据
sdio_read_byte_ext	从指定sdio_func的指定地址addr读取一字节数据，添加入参in到CMD参数中（暂不支持），并返回读取的数据
sdio_read_incr_block	从指定sdio_func的指定地址addr读取指定长度的数据到内存指定地址dst中，读取时设备地址会依次增长（比如设备的内存地址） 该接口通过CMD53命令实现
sdio_read_fifo_block	从指定sdio_func的指定地址addr读取指定长度的数据到内存指定地址dst中，读取时设备地址固定不变（比如设备的FIFO） 该接口通过CMD53命令实现

接口名	描述
sdio_write_incr_block	将指定地址src中size长度的数据写入指定sdio_func的指定地址addr中，写入时设备地址会依次增长（比如设备的内存地址） 该接口通过CMD53命令实现
sdio_write_fifo_block	将指定地址src中size长度的数据写入指定sdio_func的指定地址addr中，写入时设备地址固定不变（比如设备的FIFO） 该接口通过CMD53命令实现
sdio_write_byte	将一字节数据写入指定sdio_func的指定地址addr，该接口通过CMD52命令实现
sdio_write_byte_raw	将一字节数据写入指定sdio_func的指定地址addr，写完后读回（设置RAW Flag），该接口通过CMD52命令实现
sdio_func0_read_byte	从设备指定地址addr读取一字节数据，并返回读取的数据，该接口通过CMD52命令读取func0实现
sdio_func0_write_byte	写入一字节数据到设备指定地址addr，该接口通过CMD52命令写入func0实现
sdio_set_cur_blk_size	配置SDIO当前块大小，此块大小不应大于512
sdio_rescan	SDIO设备重新识别，实际执行操作为设备卸载和加载
sdio_reset_comm	reset SDIO设备

Huawei LiteOS基于控制器能力，提供数据的离散模式传输，并提供了一套专门的读写接口。SDIO扩展外设通过调用这套接口实现离散传输，即将多块buffer数据一次性发送或者一次性读取数据到多块buffer，以减少多块数据传输场景下的数据拷贝，提升数据传输性能。

接口名	描述
sdio_readv_incr_block	从指定sdio_func的指定地址addr读取数据，读取的数据长度和保存地址由入参struct mmc_sg *sg指定，sg中的数据个数由入参sg_nums决定，读取时设备地址会依次增长（比如设备的内存地址） 该接口通过CMD53命令实现
sdio_readv_fifo_block	从指定sdio_func的指定地址addr读取数据，读取的数据长度和保存地址由入参struct mmc_sg *sg指定，sg中的数据个数由入参sg_nums决定，读取时设备地址固定不变（比如设备的FIFO） 该接口通过CMD53命令实现

接口名	描述
sdio_writew_incr_block	将入参struct mmc_sg *sg中的数据写入指定sdio_func的指定地址addr, sg中提供数据地址和长度, sg中的数据个数由入参sg_nums决定, 写入时设备地址会依次增长(比如设备的内存地址) 该接口通过CMD53命令实现
sdio_writew_fifo_block	将入参struct mmc_sg *sg中的数据写入指定sdio_func的指定地址addr, sg中提供数据地址和长度, sg中的数据个数由入参sg_nums决定, 写入时设备地址固定不变(比如设备的FIFO) 该接口通过CMD53命令实现

mmc_sg结构体如下所示:

```
struct mmc_sg {
    size_t length; // 数据长度, 不能大于buffer的内存空间大小
    void *data;    // buffer地址, 必须cache line对齐, 其指向的buffer内存空间大小也必须cache line对齐
};
```

EMMC 存储器开发流程

1. 添加EMMC分区


```
size_t part0_start_sector = 16 * (0x100000/512);
size_t part0_count_sector = 1024 * (0x100000/512);
extern struct disk_divide_info emmc;
add_mmc_partition(&emmc, part0_start_sector, part0_count_sector);
```
2. 初始化驱动


```
MMC_HostInitById(emmc_host_id);
```

说明

emmc_host_id为EMMC存储器对应的控制器编号。

emmc介质识别成功后会生成/dev/mmcblkPy 的节点, 供文件系统挂载。(x是设备ID, 一般是0或者1, y是分区ID, 由设备分区个数决定, 从0开始)。

3. 设备识别后, 读写设备
 - a. 裸读写

调用EMMC裸读写接口进行读写。
 - b. 挂载为FAT文件系统进行读写


```
mount("/dev/mmcblk0p0", "/sd", "vfat", 0, NULL);
```

挂载后可通过文件系统标准接口对设备进行读写等操作。

说明

请根据实际情况传入/dev/mmcblk0p0为EMMC存储器被识别后注册的设备名。

TF 卡开发流程

1. 初始化驱动


```
MMC_HostInitById(sd_host_id);
```

说明

sd_host_id为TF卡对应的控制器编号。插入sd卡并识别成功后会生成/dev/mmcblkxPy的节点，供文件系统挂载。其中，x是设备ID，一般是0或者1，y是分区ID，由设备分区个数决定，从0开始。

2. 设备识别后，读写设备

a. 裸读写

调用TF卡裸读写接口进行读写。

b. 挂载为FAT文件系统读写

```
mount("/dev/mmcblk0p0", "/sd", "vfat", 0, NULL);
```

挂载后可通过文件系统标准接口对介质进行读写等操作。

说明

/dev/mmcblk0p0为TF卡被识别后注册的设备名，请根据实际情况传入。

SDIO 扩展外设开发流程

1. 管脚复用

配置IO寄存器，配置对应SDIO外围设备的CLK、CMD、DATA0、DATA1、DATA2、DATA3管脚寄存器。

2. 初始化驱动

```
MMC_HostInitById(sdio_host_id);
```

说明

sdio_host_id为SDIO扩展外设对应的控制器编号。sdio wifi设备识别成功后会生成/dev/sdio0的节点。

3. 设备识别后获取func并使能func

```
struct sdio_func* func;
func = sdio_get_func(func_num, manf_id, device_id);
sdio_en_func(func);
```

4. 设置当前block size

```
sdio_set_cur_blk_size(func, blk_size);
```

5. 注册中断处理函数

```
void handler_sample(struct sdio_func *func)
{
    dprintf( "%s,%d func:0x%x\n", __FUNCTION__, __LINE__, func);
    return;
}
sdio_require_irq(func, handler_sample);
```

后续就可以通过SDIO对外读写接口进行读写等操作了。

7.3.3 注意事项

- 正常操作过程中需要遵守的事项：
 - 保证卡的金属片与卡槽硬件接触充分良好（如果接触不好，会出现检测错误或读写数据错误），测试薄的MMC卡时，必要时可以用手按住卡槽的通讯端测试。
 - 读写过程中异常拔卡，驱动会直接卸载设备节点。在该场景下，用户必须释放正在操作的资源（比如打开的文件、目录等资源），并且对挂载点进行umount操作。否则，会造成资源泄漏或挂载点不能再次挂载。
 - SDIO外设业务调用sdio_require_irq注册中断后，在设备卸载前必须保证调用sdio_release_irq接口释放中断和相关资源，否则可能导致系统异常。

- EMMC存储器进行驱动初始化前必须先进行添加分区，否则会导致设备识别时无法获取分区，设备识别后不支持动态添加和删除分区。
- EMMC存储器挂载为FAT文件系统前需将dev设备格式化，否则可能导致挂载失败。
- SDIO扩展外设进行驱动初始化前必须先进行管脚复用配置，否则会导致设备无法识别。
- 被禁止的操作：
 - 读写SD卡时不要拔卡，否则会打印一些异常信息，并且可能会导致卡中文件或文件系统被破坏。
 - 格式化SD卡时禁止拔卡，否则可能会造成SD卡永久性损坏。
 - 上电状态下严禁拔掉SDIO扩展外设，否则可能会造成SDIO扩展外设或者硬件单板损坏。
- 不同平台控制器对应的介质可能不同，关于控制器编号与所支持的介质类型，请参考各芯片平台用户指南文档中关于EMMC/SD/SDIO控制器的章节。

7.4 USB 驱动开发

7.4.1 概述

USB，是英文Universal Serial Bus（通用串行总线）的缩写，是一个外部总线标准，用于规范电脑与外部设备的连接和通讯。Huawei LiteOS USB驱动主要包含协议层、Composite层和控制器驱动三部分。

1. 协议层

协议层是设备具体实现层，支持上层业务通过USB口进行数据收发。

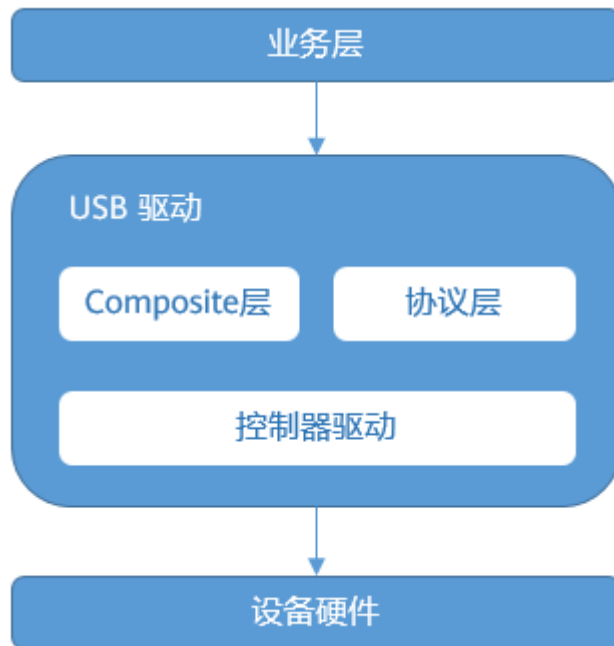
2. Composite层

Composite（复合设备）层连接协议层与控制器驱动，支持具体协议的加载、卸载及设备枚举过程。Composite作为虚拟设备，其他USB设备都需要在挂载Composite设备下面。由于Composite层封装了与控制端口交互的处理流程，USB设备只需要向Composite层注册相应的处理函数，具体协议的加载和设备枚举过程由Composite层执行。

3. 控制器驱动

控制器驱动与硬件交互，主要支持数据收发和端口控制。

图 7-3 USB 驱动基本框架



运作机制

USB协议驱动加载和数据收发主要流程如下：

1. USB驱动加载

USB设备驱动的加载通过Composite层封装统一的处理流程，由Composite调用具体协议的加载函数进行驱动程序的加载。待设备与主机通过USB线建立连接后，Composite层统一处理与主机之间的枚举过程，通过协议层注册的处理函数，进行端口申请和协议私有数据初始化。

2. USB数据收发

USB设备加载成功后，业务层通过协议层进行数据收发，协议层直接调用控制器驱动提供的端口控制接口进行端口数据收发处理。

7.4.2 开发指导

7.4.2.1 功能

功能

Hi35xx的USB功能支持Host和Device两种模式，具体协议及功能参见[表1 Host模式支持的功能](#)和[表2 Device模式支持的功能](#)。

表 7-6 Host 模式支持的功能

功能	描述	使用说明
Host U盘	支持对接U盘，进行U盘的读写访问等操作。	• 支持CD-ROM U盘 • 支持多个U盘 • 支持读卡器+SD卡热插拔 • 支持超2T硬盘 • 支持hub集线器，默认最多5个U盘设备
USB转网口	支持对接USB转网口模块，实现网络数据通过USB口的收发。	• 支持的USB转网口型号：Ax88772B、Ax88772C • 不支持设置网卡MAC地址 • 不支持二层套接字设置
Host-RNDIS	支持直接通过USB线进行网络数据的收发。	• 不支持设置网卡MAC地址 • 不支持二层套接字设置
4G-Modem	支持对接USB-4G模块。	• 支持的4G模块型号：ME909s-120、ME909s-821 4G、移远EC20/EC200T • 支持AT指令 • 只支持ECM网卡模式 • 不支持设置网卡MAC地址 • 不支持二层套接字设置 • 编译时需要同时选择：USB转串口、4G_Modem
USB-WIFI	支持对接USB-WIFI模块。	支持的USB WIFI型号：rtl8188eus、mt7601u
HOST-UVC	支持对接UVC设备。	• 支持ISOC模式传输 • 支持YUYV和MJPEG格式码流 • 不支持同时对接2个及以上UVC设备 • 不支持UVC扩展命令
HOST-HID	支持对接HID类的游戏手柄。	• 支持的游戏手柄型号：Xbox 360 • 支持同时对接2个游戏手柄

表 7-7 Device 模式支持的功能

功能	描述	使用说明
Device U盘	支持将挂载到系统的SD卡/MMC介质当做U盘，供USB Host端进行读写访问等操作。	- 支持SD卡虚拟成U盘 - 支持emmc和ram介质虚拟成U盘 - 支持emmc介质指定分区挂载识别 - 支持PC上右键盘符对U盘进行格式化、弹出等操作
UVC(USB Video Class)	支持通过USB口进行视频数据的发送。	- 支持安卓机顶盒UVC扩展命令 - 支持的视频规格： 格式：H.265/H.264/MJPEG/YUYV/NV12/NV21 分辨率@帧率：1920x1080@30fps、1280x720@30fps、640x480@30fps、640x360@30fps、352x288@30fps、320x240@30fps、176x144@30fps
UAC(USB Audio Class)	支持通过USB口进行音频数据的发送和接收。	- 不支持USB3.0 - 支持的PCM规格：48K、32K、16K、8K - 支持麦克风功能和扬声器功能 - 支持UAC1.0和UAC2.0
UAV(UVC&UAC)	支持通过USB口进行音视频数据的发送。	- 不支持USB3.0 - 支持的规格参见UVC、UAC规格 - 不支持UAC1.0扬声器功能和UAC2.0
DFU(Device Firmware Upgrade)	支持通过USB口对固件的升级，对接Host端DFU协议。	- 不支持USB3.0 - 支持PC升级 - 支持安卓机顶盒升级 - 支持升级状态查询
HID(Human Interface Device)	支持通过USB口对HID设备的数据进行接收和发送。	- 不支持USB3.0 - 不支持启动接口和启动协议 - 支持USB鼠标、键盘和带有鼠标功能的键盘设备功能 - 支持报告描述符动态可配置
虚拟网口	支持通过USB口进行网络数据的收发，模拟网口的功能。	- 不支持二层套接字设置 - 支持RNDIS协议网络传输
虚拟串口	支持通过USB口进行串行通信，模拟串口的功能。	支持调测功能

功能	描述	使用说明
复合设备 (虚拟网口&虚拟串口)	支持基于USB口同时模拟网口和串口的功能。	- 支持调测功能 - 不支持二层套接字设置

说明

- USB1.1支持OHCI，USB2.0 Host支持EHCI，USB3.0 Host支持XHCI，分别对应全速USB设备、高速USB设备和超高速USB设备的识别。
- USB3.0 Device对接USB2.0 Host时，由于设备侧硬件限制不支持降速，无法识别。

Huawei LiteOS USB驱动由于受芯片硬件资源限制，支持的规格也不尽相同，详细请参见[表3 Host 模式规格](#)和[表4 Device 模式规格](#)所示。

表 7-8 Host 模式规格

芯片平台	U 盘	4G mode m	USB转网口	Host-RNDIS	USB-WIFI	HOST-UVC	HOST-HID
Hi3516DV300 (USB2.0)	√	√	√	√	√	√	√

表 7-9 Device 模式规格

芯片平台	U 盘	UVC	UAC	UAV	DFU	HID	虚拟串口	虚拟网口	复合设备
Hi3516DV300 (USB2.0)	√	√	√	√	√	√	√	√	√

说明

Hi3516DV300/Hi3556V200/Hi3559V200的USB2.0口，是基于USB3.0的控制器操作的，编译时需要选择3.0的控制器。

7.4.2.2 配置 USB 模块的内存

USB模块的正常运行需要一段uncached内存，所以需要单独为USB模块配置内存。

配置内存可通过定义以下函数实现：

```
void board_config(void)
{
    g_sys_mem_addr_end = 0x88000000;
    g_usb_mem_addr_start = g_sys_mem_addr_end;
```

```
g_usb_mem_size = 0x100000;  
}
```

须知

- 不可以修改void board_config(void)的声明，若应用层不定义void board_config(void)函数，Huawei LiteOS会按默认值分配OS可见内存大小。建议根据实际情况自行定义。
- g_sys_mem_addr_end配置的是OS可见内存的结束位置。
- g_usb_mem_addr_start配置的是USB内存的起始位置。
- g_usb_mem_size配置的是USB内存的大小。
- 若系统完全不支持USB相关功能，则可以将g_usb_mem_addr_start和g_usb_mem_size配置为0。

7.4.2.3 动态切换 USB 协议

USB驱动支持Device和Host协议之间的动态切换。动态切换时，先调用usb_deinit卸载当前使用的协议，再通过usb_init重新加载待切换的协议。

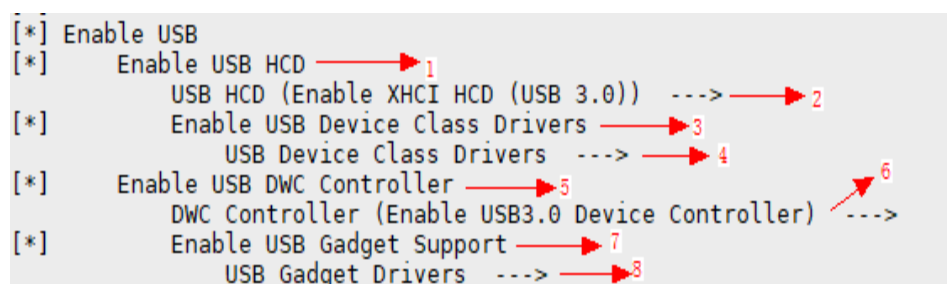
- 支持Host模式和Device模式之间的切换。
- 部分协议使用中不支持卸载（如：DFU、UVC、Camera、HID），需要先停用当前业务后再调用usb_deinit卸载。

7.4.2.4 模块编译

USB驱动的源码路径为drivers/usb。用户在Huawei LiteOS根目录下执行make即可编译出对应的USB模块库。编译成功后，会在out/<platform>/lib目录下生成名为libusb_base.a、libusb_device.a的库文件，目录中的platform为具体的平台名。

下图是Huawei LiteOS中USB驱动menuconfig配置项截图，在“Driver”配置项下，可以通过选中“Enable USB”开启USB驱动功能。

图 7-4 USB 驱动 menuconfig 配置



Host 端

配置项上[图1 USB驱动menuconfig配置](#)所示的1~4，对应描述如下：

- 1 表示是否打开USB驱动Host功能。
- 2 表示Host控制器种类，分别有2.0和3.0控制器（可视具体板子的USB口类别而定）。

- 3 表示是否开启Host模式下的USB协议功能。
- 4 表示Host模式下的具体USB协议功能，支持的USB协议如下，menuconfig配置项如下图2 USB驱动Host端配置所示：
 - Host U盘
 - Host RNDIS
 - 4G Modem
 - USB转串口
 - USB转网口
 - USB-WIFI
 - HOST-UVC
 - HOST-HID

图 7-5 USB 驱动 Host 端配置

```
[*] Enable USB Mass Storage Support
[*] Enable USB Rndis Host Support
[*] Enable USB 4G Modem Support
[*] Enable USB Serial Converter Support
[*] Enable USB Ethernet Support
[*] Enable USB Wireless Device Support
[*] Enable USB Host UVC Support
    Host UVC Source (Self-developed) --->
[*] Enable USB Host HID Support
```

须知

初始化Host模式协议可以调用usb_init(HOST, 0)，根据业务需要选择协议，也可以全部选择，协议栈会根据插入设备类型自动加载对应驱动，但是依赖的外围组件需要提前初始化，具体参见Host端的编程实例。

Device 端

配置项如为图1 USB驱动menuconfig配置所示的5~8，对应描述如下：

- 5表示是否打开USB驱动Device功能。
- 6表示Device控制器种类，分别有Dwc2.0和3.0（可视具体板子的 USB 口类别而定）。
- 7表示是否开启Device模式下的USB协议功能。
- 8表示Device模式下的具体USB协议功能，支持的USB协议如下，menuconfig配置项如下图3 USB驱动Device端配置所示：
 - Device U盘
 - UVC
 - UAC
 - UAV (UVC&UAC)
 - DFU升级
 - 虚拟串口

- 虚拟网口
- 复合设备（虚拟串口&虚拟网口）
- HID

图 7-6 USB 驱动 Device 端配置

```
[*] Enable USB Mass Storage Gadget
[*] Enable USB Video Class Gadget
[ ] Enable USB Video Class Ioctl Mode
    UVC Version (UVC 1.1) ---->
    Transfer Mode (Isochronous Mode) ---->
    YUV Format (YUYV Format) ---->
[*] Enable USB Audio Class Gadget
    UAC Version (UAC1.0) ---->
[*] Enable USB Camera Class Gadget
[*] Enable USB Dfu Gadget
[*] Enable USB Serial Gadget
    Serial Mode (Default Serial) ---->
[*] Enable USB Ethernet Class Gadget
[*] Enable USB Ethernet & Serial Gadget
[*] Enable USB HID Class Gadget
```

须知

初始化Device模式也是调用usb_init(DEVICE, xxx)，根据业务诉求开启需要的协议，也可以全部开启。与Host模式的区别在于初始化时需要指定加载的协议，可以参考[usb_init](#)，有对应协议的宏定义；另外，依赖的外围组件也需要提前加载好，具体参见[Device端](#)的编程实例。

7.4.3 编程实例

7.4.3.1 Host 端

7.4.3.1.1 Host U 盘

操作示例

步骤1 在初始化函数中调用以下接口注册USB驱动。

```
usb_init(HOST,0); // 初始化USB host模块，不需要指定dtype
```

步骤2 USB插入U盘，USB Host枚举完U盘后，会在/dev目录下生成/dev/sda节点。

步骤3 将USB设备节点挂载为正确的文件系统，即可对U盘进行读写操作。

----结束

编程示例

```
#define WRITE_BUFFER_SIZE 40
#define READ_BUFFER_SIZE 40

INT32 Udisk_fs_ctrl(void)
{
```

```

int fd = 0;
int ret = 0;
char w_buf[WRITE_BUFFER_SIZE] = "Hello,this is a test! Monday,Tuesday";
char r_buf[READ_BUFFER_SIZE];
char * charp = NULL;
unsigned int file_length = 0;
int i = 0;

memset(r_buf, 0, READ_BUFFER_SIZE);

dprintf("1.open file:\n");
fd = open("/sd0/testfile", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
if (fd == -1) {
    dprintf("open/create Testfile fail ! \n");
    return -1;
}
dprintf("open file success ! fd = %d \n\n", fd);

dprintf("2.write file:");
ret = write(fd, w_buf, WRITE_BUFFER_SIZE);
if (ret == -1) {
    dprintf("file write fail! \n");
    close(fd);
    return -1;
}
dprintf("write file success!\n\n");

ret = lseek(fd, 0, SEEK_SET);
if (ret == -1) {
    dprintf("lseek fail!\n");
    close(fd);
    return -1;
}
dprintf("lseek file success!\n\n");

dprintf("4.read file:");
if (file_length > READ_BUFFER_SIZE) {
    file_length = READ_BUFFER_SIZE;
}
ret = read(fd, r_buf, file_length);
if (ret == -1) {
    dprintf("file read fail! \n");
    close(fd);
    return -1;
}
charp = r_buf;
for (i = 0; i < ret; i++) {
    dprintf("%c", *charp);
    charp++;
}
dprintf("\n");
charp = NULL;
close(fd);
return 0;
}

void app_init()
{
    g_usb_mem_addr_start = g_sys_mem_addr_end;
    g_usb_mem_size = 0x40000; //recommend 256K nonCache for USB
    usb_init(HOST, 0); //初始化USB host模块, 不需要指定dtype

    /* 等待U盘插入, 驱动注册/dev/sdap0节点 */
    mount("/dev/sdap0", "/sd0", "vfat", 0, 0);
    Udisk_fs_ctrl();
}

```

7.4.3.1.2 Host USB 转网口&RNDIS

在初始化函数中调用以下接口实现TCP/IP协议栈和USB驱动注册：

```
tcpip_init(NULL, NULL); // 初始化TCP/IP协议栈
usb_init(HOST,0); // 初始化USB host模块, 不需要指定dtype
```

USB模块会对虚拟的网口执行netif_set_up操作，不需要用户执行，网口的具体操作方式参见[Shell](#)。

7.4.3.1.3 Host 4G-Modem

在初始化函数中调用以下接口实现TCP/IP协议栈和USB驱动注册：

```
tcpip_init(NULL, NULL); // 初始化TCP/IP协议栈
usb_init(HOST,0); // 初始化USB host模块, 不需要指定dtype
```

USB模块会注册/dev/ttyUSB0节点，并通过u3g_write_from_shell接口去打开该设备节点，并将对应的AT命令写入节点从而与4g模组进行通信。

7.4.3.1.4 Host USB-WIFI

在初始化函数中调用以下接口实现TCP/IP协议栈和USB驱动注册：

```
tcpip_init(NULL, NULL); // 初始化TCP/IP协议栈
usb_init(HOST,0); // 初始化USB host模块, 不需要指定dtype
```

 说明

- 如果通过linux_usb.c提供的接口开发WIFI驱动，在调用usb_init接口后，必须要调用usb_linux_register接口向USB驱动注册相关的钩子函数，否则无法正常识别WIFI设备。
- 在操作USB WIFI前，可先调用usb_wait_device_ready接口来检测当前WIFI设备是否已经成功被识别，如果已被识别，再进行WIFI的其他操作。

7.4.3.1.5 Host UVC

在初始化函数中调用以下接口实现USB驱动注册：

```
usb_init(HOST,0); // 初始化USB host模块, 不需要指定dtype
```

 说明

- USB插入UVC设备，且USB Host枚举UVC设备完成后，会在/dev目录下生成/dev/video设备节点，支持open、close、poll和ioctl操作；其中ioctl支持的命令参数如表1所示。
- 由于ISOC传输对实时性要求很高，即对CPU性能要求很高，因此CPU不能被其他业务长时间打断，否则会引起USB控制器重启。

表 7-10 IOCTL 命令参数表

命令参数	参数含义
VIDIOC_QUERYCAP	获取UVC设备属性（即当前设备的驱动程序名字、设备名称、驱动的版本号等）
VIDIOC_ENUM_FMT	获取UVC设备支持的码流格式
VIDIOC_ENUM_FRAME SIZES	获取UVC设备支持的分辨率信息
VIDIOC_ENUM_FRAME INTERVALS	获取UVC设备支持的帧间隔
VIDIOC_S_FMT	对UVC设备设置码流格式和分辨率

命令参数	参数含义
VIDIOC_G_FMT	获取UVC设备当前运行的码流格式和分辨率
VIDIOC_TRY_FMT	查询UVC设备是否支持某种码流格式和分辨率
VIDIOC_S_PARM	对UVC设备设置帧间隔
VIDIOC_G_PARM	获取UVC设备当前运行的帧率
VIDIOC_REQBUFS	根据应用层传入的帧缓存数量、数据类型等信息向USB申请缓存区
VIDIOC_QUERYBUF	根据应用层传入帧缓存的索引号来获取缓存帧的v4l2_buffer信息
VIDIOC_QBUF	将应用层已处理完的缓存加入空闲可捕获视频的队列
VIDIOC_DQBUF	应用层向驱动获取已经存放有视频数据的缓存帧
VIDIOC_STREAMON	启动数据流传输
VIDIOC_STREAMOFF	停止数据流传输

7.4.3.1.6 Host HID

在初始化函数中调用以下接口实现USB驱动注册：

```
usb_init(HOST,0); // 初始化USB host模块, 不需要指定dtype
```

说明

- USB插入游戏手柄，USB Host枚举游戏手柄完成后，会在/dev目录下生成/dev/uhidX-Y设备节点，其中X代表手柄的序列号，Y代表手柄接入的USB物理端口号，然后通过open、close、read和write进行操作。
- 游戏手柄会不停的发送数据给Host端，因此需要及时的读取数据，否则等DMA Buffer满了后，会产生丢包。

7.4.3.2 Device 端

7.4.3.2.1 Device 虚拟 U 盘

初始化

在初始化函数中初始化SD卡驱动和USB驱动：

```
SD_MMC_Host_init();
usbd_set_device_info(DEV_MASS, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_MASS); // dtype设置为DEV_MASS
```

说明

- 在多卡情况下(系统同时挂载2张卡以上)，USB2.0 Device默认绑定第一张卡设备节点，例如同时存在/dev/mmcblk0p0，/dev/mmcblk1p0，USB2.0 Device会绑定/dev/mmcblk0p0，因此Host端通过USB口只能访问到一张SD/MMC卡。
- USB2.0 Device连接Host端后，Host端可以通过USB端口访问到系统存储介质的文件。在Host端访问存储介质过程，系统端请勿对MMC/SD Card存储介质进行读写、格式化等操作，避免Host端访问存储介质异常。
- Device U盘对接PC进行格式化时，仅支持Windows PC进行快速格式化操作，请勿使用Windows PC的一般格式化操作或Linux的高级格式化命令，避免对分区信息进行修改，导致Device卸载虚拟U盘后mount失败。如果进行了此操作，mount失败后可重新format 存储介质来解决。

注册 Device U 盘回调函数

应用层通过fmass_register_notify(void(*notify)(void* context, int status), void* context)函数注册notify回调，驱动会在识别到接入和拔出 Host端时调用此回调函数，从而实现DeviceU盘相关拔插的处理。

请在USB模块初始化前，调用fmass_register_notify()实现Device回调函数的注册。

须知

Notify回调函数中，在usb device链接的情况下，需通过fmass_partition_startup()将需要被PC端识别的SD卡分区节点启动，并且保证其接口传进来的设备节点是存在的，如果不存在，就默认使用/dev/mmcblk0p0（前提此设备节点存在）虚拟为U盘，否则会导致失败。

注意事项

- 在正常操作过程中需要遵守的事项：
 - Device U盘将SD卡交由USB Host处理，因此在 DeviceU盘被Host端识别后，会通过回调函数通知APP软件模块，APP必须将所有录像抓拍等操作SD卡的业务停下，并umount SD卡分区，待USB Host离线后再mount和恢复相关业务。
 - USB Host端进行读写访问过程，不要拔出单板的SD/MMC卡，否则会打印一些异常信息，并且可能会导致卡中文件或文件系统被破坏。
- 在操作过程中出现异常时的操作：

若拔出Device U盘后，再极其快速地插入DeviceU盘，可能会导致Host端检测不到U盘，因为单板DeviceU盘的检测注册/注销过程需要一定的时间。

编程示例

```
extern int fmass_register_notify(void(*notify)(void* context, int status), void* context);
extern int fmass_partition_startup(char* path);
void fmass_app_notify(void* conext, int status)
{
    char *path = "/dev/mmcblk0p0";
    if(status == 1)/*usb device connect*/
    {
        /* 停止录像抓拍等对/dev/mmcblk0p0分区的读写操作，并umount该分区 */
        fmass_partition_startup(path); // startup fmass access partition
    }
    else
```

```

    {
        /* mount该分区,并通知其它模块该分区可以进行读写操作 */
    }
}
void app_init()
{
    uint32_t ret;
    int handle;

    const char manufacturer[38] = {
        'H', 0, 'i', 0, 'S', 0, 'i', 0, 'l', 0, 'i', 0, 'c', 0, 'o', 0, 'n', 0,
        '(', 0, 'S', 0, 'h', 0, 'a', 0, 'n', 0, 'g', 0, 'h', 0, 'a', 0, 'i', 0, ')', 0
    };
    struct device_string str_manufacturer = {
        .str = manufacturer,
        .len = 38
    };

    const char product[18] = {
        'U', 0, 'S', 0, 'B', 0, 'D', 0, 'E', 0, 'V', 0, 'I', 0,
        'C', 0, 'E', 0
    };
    struct device_string str_product = {
        .str = product,
        .len = 18
    };

    const char serial[16] = {
        '2', 0, '0', 0, '2', 0, '0', 0, '0', 0, '6', 0, '2', 0,
        '4', 0
    };
    struct device_string str_serial_number = {
        .str = serial,
        .len = 16
    };

    struct device_id dev_id = {
        .vendor_id = 0x3361,
        .product_id = 0x0108,
        .release_num = 0x0318
    };

    SD_MMC_Host_init();

    g_usb_mem_addr_start= g_sys_mem_addr_end;
    g_usb_mem_size= 0x40000; //recommend 256K nonCache for usb

    ret = usbd_set_device_info(DEV_MASS, &str_manufacturer, &str_product, &str_serial_number, dev_id);
    if (ret != 0) {
        dprintf("set fail!\n");
        return;
    }
    handle = fmass_register_notify(fmass_app_notify,NULL);
    if (handle == -1) {
        return;
    }
    ret = usb_init(DEVICE, DEV_MASS); // usb_init must after SD_MMC_Host_init
    if (ret != 0) {
        fmass_unregister_notify(handle);
        dprintf("usb init failed!\n");
        return;
    }
}

```

内存中 Fat32 文件系统虚拟为 U 盘

步骤1 制作FAT格式的文件系统镜像（以Ubuntu系统为例）。

1. 制作FAT格式磁盘镜像。
`mkfs.fat -s 64 -S 512 -f 2 -n "Internal SD" -F 32 -C usbdisk.tmp 2048`

其中2048表示磁盘空间大小，单位是KB。

2. 挂载FAT格式磁盘镜像。
`mount -o loop -t vfat usbdisk.tmp test/`

其中test为挂载目录。

3. 在test目录下存放所需文件，然后将文件同步到磁盘。
`sync test/`

4. 卸载磁盘镜像。
`umount test/`

5. 制作文件系统镜像。
`dd if=usbdisk.tmp of=udisk.img bs=1024 count=2048`

将usbdisk.tmp前面2MB的内容制作为文件系统镜像udisk.img。

步骤2 将制作好的文件系统镜像下载到flash上。

将udisk.img文件放到指定路径FAT_PART_IMAGE_PATH（"/jffs0/udisk.img"）。

步骤3 调用ram_mass_storage_init接口注册RAM_DEV_NAME（/dev/uram）设备节点。

步骤4 调用fmass_register_notify接口指定设备节点/dev/uram虚拟为U盘。

步骤5 调用usbd_set_device_info和usb_init接口去加载虚拟U盘协议。

须知

本次制作的文件系统镜像申请的是2M容量，请根据实际业务需求，除了需要制定特定容量的文件系统镜像之外，还要修改对应的FAT_PART_SIZE大小。

----结束

7.4.3.2.2 Device UVC

Device UVC支持UVC1.0协议和UVC1.1协议版本的切换，通过修改make menuconfig可选择UVC1.0或UVC1.1版本。当前默认为UVC1.1协议，切换为UVC1.0可支持Windows xp系统，增加兼容性。

操作指导

1. UVC上层应用在USB初始前设置UVC传输能力集，开启VI，打开UVC设备。
2. 就绪后，等待底层UVC连接，启动VPSS和VENC编码并传输码流数据，直至UVC通道断开。
3. 切换新码流格式时，应用须先关闭前一次编码并清空缓存，再启动编码。

操作示例

- 方式1 直接调用device uvc相关接口直接控制数据传输。

步骤1 通过fuvc_frame_descriptors_get接口设置UVC传输能力集（含：编码格式、分辨率和帧率）。

步骤2 在初始化函数中调用usbd_set_device_info和usb_init接口实现USB驱动注册。

```
usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_UVC); // dtype设置为DEV_UVC
```


- 步骤3** 开启VI后，调用`uvc_open_device`打开UVC设备，需指定分辨率和帧率。
- 步骤4** 通过`uvc_wait_host`等待板端与USB主机侧连接成功。
- 步骤5** 启动VPSS和VENC编码后，通过`uvc_rcv_pack`开始执行视频数据传输。
- 步骤6** 通过`uvc_video_stop`停止视频数据传输，可调用`uvc_close_device`关闭USB设备。

----结束

- 方式2 打开device uvc注册的设备节点，并通过系统调用控制数据传输。

- 步骤1** 在初始化函数中调用`usbd_set_device_info`和`usb_init`接口实现USB驱动注册。

```
usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);  
usb_init(DEVICE, DEV_UVC); // dtype设置为DEV_UVC
```

- 步骤2** 开启VI后，调用`open`打开UVC设备节点“`/dev/usb_uvc-0`”，并通过调用`select`函数监视设备连接情况。
- 步骤3** 当板端与USB主机侧连接成功并可以开始传输数据时，`select`系统调用能够接收到获取到消息，此时调用`ioctl`接口发送`VIDIOC_DQEVENT`命令，并提供结构体`struct usbd_uvc_video_event`来接收数据。根据接收的数据将视频的分辨率帧率等信息发送给VPSS和VENC。
- 步骤4** 启动VPSS和VENC编码后，通过`uvc_rcv_pack`开始执行视频数据传输。同时通过系统调用`ioctl`发送`VIDIOC_STREAMON`命令通知USB设备可以开始进行数据传输。
- 步骤5** 通过调用`VIDIOC_STREAMOFF`可以停止视频传输，再通过系统调用`close`关闭USB设备。

----结束

须知

方式1与方式2都有打开设备的操作，分别是直接调用接口`uvc_open_device`和调用系统接口`open`。当同时使用这两个接口时，只能打开一次设备，后调用的接口将无法打开设备且其各种操作都将失效。

7.4.3.2.3 Device UAC

UAC与UVC的数据处理流程类似，具体步骤如下：

- 步骤1** 在初始化函数中调用`usbd_set_device_info`和`usb_init`接口实现USB驱动注册。
- ```
usbd_set_device_info(DEV_UAC, str_manufacturer, str_product, str_serial_number, dev_id);
usb_init(DEVICE, DEV_UAC); // dtype设置为DEV_UAC
```
- 步骤2** 开启VI后，通过`uac_wait_host`等待板端与USB。
- 步骤3** 启动VPSS和VENC编码后，通过调用`fuac_send_message`发送音频数据给Host，调用`fuac_receive_message`接收Host下发的音频数据。
- 步骤4** 停止调用`fuac_send_message`和`fuac_receive_message`接口，即停止音频数据传输。

----结束

#### 7.4.3.2.4 Device UAV ( Camera )

UAV是同时支持UAC、UVC，相关操作流程，参见上面UVC、UAC使用示例。区别在于初始化时，传入的协议类型：

```
usbd_set_device_info(DEV_CAMERA, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_CAMERA); // dtype设置为 DEV_CAMERA
```

#### 7.4.3.2.5 Device 虚拟串口

在初始化函数中调用以下接口实现USB驱动注册：

```
usbd_set_device_info(DEV_SERIAL, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_SERIAL);
```

若需要基于USB虚拟的串口进行shell通信交互与调试信息打印，需要再进行console，Shell等与串口有依赖关系的模块初始化。

```
virtual_serial_init("/dev/ttyGS0");
system_console_init("/dev/serial");
```

若单板端写数据发送到USB主机，则直接调用write接口；若单板端读取从USB主机发来的数据，open串口后先调用ioctl接口，然后再调用read接口，ioctl用法如下：

```
ioctl(fd, CONSOLE_CMD_RD_BLOCK_SERIAL, 1);
```

##### 须知

- 虚拟串口功能不能与Uart串口同时使用。
- 如果有对节点的多次open操作，需要在卸载前相应的多次close，即open和close次数需要相等，才能保证协议卸载时成功，否则会出现不可预知的错误。
- 虚拟串口最大支持读写4096字节数据。
- 单串口模式下初始化后，在dev目录下生成/dev/ttyGS0节点，双串口模式下还会再生成一个/dev/ttyGS1节点。

如果Host端为Windows PC，那么首先在PC端放置驱动文件linux-cdc-acm.inf，该文件取自发布包路径为：01.software/pc/usb\_tools。

通过USB数据线将单板与Host端相连，PC端会自动加载驱动，第一次一般会失败，需要自行安装驱动，方法为：

**步骤1** 右击计算机，进入管理界面。

**步骤2** 打开设备管理器。

**步骤3** 点开其他设备，会看到Gadget Serial(COMx)，双击。

**步骤4** 打开驱动程序界面，点击更新驱动程序，进入浏览计算机以查找驱动程序软件(R)。

**步骤5** 把路径指向驱动文件(linux-cdc-acm.inf)所在的目录，点击下一步，计算机会自动进行安装驱动程序，安装成功后，关闭界面。

**须知**

- 01. software/pc/usb\_tools目录下有linux-cdc-acm.inf文件，通常win7 无法自动加载虚拟串口驱动，需要手动加载；win10通常会自动加载虚拟串口驱动，无需手动加载。
- 如果安装串口驱动失败，则需要修改linux-cdc-acm.inf文件，在[DeviceList]和[DeviceList.NTamd64]里添加USB\VID\_3361&PID\_0102&REV\_0318，其中3361对应的是VID，0102对应的是PID，0318对应的是设备版本号，这3组数字是通过调用usbd\_set\_device\_info接口来设置，请根据实际设备进行修改，需要和设置值保持一致。

单板端重新上电启动，在PC端就可以作为真正的串口来使用。

----结束

### 7.4.3.2.6 Device 虚拟网口

**步骤1** 在初始化函数中调用以下接口实现TCP/IP协议栈初始化和USB驱动注册。

```
tcpip_init(NULL, NULL); // TCP/IP协议栈初始化
usbd_set_device_info(DEV_ETHERNET, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_ETHERNET);
```

**步骤2** USB虚拟网口默认未配置IP地址，请根据网络规划自行配置IP地址。

**步骤3** 如果host端为windows 7或者windows10，那么首先在PC端放置linux.inf驱动文件，见发布包路径：01.software/pc/usb\_tools。然后修改linux.inf驱动文件，在[RndisDevices.NTx86]、[RndisDevices.NTamd64]和[RndisDevices.NTia64]添加USB\VID\_3361&PID\_0102&REV\_0318，其中3361对应的是VID，0102对应的是PID，0318对应的是设备版本号，这3组数字是通过调用usbd\_set\_device\_info接口来设置，请根据实际设备进行修改，要和设置值保持一致。

**步骤4** 通过USB数据线将单板与Host端相连。

- 如果Host端为windows7系统，通常自动加载驱动，一般会失败，需要自行安装驱动，具体操作为：
  - a. 右击计算机，进入管理界面。
  - b. 打开设备管理器。
  - c. 点开其他设备，会看到Linux USB Ethernet/RNDIS Gadget #3，双击。
  - d. 打开驱动程序界面，点击更新驱动程序，进入浏览计算机以查找驱动程序软件(R)。
  - e. 把路径指向linux.inf所在的目录，点击下一步，计算机会自动进行安装驱动程序，安装成功后，关闭界面。
- 如果Host端为windows10系统，通常会误识别为虚拟串口，需要手动更新驱动，具体操作为：
  - a. 右击计算机，进入管理界面。
  - b. 打开设备管理器。
  - c. 点开端口设备，会看到USB串行设备（COMX）。
  - d. 打开驱动程序界面，点击更新驱动程序，进入浏览计算机以查找驱动程序软件（R）。
  - e. 把路径指向linux.inf所在的目录，点击下一步，计算机会自动进行安装驱动程序，安装成功后，会生成虚拟网口设备，对应虚拟串口设备消失，关闭界

面。如果安装过程中提示第三方INF不包含数字签名信息，此关闭方法可在网上搜索，此处不再赘述。

**步骤5** 在单板端，配置IP并添加路由。

**步骤6** 当单板和PC通过USB数据线相连时，会在PC端生成USB网络节点，具体位置：打开网络和共享中心->更改适配器设置->Linux USB Ethernet /RNDIS Gadget #x。

**步骤7** 建立网桥：

1. 右键Linux USB E thernet /RNDIS Gadget #x节点，点击添加到桥选项；
2. PC端连接大网的本地连接节点右键，选择添加到桥选项；
3. 等待网桥建立完成，如下图所示。

图 7-7 成功建立网桥



此时在单板端，能正常访问大网，可作为真正的网口操作使用。

----结束

#### 须知

- 在in10 PC桥接网络，若网桥无法获取到IP地址时，请确定一下桥接顺序，桥接时请先选择物理网卡，再选择虚拟网卡，然后桥接。若配置网桥失败，请单击确定，然后再选择物理网卡右键添加到桥（这个步骤很重要），因为造成配置网桥失败的原因就是物理网卡没有添加到桥。
- 当访问桥接的网络环境时需要权限，若出现能ping通IP地址但ping不通网关和服务器的问題，请排查单板端的MAC地址（IP和MAC地址有关联性）。

### 7.4.3.2.7 Device 复合设备（串口&网口）

复合设备支持单串口与网口复合以及双串口与网口复合，在配置项里，打开串口与网口复合的配置项后，默认是单串口与网口复合，若要双串口与网口复合，需要在串口模式中选择双串口即可。

以上两种复合设备的初始化过程一样，具体过程如下：

```
tcpip_init(NULL, NULL); // TCP/IP协议栈初始化
usbd_set_device_info(DEV_SER_ETH, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_SER_ETH);
virtual_serial_init("/dev/ttyGS0");
system_console_init("/dev/serial");
```

复合设备中串口和网口的使用同上面[Device虚拟串口](#)和[Device虚拟网口](#)使用示例。

#### 须知

虚拟串口功能不能与Uart串口同时使用。

### 7.4.3.2.8 Device DFU

**步骤1** 在初始化函数中调用以下接口实现USB驱动注册。

```
usbd_set_device_info(DEV_DFU, &str_manufacturer, &str_product, &str_serial_number, dev_id);
usb_init(DEVICE, DEV_DFU);
```

**步骤2** 升级开始前，调用usb\_dfu\_init\_env\_entities接口设置升级接口类型及环境变量。

**步骤3** 调用usb\_dfu\_update\_status，等待升级事件。

**步骤4** 等到有升级事件后，调用usb\_dfu\_update\_size\_get获取升级文件大小，并对升级文件进行必要的校验，检查升级文件是否携带后缀字段（校验和、标志等关键字），如携带后缀比对校验和。

**步骤5** 根据上述升级文件大小，为DFU缓存升级文件提供足够的内存空间，通过usb\_dfu\_read获取实际升级文件。

**步骤6** 获取到实际升级文件后，即可对固件执行升级动作，写入指定Flash地址段。

**步骤7** 最后需要调用usb\_dfu\_free\_entities接口释放相关资源。

#### 说明

给升级的文件提供足够的内存空间主要与厂商提供的升级镜像大小有关，请预留足够的升级的空间。

----结束

### 7.4.3.2.9 Device HID

HID上层应用在USB初始化之前，先调用usbd\_set\_device\_info接口来设置厂商ID、产品ID、设备序列号以及字符串信息，然后调用hid\_report\_descriptor\_info或者hid\_add\_report\_descriptor接口来配置报告描述符信息，让USB主机识别为具体的HID设备，如键盘、鼠标、带有鼠标功能的键盘等

## 操作示例

- 方式1 打开Device HID注册的设备节点，通过调用VFS模块的接口进行HID数据流传输。

**步骤1** 调用usbd\_set\_device\_info和hid\_report\_descriptor\_info接口来填充各个描述符信息。

**步骤2** 在初始化函数中调用usb\_init接口实现USB驱动注册。

```
usb_init(DEVICE, DEV_HID); // dtype设置为 DEV_HID
```

**步骤3** 在上层应用中，调用open函数来打开指定的HID设备。

```
fd = open("/dev/hid", O_RDWR, 0);
```

**步骤4** 在上层应用中，按报告描述符规定的INPUT数据格式写到write函数中的入参buffer，长度也是按报告描述符规定的INPUT长度。若报告描述符定义了OUTPUT项目，则调用read函数去读取USB主机发给设备的数据，读取的大小是按报告描述符规定的OUTPUT长度，若要提高任务执行效率，可以调用poll机制，两者区别是read是永久阻塞模式，而poll机制是定时阻塞模式。

**步骤5** 数据传输结束之后，若不使用设备，便调用close函数关闭设备。

----结束

- 方式2 直接调用非VFS模块的对外接口进行HID数据流传输。

**步骤1** 调用usb\_set\_device\_info和hid\_add\_report\_descriptor接口来填充各个描述符信息。

**步骤2** 在初始化函数中调用usb\_init接口实现USB驱动注册。

```
usb_init(DEVICE, DEV_HID); // dtype设置为 DEV_HID
```

**步骤3** 在上层应用中，按报告描述符规定的INPUT数据格式和长度信息，调用fhid\_send\_data接口发送HID数据。若报告描述符定义了OUTPUT项目，则调用fhid\_rcv\_data接口去读取USB主机发给设备的数据，读取的HID数据长度与报告描述符规定的OUTPUT长度一致。

----结束

须知

- hid\_report\_descriptor\_info接口只能设置一个Report，并和VFS模块的接口配套使用；hid\_add\_report\_descriptor接口可以最多设置两个Report（循环调用接口进行设置），并和非VFS模块的接口配套使用。这两种方式不允许混用。如果需要重新设置Report，则需要加载并卸载Device HID协议才能设置。
- 如果报告描述符中没有Report ID项目（鼠标或者键盘报告表），默认ID值是0，在数据传输过程中，第一个字节不需要填充0，若定义一个以上的Report ID（带有鼠标功能的键盘报告表），此时在传输报表中的第一个字节填充相应的Report ID，输入报表、输出报表与特征报表可以分享同一个Report ID。
- 当USB主机发给设备数据的时候，建议及时把数据读走，否则USB主机第二次发来的数据会覆盖第一次的数据。
- 若调用了open函数，此时无法卸载协议，只有调用close函数，才能卸载协议。当前Device HID只允许open一次，再次open就会失败。
- 上层应用中调用VFS模块的read和write接口或者非VFS模块的fhid\_send\_data和fhid\_rcv\_data接口时，请确保传进来的buffer和len大小要一致，否则会造成发送或者读取的数据有误。
- 方式1和方式2不能同时使用，若使用方式1，需要打开VFS模块功能。HID业务运行过程中，不允许卸载HID协议，必须要停止应用层的业务操作后，才能卸载协议。
- 方式1和方式2的接收HID数据接口只能在任务中调用，不能在中断回调调用，发送HID数据接口可以在中断回调调用。

7.4.4 API 参考

该功能模块提供以下接口：

| 接口名                   | 描述                                       |
|-----------------------|------------------------------------------|
| usb_set_device_info   | 设置device设备的VID、PID、设备版本号和厂商、产品、序列号字符串信息。 |
| usb_init              | 初始化USB模块。                                |
| usb_deinit            | 卸载USB模块。                                 |
| usb_is_devicemode     | 判断当前USB Device是否已连接USB Host。             |
| fmass_register_notify | 注册Device notify回调函数。                     |



| 接口名                                        | 描述                                  |
|--------------------------------------------|-------------------------------------|
| <a href="#">fmass_unregister_notify</a>    | 注销Device notify回调函数。                |
| <a href="#">fmass_partition_startup</a>    | 启动被PC端识别的SD卡、MMC介质或RAM介质分区节点。       |
| <a href="#">ram_mass_storage_init</a>      | 使能Fat32文件系统镜像内存映射，并注册/dev/uram设备节点。 |
| <a href="#">ram_mass_storage_deinit</a>    | 非使能Fat32文件系统镜像内存映射。                 |
| <a href="#">fuvc_frame_descriptors_get</a> | 设置UVC设备视频传输能力集。                     |
| <a href="#">uvc_open_device</a>            | 打开UVC设备。                            |
| <a href="#">uvc_close_device</a>           | 关闭UVC设备。                            |
| <a href="#">uvc_wait_host</a>              | UVC等待USB HOST连接。                    |
| <a href="#">uvc_get_state</a>              | 获取UVC设备的运行状态。                       |
| <a href="#">uvc_rcv_pack</a>               | 启动UVC的视频数据传输，拷贝方式。                  |
| <a href="#">uvc_video_stop</a>             | 停止UVC视频传输。                          |
| <a href="#">uvc_format_info_get</a>        | 获取新码流格式详细信息。                        |
| <a href="#">uvc_format_status_check</a>    | 码流格式切换事件。                           |
| <a href="#">hi_camera_register_cmd</a>     | 应用注册UVC扩展命令，提供底层驱动调用。               |
| <a href="#">uac_wait_host</a>              | UAC等待USB HOST连接。                    |
| <a href="#">fuac_send_message</a>          | 音频数据传输。                             |
| <a href="#">fuac_receive_message</a>       | 音频数据传输接收。                           |
| <a href="#">fuac_opts_set</a>              | 设置uac1.0音频参数，包括通道数等。                |
| <a href="#">fuac_get_opts</a>              | 获取uac1.0音频参数，包括通道数等。                |
| <a href="#">fuac_rate_get</a>              | 获取uac1.0音频采样率。                      |
| <a href="#">fuac2_get_opts</a>             | 获取音频2.0协议的参数，包括通道数等。                |
| <a href="#">fuac2_set_opts</a>             | 设置音频2.0协议的参数，包括通道数等。                |
| <a href="#">usb_dfu_init_env_entities</a>  | 初始化配置DFU接口功能，以及配置环境变量。              |
| <a href="#">usb_dfu_update_status</a>      | 获取升级事件。                             |
| <a href="#">usb_dfu_update_size_get</a>    | 获取升级文件实际大小。                         |

| 接口名                        | 描述                           |
|----------------------------|------------------------------|
| usb_dfu_get_entity         | 获取DFU列表中指定的DFU结构体变量。         |
| usb_dfu_read               | 获取指定DFU列表中的一定大小的缓冲数据。        |
| hid_report_descriptor_info | 配置报告描述符信息，设置报告数据格式和功能。       |
| hid_add_report_descriptor  | 配置多个报告表描述符，设置报告表数据格式、功能和接口协议 |
| fhid_send_data             | HID数据传输发送。                   |
| fhid_rcv_data              | HID数据传输接收。                   |
| u3g_write_from_shell       | 对4G模组发送串口命令。                 |
| uvc_get_frame_buf          | 获取帧缓冲区的地址（自研接口）。             |
| usb_dfu_free_entities      | 释放DFU相关资源。                   |

7.4.4.1 usbd\_set\_device\_info

【描述】

设置device设备的VID、PID、设备版本号和厂商、产品、序列号字符串信息。

【语法】

```
uint32_t usbd_set_device_info(device_type dtype,
 const struct device_string *str_manufacturer,
 const struct device_string *str_product,
 const struct device_string *str_serial_number,
 struct device_id dev_id);
```

【参数】

| 参数名称             | 描述                                                                                                                                                                                                                                                                                                                          | 输入/输出 |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| dtype            | device 类型： <ul style="list-style-type: none"><li>• DEV_SERIAL，虚拟串口。</li><li>• DEV_ETHERNET，虚拟网口。</li><li>• DEV_SER_ETH，复合设备（串口&amp;网口）。</li><li>• DEV_DFU，USB 固件升级。</li><li>• DEV_MASS，虚拟U盘。</li><li>• DEV_UVC，USB 视频传输。</li><li>• DEV_UAC，USB 音频传输。</li><li>• DEV_CAMERA，USB 摄像头。</li><li>• DEV_HID，USB HID设备输入。</li></ul> | 输入    |
| str_manufacturer | 厂商字符串信息，结构体第一个成员变量是字符串指针，第二个成员变量是字符串的总长度。                                                                                                                                                                                                                                                                                   | 输入    |



| 参数名称              | 描述                                                                         | 输入/输出 |
|-------------------|----------------------------------------------------------------------------|-------|
| str_product       | 产品字符串信息，结构体成员同上。                                                           | 输入    |
| str_serial_number | 产品序列号字符串信息，结构体成员同上。                                                        | 输入    |
| dev_id            | 设备ID号，结构体第一个成员变量是VID（厂商ID号），第二个成员变量是PID（产品ID号），第三个成员变量是release_num，即设备版本号。 | 输入    |

【返回值】

| 返回值     | 描述 |
|---------|----|
| LOS_OK  | 成功 |
| LOS_NOK | 失败 |

【需求】

- 头文件：usb\_init.h
- 库文件：libusb\_base.a

【注意】

- 厂商ID号不可随意设置，需向USB协会申请，产品ID号和设备版本号可根据实际产品进行设置，如果同一个单板切换为不同的USB设备需要更换PID号，否则USB主机识别到的设备驱动会有问题。
- 字符串长度要和字符串大小一致，且长度范围2~252，传入的入参长度必须是偶数，否则USB主机无法识别字符串信息。

【举例】

```
const char manufacturer[38] = {
 'H', 0, 'i', 0, 'S', 0, 'i', 0, 'l', 0, 'i', 0, 'c', 0, 'o', 0, 'n', 0,
 'l', 0, 'S', 0, 'h', 0, 'a', 0, 'n', 0, 'g', 0, 'h', 0, 'a', 0, 'i', 0, 'i', 0
};
struct device_string str_manufacturer = {
 .str = manufacturer,
 .len = 38
};

const char product[18] = {
 'U', 0, 'S', 0, 'B', 0, 'D', 0, 'E', 0, 'V', 0, 'I', 0,
 'C', 0, 'E', 0
};
struct device_string str_product = {
 .str = product,
 .len = 18
};

const char serial[16] = {
 '2', 0, '0', 0, '2', 0, '0', 0, '0', 0, '6', 0, '2', 0,
 '4', 0
};
struct device_string str_serial_number = {
```

```
.str = serial,
.len = 16
};
struct device_id dev_id = {
 .vendor_id = 0x3361,
 .product_id = 0x0102,
 .release_num = 0x0318
};

uint32_t ret;
ret = usbd_set_device_info(DEV_UVC, &str_manufacturer, &str_product, &str_serial_number, dev_id);
if (ret != 0) {
 dprintf("set fail!\n");
}
```

 **注意**

此例仅供参考，不可直接使用，特别是VID号，请务必修改为客户自己的厂商信息。

【相关主题】

无

7.4.4.2 usb\_init

【描述】

初始化USB模块。

【语法】

```
uint32_t usb_init(controller_type ctype, device_type dtype);
```

【参数】

| 参数名称  | 描述                                                                                                                                                                                                                                                                                                                                                                                                                                    | 输入/输出 |
|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| ctype | 枚举类型，描述了usb是host类型或device类型： <ul style="list-style-type: none"><li>• HOST表示host类型。</li><li>• DEVICE表示device类型。</li></ul>                                                                                                                                                                                                                                                                                                              | 输入    |
| dtype | 枚举类型， <ul style="list-style-type: none"><li>• 作为host模式时，该入参无效，传入0。</li><li>• 作为device时，表示device类型：<ul style="list-style-type: none"><li>- DEV_SERIAL, 虚拟串口。</li><li>- DEV_ETHERNET, 虚拟网口。</li><li>- DEV_SER_ETH, 复合设备（串口&amp;网口）。</li><li>- DEV_DFU, USB 固件升级。</li><li>- DEV_MASS, 虚拟U盘。</li><li>- DEV_UVC, USB 视频传输。</li><li>- DEV_UAC, USB 音频传输。</li><li>- DEV_CAMERA, USB 摄像头。</li><li>- DEV_HID, USB HID设备输入。</li></ul></li></ul> | 输入    |

【返回值】

| 返回值     | 描述 |
|---------|----|
| LOS_OK  | 成功 |
| LOS_NOK | 失败 |

【需求】

- 头文件：usb\_init.h
- 库文件：libusb\_base.a

【注意】

- USB各协议使用时与其他模块的依赖关系，参考[编程实例](#)。
- 不支持重复执行，切换协议需要调用usb\_deinit卸载当前协议。

【举例】

参考[编程实例](#)。

【相关主题】

[usb\\_deinit](#)

7.4.4.3 usb\_deinit

【描述】

卸载USB模块。

【语法】

```
uint32_t usb_deinit(void);
```

【参数】

无

【返回值】

| 返回值     | 描述 |
|---------|----|
| LOS_OK  | 成功 |
| LOS_NOK | 失败 |

【需求】

- 头文件：usb\_init.h

- 库文件：libusb\_base.a

【注意】

无

【举例】

参考[编程实例](#)。

【相关主题】

[usb\\_init](#)

7.4.4.4 usb\_is\_devicemode

【描述】

判断当前USB Device是否已连接USB Host。

【语法】

```
bool usb_is_devicemode(void);
```

【参数】

无

【返回值】

| 返回值   | 描述                      |
|-------|-------------------------|
| TRUE  | 当前USB Device已连接USB Host |
| FALSE | 当前USB Device未连接USB Host |

【需求】

- 头文件：usb\_init.h
- 库文件：libusb\_base.a

【注意】

此接口要在加载USB Device协议后再去调用

【举例】

无

【相关主题】

无

7.4.4.5 fmass\_register\_notify

【描述】

注册Device notify回调函数。

## 【语法】

```
int fmass_register_notify(void (*notify)(void *context, int status), void *context);
```

## 【参数】

| 参数名称    | 描述                                               | 输入/输出 |
|---------|--------------------------------------------------|-------|
| notify  | Notify回调函数指针。                                    | 输入    |
| context | 需要传入Notify回调函数的私有数据指针，该指针会传入给Notify回调函数的context。 | 输入    |

## 【返回值】

| 返回值                   | 描述 |
|-----------------------|----|
| 0 ~ MAX_NOIFY_NUM - 1 | 成功 |
| -1                    | 失败 |

## 【需求】

- 头文件：f\_mass\_storage.h
- 库文件：libusb\_device.a

## 【注意】

允许注册多个notify回调函数，USB2.0 Device拔插事件发生时驱动会调用每一个notify函数。

## 【举例】

参考[编程实例](#)。

## 【相关主题】

- [fmass\\_partition\\_startup](#)
- [fmass\\_unregister\\_notify](#)

### 7.4.4.6 fmass\_unregister\_notify

## 【描述】

注销Device notify回调函数。

## 【语法】

```
int fmass_unregister_notify(int handle);
```

## 【参数】

| 参数名称   | 描述                          | 输入/输出 |
|--------|-----------------------------|-------|
| handle | fmass_register_notify返回的句柄值 | 输入    |

## 【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |

## 【需求】

- 头文件：f\_mass\_storage.h
- 库文件：libusb\_device.a

## 【注意】

无

## 【举例】

参考[编程实例](#)。

## 【相关主题】

[fmass\\_register\\_notify](#)

#### 7.4.4.7 fmass\_partition\_startup

## 【描述】

启动被PC端识别的SD卡、MMC介质或RAM介质分区节点。

## 【语法】

```
int fmass_partition_startup(const char *path);
```

## 【参数】

| 参数名称 | 描述                               | 输入/输出 |
|------|----------------------------------|-------|
| path | SD卡、MMC介质或RAM介质分区节点，该设备节点必须是有效的。 | 输入    |

## 【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |

| 返回值 | 描述 |
|-----|----|
| -1  | 失败 |

**【需求】**

- 头文件：f\_mass\_storage.h
- 库文件：libusb\_base.a

**【注意】**

无

**【举例】**

参考[编程实例](#)。

**【相关主题】**

[fmass\\_register\\_notify](#)

#### 7.4.4.8 ram\_mass\_storage\_init

**【描述】**

使能FAT32文件系统镜像内存映射，并注册/dev/uram设备节点。

**【语法】**

```
int ram_mass_storage_init(void);
```

**【参数】**

无

**【返回值】**

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |

**【需求】**

- 头文件：f\_mass\_storage\_ram.h
- 库文件：libusb\_base.a

**【注意】**

无

**【举例】**

参考[编程实例](#)。

【相关主题】

[ram\\_mass\\_storage\\_deinit](#)

7.4.4.9 ram\_mass\_storage\_deinit

【描述】

禁用FAT32文件系统镜像的内存映射。

【语法】

```
int ram_mass_storage_deinit(void);
```

【参数】

无

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |

【需求】

- 头文件：f\_mass\_storage\_ram.h
- 库文件：libusb\_base.a

【注意】

无

【举例】

参考[编程实例](#)。

【相关主题】

[ram\\_mass\\_storage\\_init](#)

7.4.4.10 fuvc\_frame\_descriptors\_get

【描述】

设置UVC设备视频传输能力集。

【语法】

```
void fuvc_frame_descriptors_get(const struct fuvc_format_info *format_info, uint32_t frames_count);
```

【参数】

| 参数名称        | 描述                | 输入/输出 |
|-------------|-------------------|-------|
| format_info | UVC支持的视频码流格式以及分辨率 | 输入    |



| 参数名称         | 描述             | 输入/输出 |
|--------------|----------------|-------|
| frames_count | 视频格式对应的分辨率的总个数 | 输入    |

【返回值】

无

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

参考[编程实例](#)。

【举例】

参考[编程实例](#)。

【相关主题】

无

7.4.4.11 uvc\_open\_device

【描述】

打开UVC设备。

【语法】

```
int uvc_open_device(uvc_t *hdl, const struct uvc_open_param *param);
```

【参数】

| 参数名称  | 描述                          | 输入/输出 |
|-------|-----------------------------|-------|
| hdl   | 用于存储UVC设备句柄                 | 输出    |
| param | 打开UVC设备的初始能力，使用中根据实际需求可变化修改 | 输入    |

【返回值】

| 返回值               | 描述       |
|-------------------|----------|
| UVC_OK            | 成功       |
| UVC_ERROR_NOMATCH | UVC状态不匹配 |

| 返回值              | 描述      |
|------------------|---------|
| UVC_ERROR_HANDLE | UVC句柄无效 |
| UVC_ERROR_PTR    | 指针非法    |
| UVC_ERROR_MEMORY | 内存不足    |
| UVC_ERROR_VALUE  | 取值非法    |

**【需求】**

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

**【注意】**

参考[编程实例](#)。

**【举例】**

参考[编程实例](#)。

**【相关主题】**

[uvc\\_close\\_device](#)

## 7.4.4.12 uvc\_close\_device

**【描述】**

关闭UVC设备。

**【语法】**

```
int uvc_close_device(uvc_t hdl);
```

**【参数】**

| 参数名称 | 描述          | 输入/输出 |
|------|-------------|-------|
| hdl  | 用于存储UVC设备句柄 | 输入    |

**【返回值】**

| 返回值             | 描述   |
|-----------------|------|
| UVC_OK          | 成功   |
| UVC_ERROR_FATAL | 致命错误 |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

参考[编程实例](#)。

【举例】

参考[编程实例](#)。

【相关主题】

[uvc\\_open\\_device](#)

7.4.4.13 uvc\_wait\_host

【描述】

等待USB HOST连接。

【语法】

```
int uvc_wait_host(uvc_t hdl, int wait_option, int *connected);
```

【参数】

| 参数名称        | 描述                                                                                                                                | 输入/输出 |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------|-------|
| hdl         | UVC设备句柄                                                                                                                           | 输入    |
| wait_option | 等待连接时的状态： <ul style="list-style-type: none"><li>• UVC_WAIT_HOST_NOP：立即返回当前状态</li><li>• UVC_WAIT_HOST_FOREVER：阻塞，直至连接再返回</li></ul> | 输入    |
| connected   | 当前连接状态： <ul style="list-style-type: none"><li>• 0：未连接</li><li>• 1：已连接</li></ul>                                                   | 输出    |

【返回值】

| 返回值               | 描述       |
|-------------------|----------|
| UVC_OK            | 成功       |
| UVC_ERROR_NOMATCH | UVC状态不匹配 |
| UVC_ERROR_HANDLE  | UVC句柄无效  |
| UVC_ERROR_PTR     | 指针非法     |

| 返回值             | 描述   |
|-----------------|------|
| UVC_ERROR_VALUE | 取值非法 |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

参考[编程实例](#)。

【举例】

参考[编程实例](#)。

【相关主题】

[uvc\\_close\\_device](#)

7.4.4.14 uvc\_get\_state

【描述】

获取UVC设备的运行状态。

【语法】

```
int uvc_get_state(uvc_t hdl, uint32_t *state);
```

【参数】

| 参数名称  | 描述                                                                                                                                           | 输入/输出 |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------|-------|
| hdl   | UVC设备句柄                                                                                                                                      | 输入    |
| state | UVC设备运行状态 <ul style="list-style-type: none"><li>• UVC_STATE_IDLE：空闲状态</li><li>• UVC_STATE_CONN：连接状态</li><li>• UVC_STATE_CLOSE：关闭状态</li></ul> | 输出    |

【返回值】

| 返回值              | 描述      |
|------------------|---------|
| UVC_OK           | 成功      |
| UVC_ERROR_HANDLE | UVC句柄无效 |
| UVC_ERROR_PTR    | 指针非法    |

| 返回值             | 描述   |
|-----------------|------|
| UVC_ERROR_VALUE | 取值非法 |

**【需求】**

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

**【注意】**

无

**【举例】**

无

**【相关主题】**

无

#### 7.4.4.15 uvc\_recv\_pack

**【描述】**

UVC零拷贝模式发送数据包。

**【语法】**

```
int uvc_recv_pack(const struct uvc_pack *pack);
```

**【参数】**

| 参数名称 | 描述          | 输入/输出 |
|------|-------------|-------|
| pack | 指向uvc数据包的指针 | 输入    |

**【返回值】**

| 返回值              | 描述      |
|------------------|---------|
| UVC_OK           | 成功      |
| UVC_ERROR_MEMORY | 内存申请失败  |
| UVC_ERROR_HANDLE | UVC句柄无效 |
| UVC_ERROR_PTR    | 指针非法    |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

参考[编程实例](#)。

【举例】

参考[编程实例](#)。

【相关主题】

[uvc\\_video\\_stop](#)

7.4.4.16 uvc\_video\_stop

【描述】

停止UVC视频传输。

【语法】

```
int uvc_video_stop(uvc_t hdl);
```

【参数】

| 参数名称 | 描述      | 输入/输出 |
|------|---------|-------|
| hdl  | UVC设备句柄 | 输入    |

【返回值】

| 返回值               | 描述       |
|-------------------|----------|
| UVC_OK            | 成功       |
| UVC_ERROR_NOMATCH | UVC状态不匹配 |
| UVC_ERROR_HANDLE  | UVC句柄无效  |
| UVC_ERROR_PTR     | 指针非法     |
| UVC_ERROR_VALUE   | 取值非法     |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

参考[编程实例](#)。

【举例】

参考[编程实例](#)。

【相关主题】

[uvc\\_rcv\\_pack](#)

7.4.4.17 uvc\_format\_info\_get

【描述】

获取新码流格式详细信息。

【语法】

```
int uvc_format_info_get(struct uvc_format_info *info);
```

【参数】

| 参数名称 | 描述                                                                                                                                                                               | 输入/输出 |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| info | struct uvc_format_info{<br>uint32_t width; // 分辨率，宽<br>uint32_t height; // 分辨率，高<br>uint32_t format; // 码流格式<br>uint32_t status; // 1-格式切换中；0-切换完成<br>} __attribute__((packed)); | 输出    |

【返回值】

| 返回值               | 描述       |
|-------------------|----------|
| UVC_OK            | 成功       |
| UVC_ERROR_NOMATCH | UVC状态不匹配 |
| UVC_ERROR_PTR     | 指针非法     |
| UVC_ERROR_FATAL   | 致命错误     |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[uvc\\_format\\_status\\_check](#)

7.4.4.18 uvc\_format\_status\_check

【描述】

码流格式切换事件。

【语法】

```
enum format_switch_status uvc_format_status_check(void);
```

【参数】

无

【返回值】

| 返回值                   | 描述     |
|-----------------------|--------|
| FORMAT_SWITCH_FINISH  | 格式切换完成 |
| FORMAT_SWITCH_PENDING | 格式切换中  |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[uvc\\_format\\_info\\_get](#)

7.4.4.19 hi\_camera\_register\_cmd

【描述】

应用注册UVC扩展命令，提供底层驱动调用。

【语法】

```
int hi_camera_register_cmd(const struct uvc_camera_cmd *cmd);
```



【参数】

| 参数名称 | 描述                                                                                                                                                                                                                                                                    | 输入/输出 |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| cmd  | UVC扩展命令:<br>struct uvc_camera_cmd {<br>uint32_t id;     // 事件码<br>uint8_t name[32]; // 名称<br>camera_control_func uvc_control_func;<br>// 处理函数指针信息的数组<br>};<br>typedef uint32_t (*camera_control_func)(void *buf, uint32_t len, uint32_t event_id, uint32_t cmd_type); | 输入    |

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |

【需求】

- 头文件：f\_uvc.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.20 uac\_wait\_host

【描述】

等待USB HOST连接。

【语法】

```
int uac_wait_host(int wait_option);
```

【参数】

| 参数名称        | 描述                                                                                                                            | 输入/输出 |
|-------------|-------------------------------------------------------------------------------------------------------------------------------|-------|
| wait_option | 等待连接时的状态： <ul style="list-style-type: none"><li>UAC_WAIT_HOST_NOP：立即返回当前状态</li><li>UAC_WAIT_HOST_FOREVER：阻塞，直至连接再返回</li></ul> | 输入    |

【返回值】

| 返回值               | 描述    |
|-------------------|-------|
| UAC_OK            | 成功    |
| UAC_ERROR_NOMATCH | 状态不匹配 |
| UAC_ERROR_PTR     | 指针非法  |
| UAC_ERROR_VALUE   | 参数非法  |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[fuac\\_send\\_message](#)

7.4.4.21 fuac\_send\_message

【描述】

音频数据传输发送。

【语法】

```
int fuac_send_message(const void *buf,int len);
```

【参数】

| 参数名称 | 描述       | 输入/输出 |
|------|----------|-------|
| buf  | 待传输的音频数据 | 输入    |

| 参数名称 | 描述         | 输入/输出 |
|------|------------|-------|
| len  | 待传输的音频数据长度 | 输入    |

【返回值】

| 返回值                  | 描述    |
|----------------------|-------|
| UAC_OK               | 成功    |
| UAC_ERROR_MEMORY     | 内存不足  |
| UAC_ERROR_PTR        | 指针非法  |
| UAC_ERROR_DISCONNECT | 设备已断开 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.22 fuac\_receive\_message

【描述】

音频数据传输接收。

【语法】

```
int fuac_receive_message(void *buf, int len, int timeout);
```

【参数】

| 参数名称    | 描述          | 输入/输出 |
|---------|-------------|-------|
| buf     | 待接收的音频数据缓存  | 输入    |
| len     | 接收音频数据缓存大小  | 输入    |
| timeout | 接收音频数据的超时时间 | 输入    |

【返回值】

| 返回值                   | 描述          |
|-----------------------|-------------|
| int                   | 接收音频数据的实际大小 |
| -UAC_ERROR_MEMORY     | 内存不足        |
| -UAC_ERROR_PTR        | 指针非法        |
| -UAC_ERROR_DISCONNECT | 设备已断开       |
| -UAC_ERROR_TIMEOUT    | 等待超时        |
| -UAC_ERROR_EVENT      | 错误事件        |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.23 fuac\_opts\_set

【描述】

设置uac1.0音频参数。

【语法】

```
int fuac_opts_set(const struct uac_opts *opts);
```

【参数】

| 参数名称 | 描述      | 输入/输出 |
|------|---------|-------|
| opts | 音频参数结构体 | 输入    |

【返回值】

| 返回值 | 描述  |
|-----|-----|
| 0   | 成功  |
| -1  | 空指针 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[fuac\\_get\\_opts](#)

7.4.4.24 fuac\_get\_opts

【描述】

获取uac1.0音频参数。

【语法】

```
int fuac_get_opts(struct uac_opts *opts);
```

【参数】

| 参数名称 | 描述      | 输入/输出 |
|------|---------|-------|
| opts | 音频参数结构体 | 输出    |

【返回值】

| 返回值            | 描述   |
|----------------|------|
| 0              | 成功   |
| -UAC_ERROR_PTR | 指针非法 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[fuac\\_opts\\_set](#)

7.4.4.25 fuac\_rate\_get

【描述】

获取uac1.0音频采样率。

【语法】

```
uint32_t fuac_rate_get(void);
```

【参数】

无

【返回值】

| 返回值          | 描述            |
|--------------|---------------|
| uint32_t类型数值 | uac1.0当前音频采样率 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.26 fuac2\_get\_opts

【描述】

获取音频2.0协议的参数，包括通道数等。

【语法】

```
int fuac2_get_opts(struct uac2_opts *opts);
```

【参数】

| 参数名称 | 描述         | 输入/输出 |
|------|------------|-------|
| opts | 当前的音频参数结构体 | 输出    |

【返回值】

| 返回值            | 描述  |
|----------------|-----|
| 0              | 成功  |
| -UAC_ERROR_PTR | 空指针 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[fuac2\\_set\\_opts](#)

7.4.4.27 fuac2\_set\_opts

【描述】

设置uac2.0音频参数。

【语法】

```
int fuac2_set_opts(const struct uac2_opts *opts);
```

【参数】

| 参数名称 | 描述      | 输入/输出 |
|------|---------|-------|
| opts | 音频参数结构体 | 输入    |

【返回值】

| 返回值              | 描述       |
|------------------|----------|
| 0                | 成功       |
| -UAC_ERROR_PTR   | 空指针      |
| -UAC_ERROR_VALUE | 不支持的参数设置 |

【需求】

- 头文件：f\_uac.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

[fuac2\\_get\\_opts](#)

7.4.4.28 usb\_dfu\_init\_env\_entities

【描述】

初始化配置DFU接口功能，配置环境变量（目前只支持RAM）。

【语法】

```
int usb_dfu_init_env_entities(const char *type, char *envstr, const char *devstr);
```

【参数】

| 参数名称   | 描述           | 输入/输出 |
|--------|--------------|-------|
| type   | DFU功能，只支持RAM | 输入    |
| envstr | DFU环境参数      | 输入    |
| devstr | 未使用          | 输入    |

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |



【需求】

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.29 usb\_dfu\_update\_status

【描述】

获取升级事件。

【语法】

```
uint32_t usb_dfu_update_status(void);
```

【参数】

无

【返回值】

| 返回值 | 描述    |
|-----|-------|
| 0   | 未启动升级 |
| 1   | 升级中   |

【需求】

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.30 usb\_dfu\_update\_size\_get

【描述】

获取升级文件实际大小。

【语法】

```
uint64_t *usb_dfu_update_size_get(void);
```

【参数】

无

【返回值】

| 返回值        | 描述            |
|------------|---------------|
| uint64_t * | 升级文件实际大小的保存地址 |

【需求】

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.31 usb\_dfu\_get\_entity

【描述】

获取列表中指定的DFU结构体变量。

【语法】

```
struct usb_dfu_entity *usb_dfu_get_entity(int alter);
```

【参数】

| 参数名称  | 描述             | 输入/输出 |
|-------|----------------|-------|
| alter | 获取列表中存储变量的序列号。 | 输入    |

【返回值】

| 返回值                     | 描述                                |
|-------------------------|-----------------------------------|
| struct usb_dfu_entity * | DFU结构体变量，存储当前使用的DFU各个参数值，参见结构体声明。 |

【需求】

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.32 usb\_dfu\_read

【描述】

获取指定DFU列表中的一定大小的缓冲数据。

【语法】

```
int usb_dfu_read(struct usb_dfu_entity *dfu, void *buf, int size, uint32_t blk_seq_num);
```

【参数】

| 参数名称        | 描述                                         | 输入/输出 |
|-------------|--------------------------------------------|-------|
| dfu         | 获取缓冲数据所在的DFU列表结构信息，通过usb_dfu_get_entity获取。 | 输入    |
| buf         | 接收缓存buffer。                                | 输入    |
| size        | 接收数据大小。                                    | 输入    |
| blk_seq_num | 获取的次数计数，须是连续自增值。                           | 输入    |

【返回值】

| 返回值 | 描述         |
|-----|------------|
| int | 获取到的数据实际长度 |

【需求】

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.33 hid\_report\_descriptor\_info

【描述】

配置报告描述符信息，设置报告数据格式和功能。

【语法】

```
int hid_report_descriptor_info(const void *report_desc, size_t report_desc_len);
```

【参数】

| 参数名称            | 描述       | 输入/输出 |
|-----------------|----------|-------|
| report_desc     | 报告描述符信息  | 输入    |
| report_desc_len | 报告描述符总大小 | 输入    |

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |
| -1  | 失败 |

【需求】

- 头文件：f\_hid.h
- 库文件：libusb\_device.a

【注意】

报告描述符的定义要遵循标准HID协议，与VFS模块接口配套使用。

【举例】

请参考HID官方文档：《Device Class Definition for Human interface Devices》和《HID Usage Tables》。

【相关主题】

无

7.4.4.34 hid\_add\_report\_descriptor

【描述】

配置多个报告表描述符，设置报告表数据格式、功能和接口协议。

【语法】

```

int hid_add_report_descriptor(const uint8_t *report_desc, size_t report_desc_len, uint8_t protocol);

```

【参数】

| 参数名称            | 描述                                   | 输入/输出 |
|-----------------|--------------------------------------|-------|
| report_desc     | 报告描述符信息                              | 输入    |
| report_desc_len | 报告描述符总大小                             | 输入    |
| protocol        | 默认值为0，如果需要在支持启动接口，需要传入1或者2，1为键盘；2为鼠标 | 输入    |

【返回值】

| 返回值          | 描述           |
|--------------|--------------|
| report_index | 成功（对应的报表索引值） |
| -1           | 失败           |

【需求】

- 头文件：f\_hid.h
- 库文件：libusb\_device.a

【注意】

报告描述符的定义要遵循标准HID协议，与fhid\_send\_data和fhid\_rcv\_data接口配套使用。

【举例】

请参考HID官方文档：《 Device Class Definition for Human interface Devices 》和《 HID Usage Tables 》。

【相关主题】

无

7.4.4.35 fhid\_send\_data

【描述】

HID数据传输发送。

【语法】

```
ssize_t fhid_send_data (uint8_t report_index, const char *buf, size_t buflen);
```

【参数】

| 参数名称         | 描述                                       | 输入/输出 |
|--------------|------------------------------------------|-------|
| report_index | 报表索引值，通过hid_add_report_descriptor接口返回值得到 | 输入    |
| buf          | 需要发送的HID数据                               | 输入    |
| buflen       | HID数据的总大小                                | 输入    |

【返回值】

| 返回值     | 描述            |
|---------|---------------|
| ssize_t | 已发送HID数据的实际大小 |
| -1      | 失败            |

【需求】

- 头文件：f\_hid.h
- 库文件：libusb\_device.a

【注意】

无

【举例】

无

【相关主题】

无

7.4.4.36 fhid\_rcv\_data

【描述】

HID数据传输接收。

【语法】

```
ssize_t fhid_rcv_data(uint8_t report_index, char *buf, size_t buflen);
```

## 【参数】

| 参数名称         | 描述                                       | 输入/输出 |
|--------------|------------------------------------------|-------|
| report_index | 报表索引值，通过hid_add_report_descriptor接口返回值得到 | 输入    |
| buf          | 需要接收HID数据的缓存                             | 输入    |
| buflen       | 接收HID数据缓存的长度                             | 输入    |

## 【返回值】

| 返回值     | 描述         |
|---------|------------|
| ssize_t | 获取到的数据实际长度 |
| -1      | 失败         |

## 【需求】

- 头文件：f\_hid.h
- 库文件：libusb\_device.a

## 【注意】

由于此接口存在读数据阻塞机制，所以只能在任务中调用接口，不能在中断回调调用。

## 【举例】

无

## 【相关主题】

无

### 7.4.4.37 u3g\_write\_from\_shell

## 【描述】

对4G模组发送串口命令

## 【语法】

```
uint32_t u3g_write_from_shell(int argc, const char **argv);
```

## 【参数】

| 参数名称 | 描述               | 输入/输出 |
|------|------------------|-------|
| argc | 参数个数             | 输入    |
| argv | 指针数组，每个元素指向一个字符串 | 输入    |

【返回值】

| 返回值     | 描述 |
|---------|----|
| LOS_OK  | 成功 |
| LOS_NOK | 失败 |

【需求】

- 头文件：usb\_host.h
- 库文件：libusb\_base.a

【注意】

每种4G模组对应的设备节点并不完全相同，有些设备节点是/dev/ttyUSB0，有些是/dev/ttyUSB1，因此要根据模组型号，修改对应设备的端口号。

【举例】

```
static void usb_4g_modem(void)
{
 const char *apOpenCmd[] = {"open", NULL};
 const char *apSetModeCmd[] = {"at+qcfg=\"usbnet\",3", NULL};
 const char *apDialCmd[] = {"at+qnetdevctl=1,1", NULL}; // EC200T模组进行拨号
 sleep(10);
 u3g_write_from_shell(1, apOpenCmd);
 sleep(1);
 u3g_write_from_shell(1, apSetModeCmd);
 sleep(1);
 u3g_write_from_shell(1, apDialCmd);
 sleep(1);
}
```

【相关主题】

无

7.4.4.38 uvc\_get\_frame\_buf

【描述】

获取帧缓冲区的地址。

【语法】

```
uint8_t *uvc_get_frame_buf(struct file *filep, uint32_t index, uint32_t offset);
```

【参数】

| 参数名称  | 描述                      | 输入/输出 |
|-------|-------------------------|-------|
| filep | 字符设备文件句柄，通过文件描述符fd进行获取。 | 输入    |
| index | 当前缓存帧的索引号。              | 输入    |



| 参数名称   | 描述                              | 输入/输出 |
|--------|---------------------------------|-------|
| offset | 帧数据内容的起点，通过VIDIOC_QUERYBUF查询申请。 | 输入    |

**【返回值】**

| 返回值       | 描述        |
|-----------|-----------|
| uint8_t * | 返回帧缓冲区的地址 |

**【需求】**

- 头文件：usb\_v4l2.h
- 库文件：libusb\_base.a

**【注意】**

无

**【举例】**

无

**【相关主题】**

无

### 7.4.4.39 usb\_dfu\_free\_entities

**【描述】**

释放DFU相关资源。

**【语法】**

```
void usb_dfu_free_entities(void)
```

**【参数】**

无

**【返回值】**

无

**【需求】**

- 头文件：f\_dfu.h
- 库文件：libusb\_device.a

**【注意】**

无

**【举例】**

无

【相关主题】

无

## 7.5 UART 驱动开发

### 7.5.1 概述

UART，通用异步收发传输器，英文全称是Universal Asynchronous Receiver/Transmitter。在嵌入式设计中，UART用来与PC进行通信，包括与监控调试器和其它器件，如EEPROM通信。

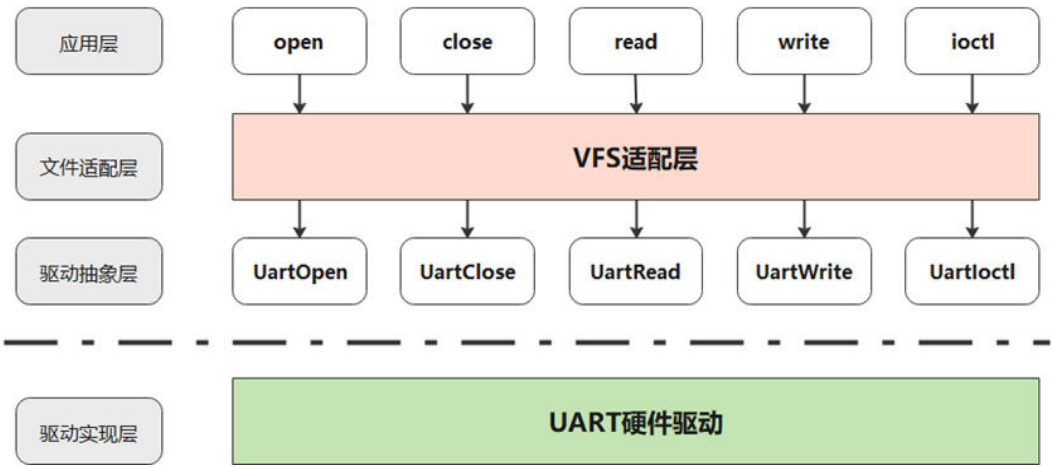
Huawei\_LiteOS UART驱动模块的基本框架包括文件适配层、驱动抽象层、驱动实现层。

文件适配层：采用LOSCFG\_FS\_VFS管控，可裁剪，提供通过文件方式控制uart。

驱动抽象层：提供对外接口，提供通过struct LosDevice设备文件方式控制uart。

驱动实现层：uart控制器硬件实现层，不提供对外接口。

图 7-8 UART 驱动基本框架



### 运作机制

UART驱动主要开发流程如下：

- UART驱动加载

UART驱动的加载由驱动抽象层提供加载接口，系统在启动过程中注册驱动。

- UART设备加载

UART设备由系统根据需要及实际情况进行加载。

- UART数据收发

驱动抽象层提供裸读写接口，文件适配层提供VFS文件方式读写。

## 7.5.2 开发指导

### 7.5.2.1 裸读写 UART

UART驱动抽象层提供如下的接口：

| 接口名                                                                     | 描述                                      |
|-------------------------------------------------------------------------|-----------------------------------------|
| struct LosDevice *UartDevGetByPort(int num)                             | 通过串口号获取UART设备。                          |
| int UartOpen(struct LosDevice *dev)                                     | 打开指定的UART设备。                            |
| int UartClose(struct LosDevice *dev)                                    | 关闭指定的UART设备。                            |
| ssize_t UartRead(struct LosDevice *dev, char *buf, size_t count)        | 读取指定的UART设备的数据。                         |
| ssize_t UartWrite(struct LosDevice *dev, const char *buf, size_t count) | 向指定的UART设备写数据。                          |
| int UartIoctl(struct LosDevice *dev, int cmd, unsigned long arg)        | 控制指定的UART设备。（具体支持的控制方式由UART控制器的驱动实现层提供） |
| int UartSuspend(struct LosDevice *dev)                                  | 挂起指定的UART设备。                            |
| int UartResume(struct LosDevice *dev)                                   | 恢复指定的UART设备。                            |

### 7.5.2.2 文件方式读写 UART

参考[VFS](#)，支持open、close、read、write、ioctl操作。

### 7.5.2.3 模块配置

打开菜单，进入Driver → Uart Type菜单，完成UART模块的配置。

| 配置项                        | 描述              | 取值范围   | 默认值 | 依赖                  |
|----------------------------|-----------------|--------|-----|---------------------|
| LOSCFG_DRIVERS_SIMPLE_UART | 简单的UART，只支持输出。  | YES/NO | YES | 无                   |
| LOSCFG_DRIVERS_UART        | 完整的UART，支持输入输出。 | YES/NO | NO  | LOSCFG_DRIVERS_BASE |

## 7.5.3 编程实例

### 7.5.3.1 simple uart

使能LOSCFG\_DRIVERS\_SIMPLE\_UART，simple uart只支持输出。

### 7.5.3.2 general uart

使能LOSCFG\_DRIVERS\_UART，general uart既支持输出，也支持输入。

#### 7.5.3.2.1 裸读写 UART

#### 实例描述

**步骤1** 通过串口号获取UART设备。

**步骤2** 打开串口。

**步骤3** 写串口。

**步骤4** 挂起串口。

**步骤5** 恢复串口。

**步骤6** 关闭串口。

----结束

#### 编程示例

```
#define BUF_SIZE 100

UINT32 TestCase(VOID)
{
 struct LosDevice *uartDev = NULL;
 int ret;
 ssize_t rtRet;
 char dataBuf[BUF_SIZE] = {"test uart\n"};

 uartDev = UartDevGetByPort(0);
 if (uartDev == NULL) {
 return LOS_NOK;
 }

 ret = UartOpen(uartDev);
 if (ret != 0) {
 return LOS_NOK;
 }

 rtRet = UartWrite(uartDev, dataBuf, BUF_SIZE);
 if (rtRet <= 0) {
 goto OUT_CLOSE;
 }

 ret = UartSuspend(uartDev);
 if (ret != 0) {
 goto OUT_CLOSE;
 }

 ret = UartResume(uartDev);
 if (ret != 0) {
 goto OUT_CLOSE;
 }

 ret = UartClose(uartDev);
 if (ret != 0) {
```

```
 return LOS_NOK;
 }

 return LOS_OK;
}

OUT_CLOSE:
(void)UartClose(uartDev);
return LOS_NOK;
}
```

### 7.5.3.2.2 文件读写 UART

#### 实例描述

**步骤1** 使能LOSCFG\_FS\_VFS。

**步骤2** 打开串口。

**步骤3** 写串口。

**步骤4** 挂起串口。

**步骤5** 恢复串口。

**步骤6** 关闭串口。

----结束

#### 编程示例

```
#define BUF_SIZE 100
#define UART_DEV_PATH "/dev/uartdev-0"

UINT32 TestCase(VOID)
{
 int ret;
 int fd;
 char dataBuf[BUF_SIZE] = {"test uart\n"};
 ssize_t writeLen;

 fd = open(UART_DEV_PATH, O_RDWR);
 if (fd == -1) {
 return LOS_NOK;
 }

 writeLen = write(fd, dataBuf, BUF_SIZE);
 if (writeLen <= 0) {
 goto OUT_CLOSE;
 }

 ret = ioctl(fd, PL011_IOCTL_SUSPEND, 0);
 if (ret != 0) {
 goto OUT_CLOSE;
 }

 ret = ioctl(fd, PL011_IOCTL_RESUME, 0);
 if (ret != 0) {
 goto OUT_CLOSE;
 }

 ret = close(fd);
 if (ret != 0) {
 return LOS_NOK;
 }

 return LOS_OK;
}
```

```
OUT_CLOSE:
 (VOID)close(fd);
 return LOS_NOK;
}
```

# 8 配置工具

- 8.1 概述
- 8.2 安装工具
- 8.3 使用指导
- 8.4 打开menuconfig菜单
- 8.5 保存savemenuconfig菜单
- 8.6 make defconfig
- 8.7 make allyesconfig
- 8.8 make allnoconfig
- 8.9 配置项说明

## 8.1 概述

Huawei LiteOS使用Kconfig文件配置系统。所用的Kconfig语言是一种菜单配置语言，config.in和Kconfig都由该语言编写而成。Huawei LiteOS使用python kconfiglib来解析、展示Kconfig文件。解析Kconfig文件后，会在根目录下生成/更新.config文件，同时在开发板的include文件夹下生成menuconfig.h。使用menuconfig前，需要先安装python和kconfiglib。

## 8.2 安装工具

- 安装python 2.7/3.2+。  
下面以python3.10为例介绍安装方法。
  - 命令行方式安装：

```
sudo apt-get install python3.10
```
  - 源码包编译安装：
    - i. 通过官网下载[python源码包](#)。
    - ii. 解压源码包。  
参考如下命令完成解压，将压缩包名替换为实际下载的源码包名：

```
tar -xf Python-3.10.2.tgz
```

iii. 检查依赖。

解压后进入到目录中，执行./configure命令以检查编译与安装python所需的依赖：

```
cd Python-3.10.2
./configure
```

如果没有报错就继续下一步操作，如果存在报错就根据提示安装依赖。

iv. 编译&安装python。

```
sudo make
sudo make install
```

v. 检查python版本并正确链接python命令。

```
python --version
```

如果显示的不是刚刚安装的python版本，则需要执行以下命令来正确链接python命令。

1) 获取python目录，例如对于python 3.10.2，执行如下命令。

```
which python3.10
```

2) 链接python命令到刚刚安装的python包。

将以下命令中的 "python3.10-path" 替换为 "which python3.10" 命令执行后的回显路径：

```
cd /usr/bin
sudo rm python3 && sudo ln -s "python3.10-path" python3
sudo rm python && sudo ln -s python3 python
```

3) 再次检查python版本。

```
python --version
python3 --version
```

- 安装pip包管理工具。

如果pip命令不存在，可以下载pip源码包进行安装。pip依赖setuptools，如果setuptools不存在，也需要安装。

- 命令行方式安装：

```
sudo apt-get install python3-setuptools python3-pip -y
sudo pip3 install --upgrade pip
```

- 源码包方式安装：

- i. 安装setuptools。

点击[setuptools源代码包下载地址](#)，可以参考下面的命令进行安装：

```
sudo unzip setuptools-50.3.2.zip
cd setuptools
sudo python setup.py install
```

### 须知

setuptools最新版本不支持python 2.7，如果使用python 2.7，请下载[setuptools 45.0.0版本](#)以支持python 2.7。

- ii. 安装pip。

点击[pip源代码包下载地址](#)，可以参考下面的命令进行安装：

```
sudo tar -xf pip-20.2.4.tar.gz
cd pip-20.2.4
sudo python setup.py install
```

- 安装kconfiglib库。



- 对于服务器可以联网的情况。

可以直接使用如下命令安装kconfiglib：

```
sudo pip install kconfiglib
```

- 对于服务器不能联网的情况。

可以采用离线的方式安装。首先在其他能联网的环境上[下载kconfiglib](#)，可以下载kconfiglib的wheel文件kconfiglib-14.1.0-py2.py3-none-any.whl或源代码文件kconfiglib-14.1.0.tar.gz，这里以14.1.0版本为例。

- wheel文件的安装，可以参考如下命令：

```
sudo pip install kconfiglib-14.1.0-py2.py3-none-any.whl
```

- 源代码文件的安装，可以参考如下命令：

```
sudo tar -zxvf kconfiglib-14.1.0.tar.gz
cd kconfiglib-14.1.0
sudo python setup.py install
```

## 8.3 使用指导

使用menuconfig配置工具前，请确保已经安装编译Huawei LiteOS的交叉编译器工具链，并加入环境变量。

为满足不同的使用场景，配置工具支持下述命令。根据使用场景，在根目录下执行下述其中一个命令即可。在执行命令前，先根据开发板拷贝tools/build/config/目录下的默认配置文件\${platform}.config到根目录，并重命名为.config。除了.config文件外，还有一个重要的配置文件tools/menuconfig/config.in文件（该文件包含了各个模块的Kconfig文件）。.config文件决定了各个配置项的默认值，config.in文件决定了展示图形化界面的配置项。

## 8.4 打开 menuconfig 菜单

在Huawei\_LiteOS根目录下执行make menuconfig命令或者./build/build\_cmake.sh menuconfig会展示图形化配置界面，用户可以根据需要自行裁剪模块或者修改设置。执行完保存菜单退出，该命令会更新根目录下.config文件。

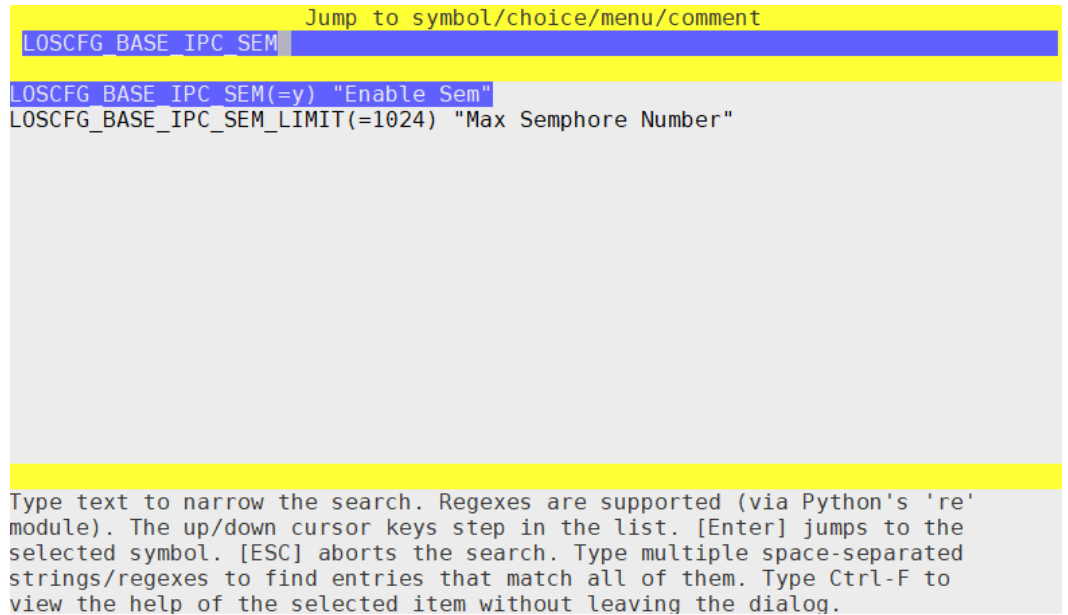
menuconfig的使用方式，主要包括：

- 上下键：选择不同的行，即移动到不同的选项上。
- 空格键/回车：用于开启或关闭选项。
  - a. 开启选项：对应的选项前面会显示“[\*]”，括号里面有一个星号，表示已经开启该选项。
  - b. 关闭选项：对应的选项前面只显示中括号“[]”，括号里面是空。
  - c. 如果选项后面有三个短横线加上一个右箭头，即--->，表示此项下面还有子选项，输入空格键/回车后可以进入子菜单。
- ESC键：返回上一级菜单，或退出menuconfig并提示保存。
- 问号?：展示配置项的帮助信息。
- 斜线/：进入搜索配置项界面，支持配置项的搜索。
- 字母F：进入帮助模式，在界面下方会显示配置项的帮助信息，再次输入字母F可以退出此模式。
- 字母C：进入name模式，在此模式下，会显示配置项对应的宏定义开关，再次输入字母C可以退出此模式。

- 字母A：进入all模式，在此模式下，会展开显示菜单中的所有子选项，再次输入字母A可以退出此模式。
- 字母S：保存配置项。
- 字母Q：退出menuconfig并提示保存。

#### 📖 说明

- 以上字母，不区分大小写。
- 可以通过斜线/进入搜索界面，在输入栏中输入要搜索的配置项宏定义，例如搜索“LOSCFG\_BASE\_IPC\_SEM”，输入该宏定义后，会自动联系出匹配到此关键字的配置项，选中所需要的配置项，按回车键即可进入。



## 8.5 保存 savemenuconfig 菜单

对于Make编译工具：在Huawei\_LiteOS根目录下执行make savemenuconfig命令会解析根目录下的.config文件，并在对应的Huawei\_LiteOS开发板工程的子目录include下生成menuconfig.h文件。

对于CMake编译工具：在Huawei\_LiteOS根目录下执行./build/build\_cmake.sh savemenuconfig会解析根目录下的.config文件，并在对应的Huawei\_LiteOS开发板工程的子目录buildcmakeout/xxx/目录下生成menuconfig.h文件。

## 8.6 make defconfig

在Huawei\_LiteOS根目录下执行make defconfig命令会解析根目录下的.config文件（cmake当前不支持该功能），使所有的配置项尽可能使用其默认配置，并更新.config，同时在对应的Huawei\_LiteOS开发板工程的子目录include下生成menuconfig.h。

## 8.7 make allyesconfig

在Huawei\_LiteOS根目录下执行make allyesconfig命令会解析根目录下的.config文件（cmake当前不支持该功能），使所有的配置项尽可能使能，即设置为Y，并更

新.config，同时在对应的Huawei\_LiteOS开发板工程的子目录include下生成menuconfig.h。

## 8.8 make allnoconfig

在Huawei\_LiteOS根目录下执行make allnoconfig命令会解析根目录下的.config文件（cmake当前不支持该功能），使所有的配置项尽可能禁用，即设置为is not set，并更新.config，同时在对应的Huawei\_LiteOS开发板工程的子目录include下生成menuconfig.h。

## 8.9 配置项说明

### 8.9.1 配置项简介

运行make menuconfig。进入Huawei LiteOS Configuration主界面，目前有Compiler, Targets, Kernel, Debug, SSP等选项。下面对Compiler, Targets和SSP选项进行介绍，其余选项在各个模块章节中均有介绍，这里不再赘述。

### 8.9.2 Compiler 配置

Compiler选项中包含了两个子菜单：Compiler交叉编译器工具链和Optimize Option编译优化选项。

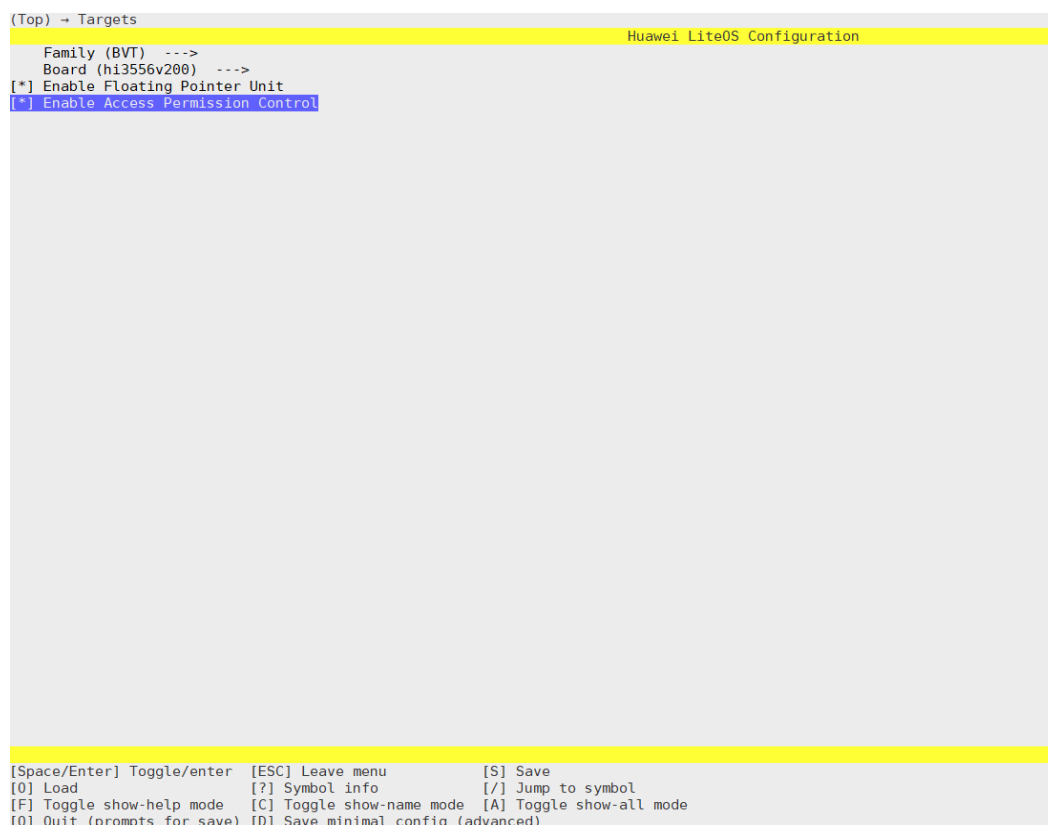
```
(Top) → Compiler
Huawei LiteOS Configuration
Compiler (arm-himix410-linux) --->
Optimize Option (Optimize Speed) --->

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

- 现有两种交叉编译器可选：arm-himix410-linux和arm-none-eabi-，请选择实际使用的编译器。
- 有三种编译优化选项可选：Optimize None（不做编译优化）、Optimize Speed（优化编译速度）和Optimize Size（优化编译体积）。

### 8.9.3 Targets 配置

Targets选项中包含Family（产品）、Board（芯片）、Enable Floating Pointer Unit（使能浮点数）、Enable Access Permission Control（使能访问权限控制）。



- Family：选择产品，其中BVT、DPT、AIOT等都是产品名，SECRET是保密项目，QEMU和STM32是Huawei LiteOS内部使用的。
- Board：选择具体芯片，目前支持hi3556v200，hi3516ev200，hi3556av100，hi3731v110等。
- Enable Access Permission Control：使能访问权限控制，可以根据不同架构使能对应的内存管理单元，包含RiscV下的PMP，Cortex-M架构下的MPU，Cortex-A/R架构下的MMU。

### 8.9.4 SSP 配置

选择Stack Smashing Protector (SSP) Compiler Feature选项，该选项可以打开和关闭堆栈保护功能。

- -fno-stack-protector：关闭堆栈保护。
- -fstack-protector：启用堆栈保护，若C或C++函数的局部变量中含有字符（char）数组（数组大小大于等于4个字节的字符数组），并且不会被优化掉，编译器会为函数插入保护代码。
- -fstack-protector-strong：启用堆栈保护，若C或C++函数中有数组定义并且不会被优化掉，或者存在局部变量的地址引用时，编译器会为函数插入保护代码（默认选项）。
- -fstack-protector-all：为所有C或C++函数插入保护代码，相比上一个，可能会大幅增加性能开销。

推荐使用"-fstack-protector"选项，在兼顾性能的同时，增强了安全性。

- ArchStackGuardInit: liteos值提供伪随机数金丝雀值，需要厂商适配真随机，以保证安全性。

```
mashing Protector (SSP) Compiler Feature → Enable stack buffer overflow detection
Huawei LiteOS Configuration
() -fno-stack-protector
() -fstack-protector
(X) -fstack-protector-strong
() -fstack-protector-all

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```



LiteOS提供弱函数ArchStackGuardInit，采用伪随机数方式初始化\_\_stack\_chk\_guard，如果用户需要达到真随机数，需重定义ArchStackGuardInit。另外部分架构没有随机种子，无法达成伪随机数，因此，用户必须自行验证\_\_stack\_chk\_guard的随机性。

### 8.9.5 NX 配置

Enable Data Sec NX Feature选项，该选项可以打开和关闭数据段不可执行功能。

- 选择该选项建议配合使用系统提供的链接脚本，不使用系统链接脚本时需要参考系统链接脚本编写私有链接脚本。
- 该选项只在支持MMU的架构上起效，不支持MMU的架构请在初始化过程中自行实现NX。

```

(Top) → Kernel
Huawei LiteOS Configuration
[*] Enable Mini Kernel
[] Enable Kernel SMP ----
 Basic Config --->
 Memory Management --->
 Interrupt Management --->
 Exception Management --->
 *- Enable Software Timer --->
 *- Enable Queue --->
 *- Enable Event ----
 *- Enable Mutex --->
 *- Enable Sem --->
[] Enable Data Sec NX Feature
[*] Enable Kernel Printf Frame
[*] Enable Kernel Printf Size Extend
[] Enable Ringbuf
[*] Enable Extend Kernel
[*] C++ Support
[*] Enable Cpu
[*] Enable Cpu include irq
[] Enable Dynamic Load Feature
[*] Enable Low Power Management Framework
 Low Power Management Configure --->
[] Enable Perf Feature
[*] Enable System Resource Check
+++++
[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [/] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

# 9 可维可测

---

- 9.1 Telnet
- 9.2 Shell
- 9.3 调度统计
- 9.4 内存调测方法
- 9.5 MMC维测方法
- 9.6 任务间通信调测方法
- 9.7 异常交互
- 9.8 低功耗调测方法
- 9.9 临终遗言使用方法
- 9.10 魔法键使用方法
- 9.11 栈估算工具使用方法
- 9.12 调试案例

## 9.1 Telnet

### 9.1.1 概述

#### 基本概念

Telnet协议是TCP/IP协议族中的一员，是Internet远程登陆服务的标准协议，也是Internet远程登陆的主要方式。在终端使用者的电脑上使用Telnet程序，用它连接到服务器。终端使用者可以在Telnet程序中输入命令，这些命令会在服务器上运行，就像直接在服务器的控制台上输入一样。

#### Huawei LiteOS Telnet

Telnet是采用命令行进行系统调试的一种工具。用户通过配置电脑IP和网关，确保与开发板在同一网段的局域网内，然后通过windows下cmd.exe工具，执行Telnet IP（开发板IP）进行网络连接，也可以说Telnet是串口的另一种调试方式。

Huawei LiteOS Telnet采用简单的Telnet协议，暂时不需要登入用户名跟密码就可以连接到开发板，主要实现了以下功能：

- 读取用户在键盘上键入的字符，通过TCP协议发送到开发板上。
- 把用户的数据经过处理后，回传到用户的终端上。

## 9.1.2 开发指导

### 开发流程

1. 在Huawei LiteOS Shell中键入命令telnet on启动telnet server。

```
Huawei LiteOS # telnet on
Huawei LiteOS # init telnet.
```

2. 在安装了Telnet客户端的windows系统上，运行cmd.exe命令行工具，输入：telnet + 开发板IP后可以连接到开发板，如下图：

```
C:\Users\Administrator>telnet 192.168.1.1
```

按下回车完成连接后，出现Huawei LiteOS #（Huawei LiteOS Shell提示符）表示已经成功连接到开发板，这时就可以执行Shell命令了。例如，执行 task 查看Huawei LiteOS系统当前的所有任务状态。

```
Huawei LiteOS # task
```

| Name                   | PID | Priority | Status      | StackSize | WaterLine | StackPoint |
|------------------------|-----|----------|-------------|-----------|-----------|------------|
| Swt_Task               | 0x0 | 0        | QueuePend   | 0x6000    | 0x320     | 0x85121038 |
| ui_MainApp             | 0x1 | 10       | Pend        | 0x6000    | 0x16b0    | 0x85126f88 |
| IdleCore000            | 0x2 | 31       | Ready       | 0x400     | 0x154     | 0x851274a4 |
| RcvAndDispatch         | 0x3 | 10       | Ready       | 0x6000    | 0x25c     | 0x8522ed2c |
| SavePara2Flash         | 0x4 | 10       | Ready       | 0x6000    | 0x244     | 0x8529fc94 |
| RecMngMsgRcv           | 0x5 | 10       | Ready       | 0x6000    | 0x984     | 0x852ba924 |
| RecMngRecTimer         | 0x6 | 10       | Ready       | 0x6000    | 0x284     | 0x852c08ec |
| RecMsgRcv              | 0x7 | 10       | Ready       | 0x6000    | 0x23c     | 0x852e3d40 |
| Isprun                 | 0x8 | 10       | Ready       | 0x6000    | 0x151c    | 0x85301424 |
| ProcessEncStreamThread | 0x9 | 2        | PendTimeOut | 0x6000    | 0xbac     | 0x85339cfc |
| aenc_get               | 0xa | 10       | Pend        | 0x6000    | 0x3a4     | 0x8539d128 |
| SendAFrameToAencThread | 0xb | 10       | Ready       | 0x6000    | 0x4130    | 0x853a30e8 |

3. 在Huawei LiteOS Shell中键入telnet off关闭telnet server。

```
Huawei LiteOS # telnet off
telnetd_accept_loop[412] Software caused connection abort
close telnet.
Huawei LiteOS #
```

## 9.1.3 注意事项

### Windows下开启Telnet服务：

- 在Windows下键入telnet+IP，如果提示：Telnet不是外部或内部命令，表示windows下的Telnet服务没有开启，需要用户在本地电脑的控制面板--->程序和功能--->打开和关闭Windows功能中选择开启Telnet服务器和客户端。



### Huawei LiteOS启动Telnet前的准备工作：

- 在Huawei LiteOS上启动Telnet前，要确保网络驱动已经初始化完成，且开发板的网卡是link up状态。
- 在Huawei LiteOS上启动Telnet前，需要确保Lwip已正常启动。使用Lwip需要初始化TCP/IP。

### Telnet使用限制：

- Telnet连接成功后再使用ifconfig设置IP会导致连接断开。
- 暂时无法支持多个客户端（telnet + IP）连接开发板。
- Telnet只支持Shell支持的命令。
- 在以下场景下不允许使用Telnet打印：锁任务、中断回调函数中、软件定时器回调函数中。
- Telnet属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

## 9.2 Shell

### 9.2.1 概述

#### 基本概念

Huawei LiteOS提供Shell命令行，它能够以命令行交互的方式访问操作系统的功能或服务：它接收并解析用户输入的命令，并处理操作系统的输出结果。

### Huawei LiteOS Shell

Huawei LiteOS提供的Shell作为在线调试工具，支持常用的基本调试功能，包含系统、文件、网络、动态加载和Trace等相关命令。同时Huawei LiteOS的Shell可以根据需求新增定制命令。

- 系统相关命令：可以查询系统任务、内核信号量、系统软件定时器、CPU占用率、当前中断等相关信息，详见[系统命令参考](#)。
- 文件相关命令：除了支持基本的ls、cd等功能外，还支持sync。sync用来同步缓存数据（文件系统的数据）到SD卡或Flash中，详见[文件命令参考](#)。
- 网络相关命令：可以查询连接到开发板的其他设备的IP，查询本机IP，测试网络连接，设置开发板的IP和station模式等相关功能，详见[网络命令参考](#)。
- 动态加载相关命令：加载obj文件或so文件，并通过查找已加载obj或so的函数地址，直接调用相关函数，同时用户也可以卸载obj文件或so文件，详见[动态加载命令参考](#)。
- Trace相关命令：可以开启/停止Trace、设置Trace事件、输出Trace缓冲区数据等，详见[Trace命令参考](#)。

## 9.2.2 开发指导

### 使用场景

Shell命令可以通过串口和Telnet工具输入。新增定制的命令，需重新编译链接后才能执行。

### 功能

- 1. Huawei LiteOS提供的Shell命令参见后面“命令参考”章节。
- 2. Huawei LiteOS的Shell模块为用户提供下面几个接口，接口详细信息可以查看API参考。

| 功能分类 | 接口名            | 描述     |
|------|----------------|--------|
| 注册命令 | SHELLCMD_ENTRY | 静态注册命令 |
|      | osCmdReg       | 动态注册命令 |

#### 说明

静态注册命令方式一般用于注册系统常用命令，动态注册命令方式一般用于注册用户命令。

静态注册命令有5个入参，动态注册命令有4个入参。下面除去第一个入参是静态注册独有的，剩余的四个入参两个注册命令是一致的。

- 第一个入参：命令变量名，用于设置链接选项（ build/mk/liteos\_tables\_ldflags.mk的 LITEOS\_TABLES\_LDFLAGS变量）。例如变量名为ls\_shellcmd，链接选项就应该设置为：LITEOS\_TABLES\_LDFLAGS += -uls\_shellcmd。这个入参是静态注册独有的，动态注册中没有这个入参。
- 第二个入参：命令类型，目前支持两种命令类型。
  - CMD\_TYPE\_EX：不支持标准命令参数输入，会把用户填写的命令关键字屏蔽掉。例如：输入ls /ramfs，传入给命令处理函数的参数只有/ramfs，对应于命令处理函数中的argv[0]，而ls命令关键字并不会被传入。
  - CMD\_TYPE\_STD：支持的标准命令参数输入，所有输入的字符都会通过命令解析后被传入。例如：输入ls /ramfs，ls和/ramfs都会被传入命令处理函数，分别对应于命令处理函数中的argv[0]和argv[1]。
- 第三个入参：命令关键字，是命令处理函数在Shell中对应的名称。**命令关键字必须唯一**，即两个不同的命令项不能有相同的命令关键字，否则只会执行其中一个。Shell在执行用户命令时，如果存在多个命令关键字相同的命令，只会执行在“help”命令中排在最前面的那个。
- 第四个入参：命令处理函数的入参最大个数。
  - 静态注册命令暂不支持设置。
  - 动态注册命令支持设置不超过32的入参最大个数，或者设置为XARGS（其在代码中被定义为0xffffffff）表示不限制参数个数。
- 第五个入参：命令处理函数名，即在Shell中执行命令时被调用的函数。

### 配置项

打开menuconfig，进入Debug ---> Enable a Debug Version ---> Enable Shell菜单，完成Shell的配置。

| 配置项                                | 含义                  | 取值范围   | 默认值 | 依赖                                                   |
|------------------------------------|---------------------|--------|-----|------------------------------------------------------|
| LOSCFG_SHELL                       | Shell功能的裁剪开关        | YES/NO | YES | LOSCFG_DEBUG_VERSION(=y) && LOSCFG_DRIVER_S_UART(=y) |
| LOSCFG_SHELL_CONSOLE ( 开源版本无该配置项 ) | 设置Shell直接与Console交互 | YES/NO | YES | LOSCFG_SHELL(=y)                                     |
| LOSCFG_SHELL_UART                  | 设置Shell直接与uart驱动交互  | YES/NO | NO  | LOSCFG_DRIVER_S_UART(=y)                             |

新增命令开发流程

下面以注册系统命令ls为例，介绍新增Shell命令的典型开发流程。

1. 定义Shell命令处理函数。

Shell命令处理函数用于处理注册的命令。例如定义一个命令处理函数osShellCmdLs，处理ls命令，并在头文件中声明命令处理函数原型。

```
int osShellCmdLs(int argc, const char **argv);
```

须知

命令处理函数的参数与C语言中main函数的参数类似，包括两个入参：

- argc: Shell命令的参数个数。个数中是否包括命令关键字，和注册命令时的命令类型有关。
- argv: 为指针数组，每个元素指向一个字符串，该字符串就是执行shell命令时传入命令处理函数的参数。参数中是否包括命令关键字，和注册命令时的命令类型有关。

2. 注册命令。

有静态注册命令和系统运行时动态注册命令两种注册方式。

- 静态注册ls命令：

```
#include "shcmd.h"
SHELLCMD_ENTRY(ls_shellcmd, CMD_TYPE_EX, "ls", XARGS, (CMD_CBK_FUNC)osShellCmdLs);
```
- 动态注册ls命令：

```
#include "shell.h"
osCmdReg(CMD_TYPE_EX, "ls", XARGS, (CMD_CBK_FUNC)osShellCmdLs);
```

- 3. 对于静态注册命令方式，在build/mk/liteos\_tables\_ldflags.mk中设置链接选项（LITEOS\_TABLES\_LDFLAGS变量）。
- 4. 打开菜单使能Shell，详见配置项。
- 5. 编译烧录系统后，可以执行新增的Shell命令。

## 9.2.3 注意事项

- 默认模式下支持英文输入。如果在UTF8格式下输入了中文字符，只能通过回退三次来删除。
- 支持Shell命令、文件名及目录名的Tab键联想补全，若有多个匹配项则补全共同字符，并打印多个匹配项。对于过多的匹配项（打印多于24行），将会进行打印询问（Display all num possibilities? (y/n)），可输入y选择全部打印，或输入n退出打印，选择全部打印并打印超过24行后，会进行--More--提示，此时回车键继续打印，q键退出（支持Ctrl+c退出）。
- 不建议使用Shell命令操作/dev目录下的设备文件，可能会引起不可预知的结果。
- Shell属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

## 9.2.4 静态注册编程实例

### 实例描述

本实例演示如何使用静态注册命令方式新增一个名为test的Shell命令。编译工具以Make为例介绍。

1. 定义一个新增命令所要调用的命令处理函数cmd\_test。
2. 使用SHELLCMD\_ENTRY函数添加新增命令项。
3. 在liteos\_tables\_ldflags.mk中添加链接该新增命令项参数。
4. 打开菜单使能Shell。
5. 重新编译代码后运行。

### 编程示例

1. 定义命令所要调用的命令处理函数cmd\_test：

```
#include "shell.h"
#include "shcmd.h"

int cmd_test(void)
{
 printf("hello everybody!\n");
 return 0;
}
```
2. 添加新增命令项：

```
SHELLCMD_ENTRY(test_shellcmd, CMD_TYPE_EX, "test", 0, (CMD_CBK_FUNC)cmd_test);
```
3. 在链接选项中添加链接该新增命令项参数：l  
在build/mk/liteos\_tables\_ldflags.mk中LITEOS\_TABLES\_LDFLAGS项下添加-utest\_shellcmd。
4. 打开菜单使能Shell，即设置LOSCFG\_SHELL=y。
5. 重新编译代码：

```
make clean;make
```

### 结果验证

烧录新系统镜像后，重启系统。使用help命令查看当前系统中所有注册的命令，可以看到test命令已经注册。

## 9.2.5 动态注册编程实例

### 实例描述

本实例演示如何使用动态注册命令方式新增一个名为test的Shell命令。编译工具以Make为例介绍。

1. 定义一个新增命令所要调用的命令处理函数cmd\_test。
2. 使用osCmdReg函数添加新增命令项。
3. 打开菜单使能Shell。
4. 重新编译代码后运行。

### 编程示例

1. 定义命令所要调用的命令处理函数cmd\_test:

```
#include "shell.h"
#include "shcmd.h"

int cmd_test(void)
{
 printf("hello everybody!\n");
 return 0;
}
```

2. 在app\_init函数中调用osCmdReg函数动态注册命令:

```
void app_init(void)
{

 osCmdReg(CMD_TYPE_EX, "test", 0,(CMD_CBK_FUNC)cmd_test);

}
```

3. 打开菜单使能Shell，即设置LOSCFG\_SHELL=y。
4. 重新编译代码：  
make clean;make

### 结果验证

烧录新系统镜像后，重启系统。使用help命令查看当前系统中所有注册的命令，可以看到test命令已经注册。

## 9.2.6 系统命令参考

### 9.2.6.1 使能系统命令

使用Shell中的系统命令前，需要打开menuconfig使能Shell，详见[配置项](#)。

### 9.2.6.2 help

#### 命令功能

help命令用于显示当前操作系统内所有的Shell命令。

命令格式

help

使用实例

举例：  
输入help。

输出说明

执行help，输出当前系统内的所有Shell命令：

```
Huawei LiteOS # help
*****shell commands:*****

cpup date dlock dmesg free help hwi
log memcheck mutex queue sem stack swtmr
systeminfo task uname watch
```

9.2.6.3 date

命令功能

date命令用于查询及设置系统时间。

命令格式

date  
date --help  
date +Format  
date -s YY/MM/DD  
date -s hh:mm:ss  
date -r Filename（开源版本暂不支持该命令）

参数说明

表 9-1 参数说明

| 参数          | 参数说明                          | 取值范围          |
|-------------|-------------------------------|---------------|
| --help      | 使用帮助                          | N/A           |
| +Format     | 根据Format格式打印时间                | --help中列出的占位符 |
| -s YY/MM/DD | 设置系统时间，用“/”分割的年月日             | >= 1970/01/01 |
| -s hh:mm:ss | 设置系统时间，用“:”分割的时分秒             | N/A           |
| -r Filename | 查询指定文件的修改时间，需要使能LOSCFG_FS_VFS | N/A           |

使用指南

- 该命令依赖于LOSCFG\_SHELL\_EXTENDED\_CMDs，该宏开关可在menuconfig菜单项中开启“Enable Shell Ext CMDs”使能。  
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell Ext CMDs
- date参数缺省时，默认显示当前系统时间。
- help、+Format、-s、-r不能混合使用。

使用实例

举例：  
输入date +%Y--%m--%d。

输出说明

执行date +%Y--%m--%d，按其指定格式打印当前系统时间：

```
Huawei LiteOS # date +%Y--%m--%d
2021--01--20
```

9.2.6.4 uname

命令功能

uname命令用于显示操作系统的名称，系统编译时间，版本信息等。

命令格式

uname [-a | -s | -t | -v | --help]

参数说明

| 参数     | 参数说明           |
|--------|----------------|
| -a     | 显示全部信息         |
| -s     | 显示操作系统名称       |
| -t     | 显示系统的编译时间      |
| -v     | 显示版本信息         |
| --help | 显示uname命令的帮助信息 |

使用指南

- 参数缺省时，默认显示操作系统名称。
- uname的参数不能混合使用。

使用实例

举例：  
输入uname -a。

输出说明

执行uname -a，获取系统信息：

```
Huawei LiteOS # uname -a
Huawei LiteOS V200R002C00SPC050B011 Huawei LiteOS 3.2.2 Mar 30 2019 16:09:28
```

9.2.6.5 task

命令功能

task命令用于查询系统任务信息。

命令格式

task [/D]

参数说明

表 9-2 参数说明

| 参数 | 参数说明                      | 取值范围            |
|----|---------------------------|-----------------|
| ID | 任务ID号，输入形式以十进制表示或十六进制表示皆可 | [0, 0xFFFFFFFF] |

使用指南

- 参数缺省时，默认打印全部运行任务信息。
- task后加ID，当ID参数在[0, 64]范围内时，返回指定ID号任务的任务名、任务ID、任务的调用栈信息（最大支持15层调用栈），其他取值时返回参数错误的提示。如果指定ID号对应的任务未创建，则提示。
- 如果在task命令中，发现任务是Invalid状态，请确保pthread\_create创建函数时有进行如下操作之一，否则资源无法正常回收。
  - 选择的是阻塞模式应该调用pthread\_join()函数。
  - 选择的是非阻塞模式应该调用pthread\_detach()函数。
  - 如果不想调用前面两个接口，就需要设置pthread\_attr\_t状态为PTHREAD\_STATE\_DETACHED，将attr参数传入pthread\_create，此设置和调用pthread\_detach函数一样，都是非阻塞模式。

使用实例

举例1：输入task 6。  
举例2：输入task。



输出说明

执行task 0xb，查询ID号为b的任务信息：

```
Huawei LiteOS # task 0xb
TaskName = SerialEntryTask
TaskId = 0xb
*****backtrace begin*****
traceback 0 -- lr = 0x1d804 fp = 0xa86bc
traceback 1 -- lr = 0x1da40 fp = 0xa86e4
traceback 2 -- lr = 0x20154 fp = 0xa86fc
traceback 3 -- lr = 0x258e4 fp = 0xa8714
traceback 4 -- lr = 0x242f4 fp = 0xa872c
traceback 5 -- lr = 0x123e4 fp = 0xa8754
traceback 6 -- lr = 0x2a9d8 fp = 0xb0b0b0b
```

执行task，查询所有任务信息：

| Huawei LiteOS # task |               |        |           |           |           |           |            |            |     |  |
|----------------------|---------------|--------|-----------|-----------|-----------|-----------|------------|------------|-----|--|
| Name                 | TaskEntryAddr | TID    | Priority  | Status    | StackSize | WaterLine | StackPoint | TopOfStack |     |  |
| EventMask            | SemID         | CPUUSE | CPUUSE10s | CPUUSE1s  | MEMUSE    |           |            |            |     |  |
| -----                |               |        |           |           |           |           |            |            |     |  |
| Swt_Task             | 0x40002770    | 0x0    | 0         | QueuePend | 0x6000    | 0x2cc     | 0x4015a318 | 0x401544e8 |     |  |
| 0x0                  | 0xffff        | 0.0    | 0.0       | 0.0       | 0         |           |            |            |     |  |
| IdleCore000          | 0x40002dc8    | 0x1    | 31        | Ready     | 0x400     | 0x15c     | 0x4015a7f4 | 0x4015a550 | 0x0 |  |
| 0xffff               | 98.6          | 98.2   | 99.9      | 0         |           |           |            |            |     |  |
| system_wq            | 0x400b80fc    | 0x3    | 1         | Pend      | 0x6000    | 0x244     | 0x40166928 | 0x40160ab8 | 0x1 |  |
| 0xffff               | 0.0           | 0.0    | 0.0       | 0         |           |           |            |            |     |  |
| SerialShellTask      | 0x40090158    | 0x5    | 9         | Running   | 0x3000    | 0x55c     | 0x40174918 | 0x40171e70 |     |  |
| 0xfff                | 0xffff        | 1.2    | 1.7       | 0.0       | 48        |           |            |            |     |  |
| SerialEntryTask      | 0x4008fe30    | 0x6    | 9         | Pend      | 0x1000    | 0x2c4     | 0x40175c78 | 0x40174e88 | 0x1 |  |
| 0xffff               | 0.0           | 0.0    | 0.0       | 72        |           |           |            |            |     |  |

 说明

## 1. 输出项说明：

- Name：任务名。
- TID：任务ID。
- Priority：任务的优先级。
- Status：任务当前的状态。
- StackSize：任务栈大小。
- WaterLine：该任务栈已经被使用的内存大小。
- StackPoint：任务栈指针，表示栈的起始地址。
- TopOfStack：栈顶的地址。
- EventMask：当前任务的事件掩码，没有使用事件，则默认任务事件掩码为0（如果任务中使用多个事件，则显示的是最近使用的事件掩码）。
- SemID：当前任务拥有的信号量ID，没有使用信号量，则默认0xFFFF（如果任务中使用了多个信号量，则显示的是最近使用的信号量ID）。
- CPUUSE：系统启动以来的CPU占用率。
- CPUUSE10s：系统最近10秒的CPU占用率。
- CPUUSE1s：系统最近1秒的CPU占用率。
- MEMUSE：截止到当前时间，任务所申请的内存大小，以字节为单位显示。MEMUSE仅针对系统内存池进行统计，不包括中断中处理的内存和任务启动之前的内存。  
任务申请内存，MEMUSE会增加，任务释放内存，MEMUSE会减小，所以MEMUSE会有正值和负值的情况。
  - 1）MEMUSE为0：说明该任务没有申请内存，或者申请的内存和释放的内存相同。
  - 2）MEMUSE为正值：说明该任务中有内存未释放。
  - 3）MEMUSE为负值：说明该任务释放的内存大于申请的内存。

## 2. 系统任务说明（Huawei LiteOS系统初始任务有以下几种）：

- Swt\_Task：软件定时器任务，用于处理软件定时器超时回调函数。
- IdleCore000：系统空闲时执行的任务。
- system\_wq：系统默认工作队列处理任务。
- SerialEntryTask：从底层buf读取用户的输入，初步解析命令，例如tab补全，方向键等。
- SerialShellTask：接收命令后进一步解析，并查找匹配的命令处理函数，进行调用。

## 3. 任务状态说明：

- Ready：任务处于就绪状态。
- Pend：任务处于阻塞状态。
- PendTime：阻塞的任务处于等待超时状态。
- Suspend：任务处于挂起状态。
- Running：该任务正在运行。
- Delay：任务处于延时等待状态。
- SuspendTime：挂起的任务处于等待超时状态。
- Invalid：非上述任务状态。

9.2.6.6 free

命令功能

free命令可显示系统内存的使用情况，同时显示系统的text段、data段、rodata段、bss段大小。

命令格式

free [-k | -m]

参数说明

表 9-3 参数说明

| 参数  | 参数说明        | 取值范围 |
|-----|-------------|------|
| 无参数 | 以Byte为单位显示  | N/A  |
| -k  | 以KByte为单位显示 | N/A  |
| -m  | 以MByte为单位显示 | N/A  |

使用指南

- 输入free显示内存使用情况，total表示系统动态内存池的总大小，used表示已使用的内存大小，free表示空闲的内存大小。text表示代码段大小，data表示数据段大小，rodata表示只读数据段大小，bss表示未初始化全局变量占用的内存大小。
- free命令能够以三种单位来显示内存使用情况，包括Byte、KByte和MByte。

使用实例

举例：

分别输入free、free -k、free -m。

输出说明

以三种单位显示内存使用情况的输出：

```
Huawei LiteOS # free
 total used free
Mem: 117631744 31826864 85804880

 text data rodata bss
Mem: 4116480 423656 1204224 6659316

Huawei LiteOS # free -k
 total used free
Mem: 114874 31080 83793

 text data rodata bss
Mem: 4020 423 1176 6503
```

```
Huawei LiteOS # free -m
```

|      |       |      |        |     |
|------|-------|------|--------|-----|
|      | total | used | free   |     |
| Mem: | 112   | 30   | 81     |     |
|      | text  | data | rodata | bss |
| Mem: | 3     | 0    | 1      | 6   |

## 9.2.6.7 memcheck

### 命令功能

检查动态申请的内存块是否完整，是否存在内存越界造成的节点损坏。

### 命令格式

memcheck

### 使用指南

- 打开内存完整性检查开关。
  - 目前只有bestfit内存管理算法支持该命令，需要使能LOSCFG\_KERNEL\_MEM\_BESTFIT。  
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
  - 该命令依赖于LOSCFG\_BASE\_MEM\_NODE\_INTEGRITY\_CHECK，使用时需要在菜单项中开启"Enable integrity check or not"。  
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable integrity check or not
- 当内存池所有节点完整时，输出"memcheck over, all passed!"。
- 当内存池存在节点不完整时，输出被损坏节点的内存块信息。

### 使用实例

举例：

输入memcheck。

### 输出说明

- 没有内存越界时，执行memcheck输出内容如下：

```
Huawei LiteOS # memcheck
system memcheck over, all passed!
```
- 发生内存越界时，执行memcheck输出内容如下：

```
Huawei LiteOS # memcheck
[ERR][OsMemIntegrityCheck], 1145, memory check error!
freeNodeInfo.pstPrev:(nil) is out of legal mem range[0x80353540, 0x84000000]

broken node head: (nil) (nil) (nil) 0x0, pre node head: 0x7fc6a31b 0x8 0x80395ccc 0x80000110

dump mem tmpNode:0x80395df4 ~ 0x80395e34

0x80395df4 :00000000 00000000 00000000 00000000
0x80395e04 :cacacaca cacacaca cacacaca cacacaca
0x80395e14 :cacacaca cacacaca cacacaca cacacaca
0x80395e24 :cacacaca cacacaca cacacaca cacacaca

dump mem :0x80395db4 ~ tmpNode:0x80395df4
```

```
0x80395db4 :00000000 00000000 00000000 00000000
0x80395dc4 :00000000 00000000 00000000 00000000
0x80395dd4 :00000000 00000000 00000000 00000000
0x80395de4 :00000000 00000000 00000000 00000000
```

```

cur node: 0x80395df4
pre node: 0x80395ce4
pre node was allocated by task:SerialShellTask
cpu0 is in exc.
cpu1 is running.
excType:software interrupt
taskName = SerialShellTask
taskId = 8
task stackSize = 12288
system mem addr = 0x80353540
excBuffAddr pc = 0x80210b78
excBuffAddr lr = 0x80210b7c
excBuffAddr sp = 0x803b2d50
excBuffAddr fp = 0x80280368
R0 = 0x59
R1 = 0x600101d3
R2 = 0x0
R3 = 0x8027a300
R4 = 0x1
R5 = 0xa0010113
R6 = 0x80395e04
R7 = 0x80317254
R8 = 0x803b2de4
R9 = 0x4
R10 = 0x803afca4
R11 = 0x80280368
R12 = 0x1
CPSR = 0x600101d3
```

以上各关键输出项含义如下：

- “mem check error”，表示检测到了内存节点被破坏。
- “cur node：”，表示该节点内存被踩，并打印内存地址。
- “pre node：”，表示被踩节点前面的节点，并打印节点地址。
- “pre node was allocated by task:SerialShellTask”表示在SerialShellTask任务中发生了踩内存。
- “taskName”和“taskId”，分别表示发生异常的任务名和任务ID。

## 9.2.6.8 memused

### 命令功能

memused命令用于查看当前系统used节点中保存的函数调用栈LR信息。通过分析数据可检测内存泄露问题。

### 命令格式

memused

### 使用指南

打开menuconfig菜单开启内存泄漏检测。

- 目前只有bestfit内存管理算法支持该功能，需要使能LOSCFG\_KERNEL\_MEM\_BESTFIT。

Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit

- 该命令依赖于LOSCFG\_MEM\_LEAKCHECK，可以在菜单项中开启“Enable Function call stack of Mem operation recorded”：

Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

开启该菜单项后，会在内存操作时，将函数调用关系LR记录到内存节点中，若相同调用栈的内存节点随时间增长而不断增多，则可能存在内存泄露，通过LR可以追溯内存申请的位置。重点关注LR重复的节点。

须知

此配置打开时，会影响内存占用和内存操作性能，因此仅在检测内存泄露问题时打开。

使用实例

举例：

输入memused。

输出说明

```
Huawei LiteOS # memused
node LR[0] LR[1] LR[2]
0x802d79e4: 0x8011d740 0x8011a990 0x00000000
0x802daa0c: 0x8011d5ec 0x8011d740 0x8011a990
0x802dca28: 0x8006e6f4 0x8006e824 0x8011d5ec
0x802e2a48: 0x8014daac 0x8014db4c 0x800f6da0
0x802e8a68: 0x8011d274 0x8011d654 0x8011d740
0x802e8a94: 0x8014daac 0x8014db4c 0x8011d494
0x802eeab4: 0x8011d4e0 0x8011d658 0x8011d740
0x802f4ad4: 0x8015326c 0x80152a20 0x800702bc
0x802f4b48: 0x8015326c 0x80152a20 0x800702bc
0x802f4bac: 0x801504d8 0x801505d8 0x80150834
0x802f4c08: 0x801504d8 0x801505d8 0x80150834
0x802f4c74: 0x801504d8 0x801505d8 0x80150834
0x802f4e08: 0x801504d8 0x801505d8 0x80150834
0x802f4e60: 0x801030e8 0x801504d8 0x801505d8
0x802f4e88: 0x80103114 0x801504d8 0x801505d8
0x802f4eb4: 0x801504d8 0x801505d8 0x80150834
0x802f4f7c: 0x801504d8 0x801505d8 0x80150834
0x802f5044: 0x801504d8 0x801505d8 0x80150834
0x802f510c: 0x800702bc 0x80118f24 0x00000000
```

9.2.6.9 hwi

命令功能

hwi命令用于查询当前中断信息。

命令格式

hwi

hwi status all

hwi status hwiNum [devId]

参数说明

表 9-4 参数说明

| 参数         | 参数说明                           | 取值范围           |
|------------|--------------------------------|----------------|
| status all | 打印所有已创建的中断状态。                  | 无              |
| hwiNum     | 中断号，十进制。                       | [0,0xFFFFFFFF] |
| devId      | 共享中断设备号，十六进制，缺省时表示查询的中断为非共享中断。 | [0,0xFFFFFFFF] |

使用指南

- 使能hwi命令，在菜单项打开宏开关LOSCFG\_DEBUG\_HWI：  
Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Hwi Debugging
- 输入hwi即显示当前中断号、中断次数及注册中断名称。
- 输入hwi status all显示当前所有已创建中断的状态。
- 输入hwi status hwiNum [devId]查询特定的中断状态；共享中断需要devId参数，非共享中断不需要参数devId。
- 若打开宏开关LOSCFG\_CPUP\_INCLUDE\_IRQ，hwi命令还会显示各个中断的处理时间（cycles）、CPU占用率以及中断类型。该宏开关可在菜单项中开启Enable Cpus include irq使能：  
Kernel ---> Enable Extend Kernel ---> Enable Cpus ---> Enable Cpus include irq  
关于该宏开关更详细的介绍，参见CPU占用率[开发流程](#)。

使用实例

举例：

输入hwi。

输入hwi status hwiNum devId。

输入hwi status hwiNum。

输入hwi status all。

输出说明

- 输入hwi，显示中断信息（LOSCFG\_CPUP\_INCLUDE\_IRQ关闭）。  
Huawei LiteOS # hwi  
InterruptNo Count Name  
35: 1364:  
36: 0:  
40: 79: uart\_pl011
- 输入hwi，显示中断信息（LOSCFG\_CPUP\_INCLUDE\_IRQ打开）。  
Huawei LiteOS # hwi  
InterruptNo Count Name CYCLECOST CPUUSE CPUUSE10s CPUUSE1s mode  
3: 1333 122 0.0 0.0 0.0 normal  
4: 0 0 0.0 0.0 0.0 normal  
5: 59 uart\_pl011 305 0.0 0.0 0.0 normal  
12: 96 ETH 131 0.0 0.0 0.0 normal

3. 输入hwi status 3 0x1，显示中断号3，设备号0x1共享中断的状态。

```
Huawei LiteOS # hwi status 3 0x1
InterruptNo DevId Priority Affinity Created Enable Pending

3 0x1 20 0x0 0 1 0
```

4. 输入hwi status 3，显示中断号3的非共享中断的状态。

```
Huawei LiteOS # hwi status 3
InterruptNo DevId Priority Affinity Created Enable Pending

3 0x0 20 0x0 0 1 0
```

5. 输入hwi status all，显示所有已创建中断的状态。

```
Huawei LiteOS # hwi status all
InterruptNo DevId Priority Affinity Created Enable Pending

0 0x0 20 0x0 1 1 0
1 0x0 20 0x0 1 1 0
2 0x0 20 0x0 1 1 0
30 0x0 20 0x0 1 1 0
33 0x0 0 0x1 1 1 0
```

9.2.6.10 queue

命令功能

queue命令用于查看队列的使用情况。

命令格式

```
queue
```

使用指南

该命令依赖于LOSCFG\_DEBUG\_QUEUE，该宏开关可在菜单项中开启"Enable Queue Debugging"使能。

```
Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Queue Debugging
```

使用实例

举例：

输入queue。

输出说明

执行queue后得到队列的使用情况：

```
Huawei LiteOS # queue
used queues information:
Queue ID <0x0> may leak, queue len is 0x10, readable cnt:0x0, writeable cnt:0x10, TaskEntry of creator:
0x0x80007d5, Latest operation time: 0x614271
```

以上各输出项含义如下：

| 输出项      | 说明   |
|----------|------|
| Queue ID | 队列编号 |



| 输出项                   | 说明         |
|-----------------------|------------|
| queue len             | 队列消息节点个数   |
| readable cnt          | 队列中可读的消息个数 |
| writable cnt          | 队列中可写的消息个数 |
| TaskEntry of creator  | 创建队列的接口地址  |
| Latest operation time | 队列最后操作时间   |

9.2.6.11 sem

命令功能

sem命令用于查询系统内核信号量的相关信息。

命令格式

sem [/D]

sem fulldata

参数说明

表 9-5 参数说明

| 参数       | 参数说明                                                                                      | 取值范围            |
|----------|-------------------------------------------------------------------------------------------|-----------------|
| ID       | 信号量ID号，输入形式以十进制表示或十六进制表示皆可                                                                | [0, 0xFFFFFFFF] |
| fulldata | 查询所有在用的信号量信息，打印信息包括如下：SemID, Count, Original Count, Creator (TaskEntry), Last Access Time | N/A             |

使用指南

- 参数缺省时，显示所有信号量的使用数及信号量总数。
- sem后加ID，当ID参数在[0, 1023]范围内时，返回指定ID号的信号量使用数（如果指定ID号的信号量未被使用则提示），其他取值时返回参数错误的提示。
- sem的参数ID和fulldata不可以混用。
- 参数fulldata依赖LOS\_CFG\_DEBUG\_SEMAPHORE，该宏开关可在菜单项中开启"Enable Semaphore Debugging"使能。

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Semaphore Debugging

## 使用实例

举例1：输入sem 1。

举例2：输入sem和sem fulldata。

## 输出说明

执行sem 1，查询指令结果：

```
Huawei LiteOS # sem 1
SemID Count

0x00000001 0x1
No task is pended on this semaphore!
```

执行sem和sem fulldata，查询所有在用的信号量信息：

```
Huawei LiteOS # sem
SemID Count

0x00000000 1
SemID Count

0x00000001 1
SemID Count

0x00000002 1
SemUsingNum : 3

Huawei LiteOS # sem fulldata
Used Semaphore List:
SemID Count OriginalCout Creator(TaskEntry) LastAccessTime

0x2 0x1 0x1 0x80164d70 0x3
0x0 0x1 0x1 0x0 0x3
0x1 0x1 0x1 0x0 0x3
```

## 9.2.6.12 mutex

### 命令功能

mutex命令用于查看互斥锁的使用情况。

### 命令格式

mutex

### 使用指南

该命令依赖于LOSCFG\_DEBUG\_MUTEX，该宏开关可在菜单项中开启"Enable Mutex Debugging"使能。

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Mutex Debugging

## 使用实例

举例：

输入mutex。

输出说明

执行mutex输出互斥锁使用情况：

```
Huawei LiteOS # mutex
used mutexs information:
Mutex ID <0x0> may leak, Owner is null, TaskEntry of creator: 0x8000711, Latest operation time: 0x0
```

以上各主要输出项含义如下：

| 输出项                   | 说明       |
|-----------------------|----------|
| Mutex ID              | 锁序号      |
| TaskEntry of creator  | 创建锁的接口地址 |
| Latest operation time | 任务最后调度时间 |

9.2.6.13 swtmr

命令功能

swtmr命令用于查询系统软件定时器的相关信息。

命令格式

swtmr [*ID*]

参数说明

表 9-6 参数说明

| 参数 | 参数说明                         | 取值范围           |
|----|------------------------------|----------------|
| ID | 软件定时器ID号，输入形式以十进制表示或十六进制表示皆可 | [0,0xFFFFFFFF] |

使用指南

- 使能swtmr命令，在菜单项打开宏开关LOSCFG\_DEBUG\_SWTMR：  
Debug ----> Enable a Debug Version ----> Enable Debug LiteOS Kernel Resource ----> Enable Swtmr Debugging
- 参数缺省时，默认显示所有软件定时器的相关信息。
- swtmr后加ID号时，当ID在[0, 当前软件定时器个数 - 1]范围内时，返回对应ID的软件定时器的相关信息，其他取值时返回错误提示。

使用实例

举例：

输入swtmr和swtmr 0。

输出说明

执行swtmr查询软件定时器相关信息，输出如下：

```
Huawei LiteOS # swtmr
SwTmrID State Mode Interval Arg handlerAddr

0x00000000 Ticking Period 1000 0x00000000 0x800442d
```

执行swtmr 0查询ID为0的软件定时器信息，输出如下：

```
Huawei LiteOS # swtmr 0
SwTmrID State Mode Interval Arg handlerAddr

0x00000000 Ticking Period 1000 0x00000000 0x800442d
```

表 9-7 输出说明

| 输出项         | 说明            |
|-------------|---------------|
| SwTmrID     | 软件定时器ID       |
| State       | 软件定时器状态       |
| Mode        | 软件定时器模式       |
| Interval    | 软件定时器使用的Tick数 |
| Arg         | 传入的参数         |
| handlerAddr | 回调函数的地址       |

9.2.6.14 systeminfo

命令功能

systeminfo命令用于显示当前操作系统的资源使用情况，包括任务、信号量、互斥锁、队列、软件定时器等。

对于信号量、互斥锁、队列和软件定时器，如果在系统镜像中已经裁剪了这些模块，那么说明系统没有使用这些资源，该命令也就不会显示这些资源的情况。

命令格式

systeminfo

使用实例

举例：

输入systeminfo。

输出说明

执行systeminfo查看系统资源使用情况，输出如下：

```
Huawei LiteOS # systeminfo
Module Used Total

Task 5 16
Sem 0 20
Mutex 1 20
Queue 1 10
SwTmr 1 16
```

以上各输出项含义如下：

| 输出项    | 说明    |
|--------|-------|
| Module | 模块名称  |
| Used   | 当前使用量 |
| Total  | 最大可用量 |

9.2.6.15 log

命令功能

log命令用于修改&查询系统的日志打印等级。

命令格式

log level [*levelNum*]

参数说明

表 9-8 参数说明

| 参数       | 参数说明     | 取值范围       |
|----------|----------|------------|
| levelNum | 设置日志打印等级 | [0x0, 0x5] |

须知

log命令的帮助信息中还显示了另外两个参数module和path，但当前暂不支持这两个参数。

使用指南

- 该命令依赖于LOSCFG\_SHELL\_LK，可在菜单项中开启"Enable Shell lk"使能。  
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell lk
- log level命令用于设置系统的日志打印等级，包括6个等级。  
TRACE\_EMG = 0  
TRACE\_COMMOM = 1

```
TRACE_ERROR = 2
TRACE_WARN = 3
TRACE_INFO = 4
TRACE_DEBUG = 5
```

- 若log level命令不加levelNum参数，则提示命令的使用方法。

使用实例

举例：  
输入log level 4。

输出说明

执行log level 4，设置系统的日志打印等级为INFO级别：

```
Huawei LiteOS # log level 4
Set current log level INFO
```

9.2.6.16 dmesg

命令功能

dmesg命令用于控制内核dmesg缓存区。

命令格式

```
dmesg
dmesg [-c / -C / -D / -E / -L / -U]
dmesg -s <size>
dmesg -l <level>
dmesg > file（开源版本暂不支持该命令）
```

参数说明

表 9-9 参数说明

| 参数      | 参数说明                       | 取值范围 |
|---------|----------------------------|------|
| -c      | 打印缓存区内容并清空缓存区，此打印不受-D、-L影响 | N/A  |
| -C      | 清空缓存区                      | N/A  |
| -D   -E | 关闭/开启控制台打印，开源版本暂不支持该参数     | N/A  |
| -L   -U | 关闭/开启串口打印，开源版本暂不支持该参数      | N/A  |

| 参数         | 参数说明                   | 取值范围   |
|------------|------------------------|--------|
| -s <size>  | 设置缓存区大小                | N/A    |
| -l <level> | 设置缓存区的日志打印等级           | 0 -- 5 |
| > file     | 将缓存区内容写入文件，开源版本暂不支持该参数 | N/A    |

须知

- 1. 写入文件需确保已挂载文件系统。
- 2. 关闭串口打印会影响Shell使用，建议先连接Telnet再尝试关闭串口。

使用指南

- 该命令依赖于LOSCFG\_SHELL\_DMESG，使用时可在菜单项中开启"Enable Shell dmesg"。  
Debug ---> Enable a Debug Version---> Enable Shell---> Enable Shell dmesg
- 参数缺省时，默认打印缓存区内容。
- 各参数均不能混合使用。

使用实例

举例：  
输入dmesg后，接着输入dmesg -C和dmesg。

输出说明

第一次执行dmesg后，可以看到输出了缓存区内容。接着执行dmesg -C清空缓存区内容，紧接着再次执行dmesg可以看到之前缓存区中的内容已经被清空：

```
Huawei LiteOS # dmesg

Huawei LiteOS # log level 4
Set current log level INFO
Huawei LiteOS # dmesg

Huawei LiteOS # dmesg -C
Huawei LiteOS # dmesg

Huawei LiteOS # dmesg
```

须知

dmesg -C是日志维测命令，只支持串口输出，不支持通过console输出到其他输出端口（Telnet、USB等）。

### 9.2.6.17 stack

#### 命令功能

stack命令用于显示当前操作系统内所有栈的信息。

#### 命令格式

stack

#### 使用指南

该功能默认开启。

#### 使用实例

举例：

输入stack。

#### 输出说明

执行stack命令查看系统内所有栈的信息：

Huawei LiteOS # stack

| stack name | cpu id | stack addr | total size | used size |
|------------|--------|------------|------------|-----------|
| udf_stack  | 3      | 0x3c800    | 0x28       | 0x0       |
| udf_stack  | 2      | 0x3c828    | 0x28       | 0x0       |
| udf_stack  | 1      | 0x3c850    | 0x28       | 0x0       |
| udf_stack  | 0      | 0x3c878    | 0x28       | 0x0       |
| abt_stack  | 3      | 0x3c8a0    | 0x28       | 0x0       |
| abt_stack  | 2      | 0x3c8c8    | 0x28       | 0x0       |
| abt_stack  | 1      | 0x3c8f0    | 0x28       | 0x0       |
| abt_stack  | 0      | 0x3c918    | 0x28       | 0x0       |
| fiq_stack  | 3      | 0x3ca40    | 0x40       | 0x0       |
| fiq_stack  | 2      | 0x3ca80    | 0x40       | 0x0       |
| fiq_stack  | 1      | 0x3cac0    | 0x40       | 0x0       |
| fiq_stack  | 0      | 0x3cb00    | 0x40       | 0x0       |
| svc_stack  | 3      | 0x3cb40    | 0x2000     | 0x524     |
| svc_stack  | 2      | 0x3eb40    | 0x2000     | 0x524     |
| svc_stack  | 1      | 0x40b40    | 0x2000     | 0x524     |
| svc_stack  | 0      | 0x42b40    | 0x2000     | 0x528     |
| irq_stack  | 3      | 0x3c940    | 0x40       | 0x0       |
| irq_stack  | 2      | 0x3c980    | 0x40       | 0x0       |
| irq_stack  | 1      | 0x3c9c0    | 0x40       | 0x0       |
| irq_stack  | 0      | 0x3ca00    | 0x40       | 0x0       |
| exc_stack  | 3      | 0x44b40    | 0x1000     | 0x0       |
| exc_stack  | 2      | 0x45b40    | 0x1000     | 0x0       |
| exc_stack  | 1      | 0x46b40    | 0x1000     | 0x0       |
| exc_stack  | 0      | 0x47b40    | 0x1000     | 0x0       |

#### 须知

stack是异常维测命令，只支持串口输出，不支持通过console输出到其他输出端口（Telnet、USB等）。



9.2.6.18 cpup

命令功能

cpup命令用于查询系统CPU的占用率，并以百分比显示占用率。

命令格式

cpup [*mode*] [*taskID*]

参数说明

表 9-10 参数说明

| 参数     | 参数说明                                                                                                                                                   | 取值范围                      |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| mode   | <ul style="list-style-type: none"><li>缺省：显示系统最近10s的CPU占用率</li><li>0：显示系统最近10s的CPU占用率</li><li>1：显示系统最近1s的CPU占用率</li><li>其他数字：显示系统启动至今总的CPU占用率</li></ul> | [0,0xFFFF]<br>或0xFFFFFFFF |
| taskID | 任务ID号                                                                                                                                                  | [0,0xFFFF]<br>或0xFFFFFFFF |

使用指南

- 参数缺省时，显示系统最近10s的CPU占用率。
- 只输入一个参数时，该参数为mode，显示系统相应时间的CPU占用率。
- 输入两个参数时，第一个参数为mode，第二个参数为taskID，显示指定任务ID的任务在相应时间的CPU占用率。
- 该功能需要使能CPU占用率模块，菜单路径为：  
Kernel ---> Enable Extend Kernel ---> Enable Cpuop  
想更多的了解CPU占用率模块，参见[CPU占用率](#)。

使用实例

举例：  
输入cpup 1 1

输出说明

执行cpup 1 1，显示ID为1的任务最近1s的CPU占用率：

Huawei LiteOS # cpup 1 1  
TaskId 1 CpuUsage in 1s: 78.7

9.2.6.19 watch

命令功能

watch命令用于周期性监听一个命令的运行结果。

命令格式

```
watch [-c / -n / -t / --count / --interval / -no-title] <command>
watch --over
```

参数说明

表 9-11 参数说明

| 参数              | 参数说明               | 缺省值        | 取值范围             |
|-----------------|--------------------|------------|------------------|
| -c   --count    | 命令执行的总次数           | 0xFFFFFFFF | ( 0, 0xFFFFFFFF] |
| -n   --interval | 命令周期性执行的时间间隔 ( s ) | 1s         | ( 0, 0xFFFFFFFF] |
| -t   -no-title  | 关闭顶端的时间显示          | N/A        | N/A              |
| command         | 需要监听的Shell命令       | N/A        | N/A              |
| --over          | 关闭当前的监听            | N/A        | N/A              |

使用指南

- 该命令依赖于LOSCFG\_SHELL\_EXTENDED\_CMDS，该宏开关可在菜单项中开启“Enable Shell Ext CMDs”使能。  
Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable Shell Ext CMDs
- command参数必须是Shell命令，对于非Shell命令，会有错误提示“command is not found”。
- 如果要监听命令，command是必填参数。
- over参数不能与其他参数混合使用。

使用实例

- 举例1：输入watch -c 5 task 1。
- 举例2：在不需要watch命令监听的情况下，执行watch --over。

输出说明

每个周期间隔1秒的执行task 1命令，共执行5次，watch命令监听到的结果如下所示：

```
Huawei LiteOS # watch -c 3 task 1
Huawei LiteOS # Thu Jan 1 16:26:26 1970
```

```
TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598

Thu Jan 1 16:26:27 1970

TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598

Thu Jan 1 16:26:28 1970

TaskName = Swt_Task
TaskId = 0x1
*****backtrace begin*****
traceback 1 -- lr = 0x08004006 -- fp = 0x0800045e
traceback 2 -- lr = 0x08004000 -- fp = 0x0800194c
traceback 3 -- lr = 0x080040da -- fp = 0x08003e50
traceback 4 -- lr = 0x080015c2 -- fp = 0x080040a8
traceback 5 -- lr = 0x0800396e -- fp = 0x08001598
```

## 9.2.6.20 schedstatstart

### 命令功能

schedstatstart命令用于开启调度统计功能。

### 命令格式

schedstatstart

### 使用指南

- 目前该功能仅支持有浮点运算的平台，且该命令依赖于LOSCFG\_DEBUG\_SCHED\_STATISTICS，该宏开关可在菜单项中开启"Enable Scheduler Statistics Debugging"使能。  
Debug ---> Enable a Debug Version ---> Enable DebugLiteOS Kernel Resource  
---> Enable Scheduler Statistics Debugging。
- schedstatstart、schedstatstop和schedstatinfo等3个命令一起配套使用。

### 使用实例

举例：输入schedstatstart。

### 输出说明

查看[输出说明](#)。

### 9.2.6.21 schedstatstop

#### 命令功能

schedstatstop命令用于关闭调度统计功能，并输出各个CPU的调度统计信息。

#### 命令格式

schedstatstop

#### 使用指南

同 [使用指南](#)。

#### 使用实例

举例：输入schedstatstop。

#### 输出说明

查看[输出说明](#)。

### 9.2.6.22 schedstatinfo

#### 命令功能

schedstatinfo命令用于输出各个任务的CPU调度统计信息。

#### 命令格式

schedstatinfo

#### 使用指南

同 [使用指南](#)。

#### 使用实例

举例：  
输入schedstatinfo。

#### 输出说明

查看[输出说明](#)。

## 9.2.7 文件命令参考

### 9.2.7.1 使能文件系统命令

使用Shell中的文件命令前，需要打开menuconfig菜单使能Shell，详见[配置项](#)。

9.2.7.2 cd

命令功能

cd命令用于改变当前目录。

命令格式

cd [*path*]

参数说明

表 9-12 参数说明

| 参数   | 参数说明 | 取值范围             |
|------|------|------------------|
| path | 目的目录 | 用户必须具有目的目录的可执行权限 |

使用指南

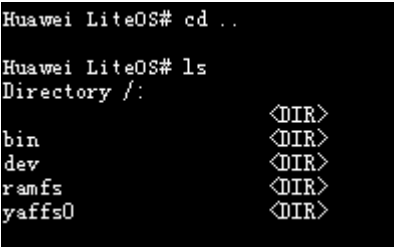
- 未指定目录参数时，会跳转至根目录。
- cd后加目录时，跳转至该目录。
- 目录以 /（斜杠）开头时，表示根目录。
- .（点）表示当前目录。
- ..（点点）表示父目录。

使用实例

举例：  
输入cd ..

输出说明

图 9-1 显示结果如下



9.2.7.3 pwd

命令功能

pwd命令用于显示当前路径。

命令格式

pwd

使用指南

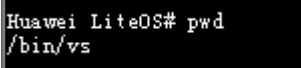
pwd 命令将当前目录的全路径名称（从根目录开始的绝对路径）写入标准输出。全部目录使用 /（斜线）分隔，第一个 / 表示根目录，最后一个目录是当前目录。

使用实例

举例：  
输入pwd。

输出说明

图 9-2 查看当前路径



9.2.7.4 mkdir

命令功能

mkdir命令用于创建一个目录。

命令格式

mkdir <dir>

参数说明

表 9-13 参数说明

| 参数  | 参数说明    | 取值范围 |
|-----|---------|------|
| dir | 需要创建的目录 | N/A  |

使用指南

- mkdir后加所需要创建的目录名，则将在当前目录下创建目录。
- mkdir后加带路径的目录名，则将在指定路径下创建目录。

使用实例

举例：  
输入mkdir share。

输出说明

图 9-3 在当前目录下创建 share 目录

```
Huawei LiteOS# mkdir share

Huawei LiteOS# ls
Directory /bin/vs:
share <DIR>
sd <DIR>
```

9.2.7.5 rmdir

命令功能

rmdir命令用于删除指定的目录。

命令格式

rmdir <dir>

参数说明

表 9-14 参数说明

| 参数  | 参数说明             | 取值范围 |
|-----|------------------|------|
| dir | 要删除的目录，该目录必须为空目录 | N/A  |

使用指南

- rmdir命令只能用来删除目录。
- rmdir一次只能删除一个目录。
- rmdir只能删除空目录。

使用实例

举例：输入rmdir dir

输出说明

图 9-4 删除一个名为 dir 的目录

```
Huawei LiteOS# ls
Directory /bin/vs:
dir <DIR>
sd <DIR>

Huawei LiteOS# rmdir dir

Huawei LiteOS# ls
Directory /bin/vs:
sd <DIR>
```

### 9.2.7.6 rm

#### 命令功能

rm命令用于删除文件或者非空目录。

#### 命令格式

rm -r <dir>  
rm <file>

#### 参数说明

表 9-15 参数说明

| 参数       | 参数说明                    | 取值范围 |
|----------|-------------------------|------|
| -r <dir> | dir为要删除的目录，支持删除非空目录和空目录 | N/A  |
| file     | 要删除文件                   | N/A  |

#### 使用指南

- rm命令一次只能删除一个文件。
- rm -r命令可以删除非空目录。

须知

如果使用rm删除系统关键资源（例如/dev），会造成系统死机等未知影响，需慎重使用该命令。

#### 使用实例

- 举例：
- 输入rm 1.c。
  - 输入rm -r dir。

#### 输出说明

图 9-5 用 rm 命令删除文件 1.c

```
Huawei LiteOS# ls
Directory /ramfs:
1.c 0

Huawei LiteOS# rm 1.c

Huawei LiteOS# ls
Directory /ramfs:
```



图 9-6 用 rm -r 删除目录 dir

```
Huawei LiteOS# ls
Directory /ramfs:
dir <DIR>

Huawei LiteOS# rm -r dir

Huawei LiteOS# ls
Directory /ramfs:
```

9.2.7.7 touch

命令功能

touch命令用于在当前目录下创建一个空文件。

命令格式

touch <filename>

参数说明

表 9-16 参数说明

| 参数       | 参数说明    | 取值范围 |
|----------|---------|------|
| filename | 需要创建的文件 | N/A  |

使用指南

- touch命令创建的空文件，其默认权限为可读写。
- touch命令一次只能创建一个文件。
- touch命令操作已存在的文件会成功，不会更新时间戳。

须知

在系统重要资源路径下使用touch命令创建文件，会对系统造成死机等未知影响，如在/dev路径下执行touch uartdev-0，会产生系统卡死现象。

使用实例

举例：  
输入touch file.c。

输出说明

图 9-7 在当前目录下创建一个名为 file.c 的文件

```
Huawei LiteOS# touch file.c

Huawei LiteOS# ls
Directory /bin/vs:
file.c 0
sd <DIR>

Huawei LiteOS#
```

9.2.7.8 cp

命令功能

cp命令用于拷贝文件，创建一份副本。

命令格式

cp <source path> <dest path>

参数说明

表 9-17 参数说明

| 参数          | 参数说明 | 取值范围          |
|-------------|------|---------------|
| source path | 源文件  | 目前只支持文件，不支持目录 |
| dest path   | 目的路径 | 支持目录以及文件      |

使用指南

- 同一路径下，源文件与目的文件不能重名。
- 源文件必须存在，且不为目录。
- 源文件路径支持“\*”和“？”通配符，“\*”代表任意多个字符，“？”代表任意单个字符。目的路径不支持通配符。当源文件路径可匹配多个文件时，目的路径必须为目录。
- 目的路径为目录时，该目录必须存在。此时目的文件以源文件名进行命名。
- 目的路径为文件时，所在目录必须存在。目的文件名就是目的路径中指定的文件名。
- 目前不支持多文件拷贝。参数大于2个时，只对前2个参数进行操作。
- 目的文件不存在时创建新文件，已存在则覆盖。

须知

拷贝系统重要资源文件时，会造成死机等重大未知影响，如拷贝/dev/uartdev-0文件时，会产生系统卡死现象。

使用实例

举例：  
输入cp 100HSCAM/FILE0087.MP4 .

输出说明



9.2.7.9 dd

命令功能

dd命令用于文件读写性能统计。

命令格式

dd file=[PATHNAME] mode=[OPS] bs=[SIZE] count=[N]

参数说明

表 9-18 参数说明

| 参数       | 参数说明                         | 取值范围            |
|----------|------------------------------|-----------------|
| PATHNAME | 带文件路径的文件名                    | 非空              |
| OPS      | 要测试的文件操作类型，包括读/写，取值1表示读，2表示写 | [1, 2]          |
| SIZE     | 一次读/写的block字节数               | [0, 0xFFFFFFFF] |
| N        | 读/写的block个数                  | [0, 0xFFFFFFFF] |

使用指南

1. dd命令可并发执行，一条dd命令对应一个线程，因此可以进行多线程的读写测试。
2. dd命令测试较大的文件读写时，存在耗时较长的情况，可能不会立即输出性能结果，这是正常现象。
3. bs字段和count字段可以不指定，系统将使用缺省值LOSCFG\_DD\_DEFAULT\_BS 1024、LOSCFG\_DD\_DEFAULT\_CNT 1。

4. bs字段的SIZE不宜过大，系统将申请SIZE大小的动态内存，过大的SIZE会导致内存申请失败，导致dd命令无法正常执行。
5. dd流程中仅会执行open、read/write、close操作，用户需自行保证操作dd前已正确挂载对应的文件系统，如有其他操作也需用户自行执行。

使用实例

举例，测试文件写性能如下：

dd file=/app/sd/testfile mode=2 bs=32768 count=32768

输出说明

```
Huawei LiteOS # dd file=/app/sd/testfile mode=2 bs=32768 count=32768
filePath : /app/sd/testfile
mode : write
blockSize : 32768
blockCount: 32768

Huawei LiteOS # total 1073741824 bytes (1.00 GB, 1024.00MB) write, time used: 48.415982(s), 21.15MB/S
```

输出写入文件的总字节数、总耗时以及速率。

9.2.7.10 ls

命令功能

ls命令用于显示当前目录的内容。

命令格式

ls [*path*]

参数说明

表 9-19 参数说明

| 参数   | 参数说明                                                                                                                                 | 取值范围                                 |
|------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| path | <ul style="list-style-type: none"><li>为为空时，显示当前目录的内容</li><li>为无效文件名时，显示失败，提示：No such directory</li><li>为有效目录路径时，显示对应目录下的内容</li></ul> | <div>1. 空</div> <div>2. 有效目录路径</div> |

使用指南

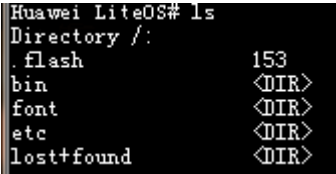
- ls可以显示文件的大小。
- proc下ls无法统计文件大小，显示为0。

使用实例

举例：  
输入ls。

输出说明

图 9-9 查看当前系统路径下的内容，显示如下



9.2.7.11 cat

命令功能

cat用于显示文本文件的内容。

命令格式

cat <pathname>

参数说明

表 9-20 参数说明

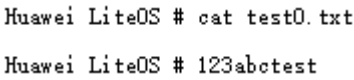
| 参数       | 参数说明                           | 取值范围   |
|----------|--------------------------------|--------|
| pathname | 带文件路径的文件名，如果文件在当前路径下，可以直接输入文件名 | 已存在的文件 |

使用实例

举例：  
输入cat w。

输出说明

图 9-10 查看 w 文件的信息



9.2.7.12 lsfd

命令功能

lsfd命令用于显示当前已经打开的文件描述符及对应的文件名。

命令格式

lsfd

使用指南

lsfd命令显示当前已经打开文件的fd号以及文件的名字。

使用实例

举例：  
输入lsfd。

输出说明

```
Huawei LiteOS # lsfd
fd filename
3 /dev/shell
```

9.2.7.13 format

命令功能

format命令用于格式化磁盘。

命令格式

format <dev\_inodename> <sectors> <option> [*label*]

参数说明

| 参数            | 参数说明                                                                        |
|---------------|-----------------------------------------------------------------------------|
| dev_inodename | 要格式化的设备名                                                                    |
| sectors       | 分配的单元内存或扇区大小，取值必须为2的幂数，FAT32下最大值为128，取其他值表示自动选择合适的簇大小。不同size的分区，可用的簇大小范围不同。 |

| 参数          | 参数说明                                                                                                                                                                                                                        |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| option      | 格式化选项，用于选择文件系统的类型，有如下几种参数可供选择： <ul style="list-style-type: none"><li>0x01: FMT_FAT</li><li>0x02: FMT_FAT32</li><li>0x07: FMT_ANY</li><li>0x08: FMT_ERASE（USB不支持该选项）</li></ul> 传入其他值皆为非法，将由系统自动选择格式化方式。若格式化U盘时低格位为1，会出现错误打印。 |
| label（可选参数） | 该参数为可选参数，输入值应为字符串，用来指定卷标名。当输入字符串“null”时，则把之前设置的卷标名清空                                                                                                                                                                        |

使用指南

- format指令用于格式化磁盘，设备名可以在dev目录下查找。format时必须安装存储卡。
- format只能格式化U盘、SD卡和EMMC卡，无法格式化Nand Flash和Nor Flash。
- sectors参数必须传入合法值，传入非法参数可能引发异常。

使用实例

举例：  
输入format /dev/mmcblk0p0 128 2。

输出说明

```
Huawei LiteOS # format /dev/mmcblk0p0 128 2
Format to FAT32, 128 sectors per cluster.
format /dev/mmcblk0p0 Success
```

9.2.7.14 mount

命令功能

mount命令用于将设备挂载到指定目录。

命令格式

mount <device> <path> <name>

参数说明

表 9-21 参数说明

| 参数     | 参数说明                      | 取值范围                                         |
|--------|---------------------------|----------------------------------------------|
| device | 要挂载的设备                    | 系统拥有的设备                                      |
| path   | 设备要挂载到的目录，用户必须具有该目录的可执行权限 | N/A                                          |
| name   | 要挂载的文件系统类型                | vfat, yaffs, jffs, ramfs, nfs, procfs, romfs |

使用指南

mount后加需要挂载的设备信息、指定目录以及设备文件格式，就能成功挂载文件系统到指定目录。

使用实例

举例：  
输入mount /dev/mmc0 /bin/vs/sd vfat。

输出说明

将/dev/mmc0以vfat文件系统类型挂载到/bin/vs/sd目录。  
Huawei LiteOS # mount /dev/mmc0 /bin/vs/sd vfat  
mount ok

9.2.7.15 umount

命令功能

umount命令用于卸载已经挂载的文件系统。

命令格式

umount <dir>

参数说明

表 9-22 参数说明

| 参数  | 参数说明             | 取值范围          |
|-----|------------------|---------------|
| dir | 需要卸载的文件系统对应的挂载目录 | 系统已挂载的文件系统的目录 |



使用指南

- umount后加上需要卸载的文件系统对应的挂载目录，即将指定文件系统卸载。

使用实例

举例：  
输入umount /bin/vs/sd。

输出说明

将挂载在/bin/vs/sd目录上的文件系统卸载。

```
Huawei LiteOS # umount /bin/vs/sd
umount ok
```

9.2.7.16 partinfo

命令功能

partinfo命令用于查看系统识别的硬盘、U盘和SD卡的分区信息。

命令格式

partinfo <dev\_inodename>

参数说明

表 9-23 参数说明

| 参数            | 参数说明    | 取值范围   |
|---------------|---------|--------|
| dev_inodename | 要查看的分区名 | 合法的分区名 |

使用实例

举例：  
输入partinfo /dev/sdap0。

输出说明

```
Huawei LiteOS # partinfo /dev/sdap0
part info :
disk id : 3
part_id in system: 0
part no in disk : 0
part no in mbr : 1
part filesystem : 0C
part dev name : sdap0
part sec start : 2048
part sec count : 167794688
```

9.2.7.17 partition

命令功能

partition命令用于查看Flash分区信息。

命令格式

partition <nand | spinor>

参数说明

表 9-24 参数说明

| 参数     | 参数说明               | 取值范围 |
|--------|--------------------|------|
| nand   | 显示Nand Flash分区信息   | N/A  |
| spinor | 显示Spinor Flash分区信息 | N/A  |

使用指南

- 当使能yaffs文件系统时才可以查看Nand Flash分区信息。
- 当使能jffs或romfs文件系统时才可以查看Spinor Flash分区信息。

使用实例

举例：

输入partition nand。

输出说明

查看Nand Flash分区信息。

```
Huawei LiteOS # partition nand
nand partition num:0, blkdev name:/dev/nandblk0, mountpt:/yaffs0, startaddr:0x00e00000, length:
0x00200000
```

9.2.7.18 statfs

命令功能

statfs命令用于打印文件系统的信息，如该文件系统类型、总大小、可用大小等信息。

命令格式

statfs <directory>

参数说明

| 参数        | 参数说明      | 取值范围                     |
|-----------|-----------|--------------------------|
| directory | 文件系统的挂载路径 | 必须是存在的文件系统，并且其支持statfs命令 |

使用指南

打印信息因文件系统而异。

使用实例

以YAFFS文件系统为例：

输入statfs /yaffs0。

输出说明

```
statfs got:
f_type = 1497497427
cluster_size = 2048
total_clusters = 704
free_clusters = 640
avail_clusters = 640
f_namelen = 255

/yaffs0
total size: 9830400 Bytes
free size: 5890048 Bytes
```

9.2.7.19 virstatfs

命令功能

virstatfs命令用于打印虚拟分区下文件系统的信息，如总大小、可用大小等信息。

命令格式

virstatfs <directory>

参数说明

| 参数        | 参数说明              | 取值范围 |
|-----------|-------------------|------|
| directory | 虚拟分区入口目录的路径或者其子路径 | N/A  |

使用指南

- virstatfs命令只有开启了虚拟分区特性时才有效（LOSCFG\_FS\_FAT\_VIRTUAL\_PARTITION=y）。

- virstatfs的输入路径，必须是已经成功应用虚拟分区的物理分区上的虚拟分区入口目录，或者其子目录。如果输入路径是该物理分区的根目录，或者该物理分区未成功应用虚拟分区特性，或者路径指向非FAT32文件系统分区，virstatfs命令均会被拒绝执行。
- 关于虚拟分区特性，详见FAT开发指导中[虚拟分区的接口说明](#)和[开发流程](#)。

## 使用实例

举例：

输入virstatfs /bin/vs/sd/virpart0。

## 输出说明

```
Huawei LiteOS # mount /dev/mmcblk0p0 /bin/vs/sd vfat
mount ok

Huawei LiteOS # ls /bin/vs/sd
Directory /bin/vs/sd:
virpart0 <DIR>
virpart1 <DIR>
virpart2 <DIR>

Huawei LiteOS # virstatfs /bin/vs/sd/virpart0
Virtual partition "/bin/vs/sd/virpart0" Info:
f_type = 19780
cluster_size = 32768
total_clusters = 0x23b73f
free_clusters = 0x23b73e
avail_clusters = 0x23b73e
f_name_len = 255

/bin/vs/sd/virpart0
total size: 76699107328 Bytes
free size: 76699074560 Bytes

Huawei LiteOS #
```

### 9.2.7.20 sync

#### 命令功能

sync命令用于同步缓存数据（FAT文件系统中的数据）到SD卡。

#### 命令格式

sync

#### 使用指南

- sync命令用来刷新缓存，当没有SD卡插入时不进行操作，有SD卡插入时缓存信息会同步到SD卡。成功时无显示信息。
- 使用sync命令前，需要使能FAT文件系统，即设置LOSCFG\_FS\_FAT = y。

使用实例

举例：  
输入sync，有SD卡时同步到SD卡，无SD卡时不操作。

输出说明

无。

9.2.7.21 writeproc

命令功能

writeproc命令用于向proc文件系统中的文件写入内容。

命令格式

writeproc <val> <operational mark> <path>

参数说明

表 9-25 参数说明

| 参数               | 参数说明                          | 取值范围        |
|------------------|-------------------------------|-------------|
| val              | 要写入文件的内容                      | 字符串         |
| operational mark | 操作符，当操作符为">>"时，表示将内容追加写到指定的文件 | 目前只允许">>"   |
| path             | 将要写入内容的文件                     | 必须为带绝对路径的文件 |

使用指南

- writeproc用于将指定内容写入proc文件系统中的文件，该文件必须是已经存在的文件，也可以是非用户创建的文件。
- 当操作符为">>"，同时文件存在时，将指定内容写入该文件。
- 使用writeproc命令前，需要使能PROC文件系统，即设置LOSCFG\_FS\_PROC = y。

使用实例

举例：  
输入writeproc 'sys=2' >> /proc/umap/logmpp。

## 输出说明

图 9-11 修改 logmmp 中的 sys 等级

```
Huawei LiteOS # writeproc 'sys=2' >> /proc/umap/logmmp
sys=2 >> /proc/umap/logmmp

Huawei LiteOS # cat /proc/umap/logmmp

Huawei LiteOS # LOG BUFFER STATE-----
MaxLen ReadPos WritePos ButtPos
 64(KB) 0 117 65536

-----CURRENT LOG LEVEL-----
vb : 3
sys : 2
chnl : 3
vpss : 3
venc : 3
h264e : 3
jpeg : 3
viu : 3
rc : 3
isp : 3
vgs : 3
```

## 9.2.8 网络命令参考

### 9.2.8.1 使能网络命令

- Shell中的网络命令需要在tcpip\_init初始化和Telnet连接后才能起作用，Huawei LiteOS默认不初始化tcpip\_init。
- 使用Shell中的网络命令前，需要打开menuconfig使能Shell，详见[配置项](#)。

### 9.2.8.2 ifconfig

#### 命令功能

ifconfig命令用于查询和设置网卡的IP地址、网络掩码、网关、硬件MAC地址等参数，同时配置协议栈启用/关闭网卡的数据处理。

#### 命令格式

```
ifconfig
[-a]
<interface> [<address>] [netmask <address>] [gateway <address>]
[hw ether <address>] [mtu <size>]
[add <address>]
[del <address>]
[up | down]
```

参数说明

表 9-26 参数说明

| 参数        | 参数说明                                                | 取值范围      |
|-----------|-----------------------------------------------------|-----------|
| 不带参数      | 打印所有网卡的IP地址、网络掩码、网关、硬件MAC地址、MTU、运行状态等信息             | N/A       |
| -a        | 打印协议栈收发数据信息                                         | N/A       |
| interface | 指定网卡名，比如eth0                                        | N/A       |
| address   | 设置IP地址，比如192.168.1.10                               | N/A       |
| netmask   | 设置子网掩码，后面是掩码参数，比如255.255.255.0                      | N/A       |
| gateway   | 设置网关，后面是网关参数，比如192.168.1.1                          | N/A       |
| hw ether  | 设置MAC地址，后面是MAC地址，比如00:11:22:33:44:55，目前只支持ether硬件类型 | N/A       |
| mtu       | 设置MTU大小，后面是MTU大小，比如1000                             | [68,1500] |
| add       | 设置IPv6的地址，比如2001:db8:2:3:4:5:6:7，需指定inet6           | NA        |
| del       | 删除IPv6的地址，需指定inet6                                  | NA        |
| up        | 启用网卡数据处理                                            | N/A       |
| down      | 关闭网卡数据处理                                            | N/A       |

须知

ifconfig命令除了不带参数和带-a参数外，使用其他参数时都需指定网卡名。

使用指南

- ifconfig命令可用于查询和设置IP地址，掩码，网关，MAC地址等。
- ifconfig命令需要启动TCP/IP协议栈后才能使用。
- 由于IP冲突检测需要反应时间，每次使用ifconfig设置IP后会有2S左右的延时。

使用实例

1. 输入命令 ifconfig eth0 192.168.100.31 netmask 255.255.255.0 gateway 192.168.100.1 hw ether 00:11:22:33:44:55。

设置开发板的eth0网卡IP192.168.100.31，掩码255.255.255.0，网关192.168.100.1，硬件MAC地址00:11:22:33:44:55。

- 2. 输入命令 `ifconfig -a` 获取协议栈统计信息。
- 3. 输入命令 `ifconfig eth0 inet6 add 2001:db8:2:3:4:5:6:7`，设置开发板 IPv6 地址为 2001:db8:2:3:4:5:6:7（前缀长度固定为64）。
- 4. 输入命令 `ifconfig eth0 inet6 del 2001:db8:2:3:4:5:6:7`，删除开发板 IPv6的地址。

输出说明

- 1. 设置网络参数。

```
Huawei LiteOS # ifconfig
eth0 ip:192.168.1.2 netmask:255.255.255.0 gateway:192.168.1.1
 HWaddr 00:11:22:33:44:55 MTU:1500 Runing Default Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
 HWaddr 00 MTU:0 Runing Link Down

Huawei LiteOS # ifconfig eth0 192.168.100.31 netmask 255.255.255.0 gateway 192.168.100.1 hw
ether 00:11:22:33:44:55

Huawei LiteOS # ifconfig
eth0 ip:192.168.100.31 netmask:255.255.255.0 gateway:192.168.100.1
 HWaddr 00:11:22:33:44:55 MTU:1500 Runing Default Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
 HWaddr 00 MTU:0 Runing Link Down
```

ifconfig 各输出项说明如下表所示：

| 输出项          | 说明              |
|--------------|-----------------|
| ip           | 开发板IP地址         |
| netmask      | 网络掩码            |
| gateway      | 网关              |
| HWaddr       | 开发板硬件MAC地址      |
| MTU          | 网络最大传输单元        |
| Running/Stop | 网卡是否正在运行        |
| Default      | 有这项说明此网卡连接到默认网关 |
| Link UP/Down | 网卡连接状态          |

- 2. 获取协议栈统计信息。

```
Huawei LiteOS # ifconfig -a
RX packets:23128 error:0 dropped:0 overrun:0 bytes:10390(10.1KB)
TX packets:40921 error:0 dropped:0 overrun:0 bytes:64008(62.5KB)
```

ifconfig -a各输出项说明如下表所示：

| 输出项        | 说明                                                          |
|------------|-------------------------------------------------------------|
| RX packets | IP层已接收的正常数据包的个数                                             |
| RX error   | IP层已接收的错误数据包的个数，错误类型包括长度错误，校验错误，IP option错误，IP首部protocol错误等 |



| 输出项        | 说明                                           |
|------------|----------------------------------------------|
| RX dropped | IP层已丢弃的数据包的个数，丢弃原因包括数据包错误，封包无法转发，本地网卡处于关闭状态等 |
| RX overrun | MAC层向上层协议栈投递封包失败的个数，失败原因主要是协议栈资源不足           |
| RX bytes   | IP层已接收的正常数据包的总长度，不包括重组未完成的分片的长度              |
| TX packets | IP层已正常发送或转发的数据包的个数                           |
| TX error   | IP层发送失败的数据包的个数，失败原因包括封包无法路由，封包在协议栈内处理失败等     |
| TX dropped | MAC层由于发送失败而丢弃的数据包个数，失败原因包括网卡驱动处理封包失败等        |
| TX overrun | 暂未使用                                         |
| TX bytes   | IP层已正常发送或者转发的数据包的总长度                         |

3. 设置IPv6的地址。

```
Huawei LiteOS # ifconfig eth0 inet6 add 2001:db8:2:3:4:5:6:7

Huawei LiteOS # ifconfig
eth1 ip:192.168.3.60 netmask:255.255.255.0 gateway:0.0.0.0
 HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
eth0 ip:192.168.2.60 netmask:255.255.255.0 gateway:0.0.0.0
 ip6: 2001:db8:2:3:4:5:6:7/64
 HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
 ip6: ::*/64
 HWaddr 00 MTU:16436 Running Link UP
```

4. 删除IPv6的地址。

```
Huawei LiteOS # ifconfig eth0 inet6 del 2001:db8:2:3:4:5:6:7

Huawei LiteOS # ifconfig
eth1 ip:192.168.3.60 netmask:255.255.255.0 gateway:0.0.0.0
 HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
eth0 ip:192.168.2.60 netmask:255.255.255.0 gateway:0.0.0.0
 HWaddr 00:11:22:33:44:55 MTU:1500 Running Link UP
lo ip:127.0.0.1 netmask:255.0.0.0 gateway:127.0.0.1
 ip6: ::*/64
 HWaddr 00 MTU:16436 Running Link UP
```

9.2.8.3 dns

命令功能

dns命令用于设置开发板的dns服务器地址。

命令格式

dns <1-2> IP

dns -a

参数说明

表 9-27 参数说明

| 参数  | 参数说明               | 取值范围 |
|-----|--------------------|------|
| 1-2 | 选择设置第一个还是第二个DNS服务器 | 1~2  |
| IP  | 要设置的DNS服务器IP地址     | -    |
| -a  | 显示DNS当前的设置状态       | -    |

使用实例

举例：

- 1. 检查当前DNS设置，输入dns -a。
- 2. 设置第二个DNS服务器IP，输入dns 2 192.168.1.2。
- 3. 检查DNS设置是否成功，输入dns -a。

输出说明

- 1. 检查当前DNS设置：  
Huawei LiteOS # dns -a  
dns1: 208.\*\*.\*\*\*.\*\*\*  
dns2: 0.0.0.0
- 2. 设置第二个DNS服务器IP：  
Huawei LiteOS # dns 2 192.168.1.2
- 3. 检查DNS设置是否成功：  
Huawei LiteOS # dns -a  
dns1: 208.\*\*.\*\*\*.\*\*\*  
dns2: 192.168.1.2

9.2.8.4 arp

命令功能

在以太网（局域网）中，主机之前的通信是直接使用MAC地址（非IP地址）的，所以对于使用IP通信的协议，必须能将IP地址转换成MAC地址，才能在局域网内通信。解决这个问题的方法就是主机存储一张IP地址和MAC地址对应的表，即ARP缓存，主机要往一个局域网内的目的IP地址发送IP包时，就可以从ARP缓存表中查询到目的MAC地址。ARP缓存由TCP/IP协议栈维护，用户可通过arp命令查看和修改ARP表。

命令格式

arp  
  
arp [-i /I] -s IPADDR HWADDR  
  
arp [-i /I] -d IPADDR

参数说明

表 9-28 参数说明

| 参数                  | 参数说明                                     | 取值范围 |
|---------------------|------------------------------------------|------|
| 无                   | 打印ARP缓存表的内容                              | N/A  |
| -i IF               | 指定网络接口（可选参数）                             | N/A  |
| -s IPADDR<br>HWADDR | 增加一条ARP表项，后面的参数是局域网中另一台主机的IP地址及其对应的MAC地址 | N/A  |
| -d IPADDR           | 删除一条ARP表项                                | N/A  |

使用指南

- arp命令用来查询和修改TCP/IP协议栈的ARP缓存表，增加非局域网内的ARP表项是没有意义的，协议栈会返回失败。
- 命令需要启动TCP/IP协议栈后才能使用。

使用实例

- 举例：
- 输入arp。
  - 输入arp -s 192.168.1.1 00:11:22:33:44:55。

输出说明

图 9-12 打印整个 ARP 缓存表

```

Huawei LiteOS # arp
Address Hwaddress Iface Type
192.168.1.3 00:D8:61:3E:49:01 eth0 dynamic
192.168.1.100 00:00:00:11:11:11 eth0 static

```

表 9-29 上图的输出说明

| 输出项       | 说明                                                 |
|-----------|----------------------------------------------------|
| Address   | 连接到开发板的网络设备IP地址                                    |
| Hwaddress | 连接到开发板的网络设备MAC地址                                   |
| Iface     | 表示该ARP表项使用的接口名                                     |
| Type      | 表示该ARP表项是动态的还是静态的，动态是指ARP表项由协议栈自动创建，静态是指ARP表项由用户增加 |

9.2.8.5 ntpdate

命令功能

ntpdate命令用于从NTP服务器同步系统时间。

命令格式

ntpdate <SERVER\_IP1> <SERVER\_IP2>...

参数说明

表 9-30 参数说明

| 参数        | 参数说明     | 取值范围 |
|-----------|----------|------|
| SERVER_IP | NTP服务器IP | -    |

使用指南

执行ntpdate <SERVER\_IP1> <SERVER\_IP2>...，会获取第一个有效服务器IP的时间并显示。

使用实例

举例：  
输入ntpdate。

输出说明

使用ntpdate命令更新系统时间：  
Huawei LiteOS # ntpdate 192.168.1.3  
time server 192.168.1.3: Mon Jun 13 09:24:25 2016

因为开发板和服务器时区的不同，获取后的显示时间可能和服务器时间有数小时的差别。

9.2.8.6 ping

命令功能

ping命令用于测试网络连接是否正常。

命令格式

ping [-n <cnt>] [-w <interval>] [-l <data\_len>] IP  
ping [-t] [-w <interval>] IP  
ping -k

参数说明

表 9-31 参数说明

| 参数            | 参数说明                                         | 取值范围    |
|---------------|----------------------------------------------|---------|
| IP            | 要测试网络是否连通的IP地址                               | -       |
| -n <cnt>      | 执行的次数，不带本参数则默认为4次                            | 1~65535 |
| -w <interval> | 发送两次ping包的时间间隔，单位毫秒                          | -       |
| -l <data_len> | ping包的长度，即ICMP echo request报文的数据长度，不包含ICMP包头 | 0~65500 |
| -t            | 表示永久ping，直到使用ping -k杀死ping线程                 | -       |
| -k            | 杀死ping线程，停止ping                              | -       |

使用指南

- ping命令用于测试到目的IP的网络连接是否正常，参数为目的IP地址。
- 如果目的IP不可达，会显示请求超时。
- 如果显示发送错误，说明没有到目的IP的路由。
- ping命令需要启动TCP/IP协议栈后才能使用。

使用实例

举例：  
输入ping 192.168.0.2。

输出说明

图 9-13 ping 192.168.0.2

```
Huawei LiteOS# ping 192.168.0.2
Reply from 192.168.0.2: time=2ms TTL=128
Reply from 192.168.0.2: time=1ms TTL=128
Reply from 192.168.0.2: time=5ms TTL=128
Reply from 192.168.0.2: time=2ms TTL=128
```

9.2.8.7 ping6

命令功能

ping6用于测试IPv6网络连接是否正常。

命令格式

```
ping6 [-c <count>] [-I <interface/sourceAddress>] <destination>
```

参数说明

表 9-32 参数说明

| 参数                 | 参数说明                 | 取值范围    |
|--------------------|----------------------|---------|
| -c <count>         | 执行的次数，不带本参数则默认为4次    | 1~65535 |
| -I <interface>     | 使用指定的IPv6网卡测试IPv6网络  | NA      |
| -I <sourceAddress> | 使用指定的源IPv6地址测试IPv6网络 | NA      |

使用指南

- ping6命令用于测试到目的IPv6的网络连接是否正常，参数为目的IPv6地址。
- 如果目的IPv6地址不可达，会显示请求超时。
- 如果显示发送错误，说明没有到达目的IPv6的路由。
- ping6命令需要启动TCP/IP协议栈后才能使用。

使用实例

- ping6 destination：使用默认的选项测试IPv6网络。
- ping -c destination：使用指定次数测试IPv6网络。
- ping -I interface destination：使用指定网卡测试IPv6。
- ping -I sourceAddress destination：使用指定的源IPv6地址测试。

输出说明

1. 输入ping6 2001:db8:2:3:4:5:6:7。

Huawei LiteOS # ping6 2001:db8:2:3:4:5:6:7  
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=1 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=2 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=3 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=4 time<1 ms  
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---  
4 packets transmitted, 4 received, 0.00% packet loss, time 20ms  
rtt min/avg/max = 0/0.00/0 ms
2. 输入ping6 -c 3 2001:db8:2:3:4:5:6:7。

Huawei LiteOS # ping6 -c 3 2001:db8:2:3:4:5:6:7  
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=1 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=2 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=3 time<1 ms  
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---  
3 packets transmitted, 3 received, 0.00% packet loss, time 20ms  
rtt min/avg/max = 0/0.00/0 ms

3. 输入ping6 -I eth0 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7。  
Huawei LiteOS # ping6 -I eth0 2001:db8:2:3:4:5:6:7  
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=1 time=10 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=2 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=3 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=4 time<1 ms  
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---  
4 packets transmitted, 4 received, 0.00% packet loss, time 30ms  
rtt min/avg/max = 0/2.50/10 ms
4. 输入ping6 -I 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7。  
Huawei LiteOS # ping6 -I 2001:db8:2:3:4:5:6:7 2001:db8:2:3:4:5:6:7  
PING 2001:db8:2:3:4:5:6:7 with 56 bytes of data.  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=1 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=2 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=3 time<1 ms  
56 bytes from 2001:db8:2:3:4:5:6:7 : icmp\_seq=4 time<1 ms  
--- 2001:db8:2:3:4:5:6:7/64 ping statistics ---  
4 packets transmitted, 4 received, 0.00% packet loss, time 20ms  
rtt min/avg/max = 0/0.00/0 ms

9.2.8.8 tftp

命令功能

TFTP（Trivial File Transfer Protocol，简单文件传输协议）是TCP/IP协议族中的用于在客户机与服务器之间进行简单文件传输的协议，提供不复杂、开销不大的文件传输服务。端口号为69。

命令格式

tftp <-g | -p> -l <FullPathLocalFile> -r <RemoteFile> <Host>

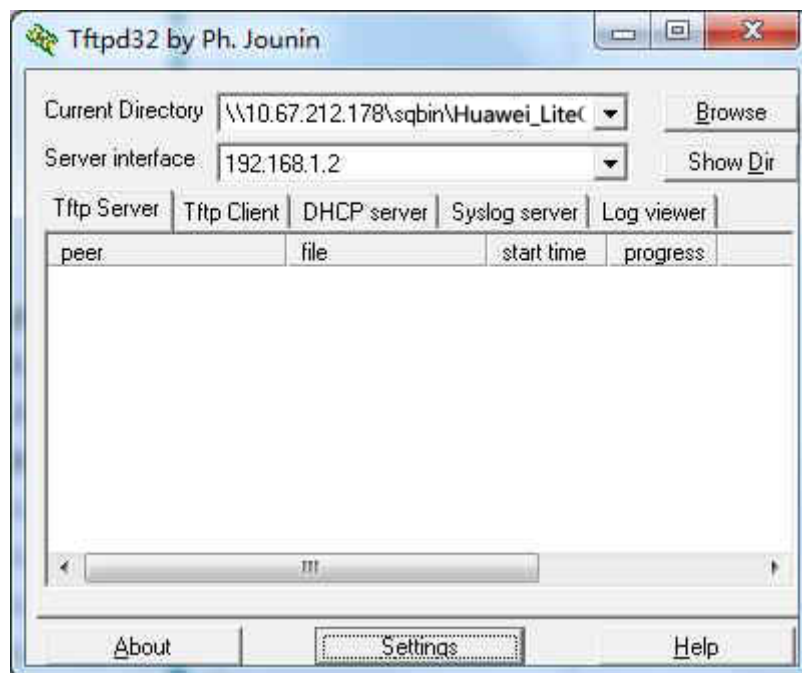
参数说明

表 9-33 参数说明

| 参数   | 参数说明                       | 取值范围        |
|------|----------------------------|-------------|
| -g   | 从服务器获取文件                   | 选择-g和-p中的一个 |
| -p   | 上传文件到服务器                   | 选择-g和-p中的一个 |
| -l   | 本地文件名（Huawei LiteOS需全路径打开） | -           |
| -r   | 服务端文件名                     | -           |
| Host | 服务端IP                      | -           |

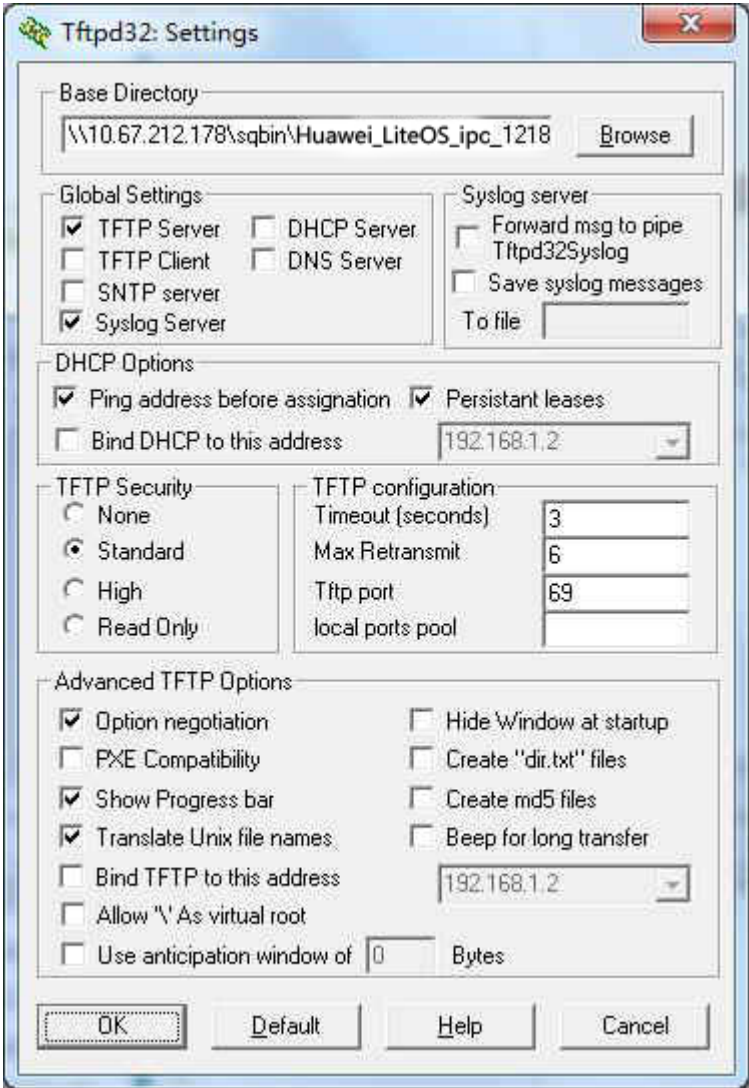
使用指南

1. 在服务器端搭建TFTP服务器。首先需保证服务器端已经安装了TFTP客户端，然后按照下图所示进行配置：



点Setting设置TFTP服务器：





这里主要设置Base Directory这项为TFTP目录，将其设置为镜像文件所在路径，然后点OK退出。

- Huawei LiteOS开发板使用tftp命令上传、下载文件。
  - 传输的文件大小不能大于32M。
  - tftp属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。
  - 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

使用实例

举例：  
从服务器下载vs\_server.bin文件。

输出说明

```

Huawei LiteOS # tftp -g -l /nfs/vs_server.bin -r vs_server.bin 192.168.1.2
TFTP transfer finish

```

tftp命令执行后，传输正常完成会显示TFTP transfer finish，失败的话会显示其他的打印信息帮助定位问题。

9.2.8.9 netstat

命令功能

netstat是控制台命令，是监测TCP/IP网络的一个非常有用的工具。它可以显示实际的网络连接以及每一个网络接口设备的状态信息。netstat用于显示与TCP、UDP协议相关的统计数据，一般用于检验本设备（开发板）各端口的网络连接情况。

命令格式

netstat

使用实例

举例：  
输入netstat。

输出说明

图 9-14 netstat 打印信息

```
Huawei LiteOS # netstat
===== total sockets 128 ===== unused sockets 122 === BootTime 27 s=====
Proto Recv-Q Send-Q Local Address Foreign Address State Time
tcp 0 0 192.168.1.102:63758 192.168.1.2:5001 ESTABLISHED 0

Proto Recv-Q Send-Q Local Address Foreign Address
udp 0 0 0.0.0.0:1337 0.0.0.0:0

Type Recv-Q Send-Q Local Address Foreign Address Protocol HDRINCL
raw 88 0 0.0.0.0 0.0.0.0 6 1
```

表 9-34 图中各输出项说明

| 输出项             | 说明                                                                                                       |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Proto           | 协议类型                                                                                                     |
| Recv-Q          | 未被用户读取的数据量，对于Listen TCP，此值为已完成三次握手，但是未被用户accept的TCP连接的数量                                                 |
| Send-Q          | 对TCP连接，已发送但未确认的数据量；对UDP连接，由于IP地址解析未完成而缓存的数据量                                                             |
| Local Address   | 本地地址和端口                                                                                                  |
| Foreign Address | 远程地址和端口                                                                                                  |
| State           | TCP连接状态，UDP此项无意义                                                                                         |
| Type            | 原始套接字类型： <ul style="list-style-type: none"><li>raw指AF_INET原始套接字</li><li>pkt-raw指PF_PACKET原始套接字</li></ul> |

| 输出项      | 说明                                                                                                                                                    |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Protocol | <ul style="list-style-type: none"> <li>对于AF_INET原始套接字，protocol指的是IP首部协议类型</li> <li>对于PF_PACKET原始套接字，protocol 指的是eth首部协议类型</li> </ul>                  |
| HDRINCL  | 用户是否对AF_INET原始套接字设置了IP_HDRINCL属性： <ul style="list-style-type: none"> <li>1表示设置了此标志</li> <li>0表示没有设置</li> </ul>                                        |
| netif    | PF_PACKET 原始套接字所绑定的网卡，None表明当前原始套接字没有绑定网卡                                                                                                             |
| Time     | <ul style="list-style-type: none"> <li>TCP套接字处于CLOSED状态时，表示套接字创建的时间</li> <li>TCP套接字CLOSE_WAIT或LAST_ACK状态时，表示套接字收到FIN的时间</li> <li>其他状态时没有意义</li> </ul> |

说明

“===== total sockets 32 ===== unused sockets 22 BootTime 27 s ===== ”

上述打印信息表示一共有32个套接字，未使用套接字22个，距系统启动已经过了27秒。

9.2.8.10 ipdebug

命令功能

ipdebug是控制台命令，是调试IPv6的一个非常有用的工具，它可以显示IPv6地址前缀，邻居条目信息，目的地缓存条目以及默认的路由条目。

命令格式

ipdebug

使用实例

举例：

输入ipdebug。

输出说明

ipdebug打印信息如下：

```

Huawei LiteOS # ipdebug
=====
|| Prefix List ||
=====
Prefix validLifetime Flags
=====
=====

```

```
0000:: 86384 1

=====
|| Neighbor Cache Entries ||
=====
0000::1 dev eth0 lladr 00:11:22:33:44:55 STALE Router
FE80::4639:C4FF:FE94:5D44 dev eth0 lladr 00:11:22:33:44:55 REACHABLE Router

=====
|| Destination Cache Entries ||
=====
0000::1 0000::1 pmtu 1500 age 152
FE80::4639:C4FF:FE94:5D44 FE80::4639:C4FF:FE94:5D44 pmtu 1500 age 6
0000::1 FE80::4639:C4FF:FE94:5D44 pmtu 1500 age 17
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219
:: :: pmtu 0 age 219

=====
|| Default Router Entries ||
=====
FE80::4639:C4FF:FE94:5D44 invalidation_timer 1784 flags 0

```

9.2.8.11 netdebug

命令功能

- 1、查询协议栈内存资源状态和关键内存信息。
- 2、查询socket全局资源使用情况或关联数据信息。

命令格式

```
netdebug memp [-i / -d <type> / -udp / -tcp / -raw / -conn]
netdebug sock [-i / -d <idx>]
```

参数说明

表 9-35 参数说明

| 参数   |              | 参数说明           | 取值范围 |
|------|--------------|----------------|------|
| memp | -i           | 查询所有内存资源信息     |      |
|      | -d<br><type> | 查询指定类型内存资源链表信息 |      |

| 参数   |          | 参数说明                             | 取值范围  |
|------|----------|----------------------------------|-------|
|      | -udp     | 查询udp pcb信息                      |       |
|      | -tcp     | 查询tcp pcb信息                      |       |
|      | -raw     | 查询raw pcb信息                      |       |
|      | -conn    | 查询netconn信息                      |       |
| sock | -i       | 查询全部socket信息                     |       |
|      | -d <idx> | 查询指定索引socket关联信息，包括netconn和pcb信息 | 0~127 |

使用指南

netdebug命令用于查看资源使用情况，可以在问题定位时查询相关数据信息。

使用实例

- 1. netdebug memmp -i 查询内存资源信息。
- 2. netdebug memmp -d 12 查询类型为12的内存资源信息。
- 3. netdebug memmp -udp 查询udp pcb信息。
- 4. netdebug memmp -tcp 查询tcp pcb信息。
- 5. netdebug memmp -raw 查询raw pcb信息。
- 6. netdebug memmp -conn 查询netconn信息。
- 7. netdebug sock -i 查询socket信息。
- 8. netdebug sock -d 2 查询索引为2的socket关联信息。

输出说明

图 9-15 输入 netdebug memmp -i 返回结果

```
Huawei LiteOS # netdebug memmp -i
```

| [MEMP]    | TYPE | START      | END        | TAB        | NUM   | USED | AVAILABLE |
|-----------|------|------------|------------|------------|-------|------|-----------|
| [MEMP] 0  |      | 0x801c8508 | 0x801c9adc | 0x801c9adc | 128   | 0    | 128       |
| [MEMP] 1  |      | 0x801c9b08 | 0x801cb0dc | 0x801cb084 | 128   | 2    | 126       |
| [MEMP] 2  |      | 0x801cb108 | 0x801d3400 | 0x801d3400 | 128   | 0    | 128       |
| [MEMP] 3  |      | 0x801d3508 | 0x801d46e4 | 0x801d46e4 | 128   | 0    | 128       |
| [MEMP] 4  |      | 0x801d4708 | 0x80261db0 | 0x80261db0 | 14482 | 0    | 14482     |
| [MEMP] 5  |      | 0x80261dd8 | 0x802641b4 | 0x802641b4 | 256   | 0    | 256       |
| [MEMP] 6  |      | 0x802641d8 | 0x802b7964 | 0x802b7964 | 17096 | 0    | 17096     |
| [MEMP] 7  |      | 0x802b7978 | 0x802c6900 | 0x802c6810 | 512   | 2    | 510       |
| [MEMP] 8  |      | 0x802c6978 | 0x802c6e64 | 0x802c6e64 | 64    | 0    | 64        |
| [MEMP] 9  |      | 0x802c6e78 | 0x802c8264 | 0x802c8250 | 256   | 0    | 256       |
| [MEMP] 10 |      | 0x802c8278 | 0x802d3480 | 0x802d3480 | 5698  | 0    | 5698      |
| [MEMP] 11 |      | 0x802d3488 | 0x802d3514 | 0x802d3500 | 8     | 1    | 7         |
| [MEMP] 12 |      | 0x802d3528 | 0x802d3598 | 0x802d3538 | 8     | 6    | 2         |
| [MEMP] 13 |      | 0x802d35a8 | 0x802d3e14 | 0x802d3e14 | 8     | 0    | 8         |
| [MEMP] 14 |      | 0x802d3f48 | 0x802d5b2c | 0x802d5b2c | 256   | 0    | 256       |
| [MEMP] 15 |      | 0x802d5b48 | 0x802ee01c | 0x802ee01c | 64    | 0    | 64        |

图 9-16 输入 netdebug memp -i -d 12 返回结果

```
Huawei LiteOS # netdebug memp -d 12
[MEMF]TYPE INDEX ADDR NEXT
[MEMF]12 0 0x802d3528 0x0
[MEMF]12 1 0x802d3538 0x802d3528
[MEMF]12 2 0x802d3548 0x802d3598
[MEMF]12 3 0x802d3558 0x802d3568
[MEMF]12 4 0x802d3568 0x802d3548
[MEMF]12 5 0x802d3578 0x0
[MEMF]12 6 0x802d3588 0x802d3578
[MEMF]12 7 0x802d3598 0x802d3588
```

图 9-17 输入 netdebug memp -udp 返回结果

```
Huawei LiteOS # netdebug memp -udp
BootTime 112 s

-----IMPPCB
pcb :0x8031a96c
l_ip :0.0.0.0
r_ip :0.0.0.0
next :0x0
flags :0
l_port :1337
r_port :0
```

图 9-18 输入 netdebug memp -tcp 返回结果

```
Huawei LiteOS # netdebug memp -tcp
BootTime 135 s

-----TCPPCB
pcb :0x80323368
l_ip :0.0.0.0
r_ip :0.0.0.0
cb_arg :0x80417b40
l_port :0
r_port :0
state :0
flags :64
```

图 9-19 输入 netdebug memp -raw 返回结果

```
Huawei LiteOS # netdebug memp -raw
BootTime 112 s

-----RAWPCB
pcb :0x8031936c
l_ip :0.0.0.0
r_ip :0.0.0.0
hdrincl :1
next :0x0
netifidx:0
proto :6
e_proto :6
```

图 9-20 输入 netdebug memp -conn 返回结果

```
Huawei LiteOS # netdebug memp -conn
BootTime 147 s

-----NETCONN
conn :0x80417ce4
pcb :0x8031a96c
socket :256
r_buf_s :2000
r_avail :0
l_err :0
is_fatal:0
type :32
state :0
shutdown:0
```

图 9-21 输入 netdebug sock -i 返回结果

```
Huawei LiteOS # netdebug sock -i
-----SOCKET
idx :0
conn :0x802c6900
r_event :0
s_event :1
e_event :0
err :0
events :1
select_w :0
sock_flag:1
poll_prev:0x802ee6a4
poll_next:0x802ee6a4

-----SOCKET
idx :1
conn :0x802c6888
r_event :0
s_event :1
e_event :0
err :0
events :0
select_w :0
sock_flag:1
poll_prev:0x802ee6f8
poll_next:0x802ee6f8
```



图 9-22 输入 netdebug sock -d 0 返回结果

```
Huawei LiteOS # netdebug sock -d 0
BootTime 165 s

-----SOCKET
idx :0
conn :0x80417ce4
r_event :0
s_event :1
e_event :0
err :0
sock_flag:1

-----NETCONN
conn :0x80417ce4
pcb :0x8031a96c
socket :256
r_buf_s :2000
r_avail :0
l_err :0
is_fatal:0
type :32
state :0
shutdown:0

-----UDPPCB
pcb :0x8031a96c
l_ip :0.0.0.0
r_ip :0.0.0.0
next :0x0
flags :0
l_port :1337
r_port :0
```

9.2.8.12 telnet

命令功能

telnet用于从终端使用者的电脑上通过网络连接到服务器。

命令格式

telnet [*on / off*]

参数说明

表 9-36 参数说明

| 参数  | 参数说明        | 取值范围 |
|-----|-------------|------|
| on  | 启动server服务端 | N/A  |
| off | 关闭server服务端 | N/A  |

### 使用指南

- telnet用于从终端使用者的电脑上通过网络连接到服务器。
- 启动telnet前要确保网络驱动及网络协议栈已经初始化完成，且开发板的网卡是link up状态。
- 暂时无法支持多个客户端（telnet + IP）连接开发板。
- telnet属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

### 使用实例

举例：  
输入telnet on。

### 输出说明

图 9-23 输入 telnet on

```
Huawei LiteOS # telnet on
Huawei LiteOS # init telnet.
```

### 9.2.8.13 iperf

#### 命令功能

iperf是一个网络性能测试工具，可以测试TCP和UDP的带宽性能、延迟抖动和数据包丢失。

#### 命令格式

iperf -s [options]  
iperf [-c <host>] [options]  
iperf -k

#### 参数说明

表 9-37 通用参数说明

| 参数 | 参数说明                          | 取值范围 |
|----|-------------------------------|------|
| -V | 使用IPv6地址（如需使用必须作为第一个参数，放在-s前） | 无参数  |
| -u | 设置为UDP连接                      | 无参数  |

| 参数              | 参数说明                                                                                                                          | 取值范围               |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------|
| -i <n>          | 设置数据报告间隔时间为n秒（默认1秒）                                                                                                           | n为无符号整数，单位：秒       |
| -l <n>          | 设置读写缓冲区大小为n，默认8KB(TCP)/1470B(UDP)                                                                                             | n为无符号整数，单位：字节      |
| -p <n>          | 设置要监听的server端网络端口号为n，默认5001                                                                                                   | n为无符号整数，无单位        |
| -S <n>          | 设置IPv4服务类型为n，默认值0                                                                                                             | n为无符号整数，取值范围为0-255 |
| -B <ip>         | 绑定主机地址 <ul style="list-style-type: none"> <li>作为server端运行时，绑定从哪些client端接收数据</li> <li>作为client端运行时，绑定从哪些server端接收数据</li> </ul> | ip可以是IPv4或IPv6地址   |
| -k              | 终止iperf的所有线程                                                                                                                  | 无参数                |
| -h -help --help | 显示帮助信息                                                                                                                        | 无参数                |

表 9-38 server 端特有参数说明

| 参数 | 参数说明       | 取值范围 |
|----|------------|------|
| -s | 设置为server端 | 无参数  |

表 9-39 client 端特有参数说明

| 参数        | 参数说明                         | 取值范围                     |
|-----------|------------------------------|--------------------------|
| -c <host> | 设置为client端，连接到指定的host        | host为server端的IPv4/IPv6地址 |
| -t <n>    | 设置数据发送的总时间为n秒，默认30秒          | n为无符号整数，单位：秒             |
| -n <m>    | 设置发送数据的总量为m字节                | m为无符号整数，单位：字节            |
| -b <n>    | 设置UDP连接场景下发送的带宽为n比特率，默认1Mbps | n为无符号整数，单位：比特率           |

## 使用指南

- iperf工具需要使能shell、lwIP、iperf后才能使用，如果需要统计UDP连接下的丢包率、抖动延迟数据，还需要使能Udp Jitter。可按如下菜单项使能。

```
Debug --> Enable a Debug Version --> Enable Shell
Net --> Enable Lwip
```

```
Net --> Enable Lwip --> Enable Iperf
Net --> Enable Lwip --> Enable Iperf --> Enable Udp Jitter
```

- 当设置-p、-l参数时，server端和client端必须将这些参数设置成相同值。
- iperf -k终止时，由于线程同步以及套接字连接操作问题，存在1秒的阻塞延迟。
- 同一设备在同一时间内只能运行一个iperf实例，当已有iperf实例运行时，再启动iperf会提示“Iperf is running”。

使用实例

- IPv4下TCP带宽测试  
server端执行：iperf -s  
client端执行：iperf -c 192.168.1.1 -i 1 -t 10
- IPv6下UDP带宽测试  
server端执行：iperf -V -s -p 5001 -i 5 -u  
client端执行：iperf -V -c FFFF::ABCD:6666:1234:5678 -p 5001 -i 1 -t 10 -u -b 10M

输出说明

- IPv4下TCP带宽测试  
server端输出如下所示，每秒都会输出数据传输量以及带宽，总测试时间为10秒，并在结束测试后输出整个过程的统计数据：  
+IPERF:  

| Interval     | Transfer      | Bandwidth  |
|--------------|---------------|------------|
| 0.0- 1.0 sec | 11.26 MBytes  | 94.42 Mb/s |
| 1.0- 2.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 2.0- 3.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 3.0- 4.0 sec | 11.28 MBytes  | 94.58 Mb/s |
| 4.0- 5.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 5.0- 6.0 sec | 11.28 MBytes  | 94.58 Mb/s |
| 6.0- 7.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 7.0- 8.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 8.0- 9.0 sec | 11.27 MBytes  | 94.57 Mb/s |
| 9.0-10.0 sec | 11.28 MBytes  | 94.58 Mb/s |
| 0.0-10.0 sec | 113.00 MBytes | 94.56 Mb/s |

  
OK  
client端的输出与server端的输出数据基本一致。
- IPv6下UDP带宽测试  
server端输出如下所示，每隔5秒输出数据传输量以及带宽，总测试时间为10秒，并在结束测试后输出整个过程的统计数据。因为client端设置了发送带宽为10Mbps，所以可以在输出打印中看到带宽速率接近10Mbps：

```
+IPERF:
Interval Transfer Bandwidth
0.0- 5.0 sec 5.57 MBytes 9.34 Mb/s
5.0-10.0 sec 5.57 MBytes 9.34 Mb/s
0.0-10.0 sec 11.14 MBytes 9.34 Mb/s
OK
```

client端输出如下所示，因为设置 -i 1，所以每秒都会输出数据传输量以及带宽：

```
+IPERF:
Interval Transfer Bandwidth
0.0- 1.0 sec 1.13 MBytes 9.48 Mb/s
1.0- 2.0 sec 1.09 MBytes 9.14 Mb/s
2.0- 3.0 sec 1.11 MBytes 9.31 Mb/s
3.0- 4.0 sec 1.12 MBytes 9.40 Mb/s
4.0- 5.0 sec 1.12 MBytes 9.40 Mb/s
5.0- 6.0 sec 1.11 MBytes 9.31 Mb/s
```

```
6.0- 7.0 sec 1.13 MBytes 9.48 Mbits/sec
7.0- 8.0 sec 1.12 MBytes 9.40 Mbits/sec
8.0- 9.0 sec 1.12 MBytes 9.40 Mbits/sec
9.0-10.0 sec 1.09 MBytes 9.14 Mbits/sec
0.0-10.0 sec 11.14 MBytes 9.34 Mbits/sec
OK
```

9.2.9 动态加载命令参考

9.2.9.1 使能动态加载命令

- 若要运行动态加载相关的Shell命令，需要事先完成动态加载模块的初始化，具体说明查看[动态加载](#)章节。
- 使用Shell中的动态加载命令前，需要先打开menuconfig使能Shell，详见[配置项](#)。并配置动态加载模块，详见[配置动态加载](#)。

9.2.9.2 mopen

命令功能

加载一个用户模块。

命令格式

mopen <module\_path>

参数说明

表 9-40 参数说明

| 参数          | 参数说明                |
|-------------|---------------------|
| module_path | 用户模块所在路径，该路径必须为绝对路径 |

使用指南

module\_path只能为.o文件，不支持加载.so文件。

使用实例

举例，加载/yafts/bin/dynload/foo.o文件：

输入mopen /yafts/bin/dynload/foo.o。

输出说明

```
Huawei LiteOS# mopen /yafts/bin/dynload/foo.o
module handle: 0x80391928

Huawei LiteOS#
```

加载成功返回模块句柄，本例中返回的模块句柄为0x80391928。

9.2.9.3 mclose

命令功能

卸载一个模块。

命令格式

mclose <module\_handle>

参数说明

表 9-41 参数说明

| 参数            | 参数说明                            |
|---------------|---------------------------------|
| module_handle | 要卸载的模块句柄，该句柄是mopen中加载该模块时返回的句柄值 |

使用指南

卸载指定句柄的模块后，将无法再从该模块中查找符号。

使用实例

举例，卸载句柄为0x80391928的用户模块：

输入mclose 0x80391928。

输出说明

Huawei LiteOS# mclose 0x80391928

Huawei LiteOS#

无错误信息返回表示卸载成功。

9.2.9.4 findsym

命令功能

从全局符合表或者指定模块中查找符号所在地址。

命令格式

findsym <handle> <symbol\_name>

参数说明

表 9-42 参数说明

| 参数          | 参数说明          |
|-------------|---------------|
| handle      | 模块句柄，0表示全局符号表 |
| symbol_name | 所要查找的符号名      |

使用指南

- handle传入0时：表示在全局符号表中查找符号地址（全局符号表中包含了内核符号以及用户模块提供的符号）。
- handle不为0且为合法句柄时：在该handle指定的模块中查找符号。

使用实例

举例：

1. 在全局符号表中查找printf符号，输入findsym 0 printf。
2. 在mopen中加载的句柄为0x80391928的用户模块中查找test\_0符号地址，输入findsym 0x80391928 test\_0。

输出说明

```
Huawei LiteOS# findsym 0 printf
symbol address:0x8004500c

Huawei LiteOS#
Huawei LiteOS# findsym 0x80391928 test_0
symbol address:0x8030f241

Huawei LiteOS#
```

9.2.9.5 call

命令功能

调用一个无参函数。

命令格式

call <func\_address>

参数说明

表 9-43 参数说明

| 参数           | 参数说明     |
|--------------|----------|
| func_address | 函数所在内存地址 |

## 使用指南

当通过findsym查找到一个函数符号所在的内存地址后，可以通过call命令调用该函数。

### 须知

使用call命令进行任意内存操作时，会造成系统崩溃。

## 使用实例

举例：调用在findsym中查找到的地址为0x8030f241的test\_0函数，输入call 0x8030f241。

## 输出说明

```
Huawei LiteOS# call 0x8030f241
test_0
```

### 9.2.9.6 lddrop

## 命令功能

销毁动态加载模块。

## 命令格式

lddrop

## 使用指南

销毁动态加载模块后无法再使用动态加载特性。如若使用动态加载特性，需要重新加载模块以初始化动态加载模块。

## 使用实例

举例：销毁动态加载模块，输入lddrop。

## 输出说明

```
Huawei LiteOS# lddrop
Huawei LiteOS#
```

无错误信息返回表示销毁成功。

## 9.2.10 Trace 命令参考

### 9.2.10.1 使能 Trace 命令

使用Shell中的Trace命令前，需要先打开menuconfig菜单使能Shell，详见[配置项](#)。



同时配置Trace模块，详见Trace开发流程中的[配置Trace](#)，下文中的“离线模式”在menuconfig中的菜单项为：Kernel ---> Enable Extend Kernel ---> Enable Trace Feature ---> Trace work mode ---> Offline mode”。关于Trace模块的详细介绍，详见[Trace](#)。

9.2.10.2 trace\_mask

命令功能

设置事件过滤掩码。

命令格式

trace\_mask [*MASK*]

参数说明

表 9-44 参数说明

| 参数   | 参数说明      | 取值范围            |
|------|-----------|-----------------|
| MASK | Trace事件掩码 | [0, 0xFFFFFFFF] |

使用指南

- 如果不设置事件掩码，或者执行该命令时参数缺省，则默认仅开启任务和中断事件记录。
- trace\_mask后加MASK，则开启对应模块的事件记录。
- 具体的模块掩码MASK参见los\_trace.h中定义的LOS\_TRACE\_MASK。

使用实例

举例：

1. 输入trace\_mask 0
2. 输入trace\_mask 0xFFFFFFFF

输出说明

执行trace\_mask 0，设置所有模块的事件都不记录，命令执行成功后，不会输出信息。

```
Huawei LiteOS # trace_mask 0
Huawei LiteOS #
```

执行trace\_mask 0xFFFFFFFF，设置所有模块的事件都进行记录，命令执行成功后，不会输出信息。

```
Huawei LiteOS # trace_mask 0xFFFFFFFF
Huawei LiteOS #
```

### 9.2.10.3 trace\_start

#### 命令功能

开启Trace。

#### 命令格式

trace\_start

#### 使用指南

输入trace\_start即开启系统Trace功能，离线模式下会记录系统发生的事件并保存在指定buffer中。

#### 使用实例

举例：开启系统Trace功能，输入trace\_start。

#### 输出说明

在离线模式下，成功执行trace\_start命令后，不会输出信息。

```
Huawei LiteOS # trace_start
```

```
Huawei LiteOS #
```

### 9.2.10.4 trace\_stop

#### 命令功能

停止Trace。

#### 命令格式

trace\_stop

#### 使用指南

输入trace\_stop即终止系统Trace功能，停止记录事件。

#### 使用实例

举例：停止系统Trace功能，输入trace\_stop。

#### 输出说明

成功执行trace\_stop命令后，不会输出信息。

```
Huawei LiteOS # trace_stop
```

```
Huawei LiteOS #
```

9.2.10.5 trace\_dump

命令功能

在离线模式下，dump出Trace buffer的信息。

命令格式

trace\_dump [1 | 0]

参数说明

表 9-45 参数说明

| 参数 | 参数说明           | 取值范围 |
|----|----------------|------|
| 1  | 将Trace数据输出到客户端 | NA   |
| 0  | 将Trace数据格式化打印  | NA   |

使用指南

- 只能在离线模式下使用trace\_dump命令。
- 参数缺省时将格式化打印Trace数据。
- trace\_dump命令打印的是trace\_start和trace\_stop之间的数据，所以需要先执行trace\_stop停止Trace后，再执行trace\_dump打印Trace buffer的信息。

使用实例

举例：  
输入trace\_dump。

输出说明

执行trace\_dump命令，格式化打印缓存中的数据。

```
Huawei LiteOS # trace_dump
*****TraceInfo begin*****
clockFreq = 180000000
CurEvtIndex = 19
Index Time(cycles) EventType CurTask Identity params
0 0x7da8da5180 0x45 0x5 0x2 0x9 0x20 0x1f
1 0x7dde8c6980 0x45 0x2 0x5 0x1f 0x4 0x9
2 0x7e1431df20 0x45 0x5 0x2 0x9 0x20 0x1f
3 0x7e49e3f720 0x45 0x2 0x5 0x1f 0x4 0x9
4 0x7e7f896cc0 0x45 0x5 0x2 0x9 0x20 0x1f
5 0x7eb53b84c0 0x45 0x2 0x5 0x1f 0x4 0x9
6 0x7eeae0fa60 0x45 0x5 0x2 0x9 0x20 0x1f
7 0x7f20931260 0x45 0x2 0x5 0x1f 0x4 0x9
8 0x7f56388800 0x45 0x5 0x2 0x9 0x20 0x1f
9 0x7f8beaa000 0x45 0x2 0x5 0x1f 0x4 0x9
10 0x7fc19015a0 0x45 0x5 0x2 0x9 0x20 0x1f
11 0x7ff7422da0 0x45 0x2 0x5 0x1f 0x4 0x9
12 0x802ce7a340 0x45 0x5 0x2 0x9 0x20 0x1f
13 0x806299bb40 0x45 0x2 0x5 0x1f 0x4 0x9
```

```

14 0x80983f30e0 0x45 0x5 0x2 0x9 0x20 0x1f
15 0x80cdf148e0 0x45 0x2 0x5 0x1f 0x4 0x9
.....
24 0x6c560a8d00 0x24 0x2 0x2d 0x0 0x0 0x0
25 0x6c8baf7600 0x25 0x2 0x2d 0x0 0x0 0x0
.....
36 0x71fe6f2000 0x24 0x2 0x2d 0x0 0x0 0x0
37 0x7234140900 0x25 0x2 0x2d 0x0 0x0 0x0
38 0x7269c20250 0x45 0x2 0x1 0x1f 0x4 0x0
39 0x734055a650 0x45 0x1 0x2 0x0 0x8 0x1f
40 0x7380b52450 0x45 0x2 0x1 0x1f 0x4 0x0
.....
48 0x77a6d3b300 0x24 0x2 0x2d 0x0 0x0 0x0
49 0x77dc789c00 0x25 0x2 0x2d 0x0 0x0 0x0
50 0x7812269550 0x45 0x2 0x1 0x1f 0x4 0x0
.....
*****TraceInfo end*****

```

### 9.2.10.6 trace\_reset

#### 命令功能

在离线模式下，清除Trace buffer中的事件数据。

#### 命令格式

```
trace_reset
```

#### 使用指南

只能在离线模式下使用trace\_reset命令。

#### 使用实例

举例：

输入 trace\_reset。

#### 输出说明

执行trace\_reset，清除事件数据：

```

Huawei LiteOS # trace_reset
Huawei LiteOS #

```

## 9.3 调度统计

#### 功能说明

Huawei LiteOS提供调度统计维测功能，该功能用于统计CPU的一些调度信息，包括idle任务启动时间、idle任务运行时长、调度切换次数、中断次数、核间迁移次数（多核）等。

使用方法

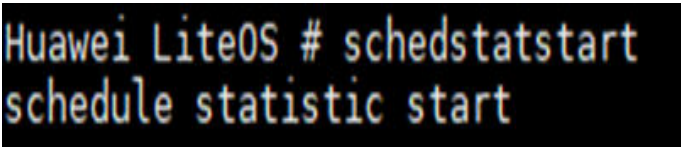
- 步骤1 在菜单项中使能Debug ---> Enable a Debug Version ---> Enable DebugLiteOS Kernel Resource ---> Enable Scheduler Statistics Debugging，可以开启调度统计维测功能（仅支持有浮点运算功能的平台）。
  - 步骤2 输入shell命令schedstatstart，开启调度统计功能。
  - 步骤3 输入shell命令schedstatstop，关闭调度统计功能，系统将输出各个CPU的调度统计信息。
  - 步骤4 输入shell命令schedstatinfo，系统将输出各个任务的CPU调度统计信息。
- 结束

注意事项

shell指令执行顺序：schedstatstart -> schedstatstop -> schedstatinfo。

输出说明

- 调用schedstatstart后系统输出如下图所示：



- 调用schedstatstop后系统输出如下图所示：

```
Huawei LiteOS # schedstatstop
Passed Time: +15694.394940 ms

CPU Idle(%) ContextSwitch HwiNum Avg Pri HiTask(%) HiTask SwiNum HiTask P(ms) MP Hwi MGR

CPU0 +99.238067 317 5 +0.225000 +0.021202 160 +0.020797 1 2
CPU1 +98.706596 0 162 +31.000000 +0.000000 0 +0.000000 162 0
CPU2 +98.908085 3 161 +9.000000 +0.003726 2 +0.292425 161 2
CPU3 +98.736067 2 162 +9.000000 +0.001431 1 +0.224580 162 1
```

参数描述如下表所示：

| 参数            | 描述                             |
|---------------|--------------------------------|
| Passed Time   | 调度统计功能运行时长                     |
| CPU           | CPU ID                         |
| Idle(%)       | Idle任务运行时长百分比                  |
| ContextSwitch | 任务调度切换次数                       |
| HwiNum        | 中断触发次数                         |
| Avg Pri       | 切入任务不为idle任务的任务优先级平均值          |
| HiTask(%)     | 高优先级任务运行时长所占百分比，定义优先级小于16为高优先级 |
| HiTask SwiNum | 切入新任务为高优先级的切换次数，定义优先级小于16为高优先级 |

|              |                              |
|--------------|------------------------------|
| HiTask P(ms) | 高优先级任务运行的平均时长，定义优先级小于16为高优先级 |
| MP Hwi       | 核间中断触发次数，仅用于多核               |
| MGR          | 核间迁移次数，仅用于多核                 |

- 调用schedstatinfo后系统输出如下图所示：

|                               |     |                  |           |           |      |                  |     |     |
|-------------------------------|-----|------------------|-----------|-----------|------|------------------|-----|-----|
| Passed Time: +15694.394940 ms |     |                  |           |           |      |                  |     |     |
| Task                          | TID | Total Time       | Total CST | Total MGR | CPU  | Time             | CST | MGR |
| Swt_Task                      | 0x0 | +2.652840 ms     | 156       | 0         |      |                  |     |     |
| IdleCore000                   | 0x1 | +15574.814130 ms | 158       | 0         | CPU0 | +2.652840 ms     | 156 | 0   |
| system_wq                     | 0x2 | +0.000000 ms     | 0         | 0         | CPU0 | +15574.814130 ms | 158 | 0   |
|                               |     |                  |           |           | CPU0 | +0.000000 ms     | 0   | 0   |
|                               |     |                  |           |           | CPU1 | +0.000000 ms     | 0   | 0   |
|                               |     |                  |           |           | CPU2 | +0.000000 ms     | 0   | 0   |
|                               |     |                  |           |           | CPU3 | +0.000000 ms     | 0   | 0   |
| Swt_Task                      | 0x4 | +0.000000 ms     | 0         | 0         | CPU2 | +0.000000 ms     | 0   | 0   |
| Swt_Task                      | 0x5 | +0.000000 ms     | 0         | 0         | CPU3 | +0.000000 ms     | 0   | 0   |
| IdleCore000                   | 0x6 | +15522.925190 ms | 1         | 0         | CPU2 | +15522.925190 ms | 1   | 0   |
| IdleCore000                   | 0x7 | +15495.945590 ms | 1         | 0         | CPU3 | +15495.945590 ms | 1   | 0   |
| Swt_Task                      | 0x8 | +0.000000 ms     | 0         | 0         | CPU1 | +0.000000 ms     | 0   | 0   |
| IdleCore000                   | 0x9 | +15491.321150 ms | 0         | 0         | CPU1 | +15491.321150 ms | 0   | 0   |
| SerialShellTask               | 0xa | +0.804040 ms     | 2         | 2         | CPU0 | +0.316340 ms     | 1   | 1   |
|                               |     |                  |           |           | CPU1 | +0.000000 ms     | 0   | 0   |
|                               |     |                  |           |           | CPU2 | +0.487700 ms     | 1   | 1   |
|                               |     |                  |           |           | CPU3 | +0.000000 ms     | 0   | 0   |
| SerialEntryTask               | 0xb | +0.576100 ms     | 4         | 3         | CPU0 | +0.254370 ms     | 2   | 1   |
|                               |     |                  |           |           | CPU1 | +0.000000 ms     | 0   | 0   |
|                               |     |                  |           |           | CPU2 | +0.097150 ms     | 1   | 1   |
|                               |     |                  |           |           | CPU3 | +0.224580 ms     | 1   | 1   |

参数描述如下表所示：

|             |                        |
|-------------|------------------------|
| 参数          | 描述                     |
| Passed Time | 调度统计功能运行时长             |
| Task        | 任务名称                   |
| TID         | 任务ID                   |
| Total Time  | 所有CPU的任务运行时长           |
| Total CST   | 所有CPU任务上下文切换次数         |
| Total MGR   | 所有CPU的核间迁移次数，仅用于多核     |
| CPU         | 任务运行的CPU ID            |
| Time        | 任务在某个CPU上的运行时长         |
| CST         | 任务在某个CPU上的上下文切换次数      |
| MGR         | 任务在某个CPU上的核间迁移次数，仅用于多核 |

## 9.4 内存调测方法

9.4.1 概述

Huawei LiteOS提供内存维测功能，在菜单项中使能Debug ---> Enable a Debug Version ---> Enable MEM Debug，可以开启内存维测。

开启内存维测后，在LOS\_MemFree和LOS\_MemRealloc接口中会根据链表关系对目标节点及前后节点进行合法性校验，若检测到异常则立即挂起系统并抛出异常。除此之外，还提供了内存维测子功能，各功能可以独立使能，详见后续章节。

9.4.2 多内存池机制

使用场景

系统中使用多个动态内存池时，需对各内存池进行管理和使用情况统计。

功能说明

系统内存机制中通过链表实现对多个内存池的管理。内存池需回收时可调用对应接口进行去初始化。

通过多内存池机制，可以获取系统各个内存池的信息和使用情况，也可以检测内存池空间分配交叉情况，当系统两个内存池空间交叉时，第二个内存池会初始化失败，并给出空间交叉的提示信息。

| 功能分类    | 接口名             | 描述                                                                                                       |
|---------|-----------------|----------------------------------------------------------------------------------------------------------|
| 初始化内存池  | LOS_MemInit     | 初始化一块指定的动态内存池，大小为size                                                                                    |
| 删除内存池   | LOS_MemDelnit   | 删除指定内存池，仅打开LOSCFG_MEM_MUL_POOL时有效                                                                        |
| 显示系统内存池 | LOS_MemPoolList | 打印系统中已初始化的所有内存池，包括内存池的起始地址、内存池大小、空闲内存总大小、已使用内存总大小、最大的空闲内存块大小、空闲内存块数量、已使用的内存块数量，仅打开LOSCFG_MEM_MUL_POOL时有效 |

使用方法

1. 打开菜单开启多内存池机制。  
功能依赖于LOSCFG\_MEM\_MUL\_POOL，使用时在菜单项中开启“Enable Memory multi-pool control”：  
Debug ---> Enable a Debug Version---> Enable MEM Debug---> Enable Memory multi-pool control
2. 调用LOS\_MemInit接口进行内存池初始化，内存池回收时调用LOS\_MemDelnit接口进行去初始化。
3. 调用LOS\_MemInfoGet获取指定内存池的信息和使用情况。
4. 调用LOS\_MemPoolList获取系统所有内存池信息和使用情况。

注意事项

- 初始化内存池时，需保证各内存池空间无交叉，若交叉则会导致初始化失败。

- malloc/free系列接口默认从OS系统内存池申请和释放内存，其它内存池的操作必须调用Huawei LiteOS内存接口（LOS\_MemAlloc等），不能调用malloc/free系列接口及其相关封装接口。
- 内存池回收必须调用LOS\_MemDelnit接口去初始化（回收前需确保池中内存块均已释放），否则二次初始化该内存池空间会失败，导致该内存池不能被重新使用。
- 内存池大小需根据业务实际情况合理分配。

## 编程实例

```
void test(void)
{
 UINT32 ret = 0;
 UINT32 size = 0x100000;

 VOID *poolAddr1 = LOS_MemAlloc(OS_SYS_MEM_ADDR, size);
 ret = LOS_MemInit(poolAddr1, size);
 if (ret != 0) {
 PRINTK("LOS_MemInit failed\n");
 return;
 }

 VOID *poolAddr2 = LOS_MemAlloc(OS_SYS_MEM_ADDR, size);
 ret = LOS_MemInit(poolAddr2, size);
 if (ret != 0) {
 PRINTK("LOS_MemInit failed\n");
 return;
 }

 PRINTK("\n*****step1 list the mem poll\n");
 LOS_MemPoolList();

 LOS_MemDelnit(poolAddr1);
 if (ret != 0) {
 PRINTK("LOS_MemDelnit failed\n");
 return;
 }

 PRINTK("\n*****step2 list the mem poll\n");
 LOS_MemPoolList();

 LOS_MemDelnit(poolAddr2);
 if (ret != 0) {
 PRINTK("LOS_MemDelnit failed\n");
 return;
 }

 PRINTK("\n*****step3 list the mem poll\n");
 LOS_MemPoolList();
}
```

log:

```
*****step1 list the mem poll
pool0 :
pool addr pool size used size free size max free node size used node num free node num

0x8017b2c0 0x100000 0x2e1fc 0xd1d20 0xd1d20 0x2b 0x1
pool1 :
pool addr pool size used size free size max free node size used node num free node num

0x8027b2c0 0x7d84d40 0x7070c8 0x767db94 0x767db94 0x1026 0x1
pool2 :
pool addr pool size used size free size max free node size used node num free node num

0x8078244c 0x100000 0x10 0xfff0c 0xffff0c 0x1 0x1
```



```

pool3 :
pool addr pool size used size free size max free node size used node num free node num

0x8088245c 0x100000 0x10 0xfff0c 0xffff0c 0x1 0x1

*****step2 list the mem poll
pool0 :
pool addr pool size used size free size max free node size used node num free node num

0x8017b2c0 0x100000 0x2e1fc 0xd1d20 0xd1d20 0x2b 0x1
pool1 :
pool addr pool size used size free size max free node size used node num free node num

0x8027b2c0 0x7d84d40 0x7070c8 0x767db94 0x767db94 0x1026 0x1
pool2 :
pool addr pool size used size free size max free node size used node num free node num

0x8088245c 0x100000 0x10 0xfff0c 0xffff0c 0x1 0x1

*****step3 list the mem poll
pool0 :
pool addr pool size used size free size max free node size used node num free node num

0x8017b2c0 0x100000 0x2e1fc 0xd1d20 0xd1d20 0x2b 0x1
pool1 :
pool addr pool size used size free size max free node size used node num free node num

0x8027b2c0 0x7d84d40 0x7070c8 0x767db94 0x767db94 0x1026 0x1

```

### 9.4.3 内存备份机制

#### 使用场景

业务代码中出现踩内存、释放野指针问题，通过异常dump信息较难定位内存非法操作的位置。

#### 功能说明

备份动态内存节点控制头信息：在前一内存节点控制头中备份当前节点控制头信息。在内存申请和释放操作中增加对当前节点的控制头信息与备份信息的检测，在节点控制头被踩而备份信息未踩时，输出节点控制头备份信息及被踩节点前一内存节点信息，用于进一步分析是否为越界踩内存问题。在释放野指针时可及时输出提示信息和调用栈信息，快速定位释放野指针的位置。

#### 使用方法

打开菜单开启内存备份机制：

- 目前只有bestfit内存管理算法支持该功能，需要使能  
LOSCFG\_KERNEL\_MEM\_BESTFIT：  
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- 同时该功能依赖于LOSCFG\_MEM\_HEAD\_BACKUP，该宏开关可以通过在菜单项中开启“Enable Node Head Backup”使能：  
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Node Head Backup

#### 注意事项

该功能开启后会增加系统内存占用（占用大小=节点个数\*节点控制头大小），且影响内存操作性能。建议仅在问题检测时开启，默认关闭。

## 9.4.4 内存合法性检查

### 使用场景

业务发生踩内存导致内存节点控制头被踩，长时间后才触发业务异常，业务逻辑复杂，难以定位发生踩内存的位置。

### 功能说明

开启该功能后，在动态内存申请接口中增加内存合法性检查，对动态内存池中所有节点控制头的合法性进行检查，若已发生动态内存节点被踩，及时触发异常，输出error信息，缩小问题定位范围。

### 使用方法

1. 打开菜单开启内存合法性检查。  
功能依赖LOSCFG\_BASE\_MEM\_NODE\_INTEGRITY\_CHECK，使用时在菜单项中开启“Enable integrity check or not”：  
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable integrity check or not
2. 发生踩内存后，下一次内存申请操作即触发异常，并给出被踩节点和前一节点信息，可初步分析定位是否是前一节点越界踩内存。踩内存发生范围为上一次内存申请和此次内存申请之间。异常信息可以通过执行Shell命令[memcheck](#)查看。

### 注意事项

该功能开启时，系统内存申请操作的性能下降明显，建议仅在定位问题时开启，默认关闭。

### 踩内存问题定位实例

通过构造超出内存长度的memset操作，构造越界踩内存，造成内存节点损坏，构造代码如下：

```
VOID SampleFunc(VOID *p)
{
 memset(p, 0, 0x110); // 超出长度的memset,设置踩内存场景
}

UINT32 Test(UINT32 argc, CHAR **args)
{
 void *p1, *p2;

 p1 = LOS_MemAlloc((void*)OS_SYS_MEM_ADDR, 0x100);
 p2 = LOS_MemAlloc((void*)OS_SYS_MEM_ADDR, 0x100);
 dprintf("p1 = %p, p2 = %p \n", p1, p2);
 SampleFunc(p1); // 因为p1和p2相邻，当memset的长度超过p1的内存大小，就会越界踩到p2内存

 LOS_MemFree(OS_SYS_MEM_ADDR, (void *)p1);
 LOS_MemFree(OS_SYS_MEM_ADDR, (void *)p2);
 return 0;
}
```

执行上述代码后，执行Shell命令memcheck，其输出内容如下：

```

Huawei_LiteOS # test
p1 = 0x8034fafc, p2 = 0x8034fc0c 1
[ERR][OsMemIntegrityCheck], 1145, memory check error!
freeNodeInfo.pstPrev:(nil) is out of legal mem range[0x80304f80, 0x84000000]

broken node head: (nil) (nil) (nil) 0x0, pre node head: 0x7fcb0513 0x8 0x80349ad8 0x80000110

dump mem tmpNode:0x8034fbfc ~ 0x8034fc3c

0x8034fbfc :00000000 00000000 00000000 00000000
0x8034fc0c :cacacaca cacacaca cacacaca cacacaca
0x8034fc1c :cacacaca cacacaca cacacaca cacacaca
0x8034fc2c :cacacaca cacacaca cacacaca cacacaca

dump mem :0x8034fbbc ~ tmpNode:0x8034fbfc

0x8034fbbc :00000000 00000000 00000000 00000000
0x8034fbcc :00000000 00000000 00000000 00000000
0x8034fbdc :00000000 00000000 00000000 00000000
0x8034fbec :00000000 00000000 00000000 00000000

cur node: 0x8034fbfc 2
pre node: 0x8034faec
pre node was allocated by task:SerialShellTask 3
cpu0 is in exc.
cpu1 is running.
excType:software interrupt
taskName = SerialShellTask
taskId = 8
task stackSize = 12288
system mem addr = 0x80304f80
excBuffAddr pc = 0x8020f8b4
excBuffAddr lr = 0x8020f8b8
excBuffAddr sp = 0x8036cb88
excBuffAddr fp = 0x802539d0
R0 = 0x59
R1 = 0x600f01d3
R2 = 0x0
R3 = 0x8024dc38
R4 = 0x1
R5 = 0xa00f0113
R6 = 0x8034fc0c
R7 = 0x802cc7f4
R8 = 0x8036cc1c
R9 = 0x4
R10 = 0x80369ae0
R11 = 0x802539d0
R12 = 0x1
CPSR = 0x600f01d3

```

从上图可以看到打印了错误信息。

- 标记2所指“cur node: 0x8034fbfc”表示该节点内存被踩，“pre node: 0x8034faec”表示被踩节点前面的节点。标记3所示“pre node was allocated by task:SerialShellTask”表示在SerialShellTask任务中发生了踩内存。
- 标记1打印的是p1和p2内存的起始地址，“p2 = 0x8034fc0c”，减去控制头大小0x10，即p2-0x10=0x8034fbfc，就是cur node打印出的地址，即p2内存被踩。从代码可以看到p1和p2是两个相邻的节点（这也可以从打印的p1和p2地址看出来，即p1+p1的size+控制头大小=p2，0x8034fafc+0x100+0x10=0x8034fc0c），所以“pre node: 0x8034faec”应该就是p1的地址，从标记1获取p1地址为“p1 = 0x8034fafc”，即pre node加上控制头大小0x10（0x8034faec+0x10=0x8034fafc）。

## 9.4.5 内存 size 检查

### 使用场景

memset和memcpy操作动态内存，发生越界踩内存问题。

功能说明

对于memset和memcpy操作，当入参为动态内存节点时，增加对内存节点实际大小与入参指定大小的检查，若指定大小大于节点实际大小时，输出error信息，并且取消该次memset或memcpy操作，所以能够防止操作越界。动态内存越界场景下，可开启该功能定位问题。

| 接口名                  | 描述               |
|----------------------|------------------|
| LOS_MemCheckLevelSet | 设置内存检查级别         |
| LOS_MemCheckLevelGet | 获取内存检查级别         |
| LOS_MemNodeSizeCheck | 获取指定内存块的总大小和可用大小 |

错误码

| 序号 | 定义                             | 实际数值       | 描述                            | 参考解决方案                                                     |
|----|--------------------------------|------------|-------------------------------|------------------------------------------------------------|
| 1  | LOS_ERRNO_MEMCHECK_PARA_NULL   | 0x02000101 | LOS_MemNodeSizeCheck的入参中存在空指针 | 传入有效指针                                                     |
| 2  | LOS_ERRNO_MEMCHECK_OUTSIDE     | 0x02000102 | 内存地址不在合法范围内                   | 输入内存地址本身不在内存管理范围之内，不做处理                                    |
| 3  | LOS_ERRNO_MEMCHECK_NO_HEAD     | 0x02000103 | 内存地址已经被释放或者是野指针               | 检查传入的内存地址，保证该地址是有效地址                                       |
| 4  | LOS_ERRNO_MEMCHECK_WRONG_LEVEL | 0x02000104 | 内存检测等级不合法                     | 通过LOS_MemCheckLevelGet检查等级，并通过LOS_MemCheckLevelSet来配置合法的等级 |
| 5  | LOS_ERRNO_MEMCHECK_DISABLED    | 0x02000105 | 内存检测被关闭                       | 通过LOS_MemCheckLevelSet使能内存检测                               |

须知

错误码定义见[错误码简介](#)。8~15位的所属模块为动态内存模块，值为0x01。

使用方法

打开菜单开启内存size检查：

- 目前只有bestfit内存管理算法支持该功能，所以需要使能LOSCFG\_KERNEL\_MEM\_BESTFIT。  
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- 同时该功能依赖于LOSCFG\_BASE\_MEM\_NODE\_SIZE\_CHECK，该宏开关可以通过在菜单项中开启“Enable size check or not”使能。  
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable size check or not

## 注意事项

开启该功能后，memset和memcpy性能下降，建议仅在需要定位越界问题时开启，默认关闭。

## 内存越界问题定位实例

通过构造超出内存长度的memset和memcpy操作，构造越界踩内存问题，构造代码如下：

```
VOID test(VOID)
{
 UINT32 size = 0x100;

 VOID *poolAddr1 = LOS_MemAlloc((VOID *)OS_SYS_MEM_ADDR, size);
 if (poolAddr1 == NULL) {
 PRINTK("malloc poolAddr1 failed\n");
 return;
 } else {
 PRINTK("malloc poolAddr1 %p succeeded\n", poolAddr1);
 }

 VOID *poolAddr2 = LOS_MemAlloc((VOID *)OS_SYS_MEM_ADDR, size);
 if (poolAddr2 == NULL) {
 PRINTK("malloc poolAddr2 failed\n");
 return;
 } else {
 PRINTK("malloc poolAddr2 %p succeeded\n", poolAddr2);
 }

 LOS_MemCheckLevelSet(LOS_MEM_CHECK_LEVEL_LOW); // 开启对memset和memcpy的长度检测

 PRINTK("memset poolAddr1 overflow\n");
 memset(poolAddr1, 0x10, size * 2); // 超出长度的memset
 PRINTK("memset poolAddr1\n");
 memset(poolAddr1, 0x10, size); // 合理长度的memset

 PRINTK("memcpy poolAddr2 overflow\n");
 memcpy(poolAddr2, poolAddr1, size * 2); // 超出长度的memcpy

 PRINTK("memcpy poolAddr2\n");
 memcpy(poolAddr2, poolAddr1, size); // 合理长度的memcpy

 LOS_MemCheckLevelSet(LOS_MEM_CHECK_LEVEL_DISABLE); // 关闭对memset和memcpy的长度检测

 LOS_MemFree((VOID *)OS_SYS_MEM_ADDR, (VOID *)poolAddr1);
 LOS_MemFree((VOID *)OS_SYS_MEM_ADDR, (VOID *)poolAddr2);

 return 0;
}
```

log：

```
malloc poolAddr1 0x80349514 succeeded
malloc poolAddr2 0x80349624 succeeded
LOS_MemCheckLevelSet: LOS_MEM_CHECK_LEVEL_LOW
memset poolAddr1 overflow
[ERR] -----
```

```

memset: dst inode availSize is not enough availSize = 0x100, memcpy length = 0x200
runTask->taskName = osMain
runTask->taskId = 64
*****backtrace begin*****
traceback 0 -- lr = 0x80209798 fp = 0x802c6930
traceback 1 -- lr = 0x80210fc4 fp = 0x802c6954
traceback 2 -- lr = 0x8020194c fp = 0x802c6994
traceback 3 -- lr = 0x80201448 fp = 0x802c699c
traceback 4 -- lr = 0x802012fc fp = 0x0
[ERR] -----
memset poolAddr1
memcpy poolAddr2 overflow
[ERR] -----
memcpy: dst inode availSize is not enough availSize = 0x100, memcpy length = 0x200
runTask->taskName = osMain
runTask->taskId = 64
*****backtrace begin*****
traceback 0 -- lr = 0x80209798 fp = 0x802c6930
traceback 1 -- lr = 0x8020dbc4 fp = 0x802c6954
traceback 2 -- lr = 0x8020194c fp = 0x802c6994
traceback 3 -- lr = 0x80201448 fp = 0x802c699c
traceback 4 -- lr = 0x802012fc fp = 0x0
[ERR] -----
memcpy poolAddr2

```

由于开启size检测，非法的memset和memcpy操作被取消，输出error信息。

“runTask->taskName = osMain”显示了该非法操作发生在 osMain函数中，并打印寄存器lr和fp的值。此时可以打开编译后生成的 .asm反汇编文件，默认生成在 Huawei\_LiteOS/out/<platform>目录下，其中的platform为具体的平台名，通过对比“寄存器lr”的值，查看函数的嵌套调用。

## 9.4.6 内存泄露检测

### 使用场景

业务运行中发生内存泄露，业务逻辑复杂或者长时间运行才出现。

### 功能说明

申请内存和释放内存时，在内存节点控制头中记录函数调用栈，发生内存泄露时，通过分析used节点信息，可定位疑似内存泄露的位置。

### 使用方法

**步骤1** 打开菜单开启内存泄漏检测。

- 目前只有bestfit内存管理算法支持该功能，需要使能 LOSCFG\_KERNEL\_MEM\_BESTFIT。  
Kernel ---> Memory Management ---> Dynamic Memory Management Algorithm ---> bestfit
- 同时该功能依赖于LOSCFG\_MEM\_LEAKCHECK，可以在菜单项中选择“Enable Function call stack of Mem operation recorded”开启该宏开关：  
Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

**步骤2** 配置调用栈回溯信息菜单配置路径：

Debug ---> Enable a Debug Version ---> Enable MEM Debug ---> Enable Function call stack of Mem operation recorded

- LOSCFG\_MEM\_OMIT\_LR\_CNT：调用栈回溯忽略层级，默认配置为2。
- LOSCFG\_MEM\_RECORD\_LR\_CNT：调用栈回溯记录个数，默认配置为3。

默认配置下，获取0~2层LR信息，忽略第0和第1两层（调用封装接口的节点0层和1层LR信息相同），记录第2层。

**步骤3** 使用Shell命令 **memused** 获取used节点数据。

系统稳定运行后，若used节点个数随时间一直增加，极大可能存在内存泄露，对数据进行对比分析，重点关注LR重复的节点是否存在内存泄露，泄漏点可通过LR信息进行回溯查找。

打印log信息如下，memused命令说明详见[memused](#)：

```
Huawei LiteOS # memused
node LR[0] LR[1] LR[2]
0x802d7b34: 0x8006d86c 0x8011c604 0x8011c758
0x802dab6c: 0x8006d16c 0x8006d8a0 0x8011c604
```

----结束

## 注意事项

该功能开启时会增加系统内存占用，影响内存操作性能，建议仅在定位问题时开启，默认关闭。

内存占用size = 内存节点个数 \* ( LOSCFG\_MEM\_RECORD\_LR\_CNT -  
LOSCFG\_MEM\_OMIT\_LR\_CNT ) \* sizeof(指针)

## 9.5 MMC 维测方法

### 9.5.1 命令失败调测方法

#### 功能说明

MMC模块在开发过程中，经常遇到某个命令发送失败的场景。出现这类情况的原因大概率是发送命令前，MMC协议寄存器的配置不对，导致命令发送失败，此时我们可以通过打开相关配置，观测寄存器的值，来判断是否有异常点。

#### 使用方法

打开LOSCFG\_DRIVERS\_MMC\_CMD\_DUMP宏，并通过修改  
LOSCFG\_DRIVERS\_MMC\_OPCODE\_DUMP的值，选择想要观测的命令。

```
Debug ---> Enable a Debug Version ---> Enable MMC Debug ---> Enable mmc command dump
```

#### 注意事项

该功能开启时会增加MMC命令发送耗时，仅在某个命令发送失败后想进行原因分析的时候开启，默认关闭。

### 9.5.2 DMA 传输调测方法

#### 功能说明

MMC模块的数据收发依赖DMA传输，所以对数据buffer的地址、长度有特殊的要求，在开发过程中经常因为buffer问题而导致数据收发出现异常，此时我们可以通过打开相关配置，观测DMA descriptor寄存器的值，来判断DMA传输是否有异常。

## 使用方法

打开LOSCFG\_DRIVERS\_MMC\_DMA\_DES\_DUMP宏。

Debug ---> Enable a Debug Version ---> Enable MMC Debug ---> Enable mmc dma descriptors dump

## 注意事项

该功能开启时会增加MMC命令发送耗时，仅在某个命令发送失败后想进行原因分析的时候开启，默认关闭。

# 9.6 任务间通信调测方法

## 9.6.1 队列调测方法

### 功能说明

队列是一种生产者消费者模型，生产者生产消息放入队列，等待被消费者使用。如果队列已满，生产者被挂起，如果队列已空，消费者被挂起。Huawei LiteOS中使用队列传递消息时，可以设置超时时间，队列的主要作用是实现任务间的异步通信。通过Shell命令`queue`可以查看队列的使用情况。

### 使用方法

`queue`命令依赖LOSCFG\_DEBUG\_QUEUE，使用时需要在menuconfig中开启"Enable Queue Debugging"。

Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Queue Debugging

### 使用实例

在Shell窗口中执行命令`queue`，打印系统中的队列信息如下：

```
Huawei LiteOS # queue
used queues information:
Queue ID <0x8> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x7> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0xb> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0xa> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x9> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x2> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x4> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x3> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x6> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
Queue ID <0x5> may leak, queue len is 0x5, readable cnt:0x0, writeable cnt:0x5, TaskEntry of creator:0x0x80242df8, Latest operation time: 0x4ee
```

其输出项的含义见`queue-输出说明`，调试过程中主要使用上图中的标识项TaskEntry of creator，即创建队列的接口函数地址（0x0x80242df8）。在.asm反汇编文件（默认在Huawei\_LiteOS/out/<platform>目录下，其中的platform为具体的平台名）中找到该地址，可以看到创建队列的函数名，比如这里的app\_init（0x0x80242df8），见下图。



```

56613
56614 80242df8 <app_init>:
56615 80242df8: e3093a04 movw r3, #39428 ; 0x9a04
56616 80242dfc: e92d4070 push {r4, r5, r6, lr}
56617 80242e00: e24dd038 sub sp, sp, #56 ; 0x38
56618 80242e04: e3483020 movt r3, #32800 ; 0x8020
56619 80242e08: e5933000 ldr r3, [r3]
56620 80242e0c: e58d3034 str r3, [sp, #52] ; 0x34
56621 80242e10: e3a03000 mov r3, #0
56622 80242e14: ebff315b bl 8020f388 <uart_dev_init>
56623 80242e18: e3500000 cmp r0, #0
56624 80242e1c: 1a000059 bne 80242f88 <app_init+0x190>
56625 80242e20: e3025d78 movw r5, #11640 ; 0x2d78
56626 80242e24: ebfffcf7 bl 80242208 <spinor_init>
56627 80242e28: e3485024 movt r5, #32804 ; 0x8024
56628 80242e2c: e30504e8 movw r0, #21736 ; 0x54e8
56629 80242e30: e3480020 movt r0, #32800 ; 0x8020
56630 80242e34: e3a04001 mov r4, #1
56631 80242e38: e3a06601 mov r6, #1048576 ; 0x100000
56632 80242e3c: ebffd2cd bl 80237978 <get_mtd>
56633 80242e40: e30c3028 movw r3, #49192 ; 0xc028
56634 80242e44: e3a02028 mov r2, #40 ; 0x28
56635 80242e48: e3483039 movt r3, #32825 ; 0x8039
56636 80242e4c: e3a01000 mov r1, #0
56637 80242e50: e5830000 str r0, [r3]
56638 80242e54: e28d000c add r0, sp, #12
56639 80242e58: ebff2cfa bl 8020e248 <memset>
56640 80242e5c: e58d5018 str r5, [sp, #24]

```

## 9.6.2 死锁检测机制

### 功能说明

在许多应用中进程需要以独占的方式访问资源，当操作系统允许多个进程并发执行时可能会出现进程永远被阻塞现象，如两个进程分别等待对方所占的资源，于是两者都不能执行而处于永远等待状态，此现象称为死锁。死锁检测模块支持自旋锁、互斥锁、二值信号量和线程锁(pthread mutex)四种类型锁的死锁、重复上锁，错误释放和锁的深度溢出的检测。

### 使用方法

打开菜单，进入Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource ---> Enable Lockdep Debugging菜单，完成死锁检测模块的配置。

| 配置项                    | 含义           | 取值范围   | 默认值 | 依赖                      |
|------------------------|--------------|--------|-----|-------------------------|
| LOSCFG_KERNEL_LOCKDEP  | 使能死锁检测功能。    | YES/NO | NO  | LOSCFG_KERNEL_MEM_ALLOC |
| LOSCFG_KERNEL_SPINDEP  | 使能自旋锁死锁检测。   | YES/NO | NO  | LOSCFG_KERNEL_SMP       |
| LOSCFG_KERNEL_MUTEXDEP | 使能互斥锁死锁检测。   | YES/NO | NO  | LOSCFG_BASE_IPC_MU X    |
| LOSCFG_KERNEL_SEMDEP   | 使能二值信号量死锁检测。 | YES/NO | NO  | LOSCFG_BASE_IPC_SE M    |

| 配置项                   | 含义         | 取值范围   | 默认值 | 依赖                   |
|-----------------------|------------|--------|-----|----------------------|
| LOSCFG_PTHREAD_MUXDEP | 使能线程锁死锁检测。 | YES/NO | NO  | LOSCFG_COMPAT_PO SIX |

自旋锁定位实例

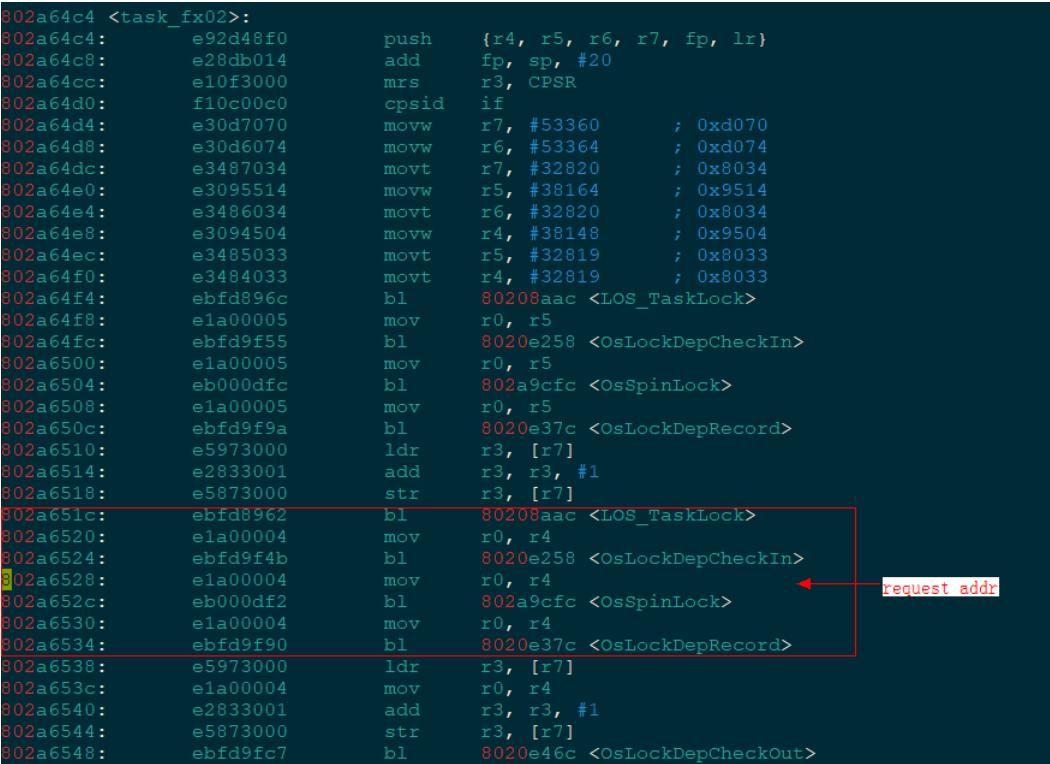
步骤1 打开自旋锁检测后，检测到死锁时会打印死锁信息，死锁检测的打印信息示例如下。

图 9-24 死锁模块打印信息

```
ERR!lockdep check failed
ERR!error type: dead lock
ERR!request addr: 0x802a6528
ERR!task name: it_ump_lockdep_004_2
ERR!task id: 7
ERR!cpu num: 1
ERR!start dumping lockdep information
ERR!() # testSpin2
ERR!() # testSpin2 <- now
ERR!task name: it_ump_lockdep_004_1
ERR!task id: 6
ERR!cpu num: 0
ERR!start dumping lockdep information
ERR!() # testSpin2
ERR!() # testSpin2 <- now
OSBackTrace Ep = 0x8034640e
main task=>taskName = it_ump_lockdep_004_2
main task=>taskID = 7
*****backtrace begin*****
backtrace 0 - lr = 0x8020a34c fp = 0x80346414
backtrace 1 - lr = 0x8020a34c fp = 0x80346414
backtrace 2 - lr = 0x802a6528 fp = 0x8034645c
backtrace 3 - lr = 0x8020a330 fp = 0x80346464
backtrace 4 - lr = 0x802a69cc fp = 0x804060b5
Name TaskEntryAddr TID Affi CPU Priority Status StackSize WaterLine StackPoint TopOfStack EventMask SemID CPURSE CPURSE10s CPURSE11s MEMUSE
IdleCore000 0x802085b4 0x0 0x0002 -1 31 Ready 0x800 0x238 0x80346518 0x80346070 0x0 0xffffffff 0.0 0.0 0.0 0
SetTask 0x80207c8c 0x1 0x0001 -1 0 Pend 0x6000 0x208 0x80397440 0x80391648 0x0 0xffffffff 0.0 0.0 0.0 0
IdleCore000 0x802085b4 0x2 0x0001 -1 31 Ready 0x800 0x148 0x80397480 0x80397738 0x0 0xffffffff 0.0 0.0 0.0 0
DevFreeArg 0x8023014c 0x3 0x0003 -1 1 Pend 0x6000 0x22c 0x80394480 0x80397f38 0x1 0xffffffff 0.0 0.0 0.0 0
M_TST_INIT 0x802a5450 0x4 0x0001 -1 25 Delay 0x30000 0x244 0x80344888 0x80394f48 0x0 0xffffffff 0.0 0.0 0.0 3112
SetTask 0x80207c8c 0x5 0x0002 -1 0 Pend 0x6000 0x208 0x80345408 0x80340010 0x0 0xffffffff 0.0 0.0 0.0 0
it_ump_lockdep_004_10x802a6528 0x6 0x0001 0 26 Running 0x600 0x148 0x80345088 0x80345428 0x0 0xffffffff 0.0 0.0 0.0 0
it_ump_lockdep_004_20x802a6404 0x7 0x0002 1 26 Running 0x600 0x33c 0x80346440 0x80346888 0x0 0xffffffff 0.0 0.0 0.0 0
```

步骤2 复制request addr的值（本例中为0x802a6528），在系统镜像的.asm反汇编文件（make编译默认在Huawei\_LiteOS/out/<platform>目录下，CMake编译默认在Huawei\_LiteOS/buildcmakeout/<platform>目录下，其中的platform为具体的平台名）中找到相应的地址，如下图所示，即可定位到调用spinlock的位置及调用函数（本例中为task\_fx02）。

图 9-25 反汇编文件中找到对应地址



**步骤3** 根据图一死锁打印信息中第二个蓝框的自旋锁，通过代码逻辑找到另一个任务持有该锁的情况，再结合代码，调整spinlock调用的时序，从而解决死锁问题。

----结束

须知

自旋锁死锁一般发生在某个CPU卡住、任务不再发生调度时。若不开启自旋锁死锁检查，可以使用JLink等调试工具halt住CPU或者触发看门狗异常，查看PC值是否为自旋锁代码，以此确定是否发生了自旋锁死锁。

互斥锁死锁定位实例

构造ABBA互斥锁死锁场景，具体如下：

有两个任务，分别为Task\_A和Task\_B，同时系统中还存在其他系统默认初始任务。在任务Task\_A中依次申请Mutex1和Mutex0，Task\_B任务依次申请Mutex1和Mutex0，构成两个任务ABBA死锁场景。

代码如下：

```
#include "los_mux.h"

static UINT32 mutexTest[2];
UINT32 g_TestTaskID01, g_TestTaskID02;

static VOID Task_A(VOID)
{
 UINT32 ret;
 ret = LOS_MuxPend(mutexTest[0], LOS_WAIT_FOREVER);
}
```

```

 if (ret == LOS_OK) {
 dprintf("%s pend muxid %d\n", __func__, mutexTest[0]);
 }

 LOS_TaskDelay(0x2);

 ret = LOS_MuxPend(mutexTest[1], 6000); // 定时阻塞模式 6000tick
 if (ret == LOS_ERRNO_MUX_TIMEOUT) {
 dprintf("%s pend muxid %d\n", __func__, mutexTest[1]);
 }

 return;
}

static VOID Task_B(VOID)
{
 UINT32 ret;
 ret = LOS_MuxPend(mutexTest[1], LOS_WAIT_FOREVER);
 if (ret == LOS_OK) {
 dprintf("%s pend muxid %d\n", __func__, mutexTest[1]);
 }
 LOS_TaskDelay(0x2);
 ret = LOS_MuxPend(mutexTest[0], 6000); // 定时阻塞模式 6000tick
 if (ret == LOS_ERRNO_MUX_TIMEOUT) {
 dprintf("%s pend muxid %d\n", __func__, mutexTest[0]);
 }
 return;
}

UINT32 mutexDeadlock(VOID)
{
 UINT32 ret;
 TSK_INIT_PARAM_S stTask1 = {0};
 stTask1.pfnTaskEntry = (TSK_ENTRY_FUNC)Task_A;
 stTask1.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
 stTask1.pcName = "Task_A";
 stTask1.usTaskPrio = 9;

 TSK_INIT_PARAM_S stTask2 = {0};
 stTask2.pfnTaskEntry = (TSK_ENTRY_FUNC)Task_B;
 stTask2.uwStackSize = LOSCFG_BASE_CORE_TSK_MIN_STACK_SIZE;
 stTask2.pcName = "Task_B";
 stTask2.usTaskPrio = 9;

 ret = LOS_MuxCreate(&mutexTest[0]);
 ret |= LOS_MuxCreate(&mutexTest[1]);
 if (ret != LOS_OK) {
 PRINT_ERR("create mutex error %u\n", ret);
 }

 ret = LOS_TaskCreate(&g_TestTaskID01, &stTask1);
 ret |= LOS_TaskCreate(&g_TestTaskID02, &stTask2);
 if (ret != LOS_OK) {
 PRINT_ERR("create task error %u\n", ret);
 }

 LOS_TaskDelay(3000); // 任务休眠3000Ticks

 ret = LOS_MuxDelete(mutexTest[0]);
 ret = LOS_MuxDelete(mutexTest[1]);

 LOS_TaskDelete(g_TestTaskID01);
 LOS_TaskDelete(g_TestTaskID02);
 return LOS_OK;
}

```

针对以上场景出现的问题，会dump以下信息：

```
Task_A pend muxid 3
Task_B pend muxid 4
lockdep check failed
error type : dead lock
request addr : 0x4200a08c
task name : Task_B
task id : 11
cpu num : 3
start dumping lockdep information
[0] Mutex ID: 0004
[1] Mutex ID: 0003 <-- now
task name : Task_A
task id : 10
cpu num : 0
start dumping lockdep information
[0] Mutex ID: 0003
[1] Mutex ID: 0004 <-- now
[ERR] the current exception level is EL1
/*****Exception Information*****/
ExcClass:0x25 => Data Abort.
Abort caused by a read instruction.
OMG!UNKNOWN fault, check ISS encoding for an exception from a Data Abort in AARM for armv8-a.
PC at 0x420083b4, FAR 0x0, ISS 0x6
ESR 0x96000006: ec 0x25, il 0x2000000, iss 0x6
taskName = Task_B
TaskId = 11
Task StackSize = 2048
system mem addr = 0x422938c0
PC = 0x420083b4
```

| Name        | TaskEntryAddr      | TID | Affi   | CPU | Priority | Status   | StackSize | WaterLine | StackPoint         | TopOfStack         | SemID      | EventMask | MEMUSE |
|-------------|--------------------|-----|--------|-----|----------|----------|-----------|-----------|--------------------|--------------------|------------|-----------|--------|
| Swt_Task    | 0x000000042000e80  | 0x0 | 0x0001 | 0   | 0        | Suspend  | 0x6000    | 0xc3f8    | 0x0000000042284530 | 0x000000004227e920 | 0xffffffff | 0x0       | 0      |
| IdleCore000 | 0x000000004200aa40 | 0x1 | 0x0001 | 0   | 31       | Running  | 0x900     | 0xc378    | 0x0000000042284c30 | 0x0000000042284950 | 0xffffffff | 0x0       | 0      |
| system_wq   | 0x00000000420114e0 | 0x2 | 0x000f | 0   | 1        | Pend     | 0x6000    | 0xc3e8    | 0x000000004228ae00 | 0x0000000042285290 | 0xffffffff | 0x0       | 0      |
| app_Task    | 0x000000004200cf20 | 0x3 | 0x0001 | 0   | 10       | Delay    | 0x6000    | 0xc638    | 0x000000004229ae00 | 0x000000004228b7c0 | 0xffffffff | 0x0       | 4288   |
| Swt_Task    | 0x000000004200e80  | 0x4 | 0x0008 | 3   | 0        | Suspend  | 0x6000    | 0xc3f8    | 0x000000004229e000 | 0x00000000422912f0 | 0xffffffff | 0x0       | 0      |
| Swt_Task    | 0x000000004200e80  | 0x5 | 0x0004 | 2   | 0        | Suspend  | 0x6000    | 0xc3f8    | 0x000000004229cf30 | 0x0000000042297320 | 0xffffffff | 0x0       | 0      |
| IdleCore000 | 0x000000004200aa40 | 0x6 | 0x0003 | 3   | 31       | Running  | 0x900     | 0xc378    | 0x000000004229d330 | 0x000000004229d350 | 0xffffffff | 0x0       | 0      |
| Swt_Task    | 0x000000004200e80  | 0x7 | 0x0002 | 1   | 0        | Suspend  | 0x6000    | 0xc3f8    | 0x00000000422a3790 | 0x000000004229db00 | 0xffffffff | 0x0       | 0      |
| IdleCore000 | 0x000000004200aa40 | 0x8 | 0x0004 | 2   | 31       | Ready    | 0x900     | 0xc668    | 0x00000000422a3d50 | 0x00000000422a3b00 | 0xffffffff | 0x0       | 0      |
| IdleCore000 | 0x000000004200aa40 | 0x9 | 0x0002 | 1   | 31       | Running  | 0x900     | 0xc668    | 0x00000000422a48c0 | 0x00000000422a43e0 | 0xffffffff | 0x0       | 0      |
| Task_A      | 0x0000000042006fd0 | 0xa | 0x000f | 2   | 9        | PendTime | 0x900     | 0xc6d8    | 0x00000000422a5080 | 0x00000000422a4c70 | 0xffffffff | 0x0       | 0      |
| Task_B      | 0x0000000042006f30 | 0xb | 0x000f | 2   | 9        | Running  | 0x900     | 0xc7f8    | 0x00000000422a58f0 | 0x00000000422a54a0 | 0xffffffff | 0x0       | 0      |

具体操作步骤如下：

- 步骤1
- 根据dump信息，可以看出任务Task\_A(0x42006fd0)与Task\_B(0x42006f30)发生死锁。
- 步骤2
- 在反汇编文件中，找到相应的任务函数。

以Task\_A(0x42006fd0)为例如下图所示，定位到互斥锁的pend位置及调用的接口。

```
0000000042006fd0 <Task_A>:
42006fd0: a9be7bfd stp x29, x30, [sp, #-32]!
42006fd4: 12800001 mov w1, #0xffffffff // #-1
42006fd8: 910003fd mov x29, sp
42006fdc: f9000bf3 str x19, [sp, #16]
42006fe0: f0001313 adrp x19, 42269000 <g_taskCBArray+0xb4a0>
42006fe4: b9444a60 ldr w0, [x19, #1006]
42006fe8: 940004de bl 42008360 <LOS_MuxPend>
42006fec: 340001e0 cbz w0, 42007028 <Task_A+0x58>
42006ff0: 91112273 add x19, x19, #0x448
42006ff4: 52800040 mov w0, #0x2 // #2
42006ff8: 94000df2 bl 4200a7c0 <LOS_TaskDelay>
42006ffc: 5282ee01 mov w1, #0x1770 // #6000
42007000: b9400660 ldr w0, [x19, #4]
42007004: 940004d7 bl 42008360 <LOS_MuxPend>
42007008: 5283a0e1 mov w1, #0x1d07 // #7431
4200700c: 72a04001 movk w1, #0x200, lsl #16
42007010: 6b01001f cmp w0, w1
42007014: 540001a0 b.eq 42007048 <Task_A+0x78> // b.none
42007018: f9400bf3 ldr x19, [sp, #16]
4200701c: a8c27bfd ldp x29, x30, [sp], #32
42007020: d65f03c0 ret
```

步骤3 根据上下文确认死锁原因，并调整申请锁的时序。  
----结束

完整实例代码

sample\_mux\_dlock.zip

9.7 异常交互

使用场景

操作系统对运行期间发生异常时，用户可以使用Shell命令进一步定位异常原因，查看系统状态。

功能说明

系统发生异常时，保留Shell基础交互功能。

配置项

| 配置项                        | 含义                  | 取值范围   | 默认值 | 依赖                                       |
|----------------------------|---------------------|--------|-----|------------------------------------------|
| LOSCFG_EXC_I<br>NTERACTION | 异常交互功<br>能的裁剪开<br>关 | YES/NO | NO  | LOSCFG_SHELL &&<br>LOSCFG_KERNEL_CONSOLE |

使用方法

步骤1 打开菜单开启异常交互功能。

该功能需要配置LOSCFG\_EXC\_INTERACTION=y，默认处于关闭状态。若关闭该选项，则异常交互失效。

开启该宏开关的菜单项为：

Debug ---> Enable a Debug Version ---> Enable Shell ---> Enable exc interaction

步骤2 输入“help”，查看当前支持的Shell命令。

----结束

注意事项

- 系统异常时，异常交互功能只会使能串口中断、其他中断都会被关闭；用户创建的任务也会被删除，只保留异常交互所使用的基本任务。
- 异常交互属于调测功能，默认配置为关闭，正式商用产品中禁止使用该功能。
- 免责声明：华为不承担在正式商用产品中使用该功能的任何风险。

9.8 低功耗调测方法

9.8.1 低功耗调测使能

Huawei LiteOS提供低功耗维测功能，在菜单项中使能Debug ---> Enable a Debug Version ---> Enable Debug LiteOS Kernel Resource--->Enable Lowpower Sleep Debugging，可以开启低功耗维测。使能低功耗维测先使能低功耗框架需要先打开menuconfig菜单使能Shell，详见[配置项](#)；同时启用低功耗框架，详见4.3。该功能仅供用户开发阶段使用，正式版本商用前请关闭此功能。

9.8.2 系统状态信息查询

功能说明

系统状态信息查询命令会打印出中断、定时器和任务的详细信息。阻止任务深睡事件查询、系统休眠唤醒打点命令配合该命令使用，方便用户定位。系统状态信息查询命令效果可参考系统命令hwi、swtmr、task，详见[系统命令参考](#)。

使用方法

lpm sysinfo

| Huawei LiteOS # lpm sysinfo |               |               |            |            |             |         |           |           |            |            |            |           |        |
|-----------------------------|---------------|---------------|------------|------------|-------------|---------|-----------|-----------|------------|------------|------------|-----------|--------|
| InterruptNo                 | Share         | ResponseCount | Name       | DevId      |             |         |           |           |            |            |            |           |        |
| 0                           | N             | 0             |            |            |             |         |           |           |            |            |            |           |        |
| 1                           | N             | 44            |            |            |             |         |           |           |            |            |            |           |        |
| 2                           | N             | 0             |            |            |             |         |           |           |            |            |            |           |        |
| 29                          | N             | 26            |            |            |             |         |           |           |            |            |            |           |        |
| 36                          | N             | 0             |            |            |             |         |           |           |            |            |            |           |        |
| 38                          | N             | 16            | uart_pi011 | 0x802b62fc |             |         |           |           |            |            |            |           |        |
| SwTmrID                     | State         | Mode          | Interval   | Arg        | handlerAddr |         |           |           |            |            |            |           |        |
| 0x00000000                  | Ticking       | Period        | 100        | 0x00000000 | 0x80204688  |         |           |           |            |            |            |           |        |
| Name                        | TaskEntryAddr | TID           | Affi       | CPU        | Priority    | Status  | StackSize | WaterLine | StackPoint | TopOfStack | SemID      | EventMask | MEMUSE |
| SwT_Task                    | 0x802062fc    | 0x0           | 0x0001     | -1         | 0           | Suspend | 0x6000    | 0x1c4     | 0x80332a38 | 0x8032cbf8 | 0xffffffff | 0x0       | 0      |
| IdleCore000                 | 0x80207064    | 0x1           | 0x0001     | -1         | 31          | Ready   | 0x800     | 0x324     | 0x803330f0 | 0x80332c10 | 0xffffffff | 0x0       | 0      |
| system_wq                   | 0x8022066c    | 0x2           | 0x0003     | -1         | 1           | Pend    | 0x6000    | 0x1bc     | 0x80339340 | 0x803334f8 | 0xffffffff | 0x0       | 0      |
| SwT_Task                    | 0x802062fc    | 0x4           | 0x0002     | -1         | 0           | Suspend | 0x6000    | 0x1c4     | 0x80345360 | 0x8033f520 | 0xffffffff | 0x0       | 0      |
| IdleCore000                 | 0x80207064    | 0x5           | 0x0002     | 1          | 31          | Running | 0x800     | 0x248     | 0x80345bf0 | 0x80345538 | 0xffffffff | 0x0       | 0      |
| SendToSer                   | 0x8020e3b0    | 0x6           | 0x0003     | -1         | 10          | Pend    | 0x6000    | 0x1e4     | 0x803522a0 | 0x8034c480 | 0xffffffff | 0x6       | 0      |
| SerialShellTask             | 0x802689c0    | 0x7           | 0x0003     | 0          | 9           | Running | 0x3000    | 0x644     | 0x80355790 | 0x80352980 | 0xffffffff | 0xfff     | 80     |
| SerialEntryTask             | 0x802682f0    | 0x8           | 0x0003     | -1         | 9           | Pend    | 0x1000    | 0x4b8     | 0x80356728 | 0x80355998 | 0xffffffff | 0x1       | 72     |

## 注意事项

打印中断与定时器信息，应先支持系统命令hwi、swtmr。详见[系统命令参考](#)。

## 9.8.3 低功耗动态开关

### 功能说明

低功耗动态开关支持手动打开、关闭、状态查询功能，默认处于打开状态。

### 使用方法

1. 打开低功耗动态开关：lpm enable。
2. 关闭低功耗动态开关：lpm disable。
3. 查询低功耗动态开关的状态：lpm status。

### 注意事项

无。

## 9.8.4 阻止系统深睡事件查询

### 功能说明

阻止系统深睡事件是指用户钩子getSleepMode返回深睡模式，但是系统经判断无法进入深睡的事件。阻止系统深睡的原因可能是距离定时器或任务到期剩余的时间达不到深睡要求的最低时间阈值、任务投票阻止等。

### 使用方法

1. 开机以来阻止系统深睡事件查询命令：lpm blockinfo cpuid。cpuid指的是具体的cpu号。
2. 阻止系统深睡事件统计信息清空命令：lpm resetblockinfo cpuid。

示例：

从下图中可以得到以下信息：

1. 0核task id为3的任务阻止系统深睡2次，任务地址为0x8008ced。
2. 0核swtmr id为2的定时任务阻止系统深睡1次，定时器任务地址为0x800fc91。
3. ( Normal, 3 )、( Light, 3 )、( Deep, 3 ) 三个投票各阻碍一次深睡。



```

Huawei LiteOS # lpm blockinfo 0
deep sleep block info:

cpuId type id blockCount handler

0 task 3 2 0x8008ced
0 swtmr 2 1 0x800fc91
user veto block deep sleep info:

sleepMode requestId blockCount

Normal 3 1
Light 3 1
Deep 3 1

```

## 注意事项

1. 当前只支持单核，因此cpuid当前只能为0，即lpm blockinfo 0或者lpm resetblockinfo 0。
2. 统计信息根据定时器id或者任务id记录，任务或定时器销毁与创建可能存在id相同的情况，会被统计到一起，用户应避免此场景。如：某a时刻任务A id为8，执行完后销毁，阻止深睡3次；经过一段时间，任务B被创建，id同样为8，阻止深睡2次；那么统计信息中type为task，id为8的阻碍次数为3次，handler为任务B的handler。
3. 被阻碍一次深睡会统计此刻所有的不满足深睡条件的因素。

## 9.8.5 系统休眠唤醒打点

### 功能说明

系统休眠唤醒打点功能会对Normal、Light、Deep模式下的睡眠时戳信息进行统计。单项信息依次为cpu号、睡眠模式、睡眠开始时刻、睡眠结束时刻，实际睡眠时间，被唤醒的中断。默认记录条数为10条，深睡时唤醒中断为固定值4294967295。

### 使用方法

1. 系统休眠唤醒打点查询命令：lpm timestamp cpuid。cpuid指的是具体的cpu号。
2. 修改最大记录条数可通过配置LOSCFG\_LOWPOWER\_RECORD\_COUNT实现，最大值为1000。

示例：

| Huawei LiteOS # lpm timestamp 0 |           |            |          |            |             |
|---------------------------------|-----------|------------|----------|------------|-------------|
| cpuId                           | sleepMode | startTicks | endTicks | sleepTicks | InterruptNo |
| 0                               | Light     | 5300       | 5399     | 99         | 29          |
| 0                               | Light     | 5400       | 5499     | 99         | 29          |
| 0                               | Light     | 5500       | 5599     | 99         | 29          |
| 0                               | Light     | 5600       | 5699     | 99         | 29          |
| 0                               | Light     | 5700       | 5781     | 81         | 38          |
| 0                               | Normal    | 5788       | 5799     | 11         | 29          |
| 0                               | Light     | 5800       | 5875     | 75         | 38          |
| 0                               | Normal    | 5882       | 5899     | 17         | 29          |
| 0                               | Light     | 5900       | 5938     | 38         | 38          |
| 0                               | Light     | 5945       | 5996     | 51         | 38          |

注意事项

- 1. 当前只支持单核，因此cpuid当前只能为0，即lpm timestamp 0。
- 2. 系统休眠唤醒打点统计的是实际睡眠时戳信息。如上图中用户配置浅睡时间为100，实际浅睡时间为99。

9.9 临终遗言使用方法

功能说明

临终遗言日志存储钩子函数，实现日志存储的读写函数的注册，接口详细信息可以查看API参考。

| 接口名                | 描述                                      |
|--------------------|-----------------------------------------|
| LOS_ExcInfoRegHook | 注册读写临终遗言日志的钩子函数，并设置记录临终遗言日志的起始地址、大小和缓冲区 |

说明

- LOS\_ExcInfoRegHook有4个参数：
- startAddr：临终遗言日志的存储起始地址。
  - space：临终遗言日志的存储空间大小。
  - buf：临终遗言日志在内存中的buffer地址，大小space。
  - LogReadWriteFunc：读写异常信息的钩子函数。该函数的入参包括startAddr、space、rwFlag和buf。rwFlag是读写标记，0为写，1为读。其余3个入参同LOS\_ExcInfoRegHook。

使用方法

- 步骤1** 打开菜单开启临终遗言记录功能，即配置LOSCFG\_SHELL\_EXCINFO\_DUMP=y，该功能默认关闭，菜单路径为：Kernel ---> Exception Management ---> Enable Shell excInfo。
- 步骤2** 注册读写钩子函数。
1. 编写读写临终遗言日志的钩子函数，示例代码如下。

```
#include "los_base.h"
#if defined(LOSCFG_SHELL_EXCINFO) && defined(LOSCFG_DRIVERS_MTD_SPI_NOR)
```

```

#include "linux/mtd/mtd.h"
#include "linux/module.h"
#include "linux/mtd/mtd_list.h"
#include "spinor.h"
#endif
#include "los_hwi.h"
#ifdef LOSCFG_FS_VFS
#include "fs/fs.h"
#endif

#ifdef LOSCFG_SHELL_EXCINFO_DUMP
STATIC struct mtd_info *g_mtdSpinor = NULL;
STATIC VOID OsSpiFlashErase(UINT32 start, size_t size)
{
 struct erase_info eraseInfo;
 (VOID)memset_s(&eraseInfo, sizeof(struct erase_info), 0, sizeof(struct erase_info));
 eraseInfo.mtd = g_mtdSpinor;
 eraseInfo.callback = NULL;
 eraseInfo.fail_addr = (UINT64)MTD_FAIL_ADDR_UNKNOWN;
 eraseInfo.addr = start;
 eraseInfo.len = size;
 eraseInfo.time = 1;
 eraseInfo.retries = 1;
 eraseInfo.dev = 0;
 eraseInfo.cell = 0;
 eraseInfo.priv = 0;
 eraseInfo.state = 0;
 eraseInfo.next = NULL;
 eraseInfo.scrub = 0;
 (VOID)g_mtdSpinor->erase(g_mtdSpinor, &eraseInfo);
}
STATIC INT32 OsWriteExclInfoToSpiFlash(UINT32 startAddr, UINT32 space, const CHAR *buf)
{
 UINT32 outLen;
 OsSpiFlashErase(startAddr, LOS_Align(space, g_mtdSpinor->erasesize));
 return g_mtdSpinor->write(g_mtdSpinor, (loff_t)startAddr, space, &outLen, buf);
}
STATIC INT32 OsReadExclInfoForSpiFlash(UINT32 startAddr, UINT32 space, CHAR *buf)
{
 UINT32 outLen;
 return g_mtdSpinor->read(g_mtdSpinor, (loff_t)startAddr, space, &outLen, buf);
}

/* Be called in the exception. */
VOID OsReadWriteExceptionInfo(UINT32 startAddr, UINT32 space, UINT32 flag, CHAR *buf)
{
 if ((buf == NULL) || (space == 0)) {
 PRINT_ERR("buffer is null or space is zero\n");
 return;
 }

 g_mtdSpinor = get_mtd("spinor");
 if (g_mtdSpinor == NULL) {
 (VOID)spinor_init();
 g_mtdSpinor = get_mtd("spinor");
 if (g_mtdSpinor == NULL) {
 PRINT_ERR("Init spinor is failed\n");
 return;
 }
 }

 if (flag == 0) {
 if (OsWriteExclInfoToSpiFlash(startAddr, space, buf) != LOS_OK) {
 PRINT_ERR("Exception information written to 0x%x flash failed\n", startAddr);
 }
 }
}
#endif
#ifdef LOSCFG_EXC_INTERACTION
/*
 * When system is in the exception interaction, this buf was free,
 * but this feature is still running. This buffer may be used again
 */

```

```
 * without malloc.
 * So, consider whether or not the "buf" is released according to actual use.
 */
 free(buf);
#endif
} elseif (flag == 1) {
 if (OsReadExclInfoForSpiFlash(startAddr, space, buf) != LOS_OK) {
 PRINT_ERR("Exception information read from 0x%x flash failed\n", startAddr);
 }
} else {
 PRINT_ERR("flag is error\n");
}
}
#endif
#endif
```

2. 在初始化函数如app\_init中注册钩子函数，示例代码如下。

```
#ifdef LOSCFG_SHELL_EXCINFO_DUMP
#define EXCINFO_RECORD_BUF_SIZE (16 * 1024) /* 用户自己定义 */
#define EXCINFO_RECORD_ADDR (0xffffffff) /* 此处是非法值，需要用户自己定义 */
CHAR *buf = (CHAR *)malloc(EXCINFO_RECORD_BUF_SIZE);
if (buf != NULL) {
 (VOID)memset_s(buf, EXCINFO_RECORD_BUF_SIZE, 0, EXCINFO_RECORD_BUF_SIZE);
 LOS_ExcInfoRegHook(EXCINFO_RECORD_ADDR, EXCINFO_RECORD_BUF_SIZE, buf,
 OsReadWriteExceptionInfo);
} else {
 PRINTK("shell excinfo malloc failed!\n");
}
#endif
#ifdef LOSCFG_FS_VFS
 los_vfs_init();
#endif
#endif
```

#### 须知

- OsReadWriteExceptionInfo函数是对Nor Flash进行读写的功能函数，宏EXCINFO\_RECORD\_BUF\_SIZE, EXCINFO\_RECORD\_ADDR需要用户根据实际情况进行配置，建议该段代码放在用户业务代码前。
- Nor Flash写数据之前要进行块擦除，块大小为64k，因此设置的临终遗言存储起始地址之后的64k区域，在写临终遗言信息时会被擦除，建议此区域不保存数据。

#### 步骤3 查看异常信息。

系统复位重启后，Shell窗口中执行exclInfo命令，会打印出已记录的临终遗言日志。

----结束

## 注意事项

保存临终遗言信息的buffer需要用户进行维护，重复调用注册函数时，需要用户自行释放前一次注册时传入的buffer。

## 9.10 魔法键使用方法

### 使用场景

系统未输出挂死相关信息，但是无响应时，可以通过魔法键查看中断是否有响应。在中断有响应的情况下，可以通过魔法键查看task信息中的CPUP（CPU占用率），找到

是哪个任务长时间占用CPU导致系统其他任务无响应（一般为比较高优先级任务一直抢占CPU，导致低优先级任务无响应）。

## 功能说明

在uart中断和usb转虚拟串口中断中，嵌入魔法键检查功能，对特殊按键进行识别，输出相关信息。

## 使用方法

### 步骤1 打开菜单开启魔法键功能。

该功能需要配置LOSCFG\_ENABLE\_MAGICKEY=y，默认处于关闭状态，若关闭该选项，则魔法键失效，开启该宏开关的菜单项为：

Debug ---> Enable MAGIC KEY

#### 说明

可以在menuconfig中光标移动到该选项上，输入“？”，查看帮助信息：

```
Answer Y to enable LiteOS Magic key.
ctrl + r : Magic key check switch;
ctrl + z : Show all magic op key;
ctrl + t : Show task information;
ctrl + p : System panic;
ctrl + e : Check system memory pool.
```

### 步骤2 输入“ctrl + r”键，打开或者关闭魔法键检测功能。

在连接uart或者usb转虚拟串口的情况下，输入“ctrl + r”键，打开魔法键检测功能，输出“Magic key on”；再输入一次后，则关闭魔法键检测功能，输出“Magic key off”。魔法键功能如下：

- ctrl + z：帮助键，输出相关魔法键简单介绍。
- ctrl + t：输出任务相关信息。
- ctrl + p：系统主动进入panic，输出panic相关信息后，系统会挂住。
- ctrl + e：系统进行简单的内存池完整性检查，检查发现错误则输出相关错误信息，检查正常则输出“system memcheck over, all passed!”。

----结束

## 注意事项

魔法键检测功能打开情况下，如果需要通过uart或者usb转虚拟串口输入特殊字符需避免与魔法键值重复，否则魔法键会被误触发，而原有设计功能可能出现错误。

## 9.11 栈估算工具使用方法

### 使用场景

对于内存资源受限的嵌入式设备，可以使用栈估算工具分析任务栈，达到精确使用栈空间的目的。

- 创建任务为任务设置任务栈大小时，可以使用栈估算工具分析任务栈，达到精确设置任务栈大小的目的。

- 系统划分内存设置中断栈大小时，可以使用栈估算工具分析中断栈，达到精确设置中断栈大小的目的。
- 可以使用栈估算工具分析任务栈溢出问题。

## 功能说明

栈估算工具是基于python 语言开发的函数栈静态分析工具，是一款离线工具。通过解析反汇编文件，计算得到任务运行中栈空间使用的最大值，为设置任务栈大小、栈溢出分析、栈内存优化等场景，提供静态分析的数据参考。同时栈估算工具还提供递归函数告警和函数调用关系树展示功能。

## 工具运作机制

栈估算工具逐行解析反汇编文件，将跳转指令转化为函数调用关系，开栈指令解析为函数栈局部空间开销，最终使用深度优先搜索，遍历任务入口函数及其调用的所有子函数，得到任务栈内存使用的峰值。

## 使用方法

栈估算工具名为stackusage.py，使用工具前，可以通过python3 stackusage.py -h命令查看帮助信息。栈估算工具支持python3.6及以上版本。如果没有安装python3.6及以上版本，可以参见官网[python安装文档](#)完成安装。

```
00451438@ubuntu:~/liteOS_test/smp_mainline/Huawei_LiteOS/tools/stackusage$ python3.8 stackusage.py -h
usage: stackusage.py [-h] [-m N] [-sp] [-t taskEntryName [taskEntryName ...]] [-out] [-j] [-r N] HuaweiLiteOS.asm

StackUsage tools. Estimate task stack size by disassembly file. Already support Arm and Risc-V platform

positional arguments:
 HuaweiLiteOS.asm Disassembly file path

optional arguments:
 -h, --help show this help message and exit
 -m N, --analyze_mode N
 analyze_mode. 0:Only analyze assembly code; 1:Analyze source code line number. 2:Analyze source code in disassembly file.
 -sp, --specify Analyze specified task entry. (default: analyze all function)
 -t taskEntryName [taskEntryName ...], --taskname taskEntryName [taskEntryName ...]
 Specified task entry name. Can be omitted if analyze all.
 -out, --outputconsole
 Output result to console.
 -j, --outputjson Output result to json file.
 -r N, --rec-stats N Recursive function loop cycles.
```

### 步骤1 生成反汇编文件（可选）。

如果没有反汇编文件，可以通过elf文件生成，例如有Huawei\_LiteOS.elf文件，可以参考如下命令生成名为Huawei\_LiteOS.asm的反汇编文件：

```
arm-none-eabi-objdump -d Huawei_LiteOS.elf > Huawei_LiteOS.asm
```

### 步骤2 修改配置文件（可选）。

统计错误分支中的异常处理方法的栈开销，有时是无意义的。因此栈估算工具支持可选地忽略部分断言函数、异常分支处理函数、打印函数等，在统计过程中会将他们的栈开销设置为0。

在修改cfg/config.ini文件时，将统计时需要忽略的函数，添加至Ignore\_function行末尾，如有多个函数可以以空格隔开。工具执行时将自动解析配置文件。

```
Ignore some assertions, exception methods, or printing methods. Set their stack cost to 0.
Ignore_function: __stack_chk_fail LOS_Panic LOS_BackTrace PrintExcInfo
Ignore_function: dprintf OsVprintf
```

### 步骤3 运行栈估算工具。

例如反汇编文件为Huawei\_LiteOS.asm，想要使用工具估算OsIdleTask任务的任务栈大小，可以参考如下命令运行工具：

```
python3 stackusage.py -sp -t OsIdleTask -out Huawei_LiteOS.asm
```

执行上面的命令后，其输出如下图所示：

```
Read manual funcPtr configuration file! ./cfg/funcptr.txt 函数调用关系配置文件
[Recursive function]: 递归函数告警
<LOS_TaskUnlock>:
<LOS_Schedule>:
<OsSchedPreempt>:
<OsSchedResched>:
<OsTaskSwitchCheck>:
<LOS_TickCountGet>:
<LOS_SpinUnlockRestore>:
<LOS_SpinUnlock>:

function <OsIdleTask>: stack cost = 476 计算结果
symbol task_entry task_stack_max_cost local_cost
symbol +--<OsIdleTask>: 476 8 调用关系树状图
|
| +--<LOS_TaskResRecyle>: 468 20
| |
| | +--<LOS_SpinUnlockRestore>: 400 20
| | |
| | | +--<LOS_SpinUnlock>: 380 20
| | | |
| | | | +--<ArchSpinUnlock>: 0 0
| | | | +--<LOS_TaskUnlock>: 360 28 *Rec 递归标记
| | | | +--<LOS_IntRestore>: 36 20
| | | | +--<ArchIntRestore>: 16 16
| | |
| | | +--<LOS_SpinLockSave>: 96 20
| | | |
| | | | +--<LOS_SpinLock>: 76 20
| | | | |
| | | | | +--<LOS_TaskLock>: 56 20
| | | | | |
| | | | | | +--<OsPercpuGet>: 24 8
| | | | | | |
| | | | | | | +--<ArchCurrCpuId>: 16 16
| | | | | | | +--<LOS_IntRestore>: 36 20
| | | | | | | +--<ArchIntRestore>: 16 16
| | | | | | | +--<LOS_IntLock>: 24 8
| | | | | | | +--<ArchIntLock>: 16 16
| | | | |
| | | | | +--<ArchSpinLock>: 0 0
| | | |
| | | | +--<LOS_IntLock>: 24 8
```

工具输出结果分为四部分：函数调用关系配置文件提示、递归函数告警、计算结果输出、调用关系树状图。调用关系树状图中，显示了任务的调用关系，以及每一层调用函数栈的最大开销和局部栈开销，并以“\*Rec”标记对递归调用的函数进行提示，以“\*Ignore”标记对忽略统计的函数进行提示。

#### 步骤4 标注函数指针（可选）。

工具将无法自动解析的函数调用关系，保存至cfg/config.ini配置文件中。以下图为例，表示在LOS\_LkPrint方法和OsIdleTask任务入口函数中，存在无法自动解析的函数调用关系，需要用户手动将方法中调用的函数名添加到Remarks\_here行末尾，如果有多个函数则以空格隔开，如果没有需要添加的函数则标注为NULL。

```
Miss_funcptr: <OsIdleTask>: 0x187f4
Remarks_here: <OsIdleTask>: NULL

Miss_funcptr: <LOS_LkPrint>: 0x2838c
Remarks_here: <LOS_LkPrint>: OsLkDefaultFunc
~
```

完成函数调用关系配置文件的编辑后，需要按步骤2重新运行工具，以获取更新后的计算结果，此时配置文件中添加的函数调用关系已添加进调用关系树中。

#### 步骤5 计算任务栈大小。

从工具输出结果中可以看到估算的任务栈大小，比如步骤2显示的OsIdleTask任务栈为476字节，由于设定任务栈时需要增加任务栈初始化开销和任务切换寄存器压栈开销（建议增加300字节的开销），所以OsIdleTask的最终任务栈大小计算为776字节。

#### 步骤6 估算中断栈（可选）。



栈估算工具无法自动识别或解析中断入口函数和中断回调函数，用户需要在cfg/config.ini配置文件中手动配置，配置格式如下，Interrupt\_entry表示中断入口，Interrupt\_isr表示中断回调函数，冒号后面需要增加中断号、中断回调函数入口。

```
Interrupt_entry: TrapVector 0sIrqHandler
Interrupt_isr: 56 HwiStack001
Interrupt_isr: 57 HwiStack002
Interrupt_isr: 58 HwiStack003
```

执行下面的命令，可以进行中断栈估算：

```
python3 stackusage.py -irq -out Huawei_LiteOS.asm
```

估算结果如下：

```
interrupt: stack cost = 444
 irq entry cost = 72
 irq 58 stack cost = 372
```

中断入口栈开销72字节，中断号58的回调函数在所有配置的中断号里栈开销最大，为372字节，所以总共的中断栈开销是444字节。

----结束

工具命令

表 9-46

| 命令           | 功能                                                             |
|--------------|----------------------------------------------------------------|
| -h, --help   | 查看帮助信息                                                         |
| -m N         | 统计模式，可配置的模式N有以下三种：<br>0：仅分析汇编代码<br>1：分析源代码行号<br>2：分析反汇编文件中的源代码 |
| -sp          | 分析指定的任务条目，如果不标明-sp，将会统计所有函数                                    |
| -t taskEntry | 指定任务入口taskEntry，默认值为OsIdleTASK，如果没有指定-sp，本命令将会被忽略              |
| -out         | 将估算结果输出到界面                                                     |
| -j           | 将估算结果输出到JSON文件中                                                |
| -r N         | 指定递归函数的递归层数N，默认为1                                              |
| -irq         | 统计中断栈。如果设置了本命令，将只统计中断栈                                         |

注意事项

- 由于基于静态的反汇编文件解析的函数调用关系，无法准确获取到动态赋值的函数指针取值，因此缺失函数调用关系，可能导致计算偏差。使用时请补全函数指针配置文件中的指针取值，以得到更准确的计算结果。



- 由于静态解析时无法确定递归函数的递归深度，工具默认以调用深度为1进行统计。如果需要设置递归深度值，可在使用工具时增加-r入参设置。
- 解析结果需要增加任务栈初始化开销和任务切换时寄存器压栈开销。这些开销一般与系统可使用的寄存器数量相关，建议增加300字节的开销。
- 栈估算工具支持解析ARM指令集和RISC-V指令集的反汇编文件，不支持其他指令集。
- 由于中断栈随着中断嵌套的深度增加而增加，而中断嵌套又与设置的中断优先级、系统运行的实际情况有关，因此栈估算工具无法静态地估算中断嵌套情景的栈开销。栈估算工具支持用户配置多个中断回调函数入口，统计结束后会给出一个最大的中断栈开销，用户可根据实际业务可能的嵌套情况，参考最大中断栈开销去设置中断栈大小。

## 9.12 调试案例

### 9.12.1 踩内存定位方法

#### 9.12.1.1 异常信息定位踩内存方法

通过异常信息定位问题，参见异常接管的[问题定位实例](#)。

#### 9.12.1.2 内存备份机制定位踩内存方法

通过内存备份机制定位问题，参见[内存备份机制](#)。

#### 9.12.1.3 内存合法性检查定位踩内存方法

通过内存合法性检查定位问题，参见[内存合法性检查](#)。

#### 9.12.1.4 内存 size 检查定位踩内存方法

通过内存size检查定位问题，参见[内存size检查](#)。

#### 9.12.1.5 全局变量踩内存定位方法

调试过程中，发现一个全局变量只在一处赋值为0，但使用时打印发现变成一个非零异常值，大概率是该全局变量被踩。

如果已知一个全局变量被踩内存，可在Huawei\_LiteOS/out/<platform>/map文件中找到该全局变量所在的地址。注意该地址前面最近被使用的变量，排查是否前面变量操作不当引发踩内存，比如对该前面变量进行memcpy，memset操作时越界，溢出覆盖了当前全局变量。

这里列举一个测试的例子，在文件中定义了两全局变量，并且初始化。

```
UINT32 g_uwEraseMap[16] = {0};
UINT32 g_uwEraseCount = 0;
```

在.map文件中可以找到这些全局变量在bss段对应的位置。

```

.bss.g_uwEraseCount
0x800ce12c 0x4 /usr1/wangyun/liuxuwei2/temp_test/HuaweiLite_OS/out/hi3516a/lib/obj/
WOW.O
0x800ce12c g_uwEraseCount
.bss.g_uwEraseMap
0x800ce130 0x40 /usr1/wangyun/liuxuwei2/temp_test/HuaweiLite_OS/out/hi3516a/lib/obj/
WOW.O
0x800ce130 g_uwEraseMap

```

若g\_uwEraseMap被踩，在.map文件中找到其地址，再查找该地址前面的变量，即g\_uwEraseCount。特别注意分析g\_uwEraseCount变量的使用情况，观察是否存在某处，对变量g\_uwEraseCount进行了越界操作。

### 9.12.1.6 task 状态判断是否踩内存

Shell命令`task`，可以查看当前系统所有任务的状态。命令输出的stackSize、WaterLine、StackPoint、TopOfStack信息，可以作为判断任务栈是否踩内存的指标。

这里举例说明如何通过task命令判断是否踩内存，如下图所示，有一任务名为shellTask。

| Name                   | PID  | Priority | Status    | StackSize | WaterLine | StackPoint | TopOfStack | EventMask | SemID  | CPUPSE | CPUPSE10s | CPUPSE1s | MEMUSE   |
|------------------------|------|----------|-----------|-----------|-----------|------------|------------|-----------|--------|--------|-----------|----------|----------|
| Set_Task               | 0x0  | 0        | QueuePend | 0x5000    | 0x380     | 0x80cfb4d0 | 0x80cf5c70 | 0x0       | 0xffff | 1      | 1         | 1        | -120080  |
| IdleCore000            | 0x1  | 31       | Ready     | 0x400     | 0x22c     | 0x80cfb22c | 0x80cfbc90 | 0x0       | 0xffff | 77     | 78        | 78       | -7566584 |
| system_wq              | 0x2  | 10       | Pend      | 0x6000    | 0x23c     | 0x80d01f90 | 0x80cfcl38 | 0x0       | 0x2    | 0      | 0         | 0        | 0        |
| app_task               | 0x3  | 10       | Delay     | 0x6000    | 0x3f50    | 0x80d01f38 | 0x80d02150 | 0x1       | 0xffff | 0      | 0         | 0        | 21010728 |
| shellTask              | 0x4  | 9        | Running   | 0x3000    | 0x2810    | 0x80d10084 | 0x80d07668 | 0xffff    | 0      | 0      | 0         | 0        | -12148   |
| cmdParseTask           | 0x5  | 9        | Pend      | 0x1000    | 0x28c     | 0x80d11540 | 0x80d10818 | 0x1       | 0xffff | 0      | 0         | 0        | 218      |
| pth01                  | 0x6  | 10       | Delay     | 0x5000    | 0x26b0    | 0x80d176c8 | 0x80d11868 | 0x0       | 0xffff | 0      | 0         | 0        | -340     |
| RevAndDispatch         | 0x7  | 10       | Ready     | 0x6000    | 0x25c     | 0x80d03a18 | 0x80d02a40 | 0x0       | 0xffff | 0      | 0         | 0        | -2704    |
| SaveParamFlash         | 0x8  | 10       | Ready     | 0x6000    | 0x294     | 0x80d0fe60 | 0x80d0f500 | 0x0       | 0xffff | 0      | 0         | 0        | -340     |
| ProcessEncStreamThread | 0x9  | 10       | Ready     | 0x10000   | 0xc40     | 0x80e11440 | 0x80e018b8 | 0x1       | 0xffff | 3      | 4         | 4        | 8533536  |
| UpdateTimeOsdThread    | 0xa  | 10       | Delay     | 0x6000    | 0x860     | 0x80e17448 | 0x80e11640 | 0x0       | 0xffff | 0      | 0         | 0        | 3008     |
| Isprun                 | 0xb  | 10       | Ready     | 0x6000    | 0x1604    | 0x80e00648 | 0x80e14860 | 0x1       | 0xffff | 11     | 9         | 10       | -465952  |
| sync_get               | 0xc  | 10       | Pend      | 0x12000   | 0x3f4     | 0x80e09f48 | 0x80e0a238 | 0x1       | 0xffff | 0      | 0         | 0        | 0        |
| SendAFrameToAencThread | 0xd  | 10       | Ready     | 0x6000    | 0x5a3c    | 0x80e05f70 | 0x80e0d250 | 0x0       | 0xffff | 4      | 4         | 4        | -345056  |
| WORKSTATE_CHECK        | 0xe  | 10       | Ready     | 0x6000    | 0x234     | 0x81cf23b0 | 0x81cece50 | 0x0       | 0xffff | 0      | 0         | 0        | -3136    |
| hinci_Task             | 0xf  | 10       | Ready     | 0x600     | 0x444     | 0x81cf4268 | 0x81cf3c20 | 0x1       | 0xffff | 0      | 0         | 0        | 853816   |
| hinci_Task             | 0x10 | 10       | Ready     | 0x800     | 0x480     | 0x81cf5a48 | 0x81cf5490 | 0x0       | 0xffff | 0      | 0         | 0        | -40      |
| TIMER_CHECK            | 0x11 | 10       | Delay     | 0x6000    | 0x7ac     | 0x81cf6a60 | 0x81cf5c40 | 0x1       | 0xffff | 0      | 0         | 0        | -1060    |

StackSize = 0x3000（创建该任务时分配的栈大小）

WaterLine = 0x2810（水线，目前为止该任务栈已经被使用的内存大小）

StackPoint = 0x80d10084（任务栈指针，指向该任务当前的地址）

TopOfStack = 0x80d0d768（栈顶）

MaxStackPoint = TopOfStack + StackSize = 0x80d10768（得到该任务栈最大的可访问地址）

- 若WaterLine > StackSize，则说明该任务踩内存。
- 若StackPoint > MaxStackPoint 或 StackPoint < TopOfStack，则说明该任务踩内存。

# 10 Security

## 10.1 地址空间布局随机化

## 10.1 地址空间布局随机化

### 10.1.1 概述

#### 基本概念

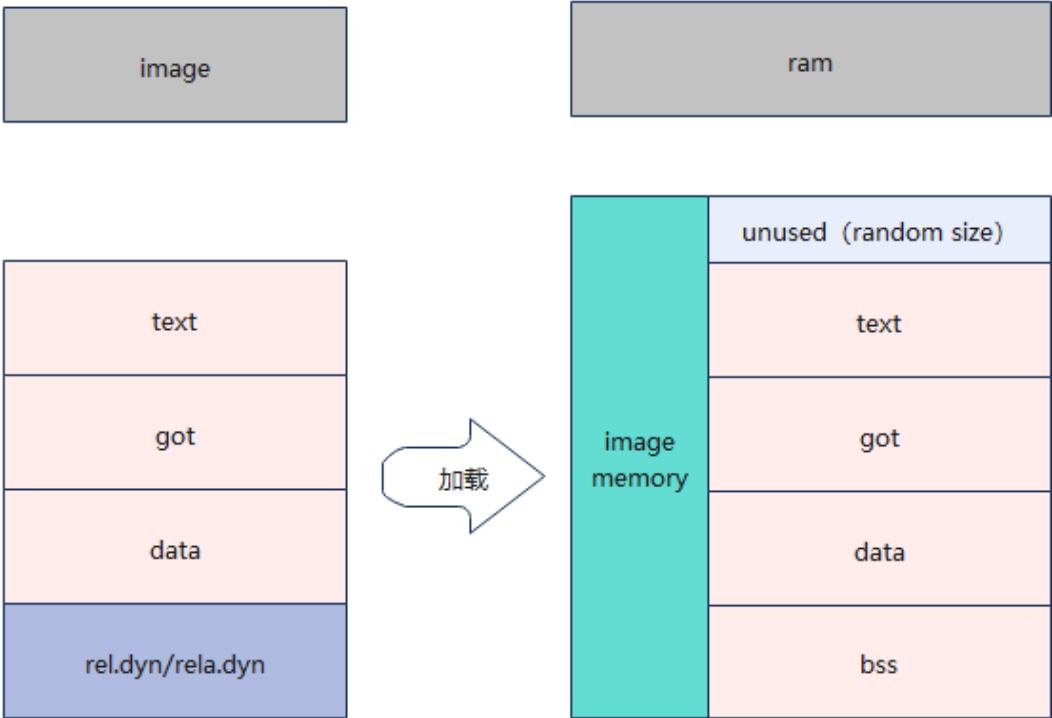
地址空间布局随机化简称ASLR（Address space layout randomization），是一种针对缓冲区溢出的安全保护技术，通过对代码段、数据段、堆等固定运行地址进行随机化，增加攻击者定位代码位置的难度，防止攻击者直接定位攻击代码位置，达到阻止溢出攻击的目的。

#### 运作机制

考虑到产品的不同使用场景，目前LiteOS支持两种地址空间布局随机化方案：GOT（Global Offset Table，全局偏移表）方案和GSPR（Global Symbol Patching Randomization，全局符号修正随机化）方案。两种方案实现机制不同，各有优劣，用户需根据产品需求来选择方案。

- **GOT地址随机化**

通过gcc的-pie/-fpie选项生成got表，使可执行文件成为地址无关代码，在系统启动后，bootloader将随机偏移值传递给重定位模块，然后重定位模块根据实际内存中的位置修正got表中对应的表项的地址。GOT方案会额外生成got段用于大部分全局变量的访问，由于全局变量访问均变成二次跳转的方式，访问效率会下降，影响运行性能。GOT方案镜像加载后，布局如图所示：

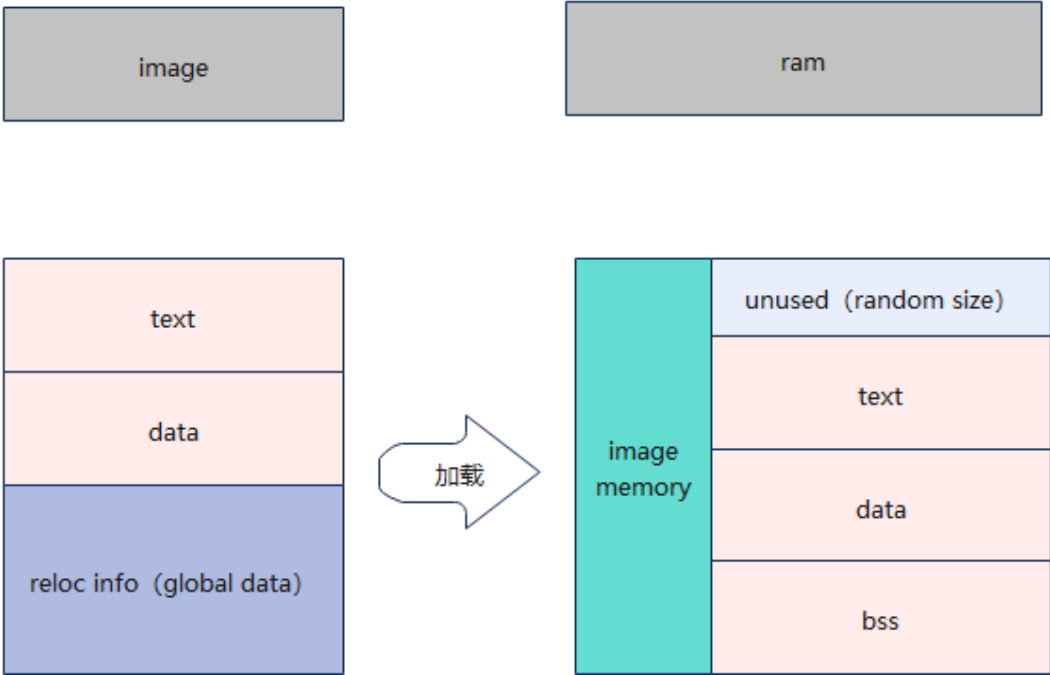


**⚠ 注意**

由于GOT方案需要在编译镜像时添加-fpie选项，链接时需要添加-pie选项，这两个选项对编译器有依赖，实际中发现有些risc-v编译器不支持该选项，因此在选择此方案时需要检查编译器是否支持该选项。

● **GSPR地址随机化**

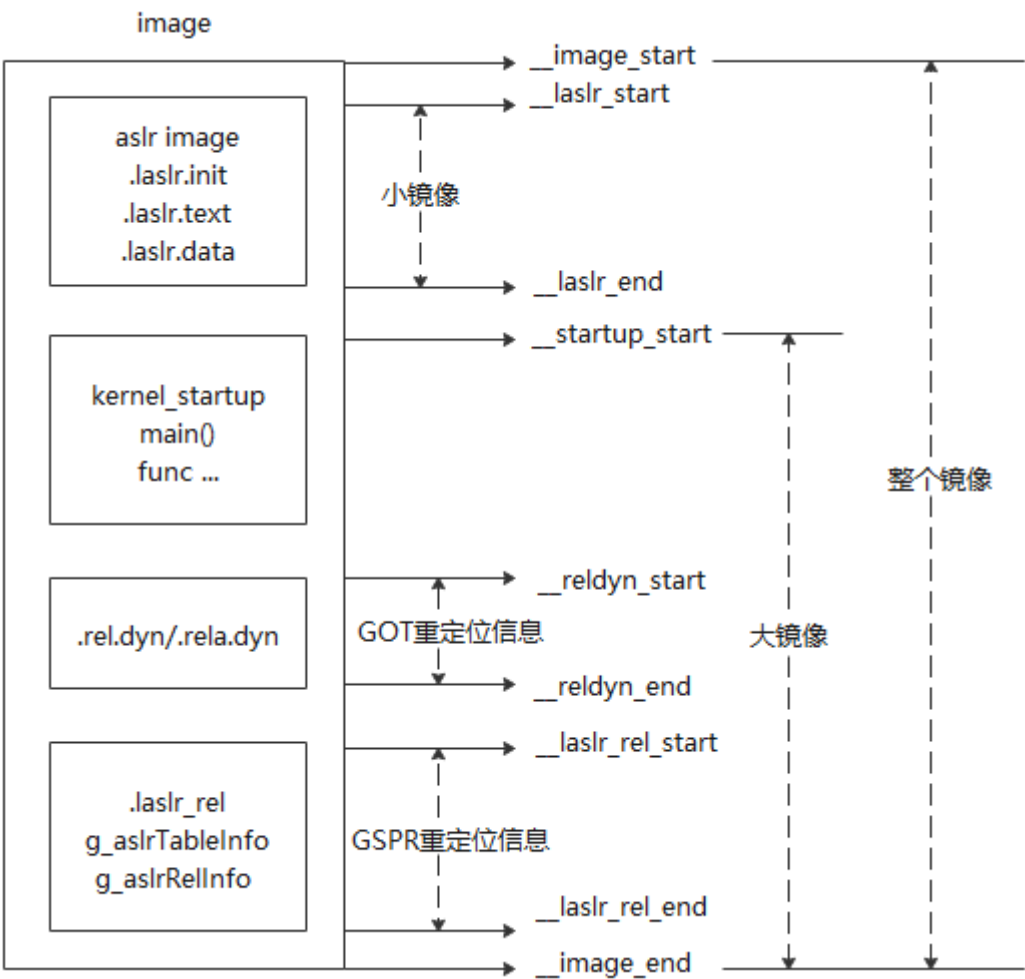
通过gcc编译器--emit-relocs选项生成重定位信息段，在系统启动后bootloader将随机偏移值传递给重定位模块，然后重定位模块根据实际内存中的偏移位置修正代码段中的指令。GSPR方案会生成一个重定位信息的段，用于重定位所有和绝对地址访问相关的指令。重定位后，运行效果与未进行随机化前的效果一致，但是启动阶段需要重定位的符号较多，影响启动性能。对性能的具体影响需要以平台实测为准。GSPR方案镜像加载后，布局如图所示：



**⚠ 注意**

GSPR方案需要编译器支持--emit-relocs链接选项，一般的编译器都支持该选项，选择使用该方案前需先确认编译器是否支持该选项。

一般情况下，不管选用哪种地址随机化方案，编译出来的镜像布局大致为下图所示：



负责对大镜像中符号进行重定位的小镜像排布在整个镜像的最开始，其主要作用就是将用户主体程序中访问绝对地址的地方进行修正，修改为因地址随机化而加了随机偏移的符号实际地址。大镜像为用户主体程序，在镜像中的布局紧随小镜像。底下存放重定位信息的段采用OVERLAY的方式与heap段进行复用，在小镜像对大镜像中符号进行重定位的过程中会遍历该段，以获取需要重定位的符号信息。在重定位完成后，小镜像程序执行完毕，跳转至大镜像的起始地址处执行，在后续内核初始化时会对内存进行初始化，这时重定位信息段的内容可以被覆盖，当做系统堆空间来使用，以节省内存空间。

重定位信息段中的重定位条目有两种格式：REL和RELA，一般某一特定架构编译出来的镜像中的重定位类型只有一种，且必是其一。每个重定位条目表示一个必须被重定位的符号引用，并指明如何计算被修改的符号引用。GOT方案的重定位条目由编译器生成，GSPR方案的重定位条目由脚本解析ELF文件生成。图中的符号为适配平台地址随机化功能链接脚本中的主要符号。其含义和用途如下表所示：

表 10-1

| 符号名           | 含义        | 作用                            |
|---------------|-----------|-------------------------------|
| __image_start | 整个镜像的起始地址 | 在小镜像中用来校验重定位信息段里的符号是否在合法地址范围内 |

| 符号名               | 含义                | 作用                                            |
|-------------------|-------------------|-----------------------------------------------|
| __image_end       | 整个镜像的结束地址         | 在小镜像中用来校验重定位信息段里的符号是否在合法地址范围内                 |
| __laslr_start     | 小镜像的起始地址          | 在小镜像中对符号进行重定位过程中根据其地址范围忽略掉小镜像中符号              |
| __laslr_end       | 小镜像的结束地址          | 在小镜像中对符号进行重定位过程中根据其地址范围忽略掉小镜像中符号              |
| __reldyn_start    | GOT方案重定位信息段的起始地址  | 在小镜像中用作遍历重定位项的起始地址（GOT方案）                     |
| __reldyn_end      | GOT方案重定位信息段的结束地址  | 在小镜像中用作遍历重定位项的结束地址（GOT方案）                     |
| __laslr_rel_start | GSPR方案重定位信息段的起始地址 | 在小镜像中用作遍历重定位项的起始地址（GSPR方案）                    |
| __laslr_rel_end   | GSPR方案重定位信息段的结束地址 | 在小镜像中用作遍历重定位项的结束地址（GSPR方案）                    |
| __startup_start   | 大镜像的起始地址          | 在小镜像执行对大镜像符号执行完重定位操作后，指示大镜像的入口地址，以便跳转到大镜像启动系统 |

10.1.2 使用指南

使用场景

为了降低固定地址类攻击、缓冲溢出类攻击的成功概率，增加攻击者定位代码位置的难度，公司安全编译选项规范要求开启地址随机化功能。使用时需要先编译出可随机化镜像，然后在加载时根据随机的加载地址对镜像符号地址进行修正。

开发流程

本部分以risc-v架构为例，讲解开发流程。

**步骤1** 打开菜单，进入Kernel ---> Enable ASLR菜单，完成任务模块的配置。

表 10-2

| 配置项               | 含义          | 取值范围   | 默认值 | 依赖                                                                                                                                         |
|-------------------|-------------|--------|-----|--------------------------------------------------------------------------------------------------------------------------------------------|
| LOSCFG_LASLR_GOT  | 选择GOT地址随机化  | YES/NO | YES | !<br>LOSCFG_KERNEL_DYNLOAD && !<br>LOSCFG_KERNEL_RUNSTOP && !<br>LOSCFG_KERNEL_SCATTER && !<br>LOSCFG_FS_VFS<br>&& !<br>LOSCFG_KERNEL_PERF |
| LOSCFG_LASLR_GSPR | 选择GSPR地址随机化 | YES/NO | NO  | !<br>LOSCFG_KERNEL_DYNLOAD && !<br>LOSCFG_KERNEL_RUNSTOP && !<br>LOSCFG_KERNEL_SCATTER && !<br>LOSCFG_FS_VFS<br>&& !<br>LOSCFG_KERNEL_PERF |

步骤2 修改平台链接脚本

以LiteOS中一个risc-v架构平台最基础通用的链接脚本为例，链接脚本要点已加粗标记。

```
ENTRY(reset_vector)

PHDRS
{
 code PT_LOAD;
 data PT_LOAD;
}

INCLUDE board.ld
SECTIONS
{
 .laslr : ALIGN(0x20) {
 __image_start = .; /* must be the first address */
 __laslr_start = .;
 KEEP (*(SORT(.laslr.init)))
 KEEP (*(SORT(.laslr.text)))
 KEEP (*(SORT(.laslr.data)))
```



```

/* 如果采用got地址随机方案且重定位表项为rela类型，重定位信息字符表通过ld脚本链接进去，以下注释
需放开 */
/* 以got方案为例 */
//ImageInfo = .;
//LONG(__image_start)
//LONG(__image_end)
//LONG(__laslr_start)
//LONG(__laslr_end)
//LONG(__reldyn_start)
//LONG(__reldyn_end)
//LONG(__startup_start)
//LONG(0x5A5A5C5C)
__laslr_end = .;
} > ram : code

/* text/read-only data */
.text ALIGN(0x20) : ALIGN(0x20) {
 . = ALIGN(0x20);
 __text_start = .;
 __startup_start = .;
 KEEP(*(entry.text))
 __startup_end = .;
 __itcm_text_start = .;
 KEEP(*(itcm.text))
 __itcm_text_end = .;
 (text.text.)
 (gnu.linkonce.t.)
 *(.plt)
 *(eh_frame_hdr)
 *(eh_frame)
} > ram : code

.initcall.init ALIGN(0x20) : ALIGN(0x20) {
 __initcall0_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall0.init)))
 __initcall0_end = ABSOLUTE(.);

 __initcall1_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall1.init)))
 __initcall1_end = ABSOLUTE(.);

 __initcall2_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall2.init)))
 __initcall2_end = ABSOLUTE(.);

 __initcall3_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall3.init)))
 __initcall3_end = ABSOLUTE(.);

 __initcall4_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall4.init)))
 __initcall4_end = ABSOLUTE(.);

 __initcall5_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall5.init)))
 __initcall5_end = ABSOLUTE(.);

 __initcall6_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall6.init)))
 __initcall6_end = ABSOLUTE(.);

 __initcall7_start = ABSOLUTE(.);
 KEEP(*(SORT (.initcall7.init)))
 __initcall7_end = ABSOLUTE(.);
} > ram : code

.gcc_except_table : {
 KEEP(*(SORT (.gcc_except_table)))
 KEEP(*(SORT (.gcc_except_table.*)))
}

```

```

} > ram : code
.exception_ranges ALIGN (0x8) : ONLY_IF_RW { *(.exception_ranges .exception_ranges*) } > ram : code

.dummy_post_text ALIGN(0x20) : ALIGN(0x20) {
 . = ALIGN(0x20);
 __text_end = .;
} > ram : code

.dynamic : { *(.dynamic) } > ram : code
.interp : { *(.interp) } > ram : code
.hash : { *(.hash) } > ram : code
.dynsym : { *(.dynsym) } > ram : code
.dynstr : { *(.dynstr) } > ram : code
.plt : { *(.plt) } > ram : code
.got : { *(.got.plt) *(.got) } > ram : data

.rodata ALIGN(0x20) : ALIGN(0x20) {
 __rodata_start = .;
 *(.srodata.cst16) *(.srodata.cst8) *(.srodata.cst4) *(.srodata.cst2)
 (.srodata .srodata*)
 (.rodata .rodata .gnu.linkonce.r*)
} > ram : data

/*
 * extra linker scripts tend to insert sections just after .rodata,
 * so we want to make sure this symbol comes after anything inserted above,
 * but not aligned to the next section necessarily.
 */
.dummy_post_rodata ALIGN(0x20) : ALIGN(0x20) {
 . = ALIGN(0x20);
 __rodata_end = .;
} > ram : data

.data ALIGN(0x20) : ALIGN(0x20) {
 /* writable data */
 __ram_data_start = .;
 (.sdata .sdata .gnu.linkonce.s*)
 (.data .data .gnu.linkonce.d*)
 . = ALIGN(0x4);
 KEEP(*(SORT (.liteos.table.*)));
} > ram : data

.ctors : ALIGN(0x4) {
 __ctor_list = .;
 KEEP (*(ctors .init_array))
 __ctor_end = .;
} > ram : data

.dtors : ALIGN(0x4) {
 __dtor_list = .;
 KEEP (*(dtors .fini_array))
 __dtor_end = .;
} > ram : data

/*
 * extra linker scripts tend to insert sections just after .data,
 * so we want to make sure this symbol comes after anything inserted above,
 * but not aligned to the next section necessarily.
 */
.dummy_post_data ALIGN(0x20) : ALIGN(0x20) {
 . = ALIGN(0x20);
 __ram_data_end = .;
 . = ALIGN(64);
} > ram : data

OVERLAY : {
 .laslr_rel {
 __laslr_rel_start = .;
 KEEP (*(SORT(.laslr_rel.*)))
 }
}

```

```

 __laslr_rel_end = .;
}
.rela.dyn {
 __reldyn_start = .;
 (.rela.)
 __reldyn_end = .;
}
.heap {
 __heap_start = .;
 KEEP(*(.heap));
 __heap_end = .;
}
} > ram : data
ASSERT((__laslr_rel_end - __laslr_rel_start) <= (__heap_end - __heap_start),
 "check liteos.ld, region laslr_rel size bigger than heap, rel address will change")

/* uninitialized data (in same segment as writable data) */
.bss ALIGN(0x20) : ALIGN(0x20) {
 __int_stack_start = .;
 *(.int_stack);
 . = ALIGN(0x4);
 __int_stack_end = .;
 KEEP(*(.bss.prebss.*))
 . = ALIGN(0x20);
 __bss_start = .;
 __global_pointer$ = .;
 (.sbss .sbss. .gnu.linkonce.sb.*)
 *(.scommon)
 (.gnu.linkonce.b.)
 (.bss .bss.)
 *(COMMON)
 . = ALIGN(0x20);
 __bss_end = .;
} > ram : data

. = ALIGN(0x20);
__image_end = .;
/* Strip unnecessary stuff */
/DISCARD/ 0 : { *(.comment .note) } > ram
}

```

### ⚠ 注意

1. 由于重定位信息和heap段复用，所以系统堆需要采用静态数组灌段的形式实现。
2. 对于got方式的地址随机化且重定位类型为rela时，由于全局变量地址存在于重定位表项的addend变量内，导致构造重定位信息的字符池不能采取静态赋值的方式，所以通过在链接脚本灌段的形式实现。

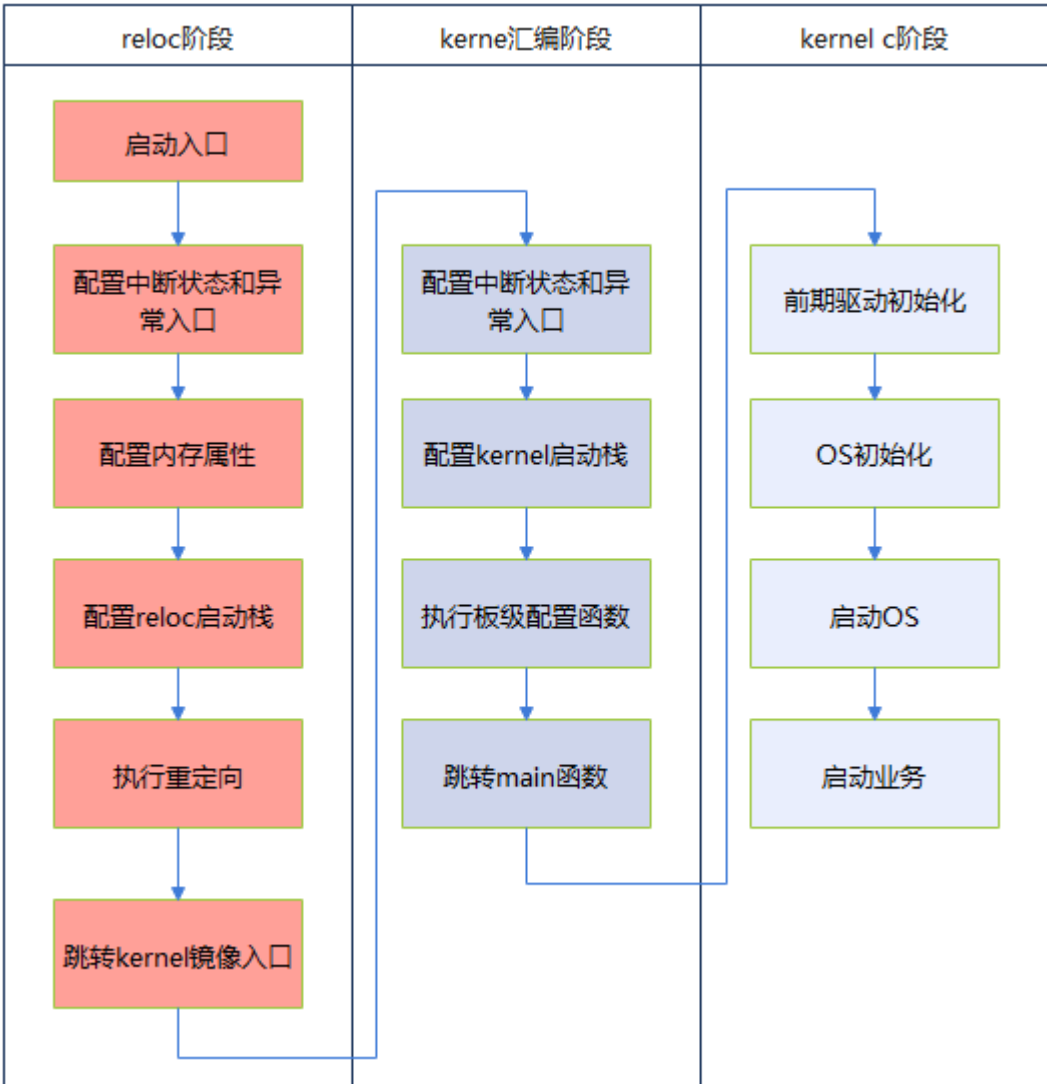
### 步骤3 编写小镜像启动文件

以LiteOS中一个risc-v架构平台最基础通用启动流程为例，主要分为如下几步：

1. 配置cache（根据需要配置，非必选项）
2. 配置PMP（一般使用GSPR方案都需要，因为在进行非对齐地址符号重定位时需要操作非对齐地址，否则系统会异常挂死）
3. 设置异常总入口（非必须，便于调试）
4. 设置小镜像启动栈（原则上启动栈地址不与镜像加载地址有重合即可，一般选用系统内存空间末尾部分作为小镜像的启动栈，跳转到大镜像执行后，这部分内存可以当做系统堆空间复用）
5. 大镜像符号重定位（直接调用封装好的函数即可）

6. 清除小镜像中的重定位相关信息并跳转至大镜像起始地址引导系统启动（直接调用封装好的函数即可）

在执行地址随机化时，系统启动基本流程如下图所示：



步骤4 编译镜像

1. 使用Make或者Cmake编译框架编译出.bin镜像文件。
2. 编译出镜像后GOT方案和GSPR方案执行的操作不同。
 
  - 对于GOT方案，进入Huawei\_LiteOS/build/scripts/aslr目录执行check\_aslr\_rel\_type.py脚本。示例如下：  
`$ ./check_aslr_rel_type.py GOT riscv32-linux-musl- out/vs_server`
  - 对于GSPR方案，进入Huawei\_LiteOS/build/scripts/aslr目录执行generate\_gspr\_rel\_info.py脚本。示例如下：  
`$ ./generate_gspr_rel_info.py GSPR riscv32-unknown-elf- out/vs_server`
3. 进入Huawei\_LiteOS/build/scripts/aslr目录执行check\_aslr\_image.py脚本。示例如下：  
`$ ./check_aslr_image.py out/vs_server out/vs_server.map riscv32-unknown-elf- ${LITEOS_LDFLAGS}`

 说明

out/vs\_server表示生成的ELF文件，\${LITEOS\_LDFLAGS}是LiteOS编译框架中的链接选项。

----结束

方案对比

GOT方案和GSPR方案除了依赖的编译器选项，以及实现机制有区别，其优劣势也十分鲜明，对比如下表所示：

表 10-3

| 对比项        | GOT方案 | GSPR方案 |
|------------|-------|--------|
| 启动时间       | 影响较小  | 影响较大   |
| 运行时text段大小 | 影响较大  | 影响较小   |
| 重定位信息段大小   | 影响较小  | 影响较大   |
| 运行时性能      | 影响较大  | 影响较小   |

由于GOT表方案相较GSPR方案的重定位项较少，所以重定位信息段较小，在小镜像中做符号重定位时，时间也较短。但是由于GOT方案要通过GOT表跳转间接访问符号，所以其运行时性能较差，运行时text段也较大。因为重定位信息段会在ELF文件中体现，所以编译出来的镜像大小，GOT方案的一般要比GSPR方案的要小。但是因为后续重定位信息段可以被系统堆空间复用，一般系统堆空间都比较大，所以一般不会占用额外运行内存。

产品依赖

- 产品编译器要支持相应的编译链接选项：GOT方案需要编译器支持-pie和-fpie选项；GSPR方案需要编译器支持--emit-relocs选项。
- 产品要提供满足公司可信要求的安全随机数，用于地址的随机加载，具体规范参考《密码算法应用规范》。且真随机数产生器必须从熵源获取足够的熵值以保证其满熵输出，产品需要结合平台的内存布局提供熵值。参考值如下：
  - a. Linux系统中32位平台熵为8位256个，64位平台熵为10位1024个。
  - b. 某系统1M范围内有7K+个。
- 无虚拟地址平台使用实地址加载镜像，需预留一定的内存范围给镜像做随机偏移。
- bootloader支持随机加载，能将镜像加载到符合平台约束的随机地址空间，并且LiteOS可以获取到这个随机值，小镜像执行完符号重定位后，会清除此随机值。

使用约束

- 使用check\_aslr\_rel\_type.py检查重定位类型，仅限于GOT方案使用，采用白名单的校验方式，目前仅支持三种架构ARM、AArch64、RISC-V。架构与对应的重定位类型对应关系为：

表 10-4

| 架构      | 重定位类型              |
|---------|--------------------|
| ARM     | R_ARM_RELATIVE     |
| AArch64 | R_AARCH64_RELATIVE |
| RISCV-V | R_RISCV_RELATIVE   |

使用GOT方案时，如果是上数三种架构之一，则其重定位信息段有且仅有上表所列其对应的那种重定位算法，否则脚本报错，需要查看非法重定位算法所对应的符号实现是否有问题。如果适配平台架构不是三者其一，则需要根据实际情况修改脚本，扩充校验白名单。

- 使用generate\_gspr\_rel\_info.py脚本构造重定位信息段数据，仅限GSPR方案使用，采用白名单的校验方式，目前仅支持ARM、RISCV-V架构。由于GSPR方案会把所有段的重定位信息生成出来，而某段代码中重定位项对应的重定位类型不仅跟架构有关，还与编译器和编译选项有关，目前支持的两种架构及其对应的重定位类型对应关系为：

表 10-5

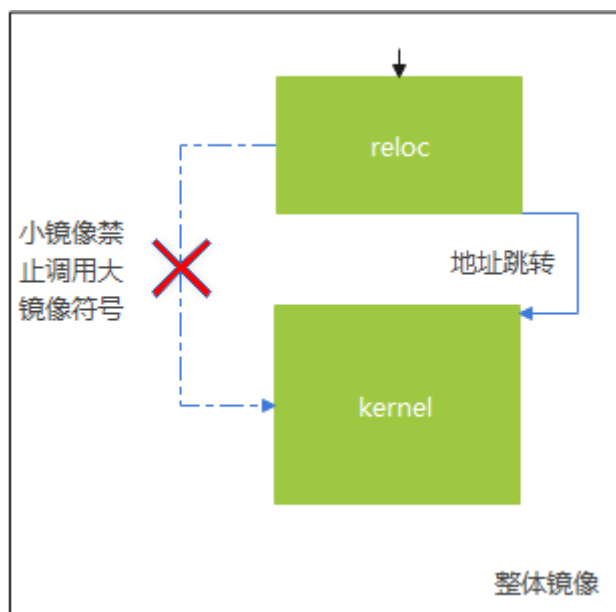
| 架构      | 重定位类型              |
|---------|--------------------|
| ARM     | R_ARM_ABS32        |
|         | R_ARM_MOVW_ABS_NC  |
|         | R_ARM_MOVT_ABS     |
|         | R_ARM_CALL         |
|         | R_ARM_JUMP24       |
|         | R_ARM_V4BX         |
|         | R_ARM_NONE         |
| RISCV-V | R_RISCV_RVC_JUMP   |
|         | R_RISCV_RVC_BRANCH |
|         | R_RISCV_JAL        |
|         | R_RISCV_BRANCH     |
|         | R_RISCV_BRANCHI    |
|         | R_RISCV_CALL       |
|         | R_RISCV_RELAX      |
|         | R_RISCV_NONE       |
|         | R_RISCV_PCREL_HI20 |

| 架构 | 重定位类型                |
|----|----------------------|
|    | R_RISCV_PCREL_LO12_I |
|    | R_RISCV_GPREL_I      |
|    | R_RISCV_GPREL_S      |
|    | R_RISCV_HI20         |
|    | R_RISCV_LO12_I       |
|    | R_RISCV_LO12_S       |
|    | R_RISCV_32           |
|    | R_RISCV_BEQ_X        |
|    | R_RISCV_RVC_LUI      |
|    | R_RISCV_BEQI_YK      |
|    | R_RISCV_GPREL17_S    |
|    | R_RISCV_GPREL17_I    |
|    | R_RISCV_GP_HI20      |
|    | R_RISCV_GP_LO12_I    |

使用GSPR方案，如果在解析编译出来的ELF文件时，发现其中有新的重定位类型不在白名单中，则会触发编译报错，出现不支持的重定位类型，则需要白名单上加上这个重定位类型，以及判断该类型是否需要被重定位。适配新架构平台的时候，需要重新实现一个包含重定位类型白名单和重定位信息解析操作函数集的python脚本，以及架构有关的重定位算法。

10.1.3 注意事项

- 代码中使用了固定地址的设计，需要修改为链接脚本中符号(例如使用IMAGE\_START宏地址判断函数的起始地址，需要修改为\_image\_start这种会跟随加载地址变化的符号)；代码中使用了链接脚本符号表示大小而非地址的，需要修改为固定值，否则会被重定向(例如使用\_heap\_size表示堆内存大小，在重定位时会被修改，需要修改为HEAP\_SIZE这种宏定义)。
- 当前地址随机化只支持向后加载，即实际加载地址大于镜像中的入口地址。
- 开启地址随机化后，禁止启用以下功能：低功耗框架（上下电保存恢复）、VFS、分散加载、Perf。
- 要确保用于重定位大镜像的小镜像不能调用大镜像中符号，因为此时大镜像中的符号都还未重定位，符号地址还未修正成正确地址，并且小镜像编码需要实现地址无关编码。具体如下图所示：

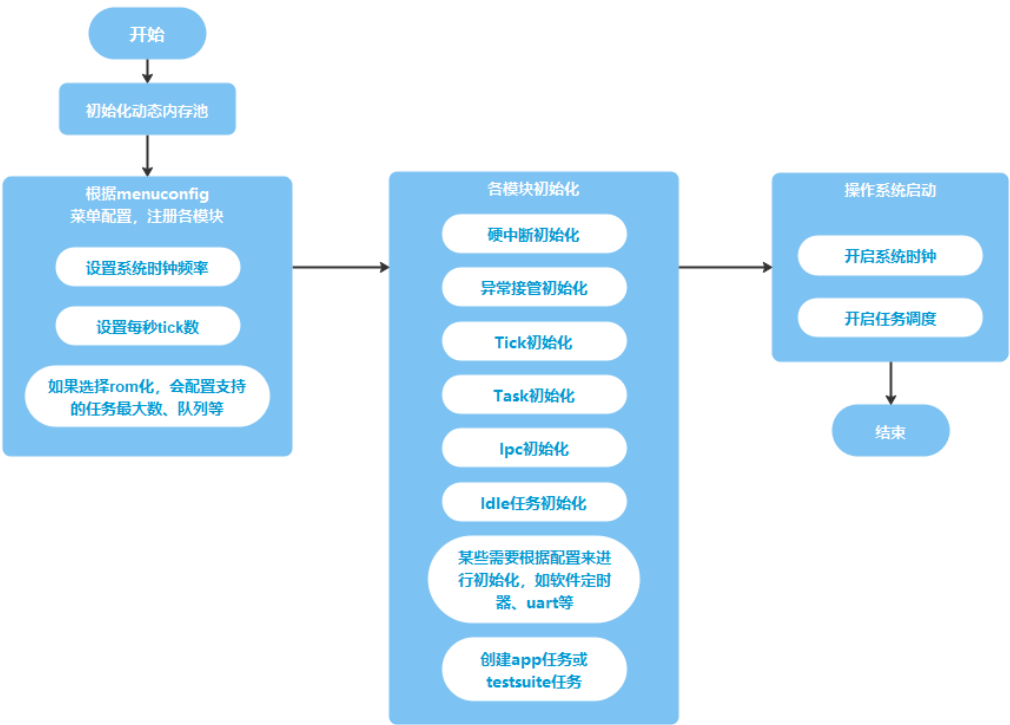




# 11 附录

- 11.1 Huawei LiteOS Kernel启动流程
- 11.2 平台兼容性适配修改要点

## 11.1 Huawei LiteOS Kernel 启动流程



上图为系统启动流程，部分模块的初始化会在模块宏开启时才生效，用户可以选择是否创建app任务（app\_init），菜单配置路径为：

Debug ---> Enable TestSuite or Applnit ---> Choose TestSuite or Applnit 菜单。

LOSCFG\_APPINIT\_TESTSUIT开启后可以选择创建app或testsuite任务，如果不开启，则认为app和testsuite任务都不创建。

## 11.2 平台兼容性适配修改要点

### 差异对比

在32位与64位模式下，涉及的主要差异点在于各个不同类型的数据位宽，开发过程中需要关注，详细如下表所示：

表 11-1 32 位与 64 位模式下数据位宽差异

| Programming type | Size in A32 | Size in A64-LP64 |
|------------------|-------------|------------------|
| char             | 8-bit       | 8-bit            |
| short            | 16-bit      | 16-bit           |
| int              | 32-bit      | 32-bit           |
| int *(pointer)   | 32-bit      | 64-bit           |
| long             | 32-bit      | 64-bit           |
| long long        | 64-bit      | 64-bit           |
| float            | 32-bit      | 32-bit           |
| double           | 64-bit      | 64-bit           |
| size_t           | 32-bit      | 64-bit           |

Huawei LiteOS的64位编译器使用的是LP64指令模式，从上表可以看出，指针、long和size\_t类型都升级为64bit。

在64位模式下，相比于32位模式，由于位宽的差异会导致一些使用上的差异，请参考如下7个修改要点，对应用代码做相关差异性的检查。

### 修改要点

1. 指针的处理。  
指针不建议强转成非指针类型，如long、int等非指针数据类型。指针可以强转成其他类型的指针和void\*，但64位的编译器会报出告警，需要消除掉。

#### 须知

也不建议指针用于数学计算时临时强转成unsigned long或int，可以转换成char\*然后再加或减常数。

2. long都要修改为int。  
除非涉及到内核公共模块，long都要修改为int，不要使用long以及相关强转。

## 须知

在某些64bit编程规范中，严格禁止使用long类型。否则很容易出错。

### 3. 仔细检查一切有关int、long、指针的赋值和比较语句。

虽然原则上不允许使用long，但存在引用的数据类型是Kernel内部原生定义为long的情况。这样在不能修改内部原生数据定义的情况下，需要注意检查引用处。

在新的编译器中，数据类型会按照如下规则自动转换：

**int + long = long**

**unsigned + signed = unsigned**

在这种原则下，下列代码：

```
long long a;
int b;
unsigned int c;
b = -2;
c = 1;
a = b + c;
```

a的值是0x00000000FFFFFFFF，而不是-1。原因是b+c是unsigned+signed，结果是个无符号数，再转成长long，只能在前面填充0。而如下代码：

```
long long a;
int b;
unsigned int c;
b = -2;
c = 1;
a = (long long) b + c;
```

a的结果就是0xffffffffffff，因为b转为long后就是0xffffffffffffe，与unsigned int c相加，就是上述结果。

### 4. 要在常量表达式中指定数据类型。

如果想将一个常量作为64位来使用，必须清楚地在数值后跟上ll或者LL，否则，编译器有可能把它当作int来对待。例如下面这个函数：

```
long long SetBitN(long long value, unsigned bitNum)
{
 long long mask;
 mask = 1 << bitNum;
 return value | mask;
}
```

在64位下乍一看以为没问题，但其实1被当成int型处理了，如果bitNum为超过32的数，实际无效，超过的部分会被截断，结果就是0。必须这样写：

```
long long SetBitN(long long value, unsigned bitNum)
{
 long long mask;
 mask = 1LL << bitNum;
 return value | mask;
}
```

另外再举例，以下两种操作会导致指针的高32比特被清0。

```
(void*) ((long long)p & ~(16 - 1))
(void*) ((long long)p & ~(alain - 1)) /* alain为u32 */
```

可以修改成：

```
(void*) ((long long)p & ~((long long)16 - 1))
(void*) ((long long)p & ~((long long)alain - 1)) /* alain为u32 */
```

也可以采用如下定义规避：

```
1LL // (long long)
1ULL // (unsigned long long)
```

## 5. 格式化输入输出问题及解决办法。

与32位系统相比，64位系统会有如下问题：

- 32位系统可以用%x打印指针，但在64位系统会有截断问题。  
解决办法：指针的输入输出需要用%p。
- 32位系统上，使用%d来打印 int 或 long 类型的值，但是在 64 位平台上会截断。  
解决办法：long的输入输出需要用%ld。

关于**size\_t**类型：在32bit平台上，它的原形是unsigned int，而在64bit平台上，它的原形是unsigned long。这导致使用printf等函数时，无论使用%u或者%llu打印都会报warning。目前可采用%lu打印方式，规避该问题。

输入的时候，注意buffer的长度，在64位下发生了变化了，确保buffer的长度足够：

```
char * buf[9];
int a = 0;
Sprintf(buf, "%p", &a);
```

上述代码在32bit模式下是正确的，一个指针的值是c0001a38，输入后是8个字符。在64bit模式下，长度是16个字符。

## 6. 最大数、无效数的宏定义。

特别注意代码中有无引用相关最大数，无效数的宏定义，在64bit下可能已经发生变化，如：

```
#define BYTES_IN_WORD 4
#define INVALID_DMA_ADDRESS 0xffffffff
```

这些是否在64bit下还是无效或最大，需要注意，如：

```
size_t count = BIG_NUMBER;
for (unsigned int index = 0; index != count; index++)
.....
```

size\_t 是64位的，而index是32位的，所以上边的循环会一直循环下去。

## 7. 检查union中成员变量大小。

union中成员变量大小要一致，32、64位差异可能会导致原union中的某些成员变量变大，各成员变量平衡被打破。检查各联合体，确保联合体内各成员变量大小一致。

如下示例是一个常见的开源代码包，可在ILP32却不可在LP64环境下运行。代码假定长度为2的unsigned short数组，占用了与long同样的空间，可这在LP64平台上却不正确。

```
union{
 unsigned long bytes;
 unsigned short len[2];
} size;
```

正确的方法是检查是否对结构体有特殊的应用，如果有，那么需要在所有代码中仔细检查联合体，以确认所有的数据成员在LP64中都为同等长度。

## 优化建议

### 1. 数据类型方面。

- 如果确定涉及到64bit计算和存储则定义为long long类型。如果32bit够用的话，数据类型建议使用int或unsigned int，以减少内存。不允许定义long类型。
- 类型定义使用体系结构无关的类型定义：u32，u16，u64，s16，s32，s64。

### 2. 结构体定义：将最常用的数据字段都定义在一起相邻位置，这样便于一次取到Cache中，提升运行性能和效率。

3. 由于对齐方式变为8字节了，结构体的排放方式需要注意新的对齐方式。

```
struct MyStruct
{
 bool m_bool;
 char *m_pointer;
 int m_int;
};
```

如上所示，结构体bool和char\*之间会填充4个字节，故需要修改结构体排布方式，以节约内存使用，如下：

```
struct MyStructOpt
{
 char *m_pointer;
 int m_int;
 bool m_bool;
};
```

## 其他建议

1. 关于非对齐操作

不同芯片平台对不同介质的非对齐操作支持程度不一致。在使用非对齐操作前，请确认目标平台是否支持以非对齐方式访问。

2. 关于联合和位域的使用

C语言标准中未定义联合和位域的比特序。在编译时，联合和位域的比特序由编译器实现决定。建议在程序开发中减少联合和位域的使用。